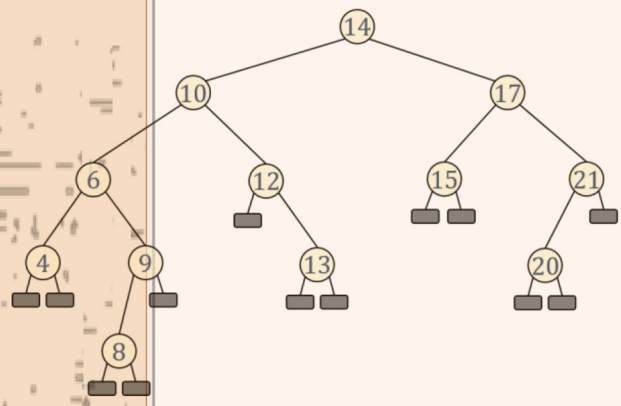
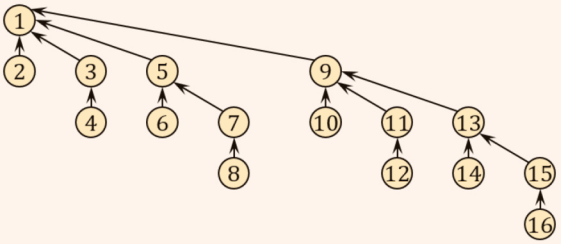
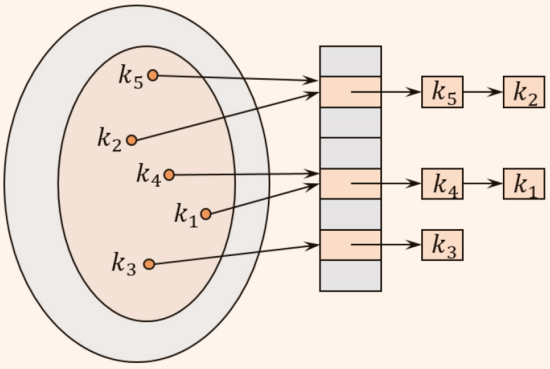
Δομες Δεδομενων

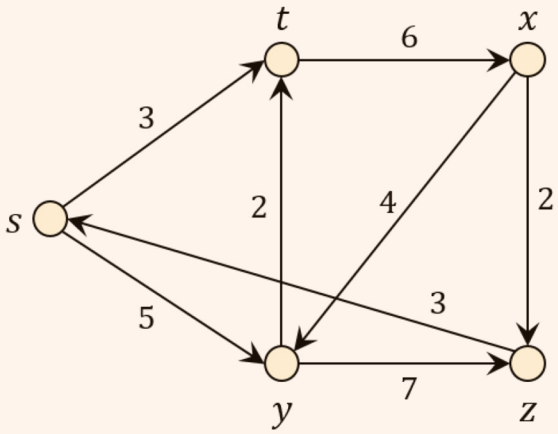
Λουκας Γεωργιαδης

Σταυρος Δ. Νικολοπουλος
Λεωνιδας Παλιος









Ελληνικά Ακαδημαϊκά Ηλεκτρονικά Συγγράμματα και Βοηθήματα
www.kallipos.gr
HEALLINK
Σύνδεσμος Ελληνικών Ακαδημαϊκών Βιβλιοθηκών
Ευρωπαϊκή Ένωση
ΕΠΙΧΕΙΡΗΣΙΑΚΟ ΠΡΟΓΡΑΜΜΑ ΕΚΠΑΙΔΕΥΣΗ ΚΑΙ ΔΙΑ ΒΙΟΥ ΜΑΘΗΣΗ
ΥΠΟΥΡΓΕΙΟ ΠΑΙΔΕΙΑΣ ΚΑΙ ΘΡΗΣΚΕΥΜΑΤΩΝ
ΕΙΔΙΚΗ ΥΠΗΡΕΣΙΑ ΔΙΑΧΕΙΡΙΣΗΣ
ΕΣΠΑ 2007-2013

ΛΟΥΚΑΣ ΓΕΩΡΓΙΑΔΗΣ

ΕΠΙΚΟΥΡΟΣ ΚΑΘΗΓΗΤΗΣ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΣΤΑΥΡΟΣ Δ. ΝΙΚΟΛΟΠΟΥΛΟΣ

ΚΑΘΗΓΗΤΗΣ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΛΕΩΝΙΔΑΣ ΠΑΛΗΟΣ

ΚΑΘΗΓΗΤΗΣ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ

Ελληνικά Ακαδημαϊκά Ηλεκτρονικά
Συγγράμματα και Βοηθήματα
www.kallipos.gr

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ

Συγγραφή

ΛΟΥΚΑΣ ΓΕΩΡΓΙΑΔΗΣ

ΣΤΑΥΡΟΣ Δ. ΝΙΚΟΛΟΠΟΥΛΟΣ

ΛΕΩΝΙΔΑΣ ΠΑΛΗΟΣ

Κριτικός Αναγνώστης

ΚΩΝΣΤΑΝΤΙΝΟΣ ΤΣΙΧΛΑΣ

Συντελεστές Έκδοσης

ΓΛΩΣΣΙΚΗ ΕΠΙΜΕΛΕΙΑ: ΣΟΦΙΑ ΤΑΚΟΥΡΛΗ

ΓΡΑΦΙΣΤΙΚΗ ΕΠΙΜΕΛΕΙΑ: ΜΑΡΙΑ ΧΡΟΝΗ

ΤΕΧΝΙΚΗ ΕΠΕΞΕΡΓΑΣΙΑ: ΝΙΚΟΛΑΟΣ ΓΙΑΝΝΑΚΕΑΣ

Copyright © ΣΕΑΒ, 2015

Το παρόν έργο αδειοδοτείται υπό τους όρους της άδειας Creative Commons Αναφορά Δημιουργού - Μη Εμπορική Χρήση - Όχι Παράγωγα Έργα 3.0.
Για να δείτε ένα αντίγραφο της άδειας αυτής επισκεφτείτε τον ιστότοπο
https://creativecommons.org/licenses/by-nc-nd/3.0/gr/

ΣΥΝΔΕΣΜΟΣ ΕΛΛΗΝΙΚΩΝ ΑΚΑΔΗΜΑΪΚΩΝ ΒΙΒΛΙΟΘΗΚΩΝ
Εθνικό Μετσόβιο Πολυτεχνείο
Ηρώων Πολυτεχνείου 9, 15780 Ζωγράφου

www.kallipos.gr

ISBN: 978-960-603-477-0

Πρόλογος

Το αντικείμενο του συγγράμματος είναι η μελέτη θεμελιωδών δομών δεδομένων οι οποίες χρησιμοποιούνται καθημερινά σε πληθώρα εφαρμογών. Ο βασικός μας στόχος είναι να προσφέρουμε στους αναγνώστες τις απαραίτητες γνώσεις έτσι ώστε να είναι σε θέση να κατανοήσουν τη λειτουργία σημαντικών δομών δεδομένων και των εφαρμογών τους. Για το σκοπό αυτό, δίνουμε τόσο το κατάλληλο θεωρητικό υπόβαθρο, όπου αναπτύσσουμε βασικές τεχνικές σχεδίασης και ανάλυσης δομών δεδομένων και αλγορίθμων, όσο και τη δυνατότητα άμεσης εφαρμογής της θεωρίας μέσω ολοκληρωμένων υλοποιήσεων.

Για τις υλοποιήσεις επιλέξαμε τη γλώσσα προγραμματισμού Java για δύο λόγους. Πρώτον, ένα σημαντικό μέρος του λογισμικού που παράγεται για σύγχρονα συστήματα αναπτύσσεται σε Java. Δεύτερον, εκμεταλλευόμαστε τη χρήση αντικειμενοστρεφούς σχεδίασης για να παρουσιάσουμε τις δομές δεδομένων μέσω αφηρημένων τύπων δεδομένων οι οποίες διαχωρίζουν την υλοποίηση από την εφαρμογή. Επιπλέον, τα προγράμματα που δίνουμε μπορούν να γίνουν κατανοητά, χωρίς ιδιαίτερη προσπάθεια, από οποιοδήποτε προγραμματιστή κάποιας άλλης σύγχρονης γλώσσας προγραμματισμού.

Η ύλη του συγγράμματος έχει οργανωθεί σε τρεις βασικές ενότητες και ένα παράρτημα, όπως περιγράφονται στον ακόλουθο πίνακα.

ΕΝΟΤΗΤΑ Α	Θεμελιώδεις Έννοιες και Στοιχειώδεις Δομές Δεδομένων	1. Εισαγωγή 2. Ανάλυση Αλγορίθμων 3. Στοιχειώδεις Δομές Δεδομένων 4. Γραφήματα και Δένδρα
ΕΝΟΤΗΤΑ Β	Βασικές Δομές Δεδομένων	5. Συλλογές, Στοίβες και Ουρές 6. Ουρές Προτεραιότητας 7. Λεξικά και Δυαδικά Δένδρα Αναζήτησης 8. Ισορροπημένα Δένδρα Αναζήτησης 9. Κατακερματισμός 10. Ψηφιακά Λεξικά 11. Ένωση Ξένων Συνόλων
ΕΝΟΤΗΤΑ Γ	Προηγμένα Θέματα	12. Διαχείριση Μνήμης 13. Αντισταθμιστική Ανάλυση 14. Προηγμένες Ουρές Προτεραιότητας
Παράρτημα		15. Γλώσσα Προγραμματισμού Java

Στην πρώτη ενότητα (Κεφάλαια 1-4), εισάγονται βασικές έννοιες και τεχνικές σχεδίασης και ανάλυσης δομών δεδομένων και αλγορίθμων. Επίσης, παρουσιάζονται στοιχειώδεις δομές δεδομένων, οι οποίες και αποτελούν τη βάση των πιο ανεπτυγμένων μεθόδων που εξετάζονται στις επόμενες δύο ενότητες. Στη δεύτερη ενότητα (Κεφάλαια 5-11), η οποία αποτελεί την κύρια ενότητα του συγγράμματος, αναλύουμε τις πιο σημαντικές δομές δεδομένων που χρησιμοποιούνται σήμερα (στοίβες και ουρές, ουρές προτεραιότητας, δένδρα αναζήτησης,

πίνακες κατακερματισμού κ.α.). Στην τρίτη ενότητα (Κεφάλαια 12-14), συζητούμε μερικά πιο προηγμένα θέματα σχεδίασης και ανάλυσης αποδοτικών δομών δεδομένων. Τέλος, στο Κεφάλαιο 15 δίνουμε μια επισκόπηση της γλώσσας προγραμματισμού Java, όπου αναφέρουμε τα βασικά χαρακτηριστικά της τα οποία χρησιμοποιούμε στα προγράμματα μας. Υποθέτουμε ότι ο αναγνώστης έχει αποκτήσει ήδη ένα υπόβαθρο σε κάποια γλώσσα προγραμματισμού όπως η Java.

Το σύγγραμμα απευθύνεται κυρίως σε φοιτητές τμημάτων Πληροφορικής και Μηχανικών Η/Υ των δύο πρώτων ετών του πρώτου κύκλου σπουδών (προπτυχιακοί φοιτητές), οι οποίοι έχουν αποκτήσει βασικές γνώσεις πάνω στον προγραμματισμό και στη λειτουργία των ηλεκτρονικών υπολογιστών. Μπορεί, ακόμα, να φανεί χρήσιμο ως βιβλίο αναφοράς σε μεταπτυχιακούς φοιτητές και σε ενδιαφερόμενους επαγγελματίες. Το σύγγραμμα προσφέρει στον αναγνώστη επαρκές υπόβαθρο για την αποτελεσματική επίλυση ενός μεγάλου φάσματος προβλημάτων και εφαρμογών, μέσω της χρήσης κατάλληλων δομών δεδομένων και αντίστοιχων αλγορίθμων. Έτσι, ο αναγνώστης, έχοντας κατανοήσει την ύλη του συγγράμματος, θα είναι σε θέση να αναλύει την επίδοση βασικών δομών δεδομένων, να συγκρίνει την αποδοτικότητα και την καταλληλόλητα διαφορετικών δομών δεδομένων για την επίλυση κάποιου προβλήματος, να σχεδιάζει σύνθετες δομές δεδομένων ή δομές δεδομένων προσαρμοσμένες σε κάποια εφαρμογή, καθώς επίσης και να υλοποιεί αποδοτικούς αλγόριθμους και ολοκληρωμένα προγράμματα.

Ευελπιστούμε ότι η θεματολογία και η δομή του συγγράμματος θα ενισχύσουν το ενδιαφέρον του αναγνώστη για την περιοχή της ανάλυσης, σχεδίασης και εφαρμογής αλγορίθμων και δομών δεδομένων. Η περιοχή αυτή βρίσκεται στο βασικό πυρήνα της Επιστήμης των Υπολογιστών και παραμένει επίκαιρη καθώς αποσκοπεί στην αποδοτική επίλυση προβλημάτων, μέσω της ανάπτυξης προγραμμάτων τα οποία είναι σε θέση να επεξεργάζονται γρήγορα μεγάλο όγκο δεδομένων.

ΛΟΥΚΑΣ ΓΕΩΡΓΙΑΔΗΣ
ΣΤΑΥΡΟΣ Δ. ΝΙΚΟΛΟΠΟΥΛΟΣ
ΛΕΩΝΙΔΑΣ ΠΑΛΗΟΣ

Ιωάννινα
Δεκέμβριος 2015

Περιεχόμενα

Κεφάλαιο 1: Εισαγωγή……...9

1.1 Αλγόριθμοι και Δομές Δεδομένων 9
1.2 Διατήρηση Διατεταγμένου Συνόλου 12
1.3 Ολοκληρωμένη Υλοποίηση σε Java 15
Ασκήσεις 18
Βιβλιογραφία 19

Κεφάλαιο 2: Ανάλυση Αλγορίθμων…….20

2.1 Εισαγωγή 20
2.2 Εμπειρική και Θεωρητική Ανάλυση Αλγορίθμων 22
2.2.1 Εμπειρική Πολυπλοκότητα 22
2.2.2 Θεωρητική Πολυπλοκότητα 23
2.3 Ανάλυση Χειρότερης και Αναμενόμενης Περίπτωσης 25
2.4 Ασυμπτωτική Πολυπλοκότητα 28
2.5 Αναδρομικές Σχέσεις 33
Ασκήσεις 37
Βιβλιογραφία 38

Κεφάλαιο 3: Στοιχειώδεις Δομές Δεδομένων……..39

3.1 Στοιχειώδεις τύποι δεδομένων 39
3.2 Πίνακες 40
3.2.1 Διδιάστατοι πίνακες 43
3.3 Συνδεδεμένες Λίστες 48
3.4 Αναδρομή 51
3.4.1 Μέθοδος «Διαίρει και Βασίλευε» 53
Ασκήσεις 57
Βιβλιογραφία 58

Κεφάλαιο 4: Γραφήματα και Δένδρα……..60

4.1 Γραφήματα 60
4.2 Δομές δεδομένων για την αναπαράσταση γραφημάτων 64
4.2.1 Υλοποίηση σε Java 66
4.3 Διερεύνηση γραφήματος 69
4.4 Δένδρα 86

4.4.1 Δυαδικά δένδρα 88
Ασκήσεις 91
Βιβλιογραφία 92

Κεφάλαιο 5: Συλλογές, Στοίβες και Ουρές 94

5.1 Αφηρημένοι τύποι δεδομένων 94
5.2 Συλλογές και Επαναλήπτες 95
5.1.1 Εφαρμογή: Υλοποίηση λιστών γειτνίασης γραφήματος 96
5.2 Στοίβα 97
5.2.1 Υλοποίηση στοίβας με συνδεδεμένη λίστα 98
5.2.3 Υλοποίηση στοίβας με πίνακα 100
5.3 Ουρά 101
5.3.1 Υλοποίηση ουράς με συνδεδεμένη λίστα 102
5.3.3 Υλοποίηση ουράς με πίνακα 104
5.4 Εφαρμογή: Συντακτική ανάλυση εκφράσεων 106
5.4 Γενικευμένες Ουρές 109
5.4.1 Υλοποίηση ουράς δύο άκρων με συνδεδεμένη λίστα 110
5.5 Παρατηρήσεις 111
Ασκήσεις 111
Βιβλιογραφία 113

Κεφάλαιο 6: Ουρές Προτεραιότητας 114

6.1 Ο αφηρημένος τύπος δεδομένων ουράς προτεραιότητας 114
6.2 Ουρές προτεραιότητας με στοιχειώδεις δομές δεδομένων 115
6.3 Δυαδικός σωρός 116
6.3.1 Υλοποίηση σε Java 125
6.3.2 Κατασκευή δυαδικού σωρού με δεδομένα κλειδιά 126
6.4 δ-Σωρός 128
6.5 Ταξινόμηση με ουρά προτεραιότητας 129
6.6 Ουρές προτεραιότητας με ευρετήριο 130
Ασκήσεις 131
Βιβλιογραφία 132

Κεφάλαιο 7: Λεξικά και Δυαδικά Δένδρα Αναζήτησης 133

7.1 Ο αφηρημένος τύπος δεδομένων λεξικού 133
7.1.1 Διατεταγμένα λεξικά 134
7.2 Στοιχειώδεις υλοποιήσεις με πίνακες και λίστες 135

7.2.1 Υλοποίηση με πίνακα ... 136
7.2.2 Υλοποίηση με αριθμοδείκτη ... 137
7.2.3 Υλοποίηση με συνδεδεμένη λίστα ... 137
7.3 Δυαδικά δένδρα αναζήτησης ... 138
7.3.1 Αναζήτηση ... 139
7.3.2 Εύρεση ελάχιστου και μέγιστου ... 141
7.3.3 Προκάτοχος και διάδοχος ... 142
7.3.4 Εισαγωγή ... 143
7.3.5 Διαγραφή ... 144
7.3.6 Επιλογή ... 146
7.3.7 Ένωση ... 147
7.3.8 Διαχωρισμός ... 148
7.4. Τυχαία κατασκευασμένα δυαδικά δένδρα αναζήτησης ... 149
7.5. Υλοποίηση δυαδικών δένδρων αναζήτησης σε Java ... 150
Ασκήσεις ... 153
Πειραματικές Μελέτες ... 154
Βιβλιογραφία ... 154

Κεφάλαιο 8: Ισορροπημένα Δένδρα Αναζήτησης ... 155

8.1 Κατηγορίες ισορροπημένων δένδρων αναζήτησης ... 155
8.1.1 Περιστροφές ... 156
8.2 Δένδρα AVL ... 157
8.2.1 Αποκατάσταση συνθήκης ισορροπίας ... 158
8.3 Αρθρωτά Δένδρα ... 166
8.3.1 Ιδιότητες των αρθρωτών δένδρων ... 171
8.3.2 Υλοποίηση σε Java ... 172
8.4 (a,b)-Δένδρα ... 174
8.4.1 Ύψος ενός (a,b)-δένδρου ... 175
8.4.2 Διάσπαση και συγχώνευση κόμβων ... 176
8.4.3 (2,4)-δένδρα ... 176
8.5 Κοκκινόμαυρα δένδρα ... 184
8.5.1 Αποκατάσταση των συνθήκων ισορροπίας ... 186
Ασκήσεις ... 195
Βιβλιογραφία ... 196

Κεφάλαιο 9: Κατακερματισμός ... 197

9.1 Εισαγωγή ... 197

9.2 Συναρτήσεις Κατακερματισμού 199
9.3 Επίλυση συγκρούσεων 200
9.3.1 Ξεχωριστές αλυσίδες 200
9.3.2 Μεταβλητές διευθύνσεις 201
Ανάλυση αναμενόμενης περίπτωσης 204
9.3.3 Κατακερματισμός του κούκου 208
9.4 Καθολικές οικογένειες συναρτήσεων κατακερματισμού 209
Ασκήσεις 210
Βιβλιογραφία 212

Κεφάλαιο 10: Ψηφιακά Λεξικά 213

10.1 Εισαγωγή 213
10.2 Ψηφιακά Δένδρα 214
10.3 Υλοποίηση σε Java 222
10.4 Συμπιεσμένα και τριαδικά ψηφιακά δένδρα 223
Ασκήσεις 225
Βιβλιογραφία 226

Κεφάλαιο 11: Ένωση Ξένων Συνόλων 227

11.1 Εισαγωγή 227
11.2 Εφαρμογή στο Πρόβλημα της Συνεκτικότητας 228
11.3 Δομή Ξένων Συνόλων με Συνδεδεμένες Λίστες 229
11.4 Δομή Ξένων Συνόλων με Ανοδικά Δένδρα 232
11.4.1 Συμπίεση Διαδρομής 234
11.5 Υλοποίηση σε Java 235
Ασκήσεις 237
Βιβλιογραφία 237

Κεφάλαιο 12: Διαχείριση Μνήμης 239

12.1 Ιεραρχία Μνήμης 239
12.2 Εξωτερική Μνήμη 241
12.2.1 Μοντέλο εξωτερικής μνήμης 241
12.2.2 Διατεταγμένο αρχείο με ευρετήριο 241
12.2.3 Β-δένδρα 242
12.3 Συλλογή Απορριμμάτων 244
Βιβλιογραφία 247

Κεφάλαιο 13: Αντισταθμιστική Ανάλυση……..248

13.1 Αντισταθμιστική Ανάλυση 248
13.2 Μέθοδοι Αντισταθμιστικής Ανάλυσης 250
13.2.1 Η χρεωπιστωτική μέθοδος 250
13.2.2 Η ενεργειακή μέθοδος 251
Ασκήσεις 252
Βιβλιογραφία 254

Κεφάλαιο 14: Προηγμένες Ουρές Προταιότητας……...…255

14.1 Διωνυμικά Δένδρα 255
14.2 Διωνυμικές Ουρές 258
14.1.1 Εισαγωγή στοιχείου σε διωνυμική ουρά 258
14.1.2 Διαγραφή μεγίστου από διωνυμική ουρά 262
14.1.3 Ένωση δύο διωνυμικών ουρών 263
14.1.4 Κατασκευή διωνυμικής ουράς με *N* κλειδιά 265
14.2 Σωροί Fibonacci 265
14.2.1 Δυναμικό σωρού Fibonacci 266
14.2.2 Εύρεση ελάχιστου κλειδιού 267
14.2.3 Εισαγωγή κλειδιού 267
14.2.4 Ένωση δύο σωρών Fibonacci 268
14.2.5 Εξαγωγή ελάχιστου κλειδιού 269
14.2.6 Μείωση κλειδιού 276
14.2.7 Διαγραφή κλειδιού 278
Βιβλιογραφία 279

Κεφάλαιο 15: Γλώσσα Προγραμματισμού Java……....280

15.1 Δομή της Java 280
15.1.1 Βασικοί Τύποι 283
15.1.2 Κλάσεις, Αντικείμενα, και Μέθοδοι 286
15.1.3 Πίνακες 290
15.1.4 Η κλάση String 290
15.2 Κληρονομικότητα και Πολυμορφισμός 291
15.3 Διασυνδέσεις και Αφηρημένες Κλάσεις 291
15.4 Γενικοί Τύποι 292
15.5 Ασκήσεις 293
Βιβλιογραφία 294

Πίνακας Συντομεύσεων-Ακρωνύμιων

AVL	*Adelson-Velskii και Landis*
BFS	*Κατά-πλάτος ή Οριζόντια Διερεύνηση* (Breadth-First Search)
BST	*Δυαδικό δένδρο αναζήτησης* (Binary Search Tree)
Dequeue	*Ουρά δύο άκρων* (Double-ended queue)
DFS	*Κατά-βάθος ή Καθοδική Διερεύνηση* (Depth-First Search)
GCD (ΜΚΔ)	*Μέγιστος Κοινός Διαιρέτης* (Greatest Common Divisor)
PQ	*Ουρά προτεραιότητας* (Priority Queue)

Κεφάλαιο 1
Εισαγωγή

Περιεχόμενα

1.1 Αλγόριθμοι και Δομές Δεδομένων 9
1.2 Διατήρηση Διατεταγμένου Συνόλου 12
1.3 Ολοκληρωμένη Υλοποίηση σε Java 15
Ασκήσεις 18
Βιβλιογραφία 19

1.1 Αλγόριθμοι και Δομές Δεδομένων

«Αλγόριθμος» είναι μια λέξη που συναντάμε πλέον πολύ συχνά, σε διάφορες δραστηριότητες της καθημερινής ζωής. Φυσικά αυτό αποτελεί μια φυσιολογική συνέπεια της εισβολής των υπολογιστικών συσκευών τις οποίες χρησιμοποιούμε για πολύ απλές έως και πολύ σύνθετες εργασίες. Τυπικά, ένας αλγόριθμος είναι μια μέθοδος επίλυσης ενός προβλήματος η οποία διέπεται από τα ακόλουθα χαρακτηριστικά:

1. Είσοδος-έξοδος: ο αλγόριθμος λαμβάνει δεδομένα εισόδου με βάση τα οποία υπολογίζει δεδομένα εξόδου.
2. Ακρίβεια: κάθε βήμα του αλγόριθμου είναι καλά ορισμένο.
3. Μοναδικότητα: τα αποτελέσματα που παράγει κάθε βήμα καθορίζονται με μοναδικό τρόπο.
4. Πεπερασμένο πλήθος βημάτων: ο αλγόριθμος κάποια στιγμή τερματίζει την εκτέλεση του.

Το δεύτερο χαρακτηριστικό, από τα παραπάνω, εξασφαλίζει ότι δεν υπάρχει καμία ασάφεια στην ερμηνεία του κάθε βήματος του αλγόριθμου, ενώ το τρίτο εξασφαλίζει ότι τα αποτελέσματα που υπολογίζει ο αλγόριθμος σε κάθε βήμα εξαρτώνται μόνο από τα δεδομένα εισόδου και από τα αποτελέσματα που έχει υπολογίσει σε προηγούμενα βήματα.

Με τον όρο δεδομένα αναφερόμαστε σε ένα σύνολο από πληροφορίες οι οποίες αποθηκεύονται στον υπολογιστή για τη λύση ενός προβλήματος. Μια «δομή δεδομένων» προσφέρει στον αλγόριθμο μεθόδους αποθήκευσης και επεξεργασίας των δεδομένων, έτσι ώστε να συμβάλλει στην κατά το δυνατό πιο αποδοτική εκτέλεση του αλγόριθμου. Κάθε πρόγραμμα εκτελεί έναν αλγόριθμο και χρησιμοποιεί ορισμένες δομές δεδομένων, γεγονός που εκφράζεται από την «εξίσωση» του Niklaus Wirth:

Αλγόριθμοι + Δομές Δεδομένων = Προγράμματα

Η περιγραφή ενός αλγόριθμου μπορεί να γίνει με διάφορους τρόπους, όπως με φυσική γλώσσα, με διάγραμμα ροής, με ψευδο-κώδικα ή με κώδικα σε κάποια γλώσσα προγραμματισμού όπως η Java.

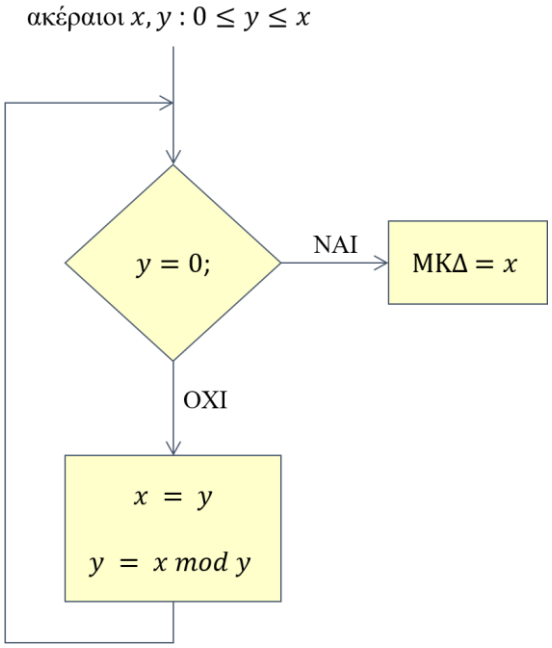
Ας θεωρήσουμε το πρόβλημα του υπολογισμού του μέγιστου κοινού διαιρέτη (ΜΚΔ) δύο μη αρνητικών ακέραιων. Ήδη πριν από το 300π.Χ. ήταν γνωστός ένας αλγόριθμος υπολογισμού του μέγιστου κοινού διαιρέτη: ο αλγόριθμος του Ευκλείδη. Έστω a και b δύο μη αρνητικοί ακέραιοι, όπου $b \neq 0$. Η ακέραιη διαίρεση του a με τον b δίνει ένα πηλίκο q και ένα υπόλοιπο r, τέτοια ώστε $a = qb + r$ και $0 \leq r \leq b - 1$. Για παράδειγμα, αν a=17 και b=6 τότε q=2 και r=5. Το πηλίκο της της ακέραιης διαίρεσης του a με τον b εκφράζει πόσες φορές μπορούμε να αφαιρέσουμε τον b από τον a και συμβολίζεται μαθηματικά ως $\lfloor a/b \rfloor$. Δηλαδή είναι ο μεγαλύτερος ακέραιος που είναι μικρότερος από το a/b. Το υπόλοιπο της ακέραιης διαίρεσης του a με τον b συμβολίζεται μαθηματικά με τη συνάρτηση mod, δηλαδή γράφουμε $r = a \bmod b = a - \lfloor a/b \rfloor b$. Ο b διαιρεί (ακριβώς) τον a όταν $a \bmod b = 0$. Έτσι, ο μέγιστος κοινός διαιρέτης δύο ακέραιων x και y είναι ο μεγαλύτερος ακέραιος που διαιρεί και τον x και τον y. Για παράδειγμα, ΜΚΔ(12,8)=4.

Ο αλγόριθμος του Ευκλείδη βασίζεται στην παρατήρηση για $x \geq y > 0$, ο μέγιστος κοινός διαιρέτης των x και y είναι ίσος με το μέγιστο κοινό διαιρέτη του y και του υπόλοιπου της διαίρεσης του x με το y, δηλαδή ισχύει ο ακόλουθος αναδρομικός ορισμός

$$\text{ΜΚΔ}(x, y) = \text{ΜΚΔ}(y, x \bmod y)$$

ενώ, για κάθε ακέραιο x ισχύει $\text{ΜΚΔ}(x, 0) = x$. Έτσι, μπορούμε να υπολογίσουμε το ΜΚΔ(x,y) αντικαθιστώντας κάθε φορά το x με το y και το y με το $x \bmod y$, μέχρι να γίνει y=0, οπότε η απάντηση δίνεται από την τρέχουσα τιμή του x.

Όπως διαπιστώνουμε πολύ συχνά, η διατύπωση ενός αλγόριθμου σε φυσική γλώσσα μπορεί να κρύβει ασάφειες ή αμφισημίες. Σε τέτοιες περιπτώσεις μπορεί να είναι πιο ενδεδειγμένη η χρήση κάποιου άλλου τρόπου περιγραφής του αλγόριθμου. Ας δούμε πρώτα πώς μπορούμε να περιγράψουμε τον αλγόριθμο του Ευκλείδη με ένα διάγραμμα ροής, όπως φαίνεται στην Εικόνα 1.1.



Εικόνα 1.1: Διάγραμμα ροής για τον υπολογισμό του μέγιστου κοινού διαιρέτη με τον αλγόριθμο του Ευκλείδη

Μπορούμε να περιγράψουμε την αντίστοιχη διαδικασία με ψευδο-κώδικα, όπως φαίνεται παρακάτω.

Αλγόριθμος του Ευκλείδη για τον υπολογισμό του Μέγιστου Κοινού Διαιρέτη (ΜΚΔ)

Είσοδος: Μη αρνητικοί ακέραιοι x και y με $x \geq y$.

Έξοδος: Μέγιστος κοινός διαιρέτης των x και y.

ΜΚΔ(x,y)

1. ενόσω $y \neq 0$
2. υπολόγισε το υπόλοιπο της διαίρεσης του x με τον y, r=x mod y.
3. θέσε x=y και y=r.
4. επίστρεψε x.

τέλος ΜΚΔ(x,y)

Ας δούμε τώρα δύο υλοποιήσεις του αλγόριθμου του Ευκλείδη σε Java. Η πρώτη υλοποίηση, gcd1, ακολουθεί πιστά τα βήματα του παραπάνω ψευδο-κώδικα.

```
static int gcd1(int x, int y)
{
     while (y!=0)
     {
         int r = x % y;  // υπόλοιπο ακέραιης διαίρεσης
         x = y;
         y = r;
     }
     return x;
}
```

Με τον προσδιορισμό static δηλώνουμε ότι η μέθοδος αφορά την κλάση η οποία την περιέχει, δηλαδή η gcd1 μπορεί να χρησιμοποιηθεί, χωρίς να υπάρχει κάποιο αντικείμενο.

Μια δεύτερη (ισοδύναμη) υλοποίηση του αλγόριθμου του Ευκλείδη, gcd2, χρησιμοποιεί απευθείας τον αναδρομικό ορισμό του μέγιστου κοινού διαιρέτη. Έτσι, λαμβάνουμε το ακόλουθο *αναδρομικό αλγόριθμο*.

```
static int gcd2(int x, int y)
{
     if (y==0) return x;
     return gcd2(y, x % y);
}
```

Ένας αναδρομικός αλγόριθμος επιλύει ένα πρόβλημα λύνοντας ένα ή περισσότερα στιγμιότυπα του ίδιου προβλήματος. Η αναδρομή αποτελεί μια από τις βασικές τεχνικές σχεδίασης αλγορίθμων, την οποία μελετάμε πιο αναλυτικά στο Κεφάλαιο 2.

Για να χρησιμοποιήσουμε τις παραπάνω μεθόδους, δημιουργούμε μια κλάση GCD, η οποία περιλαμβάνει τις μεθόδους gcd1 και η gcd2 και την οποία αποθηκεύουμε σε ένα αρχείο Java με το όνομα της κλάσης, δηλαδή GCD.java.

```
public class GCD {

     /* υπολογισμός του μέγιστου κοινού διαιρέτη δύο ακέραιων */
     public static int gcd1(int x, int y)
     {
        while (y!=0)
```

```
        {
            int r = x % y;  // υπόλοιπο ακέραιης διαίρεσης
            x = y;
            y = r;
        }
        return x;
    }

    /* αναδρομικός υπολογισμός του μέγιστου κοινού διαιρέτη δύο ακέραιων */
    static int gcd2(int x, int y)
    {
         if (y==0) return x;
         return gcd2(y, x % y);
    }


    public static void main(String args[])
    {
        int x = Integer.parseInt(args[0]);
        int y = Integer.parseInt(args[1]);

        System.out.println("gcd1 = "  + gcd1(x,y));
        System.out.println("gcd2 = "  + gcd2(x,y));
    }

}
```

Η μεταγλώττιση του προγράμματος γίνεται με την εντολή

javac GCD.java

Στο παραπάνω πρόγραμμα, η κλήση των μεθόδων gcd1 και gcd2 γίνεται από τη main μέθοδο της κλάσης. Οι τιμές των ακέραιων x και y δίνονται από τη γραμμή εντολών. Για παράδειγμα, για να υπολογίσουμε το ΜΚΔ(12,8) γράφουμε

java GCD 12 8

Το πρόγραμμα υπολογισμού του μέγιστου κοινού διαιρέτη είναι αρκετά απλό, ώστε να μην έχει ανάγκη να οργανώνει τα δεδομένα με κάποιο αποδοτικό τρόπο. Αρκούν μερικές μεταβλητές (τύπου int), για να αποθηκεύσει όλες τις απαιτούμενες πληροφορίες. Όταν έχουμε να διαχειριστούμε ένα μεγαλύτερο όγκο δεδομένων, τότε χρειαζόμαστε τη συμβολή μιας ή περισσότερων δομών δεδομένων. Από την οπτική γωνία ενός προγράμματος-χρήστη, η δομή δεδομένων υλοποιεί ένα *αφηρημένο τύπο δεδομένων*. Για να γίνει σαφής η παραπάνω διάκριση, στην επόμενη ενότητα μελετάμε το πρόβλημα της διατήρησης ενός διατεταγμένου συνόλου.

1.2 Διατήρηση Διατεταγμένου Συνόλου

Με τον όρο «διατεταγμένο σύνολο» αναφερόμαστε σε ένα σύνολο στοιχείων τα οποία έχουν μια ολική διάταξη, δηλαδή υπάρχει ένα πρώτο στοιχείο του συνόλου, ένα δεύτερο, ένα τρίτο, κ.ο.κ. Ισοδύναμα, για κάθε δύο στοιχεία x και y του συνόλου, μπορούμε να ελέγξουμε αν το x προηγείται του y στη διάταξη. Έστω S ένα διατεταγμένο σύνολο. Ορίζουμε την *τάξη* ενός στοιχείου $x \in S$ ως το πλήθος των στοιχείων που είναι μικρότερα του S. Για απλότητα θα θεωρήσουμε ότι το S είναι ένα σύνολο κατά τη μαθηματική έννοια, δηλαδή δεν περιέχει πολλαπλές εμφανίσεις του ίδιου στοιχείου. Με αυτήν την προϋπόθεση, η τάξη κάθε στοιχείου του S ορίζεται με μοναδικό τρόπο.

Ας υποθέσουμε τώρα ότι θέλουμε να υλοποιήσουμε ένα πρόγραμμα το οποίο χειρίζεται ένα τέτοιο διατεταγμένο σύνολο S, προκειμένου να εκτελεί κάποια στατιστική επεξεργασία των τιμών του. Για το σκοπό αυτό, το πρόγραμμα μας πρέπει να έχει τη δυνατότητα, ανά πάσα στιγμή, να εισάγει νέα στοιχεία στο S, καθώς και να βρίσκει το στοιχείο που έχει μια δεδομένη τάξη r στο τρέχον σύνολο S.

Μια καλή προγραμματιστική τεχνική είναι να αναθέσουμε τη διαχείριση του συνόλου S σε μια κατάλληλη δομή δεδομένων. Η καταλληλότητα της δομής καθορίζεται από το σύνολο των λειτουργιών τις οποίες πρέπει να επιτελεί πάνω στο S. Για το πρόγραμμά μας χρειαζόμαστε μια δομή η οποία υποστηρίζει τις παρακάτω λειτουργίες:

κατασκευή() : Επιστρέφει ένα κενό σύνολο S.

εισαγωγή(x) : Εισάγει στο S ένα νέο στοιχείο x.

επιλογή(j) : Επιστρέφει το j-οστό μικρότερο στοιχείο του S.

Με μια πρώτη ματιά, η σχεδίαση μιας δομής δεδομένων που υποστηρίζει το παραπάνω ρεπερτόριο λειτουργιών μπορεί να φαίνεται αρκετά απλή υπόθεση. Ωστόσο, αν θέλουμε να πετύχουμε καλή απόδοση τόσο στην εκτέλεση της εισαγωγής όσο και της επιλογής, έτσι ώστε να μην είναι απαραίτητο να εξετάζουμε πολλές θέσεις του πίνακα κάθε φορά που εκτελούμε μια από αυτές τις λειτουργίες, θα πρέπει να χρησιμοποιήσουμε μια από τις πιο προηγμένες δομές τις οποίες θα συναντήσουμε στα επόμενα κεφάλαια. Εδώ θα αρκεστούμε σε μια απλοϊκή λύση, η οποία διατηρεί τα στοιχεία του συνόλου S σε ένα πίνακα A.

Μια αρχική ιδέα είναι να τοποθετούμε τους ακέραιους, κατά τη σειρά εισαγωγής τους, στον πίνακα A. Με αυτόν τον τρόπο, το πρώτο στοιχείο που έχει εισαχθεί στο S βρίσκεται στη θέση A[0], το δεύτερο στη θέση A[1], κ.ο.κ. Όπως, όμως, γίνεται εύκολα αντιληπτό, με αυτή τη δομή δεν μπορούμε να εντοπίσουμε άμεσα, για οποιοδήποτε τιμή του j, το j-οστό μικρότερο στοιχείο του S. Κάτι τέτοιο φαίνεται να απαιτεί την ταξινόμηση των στοιχείων του S, οπότε μια εύλογη επιλογή είναι να διατηρούμε τον πίνακα A διατεταγμένο καθώς εισάγουμε νέα στοιχεία. Εδώ θα πρέπει να σημειώσουμε ότι στη βιβλιογραφία έχουν προταθεί αλγόριθμοι οι οποίοι μπορούν να βρίσκουν το j-οστό μικρότερο στοιχείο ενός συνόλου S, χωρίς να εκτελούν ταξινόμηση του. Ωστόσο, αυτοί οι αλγόριθμοι είναι πιο κατάλληλοι, όταν έχουμε να επιλέξουμε λίγα στοιχεία από το S. Στην περίπτωση μας, η δομή δεδομένων πρέπει να υποστηρίζει αποδοτικά την εκτέλεση ενός αυθαίρετου πλήθους λειτουργιών επιλογής.

Καταλήγουμε, λοιπόν, στην εξής λύση. Αποθηκεύουμε τα στοιχεία του S, κατά αύξουσα σειρά, σε ένα πίνακα A. Έτσι, αν το S έχει n στοιχεία, έχουμε $A[0] < A[1] < \cdots < A[n-1]$. Για απλότητα, θα θεωρήσουμε ότι το σύνολο S περιέχει μόνο ακέραιους (int) και θα αναπτύξουμε μια ενδεικτική υλοποίηση `SortedIntArray` με την ακόλουθη διασύνδεση.

```
class SortedIntArray        // διατεταγμένος πίνακας ακέραιων
{
   SortedIntArray(int N);   // αρχικοποίηση πίνακα N θέσεων
   void insert(Item);       // εισαγωγή αντικειμένου στη συλλογή
   int select(int j);       // επιλογή j-οστού μικρότερου στοιχείου
}
```

Στην κλάση `SortedIntArray`, εκτός από τον πίνακα ακεραίων A, χρησιμοποιούμε μια μεταβλητή n, η οποία διατηρεί το πλήθος των στοιχείων που έχουν εισαχθεί στον πίνακα A. Στην υλοποίησή μας, ο κατασκευαστής της κλάσης δέχεται ως όρισμα το μέγεθος N του πίνακα

που θα δημιουργήσει και δεσμεύει χώρο για πίνακα N θέσεων. Επίσης, καθώς ακόμα δεν έχει εισαχθεί κανένα στοιχείο, θέτει n=0.

```
class SortedIntArray {

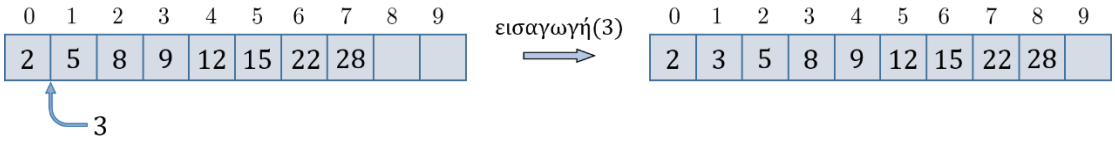
    int A[];
    int n;

    // αρχικοποίηση πίνακα N θέσεων
    SortedIntArray(int N) {
       A = new int[N];
       n = 0;
    }
}
```

Εφόσον διατηρούμε στον πίνακα A τα στοιχεία του S διατεταγμένα κατά αύξουσα σειρά, για τη λειτουργία επιλογή(j) χρειάζεται μόνο να επιστρέψουμε την τιμή A[j-1]. Έτσι, για την κλάση SortedIntArray, προσθέτουμε την παρακάτω μέθοδο.

```
/* επιλογή j-οστού μικρότερου στοιχείου */
int select(int j) {
    return A[j-1];
}
```

Ας εξετάσουμε τώρα την εισαγωγή ενός νέου στοιχείου x στο σύνολο S. Το στοιχείο αυτό θα πρέπει να τοποθετηθεί σε κατάλληλη θέση, ώστε να διατηρηθεί η διάταξη του πίνακα. Στη γενική περίπτωση, πρέπει να βρούμε τη θέση i του πίνακα για την οποία ισχύει $A[i-1] < x < A[i]$. Στη συνέχεια, πρέπει να δημιουργήσουμε χώρο, για να τοποθετηθεί το x στη θέση A[i], όπως φαίνεται στην Εικόνα 1.2: Εισαγωγή στοιχείου σε διατεταγμένο πίνακα. Για αυτό το σκοπό, μετακινούμε τα στοιχεία των θέσεων A[i], A[i+1], ..., A[n-1] κατά μια θέση δεξιά και θέτουμε A[i]=x. Η διαδικασία αυτή συμπεριλαμβάνει τις δύο ακραίες περιπτώσεις, όταν το x είναι μικρότερο από όλα τα στοιχεία του S ή όταν είναι μεγαλύτερο από όλα τα στοιχεία του S, αν θεωρήσουμε, κατά σύμβαση, ότι $A[-1] = -\infty$ και $A[n] = +\infty$.



Εικόνα 1.2: Εισαγωγή στοιχείου σε διατεταγμένο πίνακα

Στον ψευδο-κώδικα που ακολουθεί περιγράφουμε αναλυτικά τα βήματα του παραπάνω αλγόριθμου. Παρατηρήστε ότι στη γραμμή 4 ελέγχουμε αν το στοιχείο x υπάρχει ήδη αποθηκευμένο στον πίνακα A. Σε αυτή την περίπτωση, αφήνουμε τον A ως έχει, αφού κάναμε την παραδοχή ότι στο σύνολο S δεν μπορούμε να έχουμε πολλαπλές εμφανίσεις του ίδιου στοιχείου.

Αλγόριθμος εισαγωγής στοιχείου σε πίνακα ταξινομημένο σε αύξουσα σειρά

Είσοδος: Πίνακας A, ταξινομημένος σε αύξουσα σειρά, με τα στοιχεία ενός συνόλου S και ένα νέο στοιχείο x.

Έξοδος: Πίνακας A, ταξινομημένος σε αύξουσα σειρά με τα στοιχεία του συνόλου $S \cup \{x\}$.

1. θέσε n = πλήθος στοιχείων στον πίνακα A
2. βρες την πρώτη θέση i, $0 \leq i \leq n-1$, για την οποία $A[i] \geq x$
3. αν δεν υπάρχει τέτοια θέση, τότε θέσε i=n
4. αλλιώς, αν A[i]=x, επίστρεψε A
5. για j=n-1 έως i κάνε
6. θέσε A[j+1]=A[j]
7. θέσε A[i]=x
8. επίστρεψε A

τέλος

Μια υλοποίηση του αλγόριθμου εισαγωγής για την κλάση `SortedIntArray` δίνεται παρακάτω.

```
/* εισαγωγή ακέραιου x σε διατεταγμένο πίνακα */
void insert(int x) {
     int i;

     // εύρεση της θέσης εισαγωγής i
     for (i=0; i<n; i++) {
         if (A[i] >= x) break;
     }

     // αν το x υπάρχει ήδη, τότε ο πίνακας A μένει ως έχει
     if (A[i]==x) return;

     // μετακίνηση στοιχείων από τη θέση i και μετά κατά μια θέση δεξιά
     for (int j=n-1; j>=i; j--) {
         A[j+1] = A[j];
     }

     A[i]=x;
     n++;
}
```

1.3 Ολοκληρωμένη Υλοποίηση σε Java

Η υλοποίηση της κλάσης `SortedIntArray` την οποία δώσαμε στην προηγούμενη ενότητα, δεν είναι ολοκληρωμένη, καθώς δεν προβλέπει τι συμβαίνει στις ακόλουθες περιπτώσεις:

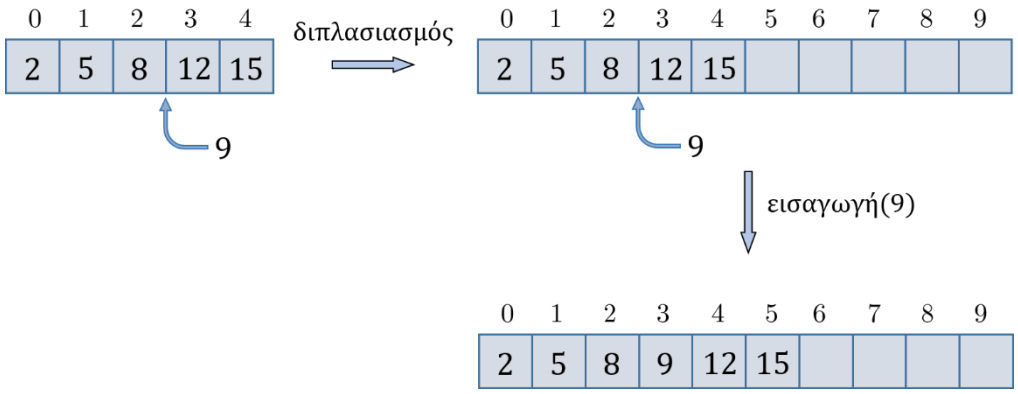
1. Όταν εκτελούμε την εισαγωγή ενός νέου στοιχείου, αλλά ο πίνακας A είναι ήδη γεμάτος (n==N).

2. Όταν σε μια λειτουργία επιλογή(j) ζητούμε ένα στοιχείο που δεν υπάρχει στον πίνακα, δηλαδή όταν j<1 ή j>n.

Σε αυτές τις περιπτώσεις, η κλήση των αντίστοιχων μεθόδων, insert και select, της κλάσης SortedIntArray θα έχει ως αποτέλεσμα τον τερματισμό του προγράμματος λόγω σφάλματος κατά το χρόνο εκτέλεσης. Τέτοιες περιπτώσεις μπορεί να μην είναι δυνατό να συμβούν σε ένα

πρόγραμμα που κάνει σωστή χρήση της δομής διατεταγμένου πίνακα, ωστόσο η υλοποίησή μας θα πρέπει να προβλέπει τέτοια πιθανά σφάλματα.

Για την αντιμετώπιση του πρώτου προβλήματος, μια επιλογή είναι απλώς να ενημερώνουμε το χρήστη ότι η δομή του διατεταγμένου πίνακα είναι γεμάτη. Αυτή η λύση δεν είναι πάντοτε ενδεδειγμένη, καθώς συχνά δεν μπορούμε να έχουμε μια καλή εκτίμηση του μεγέθους ενός συνόλου στοιχειών που πρέπει να επεξεργαστούμε. Έτσι, εναλλακτικά, μπορούμε να δεσμεύσουμε ένα μεγαλύτερο πίνακα Τ για το σύνολο μας, το οποίο χρησιμοποιούμε στη θέση του παλαιότερου πίνακα Α. Μετά τη δημιουργία του νέου πίνακα Τ, θα πρέπει να αντιγράψουμε εκεί όλα τα στοιχεία του Α. Για να αποφύγουμε το ενδεχόμενο να γίνεται μια τέτοια μετακίνηση στοιχείων πολύ συχνά, πρέπει να επιλέξουμε το μέγεθος του νέου πίνακα να είναι αρκετά μεγάλο, έτσι ώστε να χρειαστούν αρκετές εισαγωγές μέχρι να γεμίσει. Μια τυπική επιλογή είναι να θέσουμε το μέγεθος του Τ να είναι διπλάσιο από το μέγεθος του Α. Ένα τέτοιο παράδειγμα φαίνεται στην Εικόνα 1.3.



Εικόνα 1.3: Εισαγωγή στοιχείου σε δυναμικό πίνακα. Αν ο πίνακας είναι γεμάτος πριν από την εισαγωγή του νέου στοιχείου, τότε δεσμεύουμε ένα νέο πίνακα μεγαλύτερου μεγέθους (τυπικά με διπλάσιο μέγεθος από το αρχικό) στον οποίο αντιγράφουμε όλα τα στοιχεία του αρχικού πίνακα. Στη συνέχεια, μπορούμε να εκτελέσουμε τη διαδικασία εισαγωγής χωρίς πρόβλημα.

Στην υλοποίηση της κλάσης SortedIntArray προσθέτουμε την παρακάτω μέθοδο η οποία πραγματοποιεί την αλλαγή του μεγέθους του πίνακα Α.

```
// αλλαγή μεγέθους του πίνακα A
private void resize(int M) {
    int[] temp = new int[M];
    for (int i = 0; i < n; i++) {
        temp[i] = A[i];
    }
    A = temp;
}
```

Τώρα μπορούμε να τροποποιήσουμε τη μέθοδο εισαγωγής, έτσι ώστε να ελέγχει πρώτα αν ο πίνακας Α είναι ήδη γεμάτος. Σε αυτήν την περίπτωση, καλούμε τη μέθοδο resize, για να δεσμεύσουμε ένα νέο πίνακα Τ, με διπλάσιο μέγεθος, στον οποίο αντιγράφουμε τα στοιχεία του πίνακα Α. Τέλος, κάνουμε την αναφορά στον πίνακα Α να δείχνει στον πίνακα Τ. Με αυτόν τον τρόπο, αντικαθιστούμε τον παλιό πίνακα Α της δομής μας με ένα νέο πίνακα διπλάσιου μεγέθους.

```
void insert(int x) {
     // έλεγχος αν ο πίνακας είναι γεμάτος
     if (n == A.length) {
         resize(2*A.length); // διπλασιασμός του πίνακα A
     }

     int i;
     // εύρεση της θέσης εισαγωγής i
     for (i=0; i<n; i++) {
         if (A[i] >= x) break;
     }

     // αν το x υπάρχει ήδη, τότε ο πίνακας A μένει ως έχει
     if (A[i]==x) return;

     // μετακίνηση στοιχείων από τη θέση i και μετά κατά μια θέση δεξιά
     for (int j=n-1; j>=i; j--) {
         A[j+1] = A[j];
     }

     A[i]=x;
     n++;
}
```

Στη συνέχεια, πρέπει να αντιμετωπίσουμε και τη δεύτερη προβληματική περίπτωση, η οποία προκύπτει, όταν καλούμε τη λειτουργία επιλογή(j) για j<1 ή j>n. Μπορούμε να χειριστούμε μια τέτοια περίπτωση με το μηχανισμό εξαιρέσεων της Java, όπως φαίνεται στον παρακάτω κώδικα.

```
int select(int j) {
        if ( (j<=0) || (j>=n) )
           throw new NoSuchElementException("Bad index " + j);
        return A[j-1];
}
```

Οι εξαιρέσεις (exceptions) στη Java είναι αντικείμενα που ενεργοποιούνται σε περίπτωση μη αναμενόμενης λειτουργίας του προγράμματος. Ένα τέτοιο παράδειγμα είναι η κλήση της μεθόδου select(j), όταν η τιμή της παραμέτρου j είναι εκτός των ορίων του πίνακα A. Σε αυτή την περίπτωση, στην υλοποίηση μας, η μέθοδος select μεταβιβάζει μια εξαίρεση τύπου `NoSuchElementException`.

Για πληρότητα, δίνουμε την ολοκληρωμένη υλοποίηση σε Java.

```
class SortedIntArray {

    int A[];
    int n;

    SortedIntArray(int N) {
        A = new int[N];
        n = 0;
    }

    private void resize(int M) {
        int[] temp = new int[M];
        for (int i = 0; i < n; i++) {
            temp[i] = A[i];
        }
```

```
        A = temp;
    }

    void insert(int x) {
        if (n == A.length) {
            resize(2*A.length);
        }
        int i;
        for (i = 0; i < n; i++) {
            if (A[i] >= x) {
                break;
            }
        }

        if (A[i]==x) return;

        for (int j = n - 1; j >= i; j--) {
            A[j + 1] = A[j];
        }
        A[i] = k;
        n++;
    }

    int select(int j) {
        if ( (j<=0) || (j>=n) ) throw new NoSuchElementException("Error");
        return A[j-1];
    }
}
```

Στα επόμενα κεφάλαια θα παραλείψουμε την περιγραφή χρήσης παρόμοιων ελέγχων, παρόλο που, όπως προαναφέραμε, είναι απολύτως απαραίτητοι για τη δημιουργία ενός ολοκληρωμένου κώδικα. Η παράλειψη των ελέγχων, ωστόσο, θα μας βοηθήσει να επικεντρωθούμε στα κύρια σημεία ανάπτυξης της εκάστοτε δομής δεδομένων, που θα μελετήσουμε στη συνέχεια.

Ασκήσεις

1.1 Οι αριθμοί Fibonacci $F_1, F_2, \ldots$, ορίζονται από την αναδρομική σχέση $F_k = F_{k-1} + F_{k-2}$, για $k \geq 3$, ενώ $F_1 = F_2 = 1$. Έτσι, έχουμε $F_3 = 2$, $F_4 = 3$, $F_5 = 5$ κλπ. Πόσες επαναλήψεις πραγματοποιεί ο αλγόριθμος του Ευκλείδη με είσοδο $x = F_k$ και $y = F_{k-1}$;

1.2 Υλοποιήστε στην κλάση SortedIntArray την παρακάτω λειτουργία

τάξη(k) : Αναζητά τον ακέραιο k στο διατεταγμένο πίνακα A. Αν βρεθεί τότε επιστρέφει τη θέση i του k στον πίνακα A, δηλαδή έχουμε A[i]=k. Διαφορετικά, επιστρέφει την τιμή -1.

1.3 Υλοποιήστε στην κλάση SortedIntArray έναν κατασκευαστή `SortedIntArray(int[] B)`, ο οποίος δέχεται ως όρισμα ένα μη διατεταγμένο πίνακα B. Ο κατασκευαστής θα πρέπει να αρχικοποιήσει το διατεταγμένο πίνακα A με τους ακέραιους του B.

1.4 Υλοποιήστε στην κλάση SortedIntArray την παρακάτω λειτουργία:

διαγραφή(k) : Διαγράφει τον ακέραιο k από το διατεταγμένο πίνακα A.

1.5 Σκεφτείτε τι θα συμβεί, αν στη μέθοδο `insert` της κλάσης SortedIntArray αντικαταστήσουμε τις γραμμές του κώδικα

```
if (n == A.length) {
    resize(2*A.length);
}
```

με

```
if (n == A.length) {
    resize(A.length+1);
}
```

Συγκρίνετε πειραματικά την απόδοση των δύο αντίστοιχων υλοποιήσεων.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Savitch, W. (2008). *Απόλυτη Java.* Ίων.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Wirth, N. (1985). *Algorithms and data structures.* Prentice Hall.

Κεφάλαιο 2
Ανάλυση Αλγορίθμων

Περιεχόμενα

2.1 Εισαγωγή 20
2.2 Εμπειρική και Θεωρητική Ανάλυση Αλγορίθμων 22
2.2.1 Εμπειρική Πολυπλοκότητα 22
2.2.2 Θεωρητική Πολυπλοκότητα 23
2.3 Ανάλυση Χειρότερης και Αναμενόμενης Περίπτωσης 25
2.4 Ασυμπτωτική Πολυπλοκότητα 28
2.5 Αναδρομικές Σχέσεις 33
Ασκήσεις 37
Βιβλιογραφία 38

2.1 Εισαγωγή

Όταν διατυπώνουμε ένα πρόγραμμα Η/Υ για την επίλυση ενός προβλήματος Π, στην πραγματικότητα διατυπώνουμε μια *μέθοδο* (method) η οποία είναι ανεξάρτητη από την γλώσσα προγραμματισμού που χρησιμοποιούμε και η συστηματική εκτέλεση των βημάτων της οποίας οδηγεί στη λύση του προβλήματος Π. Γενικά μιλώντας, μπορούμε να πούμε ότι η μέθοδος καθορίζει τα βήματα και τους κανόνες που πρέπει να ακολουθήσουμε για την επίλυση του προβλήματος Π.

Ο όρος *αλγόριθμος* (algorithm) χρησιμοποιείται στην επιστήμη της πληροφορικής, για να περιγράψει γενικά απλά μια μέθοδο επίλυσης ενός προβλήματος Π, και πιο αναλυτικά, για να περιγράψει μια πεπερασμένη (finite), αιτιοκρατική (deterministic) και αποτελεσματική (affective) μέθοδο επίλυσης του προβλήματος Π κατάλληλη για υλοποίηση σε ένα πρόγραμμα Η/Υ.

Μπορούμε να ορίσουμε έναν αλγόριθμο περιγράφοντας μια διαδικασία για την επίλυση ενός προβλήματος Π σε μια φυσική γλώσσα, ή γράφοντας ένα πρόγραμμα Η/Υ το οποίο υλοποιεί την διαδικασία. Είναι ενδιαφέρον να σημειώσουμε ότι οι αλγόριθμοι προϋπάρχουν των υπολογιστών, καθώς είναι γνωστό ότι έχουν διατυπωθεί αποτελεσματικοί αλγόριθμοι για πλήθος προβλημάτων εδώ και 2.300 χρόνια. Κλασσικό παράδειγμα ο αλγόριθμος του Ευκλείδη για υπολογισμό του μέγιστου κοινού διαιρέτη (greater common divisor or gcd) δύο ακέραιων αριθμών, τον οποίο συναντήσαμε στο πρώτο κεφάλαιο. Όπως είδαμε, ο αλγόριθμος βασίζεται στον κανόνα $\gcd(x, y) = \gcd(y, x \bmod y)$, όπου x, y θετικοί ακέραιοι αριθμοί με $x \geq y$. Στη συνέχεια, δίνουμε τον αλγόριθμο του Ευκλείδη σε γλώσσα προγραμματισμού Java και μια εκτέλεση αυτού με είσοδο 128 και 40.

```
int Euclid(int x, int y)
{
  if y == 0 return x;
  return Euclid(y, x%y);
}
```

```
Euclid (128,40)=
  Euclid (40,8)=
    Euclid (8,0)=
       8
```

Είναι, επίσης, ενδιαφέρον να σημειώσουμε ότι για κάποιο πρόβλημα ίσως να υπάρχουν περισσότεροι του ενός αλγόριθμοι που το επιλύουν. Για παράδειγμα, είναι σε όλους μας γνωστός ο κλασσικός αλγόριθμος πολλαπλασιασμού δύο ακεραίων αριθμών. Σε λιγότερους, όμως, είναι γνωστός ο αλγόριθμος πολλαπλασιασμού a la russe. Στο παρακάτω παράδειγμα δείχνουμε τον πολλαπλασιασμό των ακεραίων 19 και 45 με τον αλγόριθμο a la russe.

19	45	45
9	90	90
4	180	
2	360	
1	720	720

		855

Ο αλγόριθμος είναι απλός και, για τους ακεραίους 19 και 45 του παραδείγματος, περιγράφεται ως εξής:

(1) Επαναληπτικά, διαιρούμε τον 19 (μικρότερο) με το 2 παίρνοντας το ακέραιο μέρος και πολλαπλασιάζουμε τον 45 (μεγαλύτερο) επί 2.

(2) Παίρνουμε από τη στήλη του 45 (μεγαλύτερου) τους ακεραίους των οποίων ο αντίστοιχος ακέραιος στην στήλη του 19 (μικρότερου) είναι περιττός.

(3) Προσθέτουμε τους ακεραίους που επιλέξαμε στο βήμα (2) και επιστρέφουμε το αποτέλεσμα της πρόσθεσης.

Έχοντας μια πρώτη προσέγγιση της έννοιας του αλγόριθμου και έχοντας δώσει έως τώρα μερικά παραδείγματα απλών αλγορίθμων, θα μπορούσαμε με βεβαιότητα να ισχυριστούμε ότι για κάθε (επιλύσιμο) πρόβλημα Π υπάρχει ένα σύνολο αλγορίθμων $\{A_1, A_2 \ldots, A_k\}$ που το επιλύουν, $k \geq 1$. Με άλλα λόγια, για το πρόβλημα Π υπάρχει αλγόριθμος A_i, $1 \leq i \leq k$, τέτοιος ώστε για κάθε είσοδο Ι του Π ο αλγόριθμος A_i δίδει σωστό αποτέλεσμα.

Έχοντας αυτό ως δεδομένο, ας έρθουμε να κάνουμε την παρακάτω βασική παρατήρηση όσον αφορά την αλγοριθμική επίλυση ενός προβλήματος μέσω ενός Η/Υ.

- Ας υποθέσουμε ότι διαθέτουμε έναν αλγόριθμο ταξινόμησης A_1 ο οποίος ταξινομεί 100.000 στοιχεία σε 30 sec σε ένα Η/Υ μεσαίου μεγέθους. Έχει παρατηρηθεί ότι, εάν χρησιμοποιήσουμε έναν άλλο αλγόριθμο ταξινόμησης A_2 για το ίδιο πρόβλημα (ταξινόμηση των 100.000 στοιχείων), είναι πολύ πιθανό ο χρόνος επίλυσης του προβλήματος να αυξηθεί απίστευτα, έστω και αν αυτή τη φορά ο Η/Υ που χρησιμοποιούμε είναι χίλιες φορές γρηγορότερος από τον προηγούμενο.

Επομένως, είναι φυσικό να δημιουργούνται ερωτήματα, όπως:

- Είναι ο αλγόριθμος Α αποτελεσματικότερος του αλγόριθμου Β; Με άλλα λόγια, επιλύει ο αλγόριθμος Α το πρόβλημα Π σε λιγότερο χρόνο από τον αλγόριθμο Β;
- Πόσο θα αυξηθεί ο χρόνος εκτέλεσης του αλγόριθμου Α ή, ισοδύναμα, ο χρόνος επίλυσης του προβλήματος Π, εάν διπλασιάσουμε τα δεδομένα εισόδου του Π;
- Μπορώ να χρησιμοποιήσω τον αλγόριθμο Α, όταν τα δεδομένα εισόδου του προβλήματος Π, που επιλύει ο αλγόριθμος Α, είναι πολύ μεγάλα;
- Υπάρχει αλγόριθμος (ή, μπορώ να σχεδιάσω έναν) ο οποίος επιλύει το πρόβλημα Π σε δεδομένο χρόνο t;

Κλείνουμε την μικρή αυτή εισαγωγή του Κεφαλαίου λέγοντας ότι δεν αποτελεί υπερβολή να ισχυριστεί κάποιος ότι οι αλγόριθμοι αποτελούν θεμελιώδες, κεντρικό και ίσως το πιο σημαντικό πεδίο μελέτης στην επιστήμη της πληροφορικής.

2.2 Εμπειρική και Θεωρητική Ανάλυση Αλγορίθμων

Κατά το σχεδιασμό ενός αλγορίθμου θα πρέπει να λαμβάνονται υπόψη, και να ενσωματώνονται σε αυτόν, όλες εκείνες οι παράμετροι που καθιστούν τον αλγόριθμο αποδοτικό ή αποτελεσματικό (efficient). Οι κύριες παράμετροι απόδοσης ενός αλγόριθμου είναι οι εξής:

- Χρόνος εκτέλεσης,
- Απαιτούμενοι πόροι, π.χ. μνήμη, επικοινωνία (π.χ. σε κατανεμημένα συστήματα),
- Βαθμός δυσκολίας υλοποίησης.

Ένας αλγόριθμος θα λέγεται αποδοτικός ή αποτελεσματικός, εάν ελαχιστοποιεί τις παραπάνω παραμέτρους.

Θα εστιάσουμε κυρίως στην παράμετρο που αφορά το χρόνο εκτέλεσης ενός αλγόριθμου, χωρίς, ωστόσο, οι παράμετροι της απόδοσης που αφορούν την απαιτούμενη μνήμη, την επικοινωνία ή το βαθμό δυσκολίας της υλοποίησης του να αποτελούν δευτερεύοντα κριτήρια, τουλάχιστον σε ένα υπολογιστικό σύστημα.

Ένας αλγόριθμος με μικρό χρόνο εκτέλεσης θα λέμε ότι έχει μικρή πολυπλοκότητα χρόνου, ενώ αντίθετα ένας με μεγάλο χρόνο εκτέλεσης θα λέμε ότι έχει μεγάλη πολυπλοκότητα.

Διαθέτοντας ένα σύνολο αλγορίθμων $\{A_1, A_2 \dots, A_\kappa\}$, $k \geq 2$, που επιλύουν το ίδιο πρόβλημα Π, το ερώτημα που καλούμαστε να απαντήσουμε είναι πώς θα αποφασίσουμε ποιος από όλους τους k αλγόριθμους είναι ο πιο αποτελεσματικός, δηλαδή ποιος αλγόριθμος έχει τη μικρότερη πολυπλοκότητα.

Στο παραπάνω ερώτημα υπάρχουν δύο προσεγγίσεις:

- **Εμπειρική Προσέγγιση** (posteriori)
- **Θεωρητική Προσέγγιση** (priori)

2.2.1 Εμπειρική Πολυπλοκότητα

Η εμπειρική πολυπλοκότητα ενός αλγόριθμου υπολογίζεται μετρώντας το χρόνο εκτέλεσης του αλγόριθμου σε συγκεκριμένη μηχανή. Πιο συγκεκριμένα, για τον υπολογισμό της εμπειρικής πολυπλοκότητας ενός αλγόριθμου εκείνο που κάνουμε είναι:

- κωδικοποιούμε τον αλγόριθμο σε μια γλώσσα προγραμματισμού,
- εκτελούμε αυτόν σε ένα Η/Υ με πολλά και διάφορα μεγέθη εισόδων και
- μετρούμε το χρόνο εκτέλεσης του (χρόνος μηχανής) στο συγκεκριμένο Η/Υ.

Το χρόνο εκτέλεσης ενός αλγόριθμου θα μπορούσε να μετρηθεί χρησιμοποιώντας το παρακάτω κώδικα Java:

```
public static void main()
{
  long startTime = System.currentTimeMillis();
  ... //run algorithm
  long endTime   = System.currentTimeMillis();
  long totalTime = endTime - startTime;
}
```

Στον παρακάτω πίνακα δίδουμε τους χρόνους εκτέλεσης σε δευτερόλεπτα για διάφορες δομές χειρισμού ξένων συνόλων (βλέπε Κεφάλαιο 9) με είσοδο τυχαίο αρχείο με *n* στοιχεία και *m* ενώσεις συνόλων.

n	m	UF	UFW	UFWP
100000	100000	2.090	0.124	0.140
100000	200000	19.172	0.249	0.264
100000	400000	63.429	0.546	0.530
100000	800000	153.692	1.075	1.045
400000	800000	378.645	1.247	1.138

όπου,

UF = βασική δομή γρήγορης ένωσης,

UFW = δομή γρήγορης ένωσης με στάθμιση και

UFWP = δομή γρήγορης ένωσης με στάθμιση και συμπίεση διαδρομής.

2.2.2 Θεωρητική Πολυπλοκότητα

Αντίθετα, η θεωρητική πολυπλοκότητα καθορίζει μαθηματικά το χρόνο (και άλλες παραμέτρους) που απαιτεί ο αλγόριθμος, συναρτήσει του μεγέθους των εξεταζόμενων εισόδων του.

Το πλεονέκτημα της θεωρητικής προσέγγισης για τον υπολογισμό της αποτελεσματικότητας ενός αλγόριθμου είναι ότι:

- δεν εξαρτάται από τον Η/Υ,
- δεν εξαρτάται από την γλώσσα προγραμματισμού,
- δεν εξαρτάται από τις ικανότητες του προγραμματιστή.

Το ερώτημα είναι **ποια μονάδα** θα χρησιμοποιήσουμε, για να εκφράσουμε θεωρητικά την αποτελεσματικότητα (πολυπλοκότητα χρόνου) ενός αλγόριθμου. Για παράδειγμα, θα την εκφράσουμε σε sec; θα την εκφράσουμε σε κύκλους μηχανής; θα την εκφράσουμε σε βήματα;

Αρχή της Σταθερότητας. Η απάντηση στο παραπάνω ερώτημα δίδεται από την *αρχή της σταθερότητας* (principle of invariance), που διατυπώνεται ως εξής:

Δύο διαφορετικές εφαρμογές (implementations) του ίδιου αλγόριθμου, δηλαδή, όταν εκτελούνται σε διαφορετικές μηχανές, όταν γράφονται σε διαφορετικές γλώσσες προγραμματισμού, όταν κωδικοποιούνται από διαφορετικούς προγραμματιστές, κλπ, δεν διαφέρουν στην αποτελεσματικότητά τους περισσότερο από ένα σταθερό πολλαπλάσιο.

Αυτό σημαίνει ότι, εάν E_1 είναι η αποτελεσματικότητα μιας εφαρμογής του αλγόριθμου και E_2 η αποτελεσματικότητα μιας άλλης εφαρμογής του ίδιου αλγόριθμου, τότε ισχύει

$$E_1 = c \cdot E_2$$

όπου c μια σταθερά.

Από την αρχή της σταθερότητας, εστιάζοντας στη χρονική αποτελεσματικότητα (πολυπλοκότητα χρόνου) ενός αλγόριθμου Α, έχουμε ότι, εάν $T_1(n)$ και $T_2(n)$ είναι οι χρόνοι εκτέλεσης δύο διαφορετικών εφαρμογών του αλγόριθμου Α, όπου n είναι το μέγεθος της εισόδου, τότε υπάρχει πάντα σταθερά c, τέτοια ώστε $T_1(n) = c \cdot T_2(n)$, με n πολύ μεγάλο.

Οφείλουμε να επισημάνουμε για μία ακόμα φορά ότι η παραπάνω αρχή ισχύει ανεξάρτητα από τον Η/Υ, τη γλώσσα προγραμματισμού και τις ικανότητες του προγραμματιστή.

Βασικές Πράξεις. Βασική πράξη (elementary operation) ονομάζεται η πράξη της οποίας ο χρόνος εκτέλεσης φράσσεται άνω από μία σταθερά η οποία εξαρτάται μόνο από την χρησιμοποιούμενη εφαρμογή (Η/Υ, γλώσσα προγραμματισμού, ικανότητα προγραμματιστή, κλπ).

Επειδή ορίζουμε το χρόνο εκτέλεσης ενός αλγόριθμου με την έννοια του «σταθερού πολλαπλασίου», για την ανάλυση της πολυπλοκότητας χρόνου του αλγόριθμου (ή, ισοδύναμα, υπολογισμό χρόνου εκτέλεσης του αλγόριθμου) θα χρειαστούμε μόνο τον **αριθμό των βασικών πράξεων** που εκτελούνται από αυτόν και **όχι τον ακριβή χρόνο** που απαιτούν κάθε μία από αυτές τις πράξεις.

Είναι προφανές ότι το πλήθος των βασικών πράξεων ενός αλγόριθμου εξαρτάται άμεσα από το μέγεθος της εισόδου του.

Παράδειγμα: Ο υπολογισμός του εσωτερικού γινομένου δίδεται από τον παρακάτω τύπο (αλγόριθμο) και πρόγραμμα.

$$z = x^T y = \sum_{i=1}^{i=n} x_i y_i$$

```
z = 0;
for (i=0; i<n; i++)
{
  t = x[i]*y[i];
  z = z+t;
}
```

Είναι πολύ εύκολο να υπολογίσουμε τον χρόνο εκτέλεσης $T(n)$ του αλγόριθμου υπολογισμού του εσωτερικού γινομένου, μετρώντας το πλήθος των βασικών πράξεων του αντίστοιχου προγράμματος. Παίρνοντας για βασική πράξη την ανάθεση τιμής σε μεταβλητή (assignment), έχουμε:

$$T(n) = 1 + n + n + n$$

όπου,

η εντολή ανάθεσης $z = 0$ εκτελείται 1 φορά, $i = 0$ εκτελείται n φορές, $t = x[i] * y[i]$ εκτελείται n φορές, καθώς και η εντολή $z = z + t$ εκτελείται n φορές.

Εάν c_1, c_2, c_3 και c_4 είναι ο χρόνος εκτέλεσης των παραπάνω τεσσάρων εντολών ανάθεσης σε μια μηχανή, τότε η πολυπλοκότητα χρόνου του αλγόριθμου είναι:

$$T(n) = c_1 + c_2 n + c_3 n + c_4 n$$

Η παραπάνω σχέση γράφεται

$$T(n) = c_1 + c_0 n$$

όπου, $c_0 = c_2 + c_3 + c_4$.

Θα εκφράσουμε τη θεωρητική αποτελεσματικότητα (πολυπλοκότητα χρόνου) με τη χρήση σταθερών πολλαπλασίων. Θα λέμε ότι:

Ένας αλγόριθμος **εκτελείται σε χρόνο** $T(n)$ ή ο αλγόριθμος **απαιτεί χρόνο** $T(n)$, εάν υπάρχει θετική σταθερά c και μια εφαρμογή του αλγόριθμου, τέτοια ώστε για κάθε είσοδο μήκους n ο χρόνος εκτέλεσης του αλγόριθμου να φράσσεται άνω από $cT(n)$, όπου n είναι το μέγεθος της εισόδου του αλγόριθμου.

Αργότερα θα δούμε ότι η παραπάνω έκφραση αποτελεί τον ασυμπτωτικό συμβολισμό (asymptotic notation) της πολυπλοκότητας των αλγορίθμων.

2.3 Ανάλυση Χειρότερης και Αναμενόμενης Περίπτωσης

Είδαμε ότι το πλήθος των βασικών πράξεων ενός αλγόριθμου και, επομένως, ο χρόνος εκτέλεσης $T(n)$ αυτού, εξαρτάται άμεσα από το μέγεθος της εισόδου του και, επίσης, ότι ο χρόνος εκτέλεσης μιας βασικής πράξης φράσσεται άνω από μία σταθερά η οποία εξαρτάται μόνο από την εφαρμογή.

Εκφράζουμε, λοιπόν, την πολυπλοκότητα χρόνου $T(n)$ ενός αλγόριθμου ως συνάρτηση του μεγέθους της εισόδου του, δηλαδή

$$T(n) = f(|I|),$$

όπου $|I|$ το μέγεθος της εισόδου.

Αναλύοντας την πολυπλοκότητα χρόνου ενός αλγόριθμου, διακρίνουμε τις παρακάτω δύο περιπτώσεις:

- **Ανάλυση Χειρότερης Περίπτωσης** (worst-case analysis)
- **Ανάλυση Αναμενόμενης Περίπτωσης** (average-case analysis)

Γενικά, η ανάλυση αναμενόμενης (ή μέσης) περίπτωσης απαιτεί γνώση της κατανομής της εισόδου. Όταν η κατανομή δεν είναι γνωστή (κάτι που συμβαίνει συχνά!), συνήθως υποθέτουμε ομοιόμορφη κατανομή. Αντίθετα, η ανάλυση χειρότερης (ή χειρίστης) περίπτωσης δεν απαιτεί καμία γνώση για την κατανομή της εισόδου και ισχύει για κάθε είσοδο.

Χειρότερη περίπτωση. Έστω D_n είναι το σύνολο όλων των εισόδων μεγέθους n. Για κάθε είσοδο $I \in D_n$, ας είναι $t(I)$ το πλήθος των βασικών πράξεων που εκτελεί ο αλγόριθμος με είσοδο I, δηλαδή

$$t(I) : D_n \to N$$

Ορίζουμε την πολυπλοκότητα χρόνου $T(n)$ χειρίστης-περίπτωσης (worst-case complexity) του αλγόριθμου, συνήθως τη συμβολίζουμε $W(n)$, ως εξής:

$$W(n) = \max\{t(I) \mid I \in D_n\}$$

Αναμενόμενη περίπτωση. Έστω ότι μπορούμε να αντιστοιχίσουμε μια πιθανότητα $p(I)$ σε κάθε είσοδο $I \in D_n$, έτσι ώστε $p(I)$ να μας δίνει την πιθανότητα εμφάνισης της εισόδου I.

Ορίζουμε την πολυπλοκότητα χρόνου $T(n)$ μέσης-περίπτωσης (average-case complexity) του αλγόριθμου, συνήθως τη συμβολίζουμε $A(n)$, ως εξής:

$$A(n) = \sum_{I \in D_n} p(I)\, t(I)$$

Στη συνέχεια θα μελετήσουμε ένα απλό πρόβλημα, αυτό της γραμμικής διερεύνησης (linear search), θα δώσουμε τον αντίστοιχο αλγόριθμο (σε γλώσσα προγραμματισμού Java) και θα δείξουμε τον υπολογισμό της πολυπλοκότητάς του στη χειρότερη και στην αναμενόμενη περίπτωση.

> **Πρόβλημα Γραμμικής Διερεύνησης.** *Δοθείσης μιας ακολουθίας στοιχείων S μήκους n και ενός στοιχείου x, να ελεγχθεί εάν το δοθέν στοιχείο x ανήκει ή όχι στην ακολουθία S. Εάν ανήκει, θέλουμε τη θέση του στην S, άλλως σχετικό μήνυμα ότι το στοιχείο δεν ανήκει στην S.*

Ο αλγόριθμος της γραμμικής διερεύνησης διατυπωμένος σε γλώσσα προγραμματισμού Java είναι ο ακόλουθος:

```
public static int Linear_Search(int S[], int x){
    int i=1;
    while (i<= S.length && S[i]!=x){
        i=i+1;
    }
    if (i > S.length) i=0;
    return i;
}
```

Θα πάρουμε για βασικές πράξεις τις συγκρίσεις $i \leq n$ $(i <= S.length)$ και $S(i) \neq x$ (`S[i]!=x`) της εντολής `while` (αρκεί να πάρουμε μία από αυτές).

Ανάλυση χειρότερης περίπτωσης. Είναι προφανές ότι ο αλγόριθμος εκτελεί το μέγιστο πλήθος συγκρίσεων, εάν το δοθέν στοιχείο x δεν ανήκει στην ακολουθία S. Τότε πολύ εύκολα μπορεί να δει κάποιος ότι το πλήθος των συγκρίσεων είναι $n + 1$ (εάν το στοιχείο x βρίσκεται στην τελευταία θέση της ακολουθίας S, τότε οι συγυρίσεις είναι n). Επομένως, $W(n) = n + 1$.

Ανάλυση αναμενόμενης περίπτωσης. Ας έρθουμε τώρα να υπολογίσουμε την πολυπλοκότητα του αλγόριθμου στην αναμενόμενη περίπτωση. Αρχικά, υποθέτουμε ότι ισχύουν τα κάτωθι:

- όλα τα στοιχεία είναι διαφορετικά μεταξύ τους,
- το στοιχείο x της αναζήτησης υπάρχει στην ακολουθία S, δηλαδή $x \in S$,
- όλες οι θέσεις του στοιχείου x στην S έχουν την ίδια πιθανότητα εμφάνισης.

Έστω I_i είναι η είσοδος για την οποία ισχύει $x = S[i],\ 1 \leq i \leq n$.

Τότε

$$t(I_i) = i$$

Επομένως,

$$A(n) = \sum_{i=1}^{n} p(I_i) t(I_i)$$

$$= \sum_{i=1}^{n} \frac{1}{n}\, i \;=\; \frac{1}{n} \sum_{i=1}^{n} i \;=\; \frac{1}{n}\ \frac{n(n+1)}{2} \;=\; \frac{n+1}{2}$$

Ας έρθουμε τώρα να άρουμε την υπόθεση ότι το στοιχείο αναζήτησης x υπάρχει στην ακολουθία S. Σε αυτή την περίπτωση θα συμβολίσουμε με I_e κάθε είσοδο μήκους η οποία δεν περιέχει το στοιχείο x, δηλαδή $x \neq S[i]$ για κάθε $i, 1 \leq i \leq n$.

Προφανώς τότε ισχύει

$$t(I_e) = n + 1$$

Έστω q είναι η πιθανότητα το ζητούμενο στοιχείο x να βρίσκεται στην ακολουθία S. Τότε ισχύει

$$t(I_i) = q \cdot (1/n) \quad \text{και} \quad t(I_e) = 1 - q$$

Επομένως,

$$A(n) = \sum_{i=1}^{n} p(I_i)\, t(I_i) \; + \; p(I_e)\, t(I_e)$$

$$= \sum_{i=1}^{n} \frac{q}{n}\, i \; + (1-q)\, n \; = \; \frac{q}{n} \sum_{i=1}^{n} i \; + (1-q)\, n \; = \; \frac{q}{n} \; \frac{n\,(n+1)}{2} + \; (1-q)\, n$$

$$= \; \frac{q\,(n+1)}{2} + \; (1-q)\, n$$

Εάν $q = 1$ (το στοιχείο x βρίσκεται στην S), τότε έχουμε

$$A(n) = \frac{n+1}{2}$$

Όπως υπολογίσαμε, εάν $q = 1/2$ (το στοιχείο x βρίσκεται στην S με πιθανότητα 0.5), τότε

$$A(n) = \frac{n+1}{4} + \frac{n}{2} \; = \; \frac{3n+1}{4}$$

Επομένως, σε αυτή την περίπτωση (δηλ., το στοιχείο x να βρίσκεται στην S με πιθανότητα 0.5) πρέπει να διερευνηθεί κατά μέσο όρο περίπου $3/4$ της ακολουθίας S.

Αντισταθμιστική Πολυπλοκότητα. Αρκετά συχνά, όταν ένας αλγόριθμος ή μία δομή δεδομένων εκτελεί μια ακολουθία από πράξεις, κάθε πράξη μπορεί να έχει διαφορετικό κόστος ανάλογα με την στιγμή που εκτελείται. Ας υποθέσουμε, για παράδειγμα, ότι μια δομή δεδομένων αποθηκεύει n στοιχεία και εκτελεί μια ακολουθία από n πράξεις σε συνολικό χρόνο $T(n)$. Μπορεί το κόστος μίας πράξης στη χειρότερη περίπτωση, έστω $t_w(n)$, να είναι πολύ μεγάλο, αλλά το μέσο κόστος ανά πράξη $t_a(n) = T(n)/n$ να είναι αρκετά μικρότερο σε *οποιαδήποτε ακολουθία πράξεων*. Δηλαδή, μπορεί να ισχύει $t_a(n) \ll t_w(n)$ *ανεξάρτητα* από την ακολουθία πράξεων που εκτελούμε. Ορίζουμε τον *αντισταθμιστικό χρόνο* εκτέλεσης $t_a(n)$ μιας πράξης ως το μέσο κόστος εκτέλεσης της πράξης όταν εκτελούμε μία ακολουθία *χειρότερης περίπτωσης*. (Προσέξτε ότι, σε αντίθεση με την αναμενόμενη περίπτωση, εδώ δεν υπολογίζουμε το μέσο όρο για κάθε δυνατή ακολουθία εισόδου.) Η σημασία του αντισταθμιστικού χρόνου είναι ότι γνωρίζουμε πως οποιαδήποτε ακολουθία n πράξεων θα εκτελεστεί το πολύ σε $n \cdot t_a(n)$ χρόνο, το οποίο μπορεί να δίνει ένα καλύτερο (μικρότερο) άνω φράγμα από το $n \cdot t_w(n)$. Θα μελετήσουμε την αντισταθμιστική πολυπλοκότητα αναλυτικά στο Κεφάλαιο 13.

2.4 Ασυμπτωτική Πολυπλοκότητα

Έως τώρα, αυτό που είδαμε είναι ο υπολογισμός του πλήθους των Βασικών Πράξεων ενός αλγόριθμου και η ανάλυση της χειρότερης περίπτωσης $W(n)$ και αναμενόμενης περίπτωσης $A(n)$ της πολυπλοκότητας χρόνου αυτού.

Ένα εύλογο ερώτημα που τίθεται είναι το εξής: Μήπως με αυτή την προσέγγιση της πολυπλοκότητας υπεισέρχονται στον υπολογισμό μας και παράγοντες οι οποίοι δεν μας διευκολύνουν να εκτιμήσουμε την αποτελεσματικότητα ενός αλγόριθμου;

Επόμενο εύλογο ερώτημα που αμέσως τίθεται: Είναι οι παράγοντες αυτοί τόσο σημαντικοί, ώστε να μην μπορούν να αγνοηθούν;

Ας αναφερθούμε στο πρώτο ερώτημα. Έστω ότι έχουμε δύο αλγορίθμους Α και Β με τις ακόλουθες πολυπλοκότητες χειρότερης περίπτωσης $W_A(n)$ και $W_B(n)$, αντίστοιχα:

$$W_A(n) = \frac{n^2}{4} \quad \text{και} \quad W_B(n) = 10n^2$$

Εύκολα μπορεί κάποιος να δει ότι ισχύει

$$\frac{n^2}{4} < 10n^2 \quad \text{για } n < 40$$

Τότε, ο αλγόριθμος Α είναι αποτελεσματικότερος του αλγορίθμου Β. Όμως, χωρίς δυσκολία, παρατηρούμε ότι για μεγάλες τιμές του n (μέγεθος εισόδου), στο παράδειγμά μας για $n \geq 40$, ο αλγόριθμος Β είναι πολύ πιο αποτελεσματικός από τον αλγόριθμο Α .

Από το παραπάνω παράδειγμα γίνεται σαφές ότι χρειαζόμαστε ένα τρόπο να εξασφαλίσουμε μερικές κατηγορίες συναρτήσεων οι οποίες **εξαλείφουν μη-σημαντικούς παράγοντες**, όπως σταθερές, μικρού μεγέθους εισόδους, κλπ.

Ακολούθως θα ορίσουμε τρεις τέτοιες κατηγορίες συναρτήσεων, οι οποίες στη συνέχεια θα χρησιμοποιηθούν για τον ορισμό της συμπτωτικής πολυπλοκότητας και των συμβολισμών Ο, Ω και Θ.

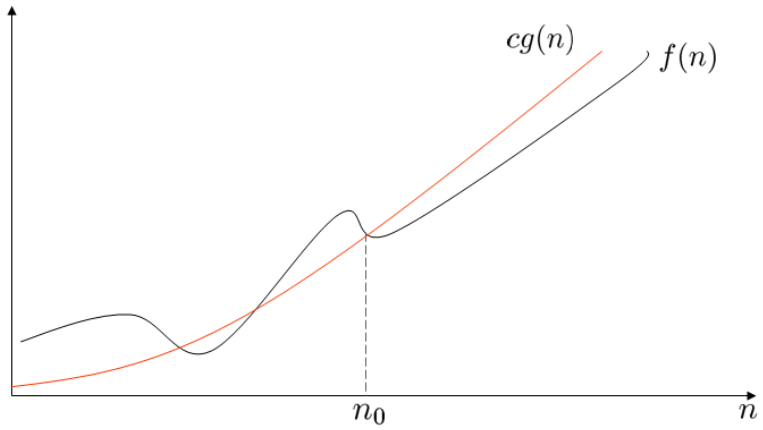
Συμβολισμός Ο (ασυμτωτικό άνω φράγμα)

Έστω η συνάρτηση $g: \mathbb{N} \to \mathbb{R}^*$. Ορίζουμε

$$O\big(g(n)\big) = \{f: \mathbb{N} \to \mathbb{R}^* \mid (\exists\, c \in \mathbb{R}^+)(\exists\, n_0 \in \mathbb{N})(\forall\, n \geq n_0)[f(n) \leq cg(n)]\}$$

να είναι το σύνολο όλων των συναρτήσεων $f: \mathbb{N} \to \mathbb{R}^*$ οι οποίες φράσσονται άνω από την $g(n)$, για κάθε $n \geq n_0$.

Το σύνολο $O\big(g(n)\big)$ θα ονομάζεται τάξη (order) της $g(n)$. Θα λέμε ότι η συνάρτηση $f(n)$ είναι τάξης $g(n)$, εάν $f(n) \in O\big(g(n)\big)$ ή, ισοδύναμα, $f(n) = O\big(g(n)\big)$.



Μια εναλλακτική τεχνική, για να δείξουμε ότι η συνάρτηση $f(n)$ ανήκει στο $O\big(g(n)\big)$, δηλαδή, είναι η ακόλουθη:

Για τις συναρτήσεις $f(n)$ και $g(n)$ ισχύει $f(n) = O\big(g(n)\big)$, εάν

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c < \infty$$

για κάποια σταθερά c, συμπεριλαμβανομένης της περίπτωσης όπου $c = 0$.

Παραδείγματα:

(α) $2n^3 + 5n^2 \log n = O(n^3)$ επειδή $\lim_{n\to\infty} \frac{2n^3 + 5n^2 \log n}{n^3} = 2$

(β) $\frac{n}{2 \log n} = O(n)$ επειδή $\lim_{n\to\infty} \frac{\frac{n}{2 \log n}}{n} = 0$

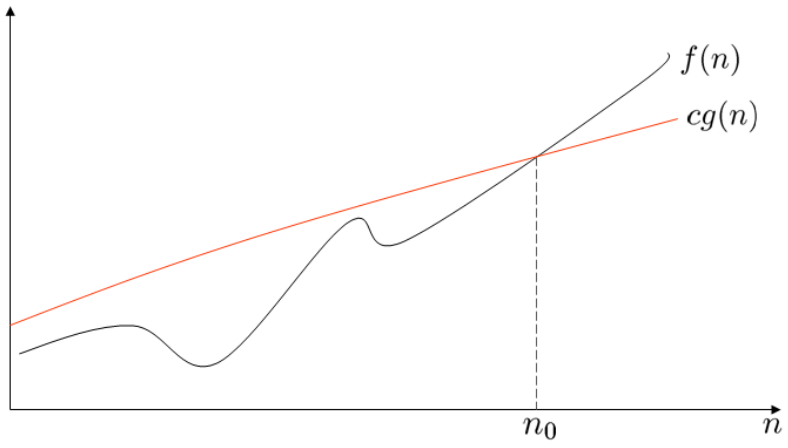
Συμβολισμός Ω (ασυμτωτικό άνω φράγμα)

Έστω $g\colon \mathbb{N} \to \mathbb{R}^*$ μια συνάρτηση. Ορίζουμε

$$\Omega\big(g(n)\big) = \{f\colon \mathbb{N} \to \mathbb{R}^* \mid (\exists\, c \in \mathbb{R}^+)(\exists\, n_0 \in \mathbb{N})(\forall\, n \geq n_0)[f(n) \geq cg(n)]\}$$

να είναι το σύνολο όλων των συναρτήσεων $f\colon \mathbb{N} \to \mathbb{R}^*$ οι οποίες φράσσονται κάτω από την $g(n)$, για κάθε $n \geq n_0$.

Το σύνολο $\Omega\big(g(n)\big)$ θα ονομάζεται ωμέγα (omega) της $g(n)$. Θα λέμε ότι η συνάρτηση $f(n)$ ανήκει στο ωμέγα της $g(n)$, εάν $f(n) \in \Omega\big(g(n)\big)$ ή, ισοδύναμα, $f(n) = \Omega\big(g(n)\big)$.



Μια εναλλακτική τεχνική, για να δείξουμε ότι η συνάρτηση $f(n)$ ανήκει στο $\Omega\big(g(n)\big)$, δηλαδή, $f(n) = \Omega\big(g(n)\big)$, είναι η ακόλουθη:

Για τις συναρτήσεις $f(n)$ και $g(n)$ ισχύει $f(n) = \Omega\big(g(n)\big)$, εάν

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c > 0$$

για κάποια σταθερά c, συμπεριλαμβανομένης της περίπτωσης όπου $c = \infty$.

Παραδείγματα:

(α) $2n^3 + 5n^2 \log n = \Omega(n^3)$ επειδή $\lim_{n\to\infty} \frac{n^3}{2n^3 + 5n^2 \log n} = \frac{1}{2}$

(β) $2n \log n = \Omega(n)$ επειδή $\lim_{n\to\infty} \frac{n}{2n \log n} = 0$

Συμβολισμός Θ (ασυμτωτικά αυστηρό φράγμα)

Έστω $g\colon \mathbb{N} \to \mathbb{R}^*$ μια συνάρτηση. Ορίζουμε

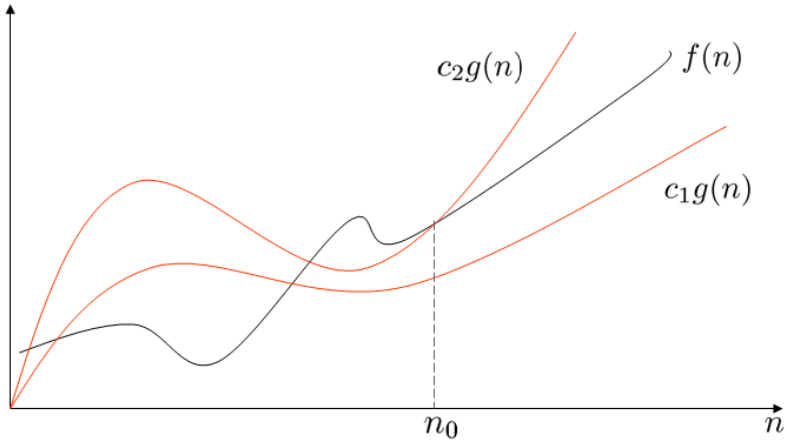
$$\Theta\big(g(n)\big) = \{f\colon \mathbb{N} \to \mathbb{R}^* \mid (\exists\, c_1, c_2 \in \mathbb{R}^+)(\exists\, n_0 \in \mathbb{N})(\forall\, n \geq n_0)[c_1 g(n) \leq f(n) \leq c_2 g(n)]\}$$

να είναι το σύνολο όλων των συναρτήσεων $f\colon \mathbb{N} \to \mathbb{R}^*$ οι οποίες φράσσονται άνω και κάτω από την $g(n)$, για κάθε $n \geq n_0$.

Ισοδύναμος ορισμός: Ορίζουμε

$$\Theta\big(g(n)\big) = O\big(g(n)\big) \cap \Omega\big(g(n)\big)$$

και ονομάζουμε το σύνολο $\Theta\big(g(n)\big)$ ακριβή τάξη της $g(n)$ (exact order of $g(n)$).



Μια εναλλακτική τεχνική, για να δείξουμε ότι η συνάρτηση $f(n)$ ανήκει στο $\Theta\big(g(n)\big)$, δηλαδή, $f(n) = \Theta\big(g(n)\big)$, είναι η ακόλουθη:

Για τις συναρτήσεις $f(n)$ και $g(n)$ ισχύει $f(n) = \Theta\big(g(n)\big)$, εάν

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$$

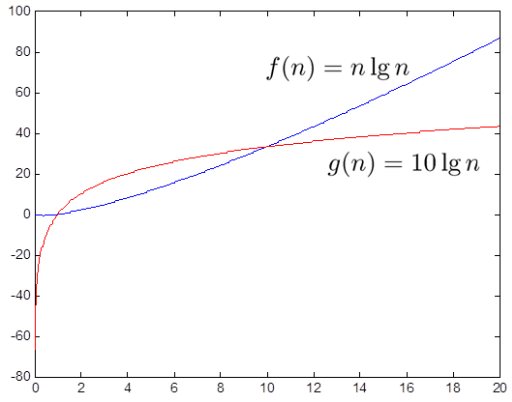
για κάποια σταθερά c, τέτοια ώστε $0 < c < \infty$.

Παράδειγμα: Έστω ότι θέλουμε να δείξουμε ότι $f(n) = \frac{1}{2}n^2 - 3n = \Theta(n^2)$. Αρκεί να βρούμε θετικές σταθερές c_1, c_2 και n_0, τέτοιες ώστε να ισχύει:

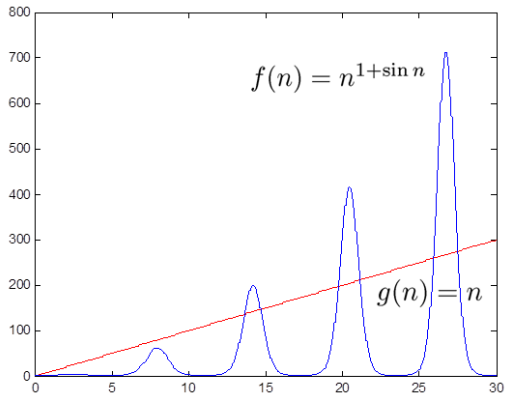
$$c_1 n^2 \le \frac{1}{2}n^2 - 3n \le c_2 n^2 \text{ για κάθε } n \ge n_0 \qquad (1)$$

Θέλουμε $c_2 \ge 1/2 - 3/n$, που ισχύει όταν $c_2 \ge 1/2$ και $n \ge 1$. Επίσης θέλουμε $0 < c_1 \le 1/2 - 3$. Έχουμε $1/2 - 3/n > 0$, και επομένως $n > 6$. Επιλέγοντας $\mathrm{n} = 7$, έχουμε $1/2 - 3/\mathrm{n} = 1/2 - 3/7 = 1/14$. Άρα, για $\mathrm{n_0} = 7$, $\mathrm{c_1} = 1/14$ και $\mathrm{c_2} = 1/2$, η συνθήκη (1) ικανοποιείται, και επομένως $f(n) = \frac{1}{2}n^2 - 3n = \Theta(n^2)$.

Προσοχή! Ακόμα και δύο γνησίως αύξουσες συναρτήσεις μπορεί να έχουν πολλά σημεία τομής (βλέπε Σχήμα 4.1(α) για τις συναρτήσεις $f(n) = n\log n$ και $g(n) = 10\ln n$. Επίσης, δεν είναι όλες οι συναρτήσεις ασυμπτωτικά συγκρίσιμες (βλέπε Σχήμα 4.1(β) για τις συναρτήσεις $f(n) = n^{1+\sin n}$ και $g(n) = n$).



(α)



(β)

Εικόνα 4.1.

Ιδιότητες

- Μεταβατικότητα: $f(n) = \Theta\big(g(n)\big)$ και $g(n) = \Theta\big(h(n)\big) \Rightarrow f(n) = \Theta\big(h(n)\big)$
- Αυτοπάθεια: $f(n) = \Theta\big(f(n)\big)$
- Συμμετρία: $f(n) = \Theta\big(g(n)\big) \Leftrightarrow g(n) = \Theta\big(f(n)\big)$

Οι ιδιότητες της μεταβατικότητα και της αυτοπάθεια ισχύουν και για τους συμβολισμούς O και Ω, όπου στη θέση της συμμετρίας έχουμε:

$$f(n) = O\big(g(n)\big) \Leftrightarrow g(n) = \Omega\big(f(n)\big)$$

$$f(n) = \Theta\big(g(n)\big) \Leftrightarrow f(n) = O\big(g(n)\big) \text{ και } g(n) = O\big(f(n)\big)$$

Κλάσεις Πολυπλοκότητας

- **Λογαριθμικοί Αλγόριθμοι.** Ένας αλγόριθμος θα λέγεται λογαριθμικός, εάν έχει χρόνο εκτέλεσης $T(n) = O(\log^k n)$, όπου k είναι μια θετική σταθερά.
- **Γραμμικοί Αλγόριθμοι.** Ένας αλγόριθμος λέγεται γραμμικός, εάν έχει χρόνο εκτέλεσης $T(n) = O(n)$, όπου n είναι το μέγεθος της εισόδου του αλγόριθμου.
- **Πολυωνυμικοί Αλγόριθμοι.** Ένας αλγόριθμος λέγεται πολυωνυμικός, εάν έχει χρόνο εκτέλεσης $T(n) = O(n^k)$, όπου k είναι μια θετική σταθερά.
- **Εκθετικοί Αλγόριθμοι.** Ένας αλγόριθμος λέγεται εκθετικός, εάν έχει χρόνο εκτέλεσης $T(n) = O(c^n)$, όπου $c > 1$.

Είναι αναγκαίο να αναφέρουμε ότι η **Κλάση Πολυπλοκότητας P** (Polynomial) περιλαμβάνει τα προβλήματα που επιδέχονται λύση σε πολυωνυμικό χρόνο (δηλ. υπάρχουν πολυωνυμικοί αλγόριθμοι που τα επιλύουν).

Πρακτικές πολυπλοκότητες

Στον παρακάτω πίνακα δείχνουμε το ρυθμό αύξησης διαφόρων τύπων συναρτήσεων για μικρές τιμές του $n = 1, 2, 4, 8$ και 16. Παρατηρούμε ότι η συνάρτηση $n!$ είναι «απαγορευτική», για να εκφράσει την πολυπλοκότητα ενός αλγόριθμου.

$\lg n$	n	$n \lg n$	n^2	n^3	2^n	$n!$
0	1	0	1	1	2	1
1	2	2	4	8	4	2
2	4	8	16	64	16	24
3	8	24	64	512	256	40320
4	16	64	256	4096	65536	20922789888000

Για καλύτερη κατανόηση του ρυθμού αύξησης των παραπάνω συναρτήσεων παραθέτουμε στη συνέχεια, πίνακα που δείχνει το ρυθμό αύξησης για λίγο μεγαλύτερες τιμές του $n = 2, 8, 32$ και 64.

	2	8	32	64
$\log n$	1 second	3 seconds	5 seconds	6 seconds
n	2 seconds	8 seconds	32 seconds	1.07 minutes
$n \log n$	2 seconds	24 seconds	2.67 minutes	6.4 minutes
n^2	4 seconds	1.07 minutes	17.07 minutes	1.14 hours
n^3	8 seconds	8.53 minutes	9.1 hours	3.03 days
2^n	4 seconds	4.27 minutes	1.36 centuries	5.86×10^9 centuries
$n!$	2 seconds	4.2 hours	8.34×10^{25} centuries	4.02×10^{79} centuries

Στον πίνακα αυτόν η τιμή της συνάρτησης εκφράζει το χρόνο εκτέλεσης ενός αλγόριθμου. Εδώ γίνεται εμφανές ότι ένας αλγόριθμος εκθετικής πολυπλοκότητας, για παράδειγμα $O(n!)$, είναι «απαγορευτικός» για πρακτικές εφαρμογές, καθώς ο χρόνος για την επίλυση ενός προβλήματος μεγέθους $n = 64$ είναι μεγαλύτερος από 10^{79} αιώνες.

Χρήσιμες συναρτήσεις στην ανάλυση αλγορίθμων

Κλείνουμε την παράγραφο παραθέτοντας (χωρίς απόδειξη) μερικές χρήσιμες συναρτήσεις που συχνά εμφανίζονται στην ανάλυση των αλγορίθμων.

(α) $\binom{N}{k} = \frac{N!}{(N-k)!\,k!}$ Διωνυμικοί συντελεστές

$\binom{N}{k} \approx \frac{N^k}{k!}$
για μικρή σταθερά k

(β) $\lg N! = \lg 1 + \lg 2 + \lg 3 + \cdots + \lg N \approx N \lg N$ Προσέγγιση Stirling

(γ) $1 + 2 + 4 + 8 + \cdots + 2^n = 2 \cdot 2^n - 1$ Γεωμετρικό άθροισμα

2.5 Αναδρομικές Σχέσεις

Πολύ συχνά ο χρόνος εκτέλεσης ενός αλγόριθμου μπορεί να εκφραστεί μέσω κάποιας αναδρομικής σχέσης.

Αναφέρουμε απλώς, χωρίς να επεκταθούμε στο σημείο αυτό, ότι υπάρχουν πολλές τεχνικές για την επίλυση μιας αναδρομικής σχέσης, όπως

- Επαναληπτική Αντικατάσταση
- Χρήση της Χαρακτηριστικής Εξίσωσης
- Γενική Μέθοδος (Master Method)

Για τις ανάγκες του συγγράμματος θα περιοριστούμε στην τεχνική της επαναληπτικής αντικατάστασης, την οποία θα δείξουμε με την επίλυση επιλεγμένων παραδειγμάτων.

Παράδειγμα 1: Έστω ότι μας δίνεται μία ακολουθία $A = (a_1, a_2, \dots, a_n)$ από n ακέραιους, ταξινομημένη κατ' αύξουσα τάξη, δηλαδή $a_1 \leq a_2 \leq \cdots \leq a_n$, και ένας ακέραιος b. Θέλουμε να βρούμε έναν ακέραιο $a_i \in A$ τέτοιο ώστε $a_i = b$.

Χρησιμοποιούμε δυαδική αναζήτηση, που σε κάθε βήμα είτε βρίσκει το b είτε αποκλείει τα μισά σχεδόν στοιχεία της ακολουθίας A που απομένουν. Για παράδειγμα, έστω η παρακάτω ακολουθία $n = 15$ στοιχείων και έστω ότι αναζητούμε το στοιχείο $b = 111$.

θέση	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
στοιχείο	3	8	9	12	28	33	45	56	88	90	102	128	233	245	356

Βρίσκουμε το μέσο της ακολουθίας A, που στο παράδειγμά μας είναι στη θέση $\frac{15+1}{2} = 8$ και είναι το στοιχείο $a_8 = 56$, και παρατηρούμε $a_8 = 56 < 111 = b$. Επομένως, διαγράφουμε τα στοιχεία $a_1, a_2, \dots, a_8$ της ακολουθίας A.

θέση	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	~~8~~	9	10	11	12	13	14	15
στοιχείο	~~3~~	~~8~~	~~9~~	~~12~~	~~28~~	~~33~~	~~45~~	~~56~~	88	90	102	128	233	245	356

⇧

Στη συνέχεια, βρίσκουμε το μέσο της ακολουθίας $A = (a_9, a_{10}, \dots, a_{15})$, που στο παράδειγμά μας είναι στη θέση $\frac{9+15}{2} = 12$ και είναι το στοιχείο $a_{12} = 128$, και παρατηρούμε $a_{12} = 128 > 111 = b$. Επομένως, διαγράφουμε τα στοιχεία $a_{12}, a_{13}, \dots, a_{15}$ της ακολουθίας A.

θέση	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	~~8~~	9	10	11	~~12~~	~~13~~	~~14~~	~~15~~
στοιχείο	~~3~~	~~8~~	~~9~~	~~12~~	~~28~~	~~33~~	~~45~~	~~56~~	88	90	102	~~128~~	~~233~~	~~245~~	~~356~~

⇧

Συνεχίζοντας βρίσκουμε το μέσο της ακολουθίας $A = (a_9, a_{10}, a_{11})$, που στο παράδειγμά μας είναι στη θέση $\frac{9+11}{2} = 10$ και είναι το στοιχείο $a_{10} = 90$, και παρατηρούμε $a_{10} = 90 < 111 = b$. Επομένως, διαγράφουμε τα στοιχεία a_9, a_{10} της ακολουθίας A.

θέση	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	~~8~~	~~9~~	10	11	~~12~~	~~13~~	~~14~~	~~15~~
στοιχείο	~~3~~	~~8~~	~~9~~	~~12~~	~~28~~	~~33~~	~~45~~	~~56~~	~~88~~	90	102	~~128~~	~~233~~	~~245~~	~~356~~

⇧

Τέλος, βρίσκουμε το μέσο της ακολουθίας $A = (a_{11})$, που στο παράδειγμά μας είναι στη θέση $\frac{11+11}{2} = 11$ και είναι το στοιχείο $a_{11} = 102$, και παρατηρούμε $a_{11} = 102 < 111 = b$. Επομένως, διαγράφουμε τα στοιχεία a_9, a_{10} της ακολουθίας A. Το στοιχείο $b = 111$ δεν υπάρχει στην ακολουθία A. Βρήκαμε, όμως, το αμέσως μικρότερο στοιχείο που είναι το 102.

θέση	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	~~8~~	~~9~~	~~10~~	11	~~12~~	~~13~~	~~14~~	~~15~~
στοιχείο	~~3~~	~~8~~	~~9~~	~~12~~	~~28~~	~~33~~	~~45~~	~~56~~	~~88~~	~~90~~	102	~~128~~	~~233~~	~~245~~	~~356~~

⇧

Το πρόγραμμα που περιγράφει τον παραπάνω αλγόριθμο δυαδικής αναζήτησης, διατυπωμένο σε γλώσσα προγραμματισμού Java, είναι το εξής :

```
public static int binarySearch(int a[], int b, int l, int r)
{   while (r >= l)
    {      int m = (l+r)/2;
           if (b == a[m]) return m;
           if (b <a[m]) r = m-1; else l = m+1;}
    return -1;
}
```

Η εντολή $\mathrm{m} = (\mathrm{l} + \mathrm{r})/2$ υπολογίζει το μέσο της ακολουθίας εισόδου, ενώ οι μεταβλητές l και r ορίζουν τα άκρα της εκάστοτε ακολουθίας.

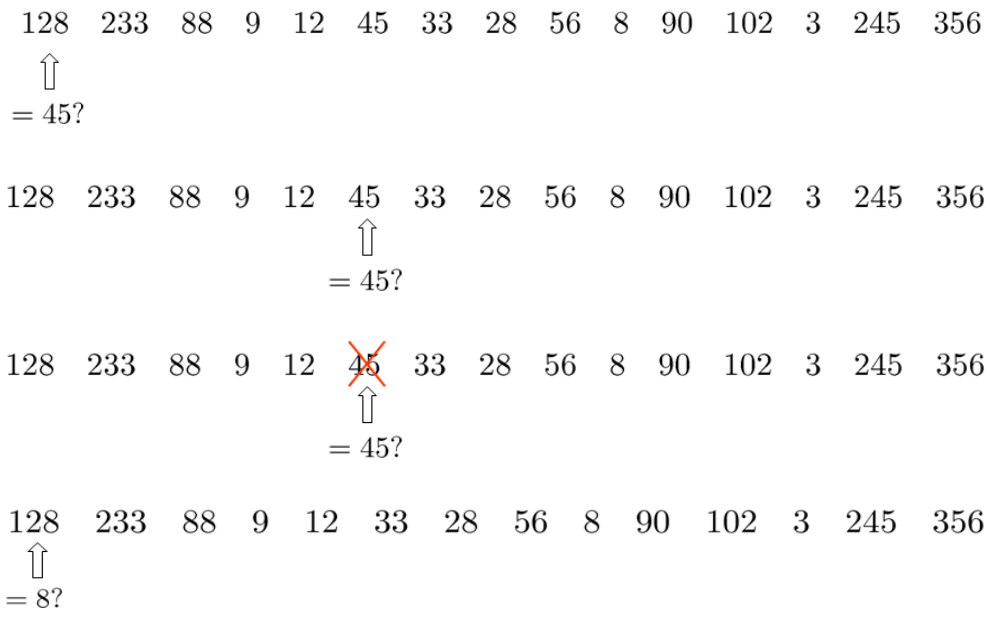
Ανάλυση Αλγόριθμου. Έστω $T(n)$ ο χρόνος της αναζήτησης σε ακολουθία n στοιχείων. Τότε, στη χειρότερη περίπτωση, ισχύει:

$$\begin{aligned} T(n) &= T(n/2) + \Theta(1) = T(n/2) + c \\ &= T(n/4) + 2c = T(n/8) + 3c = \cdots = c\log n \\ &= \Theta(\log n) \end{aligned}$$

Μάλιστα, χρησιμοποιώντας πιο περίπλοκες δομές μπορούμε να πετύχουμε $\Theta(\log\log n)$ χρόνο επίλυσης για το πρόβλημα της αναζήτησης.

Παράδειγμα 2: Έστω ότι θέλουμε να υπολογίσουμε την πολυπλοκότητα του αλγόριθμου για το πρόβλημα της ακολουθιακής αναζήτησης μιας ακολουθίας n στοιχείων και απαλοιφή ενός στοιχείου κάθε φορά, μέχρι να μείνει κενή ακολουθία.

Παρακάτω δίνουμε ένα παράδειγμα μιας ακολουθίας $n = 15$ στοιχείων (ακέραιοι αριθμοί). Αρχικά, γίνεται αναζήτηση του στοιχείου 45, το οποίο βρίσκεται στην 6η θέση και διαγράφεται. Στη συνέχεια, γίνεται αναζήτηση του στοιχείου 8, το οποίο βρίσκεται στην 9η θέση (της ακολουθίας μετά τη διαγραφή του 8) και διαγράφεται. Η διαδικασία συνεχίζεται, έως ότου η ακολουθία μείνει κενή.



Έστω $T(n)$ ο χρόνος εκτέλεσης για ακολουθία n στοιχείων στη χειρότερη περίπτωση. Τότε, ισχύει:

$$T(n) = \begin{cases} T(n-1) & n \geq 2 \\ 1 & n = 1 \end{cases}$$

Με τη μέθοδο της αντικατάστασης λαμβάνουμε:

$$\begin{aligned} T(n) &= T(n-1) + n = T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n \\ &= \cdots \\ &= T(1) + 2 + 3 + \cdots + (n-2) + (n-1) + n = \frac{n(n+1)}{2} \end{aligned}$$

Με επαγωγή θέλουμε να δείξουμε ότι $T(n) = O(n^2)$, δηλαδή ότι ισχύει $T(n) \leq cn^2$ για κάποια θετική σταθερά c.

Η βάση της επαγωγής είναι $T(1) - 1 \leq c1^2 - c$, που ισχύει για $c \geq 1$.

Επαγωγικό βήμα: Έστω ότι ισχύει $T(n-1) \leq c(n-1)^2$.

Έχουμε $T(n) = T(n-1) + n \leq c(n-1)^2 + n \leq c[(n-1)^2 + n]$
$= c(n^2 - n + 1) \leq cn^2, \quad n \geq 1$

Με επαγωγή, επίσης, μπορούμε να δείξουμε ότι ισχύει $T(n) = \Omega(n^2)$, δηλαδή ότι ισχύει $T(n) \geq c'n^2$ για κάποια θετική σταθερά c'.

Η βάση της επαγωγής είναι $T(1) = 1 \geq c'1^2 = c'$, που ισχύει για $c' \geq 1$.

Επαγωγικό βήμα: Έστω $T(n-1) \geq c'(n-1)^2$.

Έχουμε $T(n) = T(n-1) + n \geq c'(n-1)^2 + n = c'n^2 - 2c'n + c' + n$
$\geq c'n^2 + (1 - 2c')n \geq c'n^2,$ που ισχύει για $c' \leq \frac{1}{2}$.

Άρα έχουμε ότι ισχύει $T(n) = O(n^2)$ και $T(n) = \Omega(n^2)$ και, επομένως, $T(n) = \Theta(n^2)$.

Παράδειγμα 3: Έστω $T(n)$ ο χρόνος εκτέλεσης ενός αλγόριθμου στη χειρότερη περίπτωση, ο οποίος δίδεται αναδρομικά από τον παρακάτω τύπο:

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + n & n \geq 2 \\ 0 & n = 1 \end{cases}$$

Θέλουμε να υπολογίσουμε την ασυμπτωτική πολυπλοκότητα του αλγόριθμου. Με τη μέθοδο της αντικατάστασης λαμβάνουμε:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n \\ &= 4T\left(\frac{n}{4}\right) + 2n = 4\left[2T\left(\frac{n}{8}\right) + \frac{n}{4}\right] + 2n \\ &= 8T\left(\frac{n}{8}\right) + 3m = \cdots \\ &= 2^k T\left(\frac{n}{2^k}\right) + kn = \Theta(kn) = \Theta(n \log n). \end{aligned}$$

Εναλλακτικά έχουμε: $T(2^n) = 2T(2^{n-1}) + 2^n$.

Τότε, $T(2^n) = 2T(2^{n-1}) + 2^n \Rightarrow \frac{T(2^n)}{2^n} = \frac{T(2^{n-1})}{2^{n-1}} + 1$.

Άρα, $\frac{T(2^n)}{2^n} = \frac{T(2^{n-1})}{2^{n-1}} + 1 = \frac{T(2^{n-2})}{2^{n-2}} + 2 = \cdots = n$.

Οπότε ισχύει $T(2^n) = n2^n$ και, επομένως, η ασυμπτωτική πολυπλοκότητα του αλγόριθμου του παραδείγματος είναι $T(n) = \Theta(n\log n)$.

Ασκήσεις

2.1. Κατατάξτε τις ακόλουθες συναρτήσεις ως προς την τάξη αύξησής τους (by order of growth). Δηλαδή, προσδιορίστε μια διάταξη των συναρτήσεων $g_1, g_2, \ldots, g_{24}$, τέτοια ώστε: $g_1 = \Omega(g_2)$, $g_2 = \Omega(g_3)$, …, $g_{23} = \Omega(g_{24})$. Διαμερίστε τη λίστα των συναρτήσεων σε κλάσεις ισοδυναμίας έτσι ώστε $f(n)$ και $g(n)$ ανήκουν στην ίδια κλάση εάν-ν $f(n) = \Theta(g(n))$.

$\left(\frac{3}{2}\right)^n$	$\left(\sqrt{2}\right)^{\log n}$	$\log^* n$	n^2	n^3	$\log^2 n$
$n^{\frac{1}{\log n}}$	$\log\log n$	$n2^n$	$n^{\log\ \log\ n}$	$\log\ n$	2^n
$4^{\log n}$	$(n+1)!$	$\sqrt{(\log n)}$	$n!$	$2^{\sqrt{2}\ \log n}$	n
$\log\ (n!)$	$2^{\log n}$	$n\log n$	2^{2^n}	$(\log n)^{\log n}$	1

2.2. Δώστε ένα παράδειγμα δύο συναρτήσεων $f,\ g : \mathbb{N} \to \mathbb{R}^*$, τέτοιων ώστε $f \neq O(g)$ και $f \neq \Omega(g)$.
Σημείωση: Όλες οι συναρτήσεις δεν είναι ασυμπτωτικά συγκρίσιμες (asymptoticaly comperable). Για παράδειγμα, οι συναρτήσεις $f,\ g$: $\mathbb{N} \to \mathbb{R}^*$ με την παραπάνω ιδιότητα ονομάζονται ασυμπτωτικά μη-συγκρίσιμες.

2.3. Δείξτε ότι για οποιεσδήποτε πραγματικές σταθερές (for any real constants) a και b, όπου $b > 0$, ισχύει $(n+a)^b = \Theta(n^b)$.

2.4. Έστω $f,\ g : \mathbb{N} \to \mathbb{R}^*$. Δείξτε (με αντιπαράδειγμα) ότι δεν ισχύουν οι παρακάτω προτάσεις:

(a) $f(n) = O\big(g(n)\big) \Rightarrow g(n) = O(f(n))$.
(b) $f(n) + g(n) = \Theta(min(f(n), g(n))$.
(c) $f(n) = O\big(g(n)\big) \Rightarrow 2^{g(n)} = O(2^{f(n)})$.
(d) $f(n) = O(\big(f(n)\big)^2)$.
(e) $f(n) = O(f\left(\frac{n}{2}\right))$.

2.5. Επιλύστε τις παρακάτω αναδρομικές σχέσεις:

(a) $T(n) = 9\,T(\frac{n}{3}) + n$.
(b) $T(n) = T(\frac{2n}{3}) + 1$.
(c) $T(n) = 3\,T\left(\frac{n}{4}\right) + n\log n$.
(d) $T(n) = 2\,T(\frac{n}{2}) + n\log n$.

2.6. Ο χρόνος εκτέλεσης ενός αλγορίθμου A περιγράφεται από τον αναδρομικό τύπο $T(n) = 7T(\frac{n}{2}) + n^2$. Ένας άλλος ανταγωνιστικός αλγόριθμος A' έχει χρόνο εκτέλεσης $T'(n) =$

$\alpha\, T'(\frac{n}{4}) + n^2$. Προσδιορίστε τη μεγαλύτερη ακέραια τιμή του α, έτσι ώστε ο αλγόριθμος A' να είναι ασυμπτωτικά γρηγορότερος του αλγορίθμου A.

2.7. Εργαζόμαστε σε ένα εργοστάσιο παραγωγής βάζων, όπου μας ανατίθεται να προσδιορίσουμε το μεγαλύτερο ύψος h από το οποίο μπορούμε να πετάξουμε ένα συγκεκριμένο τύπο βάζου χωρίς να σπάσει. Μας αρκεί να βρούμε το h με ακρίβεια ενός μέτρου. Για το σκοπό αυτό μας δίνεται μία σκάλα που φτάνει τα N μέτρα, αλλά μόνο 2 βάζα. Πρέπει να βρούμε το h (υποθέτουμε ότι $h \leq N$) πραγματοποιώντας το μικρότερο δυνατό αριθμό $X(N)$ από ρίψεις (ως συνάρτηση του N). Προφανώς, αν κάνουμε μία ρίψη ανά ένα μέτρο τότε μπορεί να χρειαστούμε $X(N) = N$ ρίψεις, ενώ χρησιμοποιούμε μόνο το ένα βάζο. Από την άλλη, θα μπορούσαμε να κάνουμε $X(N) = O(\log N)$ ρίψεις με δυαδική αναζήτηση, αλλά έτσι μπορεί να χρειαστούμε περισσότερα από 2 βάζα.

2.8. Πως μπορούμε να λύσουμε την Άσκηση 2.7 όταν το N είναι άγνωστο;

2.9. Μας δίνεται μια αυθαίρετα μεγάλη ακολουθία στοιχείων $x_1, x_2, \ldots$, την οποία διαβάζουμε από ένα ρεύμα εισόδου. Θέλουμε να γνωρίζουμε, ανά πάσα στιγμή, ποιό είναι το στοιχείο της ακολουθίας που έχουμε δει μέχρι στιγμής, το οποίο έχει εμφανιστεί τις περισσότερες φορές. Σχεδιάστε ένα αλγόριθμο για το πρόβλημα αυτό, ο οποίος να χρησιμοποιεί μόνο $O(1)$ θέσεις μνήμης.

2.10. Μας δίνεται ένας πίνακας $A = [\, a_1 \quad a_2 \quad \ldots \quad a_n]$ με n πραγματικούς αριθμούς, καθώς και μια επιθυμητή τιμή K. Σχεδιάστε ένα αλγόριθμο ο οποίος να βρίσκει θέσεις i και j με $1 \leq i \leq j \leq n$, τέτοιες ώστε $a_i + a_{i+1} + \cdots + a_j = K$. Ποιά είναι η πολυπλοκότητα του αλγόριθμού σας;

Βιβλιογραφία

Cormen, T., Leiserson, C., Rivest, R., & Stain, C. (2001). *Introduction to Algorithms.* MIT Press (2nd edition) .

Dasgupta, S., Papadimitriou, C., & Vazirani, U. (2008). *Algorithms.* McGraw-Hill.

Garey, M., & Johnson, D. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness.* New York: W.H. Freeman.

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Κεφάλαιο 3
Στοιχειώδεις Δομές Δεδομένων

Περιεχόμενα

3.1 Στοιχειώδεις τύποι δεδομένων 39
3.2 Πίνακες 40
3.2.1 Διδιάστατοι πίνακες 43
3.3 Συνδεδεμένες Λίστες 48
3.4 Αναδρομή 51
3.4.1 Μέθοδος «Διαίρει και Βασίλευε» 53
Ασκήσεις 57
Βιβλιογραφία 58

Στο κεφάλαιο αυτό μελετούμε στοιχειώδεις δομές δεδομένων, όπως οι πίνακες και οι συνδεδεμένες λίστες. Οι δομές αυτές υποστηρίζουν μόνο ένα περιορισμένο σύνολο λειτουργιών και στις περισσότερες περιπτώσεις απαιτούν πολύ χρόνο για την εκτέλεσή τους, ωστόσο είναι απλές στην υλοποίησή τους και γι' αυτό πολύ χρήσιμες σε πολλές εφαρμογές.

3.1 Στοιχειώδεις τύποι δεδομένων

Για να διευκολύνει τους προγραμματιστές, ώστε να μπορούν να δηλώνουν τις κατάλληλες μεταβλητές που χρειάζονται στα προγράμματά τους, κάθε γλώσσα προγραμματισμού παρέχει ένα σύνολο στοιχειωδών τύπων δεδομένων, όπως, για παράδειγμα, είναι οι τύποι του ακέραιου αριθμού, του πραγματικού κ.λπ. Τέτοιοι συνήθεις στοιχειώδεις τύποι δεδομένων είναι οι τύποι του ακέραιου αριθμού, του πραγματικού αριθμού, του χαρακτήρα, του δίτιμου ή δυαδικού αριθμού (boolean), της αλφαριθμητικής συμβολοσειράς (string), της απαρίθμησης (enumeration) κ.λπ. Επιπλέον, συχνά δίδεται η δυνατότητα κατασκευής σύνθετων τύπων από συνδυασμό στοιχειωδών τύπων ή και σύνθετων τύπων που έχουν ορισθεί νωρίτερα.

Για παράδειγμα, στη γλώσσα προγραμματισμού Java υποστηρίζονται οι τύποι του ακέραιου αριθμού (`int`, `long`), του αριθμού κινητής υποδιαστολής (`float`), του αριθμού κινητής υποδιαστολής διπλής ακρίβειας (`double`), του χαρακτήρα (`char`) και του δυαδικού αριθμού (`boolean`), ενώ μπορούν να κατασκευασθούν σύνθετοι τύποι, όπως:

```
public class Point
{
        private float x;
        private float y;
}
```

Ο τύπος αυτός περιγράφει ένα διδιάστατο σημείο το οποίο ορίζεται από την x- και την y-συντεταγμένη του. Οι συντεταγμένες αποθηκεύονται στα πεδία αριθμού κινητής υποδιαστολής με ονόματα x και y, αντίστοιχα.

3.2 Πίνακες

Ένας πίνακας (array) είναι ένας τύπος δεδομένων που αποθηκεύει δεδομένα (στοιχεία)

- του ιδίου τύπου (π.χ. πίνακας ακεραίων, πίνακας δεικτών σε ακέραιους κ.λπ.),
- σε συνεχόμενες θέσεις στη μνήμη (δηλαδή στη μνήμη, το i-οστο στοιχείο κάθε πίνακα βρίσκεται αμέσως μετά το $(i-1)$-οστο στοιχείο του και αμέσως πριν από το $(i+1)$-οστο στοιχείο του) και
- στα οποία γίνεται αναφορά με χρήση ακέραιου δείκτη (δηλαδή, για έναν πίνακα A χρησιμοποιούμε τον συμβολισμό $A[2]$ για το 3ο στοιχείο του A εάν θεωρήσουμε ότι το 1ο στοιχείο του πίνακα είναι το $A[0]$ (όπως συμβαίνει στις γλώσσες προγραμματισμού C και Java)).

Λόγω του ότι τα στοιχεία ενός πίνακα A (με στοιχεία $A[0], A[1], \cdots$) αποθηκεύονται σε συνεχόμενες θέσεις στη μνήμη μπορούμε να συμπεράνουμε ότι το i-οστο στοιχείο $A[i-1]$ βρίσκεται στη θέση $X + (i-1) * L$, όπου X η θέση του πρώτου στοιχείου $A[0]$ και L το μέγεθος του κάθε στοιχείου του πίνακα A.

Στη γλώσσα προγραμματισμού Java, ένας πίνακας (π.χ., ακέραιων αριθμών) μπορεί να δηλωθεί στατικά ως εξής:

```
int[] A = { 1, 1, 2, 0, 5, 8 };
```

Επίσης, μπορούμε να έχουμε δυναμική δήλωση πίνακα, όπως φαίνεται παρακάτω

```
int[] A;
```

σε συνδυασμό με δέσμευση μνήμης με την εντολή

```
A = new int [N];
```

Σε αυτήν την περίπτωση, η δέσμευση μνήμης γίνεται κατά το χρόνο εκτέλεσης, όταν η τιμή του N είναι γνωστή. Πιο σύντομα, μπορούμε να γράψουμε:

```
int[] A = new int [N];
```

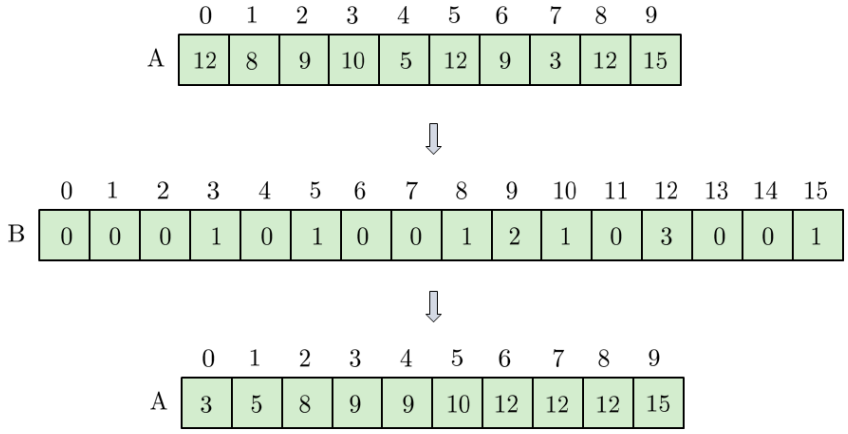
Παράδειγμα 3.2.1 (Ταξινόμηση με Κάδους)**.** Έστω ότι μας δίδεται ένας πίνακας A που αποθηκεύει N ακέραιους αριθμούς στο διάστημα $[0..M-1]$ τους οποίους θέλουμε να ταξινομήσουμε από το μικρότερο προς το μεγαλύτερο.

Μπορούμε να κάνουμε εύκολα την ταξινόμηση χρησιμοποιώντας έναν βοηθητικό πίνακα B μεγέθους M ως εξής:

1. Αρχικά θέτουμε $B[i] = 0$ για $i = 0,1,\ldots,M-1$.
2. Για $j = 0,1,\ldots,N-1$
   αυξάνουμε κατά 1 την τιμή του $B[A[j]]$

3. Θέτουμε $j = 0$ (δείκτης για την εξ αρχής διάσχιση του πίνακα A)
4. Για $i = 0,1, \dots, M-1$
   Για $k = 1, .., B[i]$
   θέτουμε $A[j] = i$
   αυξάνουμε κατά 1 την τιμή του j

Για παράδειγμα, ας θεωρήσουμε τον πίνακα A στο άνω μέρος της Εικόνας 3.1. Μετά τα Βήματα 1 και 2 του Αλγορίθμου, ο πίνακας B είναι όπως φαίνεται στην Εικόνα 3.1, ενώ μετά και τα Βήματα 3 και 4, ο πίνακας A είναι ταξινομημένος όπως φαίνεται στο κάτω μέρος της Εικόνας 3.1.



Εικόνα 3.1: Εφαρμογή του αλγορίθμου ταξινόμησης με χρήση βοηθητικού πίνακα B.

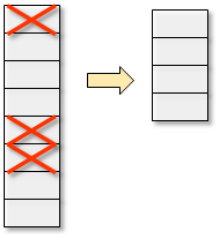
Από την περιγραφή του αλγορίθμου συμπεραίνουμε ότι αυτός εκτελείται συνολικά σε $O(M + N)$ χρόνο. Συνεπώς, ο αλγόριθμος είναι καλός εάν το M δεν είναι πολύ μεγαλύτερο από το N.

Παράδειγμα 3.2.2 (Δυναμικοί πίνακες με εισαγωγές και διαγραφές)**.** Σε έναν πίνακα, μας ενδιαφέρει να μπορούμε να εκτελούμε αποτελεσματικά εισαγωγές και διαγραφές στοιχείων. Επιθυμούμε να επιτύχουμε τις ακόλουθες ιδιότητες:

α) περιορισμένη σπατάλη χώρου και

β) μικρό συνολικό κόστος για οποιαδήποτε ακολουθία εισαγωγών και διαγραφών.

Για έναν πίνακα A που αποθηκεύει n στοιχεία, ορίζουμε το *συντελεστή πληρότητας* του πίνακα A ως το λόγο $\alpha = \frac{n}{|A|}$, όπου με $|A|$ συμβολίζουμε το μέγεθος του A. Τότε μπορούμε να εξασφαλίσουμε την ιδιότητα (α) απαιτώντας ο συντελεστής πληρότητας α να είναι τουλάχιστον ίσος με μια σταθερά. Για να το επιτύχουμε αυτό, θα πρέπει να φροντίζουμε να αντιγράφουμε τα στοιχεία σε μικρότερο πίνακα (Εικόνα 3.2) σε περίπτωση που εκτελεστούν αρκετές διαγραφές στοιχείων (εάν δεν περιοριζόμασταν σε μικρότερο πίνακα, ο συντελεστής πληρότητας του πίνακα θα μειωνόταν σε τιμή μικρότερη από την επιθυμητή σταθερά).

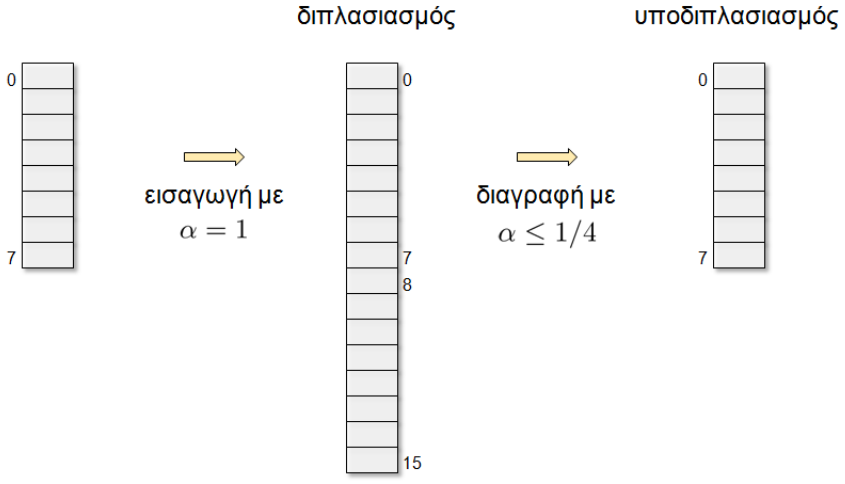


Εικόνα 3.2: Μετά την εκτέλεση αρκετών διαγραφών, αντιγράφουμε τα στοιχεία του πίνακα σε μικρότερο πίνακα.

Μια προσέγγιση που εξασφαλίζει την ισχύ και των δύο παραπάνω ιδιοτήτων είναι η εξής:

Εισαγωγή: Εάν ζητηθεί εισαγωγή στοιχείου και ο πίνακας δεν είναι πλήρης, τότε το νέο στοιχείο εισάγεται στην επόμενη διαθέσιμη θέση του πίνακα. Όμως, εάν ο πίνακας είναι ήδη πλήρης (δηλαδή, ο συντελεστής πληρότητάς του είναι ίσος με 1), τότε διπλασιάζουμε το μέγεθος του πίνακα αντιγράφοντας τα στοιχεία που περιέχει και εισάγοντας το νέο στοιχείο. Ο συντελεστής πληρότητας μετά την εισαγωγή υποδιπλασιάζεται και η τιμή του γίνεται λίγο μεγαλύτερη από $1/2$.

Διαγραφή: Εάν ζητηθεί διαγραφή στοιχείου και μετά τη διαγραφή ο συντελεστής πληρότητας έχει τιμή μεγαλύτερη από $1/4$, τότε δεν γίνεται κάτι άλλο πέραν της διαγραφής. Εάν, όμως, μετά τη διαγραφή, ο συντελεστής πληρότητας γίνει μικρότερος ή ίσος με $1/4$, τότε το μέγεθος του πίνακα υποδιπλασιάζεται. Ο συντελεστής πληρότητας μετά τη διαγραφή διπλασιάζεται και η τιμή του γίνεται ίση με ή λίγο μικρότερη από $1/2$.



Εικόνα 3.3: Διπλασιασμός του μεγέθους πίνακα για $\alpha = 1$ και υποδιπλασιασμός για $\alpha \leq 1/4$.

Η μέθοδος περιγράφεται σχηματικά στην Εικόνα 3.3. Επιτυγχάνει $\alpha \geq 1/4$ και συνολικά πολυπλοκότητα χρόνου $\Theta(n)$ για n εισαγωγές και διαγραφές. Έτσι, ο αντισταθμιστικός χρόνος ανά πράξη είναι $\Theta(1)$.

3.2.1 Διδιάστατοι πίνακες

Σε πολλές επιστήμες, είναι χρήσιμοι διδιάστατοι πίνακες (π.χ., πίνακες (μήτρες) στην γραμμική άλγεβρα). Ένας διδιάστατος πίνακας απεικονίζεται όπως φαίνεται στην Εικόνα 3.4.

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ & & \vdots & \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

Εικόνα 3.4: Ένας διδιάστατος πίνακας με m γραμμές και n στήλες.

Στη γλώσσα προγραμματισμού Java, ένας διδιάστατος πίνακας αριθμών κινητής υποδιαστολής διπλής ακρίβειας μπορεί να δηλωθεί ως εξής:

```
double[][] A = new double[m][n];
```

Γενικά, η δήλωση ενός πίνακα d διαστάσεων είναι:

```
double[][]…[] A = new double[k1][k2]…[kd];
```

όπου $k1, k2, \dots, kd$ είναι θετικοί ακέραιοι αριθμοί.

Παράδειγμα 3.2.3 (Πολλαπλασιασμός Πινάκων)**.** Έστω ότι μας δίδεται ένας διδιάστατος πίνακας A με p γραμμές και q στήλες και ένας διδιάστατος πίνακας B με q γραμμές και r στήλες (βλέπε Εικόνα 3.5) και θέλουμε να υπολογίσουμε το γινόμενό τους.

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1q} \\ a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2q} \\ & & & \vdots & & \\ a_{k1} & a_{k2} & \cdots & a_{kj} & \cdots & a_{kq} \\ & & & \vdots & & \\ a_{p1} & a_{p2} & \cdots & a_{pj} & \cdots & a_{pq} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1j} & \cdots & b_{1r} \\ b_{21} & b_{22} & \cdots & b_{2j} & \cdots & b_{2r} \\ & & & \vdots & & \\ b_{k1} & b_{k2} & \cdots & b_{kj} & \cdots & b_{kr} \\ & & & \vdots & & \\ b_{q1} & b_{q2} & \cdots & b_{qj} & \cdots & b_{qr} \end{pmatrix}$$

Εικόνα 3.5: Δύο διδιάστατοι πίνακες.

Το γινόμενο $A * B$ είναι ένας διδιάστατος πίνακας με p γραμμές και r στήλες (βλέπε Εικόνα 3.6) το (k, j)-στοιχείο του οποίου έχει τιμή $c_{kj} = \sum_{i=1}^{q} a_{ki}\ b_{ij}$, δηλαδή, για να υπολογίσουμε την τιμή του (k, j)-στοιχείου του πίνακα C πολλαπλασιάζουμε κάθε στοιχείο της k-οστής γραμμής του πίνακα A με το αντίστοιχο στοιχείο της j-οστής στήλης του πίνακα B και προσθέτουμε τις τιμές αυτών των γινομένων.

$$C = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1j} & \cdots & c_{1r} \\ c_{21} & c_{22} & \cdots & c_{2j} & \cdots & c_{2r} \\ & & & \vdots & & \\ c_{k1} & c_{k2} & \cdots & c_{kj} & \cdots & c_{kr} \\ & & & \vdots & & \\ c_{p1} & c_{p2} & \cdots & c_{pj} & \cdots & c_{pr} \end{pmatrix}$$

Εικόνα 3.6: Ο πίνακας γινόμενο $A * B$.

Ο υπολογισμός του γινομένου μπορεί να γίνει απλώς ως εξής:

```
for (i = 0; i < p; i++)
        for (j = 0; j < r; j++)
        {
                C[i][j]=0;
                for (k = 0; k < q; k++)
                {
                        C[i][j] += A[i][k]*B[k][j];
                }
        }
```

Ο παραπάνω κώδικας υπολογίζει το γινόμενο, αφού εκτελέσει $\Theta(pqr)$ πράξεις.

Αποθήκευση διδιάστατου πίνακα ως μονοδιάστατου

Ένας διδιάστατος πίνακας μπορεί να αποθηκευθεί ως μονοδιάστατος με συστηματική αποθήκευση των στοιχείων του. Δύο είναι οι κύριοι τρόποι αποθήκευσης.

$$A = \begin{pmatrix} A[0,0] & A[0,1] & \cdots & A[0,j-1] & \cdots & A[0,q-1] \\ A[1,0] & A[1,1] & \cdots & A[1,j-1] & \cdots & A[1,q-1] \\ & & & \vdots & & \\ A[k-1,0] & A[k-1,1] & \cdots & A[k-1,j-1] & \cdots & A[k-1,q-1] \\ & & & \vdots & & \\ A[p-1,0] & A[p-1,1] & \cdots & A[p-1,j-1] & \cdots & A[p-1,q-1] \end{pmatrix}$$

Εικόνα 3.7: Ένας διδιάστατος πίνακας A.

1. *Αποθήκευση κατά γραμμές*: Σε αυτήν την περίπτωση, αποθηκεύουμε πρώτα τα στοιχεία της πρώτης γραμμής του πίνακα, κατόπιν της δεύτερης γραμμής του, κ.ο.κ. έως και την τελευταία γραμμή. Σχηματικά, ο μονοδιάστατος πίνακας που προκύπτει με αυτόν τον τρόπο αποθήκευσης των στοιχείων του πίνακα A της Εικόνας 3.7 δίδεται στην Εικόνα 3.8.

$$[\ A[0,0] \quad A[0,1] \quad \ldots \quad A[0,q-1] \quad A[1,0] \quad A[1,1] \quad \ldots \quad A[1,q-1] \quad \ldots$$
$$A[k-1,0] \quad A[k-1,1] \quad \ldots \quad A[k-1,q-1] \quad \ldots \quad A[p-1,0] \quad A[p-1,1] \quad \ldots \quad A[p-1,q-1]\ ]$$

Εικόνα 3.8: Αποθήκευση κατά γραμμές των στοιχείων του πίνακα A.

Τότε, το στοιχείο $A[i,j]$ θα αποθηκευθεί στη θέση $X + (i * q + j) * L$ του νέου μονοδιάστατου πίνακα, όπου X η θέση του πρώτου στοιχείου $A[0,0]$ και L το μέγεθος του κάθε στοιχείου του πίνακα A.

2. *Αποθήκευση κατά στήλες*: Σε αυτήν την περίπτωση, αποθηκεύουμε πρώτα τα στοιχεία της πρώτης στήλης του πίνακα, κατόπιν της δεύτερης στήλης του, κ.ο.κ. έως και την τελευταία στήλη. Σχηματικά, ο μονοδιάστατος πίνακας που προκύπτει με αυτόν τον τρόπο αποθήκευσης των στοιχείων του πίνακα A της Εικόνας 3.7 δίδεται στην Εικόνα 3.9.

$$[\ A[0,0] \quad A[1,0] \quad \dots \quad A[p-1,0] \quad A[0,1] \quad A[1,1] \quad \dots \quad A[p-1,1] \quad \dots$$
$$A[0,j-1] \quad A[1,j-1] \quad \dots \quad A[p-1,j-1] \quad \dots \quad A[0,q-1] \quad A[1,q-1] \quad \dots \quad A[p-1,q-1]\ ]$$

Εικόνα 3.9: Αποθήκευση κατά στήλες των στοιχείων του πίνακα A.

Τότε, το στοιχείο $A[i,j]$ θα αποθηκευθεί στη θέση $X + (i + j * p) * L$ του νέου μονοδιάστατου πίνακα, όπου X η θέση του πρώτου στοιχείου $A[0,0]$ και L το μέγεθος του κάθε στοιχείου του πίνακα A.

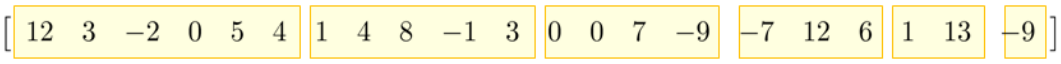
Σε πολλές εφαρμογές προκύπτουν πίνακες με ειδική μορφή, όπως αυτοί που θα δούμε στη συνέχεια. Μπορούμε να αποθηκεύσουμε τα στοιχεία τέτοιων πινάκων σε μονοδιάστατους πίνακες για εξοικονόμηση χώρου.

Συμμετρικοί πίνακες. Ένας *τετραγωνικός* πίνακας A είναι *συμμετρικός*, εάν $A[i,j] = A[j,i]$ για κάθε i, j. Ένας συμμετρικός πίνακας φαίνεται στην Εικόνα 3.10.

$$A = \begin{pmatrix} 12 & 3 & -2 & 0 & 5 & 4 \\ 3 & 1 & 4 & 8 & -1 & 3 \\ -2 & 4 & 0 & 0 & 7 & -9 \\ 0 & 8 & 0 & -7 & 12 & 6 \\ 5 & -1 & 7 & 12 & 1 & 13 \\ 4 & 3 & -9 & 6 & 13 & -9 \end{pmatrix}$$

Εικόνα 3.10: Ένας συμμετρικός πίνακας.

Μπορούμε να αποθηκεύσουμε τα στοιχεία ενός συμμετρικού πίνακα σε έναν μονοδιάστατο αποφεύγοντας να αποθηκεύσουμε τα συμμετρικά στοιχεία. Και σε αυτήν την περίπτωση μπορούμε να αποθηκεύσουμε τα στοιχεία κατά γραμμές ή κατά στήλες. Στην πρώτη περίπτωση, αρκεί να αποθηκεύσουμε για κάθε γραμμή (από την πρώτη έως την τελευταία) τα στοιχεία της από το στοιχείο της κύριας διαγωνίου και προς τα δεξιά (Εικόνα 3.11). Παρόμοια, στη δεύτερη περίπτωση, αποθηκεύουμε για κάθε στήλη (από την πρώτη έως την τελευταία) τα στοιχεία της από το στοιχείο της κύριας διαγωνίου και προς τα κάτω.



Εικόνα 3.11: Αποθήκευση των μη συμμετρικών στοιχείων του συμμετρικού πίνακα της Εικόνας 3.10 κατά γραμμές σε έναν μονοδιάστατο πίνακα.

Το πλήθος των στοιχείων που τελικά αποθηκεύονται είναι

$$n + (n-1) + (n-2) + \cdots + 2 + 1 = n(n-1)/2,$$

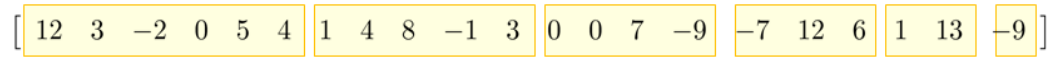
(από τα n^2 στοιχεία του πίνακα), όπου n η διάσταση του συμμετρικού πίνακα.

Άνω τριγωνικοί πίνακες. Ένας τετραγωνικός πίνακας είναι *άνω τριγωνικός*, εάν $A[i,j] = 0$ για $i > j$, δηλαδή τα στοιχεία κάτω από την κύρια διαγώνιο έχουν τιμή ίση με 0. Ένας άνω τριγωνικός πίνακας φαίνεται στην Εικόνα 3.12.

$$A = \begin{pmatrix} 12 & 3 & -2 & 0 & 5 & 4 \\ 0 & 1 & 4 & 8 & -1 & 3 \\ 0 & 0 & 0 & 0 & 7 & -9 \\ 0 & 0 & 0 & -7 & 12 & 6 \\ 0 & 0 & 0 & 0 & 1 & 13 \\ 0 & 0 & 0 & 0 & 0 & -9 \end{pmatrix}$$

Εικόνα 3.12: Ένας άνω τριγωνικός πίνακας.

Και σε αυτήν την περίπτωση μπορούμε να αποθηκεύσουμε τα στοιχεία που βρίσκονται στην κύρια διαγώνιο ή επάνω από αυτήν, σε έναν μονοδιάστατο πίνακα ακριβώς, όπως περιγράφηκε και για την περίπτωση των συμμετρικών πινάκων. Στην Εικόνα 3.13 δίδονται τα περιεχόμενα ενός μονοδιάστατου πίνακα στον οποίον έχουμε αποθηκεύσει κατά γραμμές τα στοιχεία του πίνακα A της Εικόνας 3.12 που βρίσκονται στην κύρια διαγώνιο ή επάνω από αυτήν.



Εικόνα 3.13: Αποθήκευση κατά γραμμές σε έναν μονοδιάστατο πίνακα των στοιχείων του άνω τριγωνικού πίνακα της Εικόνας 3.12 που βρίσκονται στην κύρια διαγώνιο ή επάνω από αυτήν.

Κατ' αναλογία πρός την περίπτωση των συμμετρικών πινάκων, το πλήθος των στοιχείων που αποθηκεύονται είναι $n(n-1)/2$ από τα n^2 στοιχεία του άνω τριγωνικού πίνακα.

Κάτω τριγωνικοί πίνακες. Ένας τετραγωνικός πίνακας είναι *κάτω τριγωνικός*, εάν $A[i,j] = 0$ για $i < j$, δηλαδή, τα στοιχεία πάνω από την κύρια διαγώνιο έχουν τιμή ίση με 0. Στην Εικόνα 3.14 φαίνεται ένας κάτω τριγωνικός πίνακας.

$$A = \begin{pmatrix} 12 & 0 & 0 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 & 0 & 0 \\ 0 & 4 & 5 & 0 & 0 & 0 \\ 1 & 2 & 8 & -7 & 0 & 0 \\ 10 & -6 & 12 & -9 & 1 & 0 \\ -3 & 5 & -4 & 1 & 13 & -2 \end{pmatrix}$$

Εικόνα 3.14: Ένας κάτω τριγωνικός πίνακας.

Αντίστοιχα, μπορούμε να αποθηκεύσουμε τα στοιχεία ενός κάτω τριγωνικού πίνακα που βρίσκονται στην κύρια διαγώνιο ή κάτω από αυτήν σε έναν μονοδιάστατο πίνακα κατά γραμμές ή στήλες. Στην Εικόνα 3.15 φαίνεται η αποθήκευση κατά γραμμές σε έναν μονοδιάστατο πίνακα των στοιχείων του κάτω τριγωνικού πίνακα της Εικόνας 3.14 που βρίσκονται στην κύρια διαγώνιο ή κάτω από αυτήν.

$$[\ 12 \quad 3 \quad 1 \quad 0 \quad 4 \quad 5 \quad 1 \quad 2 \quad 8 \quad -7 \quad 10 \quad -6 \quad 12 \quad -9 \quad 1 \quad -3 \quad 5 \quad -4 \quad 1 \quad 13 \quad -2\ ]$$

Εικόνα 3.15: Αποθήκευση κατά γραμμές σε έναν μονοδιάστατο πίνακα των στοιχείων του κάτω τριγωνικού πίνακα της Εικόνας 3.14 που βρίσκονται στην κύρια διαγώνιο ή κάτω από αυτήν.

Αραιοί πίνακες. Ένας πίνακας λέγεται *αραιός*, όταν το πλήθος των μη μηδενικών στοιχείων είναι πολύ μικρό σχετικά με το μέγεθός του. Στην Εικόνα 3.16 φαίνεται ένας αραιός πίνακας ο οποίος έχει μόνο 16 μη μηδενικά στοιχεία σε σύνολο 36 στοιχείων.

$$A = \begin{pmatrix} 12 & 0 & 0 & 4 & 0 & 0 \\ 3 & 0 & 6 & 0 & 0 & 0 \\ 0 & 0 & 5 & -11 & 0 & 2 \\ 0 & 0 & 8 & -7 & 12 & 0 \\ -6 & 0 & 0 & -9 & 0 & 3 \\ 0 & -3 & 0 & 0 & 1 & 12 \end{pmatrix}$$

Εικόνα 3.16: Ένας αραιός πίνακας.

Για να εξοικονομήσουμε χώρο, μπορούμε να αποθηκεύσουμε μόνο τα μη μηδενικά στοιχεία στοιχεία του αραιού πίνακα διαδοχικά κατά γραμμές σε έναν μονοδιάστατο πίνακα V (Εικόνα 3.17).

$$V = [\ 12 \quad 4 \quad 3 \quad 6 \quad 5 \quad -11 \quad 2 \quad 8 \quad -7 \quad 12 \quad -6 \quad -9 \quad 3 \quad -3 \quad 1 \quad 12\ ]$$

Εικόνα 3.17: Αποθήκευση κατά γραμμές των μη μηδενικών στοιχείων του αραιού πίνακα της Εικόνας 3.16.

Καθώς τα μη μηδενικά στοιχεία ενός αραιού πίνακα ενδέχεται να βρίσκονται σε τυχαίες θέσεις σε αυτόν, χρειάζεται να αποθηκεύσουμε, επίσης, τη στήλη και τη γραμμή που αντιστοιχούν σε

κάθε μη μηδενικό στοιχείο. Για αυτό χρησιμοποιούμε τους μονοδιάστατους πίνακες col και row (Εικόνα 3.18).

$$col = [\ 1\ \ 4\ \ 1\ \ 3\ \ 3\ \ 4\ \ 6\ \ 3\ \ 4\ \ 5\ \ 1\ \ 4\ \ 6\ \ 2\ \ 5\ \ 6\ ]$$
$$row = [\ 1\ \ 1\ \ 2\ \ 2\ \ 3\ \ 3\ \ 3\ \ 4\ \ 4\ \ 4\ \ 5\ \ 5\ \ 5\ \ 6\ \ 6\ \ 6\ ]$$

Εικόνα 3.18: Οι μονοδιάστατοι πίνακες col και row για τον αραιό πίνακα της Εικόνας 3.16.

Για παράδειγμα, το στοιχείο -11 (το 6ο στοιχείο του πίνακα V) βρίσκεται στην 4η στήλη (το 6ο στοιχείο του πίνακα col) και στην 3η γραμμή (το 6ο στοιχείο του πίνακα row) του αραιού πίνακα. Για έναν αραιό πίνακα με N μη μηδενικά στοιχεία, αυτή η αναπαράσταση απαιτεί χώρο $3N$ (ανεξάρτητα από το μέγεθος του πίνακα).

Εναλλακτικά, αντί για τον πίνακα γραμμών row, μπορούμε να χρησιμοποιήσουμε έναν άλλο μονοδιάστατο πίνακα $rowpos$, το i-οστό στοιχείο του οποίου δίνει τη θέση στον πίνακα V στην οποία βρίσκεται το αριστερότερο μη μηδενικό στοιχείο της i-οστής γραμμής του αραιού πίνακα. Σε αυτήν την περίπτωση τα περιεχόμενα των πινάκων col και $rowpos$ για τον αραιό πίνακα της Εικόνας 3.16 και τον πίνακα V της Εικόνας 3.17 είναι αυτά που φαίνονται στην Εικόνα 3.19 (παρατηρήστε ότι τα αριστερότερα μη μηδενικά κάθε γραμμής του αραιού πίνακα βρίσκονται στις θέσεις 1, 3, 5, 8, 11 και 14).

$$col = [\ 1\ \ 4\ \ 1\ \ 3\ \ 3\ \ 4\ \ 6\ \ 3\ \ 4\ \ 5\ \ 1\ \ 4\ \ 6\ \ 2\ \ 5\ \ 6\ ]$$
$$rowpos = [\ 1\ \ 3\ \ 5\ \ 8\ \ 11\ \ 14\ ]$$

Εικόνα 3.19: Οι μονοδιάστατοι πίνακες col και $rowpos$ για τον αραιό πίνακα της Εικόνας 3.16.

Για παράδειγμα, το στοιχείο -11 (το 6ο στοιχείο του πίνακα V) βρίσκεται στην 4η στήλη (το 6ο στοιχείο του πίνακα col) και στην 3η γραμμή του αραιού πίνακα (ο πίνακας $rowpos$ μάς πληροφορεί ότι τα στοιχεία στις θέσεις 5, 6 και 7 του πίνακα V βρίσκονται στην 3η γραμμή του αραιού πίνακα). Αυτή η αναπαράσταση απαιτεί χώρο $2N + m$ όπου N το πλήθος μη μηδενικών στοιχείων και m το πλήθος γραμμών του αραιού πίνακα.

3.3 Συνδεδεμένες Λίστες

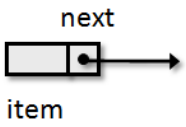
Η καταχώριση των στοιχείων των πινάκων σε συνεχόμενες θέσεις στη μνήμη έχει το σημαντικό πλεονέκτημα της εύκολης προσπέλασής τους (μέσω ακέραιου δείκτη). Την ίδια στιγμή, όμως, συνεπάγεται ότι η εισαγωγή στοιχείου σε συγκεκριμένη θέση και η διαγραφή στοιχείου ενδέχεται να απαιτήσει χρόνο γραμμικό στο πλήθος στοιχείων του πίνακα. Για παράδειγμα, εάν θέλουμε να εισαγάγουμε ένα στοιχείο στην πρώτη θέση του πίνακα διατηρώντας τη σειρά των υπόλοιπων στοιχείων (π.χ., όταν εισάγουμε ένα στοιχείο μικρότερο από τα στοιχεία ενός ταξινομημένου πίνακα), τότε θα πρέπει να αντιγράψουμε κάθε στοιχείο του πίνακα μία θέση δεξιότερα και, τέλος, να γράψουμε το νέο στοιχείο στην πρώτη θέση του πίνακα. Παρόμοια, εάν θέλουμε να διαγράψουμε το πρώτο στοιχείο ενός πίνακα χωρίς να αλλάξουμε τη σειρά των

υπόλοιπων στοιχείων (π.χ., όταν διαγράφουμε το πρώτο στοιχείο ενός ταξινομημένου πίνακα), θα πρέπει να αντιγράψουμε κάθε στοιχείο του πίνακα μία θέση αριστερότερα.

Οι συνδεδεμένες λίστες παρέχουν τη δυνατότητα γρηγορότερης εισαγωγής και διαγραφής σε πολλές περιπτώσεις. Αποθηκεύουν ένα σύνολο στοιχείων σε *κόμβους* που συνδέονται σε μια σειρά με συνδέσμους από κάθε κόμβο στον επόμενό του (έτσι, γενικά, τα περιεχόμενα των κόμβων δεν αποθηκεύονται σε συνεχόμενες θέσεις στη μνήμη). Μια τυπική δήλωση ενός κόμβου τύπου Node στην Java έχει ως εξής:

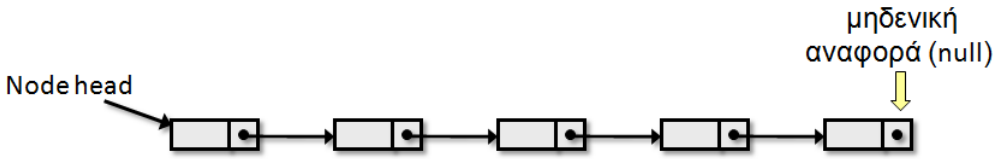
```
private class Node
{
        Item item;      // δεδομένα στον κόμβο
        Node next;      // σύνδεσμος προς επόμενο κόμβο
}
```

Σχηματικά, ένας κόμβος αποδίδεται όπως φαίνεται στην Εικόνα 3.20.



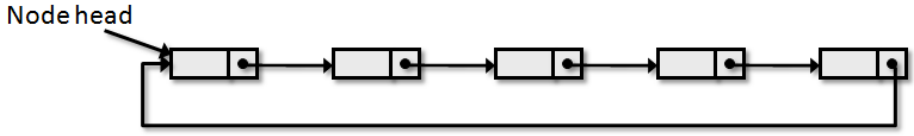
Εικόνα 3.20: Ένας κόμβος.

Τέτοιοι κόμβοι συνδέονται ο ένας μετά τον άλλον, για να σχηματίσουν μια *απλά συνδεδεμένη λίστα*. Ο τελευταίος κόμβος δεν δείχνει σε κάποιον κόμβο· για να το δηλώσουμε αυτό, λέμε ότι το πεδίο δείκτη `next` έχει τιμή `null`. Για να προσπελάσουμε τα στοιχεία της λίστας, χρειαζόμαστε μια αναφορά στον πρώτο κόμβο της λίστας. Η αναφορά αποθηκεύεται σε μια μεταβλητή, π.χ. `head` (τύπου `Node`). Βλέπε Εικόνα 3.21.



Εικόνα 3.21: Μια απλά συνδεδεμένη λίστα με 5 κόμβους.

Σε ορισμένες περιπτώσεις που θέλουμε να επεξεργαστούμε τα στοιχεία της λίστας πολλαπλές φορές είναι βολικό να κάνουμε τη λίστα *κυκλική*, δηλαδή, θέτοντας τον πρώτο κόμβο της λίστας ως επόμενο του τελευταίου κόμβου της (Εικόνα 3.22).



Εικόνα 3.22: Μια κυκλική απλά συνδεδεμένη λίστα με 5 κόμβους.

Για να δημιουργήσουμε έναν νέο κόμβο, στον οποίον θα αναφερόμαστε με τη μεταβλητή `x`, χρησιμοποιούμε την εντολή:

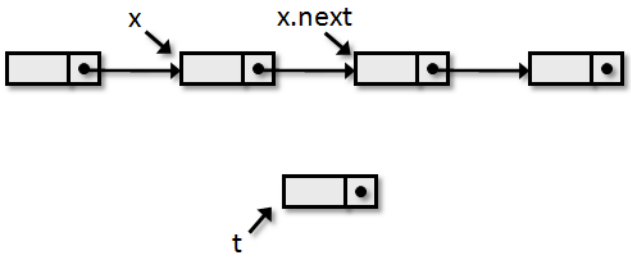
```
Node x = new Node();
```

Το αποτέλεσμα της εντολής αυτής φαίνεται σχηματικά στην Εικόνα 3.23. Για να αναφερθούμε στο πεδίο `item` του κόμβου με αναφορά `x`, χρησιμοποιούμε το `x.item`. Αναάλογα με `x.next` αναφερόμαστε στο πεδίο `next` του.



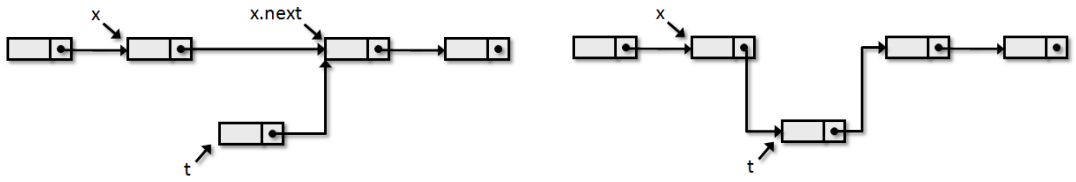
Εικόνα 3.23: Ένας νέος κόμβος με αναφορά `x`.

Ας θεωρήσουμε την εισαγωγή κόμβου με αναφορά `t` σε μια απλά συνδεδεμένη λίστα με κόμβους τύπου `Node` αμέσως μετά τον κόμβο με αναφορά `x` (Εικόνα 3.24).



Εικόνα 3.24: Εισαγωγή κόμβου με αναφορά `x` αμέσως μετά τον κόμβο με αναφορά `x`.

Πρώτα, συνδέουμε το νέο κόμβο με τον κόμβο που θα είναι επόμενός του στη λίστα με την εντολή `t.next = x.next;` (Εικόνα 3.25(α)) ενημερώνοντας το πεδίο `next` του `t` (η εντολή αυτή έχει το σωστό αποτέλεσμα, ακόμη κι αν το πεδίο δείκτη `next` του `t` έχει τιμή `null`). Κατόπιν, συνδέουμε το νέο κόμβο μετά τον κόμβο με αναφορά `x` με την εντολή `x.next = t;` (Εικόνα 3.25(β)) ενημερώνοντας το πεδίο `next` του `x`. Και η εισαγωγή ολοκληρώθηκε. Είναι σημαντικό να παρατηρήσει κανείς ότι, εάν εκτελέσουμε πρώτα την εντολή `x.next = t;`, τότε ναι μεν έχουμε συνδέσει τον `t` μετά τον `x`, αλλά δεν έχουμε πλέον πρόσβαση στον κόμβο που ήταν μετά τον `x` στη λίστα. Γι' αυτό, η εκτέλεση των δύο παραπάνω εντολών πρέπει να γίνει με τη σειρά που δόθηκε.

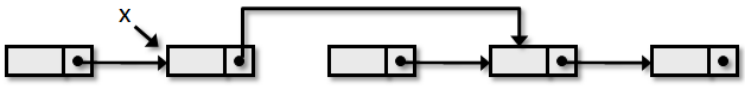


(α) (β)

Εικόνα 3.25: (α) Το αποτέλεσμα της εντολής `t.next = x.next;`. (β) Το αποτέλεσμα της εντολής `x.next = t;`.

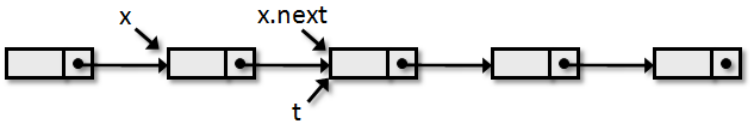
Με παρόμοιο τρόπο γίνεται και η διαγραφή κόμβου από λίστα. Ας υποθέσουμε ότι θέλουμε να διαγράψουμε τον κόμβο που βρίσκεται αμέσως μετά τον κόμβο με αναφορά `x` σε μια λίστα με κόμβους τύπου `Node`. Η διαγραφή μπορεί να γίνει απλώς με την εντολή `x.next = x.next.next;` (Εικόνα 3.26). Όπως φαίνεται και στην Εικόνα 3.26, ο κόμβος που αφαιρέθηκε από τη λίστα,

συνεχίζει να υφίσταται και συνεχίζει να δείχνει στον επόμενό του κόμβο στη λίστα. Όμως, δεν υπάρχει τρόπος να μεταβούμε στον κόμβο από κόμβους της λίστας. Επίσης, δεν έχουμε πλέον πρόσβαση στον κόμβο.

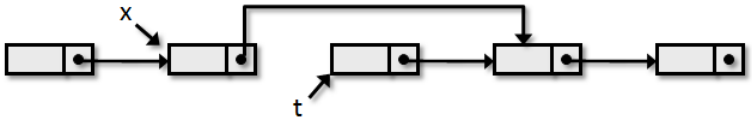


Εικόνα 3.26: Διαγραφή του κόμβου που έπεται του `x` σε μια απλά συνδεδεμένη λίστα.

Εάν θέλουμε να έχουμε πρόσβαση στον κόμβο ακόμη και μετά τη διαγραφή του (π.χ., για να τον εισαγάγουμε σε μια άλλη λίστα), τότε μπορούμε να χρησιμοποιήσουμε την εντολή `t = x.next;`, ώστε να μπορούμε να χρησιμοποιούμε τη μεταβλητή `t`, για να αναφερόμαστε σε αυτόν (Εικόνα 3.27). Με χρήση της μεταβλητής `t`, η διαγραφή του κόμβου από τη λίστα γίνεται και με την εντολή `x.next = t.next;` (Εικόνα 3.28).



Εικόνα 3.27: Το αποτέλεσμα της εντολής `t = x.next;`.



Εικόνα 3.28: Διαγραφή του κόμβου `t` με την εντολή `x.next = t.next;`.

3.4 Αναδρομή

Στην ενότητα αυτή θα μελετήσουμε αναδρομικούς αλγορίθμους. Ένας αλγόριθμος είναι *αναδρομικός* (*recursive*), εάν επιλύει ένα πρόβλημα λύνοντας ένα ή περισσότερα στιγμιότυπα του ίδιου προβλήματος.

Παράδειγμα 3.4.1 (Παραγοντικό μη αρνητικού ακέραιου αριθμού). Το παραγοντικό ενός θετικού ακέραιου αριθμού k ορίζεται ως

$$k! = 1 \times 2 \times 3 \times \cdots \times (k-1) \times k$$

ενώ, επίσης, εξ ορισμού, $0! = 1$.

Με βάση τον ορισμό του, το παραγοντικό μπορεί να υπολογιστεί εύκολα επαναληπτικά ως εξής:

```
t = 1;
for (int i=1; i<=N; i++)
      t *= i;
```

Ο ορισμός του παραγοντικού συνεπάγεται, επιπλέον, ότι για θετικό k ισχύει ότι

$$k! = k \times (k-1)!$$

το οποίο μας επιτρέπει να υπολογίσουμε το παραγοντικό του k με αναδρομικό τρόπο. Το αντίστοιχο αναδρομικό πρόγραμμα είναι:

```
int factorial (int k)
{
    if (k == 0) return 1;
    return k * factorial(k-1);
}
```

Με αφορμή το προηγούμενο παράδειγμα, σημειώνουμε ότι κάθε αναδρομικό πρόγραμμα μπορεί να μετατραπεί σε ισοδύναμο μη αναδρομικό πρόγραμμα. Ωστόσο, πολλές φορές, η χρήση αναδρομής δίνει πιο σύντομα ή/και πιο αποδοτικά προγράμματα.

Ας δούμε άλλα δύο παραδείγματα αναδρομικών αλγορίθμων.

Παράδειγμα 3.4.2 (Μέγιστος κοινός διαιρέτης). Ο μέγιστος κοινός διαιρέτης (greatest common divisor) δύο μη αρνητικών ακέραιων αριθμών (που δεν είναι και οι δύο ίσοι με 0) ορίζεται ως ο μεγαλύτερος ακέραιος που τους διαιρεί ακριβώς. Ίσως η πιο συνηθισμένη μέθοδος υπολογισμού του μέγιστου κοινού διαιρέτη είναι ο Αλγόριθμος του Ευκλείδη, ο οποίος βασίζεται στο πόρισμα που προκύπτει από την ακόλουθη πρόταση (με $x \bmod y$ συμβολίζουμε το υπόλοιπο της διαίρεσης του x δια του y: παραδείγματος χάριν, $17 \bmod 5 = 2$ γιατί $17 = 5 \times 3 + 2$, ενώ $5 \bmod 17 = 5$ γιατί $5 = 17 \times 0 + 5$).

Πρόταση 3.4.1 *Ο μέγιστος κοινός διαιρέτης δύο θετικών ακέραιων αριθμών x, y (με $x > y$) ισούται με το μέγιστο κοινό διαιρέτη των $x - y$ και y.*

Απόδειξη. Έστω d κοινός διαιρέτης των x και y. Τότε υπάρχουν ακέραιοι κ, λ, τέτοιοι ώστε $x = \kappa d$ και $y = \lambda d$. Όμως, τότε $x - y = (\kappa - \lambda)d$, δηλαδή ο d είναι διαιρέτης και του $x - y$. Καθώς ο d είναι κοινός διαιρέτης των $x - y$ και y και αυτό ισχύει για κάθε κοινό διαιρέτη d των x και y, συμπεραίνουμε ότι $\text{μκδ}(x, y) \leq \text{μκδ}(x - y, y)$.

Ας θεωρήσουμε τώρα έναν κοινό διαιρέτη d των $x - y$ και y, δηλαδή υπάρχουν ακέραιοι κ, λ τέτοιοι ώστε $x - y = \kappa d$ και $y = \lambda d$. Όμως, τότε $x = (x - y) + y = \kappa d + \lambda d = (\kappa + \lambda)d$, δηλαδή ο d είναι διαιρέτης και του x. Τότε, όπως παραπάνω, $\text{μκδ}(x - y, y) \leq \text{μκδ}(x, y)$.

Οι δύο ανισότητες συνεπάγονται την αλήθεια της πρότασης. ■

Η επαναληπτική εφαρμογή της Πρότασης 3.4.1 οδηγεί στο ακόλουθο πόρισμα.

Πόρισμα 3.4.1 *Ο μέγιστος κοινός διαιρέτης δύο θετικών ακέραιων αριθμών x, y (με $x > y$) ισούται με το μέγιστο κοινό διαιρέτη των $x \bmod y$ και y.*

Ο Αλγόριθμος του Ευκλείδη μπορεί να περιγραφεί ως υποπρόγραμμα όπως φαίνεται ακολούθως, λαμβάνοντας υπόψη ότι ο μέγιστος κοινός διαιρέτης ενός θετικού ακέραιου αριθμού x και του 0 είναι ίσος με x.

```
int gcd (int x, int y)
{
    if (y == 0) return x;
    return gcd(y, x%y);
}
```

Αξίζει να σημειωθεί ότι $x \bmod y < y$. Συνεπώς, εάν αρχικά $x > y$, το παραπάνω υποπρόγραμμα έχει πάντα το πρώτο του όρισμα μεγαλύτερο από το δεύτερο. Αλλά και στην περίπτωση που $x < y$, τότε $x \bmod y = x$, οπότε στην πρώτη αναδρομική κλήση με ορίσματα y και $x \bmod y$, το πρώτο και δεύτερο όρισμα είναι y και x, αντίστοιχα, με $y > x$, δηλαδή σε

αυτήν την αναδρομική κλήση, το πρώτο όρισμα είναι μεγαλύτερο από το δεύτερο, κάτι το οποίο θα ισχύσει και σε όλες τις υπόλοιπες αναδρομικές κλήσεις.

Ας δούμε πως λειτουργεί ο αλγόριθμος του Ευκλείδη όταν εφαρμοσθεί στους θετικούς ακέραιους 128 και 40:

$$\gcd(128,40) = \gcd(40,128 \bmod 40) = \gcd(40,8) = \gcd(8,40 \bmod 8) = \gcd(8,0) = 8.$$

Άρα, ο μέγιστος κοινός διαιρέτης των 128 και 40 είναι το 8.

Όμως, από το παραπάνω υποπρόγραμμα, δεν είναι σαφές πόσες επαναλήψεις απαιτούνται μέχρι την ολοκλήρωση του αλγορίθμου. Για να εκτιμήσουμε το πλήθος επαναλήψεων που απαιτούνται, αποδεικνύουμε την ακόλουθη ιδιότητα.

Ιδιότητα 3.4.1 *Ας θεωρήσουμε ότι* $x \geq y$. *Τότε* $x \bmod y < x/2$.

Απόδειξη. Εάν $y \leq x/2$, τότε $x \bmod y < y \leq x/2$. Αντίθετα, εάν $y > x/2$, τότε $y \leq x < 2y \Longrightarrow x \bmod y = x - y < x/2$. ■

Για να υπολογίσει το μέγιστο κοινό διαιρέτη δύο θετικών ακέραιων αριθμών x, y με $x \geq y$, ο αλγόριθμος του Ευκλείδη υπολογίζει το μέγιστο κοινό διαιρέτη των y και $x \bmod y$ και στη συνέχεια, εάν $x \bmod y \neq 0$, υπολογίζει το μέγιστο κοινό διαιρέτη των $x \bmod y$ και $y \bmod (x \bmod y)$. Αυτό σημαίνει ότι σε 2 επαναλήψεις το αρχικό ζεύγος ορισμάτων x, y αντικαθίσταται από το ζεύγος $x \bmod y, y \bmod (x \bmod y)$. Όμως, η Ιδιότητα 3.4.1 συνεπάγεται ότι σε δύο επαναλήψεις ο μεγαλύτερος αριθμός του ζεύγους τουλάχιστον υποδιπλασιάζεται, άρα, όταν ο αλγόριθμος του Ευκλείδη εφαρμοσθεί σε δύο μη αρνητικούς ακέραιους x, y ολοκληρώνεται σε $O(\log_2 \max\{x, y\})$ επαναλήψεις.

Παράδειγμα 3.4.3 (Πλήθος κόμβων λίστας). Ας θεωρήσουμε απλά συνδεδεμένες λίστες με κόμβους που δηλώνονται ως εξής:

```
private class Node {
      Item item;
      Node next;
}
```

Μας ενδιαφέρει να μετρήσουμε το πλήθος κόμβων σε μια τέτοια λίστα. Εύκολα μπορούμε να το κάνουμε αναδρομικά λαμβάνοντας υπόψη ότι το πλήθος κόμβων μιας λίστας ισούται με 1 (για τον πρώτο κόμβο της) συν το πλήθος υπόλοιπων κόμβων της λίστας. Σε αυτήν ακριβώς την ιδέα βασίζεται και η συνάρτηση `count`:

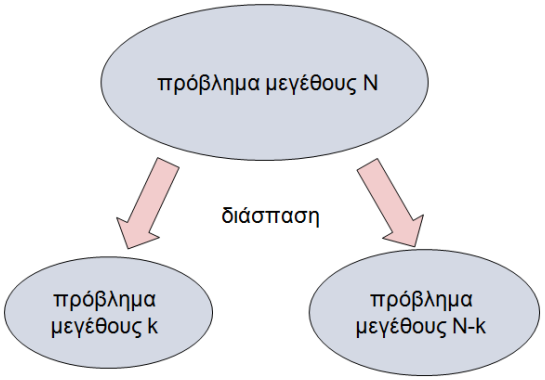
```
int count(Node x) {
      if (x == null) return 0;
      return 1 + count(x.next);
}
```

3.4.1 Μέθοδος «Διαίρει και Βασίλευε»

Μία από τις κυριότερες μεθοδολογίες που μας επιτρέπουν να κατασκευάσουμε αναδρομικούς αλγορίθμους για διάφορα προβλήματα είναι η μέθοδος «*διαίρει και βασίλευε*» (*divide and conquer*). Η βασική ιδέα της μεθόδου συνίσταται στη διάσπαση του προβλήματος που μας ενδιαφέρει σε (δύο, τις περισσότερες φορές) μικρότερα προβλήματα. Εάν αυτά δεν είναι αρκετά μικρά, ώστε να μπορούμε να τα επιλύσουμε απευθείας, επαναλαμβάνουμε τη διαδικασία διάσπασης και ούτω καθεξής. Όταν πλέον φθάσουμε σε προβλήματα τα οποία

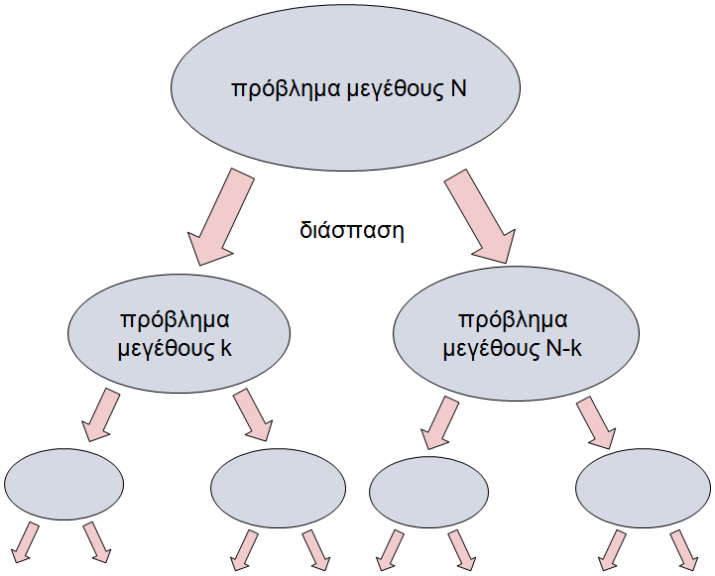
μπορούν να επιλυθούν απευθείας, τα επιλύουμε και αρχίζουμε να συνθέτουμε κατάλληλα τις λύσεις των προβλημάτων αυτών, ώστε να υπολογίσουμε τη λύση μεγαλύτερων προβλημάτων της διάσπασης και ούτω καθεξής, έως ότου φθάσουμε στο αρχικό μας πρόβλημα.

Σχηματικά, η διαδικασία περιγράφεται στις Εικόνες 3.29-3.32. Αρχικά, θεωρούμε ότι έχουμε ένα πρόβλημα μεγέθους N, το οποίο είναι αρκετά μεγάλο, ώστε να μην μπορούμε να το επιλύσουμε απευθείας. Για να το επιλύσουμε, το διασπούμε σε δύο προβλήματα με μεγέθη έστω k και $N - k$ (Εικόνα 3.29).

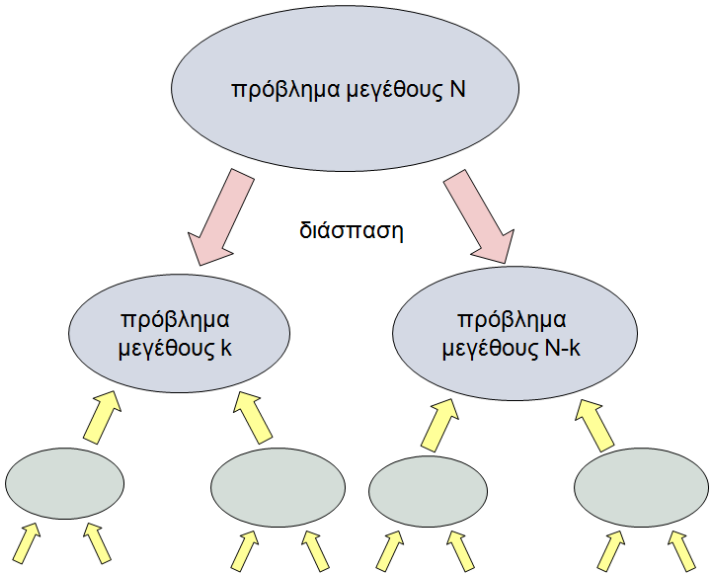


Εικόνα 3.29: Διάσπαση του αρχικού προβλήματος.

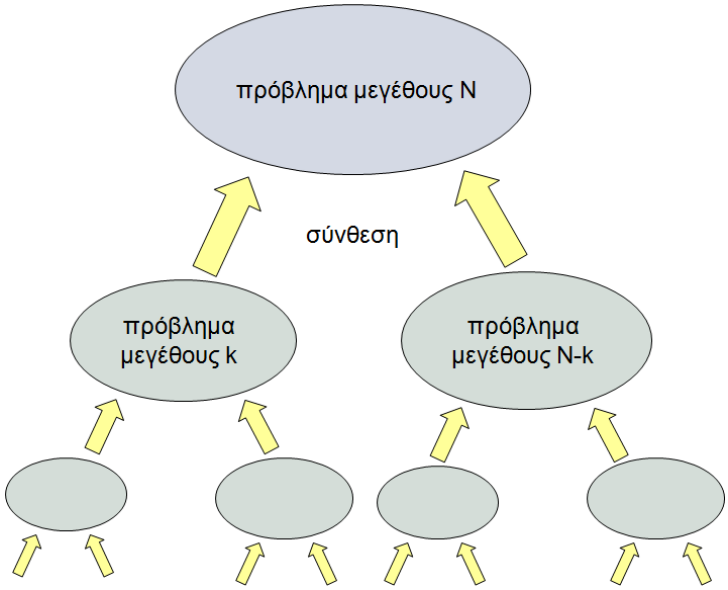
Επιλύουμε αναδρομικά τα δύο υποπροβλήματα. Εάν τα υποπροβλήματα είναι αρκετά μικρού μεγέθους, τότε τα επιλύουμε απευθείας. Εάν όχι, τότε η αναδρομική επίλυσή τους συνίσταται στην περαιτέρω διάσπασή τους (Εικόνα 3.30), στην αναδρομική επίλυση των μικρότερων προβλημάτων και στη σύνθεση των λύσεών τους σε μια λύση για τα υποπροβλήματα μεγέθους k και $N - k$ (Εικόνα 3.31). Τέλος, από τις λύσεις αυτών των υποπροβλημάτων συνθέτουμε μια λύση για το αρχικό πρόβλημα (Εικόνα 3.32).



Εικόνα 3.30: Περαιτέρω διάσπαση σε μικρότερα υποπροβλήματα.

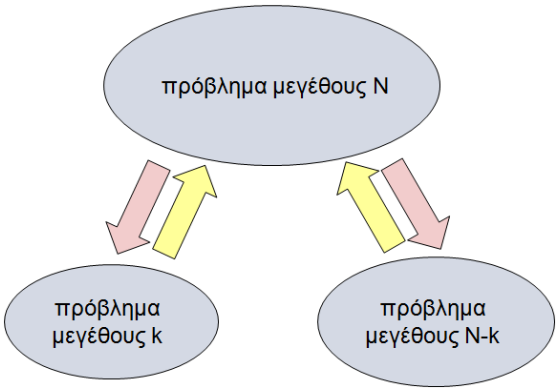


Εικόνα 3.31: Σύνθεση των λύσεων των μικρότερων προβλημάτων για την επίλυση των προβλημάτων μεγέθους k και $N-k$.



Εικόνα 3.32: Σύνθεση των λύσεων των προβλημάτων μεγέθους k και $N-k$ σε μια λύση για το πρόβλημα μεγέθους N.

Από την παραπάνω περιγραφή αντιλαμβανόμαστε ότι τα κύρια στοιχεία της μεθόδου «διαίρει και βασίλευε» είναι η κατάλληλη διάσπαση του προβλήματος σε μικρότερα προβλήματα και η κατάλληλη σύνθεση των λύσεων των μικρότερων προβλημάτων ώστε να προκύψει μια λύση του προβλήματος που διασπάστηκε (Εικόνα 3.33).



Εικόνα 3.33: Τα κύρια στοιχεία της μεθόδου «διαίρει και βασίλευε»: διάσπαση προβλήματος και σύνθεση λύσεων.

Εάν $T(N)$ συμβολίζει τον χρόνο που απαιτείται για την επίλυση ενός προβλήματος μεγέθους N με βάση τη διάσπαση που φαίνεται στην Εικόνα 3.33, τότε έχουμε

$$T(N) = D(N) + T(k) + T(N-k) + C(N)$$

όπου $D(N)$ είναι ο χρόνος που απαιτείται για τη διάσπαση του προβλήματος μεγέθους N και $C(N)$ είναι ο χρόνος που απαιτείται για τη σύνθεση μιας λύσης του προβλήματος μεγέθους N από τις λύσεις των προβλημάτων μεγέθους k και $N-k$.

Παράδειγμα 3.4.4 (Εύρεση ελάχιστου στοιχείου). Ας εφαρμόσουμε τη μέθοδο «διαίρει και βασίλευε» στο πρόβλημα της εύρεσης του ελάχιστου στοιχείου μιας ακολουθίας N αριθμών. Η μη αναδρομική επίλυση του προβλήματος συνίσταται στην επεξεργασία των στοιχείων της ακολουθίας με τη σειρά και την ενημέρωση (όποτε χρειάζεται) και αποθήκευση του τρέχοντος ελαχίστου. Αφού ολοκληρωθεί η επεξεργασία όλων των στοιχείων της ακολουθίας, το τρέχον ελάχιστο είναι το ελάχιστο στοιχείο της ακολουθίας. Με μορφή κώδικα, η διαδικασία αυτή έχει ως εξής (η μεταβλητή `t` καταχωρεί το τρέχον ελάχιστο της ακολουθίας):

```
t = a[0];
for (i = 1; i < N; i++)
    if (a[i] < t) t = a[i];
```

Για να επιλύσουμε το πρόβλημα με τη μέθοδο «διαίρει και βασίλευε», αρκεί να βρούμε αναδρομικά το ελάχιστο στοιχείο στο αριστερό μισό της ακολουθίας και το ελάχιστο στοιχείο στο δεξί μισό της ακολουθίας και να βρούμε το μικρότερο από αυτά τα δύο ελάχιστα στοιχεία. Το αντίστοιχο αναδρομικό πρόγραμμα έχει ως εξής:

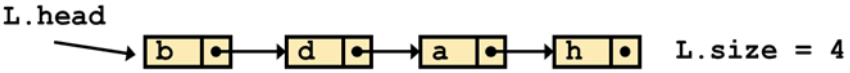
```
int min(int a[], int l, int r)
{
      int u, v, m = (l+r)/2;

      if (l == r) return a[l];

      u = min(a, l, m);   /* ελάχιστο στοιχείο στο αριστερό μισό */
      v = min(a, m+1, r);     /* ελάχιστο στοιχείο στο δεξί μισό */
      if (u < v) return u;
      else return v;
}
```

Ασκήσεις

3.1. α) Υλοποιούμε μια δομή συνδεδεμένης λίστας LinkedList, όπου ο κάθε κόμβος της, τύπου Node, αποθηκεύει ένα αντικείμενο κάποιου τύπου Item, καθώς και μια αναφορά στον επόμενο κόμβο της λίστας. Η πρόσβαση σε μια λίστα LinkedList L γίνεται από την αναφορά L.head στον πρώτο κόμβο της L. Επίσης, διατηρούμε μια μεταβλητή L.size η οποία δίνει το πλήθος των στοιχείων της λίστας, όπως φαίνεται στο παρακάτω σχήμα.



Θέλουμε η δομή LinkedList L να υποστηρίζει τις παρακάτω λειτουργίες:

- insert(Item item) : Εισάγει το στοιχείο item στη λίστα L. (Η θέση που τοποθετείται το item στην L δεν μας ενδιαφέρει.)
- delete(Node x) : Διαγράφει από τη λίστα L τον κόμβο x.
- join(LinkedList M) : Προσθέτει στην L τα στοιχεία της LinkedList M. Στο τέλος της λειτουργίας η λίστα M καταστρέφεται.

Δώστε όσο το δυνατόν πιο αποδοτικές υλοποιήσεις των παραπάνω λειτουργιών. Ποιος είναι ο χρόνος εκτέλεσης κάθε εντολής στη χειρότερη περίπτωση;

β) Δείξτε πώς μπορεί να τροποποιηθεί η παραπάνω δομή, ώστε όλες οι λειτουργίες insert, delete και join να εκτελούνται σε $O(1)$ χρόνο χειρότερης περίπτωσης.

3.2. Μία ακολουθία αριθμών $A = (a_1\ a_2\ \ldots\ a_n)$ ονομάζεται *μονοκόρυφη*, εάν για κάποιο p με $1 \le p \le n$, έχουμε $a_1 < a_2 < \ldots < a_p$ και $a_p > a_{p+1} > \ldots > a_n$. Θέλουμε να βρούμε το μέγιστο στοιχείο a_p μιας μονοκόρυφης ακολουθίας, διαβάζοντας όσο το δυνατό λιγότερα στοιχεία της. Σχεδιάστε έναν αποδοτικό αλγόριθμο για αυτό το πρόβλημα και αναλύστε τον ασυμπτωτικό χρόνο εκτέλεσης του. Υποθέστε ότι η ακολουθία είναι αποθηκευμένη σε έναν πίνακα, οπότε η προσπέλαση ενός στοιχείου της γίνεται σε $O(1)$ χρόνο.

3.3. Η παρακάτω συνάρτηση υπολογίζει τα αθροίσματα $A[i] + A[i+1] + \cdots + A[j]$ για όλα τα i και j, τέτοια ώστε $0 \le i \le j \le N-1$, όπου A ένας μονοδιάστατος πίνακας N ακέραιων. Τα αθροίσματα αποθηκεύονται σε ένα διδιάστατο $N \times N$ πίνακα ακεραίων B. (Υποθέτουμε ότι κάθε $B[i][j]$ έχει ήδη αρχικοποιηθεί με την τιμή 0.)

```
void partialSums(int *A, int N, int **B)
{
   int i,j,k;
   for (i=0; i<=N-1; i++)
     for (j=i; j<=N-1; j++)
       for (k=i; k<=j; k++)
          B[i][j] += A[k];
}
```

α) Ποίος είναι ο ασυμπτωτικός χρόνος εκτέλεσης $T(N)$ της partialSums;

β) Δώστε ένα βελτιωμένο τρόπο υπολογισμού του πίνακα B, με ασυμπτωτικά καλύτερο χρόνο εκτέλεσης $T'(N)$. Δηλαδή θέλουμε $T'(N) = o(T(N))$.

3.4. α) Έστω ότι μας δίνεται ένας ακέραιος αριθμός v και ένας διατεταγμένος πίνακας A με N ακέραιους, όπου $A[0] < A[1] < \ldots < A[N-1]$. Θέλουμε να βρούμε αν ο A περιέχει ένα αριθμό $A[i]$, τέτοιο ώστε $v + A[i] = 0$. Περιγράψτε μια γρήγορη μέθοδο που να εντοπίζει ένα τέτοιο $A[i]$, αν υπάρχει. Ποιός είναι ο ασυμπτωτικός χρόνος εκτέλεσης της μεθόδου σας στη χειρότερη περίπτωση;

β) Η παρακάτω μέθοδος δέχεται στην είσοδο έναν πίνακα A με N ακέραιους αριθμούς και μετράει πόσες τριάδες $(A[i], A[j], A[k])$ δίνουν άθροισμα 0.

```
public static int countTriples(int[] a) {
     int N = a.length;
     int count = 0;
     for (int i = 0; i < N; i++)
          for (int j = i+1; j < N; j++)
            for (int k = j+1; k < N; k++)
                 if (a[i] + a[j] + a[k] == 0) count++;
     return count;
}
```

Η μέθοδος countTriples εκτελείται σε χρόνο $\Theta(N^3)$. Περιγράψτε έναν ασυμπτωτικά ταχύτερο αλγόριθμο με τη βοήθεια της μεθόδου του ερωτήματος (α). Ποίος είναι ο ασυμπτωτικός χρόνος εκτέλεσης του αλγόριθμού σας στη χειρότερη περίπτωση;

3.5. Μας δίνεται ένας $N \times N$ διδιάστατος πίνακας A με N^2 διαφορετικούς ακέραιους. Ένα στοιχείο $A[i][j]$ είναι τοπικό ελάχιστο αν $A[i][j] < A[i+1][j]$, $A[i][j] < A[i][j+1]$, $A[i][j] < A[i-1][j]$ και $A[i][j] < A[i][j-1]$. Σχεδιάστε ένα αλγόριθμο ο οποίος να υπολογίζει ένα τοπικό ελάχιστο σε $O(N)$ χρόνο.

Υπόδειξη: Βρείτε το ελάχιστο $A[N/2][j]$ της γραμμής $N/2$ και ελέγξτε τους δύο γείτονες του $A[N/2-1][j]$ και $A[N/2+1][j]$ στην κάτω και στην πάνω γραμμή, αντίστοιχα. Χρησιμοποιήστε αναδρομή στο μισό πίνακα που περιέχει τον μικρότερο γείτονα και βρείτε το ελάχιστο στοιχείο της στήλης $N/2$.

Βιβλιογραφία

Golub, G., & Van Loan, C. F. (1996). *Matrix Computations (3rd Ed.).* John Hopkins University Press.

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Κεφάλαιο 4
Γραφήματα και Δένδρα

Περιεχόμενα

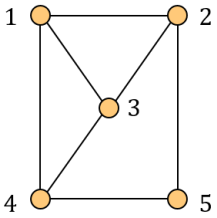
4.1 Γραφήματα 60
4.2 Δομές δεδομένων για την αναπαράσταση γραφημάτων 64
4.2.1 Υλοποίηση σε Java 66
4.3 Διερεύνηση γραφήματος 69
4.4 Δένδρα 86
4.4.1 Δυαδικά δένδρα 88
Ασκήσεις 91
Βιβλιογραφία 92

4.1 Γραφήματα

Τα γραφήματα είναι μαθηματικά αντικείμενα τα οποία κατέχουν κεντρικό ρόλο στη Επιστήμη της Πληροφορικής, καθώς χρησιμεύουν στη μοντελοποίηση πολλών σημαντικών οντοτήτων, όπως είναι τα δίκτυα υπολογιστών, τα οδικά και τα κοινωνικά δίκτυα και τα ηλεκτρονικά κυκλώματα. Ένα γράφημα $G = (V, E)$ απαρτίζεται από ένα σύνολο κόμβων V και ένα σύνολο ακμών E, οι οποίες εκφράζουν μια διμελή σχέση μεταξύ των κόμβων. Δηλαδή, κάθε ακμή e αντιστοιχεί σε ένα ζεύγος κόμβων u και v, το οποίο εκφράζουμε ως $e = (u, v)$. Η σειρά με την οποία εμφανίζονται οι κόμβοι στην ακμή (u, v) δεν έχει σημασία, όταν η σχέση που αναπαριστούν οι ακμές είναι συμμετρική. Στην περίπτωση αυτή, οι εκφράσεις (u, v) και (v, u) αναπαριστούν την ίδια ακμή μεταξύ των κόμβων u και v, οπότε αναφερόμαστε σε ένα μη κατευθυνόμενο γράφημα. Αντίθετα, όταν έχουμε μη συμμετρική σχέση, τότε λέμε ότι μια ακμή (u, v) έχει κατεύθυνση από τον κόμβο u προς τον κόμβο v. Για παράδειγμα, σε ένα κοινωνικό δίκτυο, οι κόμβοι του γραφήματος μπορεί να αντιστοιχούν σε άτομα και οι ακμές να εκφράζουν σχέση φιλίας μεταξύ των ατόμων. Στην περίπτωση αυτή έχουμε ένα μη κατευθυνόμενο γράφημα, καθώς εκλαμβάνουμε τη σχέση φιλίας ως συμμετρική (αν ο u θεωρεί τον v φίλο τότε και v θεωρεί τον u φίλο). Από την άλλη, ένα οδικό δίκτυο αναπαρίσταται από ένα κατευθυνόμενο γράφημα, που οι κόμβοι του αντιστοιχούν σε διασταυρώσεις και οι ακμές του σε δρόμους που συνδέουν τους κόμβους προς μία συγκεκριμένη κατεύθυνση. Ένας δρόμος διπλής κατεύθυνσης που συνδέει τους κόμβους u και v περιγράφεται με τις κατευθυνόμενες ακμές (u, v) και (v, u).

Ένα (μη κατευθυνόμενο ή κατευθυνόμενο) γράφημα μπορεί να αναπαρασταθεί σχηματικά όπως φαίνεται στην Εικόνα 4.1. Αναπαριστούμε τους κόμβους με σημεία (σχηματικά, με μικρούς κύκλους), ενώ τις ακμές με ευθύγραμμα τμήματα ή καμπύλες που συνδέουν δύο κόμβους. Φυσικά, μια τέτοια σχηματική αναπαράσταση ενός γραφήματος στο επίπεδο δεν είναι μοναδική και εξαρτάται από τη θέση στην οποία τοποθετούμε τους κόμβους. Στα κατευθυνόμενα γραφήματα δηλώνουμε την κατεύθυνση μιας ακμής (u, v) από τον κόμβο u προς τον κόμβο v με ένα βέλος που καταλήγει στον v. Παρατηρούμε, επίσης, ότι στην Εικόνα

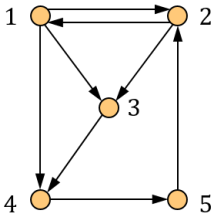
4.1 έχουμε ονομάσει τους κόμβους με ακεραίους αριθμούς. Γενικά, για ένα γράφημα $G = (V, E)$ με n κόμβους θα υποθέτουμε ότι $V = \{1,2, \dots, n\}$. Αυτή η υπόθεση είναι βολική στην ανάπτυξη κατάλληλων δομών δεδομένων για την αναπαράσταση γραφημάτων, όπως θα δούμε στη συνέχεια.



$G = (V, E)$

$V = \{1,2,3,4,5\}$

$E = \{(1,2), (1,3), (1,4), (2,3), (2,5), (3,4), (4,5)\}$



$G = (V, E)$

$V = \{1,2,3,4,5\}$

$E = \{(1,2), (1,3), (1,4), (2,1), (2,3), (3,4), (4,5), (5,2)\}$

Εικόνα 4.1: Ένα μη-κατευθυνόμενο γράφημα με 5 κόμβους και 7 ακμές (πάνω) και ένα κατευθυνόμενο γράφημα με 5 κόμβους και 8 ακμές (κάτω).

Στο παρόν κεφάλαιο επικεντρωνόμαστε στο σχεδιασμό δομών δεδομένων για την αποδοτική επεξεργασία γραφημάτων, καθώς και σε βασικούς αλγόριθμους επεξεργασίας γραφημάτων. Η Θεωρία Γραφημάτων αποτελεί μια πλούσια περιοχή των Μαθηματικών, με πολλές πρακτικές εφαρμογές. Ωστόσο, μια εκτενής αναφορά στην περιοχή αυτή ξεφεύγει από τους σκοπούς του συγγράμματος και έτσι θα αρκεστούμε μόνο σε μερικούς βασικούς ορισμούς που χρειαζόμαστε για την ανάπτυξη μερικών βασικών αλγόριθμων.

Θα ασχοληθούμε με γραφήματα στα οποία δεν υπάρχουν βρόχοι (ακμές της μορφής (v, v)) και παράλληλες ακμές, δηλαδή δύο ή περισσότερα αντίγραφα της ίδιας ακμής (u, v). Τέτοια γραφήματα ονομάζονται *απλά*.

Υπογραφήματα. Ένα *υπογράφημα* ενός γραφήματος G=(V,E) είναι ένα γράφημα $H = (V', E')$, τέτοιο ώστε $V' \subseteq V$ και $E' \subseteq E$. Εάν το E' περιέχει όλες τις ακμές του E μεταξύ των κόμβων του V', δηλαδή $E' = \{(u, v) \in E \mid u \in V' \text{ και } v \in V'\}$, τότε λέμε ότι το H είναι το επαγόμενο υπογράφημα από το σύνολο κόμβων V'. Θα συμβολίζουμε με $G[A]$ το επαγόμενο υπογράφημα από ένα σύνολο κόμβων A του γραφήματος G. Δείτε την Εικόνα 4.2.



Εικόνα 4.2: Τα επαγόμενα υπογραφήματα $G[A]$ των γραφημάτων της Εικόνας 4.1, για $A = \{1,2,5\}$.

Έστω $G = (V, E)$ ένα απλό, μη κατευθυνόμενο γράφημα με n κόμβους και m ακμές. Αφού κάθε ακμή είναι ένα μη διατεταγμένο ζεύγος κόμβων, ισχύει $0 \leq m \leq n(n-1)/2$.

Ορίζουμε διαφόρους τύπους ακολουθιών κόμβων και ακμών του G.

Περίπατος. Μία ακολουθία κόμβων $W = (v_0, v_1, v_2, \ldots, v_k)$ του G ονομάζεται *περίπατος*, εάν υπάρχει η ακμή $e_i = (v_{i-1}, v_i)$ στο G, για κάθε i=1,2,...,k. Οι ακμές e_i ονομάζονται ακμές του περιπάτου W. Το *μήκος* $l(W)$ του περιπάτου W ισούται με το πλήθος των ακμών του, δηλαδή $l(W) = k$. Για παράδειγμα, οι ακολουθίες κόμβων (1,2,3,1,4) και (5,2,3,4) αποτελούν περίπατους του μη κατευθυνόμενου γραφήματος της στην Εικόνα 4.1, με μήκη 4 και 3, αντίστοιχα.

Διαδρομή ή μονοπάτι. Μία ακολουθία κόμβων $P = (v_0, v_1, v_2, \ldots, v_k)$ του G ονομάζεται *διαδρομή*, εάν είναι περίπατος του οποίου δεν επαναλαμβάνεται κάποιος κόμβος. Το *μήκος* $l(P)$ μίας διαδρομής P ισούται με το πλήθος των ακμών της. Στο προηγούμενο παράδειγμα, ο περίπατος (5,2,3,4) του μη κατευθυνόμενου γραφήματος της Εικόνα 4.1 είναι διαδρομή, ενώ ο περίπατος (1,2,3,1,4) όχι, καθώς επαναλαμβάνεται ο κόμβος 1.

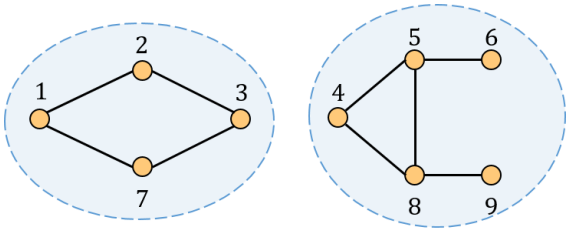
Κύκλος. Μία ακολουθία κόμβων $C = (v_0, v_1, v_2, \ldots, v_{k-1}, v_k)$ ονομάζεται κύκλος, εάν είναι περίπατος ο οποίος καταλήγει στον κόμβο από τον οποίο ξεκίνησε, δηλαδή όταν $v_0 = v_k$. Αν η ακολουθία $(v_0, v_1, v_2, \ldots, v_{k-1})$ είναι διαδρομή, τότε ο κόμβος v_0 είναι ο μόνος που επαναλαμβάνεται στην ακολουθία $C = (v_0, v_1, v_2, \ldots, v_{k-1}, v_0)$. Σε αυτήν την περίπτωση λέμε ότι ο C είναι ένας απλός κύκλος. Στο μη-κατευθυνόμενο γράφημα της Εικόνα 4.1, οι περίπατοι (1,2,3,1,4,3,1) και (4,3,1,2,5,4) είναι κύκλοι, αλλά μόνο ο δεύτερος κύκλος είναι απλός.

Οι παραπάνω έννοιες ορίζονται ακριβώς με τον ίδιο τρόπο και σε κατευθυνόμενα γραφήματα. Έτσι, για παράδειγμα, οι ακολουθίες κόμβων (1,2,1,4), (2,1,3,4), (1,2,3,4,5,2,1) και (1,3,4,5,2,1) αποτελούν, αντίστοιχα, περίπατο, διαδρομή, κύκλο και απλό κύκλο του κατευθυνόμενου γραφήματος της Εικόνα 4.1.

Συνεκτικό γράφημα. Ένα μη κατευθυνόμενο γράφημα ονομάζεται *συνεκτικό*, εάν μεταξύ οποιωνδήποτε δύο κόμβων του x και y υπάρχει μία διαδρομή $(x, v_1, v_2, \ldots, v_p, y)$ που συνδέει τους κόμβους x και y.

Συνεκτικές συνιστώσες. Έστω $G = (V, E)$ ένα μη κατευθυνόμενο και μη συνεκτικό γράφημα και έστω $V_1, V_2, \ldots, V_k$ μία διαμέριση του συνόλου V των κόμβων του γραφήματος G σε k υποσύνολα, με την εξής ιδιότητα: δύο κόμβοι x και y συνδέονται με μία διαδρομή $(x, v_1, v_2, \ldots, v_p, y)$ στο G, αν και μόνο αν οι x και y ανήκουν στο ίδιο σύνολο κόμβων V_i της διαμέρισης του V. Τότε τα επαγόμενα υπογραφήματα $G[V_1]$, $G[V_2]$, …, $G[V_k]$ ονομάζονται *συνεκτικές συνιστώσες* του γραφήματος G. Οι συνεκτικές συνιστώσες είναι τα μέγιστα συνεκτικά υπογραφήματα του G.

Ένα μη κατευθυνόμενο και μη συνεκτικό γράφημα G αποτελείται από $k \geq 2$ συνεκτικές συνιστώσες. Το μη κατευθυνόμενο γράφημα της Εικόνα 4.1 είναι συνεκτικό, ενώ αυτό της Εικόνα 4.3 δεν είναι συνεκτικό.

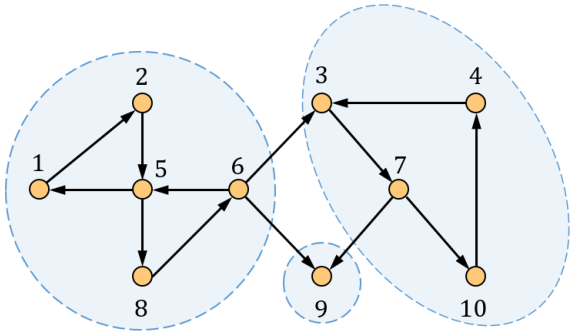


Εικόνα 4.3: Ένα μη κατευθυνόμενο και μη συνεκτικό γράφημα. Αποτελείται από δύο συνεκτικές συνιστώσες με σύνολα κόμβων {1,2,3,7} και {4,5,6,8,9}.

Ισχυρά συνεκτικό γράφημα. Ένα κατευθυνόμενο γράφημα ονομάζεται *ισχυρά συνεκτικό*, εάν μεταξύ οποιωνδήποτε δύο κόμβων του x και y υπάρχει μία διαδρομή $(x, v_1, v_2, \ldots, v_p, y)$ από τον κόμβο x στον κόμβο y και μία διαδρομή $(y, u_1, u_2, \ldots, u_q, x)$ από τον κόμβο y στον κόμβο x.

Ισχυρά συνεκτικές συνιστώσες. Όμοια με τα μη κατευθυνόμενα γραφήματα, ένα μη ισχυρά συνεκτικό κατευθυνόμενο γράφημα $G = (V, E)$ χωρίζεται σε ισχυρά συνεκτικές συνιστώσες. Συγκεκριμένα, έστω $V_1, V_2, \ldots, V_k$ μία διαμέριση του συνόλου V των κόμβων του γραφήματος G σε k υποσύνολα, με την εξής ιδιότητα: δύο κόμβοι x και y συνδέονται με μία διαδρομή $(x, v_1, v_2, \ldots, v_p, y)$ και με μία διαδρομή $(y, u_1, u_2, \ldots, u_q, x)$στο G, αν και μόνο αν οι x και y ανήκουν στο ίδιο σύνολο κόμβων V_i της διαμέρισης του V. Τότε τα επαγόμενα υπογραφήματα $G[V_1]$, $G[V_2]$, …, $G[V_k]$ ονομάζονται *ισχυρά συνεκτικές συνιστώσες* του γραφήματος G. Οι ισχυρά συνεκτικές συνιστώσες είναι τα μέγιστα ισχυρά συνεκτικά υπογραφήματα του G.

Ένα κατευθυνόμενο και μη ισχυρά συνεκτικό γράφημα G αποτελείται από $k \geq 2$ ισχυρά συνεκτικές συνιστώσες. Το κατευθυνόμενο γράφημα της Εικόνας 4.1 είναι ισχυρά συνεκτικό, ενώ αυτό της Εικόνα 4.4 δεν είναι ισχυρά συνεκτικό.



Εικόνα 4.4: Ένα κατευθυνόμενο και μη ισχυρά συνεκτικό γράφημα. Αποτελείται από τρεις ισχυρά συνεκτικές συνιστώσες με σύνολα κόμβων {1,2,5,6,8},{3,4,7,10} και {9}.

Αποστάσεις σε γραφήματα. Η απόσταση μεταξύ δύο κόμβων x και y ενός γραφήματος G ορίζεται ως το ελάχιστο μήκος μιας διαδρομής μεταξύ των κόμβων x και y.

Σε πολλά προβλήματα που αφορούν την εύρεση ελάχιστων διαδρομών, οι ακμές e του γραφήματος έχουν κάποια βάρη w(e). Σε αυτήν την περίπτωση, ορίζουμε το βάρος μιας διαδρομής P ως το άθροισμα των βαρών των ακμών της P, δηλαδή $w(P) = \sum_{e \in P} w(e)$.

4.2 Δομές δεδομένων για την αναπαράσταση γραφημάτων

Για την αναπαράσταση ενός γραφήματος στη μνήμη του υπολογιστή, χρειαζόμαστε να αποθηκεύσουμε τις ακμές του γραφήματος με τέτοιο τρόπο, ώστε να είναι εύκολο να βρούμε τους γείτονες ενός κόμβου. Υπάρχουν δύο βασικοί τρόποι αναπαράστασης: ο πίνακας γειτνίασης και οι λίστες γειτνίασης. Θα αναφερθούμε, επίσης, σε μια ακόμα αναπαράσταση με μονοδιάστατους πίνακες, η οποία είναι κατάλληλη για γραφήματα που δεν επιδέχονται αλλαγές, με την προσθήκη ή διαγραφή ακμών ή κόμβων. Όπως αναφέραμε και στην εισαγωγή του κεφαλαίου, υποθέτουμε ότι οι n κόμβοι του γραφήματος $G = (V, E)$ που θέλουμε να αποθηκεύσουμε είναι αριθμημένοι από το 1 έως και το n, δηλαδή μπορούμε να θεωρήσουμε ότι $V = \{1,2, \dots, n\}$.

Οι επιδόσεις των δομών δεδομένων που θα μελετήσουμε στη συνέχεια συνοψίζονται στον παρακάτω πίνακα.

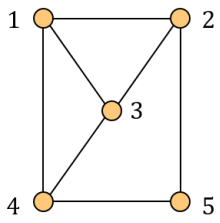
Πίνακας 4.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης μερικών βασικών λειτουργιών μιας δομής αναπαράστασης γραφήματος με n κόμβους και m ακμές.

	αναζήτηση ακμής	εισαγωγή ακμής	προσπέλαση όλων των ακμών
πίνακας γειτνίασης	$O(1)$	$O(1)$	$O(n^2)$
λίστα γειτνίασης	$O(n)$	$O(1)$	$O(m)$

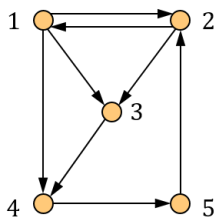
Πίνακας Γειτνίασης. Έστω $G = (V, E)$ ένα γράφημα με n κόμβους. Ο πίνακας γειτνίασης $A = (a_{i,j})$ του G είναι ένας $n \times n$ πίνακας, τέτοιος ώστε:

$$a_{i,j} = \begin{cases} 0, & \text{εάν } (i,j) \notin E \\ 1, & \text{εάν } (i,j) \in E \end{cases}$$

Εξ ορισμού, τα στοιχεία της κυρίας διαγωνίου του πίνακα A είναι μηδέν και ο A είναι συμμετρικός ως προς την κύρια διαγώνιο, εάν το γράφημα είναι μη-κατευθυνόμενο.



$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

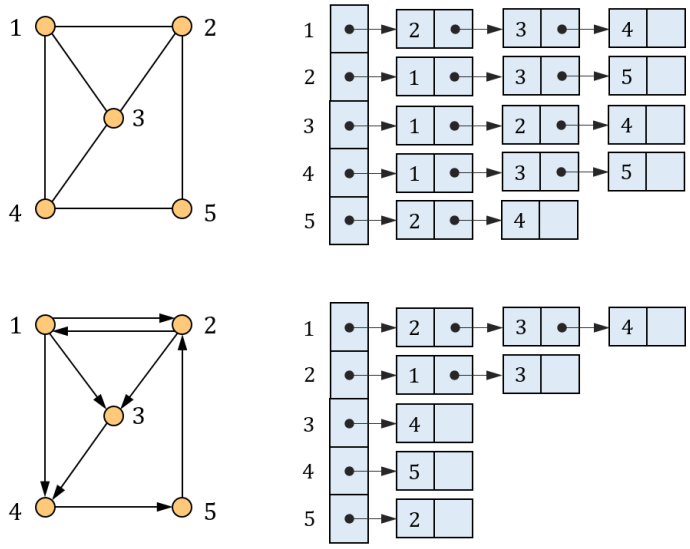


$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Εικόνα 4.5: Τα γραφήματα της Εικόνας 4.1 και οι πίνακες γειτνίασης τους.

Σε μια γλώσσα προγραμματισμού όπως η Java, μπορούμε να αποθηκεύσουμε απευθείας τον πίνακας γειτνίασης A σε ένα διδιάστατο πίνακα τύπου boolean. Έτσι, μπορούμε να απαντήσουμε άμεσα αν ένα ζεύγος κόμβων συνδέεται με μια ακμή. Επιπλέον, μπορούμε εύκολα να εισαγάγουμε ή να διαγράψουμε μια δεδομένη ακμή. Οι παραπάνω λειτουργίες πραγματοποιούνται σε σταθερό χρόνο, ωστόσο, σε μία τέτοια αναπαράσταση, υπάρχουν λειτουργίες που δεν υποστηρίζονται αποδοτικά. Για παράδειγμα, η εύρεση όλων των γειτόνων ενός κόμβου v απαιτεί $O(n)$ χρόνο, δηλαδή συνολικά $O(n^2)$ χρόνο, για να προσπελάσουμε όλες τις ακμές του γραφήματος, όσος είναι και ο απαιτούμενος αποθηκευτικός χώρος για τη δομή. Είναι σημαντικό να παρατηρήσουμε ότι αυτός ο χρόνος προσπέλασης των ακμών απέχει πολύ από τον επιθυμητό χρόνο $O(m)$, όταν το γράφημα είναι αραιό, δηλαδή όταν $m = O(n)$. Οι επόμενες δύο δομές που αναφέρουμε αντιμετωπίζουν αυτό ακριβώς το πρόβλημα.

Λίστα Γειτνίασης ενός Γραφήματος. Για κάθε κόμβο v του γραφήματος G δημιουργούμε μία λίστα $N(v)$ που περιέχει όλους τους κόμβους w για τους οποίους υπάρχει η ακμή (v, w) στο G. Ονομάζουμε την $N(v)$ *λίστα γειτνίασης του κόμβου* v. Μπορούμε να αναπαραστήσουμε τη $N(v)$ εσωτερικά σε έναν υπολογιστή ως μία απλά συνδεδεμένη λίστα, όπως φαίνεται στην Εικόνα 4.6. Η δομή που αποτελείται από τις n συνδεδεμένες λίστες των κόμβων του G ονομάζεται *λίστα γειτνίασης* του γραφήματος G.



Εικόνα 4.6: Τα γραφήματα της Εικόνας 4.1 και οι λίστες γειτνίασης τους.

Παρατηρήστε ότι στη λίστα γειτνίασης ενός μη κατευθυνόμενου γραφήματος, η κάθε ακμή (v, w) εμφανίζεται ακριβώς δύο φορές, καθώς ο κόμβος w αποθηκεύεται στη λίστα $N(v)$ και ο κόμβος v αποθηκεύεται στη λίστα $N(w)$. Αντίθετα, σε ένα κατευθυνόμενο γράφημα, η ακμή (v, w) εμφανίζεται μια φορά, αποθηκεύοντας τον w στη $N(v)$. Και στις δύο περιπτώσεις, ο χώρος που απαιτείται για την αποθήκευση στη μνήμη του υπολογιστή της λίστας γειτνίασης ενός γραφήματος με n κόμβους και m ακμές είναι $O(n + m)$. Γίνεται, επομένως, προφανές ότι για την εσωτερική αναπαράσταση ενός αραιού γραφήματος G είναι προτιμότερο, όσον αφορά στο χώρο αποθήκευσης, να χρησιμοποιούμε τη λίστα γειτνίασης του G αντί του πίνακα γειτνίασής του. Συχνά, η λίστα γειτνίασης είναι, επίσης, προτιμότερη όσον αφορά ακόμα και στο χρόνο εκτέλεσης ορισμένων λειτουργιών. Για παράδειγμα, πολλοί βασικοί αλγόριθμοι σε γραφήματα χρειάζεται να εξετάζουν τους γείτονες ενός κόμβου v. Αυτό απαιτεί τη διερεύνηση της λίστας $N(v)$ και, επομένως, εκτελείται σε χρόνο ανάλογο του πλήθους των κόμβων σε αυτή τη λίστα. Έτσι, ο χρόνος προσπέλασης όλων των ακμών του γραφήματος είναι $O(m)$. Η δομή της λίστας γειτνίασης επιτρέπει, επίσης, την εισαγωγή μιας νέας ακμής (v, w) σε σταθερό χρόνο. Αντίθετα, λειτουργίες όπως ο έλεγχος για το αν δύο κόμβοι συνδέονται με μια ακμή

απαιτούν την αναζήτηση σε μια συνδεδεμένη λίστα, δηλαδή $O(n)$ χρόνο στη χειρότερη περίπτωση.

Επομένως, μπορούμε με βεβαιότητα να πούμε ότι δεν υπάρχει αναπαράσταση γραφήματος η οποία να είναι ιδανική ή καλύτερη σε σχέση με το χρόνο και το χώρο για όλες τις *λειτουργίες*.

Στατική Υλοποίηση Λίστας Γειτνίασης. Σε πολλές εφαρμογές χειριζόμαστε γραφήματα τα οποία δεν μεταβάλλονται με την εισαγωγή ή διαγραφή ακμών ή και κόμβων. Σε μια τέτοια περίπτωση, μπορούμε να χρησιμοποιήσουμε μια πιο συμπαγή υλοποίηση της λίστας γειτνίασης, με δύο πίνακες target και first. Ο πίνακας αποθηκεύει τους κόμβους της κάθε λίστας γειτνίασης $N(v)$ σε διαδοχικές θέσεις, ξεκινώντας από τη λίστα $N(1)$ του κόμβου $v = 1$. Στον πίνακα first αποθηκεύουμε τη θέση του πρώτου κόμβου της κάθε λίστας $N(v)$. Συγκεκριμένα, για κάθε κόμβο $v \in V = \{1,2,\dots,n\}$, η τιμή first$[v]$ δίνει τη θέση του πίνακα target στην οποία είναι αποθηκευμένος ο πρώτος κόμβος της λίστας γειτνίασης $N(v)$. Έτσι, οι κόμβοι της λίστας $N(v)$ βρίσκονται στον πίνακα target στις θέσεις από first$[v]$ έως και first$[v+1]-1$, για $v \in \{1,2,\dots,n-1\}$, και στις θέσεις από first$[v]$ έως και n, για $v = n$. Στην Εικόνα 4. φαίνονται τα περιεχόμενα των δύο πινάκων για το μη-κατευθυνόμενο γράφημα της Εικόνα 4.1.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
target = [	2	3	4	1	3	5	1	2	4	1	3	5	2	4]
first = [	–	0	3	6	9	12]								

Εικόνα 4.7: Στατική υλοποίηση της λίστας γειτνίασης του μη-κατευθυνόμενου γραφήματος της Εικόνας 4.1.

Οι επιδόσεις της στατικής υλοποίησης της λίστας γειτνίασης είναι ανάλογες με την υλοποίηση με συνδεδεμένες λίστες, που είδαμε παραπάνω. Δηλαδή, απαιτεί $O(n+m)$ αποθηκευτικό χώρο για γράφημα με n κόμβους και m ακμές, επιτρέπει την πρόσβαση σε όλους τους $|N(v)|$ γείτονες ενός κόμβου v σε $O(|N(v)|)$ χρόνο, ενώ απαιτεί τον ίδιο χρόνο, $O(|N(v)|)$ στη χειρότερη περίπτωση, για να εξετάσει αν υπάρχει ένας συγκεκριμένος κόμβος w στη λίστα $N(v)$. Η στατική δομή, ωστόσο, αποδίδει καλύτερα στην πράξη για δύο λόγους: α) αποφεύγει την αποθήκευση αναφορών σε κόμβους συνδεδεμένης λίστας, επομένως χρειάζεται μικρότερο αποθηκευτικό χώρο και β) η αποθήκευση των κόμβων κάθε λίστας γειτνίασης $N(v)$ σε συνεχόμενες θέσεις στον πίνακα target μπορεί να επιτύχει μεγαλύτερη τοπικότητα αναφορών στη μνήμη κατά την εκτέλεση ενός αλγόριθμου.

4.2.1 Υλοποίηση σε Java

Θα περιγράψουμε την υλοποίηση μιας δομής γραφήματος πρώτα με τον πίνακα γειτνίασης και, στη συνέχεια, με λίστα γειτνίασης.

Στην παρακάτω υλοποίηση, ορίζουμε μια κλάση Graph η οποία αναπαριστά ένα μη-κατευθυνόμενο γράφημα με τον πίνακα γειτνίασης του. Για το σκοπό αυτό, χρησιμοποιούμε ένα $(n+1) \times (n+1)$ boolean πίνακα A, όπου n είναι το πλήθος των κόμβων του γραφήματος. Χρησιμοποιούμε μια παραπάνω γραμμή και στήλη στον πίνακα A γιατί υποθέτουμε ότι οι κόμβοι είναι αριθμημένοι από 1 έως n, ενώ οι πίνακες στη Java ξεκινούν από τη θέση 0.

Υποθέτουμε, επίσης, ότι το πλήθος των κόμβων δεν μεταβάλλεται, γι' αυτό και αποθηκεύεται σε μια final η μεταβλητή.

Μετά την κατασκευή του πίνακα A, η εισαγωγή μιας (μη-κατευθυνόμενης) ακμής (i,j) γίνεται θέτοντας true τις αντίστοιχες θέσεις του πίνακα A, δηλαδή τις A[i][j] και A[j][i].

```
/* μη-κατευθυνόμενο γράφημα: υλοποίηση με πίνακα γειτνίασης */
public class Graph
{
    private final int n;    // πλήθος κόμβων
    private int m;          // πλήθος ακμών
    private boolean A[][];  // πίνακας γειτνίασης

    /* κατασκευή κενού πίνακα γειτνίασης */
    public Graph(int n)
    {
        this.n = n;
        this.m = 0;
        A = new boolean[n+1][n+1];
    }

    /* εισαγωγή ακμής (i,j) */
    public void addEdge(int i, int j)
    {
        A[i][j] = A[j][i] = true;
        m++;
    }

    /* τυπώνει τον πίνακα γειτνίασης */
    public void printGraph()
    {
        System.out.println("Number of vertices = " + n + ", number of edges = " + m);
        for (int i=1; i<=n; i++)
        {
            for (int j=1; j<=n; j++)
            {
                if (A[i][j]) System.out.print("1 ");
                else  System.out.print("0 ");
            }

            System.out.println("");
        }
    }

    public static void main(String args[])
    {
        // κατασκευάζει και τυπώνει το μη-κατευθυνόμενο γράφημα της Εικόνας 4.1
        int n = 5;
        Graph G = new Graph(n);

        G.addEdge(1,2);
        G.addEdge(1,3);
        G.addEdge(1,4);
        G.addEdge(2,3);
        G.addEdge(2,5);
        G.addEdge(3,4);
        G.addEdge(4,5);

        G.printGraph();
    }
}
```

Τώρα περιγράφουμε την αντίστοιχη υλοποίηση με λίστα γειτνίασης. Η λίστα γειτνίασης του κάθε κόμβου του γραφήματος αποθηκεύεται σε μια απλά συνδεδεμένη λίστα. Κάθε κόμβος της συνδεδεμένης λίστας αποθηκεύει τον αριθμό ενός κόμβου του γραφήματος, καθώς και μια αναφορά στην επόμενο κόμβο της λίστας. Η αρχή της κάθε λίστας αποθηκεύεται σε ένα πίνακα αναφορών A μεγέθους n+1, όπου n είναι το πλήθος των κόμβων του γραφήματος. Όπως και στην υλοποίηση με πίνακα γειτνίασης, χρησιμοποιούμε μια παραπάνω θέση στον πίνακα A γιατί υποθέτουμε ότι οι κόμβοι είναι αριθμημένοι από 1 έως n και, επομένως, δεν χρησιμοποιούμε τη θέση A[0].

Μετά την κατασκευή του πίνακα A, η εισαγωγή μιας (μη-κατευθυνόμενης) ακμής (i,j) γίνεται εισάγοντας την τιμή j στη λίστα γειτνίασης του κόμβου i και, αντίστοιχα, εισάγοντας την τιμή i στη λίστα γειτνίασης του κόμβου j. Οι εισαγωγές αυτές γίνονται στην αρχή της κάθε λίστας, και έτσι εκτελούνται σε σταθερό χρόνο.

```
/* μη-κατευθυνόμενο γράφημα: υλοποίηση με λίστα γειτνίασης */
public class Graph
{
    private final int n;  // πλήθος κόμβων
    private int m;        // πλήθος ακμών
    Node A[];             // πίνακας αναφορών σε λίστες γειτνίασης

    /* κόμβος συνδεδεμένης λίστας */
    static class Node {
        int v;      // τιμή = αριθμός κόμβου του γραφήματος
        Node next;  // αναφορά στον επόμενο

        // κατασκευή νέου κόμβου συνδεδεμένης λίστας
        // με τιμή v και επόμενο κόμβο t
        Node(int v, Node t) {
            this.v = v;
            next = t;
        }
    }
    /* κατασκευή κενής λίστας γειτνίασης */
    public Graph(int n)
    {
        this.n = n;
        this.m = 0;
        A = new Node[n+1];
    }

    /* εισαγωγή ακμής (i,j) */
    public void addEdge(int i, int j)
    {
        A[i] = new Node(j, A[i]); // εισαγωγή στην αρχή της λίστας γειτνίασης του i
        A[j] = new Node(i, A[j]); // εισαγωγή στην αρχή της λίστας γειτνίασης του j
        m++;
    }

    /* τυπώνει τη λίστα γειτνίασης */
    public void printGraph()
    {
        System.out.println("Number of vertices = " + n + ", number of edges = " + m);
        for (int i=1; i<=n; i++)
        {
            System.out.print(i + " :");
            for (Node t = A[i]; t!=null;  t=t.next) // λίστα γειτνίασης του κόμβου i
            {
                System.out.print(" " + t.v);
            }
```

```
            System.out.println("");
        }
    }


    public static void main(String args[])
    {
        // κατασκευάζει και τυπώνει το μη-κατευθυνόμενο γράφημα της Εικόνας 4.1
        int n = 5;
        Graph2 G = new Graph2(n);

        G.addEdge(1,2);
        G.addEdge(1,3);
        G.addEdge(1,4);
        G.addEdge(2,3);
        G.addEdge(2,5);
        G.addEdge(3,4);
        G.addEdge(4,5);

        G.printGraph();
    }

}
```

4.3 Διερεύνηση γραφήματος

Η διερεύνηση (ή διάσχιση) ενός γραφήματος G αποτελεί βασικό συστατικό στοιχείο πολλών αλγόριθμων επεξεργασίας γραφημάτων. Με τον όρο *διερεύνηση του* G αναφερόμαστε στη συστηματική αναζήτηση στο γράφημα G, με την οποία επισκεπτόμαστε όλους τους κόμβους του και εξετάζουμε όλες τις ακμές του. Ένας αλγόριθμος διερεύνησης του G χρησιμοποιεί μια από τις δομές αναπαράστασης γραφημάτων που είδαμε στην Ενότητα 4.2, με τη βοήθεια της οποίας μπορούμε να μεταβούμε από έναν οποιοδήποτε κόμβο v σε ένα γειτονικό του κόμβο u ακολουθώντας την ακμή (v, u). Σε ένα ενδιάμεσο βήμα μίας διαδικασίας διερεύνησης ενός γραφήματος υπάρχουν κόμβοι τους οποίους έχουμε ανακαλύψει, κόμβοι τους οποίους έχουμε επισκεφτεί και κόμβοι τους οποίους δεν έχουμε ανακαλύψει ακόμα. Επιπλέον, η διερεύνηση κάθε χρονική στιγμή βρίσκεται σε ένα τρέχοντα κόμβο, τον κόμβο τον οποίο επισκέπτεται τη συγκεκριμένη χρονική στιγμή. Κάθε κόμβος v μπορεί να βρίσκεται σε μία από τις ακόλουθες καταστάσεις:

α. να μην τον έχουμε ανακαλύψει,
β. να τον έχουμε ανακαλύψει, αλλά να μην τον έχουμε επισκεφτεί ακόμα,
γ. να τον έχουμε επισκεφτεί.

Αυτό σημαίνει ότι η διερεύνηση μπορεί να επισκεφτεί έναν κόμβο, μόνο εφόσον τον έχει προηγουμένως ανακαλύψει. Από τις παραπάνω καταστάσεις, συνήθως μας αρκεί να γνωρίζουμε αν έχουμε ανακαλύψει ένα κόμβο ή όχι. Μπορούμε να αποθηκεύσουμε αυτήν την πληροφορία για κάθε κόμβο σε ένα boolean πίνακα mark, όπου σημειώνουμε τους κόμβους που έχουμε ανακαλύψει. Συγκεκριμένα, για κάθε κόμβο v που έχουμε ανακαλύψει θέτουμε $\text{mark}[v] = \text{true}$, διαφορετικά, αν δεν έχουμε ανακαλύψει τον v, έχουμε $\text{mark}[v] = \text{false}$. Αρχικά, ξεκινώντας τη διαδικασία διερεύνησης, θέτουμε $\text{mark}[s]$=true και $\text{mark}[v]$=false για κάθε κόμβο v διαφορετικό του s.

Η διαδικασία διερεύνησης ξεκινά από ένα αφετηριακό κόμβο s, ο οποίος είναι ο πρώτος κόμβος που ανακαλύπτουμε και επισκεπτόμαστε. Κατά τη διάρκεια της διερεύνησης, βρισκόμαστε σε ένα τρέχοντα κόμβο v, από τον οποίο εξετάζουμε τους γειτονικούς του κόμβους. Αυτό γίνεται εξετάζοντας τις ακμές (v, w) που ξεκινούν από τον v. Κάθε ακμή εξετάζεται μόνο μια φορά. Όταν εξετάζουμε την ακμή (v, w), ελέγχουμε την κατάσταση του

w. Αν mark[w]=true, τότε έχουμε ανακαλύψει τον w σε προηγούμενο βήμα, οπότε συνεχίζουμε την εξέταση μιας άλλης ακμής από τον v. Αντίθετα, αν mark[w]=false, τότε έχουμε μόλις ανακαλύψει τον w και θέτουμε mark[w]=true. Σε αυτήν την περίπτωση, ανάλογα με το είδος της διερεύνησης, μπορούμε είτε να συνεχίσουμε με την εξέταση των ακμών του v ή να μεταβούμε σε κάποιο κόμβο που έχουμε ανακαλύψει.

Σειρά Ανακάλυψης Κόμβων. Σε πολλούς αλγόριθμους που βασίζονται σε κάποια διαδικασία διερεύνησης ενός γραφήματος, χρειάζεται να γνωρίζουμε τη σειρά ανακάλυψης των κόμβων. Όπως αναφέραμε παραπάνω, η διαδικασία διερεύνησης ανακαλύπτει ένα κόμβο w, όταν εξετάζει μια ακμή (v, w), όπου v ο τρέχων κόμβος ο οποίος είναι γειτονικός του w, και ισχύει mark[w]=false. Καταγράφουμε τη σειρά πρώτης επίσκεψης σε ένα πίνακα d, έτσι ώστε η μεταβλητή $d[v]$ να δίνει τη σειρά ανακάλυψης του κόμβου v. Για να υπολογίσουμε αυτή τη σειρά, διατηρούμε μια μεταβλητή t, την οποία αρχικοποιούμε με την τιμή μηδέν, και την αυξάνουμε κατά ένα, μόλις ανακαλύψουμε ένα νέο κόμβο v. Εκείνη τη στιγμή θέτουμε $d[v] = t$. Έτσι, για τον αφετηριακό κόμβο s θα θέσουμε $d[s] = 1$ και οι υπόλοιποι κόμβοι $v \neq s$ θα λάβουν αρίθμηση $d[v]$ στο διάστημα $[2, n]$.

Γονέας ενός Κόμβου. Μια ακόμα πληροφορία που μας είναι πολλές φορές χρήσιμη στις εφαρμογές είναι να γνωρίζουμε, για κάθε κόμβο w, τον κόμβο v από τον οποίο ανακαλύφθηκε ο w. Αυτό σημαίνει ότι είδαμε τον w για πρώτη φορά κατά την εξέταση της ακμής (v, w). Σε αυτήν την περίπτωση λέμε ότι ο v είναι ο *γονέας* του w στη διερεύνηση. Όπως θα δούμε στη συνέχεια, μετά την ολοκλήρωση μιας διερεύνησης σε ένα συνεκτικό γράφημα G με n κόμβους, οι ακμές (v, w), όπου ο v είναι ο γονέας του w, ορίζουν ένα άκυκλο και συνεκτικό υπογράφημα του G, το οποίο ονομάζουμε *δένδρο της διερεύνησης*. Αν το G δεν είναι συνεκτικό, τότε μπορούμε να εκτελέσουμε μια διαδικασία διερεύνησης για κάθε συνεκτική συνιστώσα του G, οπότε λαμβάνουμε μια συλλογή από δένδρα διερεύνησης, ένα για κάθε συνιστώσα. Ονομάζουμε αυτή τη συλλογή, *δάσος της διερεύνησης*.

Κριτήρια Διερεύνησης. Ένας αλγόριθμος διερεύνησης χαρακτηρίζεται από το κριτήριο με το οποίο επιλέγει τον επόμενο κόμβο που θα επισκεφτεί. Θα εστιάσουμε την προσοχή μας σε δύο βασικά κριτήρια επιλογής που αποδεικνύονται ιδιαίτερα χρήσιμα και καθορίζουν τις δύο πιο γνωστές διαδικασίες διερεύνησης, την *κατά-βάθος* ή *καθοδική διερεύνηση* (depth-first search - DFS) και την *κατά-πλάτος ή οριζόντια διερεύνηση* (breadth-first search - BFS), που θα παρουσιάσουμε στη συνέχεια.

Κατά-βάθος Διερεύνηση (DFS). Στην κατά-βάθος διερεύνηση ενός γραφήματος επισκεπτόμαστε κάθε φορά τον κόμβο που μόλις έχουμε ανακαλύψει. Η διερεύνηση ξεκινά από τον αφετηριακό κόμβο s, απ' όπου μεταβαίνουμε στον πρώτο γείτονα του s που βρίσκουμε με τη βοήθεια της δομής δεδομένων με την οποία αναπαριστούμε το γράφημα. Γενικά, όταν βρισκόμαστε στον τρέχοντα κόμβο v, επιλέγουμε να επισκεφτούμε τον επόμενο γείτονα w του v που δεν έχουμε ανακαλύψει ακόμα, οπότε ο v γίνεται γονέας του w στη διερεύνηση. Αν έχουμε εξετάσει όλες τις ακμές που ξεκινούν από τον v, οπότε όλοι οι γείτονες του v έχουν ανακαλυφθεί, τότε *εγκαταλείπουμε* τον v. Σε αυτήν την περίπτωση, δεν πρόκειται να επισκεφτούμε ξανά τον v στο υπόλοιπο της διερεύνησης. Με την εγκατάλειψη του v, αν $v \neq s$, τότε μεταβαίνουμε στο γονέα του v, ο οποίος γίνεται ο τρέχων κόμβος της αναζήτησης. Διαφορετικά, αν $v = s$, τότε τερματίζουμε τη διερεύνηση από τον s.

Σημειώνουμε ότι, εάν το γράφημα G είναι μη-κατευθυνόμενο και συνεκτικό, τότε κάθε ακμή (v, w) την εξετάζουμε ακριβώς μία φορά και στις δύο κατευθύνσεις και επισκεπτόμαστε κάθε κόμβο τουλάχιστον μία φορά. Εάν το γράφημα δεν είναι συνεκτικό, τότε η διερεύνηση πραγματοποιείται σε κάθε μία συνεκτική συνιστώσα του G. Μπορούμε να συνεχίσουμε την ίδια διαδικασία και στις επόμενες συνιστώσες, επιλέγοντας ως αφετηρία ένα κόμβο που δεν έχουμε ανακαλύψει ακόμα.

Η διαδικασία της κατά-βάθος διερεύνησης ενός γραφήματος $G = (V, E)$ περιγράφεται στον παρακάτω αλγόριθμο. Ο αλγόριθμος δέχεται ως είσοδο τη λίστα γειτνίασης του G και υπολογίζει το dfs-δάσος μαζί με την αρίθμηση των κόμβων από 1 έως n σύμφωνα με την σειρά ανακάλυψης τους. Στην περιγραφή που δίνουμε δεν χρησιμοποιούμε τον πίνακα mark, καθώς μπορούμε να ελέγξουμε αν έχουμε ανακαλύψει ένα κόμβο v ελέγχοντας αν έχει λάβει τιμή $d[v] > 0$. Προσέξτε ότι ο αλγόριθμος χρησιμοποιεί την αναδρομική διαδικασία DFS(v). Με αυτόν τον τρόπο, όταν εξετάζουμε τις ακμές (v, w) του v, μπορούμε να μεταβούμε άμεσα σε ένα κόμβο w που μόλις ανακαλύψαμε με την κλήση DFS(w). Επισκεπτόμαστε ξανά τον v, μόλις επιστρέψει η αναδρομική κλήση DFS(w), οπότε μόλις έχουμε εγκαταλείψει τον w. Με την επιστροφή της αναδρομικής κλήσης DFS(w), ο έλεγχος επιστρέφει στην DFS(v), η οποία συνεχίζει την εξέταση των ακμών του v από το σημείο που διακόπηκε.

Αλγόριθμος κατά-βάθος διερεύνησης γραφήματος (DFS)

$DFS(v)$

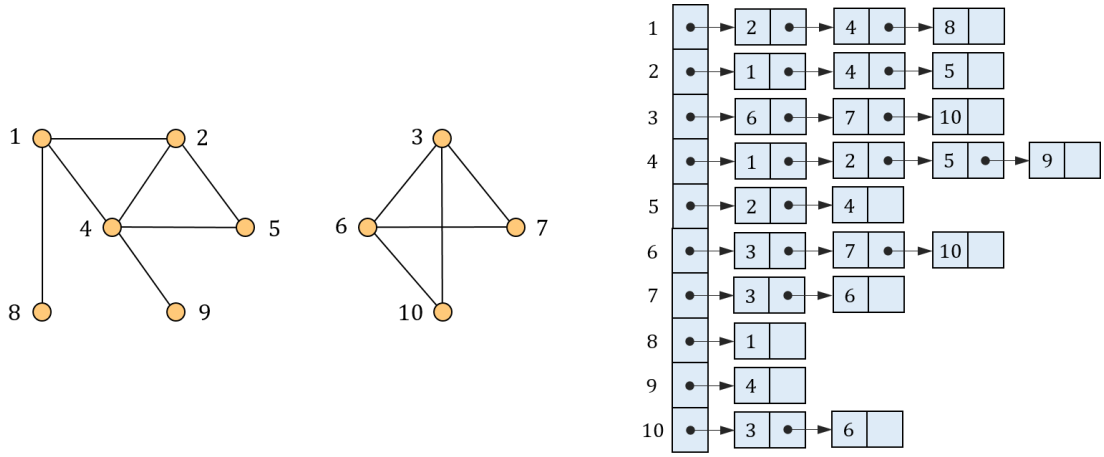
1. θέσε $t = t + 1$ και $d[v] = t$.
2. για κάθε κόμβο $w \in N(v)$
3. αν $d[w] = 0$, τότε
4. θέσε $p[w] = v$
5. εκτέλεσε αναδρομικά DFS(w)

τέλος DFS(v)

αρχή διερεύνησης

6. αρχικοποίηση: $t = 0$
7. για κόμβο $v \in V$ θέσε $d[v] = 0$ και $p[v] = 0$
8. για κάθε κόμβο $s \in V$
9. αν $d[s] = 0$, τότε
10. εκτέλεσε DFS(s)

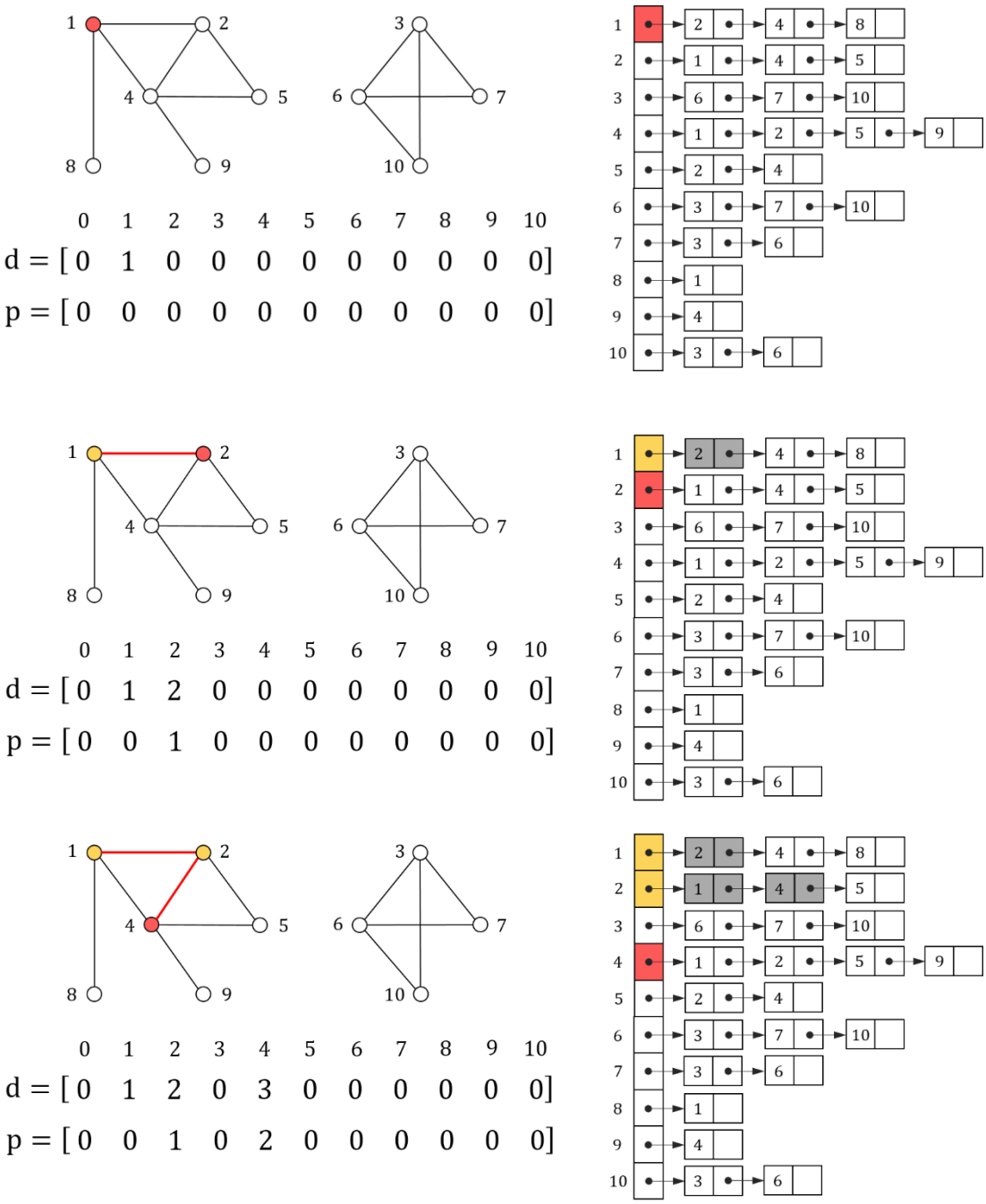
τέλος διερεύνησης



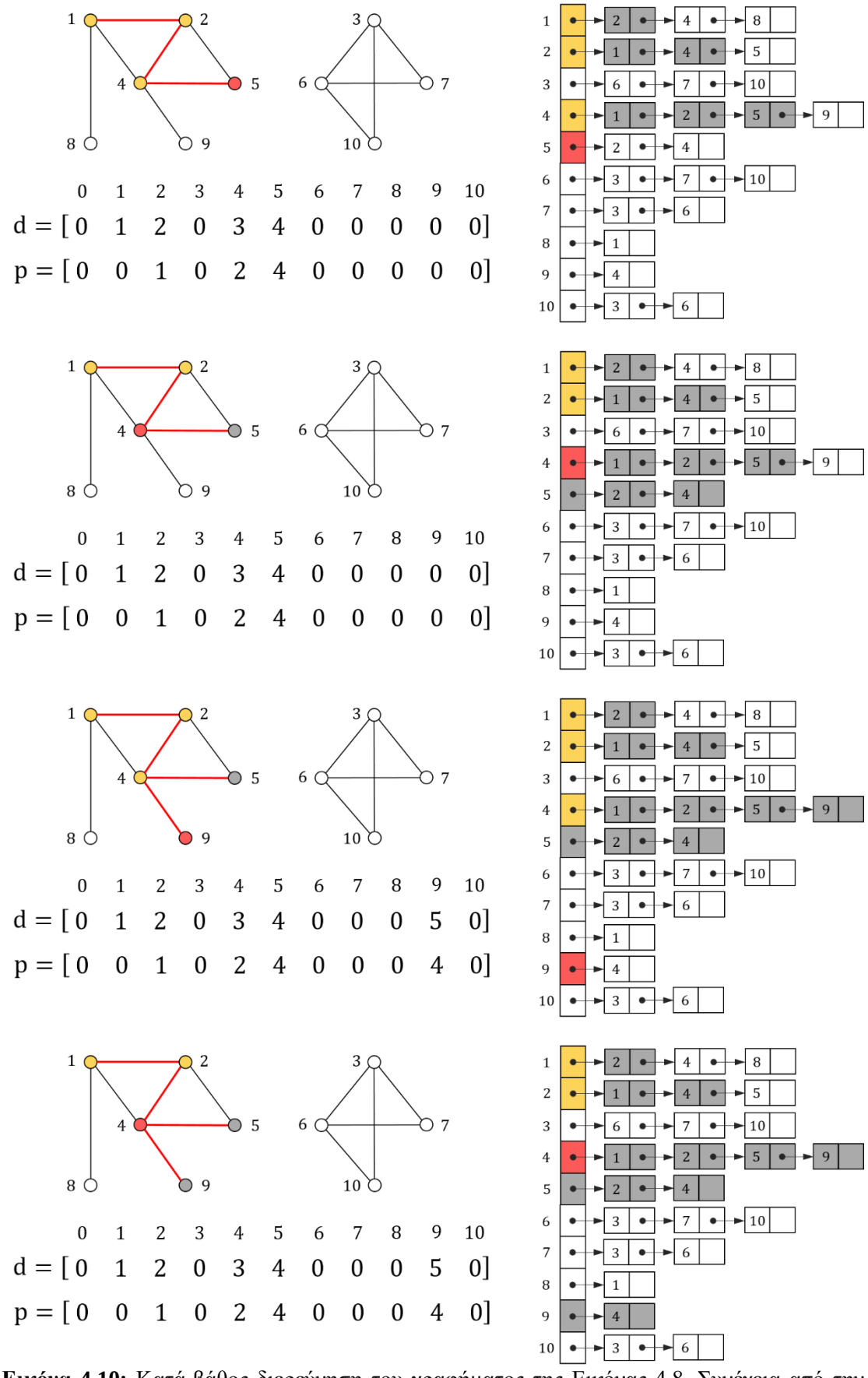
Εικόνα 4.8: Ένα μη-κατευθυνόμενο γράφημα με 2 συνεκτικές συνιστώσες και η αναπαράσταση του με λίστα γειτνίασης.

Όπως είδαμε και στη γενική περίπτωση μίας διαδικασίας διερεύνησης, έτσι και στην κατά-βάθος διερεύνηση ενός γραφήματος $G = (V, E)$ το σύνολο των ακμών του γραφήματος διαμερίζεται σε δύο σύνολα, έστω T και B, όπου το T ορίζει ένα συνδετικό δάσος του G, το οποίο περιέχει ένα δένδρο για κάθε συνεκτική συνιστώσα του. Η ακμή (v, w) προστίθεται στο T, αν ανακαλύψουμε τον κόμβο w ενόσω εξετάζουμε τις ακμές από τον v, δηλαδή όταν ο v

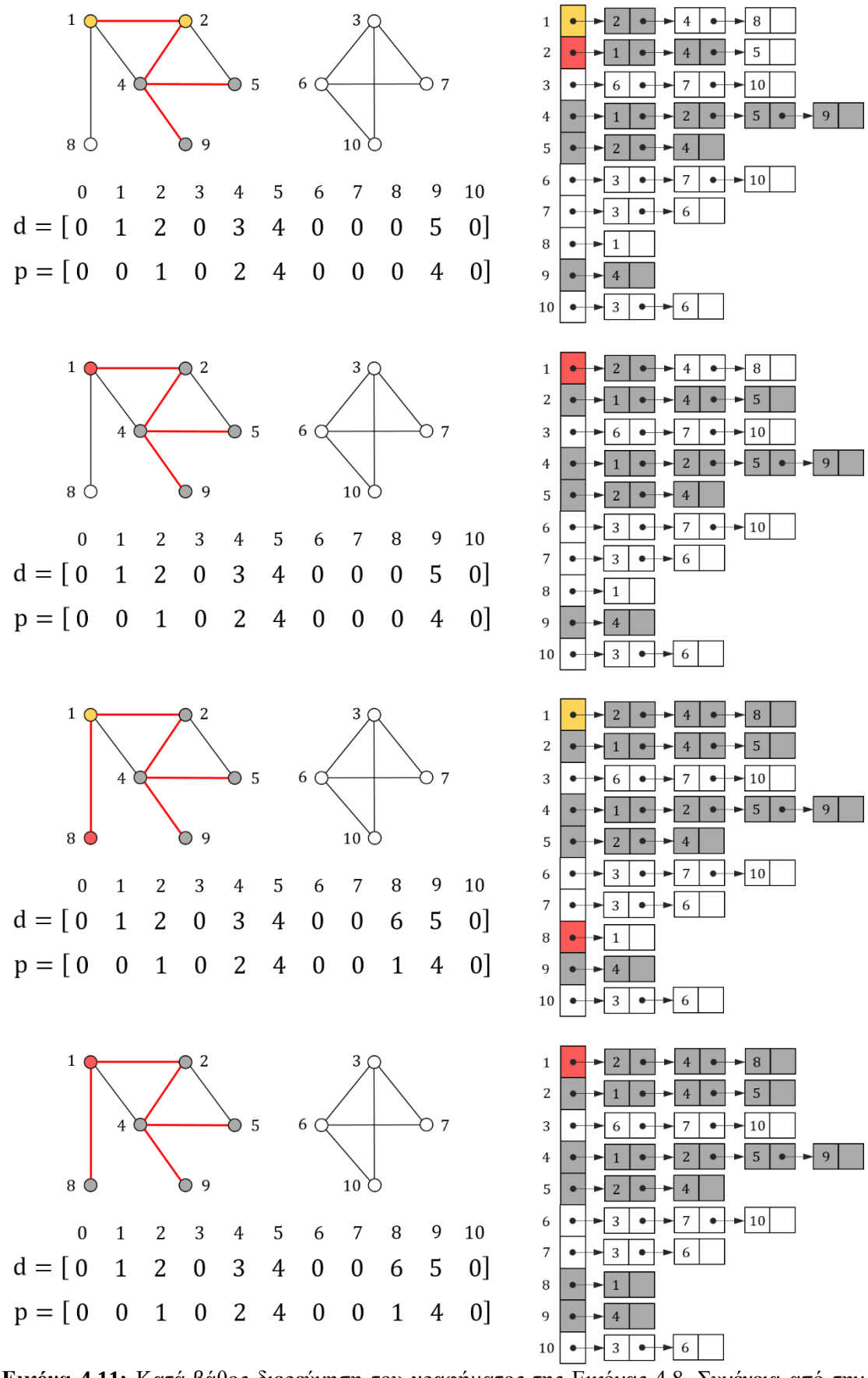
γίνει γονέας του w. Οι ακμές του συνόλου T ονομάζονται *δενδρικές-ακμές*, ενώ οι υπόλοιπες του συνόλου B *οπίσθο-ακμές*. Εάν το γράφημα G είναι συνεκτικό, τότε το T είναι δένδρο και ονομάζεται dfs-δένδρο του G. Θεωρούμε κάθε dfs-δένδρο T_i ενός dfs-δάσους ότι έχει ρίζα τον κόμβο εκκίνησης s_i της κατά-βάθος διερεύνησης της συνεκτικής συνιστώσας του δένδρου T_i. Στις παρακάτω Εικόνες 4.9 – 4.12 δίνουμε ένα παράδειγμα εκτέλεσης της κατά-βάθους διερεύνησης ενός γραφήματος. Σημειώνουμε με κόκκινο χρώμα τον τρέχοντα κόμβο, καθώς και τις ακμές του dfs-δένδρου. Σημειώνουμε με κίτρινο χρώμα κάθε άλλο κόμβο (εκτός από τον τρέχοντα κόμβο) που έχουμε επισκεφτεί, αλλά δεν έχουμε εγκαταλείψει ακόμα. Επίσης, σημειώνουμε με γκρίζο χρώμα τους κόμβους που έχουμε εγκαταλείψει, καθώς και τις ακμές στη λίστα γειτνίασης, τις οποίες έχουμε εξετάσει.



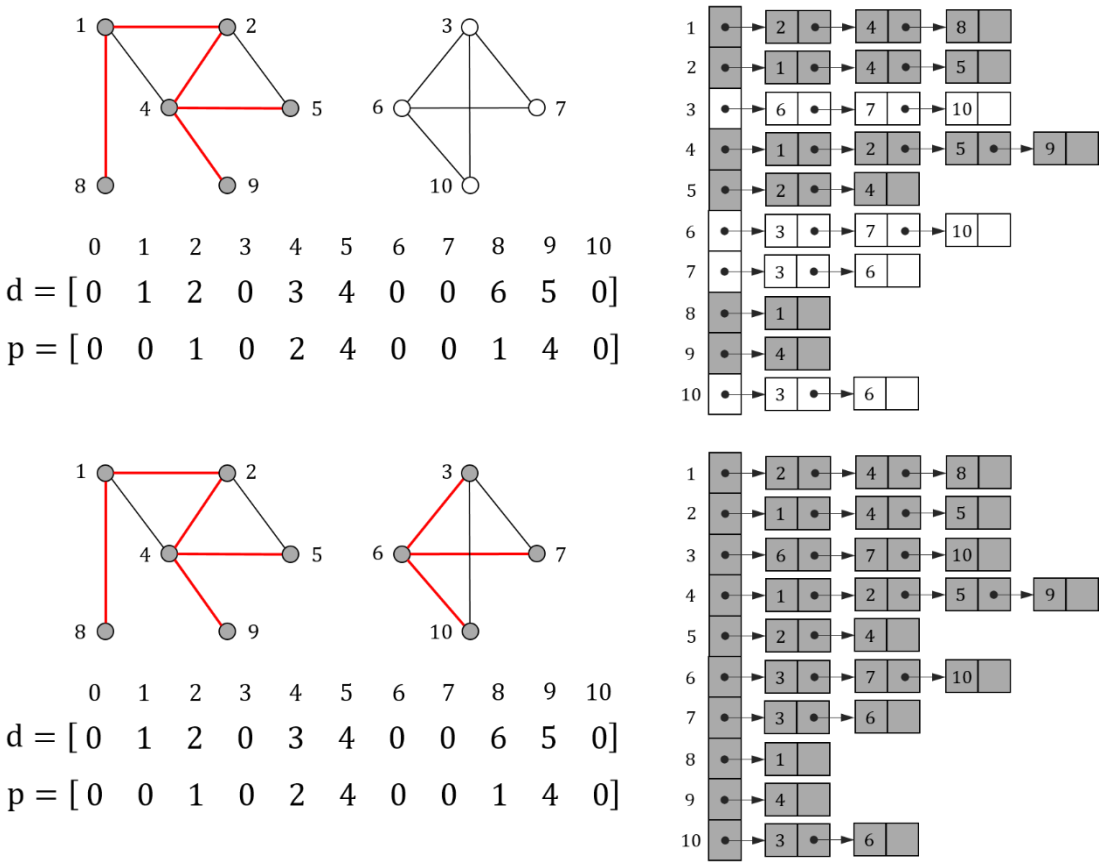
Εικόνα 4.9: Κατά-βάθος διερεύνηση του γραφήματος της Εικόνας 4.8.



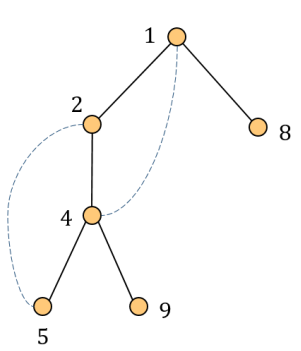
Εικόνα 4.10: Κατά-βάθος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.11.



Εικόνα 4.11: Κατά-βάθος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.10.



Εικόνα 4.12: Κατά-βάθος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.11.



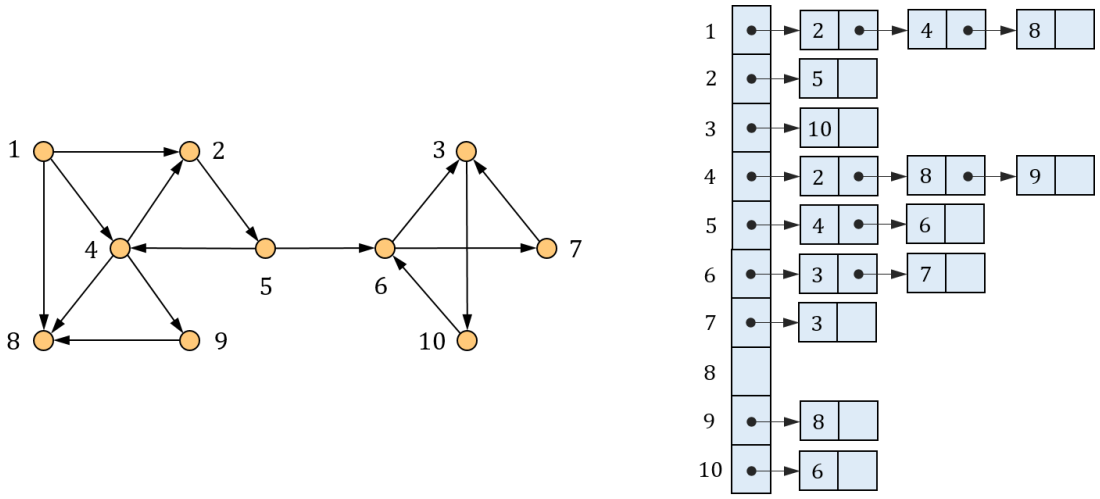
Εικόνα 4.13: Το δάσος της κατά-βάθος διερεύνησης του γραφήματος της Εικόνα 4.. Οι ακμές που δεν ανήκουν στο δάσος είναι οπισθο-ακμές και σημειώνονται με διακεκομμένες γραμμές.

Γενικά, ένα γράφημα μπορεί να έχει περισσότερα από ένα διαφορετικά dfs-δένδρα ή dfs-δάση. Τυπικά, το dfs-δένδρο ή dfs-δάσος που παράγεται από την κατά-βάθος διερεύνηση εξαρτάται από τη σειρά με την οποία εμφανίζονται οι κόμβοι στις λίστες γειτνίασης. Ωστόσο, ένα dfs-δένδρο T έχει μερικές σημαντικές και χρήσιμες ιδιότητες, όπως οι εξής:

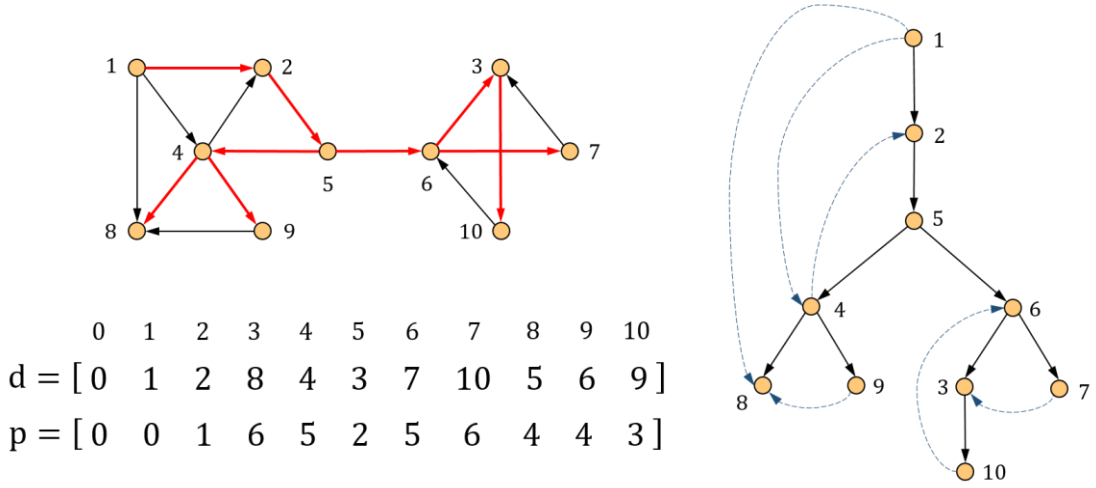
1. Εάν ο κόμβος v είναι πρόγονος του κόμβου w στο dfs-δένδρο T, τότε $d[v] < d[w]$.

2. Εάν το γράφημα G είναι μη-κατευθυνόμενο, τότε για κάθε ακμή (v,u) του G, είτε είναι δενδρική ακμή είτε οπισθο-ακμή, ισχύει ότι ένας από τους κόμβους της ακμής είναι πρόγονος του άλλου.

Η κατά-βάθος διερεύνηση εκτελείται με ακριβώς τον ίδιο τρόπο και σε ένα κατευθυνόμενο γράφημα $G = (V, E)$. Η μόνη διαφορά είναι ότι μια ακμη (u, v) του G δεν είναι απαραίτητα δενδρική-ακμή ή οπισθο-ακμή. Συγκεκριμένα, αν ο u είναι πρόγονος του v στο dfs-δένδρο (ή στο dfs-δάσος) τότε η (u, v) είναι μια *εμπρόσθια-ακμή* (forward-edge). Αν οι u και v δεν έχουν σχέση προγόνου-απογόνου τότε η (u, v) είναι μια *διασχίζουσα-ακμή* (cross-edge). Δείτε τις Εικόνες 4.14 και 4.15.



Εικόνα 4.14: Ένα κατευθυνόμενο γράφημα και η αναπαράσταση του με λίστα γειτνίασης.



Εικόνα 4.15: Το δάσος (δένδρο) της κατά-βάθος διερεύνησης του γραφήματος της Εικόνας 4.14. Οι ακμές που δεν ανήκουν στο δάσος σημειώνονται με διακεκομμένες γραμμές.

Κατά-πλάτος Διερεύνηση (BFS). Αντίθετα με την *κατά-βάθος διερεύνηση* (DFS), στην οποία επισκεπτόμαστε τον κόμβο που μόλις έχουμε ανακαλύψει, η κατά-πλάτος διερεύνηση αποθηκεύει τους κόμβους που ανακαλύπτει σε μία δομή δεδομένων την οποία ονομάζουμε ουρά και θα εξετάσουμε αναλυτικά στο Κεφάλαιο 5. Η ουρά Q διατηρεί τους κόμβους v του γραφήματος σε σειρά ανακάλυψης, δηλαδή σε αύξουσα σειρά ως προς τις τιμές $d[v]$. Αρχικά, η Q είναι κενή και εισάγουμε ένα αφετηριακό κόμβο s. Επαναληπτικά διαγράφουμε από την Q τον κόμβο v, ο οποίος έχει εισαχθεί πριν από τους υπόλοιπους στην Q, και εξετάζουμε τις ακμές του v. Για κάθε ακμή (v, w), αν ο w δεν έχει ανακαλυφθεί, τότε εισαγάγουμε στην Q. Η διαδικασία τερματίζεται, όταν η ουρά αδειάσει.

Όπως η *κατά-βάθος διερεύνηση*, έτσι και η *κατά-πλάτος* εκτελείται μία φορά για κάθε συνεκτική συνιστώσα του γραφήματος εισόδου. Ωστόσο, στην *κατά-πλάτος διερεύνηση* επισκεπτόμαστε κάθε κόμβο μόνο μία φορά, τη στιγμή που τον διαγράφουμε από την ουρά.

Η *κατά-πλάτος διερεύνηση* ενός μη-κατευθυνόμενου γραφήματος $G = (V, E)$ επίσης διαμερίζει τις ακμές του γραφήματος σε δύο σύνολα: τις *δενδρικές-ακμές* και τις *μη-δενδρικές ακμές*. Μία ακμή (v, w) επιλέγεται ως δενδρική ακμή όταν ο κόμβος w εισάγεται στην ουρά Q κατά την εξέταση της (v, w). Εάν το γράφημα G είναι συνεκτικό, τότε οι δενδρικές ακμές σχηματίζουν ένα συνδετικό δένδρο του G, το οποίο ονομάζουμε bfs-δένδρο του G. Διαφορετικά, όταν το G δεν είναι συνεκτικό, οι δενδρικές δομές σχηματίζουν ένα bfs-δένδρο για κάθε συνεκτική συνιστώσα του G.

Η διαδικασία της κατά-πλάτος διερεύνησης ενός γραφήματος $G = (V, E)$ περιγράφεται στον παρακάτω αλγόριθμο. Ο αλγόριθμος δέχεται ως είσοδο τη λίστα γειτνίασης του G και υπολογίζει το bfs-δάσος μαζί με την αρίθμηση των κόμβων από 1 έως n σύμφωνα με την σειρά ανακάλυψης τους. Όπως και στην περιγραφή της DFS, διαπιστώνουμε αν έχουμε ανακαλύψει ένα κόμβο v ελέγχοντας αν έχει λάβει τιμή $d[v] > 0$.

Αλγόριθμος κατά-πλάτος διερεύνησης γραφήματος (BFS)

BFS(s)

1. θέσε $t = t + 1$ και $d[s] = t$.
2. Q.εισαγωγή(s)
3. ενόσω η Q δεν είναι κενή
4. $v = Q$.εξαγωγή
5. για κάθε κόμβο $w \in N(v)$
6. αν $d[w] = 0$, τότε
7. θέσε $t = t + 1$, $d[s] = t$ και $p[w] = v$
8. Q.εισαγωγή(w)

τέλος BFS(s)

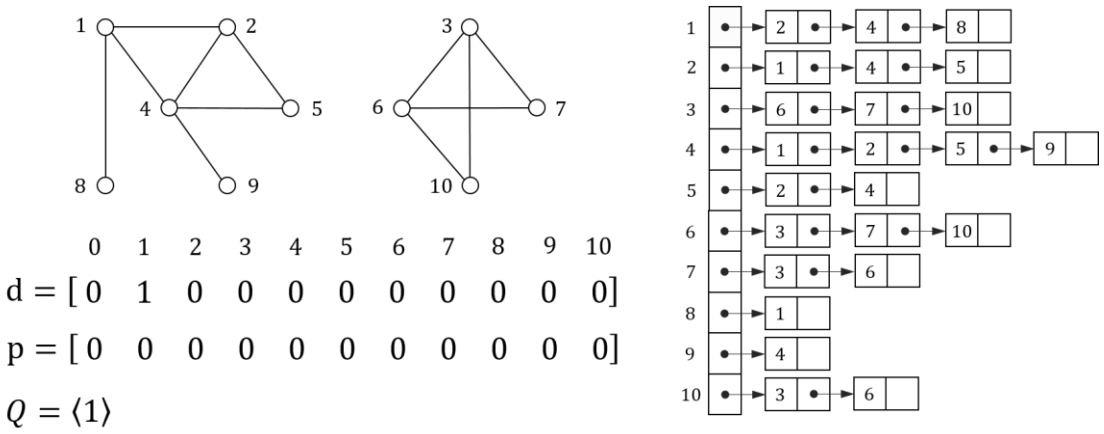
αρχή διερεύνησης

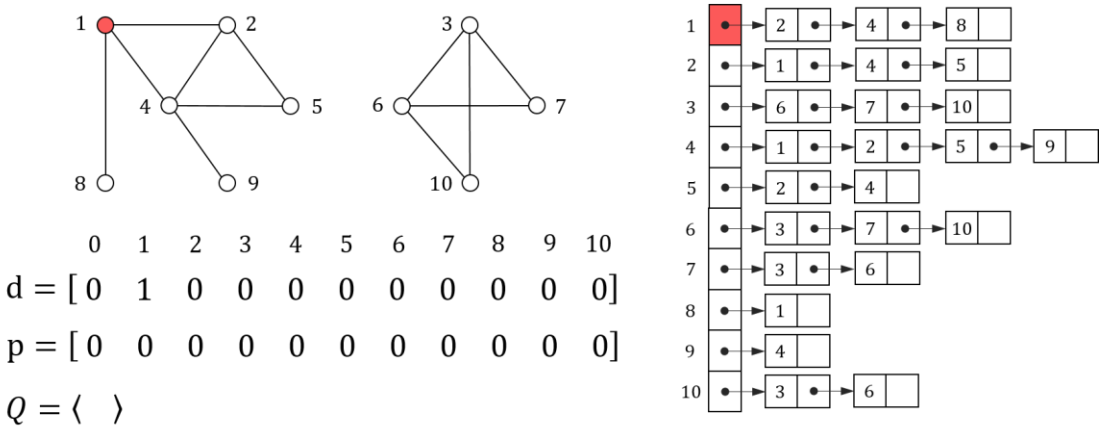
9. αρχικοποίηση: $t = 0$
10. για κόμβο $v \in V$ θέσε $d[v] = 0$ και $p[v] = 0$
11. για κάθε κόμβο $s \in V$
12. αν $d[s] = 0$, τότε
13. εκτέλεσε BFS(s)

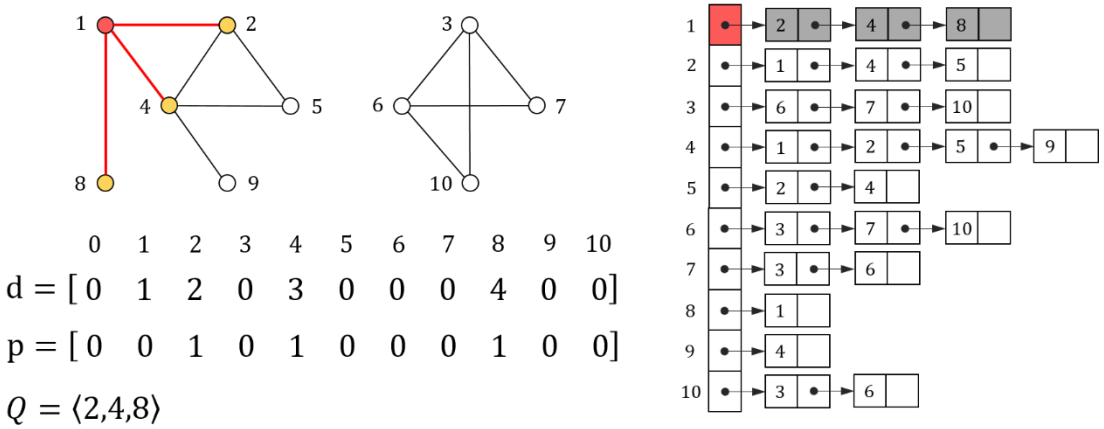
τέλος διερεύνησης

Στις παρακάτω Εικόνες 4.16 – 4.23 δίνουμε ένα παράδειγμα εκτέλεσης της κατά-πλάτους διερεύνησης ενός γραφήματος. Σημειώνουμε με κόκκινο χρώμα τον τρέχοντα κόμβο, καθώς και τις ακμές του bfs-δένδρου. Σημειώνουμε με κίτρινο χρώμα τους κόμβους που έχουμε

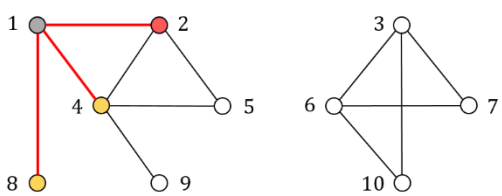
τοποθετήσει στην ουρά Q, αλλά δεν τους εξαγάγει ακόμα. Επίσης, σημειώνουμε με γκρίζο χρώμα τους κόμβους που έχουμε εξαγάγει από την Q, καθώς και τις ακμές της λίστας γειτνίασης, τις οποίες έχουμε εξετάσει.





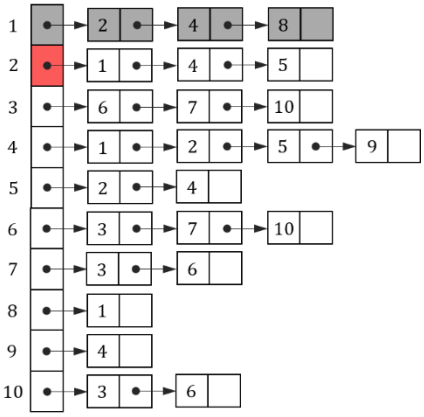


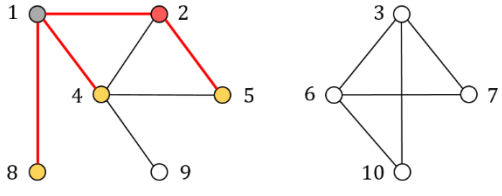
Εικόνα 4.16: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8.



	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	0	0	0	4	0	0
p =	0	0	1	0	1	0	0	0	1	0	0

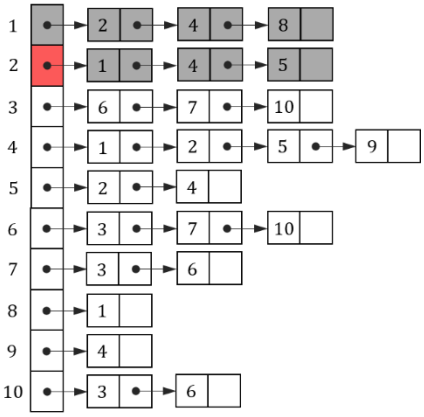
$Q = \langle 4,8 \rangle$

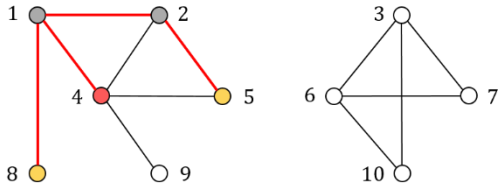




	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	5	0	0	4	0	0
p =	0	0	1	0	1	2	0	0	1	0	0

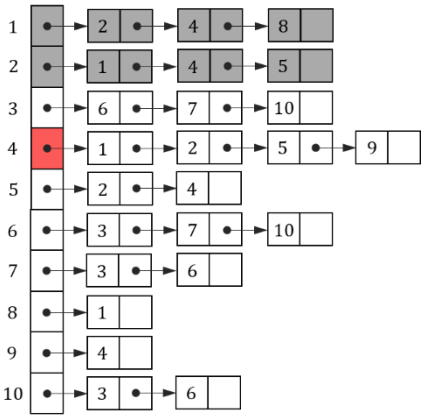
$Q = \langle 4,8,5 \rangle$



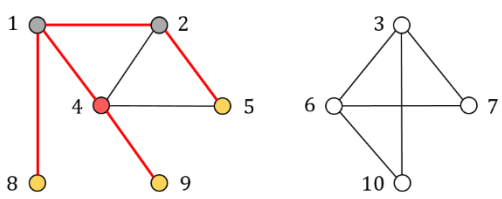


	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	5	0	0	4	0	0
p =	0	0	1	0	1	2	0	0	1	0	0

$Q = \langle 8,5 \rangle$

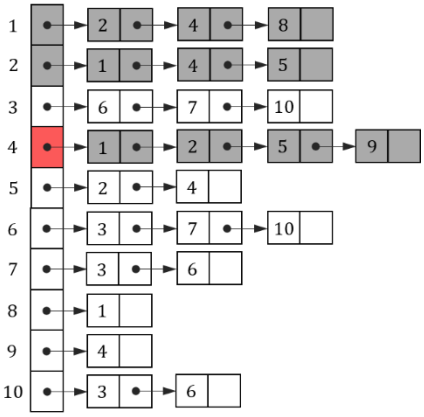


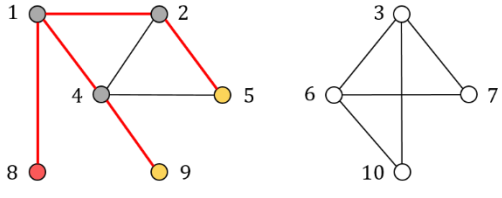
Εικόνα 4.17: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.16.



	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	5	0	0	4	6	0
p =	0	0	1	0	1	2	0	0	1	4	0

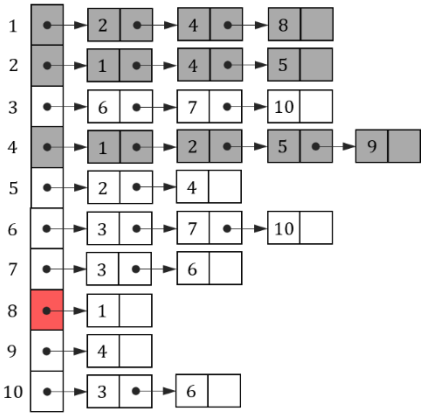
$Q = \langle 8,5,9 \rangle$

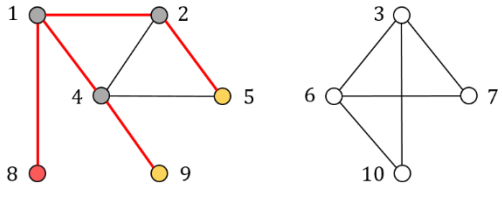




	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	5	0	0	4	6	0
p =	0	0	1	0	1	2	0	0	1	4	0

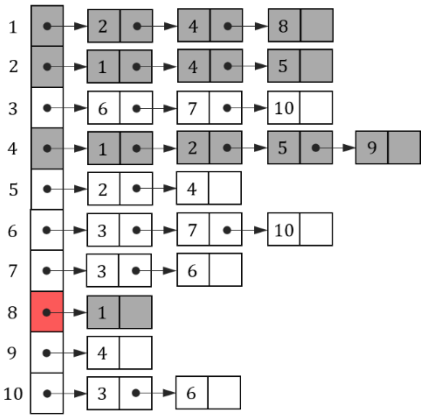
$Q = \langle 5,9 \rangle$



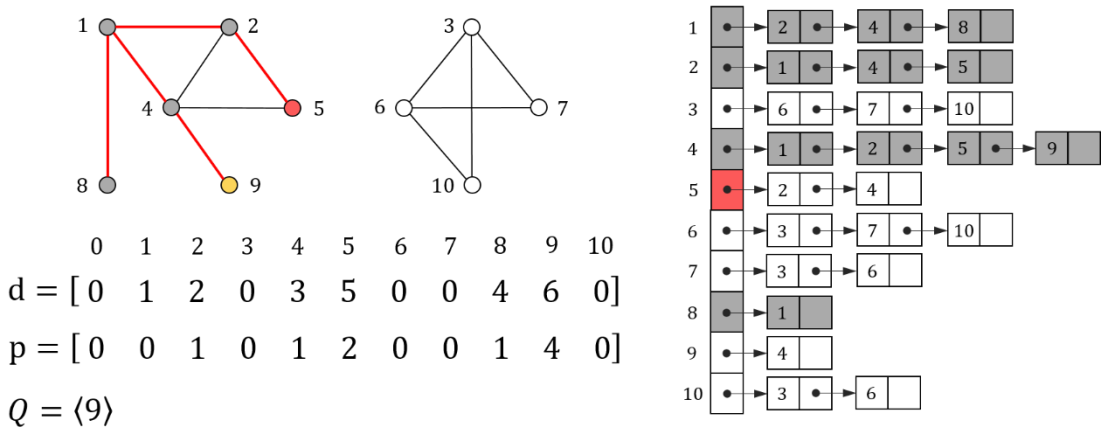


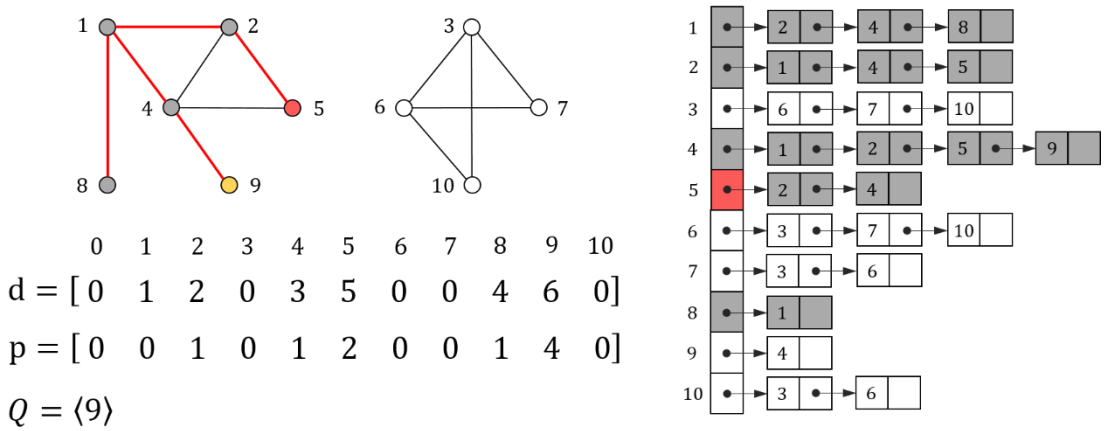
	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	5	0	0	4	6	0
p =	0	0	1	0	1	2	0	0	1	4	0

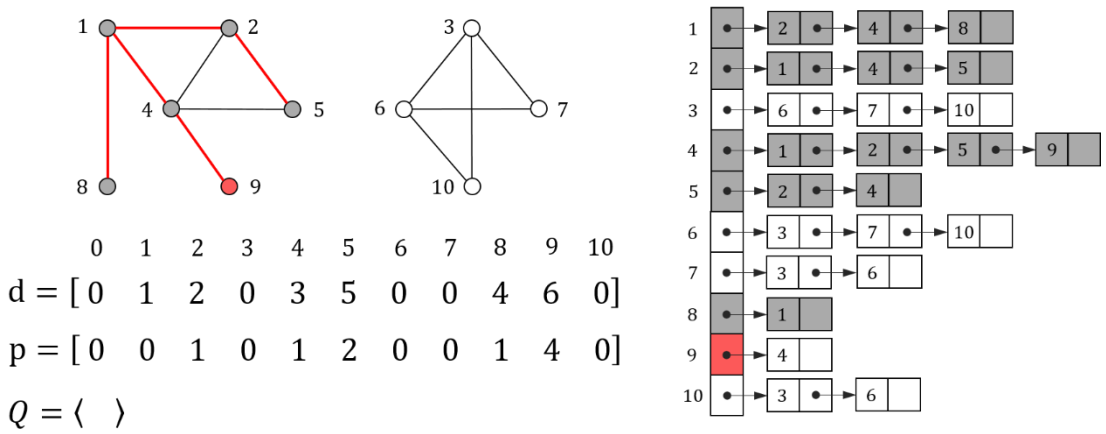
$Q = \langle 5,9 \rangle$



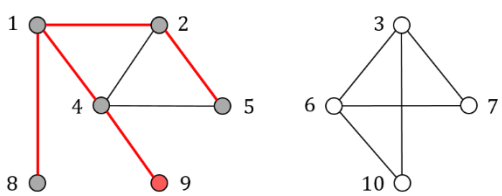
Εικόνα 4.18: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.17.





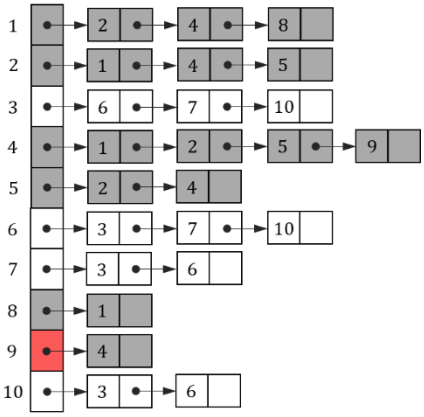


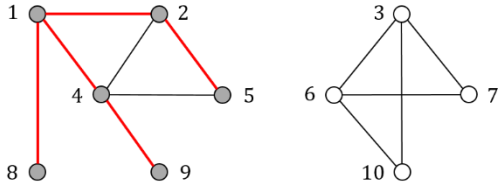
Εικόνα 4.19: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.18.



	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	0	3	5	0	0	4	6	0
p =	0	0	1	0	1	2	0	0	1	4	0

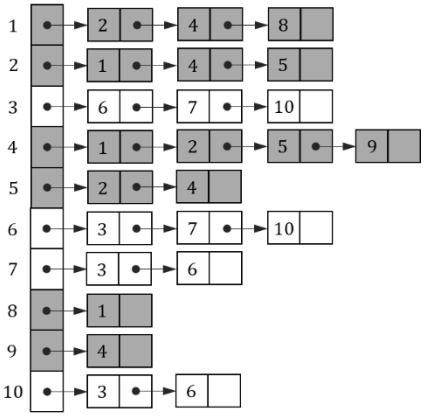
$Q = \langle \quad \rangle$

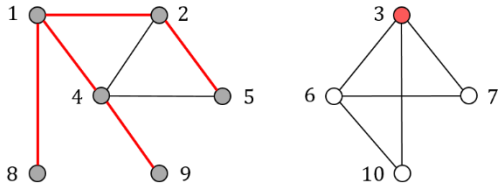




	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	7	3	5	0	0	4	6	0
p =	0	0	1	0	1	2	0	0	1	4	0

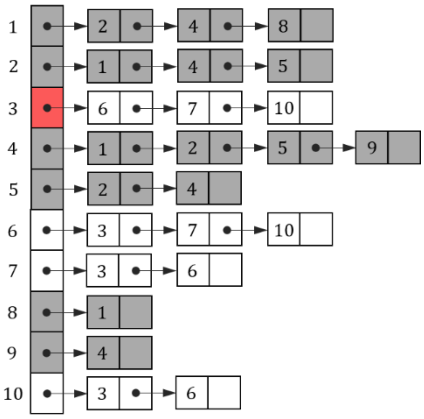
$Q = \langle 3 \rangle$



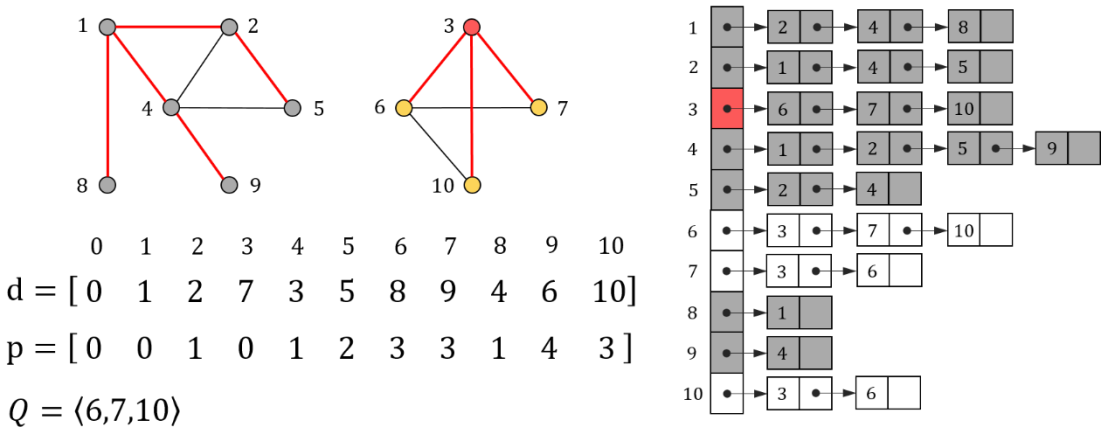


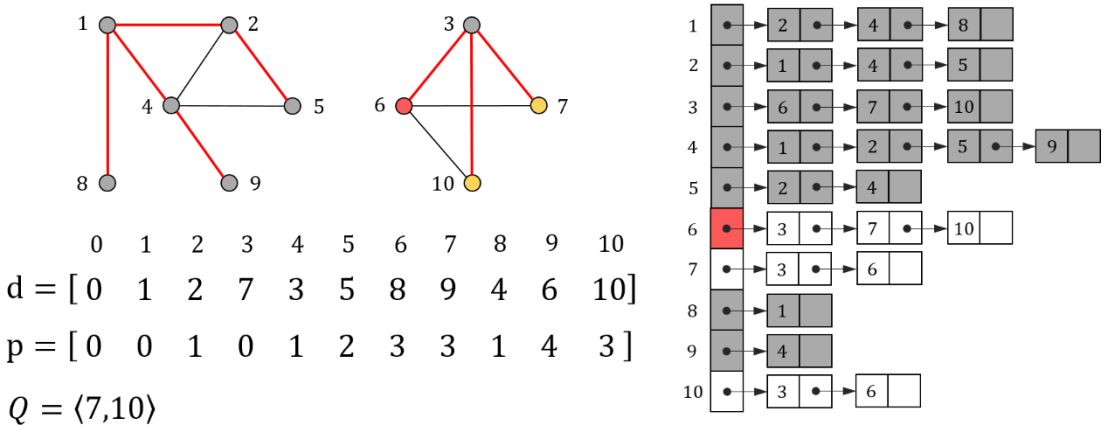
	0	1	2	3	4	5	6	7	8	9	10
d =	0	1	2	7	3	5	0	0	4	6	0
p =	0	0	1	0	1	2	0	0	1	4	0

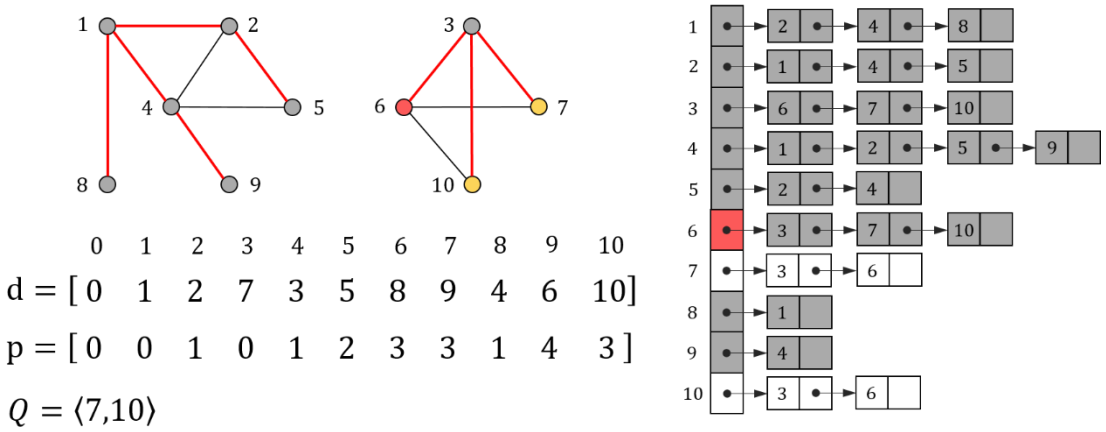
$Q = \langle \quad \rangle$



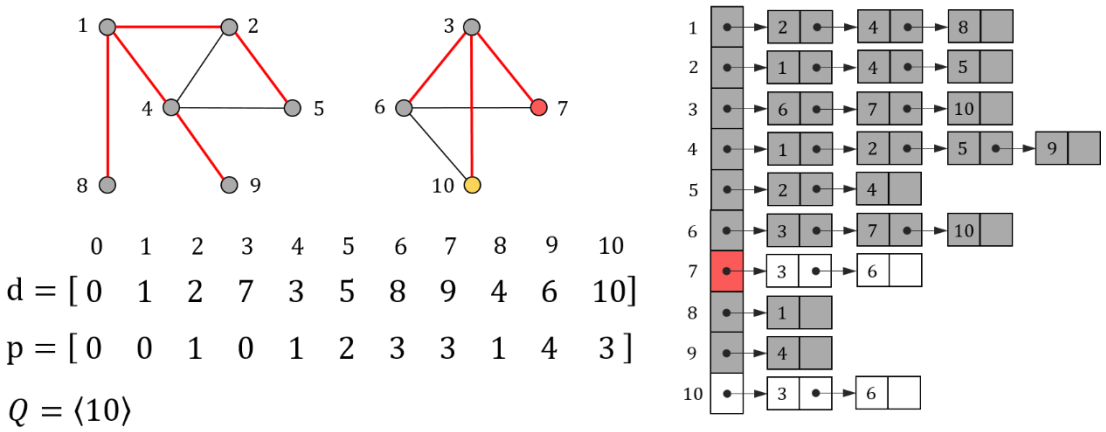
Εικόνα 4.20: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.19.

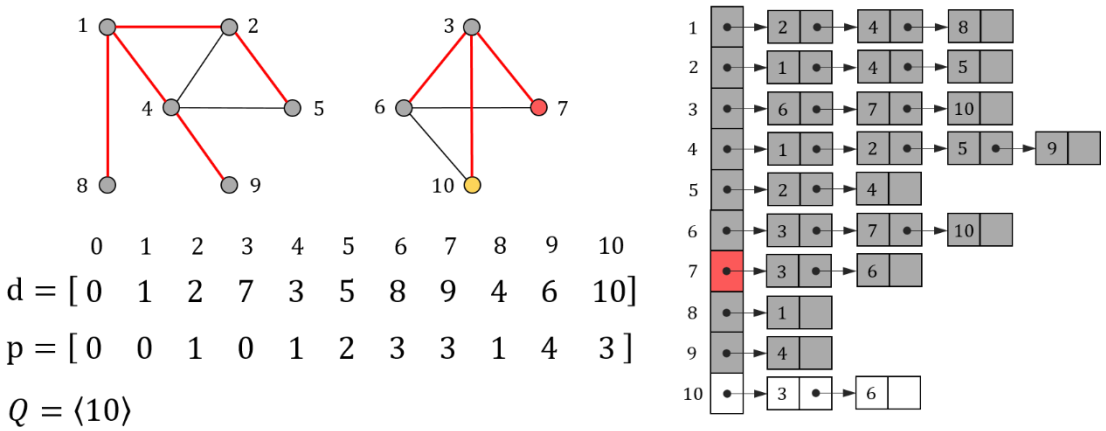


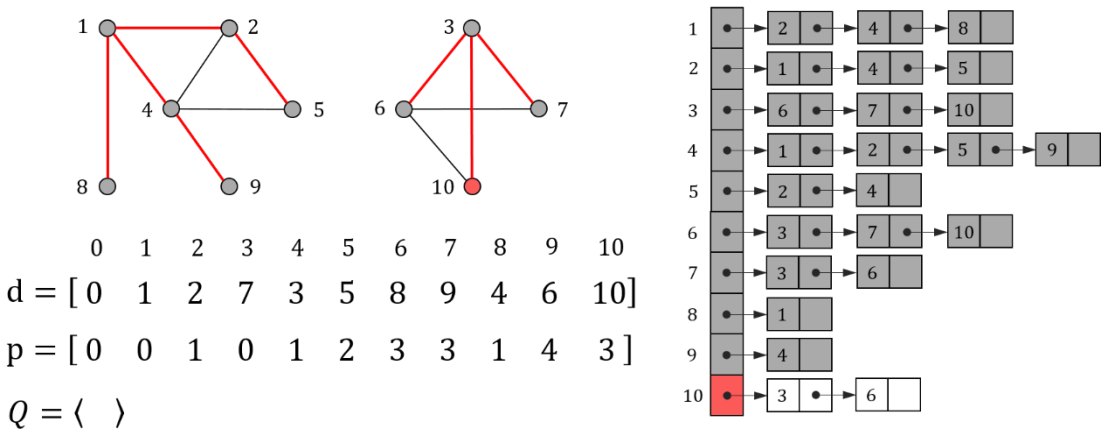




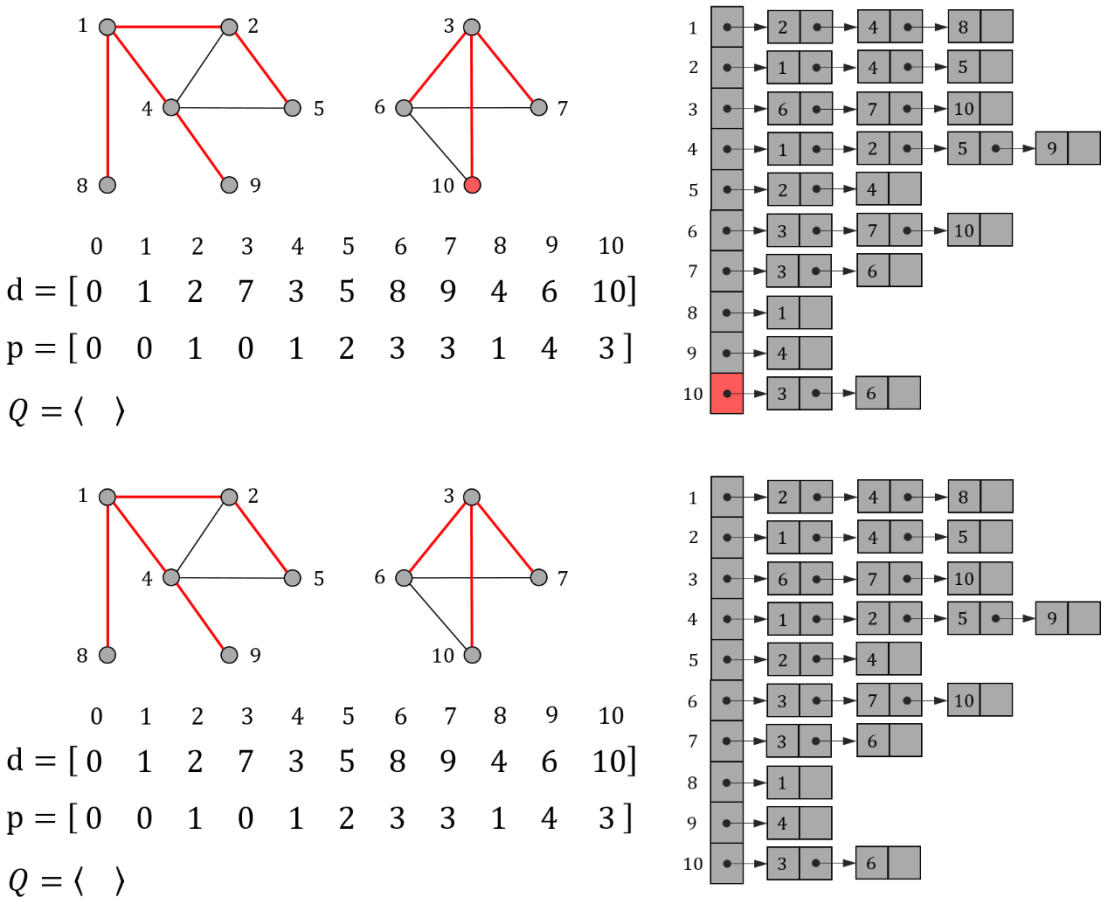
Εικόνα 4.21: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.20.



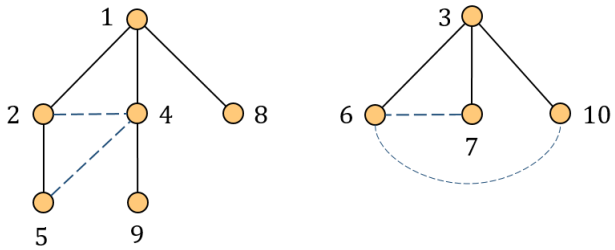




Εικόνα 4.22: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.21.



Εικόνα 4.23: Κατά-πλάτος διερεύνηση του γραφήματος της Εικόνας 4.8. Συνέχεια από την Εικόνα 4.22.



Εικόνα 4.24: Το δάσος της κατά-πλάτος διερεύνησης του γραφήματος της Εικόνας 4.8. Οι ακμές που δεν ανήκουν στο δάσος σημειώνονται με διακεκομμένες γραμμές.

Όπως και στην περίπτωση της κατά-βάθος διερεύνησης, ένα γράφημα μπορεί να έχει περισσότερα από ένα διαφορετικά bfs-δένδρα ή bfs-δάση. Το επίπεδο ενός κόμβου v στο bfs-δένδρο T, στο οποίο ανήκει, ορίζεται αναδρομικά ως εξής:

$$level(v) = \begin{cases} 0, & \text{εάν ο κόμβος } v \text{ είναι ρίζα του δένδρου } T \\ 1 + level\big(p(v)\big), & \text{διαφορετικά} \end{cases}$$

όπου $p(v)$ είναι ο γονέας του κόμβου v στο bfs-δένδρο T.

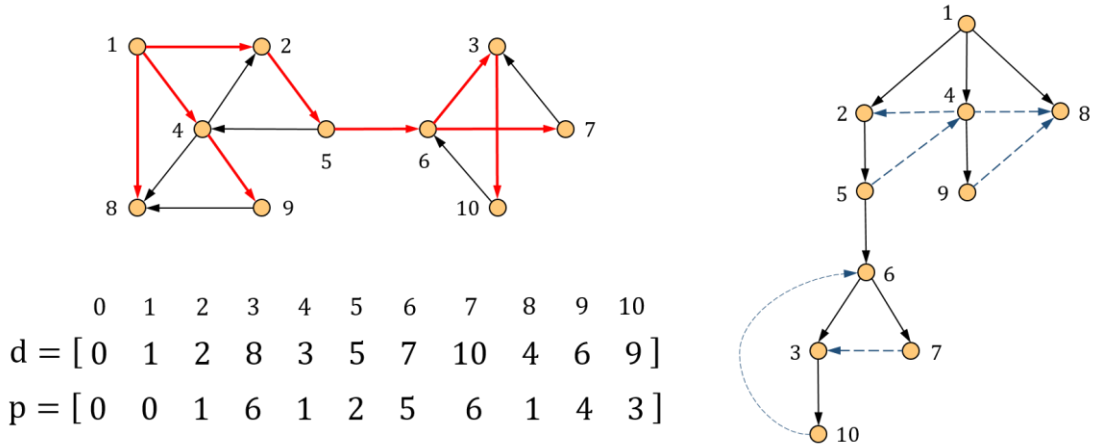
Το bfs-δένδρο T ενός μη-κατευθυνόμενου γραφήματος G έχει τις εξής σημαντικές και χρήσιμες ιδιότητες.

1. Εάν ο κόμβος v είναι πρόγονος του κόμβου w στο bfs-δένδρο T, τότε $d[v] < d[w]$.

2. Εάν το γράφημα G είναι μη-κατευθυνόμενο, τότε για κάθε ακμή (v, w) του G, είτε είναι δενδρική είτε μη-δενδρική, ισχύει ότι τα επίπεδα των κόμβων v και u διαφέρουν το πολύ κατά ένα.

3. Εάν ο κόμβος v ανήκει σε μία συνεκτική συνιστώσα G_i του γραφήματος G με αντίστοιχο bfs-δένδρο T_i με ρίζα r, τότε το επίπεδο του κόμβου v ισούται με το μήκος της ελάχιστης διαδρομής από τον κόμβο r στον v στο γράφημα G.

Όπως ισχύει με την κατά-βάθος διερεύνηση, και η κατά-πλάτος διερεύνηση εκτελείται με ακριβώς τον ίδιο τρόπο σε ένα κατευθυνόμενο γράφημα $G = (V, E)$. Δείτε την Εικόνα 4.25.



Εικόνα 4.25: Το δάσος (δένδρο) της κατά-πλάτος διερεύνησης του γραφήματος της Εικόνας 4.14. Οι ακμές που δεν ανήκουν στο δάσος σημειώνονται με διακεκομμένες γραμμές.

Υλοποίηση και Πολυπλοκότητα. Τόσο η κατά-βάθος όσο και η κατά-πλάτος διερεύνηση ενός γραφήματος G με m ακμές και n κόμβους μπορούν να εκτελεστούν σε $O(n + m)$ χρόνο και χώρο. Μία τέτοια υλοποίηση λέμε ότι είναι γραμμική ως προς το μέγεθος του γραφήματος και είναι η καλύτερη δυνατή που μπορούμε να επιτύχουμε, μίας και πρέπει να εξετάσουμε όλους τους κόμβους και όλες τις ακμές του γραφήματος εισόδου. Και οι δύο αλγόριθμοι διερεύνησης επιτυγχάνουν γραμμικό χρόνο εκτέλεσης με την προϋπόθεση ότι χρησιμοποιούμε λίστα γειτνίασης για την αναπαράσταση του γραφήματος.

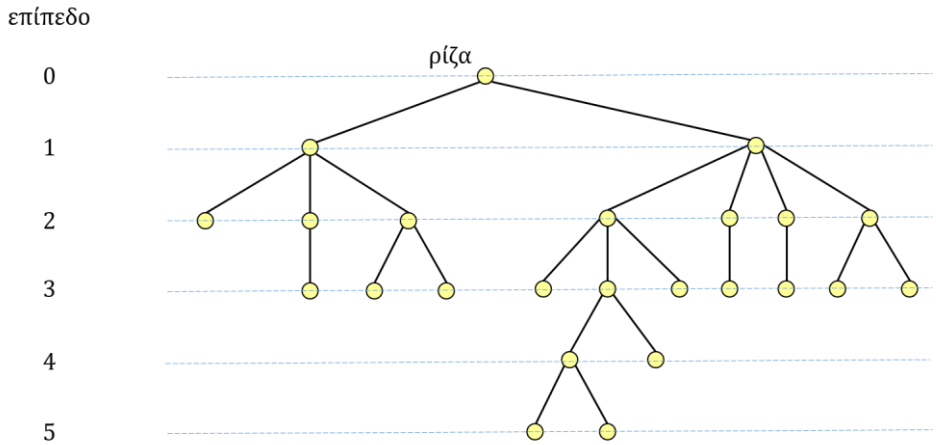
4.4 Δένδρα

Ένα δένδρο (tree) ορίζεται ως ένα μη-κατευθυνόμενο γράφημα το οποίο είναι συνεκτικό και άκυκλο. Στο επόμενο θεώρημα συνοψίζονται βασικές ιδιότητες των δένδρων, οι οποίες αφορούν στην τάξη τους, το μέγεθός τους και στοιχεία της συνεκτικότητάς τους.

Θεώρημα 4.1 Έστω G ένα μη-κατευθυνόμενο γράφημα με n κόμβους. Οι ακόλουθες προτάσεις είναι ισοδύναμες.

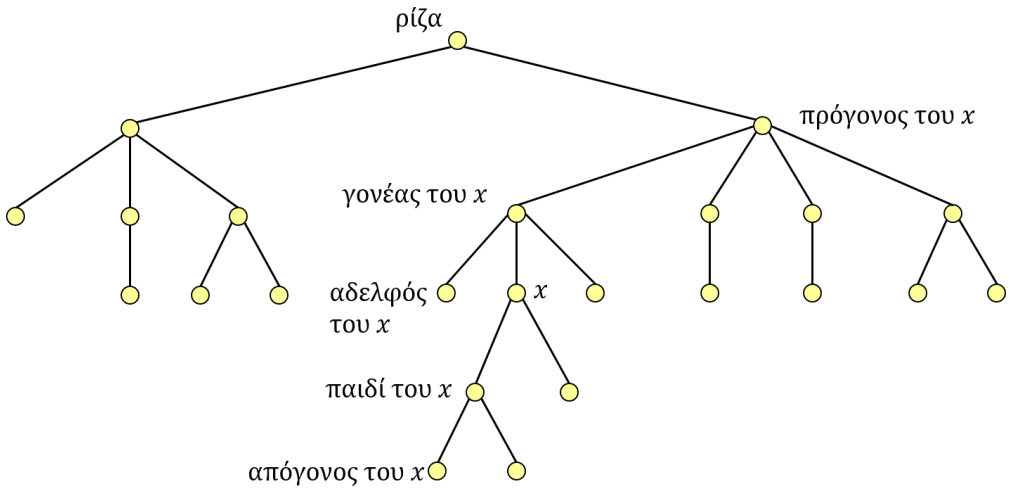
1. Το γράφημα G είναι δένδρο.
2. Υπάρχει μοναδική διαδρομή μεταξύ κάθε ζεύγους κόμβων του G.
3. Το γράφημα G είναι συνεκτικό και κάθε ακμή του είναι γέφυρα.
4. Το γράφημα G είναι συνεκτικό και έχει $n-1$ ακμές.
5. Το γράφημα G είναι άκυκλο και έχει $n-1$ ακμές.
6. Το γράφημα G είναι άκυκλο και οποτεδήποτε δύο τυχαίοι μη-γειτονικοί κόμβοι του G ενωθούν με ακμή, το προκύπτον γράφημα G' έχει ένα μοναδικό κύκλο.
7. Το γράφημα G είναι συνεκτικό και οποτεδήποτε δύο τυχαίοι μη-γειτονικοί κόμβοι του G ενωθούν με ακμή, το προκύπτον γράφημα G' έχει ένα μοναδικό κύκλο.

Μπορούμε να ορίσουμε μια ιεραρχία των κόμβων ενός δένδρου με βάση ένα διακεκριμένο αφετηριακό κόμβο τον οποίο καλούμε *ρίζα* του δένδρου. Οι κόμβοι ενός δένδρου με ρίζα χωρίζονται σε επίπεδα, όπου του επίπεδο i περιλαμβάνει τους κόμβους σε απόσταση i από τη ρίζα. Δείτε την Εικόνα 4..



Εικόνα 4.26: Ένα δένδρο με ρίζα και τα επίπεδα του.

Κάθε κόμβος x, εκτός της ρίζας, έχει ένα γείτονα p στο αμέσως προηγούμενο επίπεδο. Καλούμε αυτό το γείτονα p, *γονέα* του x. Σε αυτήν την περίπτωση λέμε, επίσης, ότι ο x είναι *παιδί* του p. Ένας κόμβος y που έχει τον ίδιο γονέα με τον x είναι αδελφός του x. Κάθε κόμβος στο μονοπάτι από τη ρίζα προς τον x ονομάζεται *πρόγονος* του x. Αν ένας κόμβος y είναι πρόγονος του x, τότε λέμε και ότι ο x είναι απόγονος του y. Ένα παράδειγμα αυτών των σχέσεων φαίνεται στην Εικόνα 4.27.



Εικόνα 4.27: Ένα δένδρο με ρίζα και οι διάφορες σχέσεις μεταξύ των κόμβων του.

Ένα δένδρο στο οποίο έχουμε ορίσει μια διάταξη $x_1, x_2, \ldots, x_k$ των παιδιών κάθε κόμβου x ονομάζεται διατεταγμένο. Ένα διατεταγμένο δένδρο στο οποίο κάθε κόμβος έχει το πολύ m παιδιά ονομάζεται m-αδικό δένδρο.

Τα δένδρα αποτελούν πολύ σημαντικές δομές δεδομένων για την οργάνωση και επεξεργασία πληροφοριών. Όπως θα δούμε στα επόμενα κεφάλαια, ανάλογα με τις λειτουργίες που χρειάζεται να υποστηρίξουμε, μπορούμε να καθορίσουμε μια κατάλληλη μορφή και αναπαράσταση ενός δένδρου. Στη συνέχεια θα εστιάσουμε σε μια χρήσιμη κατηγορία δένδρων, τα δυαδικά δένδρα.

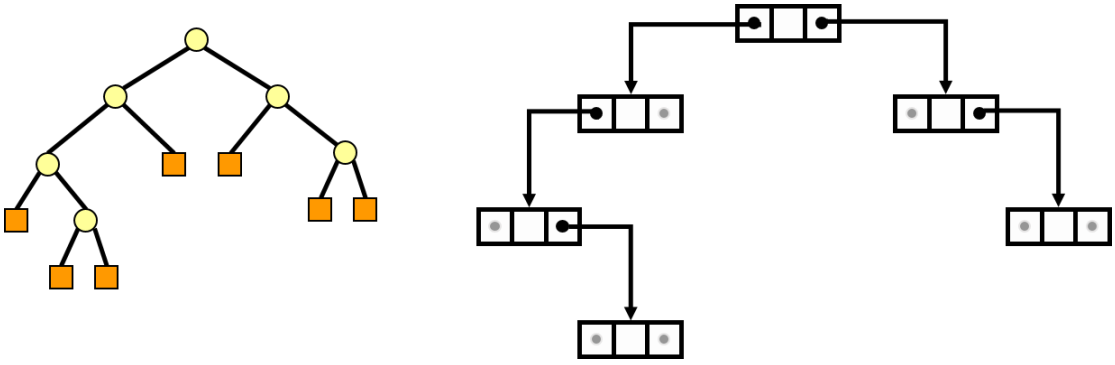
4.4.1 Δυαδικά δένδρα

Ένα *δυαδικό δένδρο* είναι ένα διατεταγμένο δένδρο όπου κάθε κόμβος έχει το πολύ δύο μη κενά παιδιά. Θα καλούμε έναν κόμβο v εσωτερικό, αν δεν είναι κενός (v!=null), και εξωτερικό διαφορετικά (v==null). Ένα αναδρομικός ορισμός των δυαδικών δένδρων έχει ως εξής: Ένα δυαδικό δένδρο είναι ένας εξωτερικός κόμβος ή ένας εσωτερικός κόμβος ο οποίος συνδέεται με ένα δυαδικό δένδρο στα αριστερά και ένα δυαδικό δένδρο στα δεξιά.

Στη Java μπορούμε να ορίσουμε μια κατάλληλη κλάση BSTNode κόμβου δυαδικού δένδρου ως εξής:

```
class BSTNode {
	Item item; BSTNode l; BSTNode r;
	Node(Item v, BSTNode l, BSTNode r) {
		this.item = v;
		this.l = l;
		this.r = r;
	}
}
```

Κάθε κόμβος v αποθηκεύει ένα στοιχείο τύπου Item, καθώς και δύο αναφορές σε κόμβους τύπου BSTNode: μια αναφορά $v.l$ στο αριστερό παιδί και μια αναφορά $v.r$ στο δεξί παιδί του κόμβου v. Δείτε την Εικόνα 4..



Εικόνα 4.28: Ένα δυαδικό δένδρο και η υλοποίηση του με μια δομή δεδομένων. Αναπαριστούμε τους εσωτερικούς κόμβους με κύκλους και τους εξωτερικούς κόμβους με τετράγωνα. Στην υλοποίηση, οι εσωτερικοί κόμβοι αντιστοιχούν σε αντικείμενα τύπου Node και οι εξωτερικοί κόμβοι είναι null.

Το *ύψος* ενός δένδρου είναι ίσο με το μέγιστο μήκος διαδρομής από τη ρίζα προς ένα εξωτερικό κόμβο. Ισοδύναμα, το ύψος του δένδρου είναι ίσο με το μέγιστο επίπεδο ενός εξωτερικού κόμβου. Το ύψος μιας δενδρικής δομής δεδομένων καθορίζει την αποδοτικότητα διαφόρων λειτουργιών της. Στα δυαδικά δένδρα ισχύει η ακόλουθη ιδιότητα (δείτε την Εικόνα 4.29Εικόνα 4.).

Ιδιότητα 4.1 Ένα δυαδικό δένδρο με n εσωτερικούς κόμβους έχει ύψος μεταξύ $\lg n$ και n.

Απόδειξη

Έστω h το ύψος ενός δυαδικού δένδρου T με n εσωτερικούς κόμβους. Το άνω φράγμα $h \leq n$ προκύπτει άμεσα από τον ορισμό του ύψους. Απομένει, λοιπόν, να δείξουμε ότι $h \geq \lg n$. Παρατηρούμε ότι κάθε επίπεδο j του T έχει το πολύ 2^j εσωτερικούς κόμβους. Καθώς το μέγιστο επίπεδο του T είναι το $h-1$, έχουμε συνολικά το πολύ $\sum_{j=0}^{h-1} 2^j$ εσωτερικούς κόμβους. Άρα

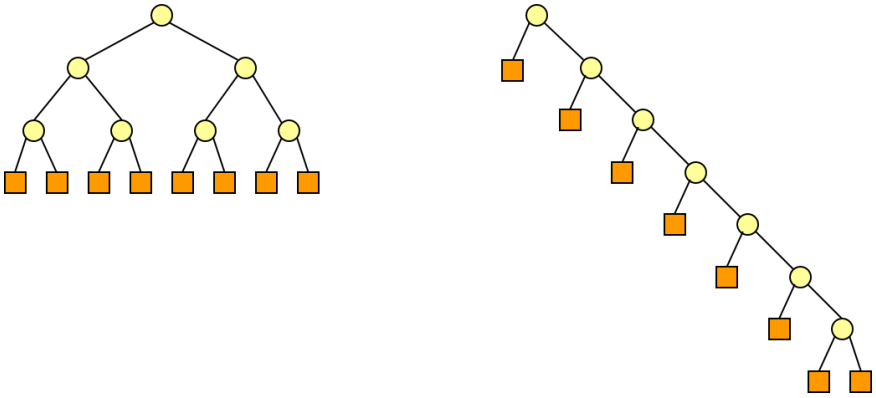
$$n \leq \sum_{j=0}^{h-1} 2^j = 2^h - 1.$$

Λύνοντας ως προς h λαμβάνουμε

$$2^h \geq n + 1,$$

άρα

$$h \geq \lg(n+1) \geq \lg n.$$



Εικόνα 4.29: Δύο ακραίες περιπτώσεις δυαδικών δένδρων με 7 εσωτερικούς κόμβους.

Διάσχιση δένδρου

Αντίστοιχα με τις διασχίσεις ενός γραφήματος, μπορούμε να κάνουμε μια συστηματική αναζήτηση σε ένα δένδρο T, με την οποία επισκεπτόμαστε όλους τους κόμβους του δένδρου. Σε ένα δυαδικό δένδρο χρησιμοποιούμε συνήθως μια από τις παρακάτω μεθόδους, οι οποίες αποτελούν ειδικές περιπτώσεις της κατά-βάθος διερεύνησης γραφήματος.

Προδιάταξη: Επεξεργαζόμαστε πρώτα το γονέα, στη συνέχεια το αριστερό παιδί και, τέλος, το δεξί παιδί.

```
void preorder(BSTNode v) {
       if (v == null) return;
       v.process();
       preorder(v.l);
       preorder(v.r);
}
```

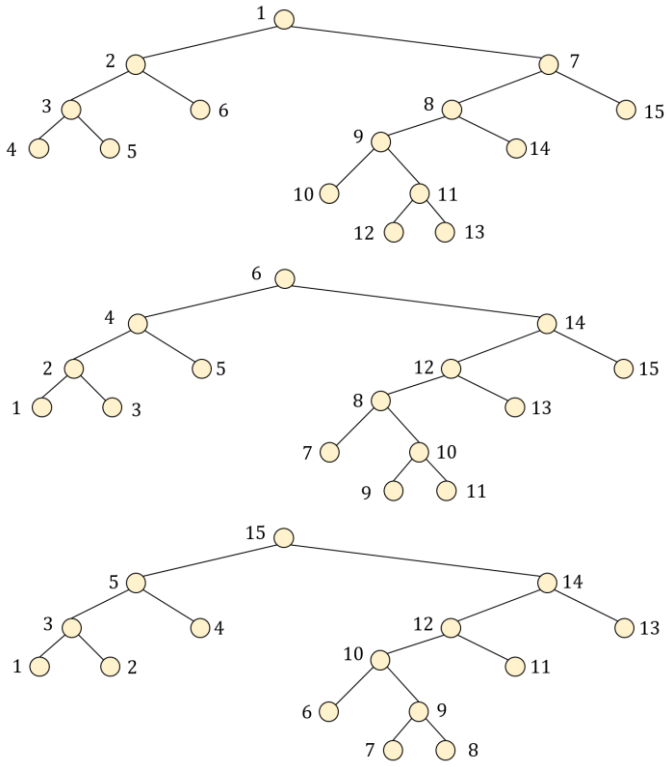
Ενδοδιάταξη: Επεξεργαζόμαστε πρώτα το αριστερό παιδί, στη συνέχεια το γονέα και, τέλος, το δεξί παιδί.

```
void inorder(BSTNode v) {
       if (v == null) return;
       v.process();
       inorder(v.l);
       inorder(v.r);
}
```

Μεταδιάταξη: Επεξεργαζόμαστε πρώτα το αριστερό παιδί, στη συνέχεια το δεξί παιδί και, τέλος, το γονέα.

```
void postorder(BSTNode v) {
       if (v == null) return;
       preorder(v.l);
       preorder(v.r);
       v.process();
}
```

Η αντίστοιχη σειρά επεξεργασίας των κόμβων φαίνεται στην Εικόνα 4..



Εικόνα 4.30: Προσδιάταξη, ενδοδιάταξη και μεταδιάταξη δυαδικού δένδρου.

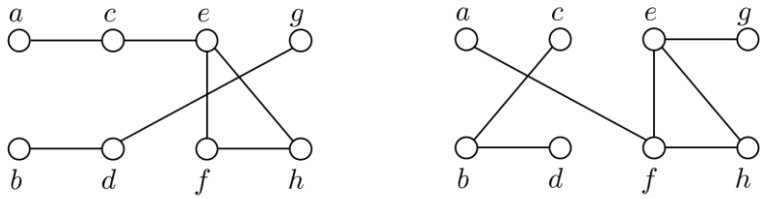
Ασκήσεις

4.1 Υλοποιήστε σε Java τη δομή αναπαράστασης γραφήματος με στατική λίστα γειτνίασης. Μπορείτε να υποθέσετε ότι το πλήθος των ακμών και των κόμβων είναι γνωστό και μας δίνεται εξαρχής. Οι ακμές πρέπει να διαβάζονται από την είσοδο, ως ζεύγη κόμβων, σε αυθαίρετη σειρά.

4.2 Έστω $G = (V, E)$ ένα συνεκτικό μη κατευθυνόμενο γράφημα. Σχεδιάστε έναν γραμμικό αλγόριθμο (δηλαδή με χρόνο εκτέλεσης $O(|V| + |E|)$) που να προσδιορίζει εάν υπάρχει κάποια ακμή $e \in E$, τέτοια ώστε, το G να παραμένει συνεκτικό μετά τη διαγραφή της e. Μπορείτε να κάνετε τον αλγόριθμο σας να εκτελείται σε χρόνο $O(|V|)$;

4.3 Ένα μη κατευθυνόμενο γράφημα $G = (V, E)$ είναι *διμερές*, όταν οι κόμβοι του μπορούν να χωριστούν σε δύο ξένα μεταξύ τους σύνολα A και B, δηλαδή $V = A \cup B$ και $A \cap B = \emptyset$, τέτοια ώστε για κάθε ακμή $\{a, b\} \in E$ να έχουμε $a \in A$ και $b \in B$. Δώστε έναν αποδοτικό αλγόριθμο που να αναγνωρίζει εάν ένα γράφημα είναι διμερές. Επίσης, αν το γράφημα είναι διμερές, ο αλγόριθμος πρέπει να παράγει τα σύνολα A και B. Ποιος είναι ο χρόνος εκτέλεσης του αλγορίθμου σας;

4.4 Έστω ότι έχουμε μια φυσική διεργασία που περιγράφεται από ένα χρονικά μεταβαλλόμενο γράφημα $G(t)$, στο οποίο μπορεί να προστίθενται και να αφαιρούνται ακμές με την πάροδο του χρόνου t. Μας ενδιαφέρει να βρούμε τα σύνολα των κόμβων που ανήκουν στην ίδια συνεκτική συνιστώσα, τόσο στο αρχικό γράφημα $G(0)$ όσο και στο τελικό $G(T)$.



Για παράδειγμα, στο παραπάνω σχήμα, οι συνεκτικές συνιστώσες του $G(0)$ είναι οι $\{a, c, e, f, h\}$ και $\{b, d, g\}$, ενώ οι συνεκτικές συνιστώσες του $G(T)$ είναι οι $\{a, e, f, g, h\}$ και $\{b, c, d\}$. Επομένως, τα σύνολα των κόμβων που βρίσκονται στην ίδια συνεκτική συνιστώσα τόσο στο αρχικό γράφημα $G(0)$ όσο και στο τελικό $G(T)$ είναι τα $\{a, e, f, h\}$, $\{b, d\}$, $\{c\}$ και $\{g\}$. Παρατηρήστε ότι τα σύνολα αυτά προκύπτουν από τις τομές των συνεκτικών συνιστωσών στο $G(0)$ και το $G(T)$, π.χ. $\{a, c, e, f, h\} \cap \{a, e, f, g, h\} = \{a, e, f, h\}$ και $\{a, c, e, f, h\} \cap \{b, c, d\} = \{c\}$.

Περιγράψτε έναν απλό και αποδοτικό αλγόριθμο που να λύνει το παραπάνω πρόβλημα. Ποιος είναι ο ασυμπτωτικός χρόνος εκτέλεσης του αλγόριθμού σας; *Υπόδειξη:* Ο αλγόριθμος θα υπολογίζει πρώτα τις συνεκτικές συνιστώσες των $G(0)$ και $G(T)$. Στη συνέχεια, μπορεί να χρησιμοποιήσει ταξινόμηση, για να βρει γρήγορα τις τομές των διαφορετικών συνεκτικών συνιστωσών.

4.5 Δώστε μια μη αναδρομική υλοποίηση της προδιατεταγμένης διάσχισης ενός δυαδικού δένδρου.

4.6 Ένα δυαδικό δένδρο με n εσωτερικούς κόμβους μπορεί να αναπαρασταθεί από μια ακολουθία $b_1 b_2 \ldots b_{2n+1}$ από $2n + 1$ δυαδικά ψηφία, η οποία ικανοποιεί τις ακόλουθες συνθήκες: 1) $n + 1$ ψηφία είναι $b_i = 0$ και τα υπόλοιπα n είναι $b_i = 1$, 2) Για κάθε θέση $1 \leq i \leq 2n + 1$, ο αριθμός των 0 που βρίσκονται πριν από το b_i είναι μεγαλύτερος ή ίσος του αριθμού των 1 που βρίσκονται πριν από το b_i. Δείξτε πώς μπορούμε να κατασκευάσουμε μια τέτοια αναπαράσταση ενός δυαδικού δένδρου.
Υπόδειξη: Χρησιμοποιήστε την προδιάταξη του δένδρου.

Βιβλιογραφία

Cormen, T., Leiserson, C., Rivest, R., & Stain, C. (2001). *Introduction to Algorithms.* MIT Press (2nd edition).

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Νικολόπουλος, Σ. Δ., Γεωργιάδης, Λ., & Παληός, Λ. (2015). *Αλγοριθμική Θεωρία Γραφημάτων.* Εκδόσεις Κάλλιπος (http://hdl.handle.net/11419/2067).

Κεφάλαιο 5
Συλλογές, Στοίβες και Ουρές

Περιεχόμενα

5.1 Αφηρημένοι τύποι δεδομένων 94
5.2 Συλλογές και Επαναλήπτες 95
5.1.1 Εφαρμογή: Υλοποίηση λιστών γειτνίασης γραφήματος 96
5.2 Στοίβα 97
5.2.1 Υλοποίηση στοίβας με συνδεδεμένη λίστα 98
5.2.3 Υλοποίηση στοίβας με πίνακα 100
5.3 Ουρά 101
5.3.1 Υλοποίηση ουράς με συνδεδεμένη λίστα 102
5.3.3 Υλοποίηση ουράς με πίνακα 104
5.4 Εφαρμογή: Συντακτική ανάλυση εκφράσεων 106
5.4 Γενικευμένες Ουρές 109
5.4.1 Υλοποίηση ουράς δύο άκρων με συνδεδεμένη λίστα 110
5.5 Παρατηρήσεις 111
Ασκήσεις 111
Βιβλιογραφία 113

5.1 Αφηρημένοι τύποι δεδομένων

Σε αυτό το κεφάλαιο θα μελετήσουμε ορισμένες δομές δεδομένων οι οποίες, παρόλο που υποστηρίζουν ένα πολύ περιορισμένο σύνολο λειτουργιών, είναι εξαιρετικά χρήσιμες σε πληθώρα εφαρμογών.

Η πρώτη δομή που θα περιγράψουμε, η συλλογή, αποθηκεύει ένα σύνολο στοιχείων με σκοπό την επεξεργασία τους, χωρίς να παίζει ρόλο η σειρά με την οποία θα γίνει η επεξεργασία αυτή. Οι άλλες δύο δομές, η στοίβα και η ουρά, διατηρούν αποθηκευμένα τα στοιχεία σε «χρονολογική σειρά» εισαγωγής και επιτρέπουν την πρόσβαση μόνο στο νεότερο ή το παλαιότερο στοιχείο, αντίστοιχα.

Οι παραπάνω περιορισμοί στην πρόσβαση των στοιχείων μιας συλλογής, στοίβας ή ουράς, μας επιτρέπουν να επιτύχουμε σταθερό χρόνο εκτέλεσης ανά λειτουργία. Σκοπός αυτού του κεφαλαίου είναι να αναδείξει τους πιο απλούς και αποτελεσματικούς τρόπους που επιτυγχάνουν αυτό το στόχο.

5.2 Συλλογές και Επαναλήπτες

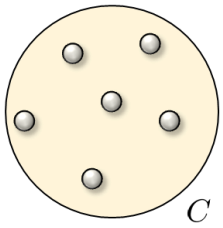
Μια συλλογή C διατηρεί ένα σύνολο στοιχείων και υποστηρίζει τις παρακάτω βασικές λειτουργίες:

κατασκευή() : Επιστρέφει μια κενή συλλογή.

εισαγωγή(x) : Εισάγει στη C ένα στοιχείο x.

πλήθος() : Επιστρέφει το πλήθος των στοιχείων της C.

είναι κενή() : Ελέγχει αν η C είναι κενή.



Είναι εύκολο να πετύχουμε σταθερό χρόνο εκτέλεσης για κάθε μία από τις παραπάνω λειτουργίες χρησιμοποιώντας απλές δομές. Παρατηρήστε ότι δεν έχουμε συμπεριλάβει κάποια εντολή πρόσβασης στα στοιχεία της συλλογής. Ο λόγος είναι ότι θα εκμεταλλευτούμε τις δυνατότητες που προσφέρουν οι επαναλήπτες (iterators) της Java, για να υλοποιήσουμε έναν πολύ κομψό τρόπο πρόσβασης όλων των n στοιχείων της συλλογής σε $O(n)$ χρόνο, δηλαδή σε σταθερό χρόνο ανά στοιχείο. Οι επιδόσεις μιας τέτοιας δομής συνοψίζονται στον παρακάτω πίνακα.

Πίνακας 5.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης των λειτουργιών μιας συλλογής με n στοιχεία, υλοποιημένης με στοιχειώδεις δομές.

	πλήθος	εισαγωγή	προσπέλαση όλων
μη διατεταγμένος πίνακας	$O(1)$	$O(1)$	$O(n)$
μη διατεταγμένη λίστα	$O(1)$	$O(1)$	$O(n)$

Παρακάτω δίνουμε μια ενδεικτική υλοποίηση `ListCollection` με συνδεδεμένη λίστα η οποία υλοποιεί την ακόλουθη διασύνδεση.

```
class Collection<Item>   // συλλογή αντικειμένων γενικού τύπου Item
{
   Collection ();        // αρχικοποίηση κενής συλλογής
   boolean isEmpty();    // έλεγχος αν η συλλογή είναι άδεια
   int size();           // πλήθος στοιχείων στη συλλογή
   void insert (Item);   // εισαγωγή αντικειμένου στη συλλογή
}
```

Καθώς στη `ListCollection` χρησιμοποιούμε κόμβους λίστας με μόνο μια αναφορά στον επόμενο κόμβο, η πρόσβαση στα στοιχεία της συλλογής γίνεται μέσω της αναφοράς `first` στον πρώτο κόμβο της λίστας. Έτσι, για την εισαγωγή ενός νέου στοιχείου `item`, αποθηκεύουμε το `item` σε ένα νέο κόμβο, τον οποίο τοποθετούμε στην αρχή της λίστας.

```
/* υλοποίηση συλλογής με απλά συνδεδεμένη λίστα */
public class ListCollection<Item> implements Iterable<Item> {

   /* τύπος κόμβου συνδεδεμένης λίστας */
   private class Node {
      Item item;
      Node next;
      Node(Item item, Node next) {
          this.item = item;
          this.next = next;
      }
   }
```

```
   private Node first;  // πρώτος κόμβος της λίστας

   /* κατασκευή κενής συλλογής */
   ListCollection() { first = null; }

   /* έλεγχος αν η συλλογή είναι κενή */
   public boolean isEmpty() {
      return first == null;
   }

   /* εισαγωγή αντικειμένου στη συλλογή */
   public void insert(Item item) {
      first = new Node(item, first);
   }

   /* επαναλήπτης συλλογής */
   public Iterator<Item> iterator() {
      return new ListIterator();
   }

   private class ListIterator implements Iterator<Item> {
      private Node current = first;  // τρέχων κόμβος της συλλογής

      /* έλεγχος αν ο τρέχων κόμβος έχει φτάσει στο τέλος της λίστας */
      public boolean hasNext() {
         return current!=null;
      }

      /* διαγραφή αντικειμένου: δεν ορίζεται στην περίπτωση της συλλογής μας */
      public void remove() {}

      /* επιστρέφει το επόμενο στοιχείο της συλλογής */
      public Item next() {
         Item item = current.item;
         current = current.next;
         return item;
      }
    }
  }
```

Ο επαναλήπτης της `ListCollection` επιστρέφει τα στοιχεία της συλλογής με τη σειρά κατά την οποία εμφανίζονται στη λίστα από τον πρώτο προς τον τελευταίο κόμβο. Έτσι, η προσπέλαση του επόμενου στοιχείου γίνεται κάθε φορά σε σταθερό χρόνο.

5.1.1 Εφαρμογή: Υλοποίηση λιστών γειτνίασης γραφήματος

Η χρήση μιας συλλογής με επαναλήπτη δίνει μια κομψή υλοποίηση των λιστών γειτνίασης για την αναπαράσταση ενός γραφήματος. Στο Κεφάλαιο 3 αναπτύξαμε μια σύνθετη δομή δεδομένων, βασισμένη σε έναν πίνακα αναφορών adj, ο οποίος αποθηκεύει αναφορές στις λίστες των γειτόνων του κάθε κόμβου του γραφήματος. Εδώ θα κάνουμε μια μικρή τροποποίηση αυτής της δομής, ώστε κάθε λίστα γειτόνων να αποθηκεύεται σε μία συλλογή. Τώρα, ο πίνακας adj αποθηκεύει αναφορές σε συλλογές, ενώ η πρόσβαση όλων των γειτόνων ενός κόμβου v γίνεται με ένα βρόχο της μορφής

```
for (int w : adj(v)) { … }
```

Το παρακάτω πρόγραμμα παρουσιάζει την υλοποίηση ολόκληρης της δομής λιστών γειτνίασης με συλλογές.

```
/* υλοποίηση λιστών γειτνίασης γραφήματος με συλλογές */
public class Graph {

   private final int n;                // πλήθος κόμβων
   private int m;                      // πλήθος ακμών
   private Collection<Integer>[] adj;  // λίστες γειτνίασης

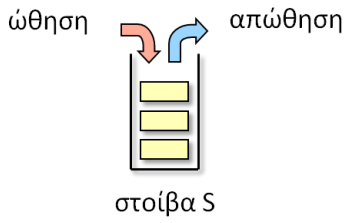
   /* δημιουγία γραφήματος με n κόμβους και καμία ακμή */
   public Graph(int n) {
      this.n = n;
      this.m = 0;
      // πίνακας αναφορών σε συλλογές
      adj = (Collection<Integer>[]) new Collection[n];
      for (int i = 0; i < N; i++)
         adj[i] = new Collection<Integer>();
   }

   /* προσθήκη ακμής {v,w} */
   public void addEdge(int v, int w) {
      adj[v].insert(w);
      adj[w].insert(v);
      m++;
   }
   /* επαναλήπτης για γειτονικούς κόμβους του κόμβου v */
   public Iterable<Integer> adj(int v) {
      return adj[v];
   }

   /* τυπώνει τις λίστες γειτνίασης */
   public void printGraph() {
      System.out.println("adjacency lists");
      for (int v = 0; v < n; v++) {
         System.out.print(v + " : ");
         for (int w : adj(v))
            System.out.print(w + " ");
         System.out.println("");
      }
   }
}
```

5.2 Στοίβα

Η στοίβα είναι μια βασική δομή δεδομένων η οποία διατηρεί ένα σύνολο αντικειμένων σε σειρά εισαγωγής και δίνει γρήγορη πρόσβαση στα στοιχεία που έχουν εισαχθεί πιο πρόσφατα. Συγκεκριμένα, μια στοίβα S υποστηρίζει τις παρακάτω βασικές λειτουργίες:



κατασκευή() : Επιστρέφει μια κενή στοίβα.

ώθηση(x) : Εισάγει το στοιχείο x στην κορυφή της S.

απώθηση() : Διαγράφει από την S το στοιχείο x που βρίσκεται στην κορυφή της S και επιστρέφει το x.

κορυφή() : Επιστρέφει το στοιχείο που βρίσκεται στην κορυφή της S. Το στοιχείο αυτό παραμένει στην S.

πλήθος() : Επιστρέφει το πλήθος των στοιχείων της S.

είναι κενή() : Ελέγχει αν η S είναι κενή.

Όπως και στην περίπτωση της συλλογής, θα αναπτύξουμε δύο αποδοτικές λύσεις με στοιχειώδεις δομές δεδομένων, δηλαδή με συνδεδεμένες λίστες και πίνακες.

Πίνακας 5.2: Χρόνοι εκτέλεσης των λειτουργιών μιας στοίβας με n στοιχεία, υλοποιημένης με στοιχειώδεις δομές. Για την υλοποίηση με λίστα ή με πίνακα σταθερού μεγέθους οι χρόνοι είναι χειρότερης περίπτωσης. Για την υλοποίηση με δυναμικό πίνακα, οι χρόνοι είναι αντισταθμιστικοί.

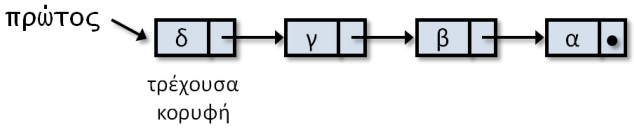
	ώθηση	απώθηση	πλήθος
πίνακας	$O(1)$	$O(1)$	$O(1)$
απλά συνδεδεμένη λίστα	$O(1)$	$O(1)$	$O(1)$

Η διασύνδεση της στοίβας έχει την παρακάτω μορφή:

```
class Stack<Item>      // στοίβα αντικειμένων γενικού τύπου Item
{
   Stack(int);         // αρχικοποίηση στοίβας με δεδομένη χωρητικότητα
   Stack();            // αρχικοποίηση κενής στοίβας
   void push(Item);    // ώθηση αντικειμένου
   Item pop();         // απώθηση αντικειμένου
   Item top();         // επιστρέφει το στοιχείο στην κορυφή της στοίβας
   int size();         // πλήθος στοιχείων στη στοίβα
   boolean isEmpty();  // έλεγχος αν η στοίβα είναι κενή
}
```

Παρέχουμε δύο μεθόδους κατασκευής, οι οποίες δίνουν στο χρήστη τη δυνατότητα να δηλώσει το μέγεθος της στοίβας, δηλαδή το μέγιστο πλήθος των στοιχείων που μπορεί να αποθηκεύσει η στοίβα, ή να το αφήσει απροσδιόριστο. Η διαφοροποίηση αυτή είναι χρήσιμη στην υλοποίηση με πίνακα, όταν το πλήθος των αντικειμένων που χειριζόμαστε είναι γνωστό. Στην κατασκευή μιας στοίβας με συνδεδεμένη λίστα δεν χρησιμοποιούμε την πληροφορία για το πλήθος των αντικειμένων και, επομένως, δεν υπάρχει διαφοροποίηση των δύο μεθόδων κατασκευής στην περίπτωση αυτή.

5.2.1 Υλοποίηση στοίβας με συνδεδεμένη λίστα

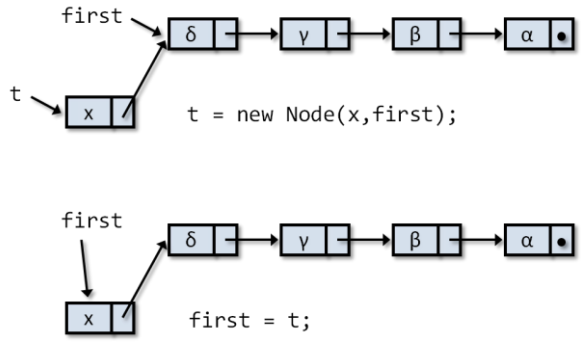


Εικόνα 5.4: Υλοποίηση στοίβας με απλά συνδεδεμένη λίστα. Η σειρά εισαγωγής των στοιχείων είναι α, β, γ, δ. Ο πρώτος κόμβος της λίστας αντιστοιχεί στην κορυφή της στοίβας.

Όπως και στην υλοποίηση της συλλογής που είδαμε παραπάνω, χρησιμοποιούμε μια απλά συνδεδεμένη λίστα που κάθε κόμβος της αποθηκεύει ένα στοιχείο και μια αναφορά στον επόμενο κόμβο της λίστας. Η πρόσβαση στη λίστα γίνεται μέσω της μεταβλητής first, η οποία αποθηκεύει μια αναφορά στον πρώτο κόμβο της λίστας και έτσι επιτρέπει τη γρήγορη πρόσβαση στα στοιχεία που αποθηκεύονται στην αρχή της λίστας. Αυτή η παρατήρηση μας

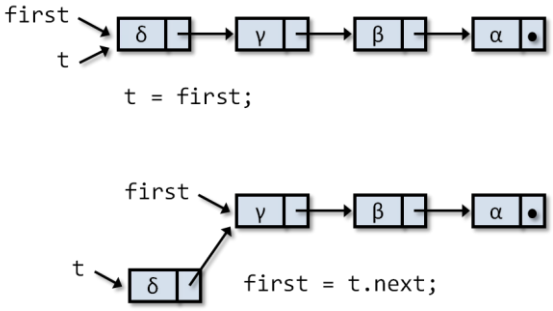
οδηγεί στο να αποθηκεύσουμε τα στοιχεία στη λίστα σε αντίστροφη σειρά εισαγωγής τους στη στοίβα, όπως φαίνεται στην Εικόνα 5.4. Έτσι, η μέθοδος top() απλώς επιστρέφει το στοιχείο first.item.

Για την ώθηση ενός στοιχείου x δημιουργούμε ένα νέο κόμβο t, όπου αποθηκεύουμε το στοιχείο x και θέτουμε στο πεδίο next την αναφορά first. Τέλος, θέτουμε τη μεταβλητή first να αναφέρεται στον κόμβο t. Αυτή η διαδικασία τοποθετεί το νέο κόμβο στην αρχή της λίστας και, επομένως, το νέο στοιχείο x βρίσκεται πλέον στην κορυφή της στοίβας.



Εικόνα 5.5: Ώθηση του στοιχείου x στην κορυφή της στοίβας της Εικόνα 5.4. Το στοιχείο αποθηκεύεται σε ένα νέο κόμβο ο οποίος τοποθετείται στην αρχή της λίστας.

Η απώθηση του στοιχείου που βρίσκεται στην κορυφή της στοίβας πραγματοποιείται με τη μεταφορά της αναφοράς first από τον πρώτο στο δεύτερο κόμβο της λίστας.



Εικόνα 5.6: Απώθηση του στοιχείου που βρίσκεται στην κορυφή της στοίβας της Εικόνα 5.4. Η προσωρινή μεταβλητή t αποθηκεύει την αναφορά στον πρώτο κόμβο της λίστας. Στη συνέχεια η μεταβλητή first αποθηκεύει την αναφορά στον επόμενο κόμβο από τον t.

```
/* υλοποίηση στοίβας με απλά συνδεδεμένη λίστα */
public class ListStack<Item> {

   private int n;   // πλήθος στοιχείων στη στοίβα

   /* τύπος κόμβου συνδεδεμένης λίστας */
   private class Node {
      Item item;
      Node next;
      Node(Item item, Node next) {
          this.item = item;
          this.next = next;
      }
   }

   private Node first;  // πρώτος κόμβος της λίστας
```

```
    /* κατασκευή κενής στοίβας */
    ListStack() {
       first = null;
       n = 0;
    }

    /* πλήθος στοιχείων στη στοίβα */
    public int size() {
       return n;
    }

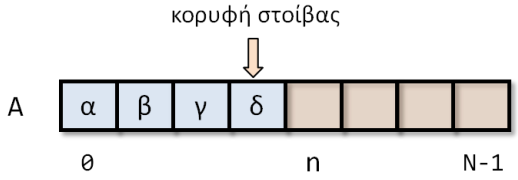
    /* έλεγχος αν η στοίβα είναι κενή */
    public boolean isEmpty() {
       return n == 0;
    }

    /* ώθηση στοιχείου στην κορυφή της στοίβας */
    public void push(Item item) {
       first = new Node(item, first);
       n++;
    }

    /* απώθηση του στοιχείου της κορυφής της στοίβας */
    public Item pop() {
       Node t = first;
       first = t.next;
       n--;
       return t.item;
    }

    /* στοιχείο της κορυφής της στοίβας */
    public Item top() {
       return first.item;
    }
}
```

5.2.3 Υλοποίηση στοίβας με πίνακα



Εικόνα 5.7: Υλοποίηση στοίβας με πίνακα μεγέθους N. Η μεταβλητή n μετρά το πλήθος των στοιχείων της στοίβας.

Η υλοποίηση της στοίβας με πίνακα σταθερού μεγέθους είναι εξίσου απλή. Η μόνη πληροφορία που χρειάζεται να διατηρούμε είναι το πλήθος n των στοιχείων της στοίβας. Έτσι, όταν η στοίβα δεν είναι κενή, το στοιχείο που βρίσκεται στην κορυφή της είναι αποθηκευμένο στη θέση $A[n-1]$. Η ώθηση ενός αντικειμένου τοποθετεί το νέο αντικείμενο στη θέση $A[n]$ και, στη συνέχεια, αυξάνει κατά 1 την τιμή του n. Η απώθηση του στοιχείου που βρίσκεται στην κορυφή της στοίβας γίνεται μειώνοντας κατά 1 την τιμή του n και, στη συνέχεια, θέτουμε null το περιεχόμενο της θέσης $A[n]$. Αυτό είναι απαραίτητο, όταν ο πίνακας A αποθηκεύει αναφορές σε αντικείμενα, έτσι ώστε το σύστημα της Java να γνωρίζει ότι η αντίστοιχη θέση μνήμης του αντικειμένου που απωθείται από τη στοίβα μπορεί να απελευθερωθεί.

```
/* υλοποίηση στοίβας με πίνακα σταθερού μεγέθους */
public class ArrayStack<Item> {

   private int n;      // πλήθος στοιχείων στη στοίβα
   private Item[] A;   // πίνακας που υλοποιεί τη στοίβα

   /* κατασκευή στοίβας χωρητικότητας N στοιχείων */
   ArrayStack(int N) {
      private Item[] A = (Item[]) new Object[N];
      n = 0;
   }

   /* πλήθος στοιχείων στη στοίβα */
   public int size() {
      return n;
   }

   /* έλεγχος αν η στοίβα είναι κενή */
   public boolean isEmpty() {
      return n == 0;
   }

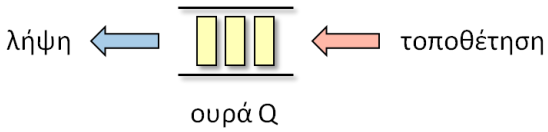
   /* ώθηση στοιχείου στην κορυφή της στοίβας */
   public void push(Item item) {
      A[n++] = item;
   }

   /* απώθηση του στοιχείου της κορυφής της στοίβας */
   public Item pop() {
      Item item = A[--n];
      A[n] = null;
      return item;
   }

   /* στοιχείο της κορυφής της στοίβας */
   public Item top() {
      return first.item;
   }
}
```

Όταν το μέγιστο μέγεθος της στοίβας δεν είναι γνωστό εξαρχής, μπορούμε να χρησιμοποιήσουμε τη μέθοδο των δυναμικών πινάκων. Οι μόνες αλλαγές στον παραπάνω κώδικα είναι η προσθήκη της μεθόδου resize(), που είδαμε στο Κεφάλαιο 2, η οποία καλείται από την push() πριν από την ώθηση του νέου στοιχείου, όταν το πλήθος n των στοιχείων της στοίβας είναι ίσο με το μέγεθος του πίνακα, και από την pop() μετά την απώθηση του στοιχείου στην κορυφή της στοίβας, όταν $n == (A.\text{length}/4)$.

5.3 Ουρά



Η ουρά είναι η τρίτη βασική δομή δεδομένων που εξετάζουμε σε αυτό το κεφάλαιο. Διατηρεί, όπως και η στοίβα, ένα σύνολο αντικειμένων σε σειρά εισαγωγής, αλλά επιτρέπει τη γρήγορη

διαγραφή του παλαιότερου στοιχείου. Συγκεκριμένα, μια ουρά Q υποστηρίζει τις παρακάτω βασικές λειτουργίες:

κατασκευή() :	Επιστρέφει μια κενή ουρά.
τοποθέτηση(x) :	Εισάγει το στοιχείο x στην αρχή της Q.
λήψη() :	Διαγράφει από την Q το στοιχείο x που βρίσκεται στο τέλος της Q και επιστρέφει το x.
πλήθος() :	Επιστρέφει το πλήθος των στοιχείων της Q.
είναι κενή() :	Ελέγχει αν η Q είναι κενή.

Και στην περίπτωση της ουράς θα αναπτύξουμε δύο αποδοτικές λύσεις με συνδεδεμένες λίστες και πίνακες.

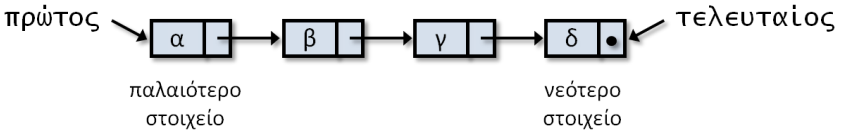
Πίνακας 5.3: Χρόνοι εκτέλεσης των λειτουργιών μιας ουράς Q με n στοιχεία, υλοποιημένης με στοιχειώδεις δομές. Για την υλοποίηση με λίστα ή με πίνακα σταθερού μεγέθους οι χρόνοι είναι χειρότερης περίπτωσης. Για την υλοποίηση με δυναμικό πίνακα οι χρόνοι είναι αντισταθμιστικοί.

	τοποθέτηση	λήψη	πλήθος
πίνακας	$O(1)$	$O(1)$	$O(1)$
απλά συνδεδεμένη λίστα	$O(1)$	$O(1)$	$O(1)$

Η διασύνδεση της ουράς έχει την παρακάτω μορφή:

```
class Queue<Item>      // ουρά αντικειμένων γενικού τύπου Item
{
   Queue(int);         // αρχικοποίηση ουράς με δεδομένη χωρητικότητα
   Queue();            // αρχικοποίηση κενής στοίβας
   void put(Item);     // τοποθέτηση αντικειμένου
   Item get();         // λήψη αντικειμένου
   int size();         // πλήθος στοιχείων στην ουρά
   boolean isEmpty();  // έλεγχος αν η ουρά είναι άδεια
}
```

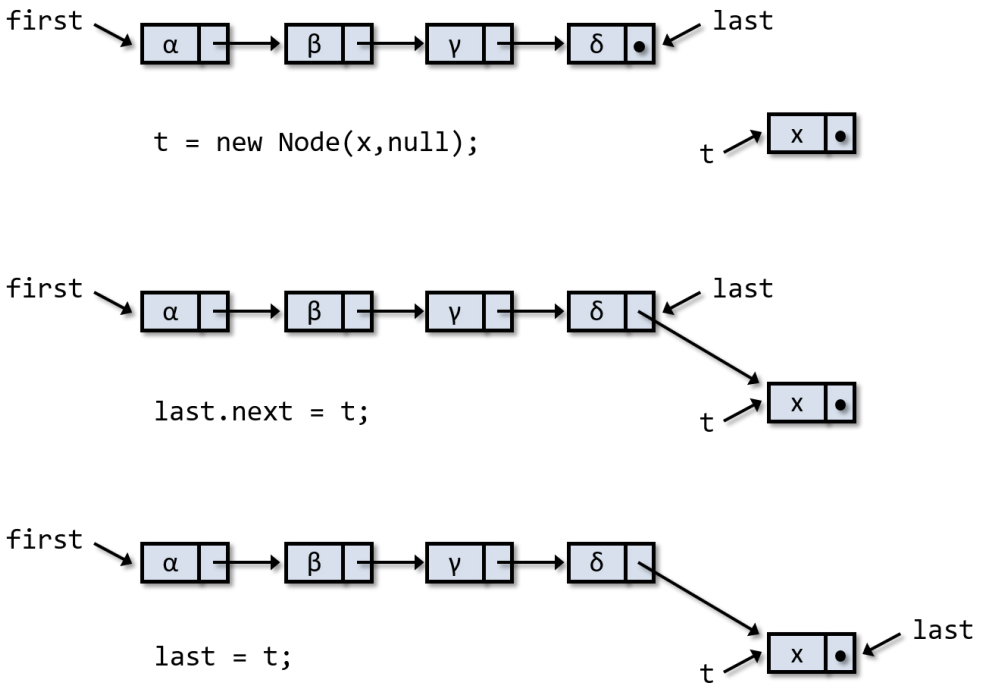
5.3.1 Υλοποίηση ουράς με συνδεδεμένη λίστα



Εικόνα 5.8: Υλοποίηση ουράς με απλά συνδεδεμένη λίστα. Η σειρά εισαγωγής των στοιχείων είναι α, β, γ, δ. Το παλαιότερο στοιχείο βρίσκεται στον πρώτο κόμβο της λίστας, ενώ το νεότερο στον τελευταίο.

Μια απλά συνδεδεμένη λίστα, αρκεί για να επιτύχουμε σταθερό χρόνο εκτέλεσης για κάθε λειτουργία της ουράς. Σε αντίθεση με την υλοποίηση της στοίβας με συνδεδεμένη λίστα, εδώ τοποθετούμε ένα νέο στοιχείο στο τέλος της λίστας, έτσι ώστε το παλαιότερο στοιχείο να βρίσκεται στην αρχή και να μπορεί να διαγραφεί γρήγορα. Με αυτόν τον τρόπο, η μέθοδος

get() είναι ακριβώς ίδια με την μέθοδο pop() της υλοποίησης της στοίβας με συνδεδεμένη λίστα, όπως φαίνεται στην Εικόνα 5.6, όπου η πρόσβαση στο πρώτο στοιχείο της λίστας γίνεται μέσω της αναφοράς first. Για να επιτρέψουμε τη γρήγορη τοποθέτηση νέων στοιχείων στην ουρά διατηρούμε επιπλέον μια αναφορά last στον τελευταίο κόμβο της λίστας. Έτσι, η τοποθέτηση ενός νέου στοιχείου x γίνεται ως εξής: δημιουργούμε ένα νέο κόμβο t όπου αποθηκεύουμε το στοιχείο x, θέτουμε null το πεδίο t.next. Επιπλέον, αν η ουρά δεν είναι κενή, τότε αποθηκεύουμε την αναφορά t στο πεδίο last.next. Τέλος, θέτουμε τη μεταβλητή last να αναφέρεται στον κόμβο t.



Εικόνα 5.9: Τοποθέτηση του στοιχείου x στο τέλος της ουράς της Εικόνα 5.8. Το στοιχείο αποθηκεύεται σε ένα νέο κόμβο ο οποίος τοποθετείται στο τέλος της λίστας.

```
/* υλοποίηση ουράς με απλά συνδεδεμένη λίστα */
public class ListQueue<Item> {

   private int n;   // πλήθος στοιχείων στην ουρά

   /* τύπος κόμβου συνδεδεμένης λίστας */
   private class Node {
      Item item;
      Node next;
      Node(Item item, Node next) {
          this.item = item;
          this.next = next;
      }
   }

   private Node first, last;  // πρώτος και τελευταίος κόμβος της λίστας

   /* κατασκευή κενής ουράς */
   ListQueue() {
      first = last = null;
      n = 0;
   }
```

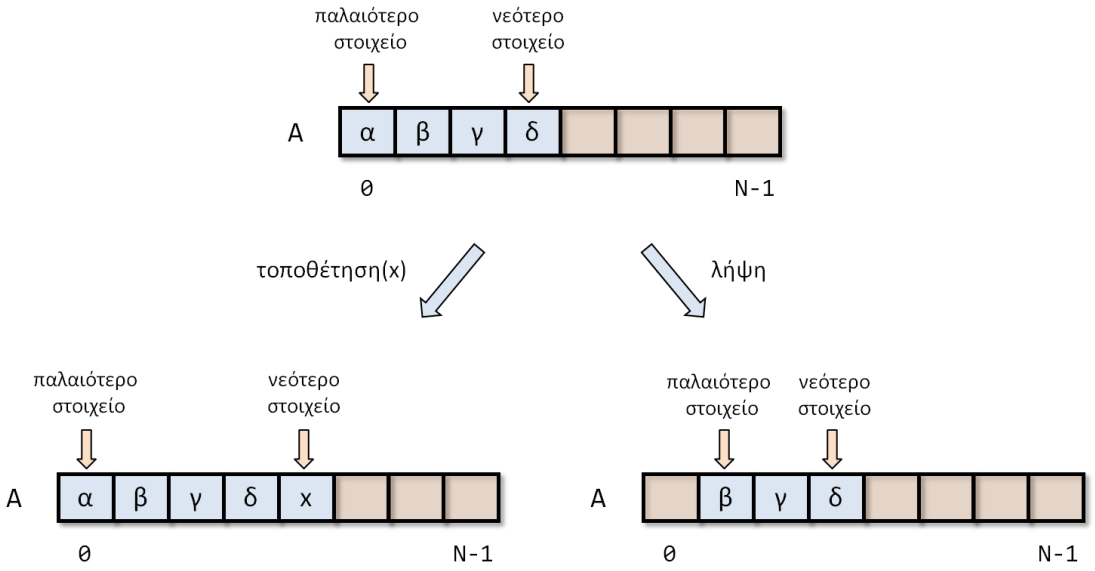
```
    /* πλήθος στοιχείων στην ουρά */
    public int size() {
       return n;
    }

    /* έλεγχος αν η ουρά είναι κενή */
    public boolean isEmpty() {
       return n == 0;
    }

    /* τοποθέτηση στοιχείου στο τέλος της ουράς */
    public void put(Item item) {
       Node t = last;
       last = new Node(item,null);
       if (isEmpty()) first = last; else t.next = first;
       n++;
    }

    /* λήψη του πρώτου στοιχείου της ουράς */
    public Item get() {
       Node t = first;
       first = t.next;
       n--;
       return t.item;
    }
}
```

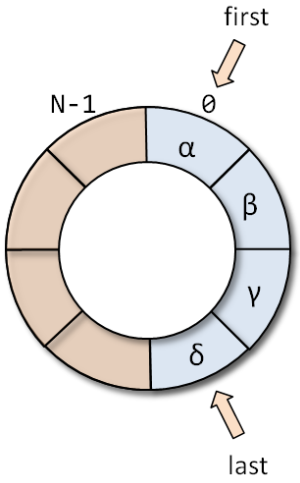
5.3.3 Υλοποίηση ουράς με πίνακα



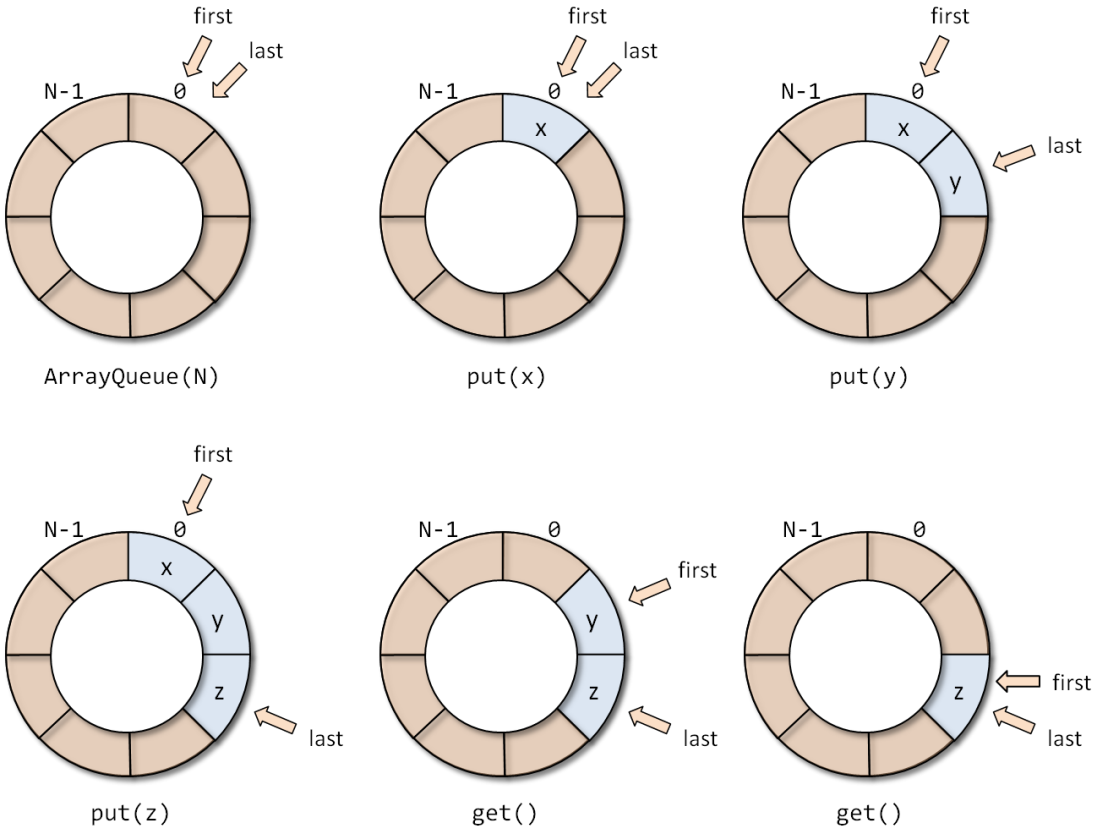
Εικόνα 5.10: Υλοποίηση ουράς με πίνακα μεγέθους N. Διατηρούμε δύο μεταβλητές, οι οποίες δίνουν τη θέση του παλαιότερου και τη θέση του νεότερου στοιχείου της ουράς, αντίστοιχα. Τα στοιχεία τοποθετούνται σε διαδοχικές θέσεις του πίνακα, από το παλαιότερο προς το νεότερο.

Για να δώσουμε μια αποδοτική υλοποίηση της ουράς με πίνακα σταθερού μεγέθους, αποθηκεύουμε στον πίνακα τα στοιχεία της ουράς διατεταγμένα ως προς τη σειρά εισαγωγής τους. Διατηρούμε, επίσης, δύο μεταβλητές, first και last, οι οποίες δίνουν τη θέση του παλαιότερου και τη θέση του νεότερου στοιχείου της ουράς, αντίστοιχα. Για τη λήψη του

παλαιότερου στοιχείου, επιστρέφουμε το στοιχείο του πίνακα A[first], θέτουμε null το περιεχόμενο της θέσης A[first] και αυξάνουμε κατά 1 την τιμή της μεταβλητής first. Η τοποθέτηση ενός στοιχείου αυξάνει κατά 1 την τιμή της μεταβλητής last και τοποθετεί το νέο αντικείμενο στη θέση A[last].



Εικόνα 5.11: Υλοποίηση ουράς με αναδιπλωμένο πίνακα μεγέθους N. Διατηρούμε δύο μεταβλητές οι οποίες δίνουν τη θέση του παλαιότερου και τη θέση του νεότερου στοιχείου της ουράς, αντίστοιχα. Τα στοιχεία τοποθετούνται κυκλικά σε διαδοχικές θέσεις του πίνακα, από το παλαιότερο προς το νεότερο.



Εικόνα 5.12: Παράδειγμα εκτέλεσης μιας ακολουθίας λειτουργιών σε ουρά η οποία είναι υλοποιημένη με αναδιπλωμένο πίνακα μεγέθους N.

Η υλοποίησή μας διατηρεί τις ακόλουθες αναλλοίωτες συνθήκες:

1. Οι μεταβλητές first και last λαμβάνουν τιμές στο διάστημα $[0, N-1]$.
2. Αν η ουρά είναι άδεια, τότε $n == 0$ και first $==$ last.
3. Αν ουρά έχει $n \geq 1$ στοιχεία, τότε αυτά βρίσκονται διαδοχικά, από το παλαιότερο προς το νεότερο, στις θέσεις first, first $+ 1, \ldots,$ first $+ (n-1)$ του πίνακα A, όπου οι προσθέσεις γίνονται $\bmod N$ και $(\text{first} + (n-1)) \bmod N == \text{last}$.

```
/* υλοποίηση στοίβας με αναδιπλωμένο πίνακα σταθερού μεγέθους */
public class ArrayQueue<Item> {

   private int n;      // πλήθος στοιχείων στην ουρά
   private Item[] A;   // πίνακας που υλοποιεί την ουρά

   /* κατασκευή ουράς χωρητικότητας N στοιχείων */
   ArrayQueue(int N) {
      private Item[] A = (Item[]) new Object[N];
      n = 0;
   }

   /* πλήθος στοιχείων στην ουρά */
   public int size() {
      return n;
   }

   /* έλεγχος αν η ουρά είναι κενή */
   public boolean isEmpty() {
      return n == 0;
   }

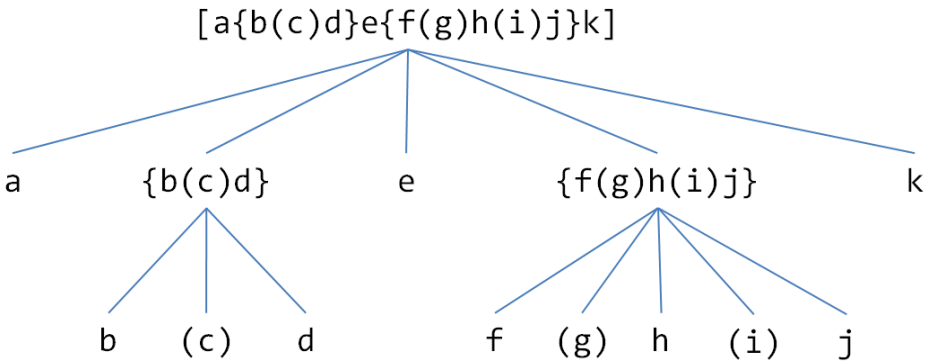
   /* τοποθέτηση στοιχείου στο τέλος της ουράς */
   public void put(Item item) {
      last = (last +1) % N;
      A[last] = item;
      n++;
   }

   /* λήψη στοιχείου από την αρχή της ουράς */
   public Item get() {
      Item item = A[first];
      A[first] = null;
      --n;
      if (n > 0) first = (first + 1) % N;
      return item;
   }
}
```

5.4 Εφαρμογή: Συντακτική ανάλυση εκφράσεων

Σε αυτή την ενότητα θα περιγράψουμε μια εφαρμογή που αναλύει τη μορφή εκφράσεων με παρενθέσεις. Θα δώσουμε ένα πρόγραμμα το οποίο επιτελεί την ακόλουθη λειτουργία. Το πρόγραμμα δέχεται στην είσοδο μια έκφραση η οποία περιλαμβάνει παρενθέσεις τριών τύπων: ( ), [ ] και { }. Σκοπός του προγράμματος είναι να ελέγξει αν η ακολουθία έχει ορθή σύνταξη, δηλαδή αν οι παρενθέσεις είναι φωλιασμένες σωστά. Επιπλέον, εφόσον η ακολουθία είναι συντακτικά ορθή, πρέπει να αποθηκεύει τις θέσεις των αντίστοιχων αριστερών και δεξιών παρενθέσεων με τη σειρά με την οποία συναντώνται.

Για παράδειγμα, η ακολουθία [a { b (c) d } e { f (g) h (i) j } k] είναι έγκυρη και αναλύεται ως εξής:



Εικόνα 5.13: Συντακτική ανάλυση έκφρασης με παρενθέσεις.

Οι αντιστοιχίες των παρενθέσεων είναι: 4-6 ( ), 2-8 { }, 12-14 ( ), 16-18 ( ), 10-20 { }, 0-22 [ ]. Δηλαδή, η παρένθεση (στη θέση 4 της ακολουθίας αντιστοιχεί στην παρένθεση) στη θέση 6 της ακολουθίας, και ούτω καθεξής. Αντίθετα, ακολουθίες όπως { a (b } c),) a (, [a (b) και [a) b (c] δεν είναι έγκυρες.

Ο παραπάνω έλεγχος μπορεί να γίνει με τη βοήθεια μιας στοίβας, η οποία θα αποθηκεύει τις αριστερές παρενθέσεις μαζί με τον αριθμό των θέσεων που βρίσκονται στην ακολουθία. Η αποθήκευση των θέσεων των αντίστοιχων αριστερών και δεξιών παρενθέσεων θα γίνει σε μία ουρά. Συγκεκριμένα, διαβάζουμε διαδοχικά τους χαρακτήρες της ακολουθίας εισόδου, από τα αριστερά (θέση 0) προς τα δεξιά. Έστω ότι βρισκόμαστε στη θέση i όπου διαβάζουμε τον χαρακτήρα c. Τότε κάνουμε ένα από τα παρακάτω:

- Αν ο c δεν είναι παρένθεση, τότε τον αγνοούμε.
- Αν ο c είναι αριστερή παρένθεση, (, [ή {, τότε ωθούμε στη στοίβα το ζεύγος $\langle c, i \rangle$.
- Αν ο c είναι δεξιά παρένθεση,),], ή }, τότε απωθούμε από τη στοίβα το ζεύγος $\langle d, j \rangle$, που βρίσκεται στην κορυφή της. Ο χαρακτήρας d είναι αριστερή παρένθεση (αφού μόνο αυτοί οι χαρακτήρες ωθούνται στη στοίβα), οπότε ελέγχουμε αν είναι του ίδιου τύπου με τον c, π.χ., $d = \{$ και $c = \}$. Αν είναι, τότε τοποθετούμε στο τέλος της ουράς το ζεύγος των αριθμών $\langle j, i \rangle$ που αντιστοιχούν στο διάστημα που καλύπτουν οι παρενθέσεις d και c. Διαφορετικά δηλώνουμε ότι υπάρχει συντακτικό λάθος και διακόπτουμε την εκτέλεση του προγράμματος.

```
/* συντακτική ανάλυση έκφρασης με παρενθέσεις */
public class SyntaxAnalysis {

   private class Pair {
      private Character c;   // χαρακτήρας
      private int p;         // θέση

      Pair(Character c, int p) {
         this.c = c; this.p = p;
      }

      public int getP() { return this.p; }
      public Character getC() { return this.c; }
   }

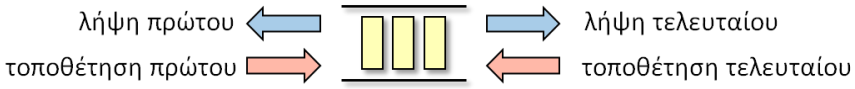
   private static ListQueue<Integer> ParseInput(String str) {
      ListStack<Pair> S    = new ListStack<Pair>();
      ListQueue<Integer> Q = new ListQueue<Integer>();
```

```
      Pair p;

       for (int i=0; i<str.length; i++) {
          switch ( str.charAt(i) ) {
             case '(' :
             case '{' :
             case '[' :   p = new Pair(,i);
                          S.push(p); break;
             case ')' :   if ( S.isEmpty() ) {
                             System.out.println("Syntax error! ");
                             return null;
                          }
                          p = S.pop();
                          if ( p.getC() != '(') {
                               System.out.println("Syntax error!");
                               Return null;
                          }
                          Q.put(p.getP());
                          Q.put(i);
                          break;
              case ']' :  if ( S.isEmpty() ) {
                             System.out.println("Syntax error!");
                             Return null;
                          }
                          p = S.pop();
                          if ( p.getC() != '[') {
                            System.out.println("Syntax error!");
                            Return null;
                          }
                          Q.put(p.getP());
                          Q.put(i);
                          break;
              case '}' :  if ( S.isEmpty() ) {
                             System.out.println("Syntax error!");
                             return null;
                          }
                          p = S.pop();
                          if ( p.getC() != '{') {
                             System.out.println("Syntax error!");
                             return null;
                          }
                          Q.put(p.getP());
                          Q.put(i);
                          break;
              default :   break;
          }
       }

       if ( !S.isEmpty() ) {
           System.out.println("Syntax error!");
           return null;
       }
       else return Q;
    }
}
```

5.4 Γενικευμένες Ουρές



ουρά δύο άκρων DQ

Ορισμένες φορές χρειαζόμαστε τη δυνατότητα να διατηρούμε τα στοιχεία ενός συνόλου σε μια διάταξη, όπου είναι δυνατή η εισαγωγή και διαγραφή στοιχείων τόσο από την αρχή όσο και από το τέλος. Για παράδειγμα, τέτοιες λειτουργίες χρειάζονται για την υλοποίηση συγκεκριμένων αλγόριθμων χρονοδρομολόγησης εργασιών σε παράλληλα συστήματα. Η ουρά δύο άκρων (double-ended queue, ή dequeue) αποτελεί μια άμεση γενίκευση των δομών στοίβας και ουράς, η οποία παρέχει αυτή τη δυνατότητα. Συγκεκριμένα, μια ουρά δύο άκρων DQ υποστηρίζει τις παρακάτω βασικές λειτουργίες:

κατασκευή() :	Επιστρέφει μια κενή ουρά δύο άκρων.
τοποθέτηση πρώτου(x) :	Εισάγει το στοιχείο x στην αρχή της DQ.
τοποθέτηση τελευταίου(x) :	Εισάγει το στοιχείο x στο τέλος της DQ.
λήψη πρώτου() :	Διαγράφει από την DQ το στοιχείο x που βρίσκεται στην αρχή της DQ και επιστρέφει το x.
λήψη τελευταίου() :	Διαγράφει από την DQ το στοιχείο x που βρίσκεται στο τέλος της DQ και επιστρέφει το x.
πλήθος() :	Επιστρέφει το πλήθος των στοιχείων της DQ.
είναι κενή() :	Ελέγχει αν η DQ είναι κενή.

Στη συνέχεια, θα περιγράψουμε μια αποδοτική λύση με χρήση διπλά συνδεδεμένης λίστας. Μπορούμε, επίσης, να προσαρμόσουμε την υλοποίηση ουράς με πίνακα, έτσι ώστε να υποστηρίζονται οι παραπάνω λειτουργίες.

Πίνακας 5.4: Χρόνοι εκτέλεσης των λειτουργιών μιας ουράς δύο άκρων DQ με n στοιχεία, υλοποιημένης με στοιχειώδεις δομές. Για την υλοποίηση με λίστα ή με πίνακα σταθερού μεγέθους οι χρόνοι είναι χειρότερης περίπτωσης. Για την υλοποίηση με δυναμικό πίνακα, οι χρόνοι είναι αντισταθμιστικοί.

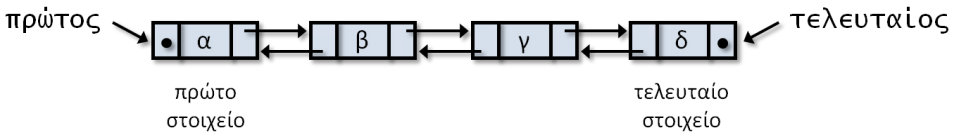
	τοποθέτηση πρώτου/τελευταίου	λήψη πρώτου/τελευταίου	πλήθος
πίνακας	$O(1)$	$O(1)$	$O(1)$
διπλά συνδεδεμένη λίστα	$O(1)$	$O(1)$	$O(1)$

Η διασύνδεση της ουράς δύο άκρων έχει την παρακάτω μορφή:

```
class Dequeue<Item>         // ουρά δύο άκρων με αντικείμενα γενικού τύπου Item
{
   Dequeue(int);            // αρχικοποίηση ουράς δύο άκρων με δεδομένη χωρητικότητα
   Dequeue();               // αρχικοποίηση κενής ουράς δύο άκρων
   void putFirst(Item);     // τοποθέτηση αντικειμένου στην αρχή της ουράς
   void putLast(Item);      // τοποθέτηση αντικειμένου στο τέλος της ουράς
   Item getFirst();         // λήψη αντικειμένου από την αρχή της ουράς
```

```
   Item getLast();         // λήψη αντικειμένου από το τέλος της ουράς
   int size();             // πλήθος στοιχείων στην ουρά
   boolean isEmpty();      // έλεγχος αν η ουρά είναι κενή
}
```

5.4.1 Υλοποίηση ουράς δύο άκρων με συνδεδεμένη λίστα



Εικόνα 5.14: Υλοποίηση ουράς δύο άκρων με διπλά συνδεδεμένη λίστα. Η σειρά εισαγωγής των στοιχείων που δίνει την παραπάνω λίστα δεν είναι μοναδική. Π.χ. θα μπορούσαμε να έχουμε εισαγωγή των β και α στην αρχή της λίστας και των δ και γ στο τέλος της.

```
/* υλοποίηση ουράς δύο άκρων με διπλά συνδεδεμένη λίστα */
public class ListDequeue<Item> {

   private int n;   // πλήθος στοιχείων στην ουρά

   /* τύπος κόμβου συνδεδεμένης λίστας */
   private class Node {
      Item item;
      Node previous;  // προηγούμενος κόμβος στη λίστα
      Node next;      // επόμενος κόμβος στη λίστα
      Node(Item item, Node previous, Node next) {
          this.item = item;
          this.previous = previous;
          this.next = next;
      }
   }

   private Node first, last;  // πρώτος και τελευταίος κόμβος της λίστας

   /* κατασκευή κενής ουράς */
   ListDequeue() {
      first = last = null;
      n = 0;
   }

   /* πλήθος στοιχείων στην ουρά */
   public int size() {
      return n;
   }

   /* έλεγχος αν η ουρά είναι κενή */
   public boolean isEmpty() {
      return n == 0;
   }

   /* τοποθέτηση στοιχείου στην αρχή της ουράς */
   public void putFirst(Item item) {
      Node t = first;
      first = new Node(item,null,first);
      if (isEmpty()) first = last; else t.previous = first;
      n++;
   }

   /* τοποθέτηση στοιχείου στο τέλος της ουράς */
   public void putLast(Item item) {
      Node t = last;
```

```
            last = new Node(item,last,null);
            if (isEmpty()) first = last; else t.next = last;
            n++;
        }

        /* λήψη του πρώτου στοιχείου της ουράς */
        public Item get() {
            Node t = first;
            first = t.next;
            first.previous = null;
            n--;
            return t.item;
        }

        /* λήψη του τελευταίου στοιχείου της ουράς */
        public Item getLast() {
            Node t = last;
            last = t.previous;
            last.next = null;
            n--;
            return t.item;
        }

    }
```

5.5 Παρατηρήσεις

Περιγράψαμε απλές δομές δεδομένων οι οποίες υλοποιούν με αποδοτικό τρόπο τους αφηρημένους τύπους δεδομένων της συλλογής, της στοίβας και της ουράς. Οι υλοποιήσεις τόσο με συνδεδεμένες λίστες όσο και με πίνακες επιτυγχάνουν σταθερό χρόνο ανά λειτουργία, ωστόσο η χρήση πινάκων είναι αρκετά πιο γρήγορη στην πράξη.

Ασκήσεις

5.1 Περιγράψτε μια αποδοτική υλοποίηση της δομής δεδομένων συλλογής με δυναμικούς πίνακες.

5.2 Συμπληρώστε την υλοποίηση μιας στοίβας και μιας ουράς με δυναμικούς πίνακες.

5.3 Έστω ότι εκτελούμε μια μεικτή ακολουθία από ωθήσεις και απωθήσεις σε μια αρχική στοίβα, όπου οι ωθήσεις τοποθετούν τους ακέραιους από 0 έως και 5 σε αύξουσα σειρά, ενώ η στοίβα μένει κενή μετά το πέρας της ακολουθίας. Π.χ., μια τέτοια ακολουθία είναι η ώθηση(0), ώθηση(1), ώθηση(2), απώθηση(), απώθηση(), ώθηση(3), απώθηση(), ώθηση(4), ώθηση(5), απώθηση(), απώθηση(), απώθηση(). Έστω ότι τυπώνουμε τα στοιχεία με τη σειρά που απωθούνται από τη στοίβα. Ποιες από τις παρακάτω ακολουθίες είναι αδύνατο να εμφανιστούν;

α) 0, 1, 2, 3, 4, 5
β) 5, 4, 3, 2, 1, 0
γ) 0, 2, 4, 1, 3, 5
δ) 0, 2, 4, 3, 5, 1
ε) 3, 4, 2, 0, 1, 5

5.4 Ας θεωρήσουμε τώρα τη γενίκευση της διαδικασίας που περιγράψαμε στην προηγούμενη άσκηση. Έχουμε ένα πρόγραμμα το οποίο εκτελεί μια μεικτή ακολουθία από ωθήσεις και

απωθήσεις σε μια αρχική στοίβα, όπου οι ωθήσεις τοποθετούν τους ακέραιους από 0 έως και $N-1$ σε αύξουσα σειρά. Όπως και πριν, η στοίβα μένει κενή στο τέλος της ακολουθίας, ενώ τυπώνουμε τους ακέραιους με τη σειρά απώθησης τους. Από την Άσκηση 5.3 συνάγουμε το συμπέρασμα ότι το πρόγραμμα μας δεν μπορεί να δώσει όλες τις δυνατές μεταθέσεις των N ακέραιων. Περιγράψτε έναν αποδοτικό αλγόριθμο ο οποίος να ελέγχει αν μια δοθείσα μετάθεση μπορεί να κατασκευαστεί από το πρόγραμμα αυτό.

5.5 Ας θεωρήσουμε το πρόβλημα υπολογισμού της τιμής αριθμητικών παραστάσεων, οι οποίες αποτελούνται από αριθμούς, παρενθέσεις και τελεστές από το σύνολο $\{+,-,\times,\div\}$. Υποθέτουμε ότι έχουμε πλήρη χρήση παρενθέσεων, δηλαδή κάθε αριθμητική παράσταση είναι ένας αριθμός ή είναι παράσταση της μορφής (A⊙B), όπου A και B αριθμητικές παραστάσεις και ⊙ ένας τελεστής από το σύνολο $\{+,-,\times,\div\}$. Για παράδειγμα, η ακόλουθη αριθμητική παράσταση έχει την παραπάνω μορφή

$$\big(\,(\,(\,(9+5)\div(9-5)\,)\times 3\,)+(7\times 2)\big).$$

Περιγράψτε έναν αποδοτικό αλγόριθμο υπολογισμού αριθμητικών παραστάσεων αυτής της μορφής.

Υπόδειξη: Μπορείτε να κάνετε χρήση δύο δομών στοίβας, μία για αριθμούς και μία για τελεστές.

5.6 Ένα πρόγραμμα επεξεργασίας κειμένου χρησιμοποιεί μια δομή δεδομένων buffer, της οποίας η διασύνδεση δίνεται από τον παρακάτω αφηρημένο τύπο δεδομένων :

buffer()	δημιουργεί μια κενή δομή τύπου buffer
void insert(char c)	εισάγει το χαρακτήρα c στην τρέχουσα θέση του buffer
char delete()	διαγράφει και επιστρέφει το χαρακτήρα της τρέχουσας θέσης του buffer
void left(int k)	μετακινεί την τρέχουσα θέση του buffer κατά k θέσεις αριστερά
void right(int k)	μετακινεί την τρέχουσα θέση του buffer κατά k θέσεις δεξιά
int size()	επιστρέφει τον αριθμό των χαρακτήρων του buffer

Δώστε μια αποδοτική υλοποίηση του παραπάνω αφηρημένου τύπου δεδομένων.

Υπόδειξη: Μπορείτε να κάνετε χρήση δύο δομών στοίβας.

5.7 Περιγράψτε μια υλοποίηση του αφηρημένου τύπου δεδομένων της στοίβας με τη χρήση δύο ουρών. Ποιος είναι χρόνος εκτέλεσης της ώθησης και της απώθησης ενός αντικειμένου στη δομή που προτείνετε;

5.8 Μια *τυχαιοποιημένη ουρά* είναι παρόμοια με την ουρά που έχουμε δει στην Ενότητα 5.3, με τη διαφορά ότι το στοιχείο που επιστρέφει και διαγράφει από την ουρά επιλέγεται με ίση πιθανότητα (ομοιόμορφα τυχαία) από όλα τα στοιχεία που περιέχει η ουρά. Θέλουμε να σχεδιάσουμε μια δομή τυχαιοποιημένης ουράς randQueue Q η οποία να υποστηρίζει τις παρακάτω λειτουργίες:

randQueue(int N)	δημιουργεί μια κενή τυχαιοποιημένη ουρά μεγέθους N
void put(Item item)	εισάγει το στοιχείο item στην ουρά Q

get( ) επιστρέφει ένα ομοιόμορφα τυχαία επιλεγμένο στοιχείο item της ουράς Q το οποίο διαγράφεται από την Q

isEmpty( ) επιστρέφει true, αν η Q είναι άδεια, διαφορετικά επιστρέφει false

Δώστε μια αποδοτική υλοποίηση της δομής randQueue με πίνακα. Για τη λειτουργία get() υποθέτουμε ότι έχουμε διαθέσιμη μια συνάρτηση rand(a, b) η οποία επιλέγει ομοιόμορφα τυχαία έναν ακέραιο στο διάστημα $[a, b]$, όπου a και b ακέραιοι με $a \leq b$.

Υπόδειξη: Μπορούμε να υποστηρίζουμε όλες τις παραπάνω λειτουργίες σε σταθερό χρόνο χειρότερης περίπτωσης.

5.9 Περιγράψτε μια αποδοτική υλοποίηση μιας ουράς δύο άκρων με πίνακα μεταβλητού μεγέθους.

Υπόδειξη: Προσαρμόστε την υλοποίηση της απλής ουράς με πίνακα.

5.10 Υλοποιήστε δύο δομές στοίβας χρησιμοποιώντας μία μόνο ουρά δύο άκρων. Κάθε λειτουργία θα πρέπει να εκτελείται με σταθερό πλήθος λειτουργιών της ουράς δύο άκρων.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Κεφάλαιο 6
Ουρές Προτεραιότητας

Περιεχόμενα

6.1 Ο αφηρημένος τύπος δεδομένων ουράς προτεραιότητας 114
6.2 Ουρές προτεραιότητας με στοιχειώδεις δομές δεδομένων 115
6.3 Δυαδικός σωρός 116
6.3.1 Υλοποίηση σε Java 125
6.3.2 Κατασκευή δυαδικού σωρού με δεδομένα κλειδιά 126
6.4 δ-Σωρός 128
6.5 Ταξινόμηση με ουρά προτεραιότητας 129
6.6 Ουρές προτεραιότητας με ευρετήριο 130
Ασκήσεις 131
Βιβλιογραφία 132

6.1 Ο αφηρημένος τύπος δεδομένων ουράς προτεραιότητας

Η ουρά προτεραιότητας αποτελεί γενίκευση των δομών της στοίβας και της ουράς, υπό την έννοια ότι επιτρέπουν τη γρήγορη πρόσβαση σε στοιχεία όχι με βάση τη χρονική σειρά εισαγωγής αλλά με βάση τα κλειδιά που υποδηλώνουν την προτεραιότητα του κάθε στοιχείου. Είναι μια εξαιρετικά χρήσιμη δομή δεδομένων με πολλές εφαρμογές, όπως π.χ. στην ταξινόμηση και σε προσομοίωση συστημάτων διακριτών γεγονότων.

Μια ουρά προτεραιότητας PQ διατηρεί ένα σύνολο στοιχείων με κλειδιά και υποστηρίζει τις παρακάτω βασικές λειτουργίες:

κατασκευή() :	Επιστρέφει μια κενή ουρά προτεραιότητας.
εισαγωγή(x, k) :	Εισάγει στην PQ ένα στοιχείο x με κλειδί k.
εύρεση μέγιστου() :	Επιστρέφει ένα στοιχείο με μέγιστο κλειδί.
διαγραφή μέγιστου() :	Διαγράφει από την PQ ένα στοιχείο x με μέγιστο κλειδί και επιστρέφει το x .
πλήθος() :	Επιστρέφει το πλήθος των στοιχείων της PQ.
είναι κενή() :	Ελέγχει αν η PQ είναι κενή.

```
// συλλογή στοιχείων γενικού τύπου Item με κλειδιά τύπου Key
class MaxPriorityQueue<Key extends Comparable<Key>, Item>
{
   MaxPriorityQueue();      // αρχικοποίηση κενής ουράς προτεραιότητας
   boolean isEmpty();       // έλεγχος αν η ουρά προτεραιότητας είναι άδεια
   int size();              // πλήθος στοιχείων στην ουρά προτεραιότητας
   void insert(Item);       // εισαγωγή αντικειμένου στην ουρά προτεραιότητας
   Item findMax();          // επιστρέφει ένα στοιχείο με μέγιστο κλειδί
   Item deleteMax();        // διαγράφει από τη δομή ένα στοιχείο με μέγιστο κλειδί
                            // και το επιστρέφει
}
```

Μέσω των παραπάνω βασικών λειτουργιών μπορούμε να υποστηρίξουμε ορισμένες επιπρόσθετες λειτουργίες, όπως οι παρακάτω:

αλλαγή κλειδιού(Item item, Key key) : Αναθέτει στο στοιχείο item το κλειδί key.

διαγραφή(Item item) : Διαγράφει από την PQ το στοιχείο item.

Σε ορισμένες εφαρμογές είναι χρήσιμη η δυνατότητα ένωσης δύο ουρών προτεραιότητας. Για το σκοπό αυτό μπορούμε να ορίσουμε την παρακάτω λειτουργία για μία ουρά προτεραιότητας PQ:

ένωση(PQ′) : Επιστρέφει μια νέα ουρά προτεραιότητας, η οποία προκύπτει από την ένωση των στοιχείων των ουρών προτεραιότητας PQ και PQ′. Η λειτουργία αυτή καταστρέφει την PQ′ και η νέα ουρά προτεραιότητας παίρνει τη θέση της PQ.

6.2 Ουρές προτεραιότητας με στοιχειώδεις δομές δεδομένων

Είναι εύκολο να διαπιστώσουμε ότι σε μια απλή υλοποίηση ουράς προτεραιότητας, με πίνακα ή λίστα, θα έχουμε τουλάχιστον μια λειτουργία η οποία δεν μπορεί να υποστηριχθεί αποδοτικά. Δείτε τον Πίνακας 6.1. Για παράδειγμα, σε ένα διατεταγμένο πίνακα μπορούμε να βρούμε άμεσα το μέγιστο στοιχείο, αλλά η εισαγωγή ενός νέου στοιχείου απαιτεί $O(n)$ χρόνο στη χειρότερη περίπτωση. Αντίστοιχα, αν ο πίνακας δεν είναι διατεταγμένος, τότε η εισαγωγή γίνεται εύκολα, αλλά η εύρεση του μέγιστου στοιχείου απαιτεί την εξέταση ολόκληρου του πίνακα. Ανάλογες παρατηρήσεις ισχύουν και για τις υλοποιήσεις με συνδεδεμένη λίστα.

Πίνακας 6.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης μερικών βασικών λειτουργιών ουράς προτεραιότητας PQ με n στοιχεία, υλοποιημένης με στοιχειώδεις δομές.

	εισαγωγή	εύρεση μέγιστου	διαγραφή μέγιστου
μη διατεταγμένος πίνακας	$O(1)$	$O(n)$	$O(n)$
διατεταγμένος πίνακας	$O(n)$	$O(1)$	$O(1)$
μη διατεταγμένη λίστα	$O(1)$	$O(n)$	$O(n)$
διατεταγμένη λίστα	$O(n)$	$O(1)$	$O(1)$

Από τον παραπάνω πίνακα, συμπεραίνουμε ότι χρησιμοποιώντας οποιαδήποτε από αυτές τις απλοϊκές υλοποιήσεις, η εκτέλεση μιας μεικτής ακολουθίας n εισαγωγών και n διαγραφών μέγιστου θα χρειαστεί $O(n^2)$ χρόνο στη χειρότερη περίπτωση. Ένας τέτοιος χρόνος εκτέλεσης

μπορεί να είναι απαγορευτικά μεγάλος σε πολλές εφαρμογές. Θα πρέπει, λοιπόν, να αναζητήσουμε πιο αποδοτικές λύσεις.

6.3 Δυαδικός σωρός

Μια δομή δεδομένων η οποία μπορεί να χρησιμοποιηθεί για την υλοποίηση μιας ουράς προτεραιότητας είναι ο δυαδικός σωρός.

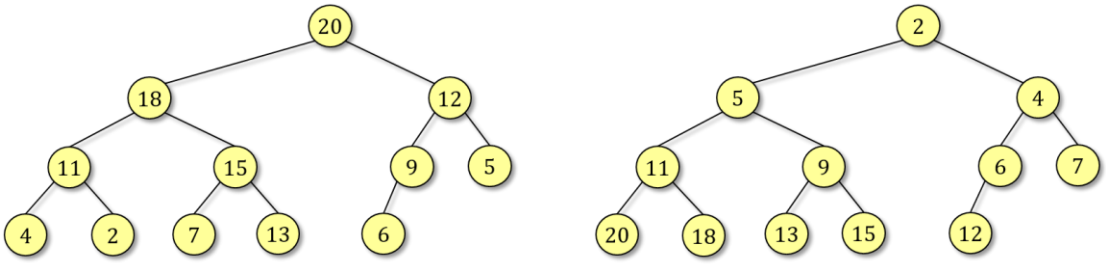
Ο δυαδικός σωρός είναι ένα πλήρες δυαδικό δένδρο, δηλαδή κάθε κόμβος του έχει ακριβώς δύο παιδιά, με εξαίρεση τους κόμβους του προτελευταίου επιπέδου, οι οποίοι μπορεί να έχουν κανένα, ένα ή δύο παιδιά, και οι κόμβοι του τελευταίου επιπέδου καταλαμβάνουν συνεχόμενες θέσεις από αριστερά προς τα δεξιά. Αυτό σημαίνει ότι, εάν ένας δυαδικός σωρός αποθηκεύει n κλειδιά και έχει ℓ επίπεδα, τότε ισχύει

$$2^{\ell-1} \leq n \leq 2^{\ell} - 1.$$

Οι παραπάνω ανισότητες προκύπτουν από το γεγονός ότι τα επίπεδα από 0 έως και $\ell - 2$ έχουν ακριβώς $2^0 + 2^1 + 2^2 + \cdots + 2^{\ell-2} = 2^{\ell-1} - 1$ κόμβους, ενώ το τελευταίο επίπεδο, $\ell - 1$, έχει από 1 έως $2^{\ell-1}$ κόμβους. Έτσι, έχουμε ότι το πλήθος των επιπέδων του σωρού είναι $\ell \leq \lfloor \lg n \rfloor + 1$ και, άρα, το ύψος h του σωρού είναι $h \leq \lfloor \lg n \rfloor$.

Αναλλοίωτη συνθήκη δυαδικού σωρού μέγιστου: Οποιοσδήποτε κόμβος του δυαδικού σωρού έχει κλειδί μικρότερο ή ίσο του κλειδιού του γονέα του.

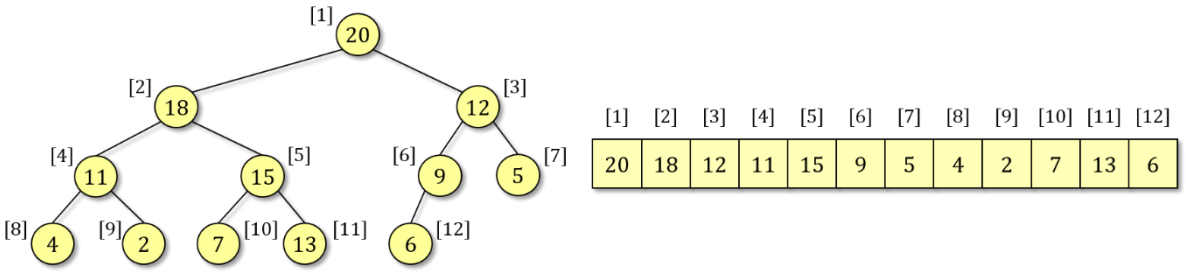
Αναλλοίωτη συνθήκη δυαδικού σωρού ελάχιστου: Οποιοσδήποτε κόμβος του δυαδικού σωρού έχει κλειδί μεγαλύτερο ή ίσο του κλειδιού του γονέα του.



Εικόνα 6.1: Δυαδικός σωρός μέγιστου (αριστερά) και δυαδικός σωρός ελάχιστου (δεξιά).

Παραδείγματα δυαδικών σωρών μέγιστου και ελάχιστου δίνονται στην Εικόνα 6.1. Στη συνέχεια της ενότητας αναπτύσσουμε τους αλγόριθμους χειρισμού ενός δυαδικού σωρού μέγιστου. Για το χειρισμό ενός δυαδικού σωρού ελάχιστου χρησιμοποιούμε ανάλογους αλγόριθμους, όπου η μόνη διαφορά είναι η φορά των συγκρίσεων. Δείτε την Άσκηση 6.1.

Η μορφή του δυαδικού σωρού μάς επιτρέπει να τον αναπαραστήσουμε με μόνο ένα πίνακα, όπου αντιστοιχίζουμε κάθε κόμβο του σωρού σε μια θέση του πίνακα ως εξής: Αριθμούμε τους κόμβους του σωρού σύμφωνα με την οριζόντια διερεύνηση του δένδρου, δηλαδή σε αύξουσα σειρά από το ένα, ξεκινώντας από το υψηλότερο επίπεδο και από τα αριστερά προς τα δεξιά. Ο κόμβος με αριθμό i αντιστοιχεί στη θέση i του πίνακα, όπως φαίνεται στην Εικόνα 6.2.



Εικόνα 6.2: Αναπαράσταση δυαδικού σωρού μέγιστου με πίνακα.

Αυτή η αντιστοιχία μάς επιτρέπει να βρίσκουμε άμεσα τις θέσεις του γονέα και των παιδιών οποιουδήποτε κόμβου.

Ιδιότητα 6.1. Στην αναπαράσταση ενός δυαδικού σωρού n κλειδιών με πίνακα ισχύει ότι:

- Ο γονέας ενός κόμβου $i > 1$ βρίσκεται στη θέση $\lfloor i/2 \rfloor$.
- Ένας κόμβος i έχει παιδιά αν $i \leq \lfloor n/2 \rfloor$. Στην περίπτωση αυτή, το αριστερό παιδί του i βρίσκεται στη θέση $2i$ και το δεξί παιδί βρίσκεται στη θέση $2i + 1$ (αν $2i + 1 \leq n$).

Η παραπάνω ιδιότητα σε συνδυασμό με το λογαριθμικό ύψος του δυαδικού σωρού μάς επιτρέπουν να επιτύχουμε τους χρόνους εκτέλεσης που δίνονται στον Πίνακα 6.1. Όπως θα δούμε παρακάτω, οι λειτουργίες εισαγωγής και διαγραφής μέγιστου εκτελούν ένα σταθερό πλήθος βημάτων σε κάθε επίπεδο του σωρού, με αποτέλεσμα να εκτελούνται σε $O(\log n)$ χρόνο.

Πίνακας 6.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης μερικών βασικών λειτουργιών ουράς προτεραιότητας PQ με n στοιχεία, υλοποιημένης με δυαδικό σωρό.

εισαγωγή	εύρεση μέγιστου	διαγραφή μέγιστου
$O(\log n)$	$O(1)$	$O(\log n)$

Προτού περιγράψουμε τις βασικές λειτουργίες του δυαδικού σωρού θα αναφερθούμε σε δύο βασικές διαδικασίες χειρισμού του σωρού, τις οποίες αποκαλούμε αποκατάσταση άνω και αποκατάσταση κάτω. Ο ρόλος τους είναι να αποκαταστήσουν τη συνθήκη σωρού μέγιστου, όταν υπάρχει ένα κλειδί k στη θέση i το οποίο παραβιάζει αυτή τη συνθήκη. Αυτό συμβαίνει όταν το k είτε είναι μεγαλύτερο από το κλειδί του κόμβου $\lfloor i/2 \rfloor$ (του γονέα του i), είτε μικρότερο από το κλειδί ενός παιδιού του i.

Στην πρώτη περίπτωση, η αποκατάσταση της συνθήκης σωρού γίνεται μέσω της διαδικασίας αποκατάσταση_άνω, η οποία μετακινεί το κλειδί k προς τα άνω επίπεδα στο σωρό. Αυτό γίνεται ανταλλάσσοντας κάθε φορά το k με το κλειδί του κόμβου $\lfloor i/2 \rfloor$, όπου i είναι η τρέχουσα θέση του κόμβου που περιέχει το k. Η ανταλλαγή αποκαθιστά τη συνθήκη σωρού στη θέση i, αλλά μπορεί να προκαλεί την παραβίαση της συνθήκης στη θέση $\lfloor i/2 \rfloor$. Επομένως, οι ανταλλαγές πρέπει να συνεχιστούν μέχρι το κλειδί k να καταλήξει σε μία θέση όπου παύει να παραβιάζεται η συνθήκη σωρού, δηλαδή να βρεθεί στη ρίζα ή σε ένα κόμβο στη θέση j, τέτοιο ώστε ο γονέας του $\lfloor j/2 \rfloor$ να έχει μεγαλύτερο κλειδί.

Αντίστοιχα, όταν το κλειδί k είναι μικρότερο από το κλειδί ενός παιδιού του κόμβου i, τότε η αποκατάσταση της συνθήκης σωρού μπορεί να γίνει με την μετακίνηση του k προς τα κάτω επίπεδα του σωρού. Τη μετακίνηση αυτή την αναλαμβάνει η διαδικασία αποκατάσταση_κάτω. Τώρα, προκειμένου να αποκατασταθεί η συνθήκη σωρού στη θέση i, ανταλλάσσουμε το k με το μέγιστο από τα κλειδιά των παιδιών του κόμβου i. Όπως και πριν, η ανταλλαγή μπορεί να προκαλέσει την παραβίαση της συνθήκης σωρού στη θέση $2i$ ή $2i + 1$, ανάλογα με το ποιο παιδί έχει το μεγαλύτερο κλειδί. Επομένως και εδώ, οι ανταλλαγές πρέπει να συνεχιστούν μέχρι το κλειδί k να καταλήξει σε μία θέση όπου παύει να παραβιάζεται η συνθήκη σωρού.

Παρακάτω δίνουμε την αναλυτική περιγραφή των δύο βοηθητικών διαδικασιών, όπου συμβολίζουμε με κλειδί(i) το κλειδί που αποθηκεύεται στον κόμβο i του δυαδικού σωρού.

Αλγόριθμος αποκατάσταση άνω(i)

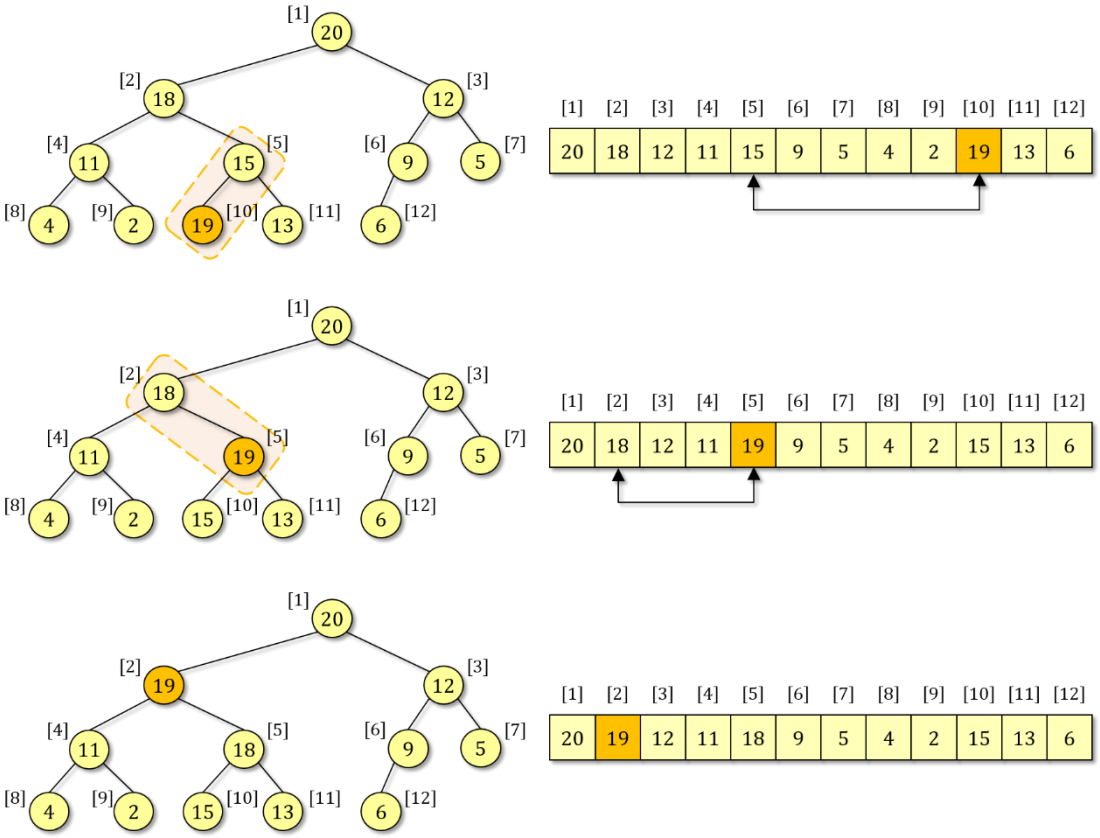
ενόσω $i > 1$

αν κλειδί$(i) \leq$ κλειδί$(\lfloor i/2 \rfloor)$ **επιστροφή** // η συνθήκη σωρού ισχύει στη θέση i

ανταλλαγή των κλειδιών στις θέσεις i και $\lfloor i/2 \rfloor$

$i \leftarrow \lfloor i/2 \rfloor$ // εξετάζουμε στη συνέχεια το γονέα του κόμβου i

επιστροφή



Εικόνα 6.3: Παράδειγμα εκτέλεσης της διαδικασίας αποκατάσταση άνω(10) στο δυαδικό σωρό μέγιστου της Εικόνας 6.2, μετά την αλλαγή του κλειδιού στη θέση 10.

Η Εικόνα 6.3 δείχνει ένα παράδειγμα για το πώς εκτελείται αυτή η διαδικασία.

Αλγόριθμος αποκατάσταση κάτω(i)

ενόσω $i \leq n/2$

// εύρεση του παιδιού j με το μεγαλύτερο κλειδί

αν $2i < n$ **και** κλειδί$(2i+1) >$ *κλειδί*$(2i)$, **τότε** $j \leftarrow 2i+1$

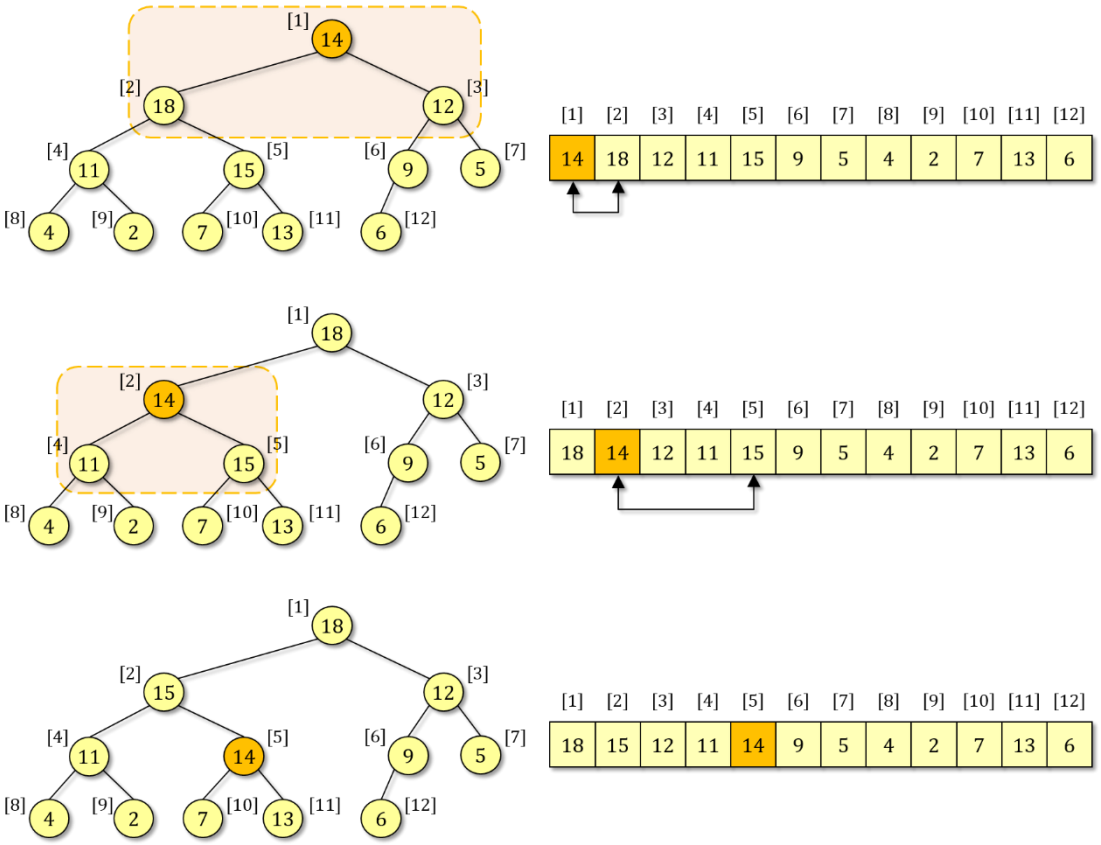
διαφορετικά $j \leftarrow 2i$

αν κλειδί$(j) \leq$ κλειδί(i) **επιστροφή** // η συνθήκη σωρού ισχύει στη θέση i

ανταλλαγή των κλειδιών στις θέσεις j και i

$i \leftarrow j$ // εξετάζουμε στη συνέχεια τον κόμβο j

επιστροφή



Εικόνα 6.4: Παράδειγμα εκτέλεσης της διαδικασίας αποκατάσταση κάτω(1) στο δυαδικό σωρό μέγιστου της Εικόνας 6.2, μετά την αλλαγή του κλειδιού στη θέση 1.

Τώρα είμαστε σε θέση να περιγράψουμε τις λειτουργίες εισαγωγής και διαγραφής μέγιστου. Υποθέτουμε ότι ο δυαδικός σωρός αποθηκεύεται σε ένα στατικό πίνακα μεγέθους N, που επαρκεί για την αποθήκευση όλων των στοιχείων.

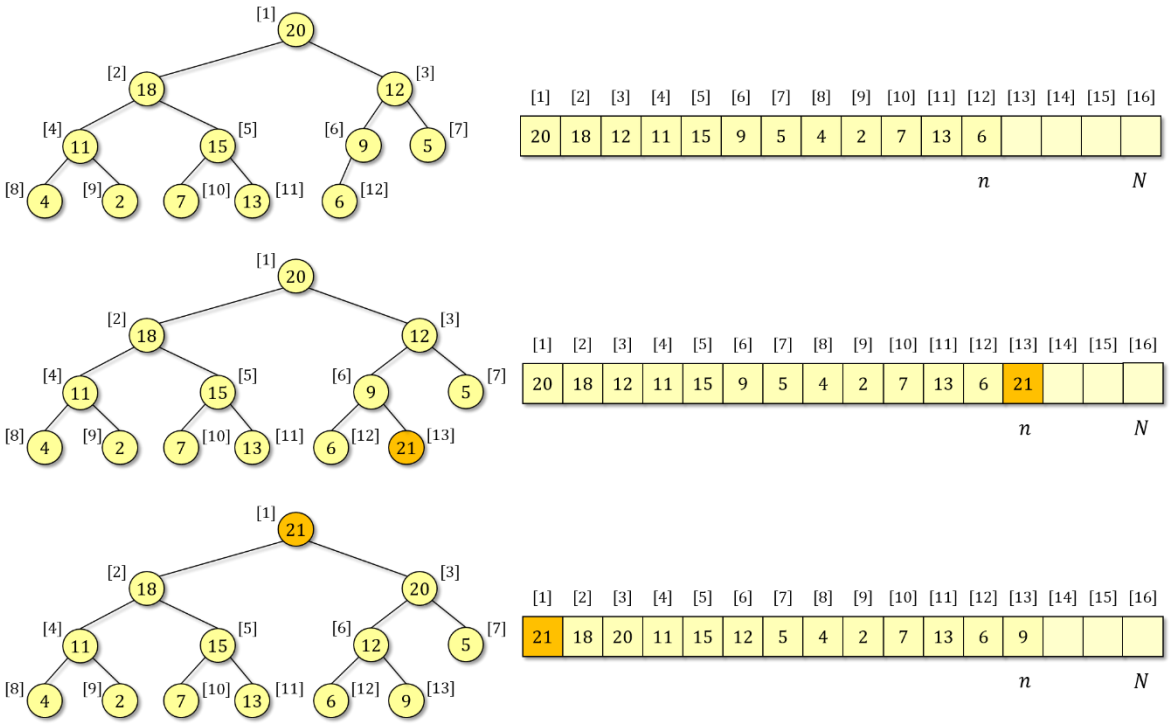
Ο αλγόριθμος εισαγωγής ενός νέου στοιχείου x με κλειδί k επαυξάνει την τιμή του n (του πλήθους των στοιχείων στο σωρό) και τοποθετεί αρχικά το x στη θέση n του πίνακα, που είναι η επόμενη κενή θέση. Έτσι, το νέο στοιχείο τοποθετείται στο τελευταίο επίπεδο του σωρού και μπορεί να προκαλέσει την παραβίαση της συνθήκης σωρού, μόνο αν ο γονέας του κόμβου n έχει κλειδί μικρότερο του k. Αρκεί, λοιπόν, να κληθεί η διαδικασία αποκατάσταση_άνω(n), για να αποκατασταθεί η συνθήκη σωρού. Η λειτουργία εισαγωγής περιγράφεται από τον παρακάτω αλγόριθμο.

Αλγόριθμος εισαγωγή(x,k)

$n \leftarrow n + 1$

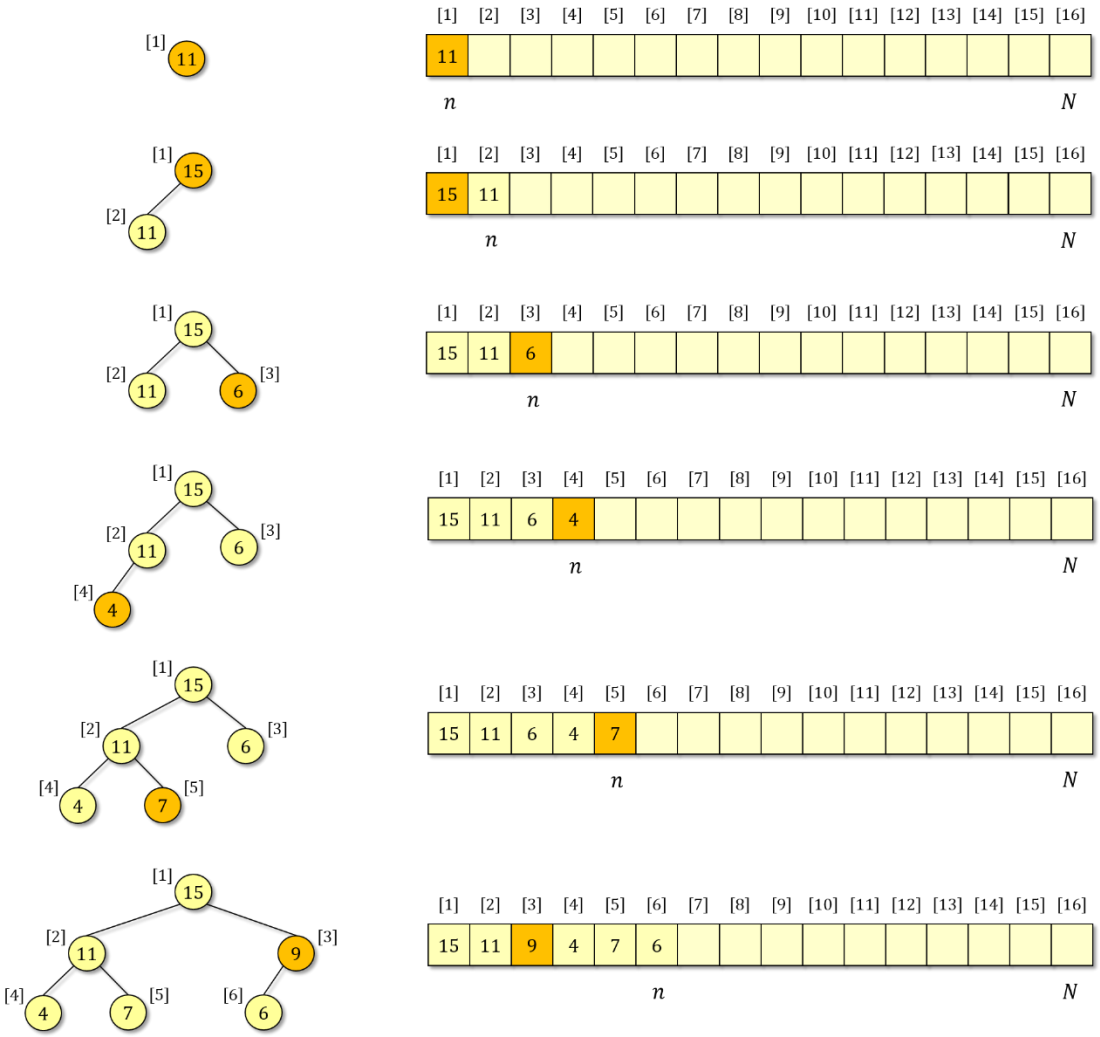
αποθήκευσε το στοιχείο x με κλειδί k στη θέση n του δυαδικού σωρού

αποκατάσταση άνω(n)

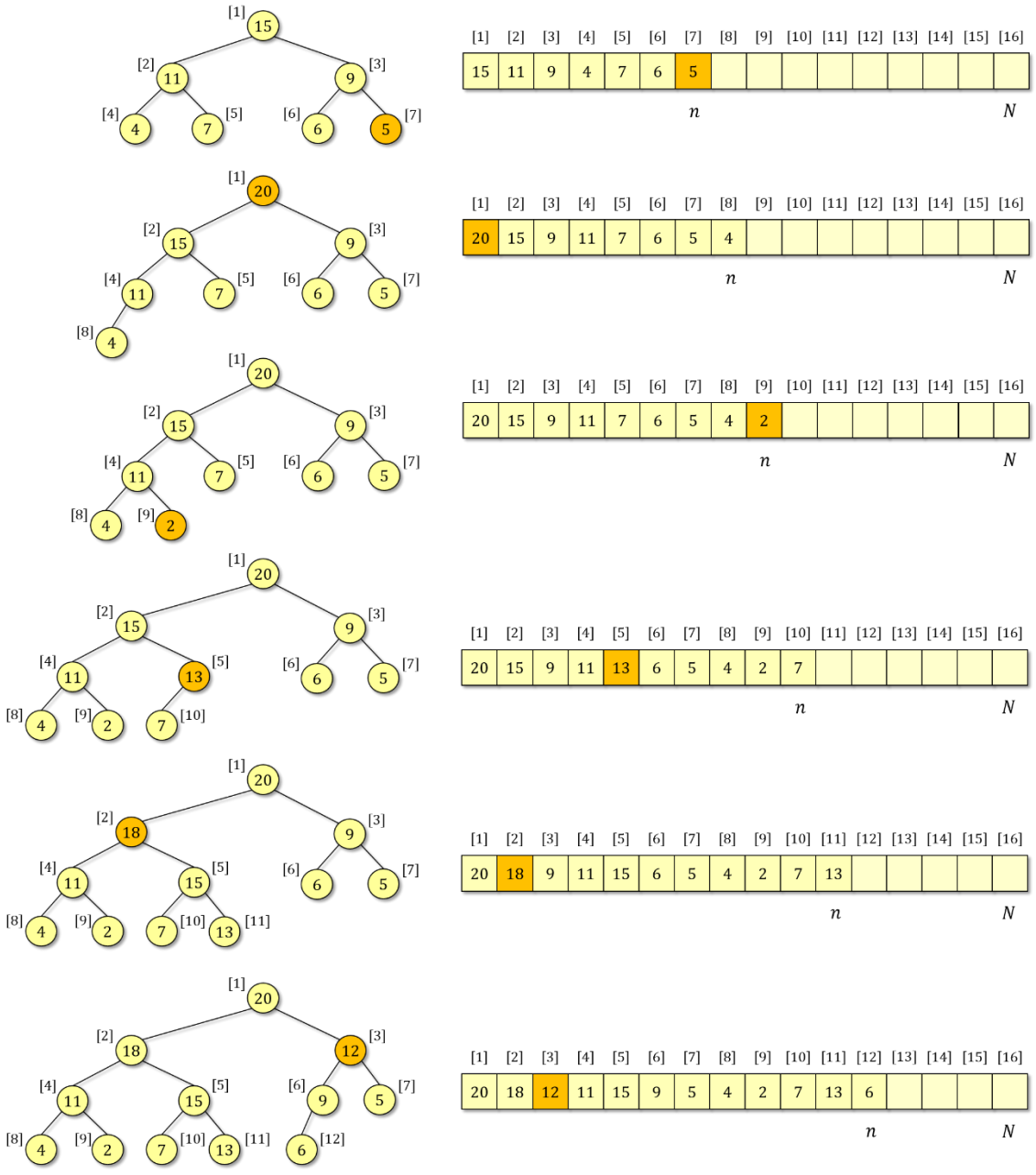


Εικόνα 6.5: Εισαγωγή του κλειδιού 21 στο δυαδικό σωρό μέγιστου της Εικόνας 6.2, όπου χρησιμοποιούμε ένα πίνακα μεγέθους $N = 16$ για την αποθήκευση του σωρού.

Ένα παράδειγμα εισαγωγής νέου κλειδιού δίνεται στην Εικόνα 6.5. Στις Εικόνες 6.6 και 6.7 φαίνεται η μορφή ενός δυαδικού σωρού μέγιστου κατά τη διάρκεια εκτέλεσης μιας ακολουθίας εισαγωγών.



Εικόνα 6.6: Διαδοχική εισαγωγή των κλειδιών 11, 15, 6, 4, 7 και 9 σε αρχικά κενό δυαδικό σωρό μέγιστου, όπου χρησιμοποιούμε ένα πίνακα μεγέθους N = 16 για την αποθήκευση του σωρού.



Εικόνα 6.7: Διαδοχική εισαγωγή των κλειδιών 5, 20, 2, 13, 18 και 12 στο δυαδικό σωρό μέγιστου της Εικόνας 6.6.

Η διαδικασία που ακολουθούμε για τη διαγραφή του μέγιστου κλειδιού του σωρού είναι εξίσου απλή. Το μέγιστο κλειδί, το οποίο βρίσκεται στη ρίζα του σωρού, μπορεί να αντικατασταθεί προσωρινά από το κλειδί που βρίσκεται στην τελευταία θέση. Η αντικατάσταση του κλειδιού της ρίζας μπορεί να προκαλέσει την παραβίαση της συνθήκης σωρού μόνο αν κάποιο από τα παιδιά της ρίζας έχει μεγαλύτερο κλειδί. Οπότε αρκεί τώρα να κληθεί η διαδικασία αποκατάσταση_κάτω(1) για να αποκατασταθεί η συνθήκη σωρού. Η λειτουργία διαγραφής μέγιστου περιγράφεται από τον παρακάτω αλγόριθμο:

Αλγόριθμος διαγραφή μέγιστου()

$k \leftarrow$ κλειδί που βρίσκεται στη ρίζα του δυαδικού σωρού

// μεταφορά κλειδιού από τον τελευταίο κόμβο στη ρίζα του σωρού

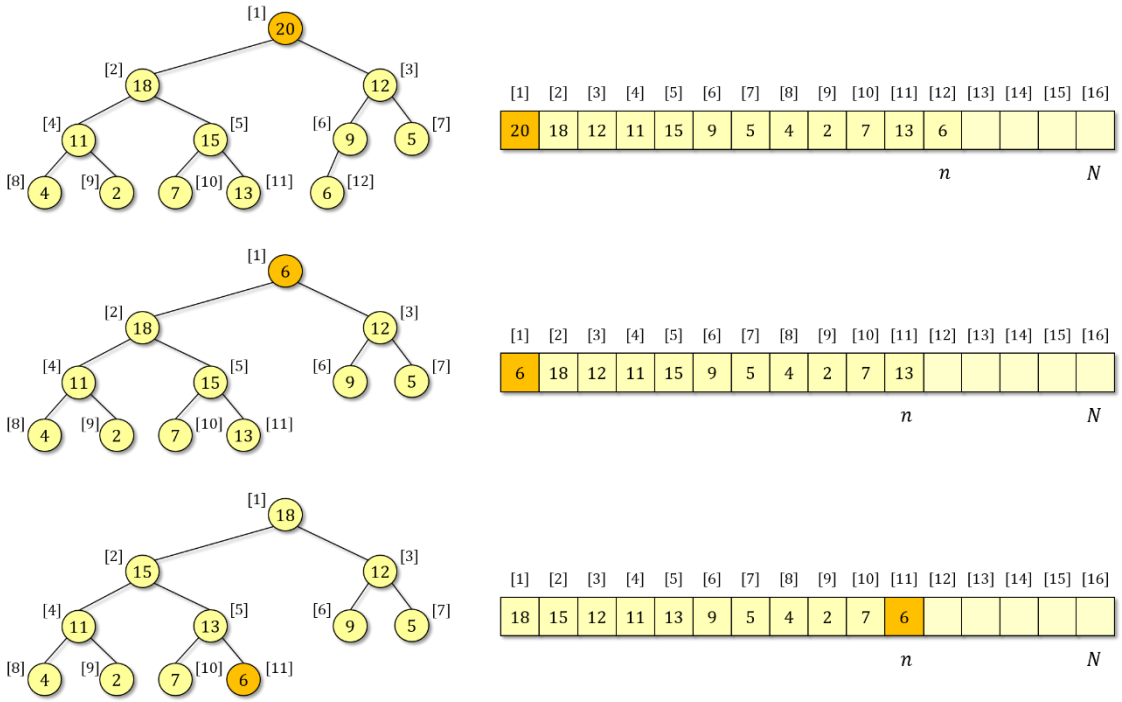
κλειδί$(1) \leftarrow$ κλειδί(n), κλειδί$(n) \leftarrow$ κενό

$n \leftarrow n - 1$

αποκατάσταση_κάτω(1)

επιστροφή k

Η Εικόνα 6.8 δίνει ένα παράδειγμα εκτέλεσης του παραπάνω αλγόριθμου.



Εικόνα 6.8: Διαγραφή του μέγιστου κλειδιού στο δυαδικό σωρό της Εικόνας 6.2, όπου χρησιμοποιούμε ένα πίνακα μεγέθους $\mathrm{N} = 16$ για την αποθήκευση του σωρού.

6.3.1 Υλοποίηση σε Java

Το παρακάτω πρόγραμμα υλοποιεί μια ουρά προτεραιότητας μέγιστου με δυαδικό σωρό. Η δομή αποθηκεύει αντικείμενα τα οποία έχουν ένα κλειδί γενικού τύπου Key.

```
public class MaxHeap<Key extends Comparable<Key>> {

    private int N = 0;
    private Key[] pq = (Key[]) new Comparable[2];

    private boolean less(int i, int j) {
        return pq[i].compareTo(pq[j]) < 0;
    }

    private void exch(int i, int j) {
        Key t = pq[i];
        pq[i] = pq[j];
        pq[j] = t;
    }

    // αποκατάσταση προς τους πρόγονους του κόμβου k
    private void fixUp(int k) {
        while (k > 1 && less(k / 2, k)) {
            exch(k, k / 2);
            k = k / 2;
        }
    }

    // αποκατάσταση προς τους απογόνους του κόμβου k
    private void fixDown(int k) {
        int j;
        while (2 * k <= N) {
            j = 2 * k;
            if (j < N && less(j, j + 1)) {
                j++;
            }
            if (!less(k, j)) {
                break;
            }
            exch(k, j);
            k = j;
        }
    }

    public boolean isEmpty() {
        return N == 0;
    }

    public int size() {
        return N;
    }

    // μετακινεί το σωρό pq σε νέο πίνακα μεγέθους max
    private void resize(int max) {
        System.out.println("resize " + max);
        Key[] temp = (Key[]) new Comparable[max];
        for (int i = 1; i <= N; i++) {
            temp[i] = pq[i];
        }
        pq = temp;
    }
```

```
    public void insert(Key v) {
        if (N == (pq.length - 1)) {
            resize(2 * pq.length);
        }
        pq[++N] = v;
        fixUp(N);
    }

    public Key delMax() {
        Key max = pq[1];
        exch(1, N);
        pq[N--] = null;
        fixDown(1);
        if (N > 0 && N == (pq.length-1) / 4) {
            resize(pq.length / 2);
        }
        return max;
    }

    public Key findMax() {
        Key max = pq[1];
        return max;
    }
}
```

6.3.2 Κατασκευή δυαδικού σωρού με δεδομένα κλειδιά

Σε πολλές περιπτώσεις θέλουμε να κατασκευάσουμε μια ουρά προτεραιότητας από ένα σύνολο n κλειδιών που είναι γνωστό εξαρχής. Η κατασκευή μπορεί να γίνει με τη διαδοχική εισαγωγή των κλειδιών του δοθέντος συνόλου. Αν η ουρά προτεραιότητας υλοποιηθεί ως δυαδικός σωρός, τότε η εισαγωγή των n κλειδιών θα ολοκληρωθεί σε $O(n \log n)$ χρόνο (δείτε την Άσκηση 6.2). Στην περίπτωση του δυαδικού σωρού, όμως, μπορούμε επίσης να εκμεταλλευτούμε το γεγονός ότι η συνθήκη σωρού μπορεί να επιβληθεί με διαδοχικές κλήσεις των διαδικασιών αποκατάσταση άνω() ή αποκατάσταση κάτω(). Εδώ θα δείξουμε ότι οι διαδοχικές κλήσεις της διαδικασίας αποκατάσταση κάτω(i), για $i = \lfloor n/2 \rfloor, \lfloor n/2 \rfloor - 1, \ldots, 1$, εκτελείται σε $O(n)$ χρόνο. Στην Άσκηση 6.3 μελετάμε τη αποκατάσταση της συνθήκης σωρού με διαδοχικές κλήσεις της διαδικασίας αποκατάσταση άνω().

Αλγόριθμος κατασκευή δυαδικού σωρού(πίνακας κλειδιών A)

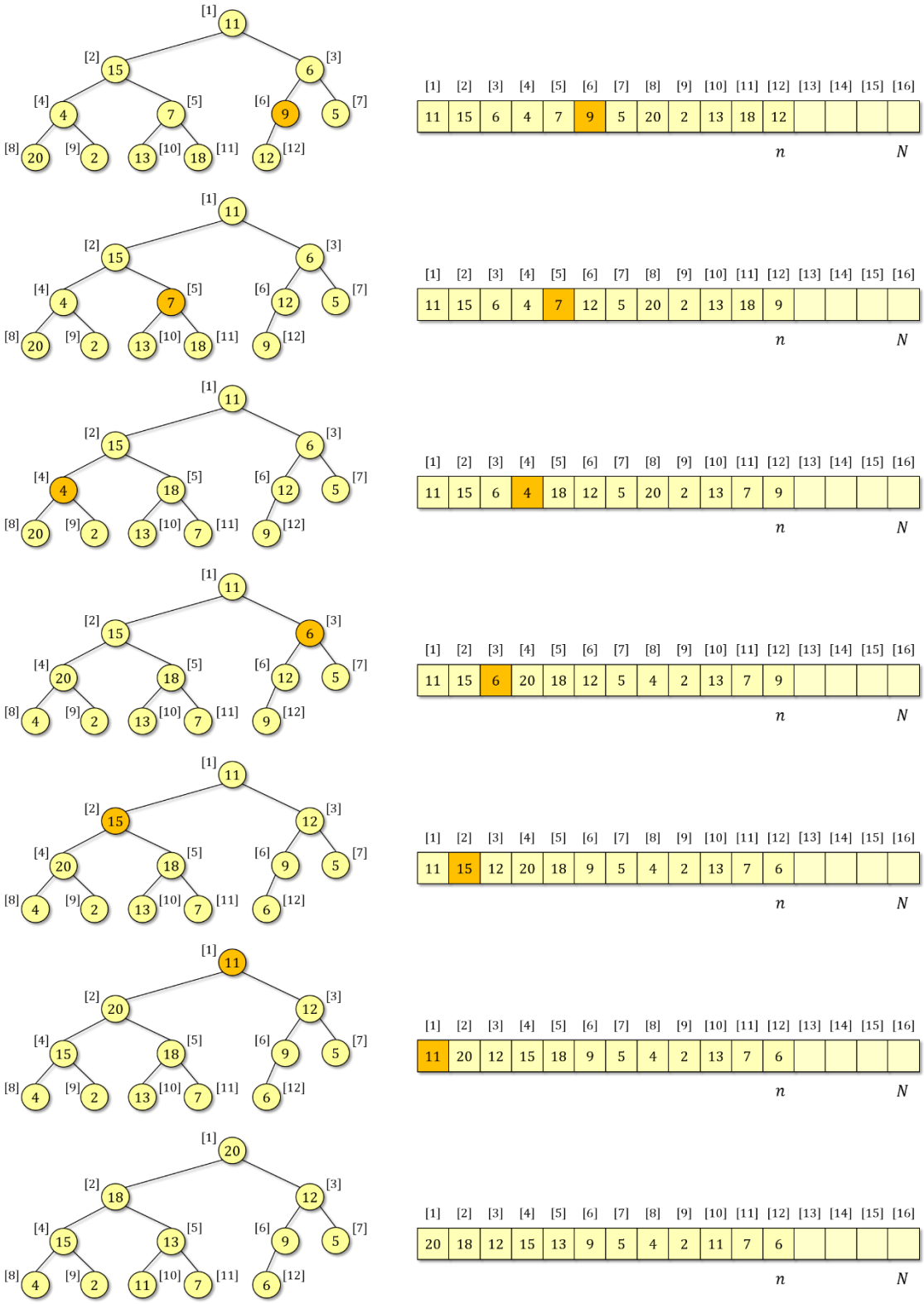
$n \leftarrow$ πλήθος κλειδιών του πίνακα A

$H \leftarrow$ δυαδικός σωρός για τα κλειδιά του πίνακα A

για $i = \lfloor n/2 \rfloor$ **έως** 1

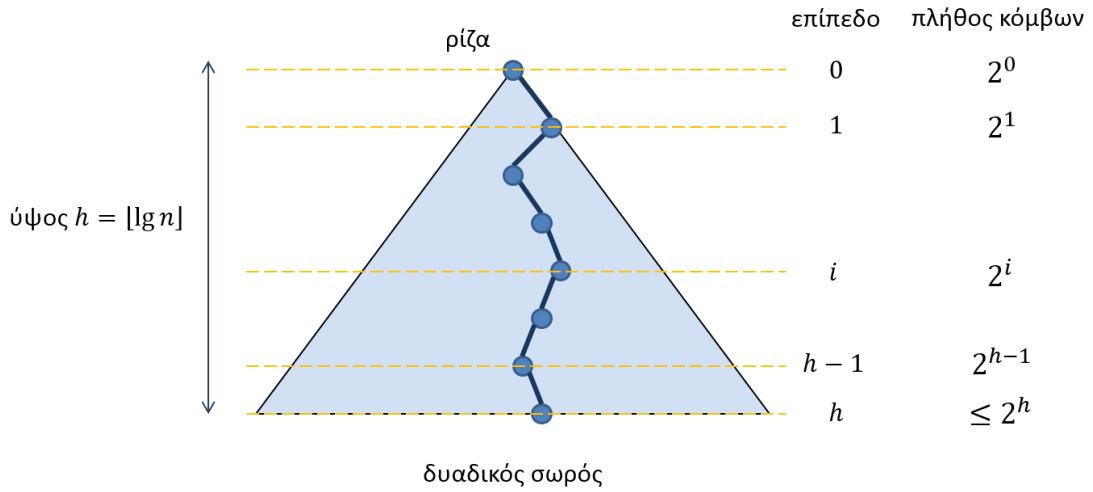
H.αποκατάσταση_κάτω(i)

επιστροφή H



Εικόνα 6.9: Κατασκευή δυαδικού σωρού μέγιστου με δεδομένα κλειδιά.

Έστω h το ύψος του δυαδικού σωρού. Θεωρούμε ένα κλειδί που αρχικά βρίσκεται στο επίπεδο i. Το κλειδί μπορεί να μετακινηθεί το πολύ κατά $h - i$ επίπεδα.



Εικόνα 6.10: Ανάλυση του χρόνου εκτέλεσης της διαδικασίας κατασκευής δυαδικού σωρού από δεδομένα κλειδιά.

Αφού το πλήθος των κλειδιών στο επίπεδο i είναι 2^i έχουμε ότι το συνολικό πλήθος των ανταλλαγών είναι το πολύ

$$\sum_{i=0}^{h}(h-i)2^i = \sum_{i=0}^{h}h2^i - \sum_{i=0}^{h}i2^i$$

Γνωρίζουμε ότι $\sum_{i=0}^{h}2^i = 2^{h+1} - 1$ και $\sum_{i=0}^{h}i2^i = (h-1)2^{h+1} + 2$, άρα

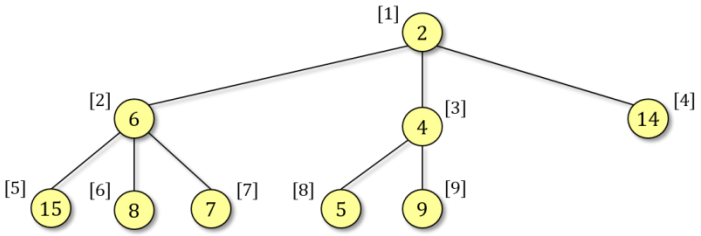
$$\sum_{i=0}^{h}(h-i)2^i = h2^{h+1} - 1 - (h-1)2^{h+1} - 2 = 2^{h+1} - 3 < n$$

και, επομένως, αποδείξαμε την παρακάτω ιδιότητα:

Ιδιότητα 6.1 Η κατασκευή ενός δυαδικού σωρού από n δεδομένα κλειδιά μπορεί να γίνει σε $O(n)$ χρόνο.

6.4 δ-Σωρός

Ο δ-σωρός αποτελεί γενίκευση της δομής του δυαδικού σωρού. Ένας δ-σωρός είναι ένα πλήρες δ-αδικό δένδρο με ρίζα, στο οποίο κάθε κόμβος έχει έως δ παιδιά, όπου $\delta \geq 2$ ακέραιος, και οι κόμβοι προστίθενται από αριστερά προς τα δεξιά και από μικρότερο προς μεγαλύτερο επίπεδο, όπως φαίνεται στην Εικόνα 6.11. (Δηλαδή υπάρχουν κόμβοι στο επίπεδο k, μόνο αν όλα τα παραπάνω επίπεδα είναι πλήρη.) Μια ανάλυση παρόμοια με αυτή που κάναμε για το δυαδικό σωρό δείχνει ότι ο δ-σωρός έχει ύψος $\lfloor \log_\delta n \rfloor + O(1)$.



Εικόνα 6.11: Ένας δ-σωρός ελάχιστου με $\delta = 3$.

Ο δ-σωρός μπορεί να αποθηκευτεί σε ένα πίνακα με τον ίδιο τρόπο όπως και ο δυαδικός σωρός, δηλαδή, αφού αριθμήσουμε τους κόμβους σύμφωνα με την οριζόντια διερεύνηση του δένδρου και αντιστοιχούμε τον κόμβο με αριθμό i στη θέση i του πίνακα. Έτσι, τα παιδιά του κόμβου i βρίσκονται στις θέσεις $\delta(i-1)+2, \delta(i-1)+3, \ldots, \min\{\delta i+1, n\}$, ενώ ο γονέας του στη θέση $\lceil (i-1)/d \rceil$.

6.5 Ταξινόμηση με ουρά προτεραιότητας

Η ταξινόμηση ενός συνόλου κλειδιών αποτελεί μια από τις κύριες εφαρμογές μιας ουράς προτεραιότητας. Θα περιγράψουμε πώς μπορούμε να ταξινομήσουμε έναν πίνακα A χρησιμοποιώντας μια ουρά προτεραιότητας μέγιστου. Η ταξινόμηση μπορεί να γίνει και με μια ουρά προτεραιότητας ελάχιστου με παρόμοιο τρόπο. Πρώτα εισάγουμε τα κλειδιά του πίνακα A στην ουρά προτεραιότητας μέγιστου. Στη συνέχεια, πραγματοποιούμε διαδοχικές διαγραφές του μέγιστου κλειδιού της ουράς. Το κλειδί που λαμβάνουμε με την i-οστή εξαγωγή τοποθετείται στη θέση $n-i+1$ του πίνακα A. Έτσι, μετά το πέρας αυτής της διαδικασίας, όλα τα κλειδιά είναι τοποθετημένα στον πίνακα A σε αύξουσα σειρά.

Αλγόριθμος ταξινόμηση_με_ουρά_προτεραιότητας(πίνακας κλειδιών A)

$n \leftarrow$ πλήθος κλειδιών του πίνακα A

PQ $\leftarrow$ ουρά προτεραιότητας μέγιστου για n κλειδιά

για $i = 1$ **έως** n

  PQ.εισαγωγή($A[i]$)

$j \leftarrow n$

ενόσω η PQ δεν είναι κενή

  $A[j] \leftarrow$ PQ.διαγραφή_μέγιστου()

  $j \leftarrow j-1$

Αν υλοποιήσουμε την ουρά προτεραιότητας μέγιστου με ένα δυαδικό σωρό, τότε τόσο η διαδοχική εισαγωγή όσο και η διαδοχική εξαγωγή n κλειδιών απαιτεί $O(n \log n)$ χρόνο στη χειρότερη περίπτωση. Δείτε την Άσκηση 6.2.

Ο παραπάνω τρόπος ταξινόμησης απαιτεί επιπλέον χώρο για την αποθήκευση των n κλειδιών στην ουρά προτεραιότητας. Επίσης, δεν εκμεταλλεύεται το γεγονός ότι όλα τα κλειδιά είναι γνωστά, προτού γίνει η ταξινόμηση, το οποίο σημαίνει ότι δε χρειάζεται να εισαχθούν ένα προς ένα στη δομή. Μια καλύτερη λύση είναι να χρησιμοποιήσουμε τον ίδιο τον πίνακα εισόδου A ως δυαδικό σωρό. Με αυτόν τον τρόπο, η κατασκευή του σωρού μπορεί να γίνει με τον

αλγόριθμο της Ενότητας 6.2.1. Επιπλέον, η τοποθέτηση του κάθε κλειδιού στη σωστή του θέση μπορεί να γίνει με διαδοχικές κλήσεις της ρουτίνας αποκατάστασης προς τα κάτω.

Αλγόριθμος ταξινόμηση_με_σωρό(πίνακας κλειδιών A)

$n \leftarrow$ πλήθος κλειδιών του πίνακα A

$H \leftarrow$ κατασκευή_δυαδικού_σωρού(πίνακας κλειδιών A)

για $i = 1$ **έως** n

ανταλλαγή των κλειδιών της ρίζας και του τελευταίου κόμβου του H

// $A[1] \leftrightarrow A[n]$

$n \leftarrow n - 1$

H.αποκατάσταση_κάτω(1)

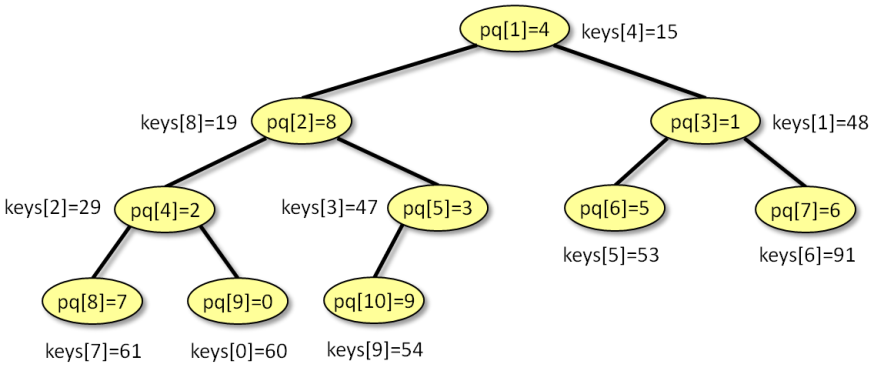
Ο αλγόριθμος ταξινόμησης με σωρό έχει τον ίδιο ασυμπτωτικό χρόνο εκτέλεσης, $O(n \log n)$ για n κλειδιά, ωστόσο αποδίδει καλύτερα στην πράξη, γιατί η κατασκευή του δυαδικού σωρού γίνεται σε $O(n)$ χρόνο.

6.6 Ουρές προτεραιότητας με ευρετήριο

Σε ορισμένες εφαρμογές όπου χειριζόμαστε αντικείμενα με κλειδιά, θέλουμε να εκτελούμε πράξεις πάνω στα κλειδιά συγκεκριμένων αντικειμένων. Ας θεωρήσουμε, για παράδειγμα, τη λειτουργία αλλαγή κλειδιού(Item item, Key key), που αναφέραμε στην εισαγωγή. Σε μια δομή σωρού μπορούμε να υλοποιήσουμε εύκολα αυτή τη λειτουργία χρησιμοποιώντας τις βοηθητικές μεθόδους αποκατάσταση άνω και αποκατάσταση κάτω, με την προϋπόθεση ότι γνωρίζουμε τη θέση στην οποία βρίσκεται το κλειδί του αντικειμένου item στο σωρό. Επομένως, το ερώτημα που καλούμαστε να απαντήσουμε είναι το πώς μπορούμε να εντοπίσουμε το κλειδί ενός δεδομένου αντικειμένου. Εδώ θα περιγράψουμε μια απλή λύση για την περίπτωση όπου τα αντικείμενα έχουν μια ακέραιη ταυτότητα από 0 έως $N - 1$. Για το σκοπό αυτό χρησιμοποιούμε ένα πίνακα keys[] τύπου Key και δύο πίνακες ακεραίων pq[] και index[].

Το κλειδί ενός αντικειμένου με ταυτότητα j αποθηκεύεται στη θέση keys[j]. Ο πίνακας pq[] αποθηκεύει τις ταυτότητες των αντικειμένων και είναι διατεταγμένος σε δυαδικό σωρό ως προς τα κλειδιά των αντικειμένων. Η θέση του αντικειμένου με ταυτότητα j στον πίνακα pq[] δίνεται από την τιμή index[j], δηλαδή $\text{pq}[\text{index}[\text{j}]] = \text{index}[\text{pq}[\text{j}]] = \text{j}$. Αν δεν έχει εισαχθεί στην ουρά προτεραιότητας το αντικείμενο με ταυτότητα j, τότε έχουμε $\text{index}[\text{j}] = -1$.

Για παράδειγμα, μετά την εισαγωγή των αντικειμένων με ταυτότητες και αντίστοιχα κλειδιά (0,60), (1,48), (2,29), (3,47), (4,15), (5,53), (6,91), (7,61), (8,19), (9,54) έχουμε $\text{keys}[0:9] = [60,48,29,47,15,53,91,61,19,54]$, $\text{pq}[1:10] = [4,8,1,2,3,5,6,7,0]$ και $\text{index}[0:9] = [9,3,4,5,1,6,7,8,2,10]$, όπου η διάταξη σε δυαδικό σωρό απεικονίζεται στο ακόλουθο σχήμα.



Εικόνα 6.12: Ένας δυαδικός σωρός με ευρετήριο.

Μπορούμε να τροποποιήσουμε εύκολα τις βοηθητικές μεθόδους αποκατάσταση άνω και αποκατάσταση κάτω, έτσι ώστε να ενημερώνονται σωστά οι πίνακες pq[], index[] και keys[]. Με αυτόν τον τρόπο, μπορούμε να υποστηρίξουμε το ίδιο αποδοτικά τις βασικές λειτουργίες μιας ουράς προτεραιότητας και, επιπλέον, να μπορούμε να υποστηρίξουμε τη λειτουργία αλλαγή κλειδιού.

Ασκήσεις

6.1 Περιγράψτε αποδοτικούς αλγόριθμους οι οποίοι να υλοποιούν τις βασικές λειτουργίες μιας ουράς προτεραιότητας ελάχιστου.

6.2 Περιγράψτε μια ακολουθία n εισαγωγών σε δυαδικό σωρό η οποία απαιτεί συνολικό χρόνο $O(n \log n)$.

6.3 Ας υποθέσουμε ότι θέλουμε να δώσουμε μια εναλλακτική υλοποίηση της διαδικασίας κατασκευή δυαδικού σωρού, όπου χρησιμοποιούμε διαδοχικές κλήσεις της διαδικασίας αποκατάσταση άνω(). Δώστε μια πλήρη περιγραφή αυτής της διαδικασίας. Ποιος πιστεύετε ότι είναι ο ασυμπτωτικός χρόνος εκτέλεσής της στη χειρότερη περίπτωση; Δικαιολογήστε την απάντησή σας.

6.4 Δείξτε ότι ένας δ-σωρός με n κλειδιά έχει ύψος $\lfloor \log_\delta n \rfloor + O(1)$.

6.5 Δώστε αποδοτικές υλοποιήσεις των λειτουργιών εισαγωγής και διαγραφής ελάχιστου σε ένα δ-σωρό ελάχιστου.

6.6 Ένα υπολογιστικό σύστημα δέχεται στην είσοδο ένα ρεύμα εισόδου που αποτελείται από N αριθμούς, από τους οποίους πρέπει να κρατήσει τους M μεγαλύτερους, όπου $M \ll N$. Αν το N είναι αρκετά μικρό τότε μπορούμε να αποθηκεύσουμε ολόκληρο το ρεύμα εισόδου σε ένα πίνακα μεγέθους N, να τον ταξινομήσουμε και να κρατήσουμε τις M τελευταίες θέσεις. Αυτό γίνεται σε χρόνο $O(N \log N)$, αλλά προϋποθέτει ότι το σύστημά μας έχει αρκετή μνήμη, για να αποθηκεύσει N αριθμούς. Θέλουμε να εξετάσουμε την περίπτωση όπου το N είναι πολύ μεγάλο και η μνήμη του υπολογιστή δεν επαρκεί, για να αποθηκεύσει ολόκληρη την ακολουθία εισόδου. (Για παράδειγμα, το ρεύμα εισόδου μπορεί να αντιστοιχεί στις θερμοκρασίες που καταγράφουν σε τακτά χρονικά διαστήματα οι μετεωρολογικοί σταθμοί μιας χώρας, ή τις χρηματιστηριακές συναλλαγές που λαμβάνουν χώρα στο διάστημα μιας ημέρας, οπότε ο όγκος των δεδομένων είναι πολύ μεγάλος.) Περιγράψτε μια όσο το δυνατό πιο

απλή και αποδοτική λύση για αυτό το πρόβλημα. Ποιος είναι ο χρόνος εκτέλεσης του αλγόριθμού σας;

6.7 Ο διάμεσος ενός συνόλου S με n αριθμούς είναι ο αριθμός $k \in S$, ο οποίος είναι μεγαλύτερος από $\lfloor n/2 \rfloor$ αριθμούς του S. Π.χ., ο 5 είναι διάμεσος του συνόλου $\{1, 2, \mathbf{5}, 6, 9\}$, ενώ ο 9 είναι διάμεσος του συνόλου $\{1, 2, 5, 6, \mathbf{9}, 10, 11, 12\}$. Σχεδιάστε μια δομή δεδομένων η οποία αποθηκεύει ένα δυναμικό σύνολο ακέραιων αριθμών S και υποστηρίζει τις παρακάτω μεθόδους:

void insert(int k)	εισάγει στο σύνολο S τον ακέραιο αριθμό k
int findMedian()	επιστρέφει το διάμεσο των ακέραιων του συνόλου S
void deleteMedian(int k)	διαγράφει από το σύνολο S τον ακέραιο αριθμό k

Η δομή πρέπει να εκτελεί τη μέθοδο findMedian σε χρόνο $O(1)$ και τις insert και deleteMedian σε χρόνο $O(\log n)$. *Υπόδειξη:* Χρησιμοποιήστε μια ουρά προτεραιότητας ελάχιστου και μια ουρά προτεραιότητας μέγιστου.

6.8 Περιγράψτε τις λεπτομέρειες της υλοποίησης ενός δυαδικού σωρού με ευρετήριο.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Κεφάλαιο 7
Λεξικά και Δυαδικά Δένδρα Αναζήτησης

Περιεχόμενα

7.1 Ο αφηρημένος τύπος δεδομένων λεξικού 133
7.1.1 Διατεταγμένα λεξικά 134
7.2 Στοιχειώδεις υλοποιήσεις με πίνακες και λίστες 135
7.2.1 Υλοποίηση με πίνακα 136
7.2.2 Υλοποίηση με αριθμοδείκτη 137
7.2.3 Υλοποίηση με συνδεδεμένη λίστα 137
7.3 Δυαδικά δένδρα αναζήτησης 138
7.3.1 Αναζήτηση 139
7.3.2 Εύρεση ελάχιστου και μέγιστου 141
7.3.3 Προκάτοχος και διάδοχος 142
7.3.4 Εισαγωγή 143
7.3.5 Διαγραφή 144
7.3.6 Επιλογή 146
7.3.7 Ένωση 147
7.3.8 Διαχωρισμός 148
7.4. Τυχαία κατασκευασμένα δυαδικά δένδρα αναζήτησης 149
7.5. Υλοποίηση δυαδικών δένδρων αναζήτησης σε Java 150
Ασκήσεις 153
Πειραματικές Μελέτες 154
Βιβλιογραφία 154

7.1 Ο αφηρημένος τύπος δεδομένων λεξικού

Η αποθήκευση και η λήψη πληροφοριών αποτελούν θεμελιώδεις λειτουργίες ενός υπολογιστικού συστήματος. Με τον όρο *λεξικό* (dictionary) αναφερόμαστε σε μια δομή δεδομένων η οποία υποστηρίζει αυτές τις λειτουργίες, μαζί ενδεχομένως με ένα μεγαλύτερο σύνολο άλλων χρήσιμων λειτουργιών.

Ένα λεξικό D αποθηκεύει ένα σύνολο S από στοιχεία (εγγραφές). Κάθε στοιχείο διαθέτει ένα κλειδί που μπορεί να χρησιμοποιηθεί για την αναζήτησή του στη δομή. Εάν το σύνολο όλων

των κλειδιών είναι ολικά διατεταγμένο, τότε το λεξικό είναι *διατεταγμένο*. Σε αντίθετη περίπτωση, έχουμε ένα *μη διατεταγμένο* λεξικό.

Ένα λεξικό D υποστηρίζει τις παρακάτω βασικές λειτουργίες:

κατασκευή() : Επιστρέφει ένα κενό λεξικό.

αναζήτηση(k) : Αν το D περιέχει ένα στοιχείο με κλειδί k, τότε επιστρέφει ένα τέτοιο στοιχείο. Διαφορετικά επιστρέφει το κενό στοιχείο.

εισαγωγή(x, k) : Εισάγει στο D ένα νέο στοιχείο x με κλειδί k.

διαγραφή(x) : Διαγράφει από το D το στοιχείο x και επιστρέφει το διαγεγραμμένο στοιχείο.

Η εκτέλεση της λειτουργίας διαγραφή(x) προϋποθέτει ότι το στοιχείο x είναι αποθηκευμένο στο λεξικό. Σε αντίθετη περίπτωση, πρέπει να ενημερώνει ότι η δομή δεν περιέχει το στοιχείο x, π.χ. παράγοντας την αντίστοιχη εξαίρεση στη Java ή επιστρέφοντας το κενό στοιχείο. Μπορούμε εναλλακτικά να θέσουμε ως όρισμα της λειτουργίας διαγραφή το κλειδί του στοιχείου που θέλουμε να διαγράψουμε. Μια τέτοια διαφοροποίηση μπορεί να απαιτεί την εκτέλεση μιας αναζήτησης του στοιχείου, αλλά δεν επηρεάζει ουσιαστικά τον τρόπο με τον οποίο εκτελείται η διαγραφή στις δομές που θα περιγράψουμε παρακάτω.

Ένα ακόμα σημείο που χρήζει προσοχής είναι εάν επιτρέπουμε στο λεξικό να αποθηκεύει στοιχεία που έχουν το ίδιο κλειδί. Για παράδειγμα, σε μια βάση δεδομένων των φοιτητών μιας σχολής, μπορούμε να επιλέξουμε τον αριθμό μητρώου ως κλειδί αναζήτησης, με αποτέλεσμα το κλειδί κάθε στοιχείου να είναι μοναδικό. Αντίθετα, σε μια επιχείρηση, που διατηρεί ένα πελατολόγιο, το κλειδί μπορεί να είναι το ονοματεπώνυμο του κάθε πελάτη, οπότε υπάρχει το ενδεχόμενο πολλαπλής εμφάνισης του ίδιου κλειδιού. Σε μια τέτοια περίπτωση πρέπει να καθορίσουμε τη συμπεριφορά της λειτουργίας αναζήτηση(k), δηλαδή αν μας αρκεί να επιστρέφει οποιοδήποτε στοιχείο με κλειδί k, αν θέλουμε να βρίσκουμε όλα τα στοιχεία με κλειδί k ή αν η επιλογή του στοιχείου που θα επιστραφεί θα γίνεται με ένα δευτερεύον κριτήριο.

Σε ορισμένες εφαρμογές μάς είναι χρήσιμη η δυνατότητα συγχώνευσης δύο λεξικών. Για το σκοπό αυτό μπορούμε να ορίσουμε την παρακάτω λειτουργία για ένα λεξικό D:

συγχώνευση(D′) : Επιστρέφει ένα νέο λεξικό το οποίο προκύπτει από την ένωση των στοιχείων των λεξικών D και D′. Η λειτουργία αυτή καταστρέφει το D′ και το νέο λεξικό παίρνει τη θέση του D.

7.1.1 Διατεταγμένα λεξικά

Η ύπαρξη ολικής διάταξης για το σύνολο των κλειδιών επιτρέπει την εκτέλεση επιπρόσθετων λειτουργιών, όπως οι παρακάτω:

ελάχιστο() : Επιστρέφει ένα στοιχείο του D με το ελάχιστο κλειδί.

μέγιστο() : Επιστρέφει ένα στοιχείο του D με το μέγιστο κλειδί.

επιλογή(j) : Επιστρέφει ένα στοιχείο του D με το j-οστό μικρότερο κλειδί. Υποθέτει ότι $1 \leq j \leq n$, όπου n το πλήθος των κλειδιών στο λεξικό.

ταξινόμηση() : Επιστρέφει τα στοιχεία του D διατεταγμένα σε αύξουσα σειρά ως προς τα κλειδιά τους.

προκάτοχος(k) : Επιστρέφει ένα στοιχείο με το μεγαλύτερο κλειδί που είναι μικρότερο του k.

διάδοχος(k) : Επιστρέφει ένα στοιχείο με το μικρότερο κλειδί που είναι μεγαλύτερο του k.

ένωση(x, D′) : Επιστρέφει ένα νέο λεξικό το οποίο προκύπτει από την ένωση του στοιχείου x και των στοιχείων των λεξικών D και D′. Η λειτουργία αυτή καταστρέφει το D′ και το νέο λεξικό που δημιουργείται παίρνει τη θέση του D. Προϋποθέτει ότι κάθε στοιχείο του D έχει κλειδί μικρότερο από το κλειδί του x και κάθε στοιχείο του D′ έχει κλειδί μεγαλύτερο από το κλειδί του x.

διαχωρισμός(k) : Χωρίζει το λεξικό D σε δύο λεξικά D_1 και D_2, όπου το D_1 περιέχει τα στοιχεία με κλειδιά μικρότερα ή ίσα του k και το D_2 περιέχει τα στοιχεία με κλειδιά μεγαλύτερα του k.

Η σχεδίαση αποδοτικών δομών δεδομένων που υποστηρίζουν τις λειτουργίες λεξικού, διατεταγμένου ή μη, είναι ένα βασικό πρόβλημα το οποίο έχει μελετηθεί εκτενώς. Στη σχετική βιβλιογραφία έχουν προταθεί λύσεις με διαφορετικά χαρακτηριστικά επιδόσεων, τις πιο σημαντικές από τις οποίες θα μελετήσουμε σε αυτό και σε επόμενα κεφάλαια. Στις υπόλοιπες ενότητες του κεφαλαίου θα αναφερθούμε πρώτα σε στοιχειώδεις μεθόδους, βασισμένες σε λίστες και πίνακες, οι οποίες μπορούν να εφαρμοστούν για την αναπαράσταση τόσο διατεταγμένων όσο και μη διατεταγμένων λεξικών. Στη συνέχεια, θα μελετήσουμε τη χρήση δυαδικών δένδρων αναζήτησης για την αναπαράσταση διατεταγμένων λεξικών. Τα δένδρα αυτά, σε συνδυασμό με τις τεχνικές ισορρόπησης τους, που αναπτύσσουμε στο Κεφάλαιο 8, δίνουν πολύ αποδοτικές λύσεις στο πρόβλημα του διατεταγμένου λεξικού.

7.2 Στοιχειώδεις υλοποιήσεις με πίνακες και λίστες

Μπορούμε να δώσουμε μια προφανή λύση στο πρόβλημα του λεξικού χρησιμοποιώντας μια από τις στοιχειώδεις δομές, πίνακα ή συνδεδεμένη λίστα, όπως είδαμε στο Κεφάλαιο 2. Αν το λεξικό είναι μη διατεταγμένο, τότε τα στοιχεία του τοποθετούνται στην αντίστοιχη δομή με τη σειρά εισαγωγής τους. Σε ένα διατεταγμένο λεξικό έχουμε τη δυνατότητα να διατηρούμε τα στοιχεία διατεταγμένα, π.χ. σε αύξουσα σειρά ως προς τα κλειδιά τους. Οι επιλογές αυτές επηρεάζουν το χρόνο εκτέλεσης των διαφόρων λειτουργιών του λεξικού, όπως φαίνεται στον Πίνακα 7.1.

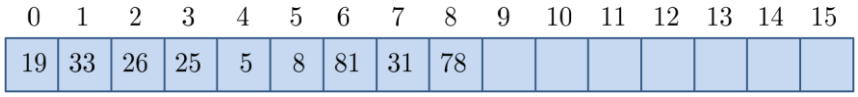
Πίνακας 7.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης μερικών βασικών λειτουργιών ενός λεξικού D με n στοιχεία, υλοποιημένου με στοιχειώδεις δομές. Για τη λειτουργία συγχώνευση(D′), n' είναι το πλήθος των στοιχείων στο λεξικό D′.

	αναζήτηση	εισαγωγή	συγχώνευση
μη διατεταγμένος πίνακας	$O(n)$	$O(1)$	$O(n+n')$
διατεταγμένος πίνακας	$O(\log n)$	$O(n)$	$O(n+n')$
μη διατεταγμένη λίστα	$O(n)$	$O(1)$	$O(1)$
διατεταγμένη λίστα	$O(n)$	$O(n)$	$O(n+n')$

Οποιαδήποτε από τις παραπάνω υλοποιήσεις λεξικού απαιτεί $O(n^2)$ χρόνο για την εκτέλεση μιας μικτής ακολουθίας από $O(n)$ εισαγωγές και αναζητήσεις στη χειρότερη περίπτωση. Σε

ορισμένες ειδικές περιπτώσεις, όπως αυτή που αναλύουμε στην Ενότητα 7.2.2, μπορούμε να επιτύχουμε πολύ καλύτερη απόδοση.

7.2.1 Υλοποίηση με πίνακα



Εικόνα 7.1: Αποθήκευση $n=9$ κλειδιών σε μη διατεταγμένο πίνακα.

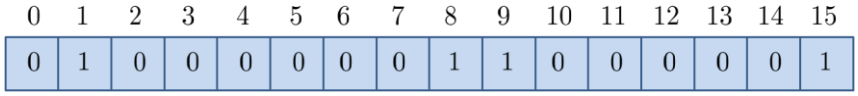
Χρησιμοποιούμε έναν πίνακα $T[0: N-1]$, N θέσεων. Υποθέτουμε ότι γνωρίζουμε ένα άνω φράγμα του πλήθους των στοιχείων, που χρειάζεται να αποθηκεύσει η δομή μας σε κάθε χρονική στιγμή, και θέτουμε την τιμή του N ίση με αυτό το άνω φράγμα. Μπορούμε να χειριστούμε την περίπτωση, όπου ένα τέτοιο άνω φράγμα για την τιμή του N δεν μας είναι γνωστό, όπως στο Κεφάλαιο 2. (Δείτε, επίσης, την ανάλυση στο Κεφάλαιο 14.)

Η αποθήκευση των στοιχείων σε ένα μη διατεταγμένο πίνακα γίνεται με τη σειρά εισαγωγής τους. Ένα νέο στοιχείο εισάγεται στην επόμενη κενή θέση του πίνακα. Επομένως, η εισαγωγή γίνεται σε σταθερό χρόνο, αρκεί να διατηρούμε σε μια ακέραιη μεταβλητή την τιμή της επόμενης κενής θέσης του πίνακα. Αυτό ισοδυναμεί με το να διατηρούμε το πλήθος n των στοιχείων που αποθηκεύονται στον πίνακα, όπως φαίνεται στην Εικόνα 7.1. Η αναζήτηση ενός στοιχείου πρέπει να διατρέξει ολόκληρο τον πίνακα στη χειρότερη περίπτωση. Αυτό συμβαίνει κάθε φορά που έχουμε μια ανεπιτυχή αναζήτηση ή όταν το στοιχείο που αναζητούμε είναι το τελευταίο, δηλαδή βρίσκεται στη θέση $T[n-1]$. Μια επιτυχής αναζήτηση θα χρειαστεί να εξετάσει τα πρώτα $n/2$ στοιχεία του πίνακα κατά μέσο όρο (υποθέτοντας ότι αναζητούμε ένα τυχαίο στοιχείο του πίνακα).

Η αποθήκευση σε διατεταγμένο πίνακα βελτιώνει το χρόνο αναζήτησης σε $O(\log n)$, χρησιμοποιώντας τη δυαδική αναζήτηση, που είδαμε στο Κεφάλαιο 2. Η διατήρηση της διάταξης μετά από κάθε εισαγωγή ή διαγραφή έχει ως αποτέλεσμα τη μετακίνηση έως και n στοιχείων στη χειρότερη περίπτωση.

Στην ειδική περίπτωση όπου ορίζονται αριθμητικές πράξεις πάνω στα κλειδιά (π.χ. όταν αυτά είναι τύπου Integer ή Double), μπορούμε να χρησιμοποιήσουμε τη μέθοδο της *αναζήτησης με παρεμβολή* (interpolation search), η οποία αποτελεί μια παραλλαγή της δυαδικής αναζήτησης που λαμβάνει υπόψη την κατανομή των κλειδιών. Η μέθοδος αυτή κατά κάποιο τρόπο μιμείται την αναζήτηση σε ένα ονομαστικό κατάλογο, όπου κοιτάμε πιο κοντά στην αρχή του καταλόγου, όταν το ζητούμενο όνομα ξεκινά με ένα από τα πρώτα γράμματα του αλφάβητου. Έστω k_1 και k_n το μικρότερο και το μεγαλύτερο κλειδί του λεξικού, αντίστοιχα, και έστω k το ζητούμενο κλειδί. Εκτελούμε δυαδική αναζήτηση, με τη διαφορά ότι, αντί να χωρίσουμε τον πίνακα στη μέση συγκρίνοντας με το μεσαίο κλειδί, τον χωρίζουμε στη θέση $\lfloor (k-k_1)/(k_n-k_1) \rfloor$. Η αναζήτηση επαναλαμβάνεται αναδρομικά στο κατάλληλο τμήμα του πίνακα, όπως και στη συνήθη δυαδική αναζήτηση.

7.2.2 Υλοποίηση με αριθμοδείκτη

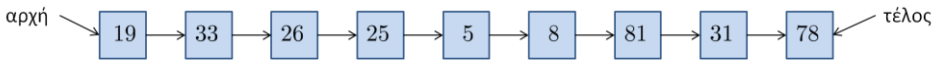


Εικόνα 7.2: Αποθήκευση των κλειδιών $1, 8, 9, 15$ με αριθμοδείκτη σε πίνακα τύπου boolean με $M = 16$ θέσεις.

Μια σημαντική ειδική περίπτωση, κατά την οποία μπορούμε να δώσουμε μια απλή αλλά αποδοτική λύση, είναι όταν τα κλειδιά των στοιχείων είναι μικροί ακέραιοι. Ας υποθέσουμε, αρχικά, ότι τα στοιχεία έχουν διακριτά κλειδιά. Σε αυτήν την περίπτωση, μπορούμε απλώς να αποθηκεύσουμε τα στοιχεία σε ένα πίνακα $T[0: M-1]$, όπου το στοιχείο x με κλειδί k αποθηκεύεται στη θέση T[k]. Το μέγεθος M του πίνακα T καθορίζεται από το πλήθος των διαφορετικών κλειδιών που μπορούν να έχουν τα στοιχεία που αποθηκεύουμε. Αν τα στοιχεία έχουν διακριτά κλειδιά, τότε ο πίνακας T μπορεί να είναι τύπου boolean, όπως δείχνει η Εικόνα 7.2. Οι λειτουργίες εισαγωγή, διαγραφή και αναζήτηση μπορούν να πραγματοποιηθούν σε $O(1)$ χρόνο. Οι υπόλοιπες λειτουργίες, όμως, απαιτούν χρόνο ανάλογο του M στη χειρότερη περίπτωση. Αν τα κλειδιά δεν είναι διακριτά, τότε μπορούμε να συγκεντρώσουμε κάθε στοιχείο με κλειδί k σε ένα υποσύνολο (υλοποιημένο π.χ. ως μια συνδεδεμένη λίστα) το οποίο δεικτοδοτείται από τη θέση T[k].

Η παραπάνω απλή δομή αποτελεί τη βάση της τεχνικής του κατακερματισμού, που μελετάμε στο Κεφάλαιο 10.

7.2.3 Υλοποίηση με συνδεδεμένη λίστα



Εικόνα 7.3: Αποθήκευση κλειδιών σε μη διατεταγμένη λίστα.

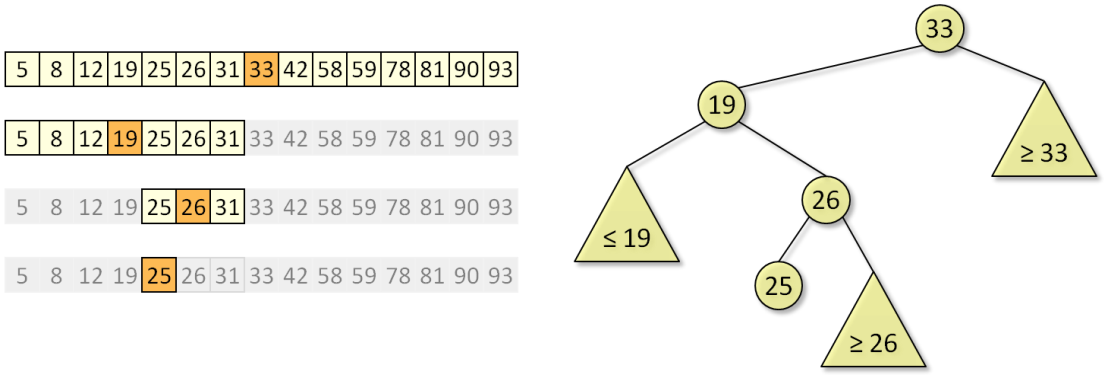
Αποθηκεύουμε τα στοιχεία σε μια απλά συνδεδεμένη λίστα, όπου η πρόσβαση γίνεται με μία αναφορά αρχή στον πρώτο κόμβο της λίστας. Όπως είδαμε στο Κεφάλαιο 2, η εισαγωγή ενός νέου στοιχείου πραγματοποιείται σε $O(1)$ χρόνο τοποθετώντας ένα νέο κόμβο στην αρχή της λίστας. Όπως και στην περίπτωση του μη διατεταγμένου πίνακα, η αναζήτηση ενός στοιχείου γίνεται ακολουθιακά, διατρέχοντας τη λίστα από τον πρώτο κόμβο που δείχνει η αναφορά αρχή. Επομένως, χρειάζεται να εξεταστούν n κόμβοι στη χειρότερη περίπτωση (όταν έχουμε μια ανεπιτυχή αναζήτηση ή όταν το στοιχείο που αναζητούμε βρίσκεται στον τελευταίο κόμβο). Αντίστοιχα, σε επιτυχή αναζήτηση θα χρειαστεί να εξεταστούν οι πρώτοι $n/2$ κόμβοι κατά μέσο όρο (υποθέτοντας ότι αναζητούμε ένα τυχαίο στοιχείο της λίστας).

Για να πραγματοποιήσουμε την ένωση του λεξικού D με ένα λεξικό D' χρειάζεται να διατρέξουμε μια από τις δύο λίστες –κατά προτίμηση τη μικρότερη από τις δύο– για να αποκτήσουμε πρόσβαση στον τελευταίο κόμβο. Αν είναι γνωστό το πλήθος των στοιχείων σε κάθε λεξικό, έστω n και n' αντίστοιχα, τότε η ένωσή τους με τον παραπάνω τρόπο απαιτεί $O(\min\{n, n'\})$ χρόνο. Ο χρόνος της ένωσης μπορεί να βελτιωθεί σε $O(1)$ με μια απλή προσθήκη μιας αναφοράς τέλος στον τελευταίο κόμβο της λίστας, όπως φαίνεται στην Εικόνα 7.3.

Η χρήση μιας διατεταγμένης λίστας έχει ως αποτέλεσμα όλες οι λειτουργίες που αναλύσαμε πιο πάνω να εκτελούνται σε χρόνο ανάλογο του πλήθους των στοιχείων της. Στη βιβλιογραφία έχει προταθεί η ιδέα των λιστών παράλειψης, οι οποίες γενικεύουν τη διατεταγμένη συνδεδεμένη λίστα, τοποθετώντας στους κόμβους της ένα μικρό αριθμό επιπλέον δεικτών. Με αυτόν τον τρόπο, κατά την αναζήτηση ενός στοιχείου μπορούμε να παραλείπουμε την εξέταση πολλών ενδιάμεσων κόμβων που έχει ως αποτέλεσμα την βελτίωση του χρόνου εκτέλεσης πολλών λειτουργιών.

7.3 Δυαδικά δένδρα αναζήτησης

Όπως είδαμε στην Ενότητα 7.2.1, η υλοποίηση ενός διατεταγμένου λεξικού με διατεταγμένο πίνακα προσφέρει καλό χρόνο αναζήτησης (μέσω δυαδικής αναζήτησης), αλλά είναι αναποτελεσματική ως προς τις υπόλοιπες λειτουργίες.



Εικόνα 7.4: Δυαδική αναζήτηση του κλειδιού 25 και η αναπαράστασή της με ένα δυαδικό δένδρο.

Η αναζήτηση μπορεί να γίνει με παρόμοιο τρόπο αν αποθηκεύσουμε τα κλειδιά στους κόμβους ενός δυαδικού δένδρου, όπως δείχνει η Εικόνα 7.4. Η δομή, που προκύπτει με αυτόν τον τρόπο ονομάζεται *δυαδικό δένδρο αναζήτησης*.

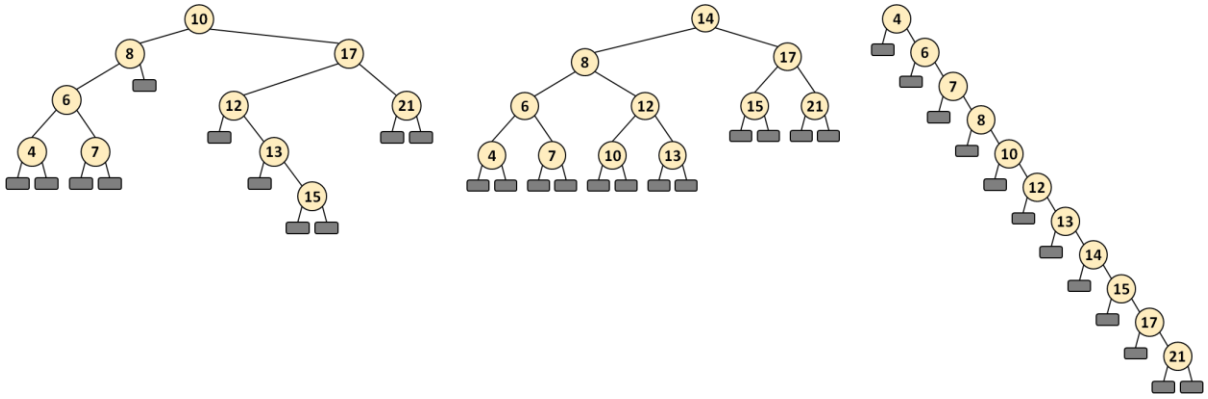
Ένα δυαδικό δένδρο αναζήτησης T είναι ένα διατεταγμένο δυαδικό δένδρο, στο οποίο κάθε κόμβος v αποθηκεύει ένα κλειδί, κλειδί(v), από ένα διατεταγμένο σύνολο, έτσι ώστε:

- Τα κλειδιά που είναι αποθηκευμένα στους κόμβους του αριστερού υποδένδρου του κόμβου v είναι μικρότερα ή ίσα του κλειδί(v).

- Τα κλειδιά που είναι αποθηκευμένα στους κόμβους του δεξιού υποδένδρου του κόμβου v είναι μικρότερα ή ίσα του κλειδί(v).

Από τον παραπάνω ορισμό προκύπτει ότι ένα δεδομένο σύνολο κλειδιών μπορεί να αναπαρασταθεί από πολλά δυαδικά δένδρα αναζήτησης, όπως φαίνεται στην Εικόνα 7.5. Όπως θα δούμε στις επόμενες ενότητες, η μορφή του δένδρου επηρεάζει την απόδοση των λειτουργιών που εκτελούνται στο δένδρο.

Η συμμετρική διάνυση (ενδοδιάταξη) σε οποιοδήποτε δυαδικό δένδρο αναζήτησης επιστρέφει τα κλειδιά, που αποθηκεύονται στο δένδρο σε διάταξη, από το μικρότερο προς το μεγαλύτερο. Με τον τρόπο αυτό, λαμβάνουμε μια υλοποίηση της λειτουργίας ταξινόμηση, η οποία, όπως έχουμε δει στο Κεφάλαιο 4, εκτελείται σε χρόνο $O(n)$ σε ένα δυαδικό δένδρο με n κόμβους.



Εικόνα 7.5: Διάφορα δυαδικά δένδρα αναζήτησης για δεδομένο σύνολο κλειδιών.

Πίνακας 7.2: Χρόνοι εκτέλεσης χειρότερης περίπτωσης μερικών βασικών λειτουργιών ενός λεξικού **D**, υλοποιημένου με δυαδικό δένδρο αναζήτησης ύψους $\boldsymbol{h}$. Για τη λειτουργία **συγχώνευση**(**D**′), $\boldsymbol{h'}$ είναι το ύψος του δυαδικού δένδρου αναζήτησης που αποθηκεύει το λεξικό **D**′.

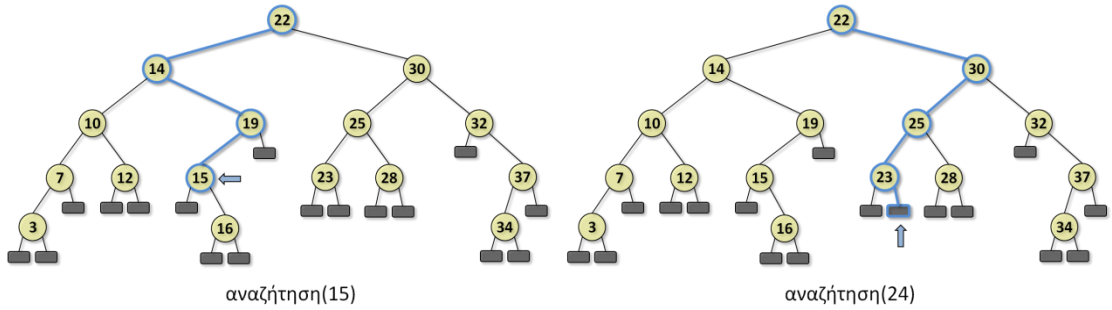
	αναζήτηση	**εισαγωγή**	**συγχώνευση**
Δυαδικό δένδρο αναζήτησης	$O(h)$	$O(h)$	$O(h + h')$

Ο Πίνακας 7.2 δίνει τους χρόνους εκτέλεσης μερικών λειτουργιών λεξικού υλοποιημένου με δυαδικό δένδρο αναζήτησης. Στις επόμενες ενότητες περιγράφουμε αλγόριθμους που εκτελούν τις βασικές λειτουργίες διατεταγμένου λεξικού σε ένα δυαδικό δένδρο αναζήτησης T. Η πρόσβαση στο δένδρο γίνεται μέσω της ρίζας του δένδρου, στην οποία αναφερόμαστε με το συμβολισμό T. ρίζα. Θα υποθέσουμε ότι τα στοιχεία που αποθηκεύει το λεξικό έχουν διακριτά κλειδιά (δύο διαφορετικά στοιχεία δεν μπορούν να έχουν το ίδιο κλειδί). Στην περίπτωση που εκτελείται μια λειτουργία εισαγωγή(x, k), όπου το κλειδί k υπάρχει ήδη στο δένδρο και αντιστοιχεί σε ένα στοιχείο y, η εισαγωγή αντικαθιστά το y με το x. Οι αλγόριθμοι που θα παρουσιάσουμε μπορούν να προσαρμοστούν ώστε να χειρίζονται την περίπτωση μη διακριτών κλειδιών. Δείτε τις Ασκήσεις 7.5 και 7.6.

7.3.1 Αναζήτηση

Η αναζήτηση ενός κλειδιού k ξεκινάει από τη ρίζα του δένδρου T και επισκέπτεται τους κόμβους ενός μονοπατιού, μέχρι να βρει ένα κόμβο v με κλειδί(v) = k, οπότε η αναζήτηση είναι επιτυχής, ή να καταλήξει σε κενό κόμβο, οπότε η αναζήτηση είναι ανεπιτυχής. Ο

αλγόριθμος αναζήτησης, όταν βρεθεί σε ένα ενδιάμεσο κόμβο v, θα μεταβεί στο αριστερό παιδί του v αν $k <$ κλειδί(v) ή στο δεξί παιδί του v αν $k >$ κλειδί(v). Δείτε την Εικόνα 7.6.



Εικόνα 7.6: Αναζήτηση στοιχείου με δεδομένο κλειδί. Το μονοπάτι αναζήτησης σημειώνεται με μπλε χρώμα.

Αλγόριθμος αναζήτηση(k)

v ← Τ.ρίζα

ενόσω v ≠ κενό

αν k = κλειδί(v), **τότε επιστροφή** v

αν k < κλειδί(v), **τότε** v ← αριστερός(v) **διαφορετικά** v ← δεξιός(v)

επιστροφή v

Η διαδικασία αναζήτησης μπορεί να περιγραφεί και με τη βοήθεια αναδρομής, όπως φαίνεται παρακάτω.

Αλγόριθμος αναζήτηση(v, k) – **Αναδρομική εκδοχή**

αν v = κενό, **τότε επιστροφή** v

αν k < κλειδί(v), **τότε επιστροφή** αναζήτηση(k, αριστερός(v))

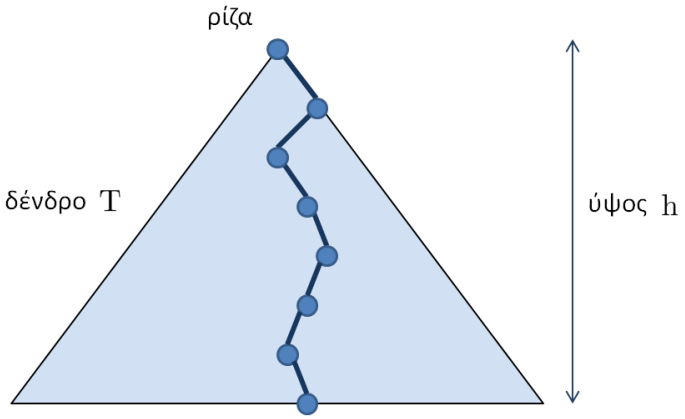
αν k > κλειδί(v), **τότε επιστροφή** αναζήτηση(k, δεξιός(v))

επιστροφή v

Στην αναδρομική εκδοχή της αναζήτησης χρησιμοποιούμε ένα επιπλέον όρισμα, τον κόμβο v ο οποίος δίνει την τρέχουσα θέση της αναζήτησης. Για να εκτελέσουμε την αναζήτηση του κλειδιού k σε ολόκληρο το δένδρο, καλούμε αναζήτηση(Τ.ρίζα, k).

Μια ανεπιτυχής αναζήτηση του κλειδιού k καταλήγει σε ένα κενό κόμβο του δένδρου, που δίνει, όμως, τη σωστή θέση του k στη διάταξη των κλειδιών του δένδρου. Η ιδιότητα αυτή είναι χρήσιμη για την εισαγωγή ενός νέου στοιχείου στο δένδρο, όπως θα δούμε στη συνέχεια.

Ο αλγόριθμος αναζήτησης δαπανά $O(1)$ χρόνο σε κάθε κόμβο v που επισκέπτεται. Αν η αναζήτηση δεν τερματιστεί στον v (δηλαδή κλειδί(v) ≠ k), τότε ο επόμενος κόμβος στον οποίο μεταβαίνει βρίσκεται στο αμέσως επόμενο επίπεδο του δένδρου. Επομένως, ο χρόνος αναζήτησης στη χειρότερη περίπτωση είναι $O(h)$, όπου h το ύψος του δένδρου. Δείτε την Εικόνα 7.7.



Εικόνα 7.7: Απεικόνιση της λειτουργίας αναζήτησης σε ένα δυαδικό δένδρο αναζήτησης. Ο αλγόριθμος αναζήτησης δαπανά σταθερό χρόνο σε κάθε επίπεδο του δένδρου.

Όπως έχουμε δει στο Κεφάλαιο 3, το ύψος ενός δυαδικού δένδρου h με n κόμβους ικανοποιεί την ανισότητα $\lg n \leq h \leq n$. Αυτό σημαίνει ότι ο χρόνος αναζήτησης είναι $O(n)$ στη χειρότερη περίπτωση, δηλαδή ανάλογος του χρόνου αναζήτησης σε μια συνδεδεμένη λίστα ή ένα μη ταξινομημένο πίνακα. Ωστόσο, αναμένουμε ότι σε αρκετές πρακτικές περιπτώσεις η τιμή του h μπορεί να είναι αρκετά μικρότερη. Πράγματι, στην Ενότητα 7.4, θα δούμε ότι το αναμενόμενο ύψος ενός κόμβου σε τυχαία κατασκευασμένο δυαδικό δένδρο αναζήτησης με n κλειδιά είναι $O(\log n)$. Στο Κεφάλαιο 8, αναπτύσσουμε διάφορες τεχνικές που εγγυώνται ότι ένα δυαδικό δένδρο διατηρεί ύψος $O(\log n)$ στη χειρότερη περίπτωση.

7.3.2 Εύρεση ελάχιστου και μέγιστου

Η εύρεση του ελάχιστου κλειδιού αντιστοιχεί σε μια αναζήτηση από τη ρίζα κατά την οποία ακολουθούμε πάντα τους συνδέσμους προς το αριστερό παιδί μέχρι να καταλήξουμε σε κόμβο v που δεν έχει αριστερό παιδί. Ο κόμβος v περιέχει το ελάχιστο κλειδί του δένδρου.

Αλγόριθμος ελάχιστο(v)

  ενόσω αριστερός(v) $\neq$ κενό

    $v \leftarrow$ αριστερός(v)

  επιστροφή v

Η εύρεση του μέγιστου κλειδιού είναι ανάλογη, με μόνη διαφορά ότι ακολουθούμε τους συνδέσμους προς το δεξί παιδί.

Αλγόριθμος μέγιστο(v)

ενόσω δεξιός(v) $\neq$ κενό

  $v \leftarrow$ δεξιός(v)

επιστροφή v

Επομένως, ο χρόνος εύρεσης του ελάχιστου ή του μέγιστου κλειδιού είναι $O(h)$.

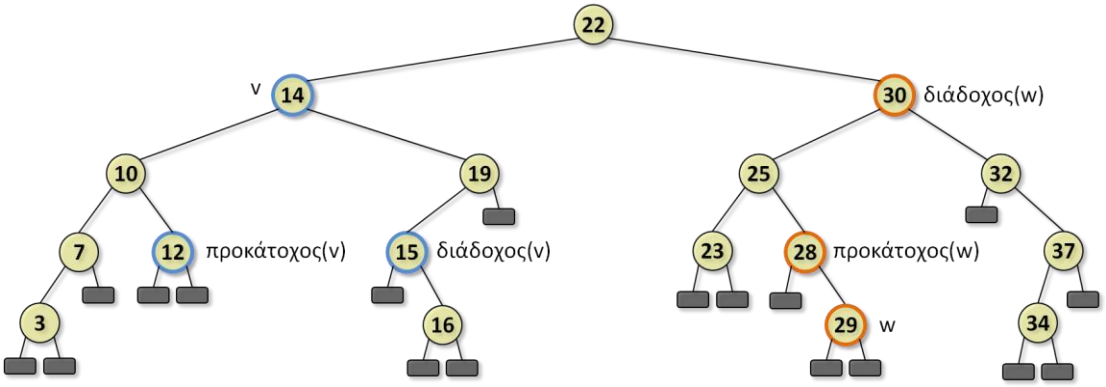
7.3.3 Προκάτοχος και διάδοχος

Στην αρχή του κεφαλαίου αναφέραμε τις λειτουργίες εύρεσης προκατόχου και διαδόχου ενός κλειδιού σε διατεταγμένο λεξικό. Εδώ εξετάζουμε μια χρήσιμη παραλλαγή των λειτουργιών αυτών, όπου η αναζήτηση γίνεται με βάση ένα κόμβο v του δένδρου. Η εύρεση προκατόχου και διαδόχου ενός κλειδιού μπορεί να γίνει με ανάλογο τρόπο. Δείτε την Άσκηση 7.3.

προκάτοχος(v) : Επιστρέφει τον κόμβο u του δένδρου με το μεγαλύτερο κλειδί, που είναι μικρότερο του κλειδί(v).

διάδοχος(v) : Επιστρέφει τον κόμβο u του δένδρου με το μικρότερο κλειδί, που είναι μεγαλύτερο του κλειδί(v).

Οι αλγόριθμοι εντοπισμού προκατόχου και διαδόχου βασίζονται στην παρατήρηση ότι οι θέσεις αυτών των κόμβων καθορίζονται μόνο από την τοπολογία του δένδρου, δίχως να χρειάζεται να εξετάσουμε τα κλειδιά. Δείτε την Εικόνα 7.8.



Εικόνα 7.8: Προκάτοχος και διάδοχος κόμβος σε δυαδικό δένδρο.

Περιγράφουμε την εύρεση διαδόχου. Η εύρεση προκατόχου γίνεται με ανάλογο τρόπο. Αν ο v έχει δεξί παιδί, τότε ο διάδοχος(v) είναι ο κόμβος με το μικρότερο κλειδί στο δεξί υποδένδρο του v. Διαφορετικά ο διάδοχος(v) είναι ο κοντινότερος πρόγονος u του v με κλειδί(u) ≥ κλειδί(v). Η εύρεση του u γίνεται εύκολα ως εξής: Με αφετηρία τον v, διατρέχουμε το μονοπάτι προς τη ρίζα, μέχρι να βρούμε τον πρώτο κόμβο το αριστερό παιδί του οποίου βρίσκεται, επίσης, στο ίδιο μονοπάτι από τον v προς τη ρίζα.

Αλγόριθμος διάδοχος(v)

αν δεξιός(v) ≠ κενό, **τότε επιστροφή** ελάχιστο(δεξιός(v))

u ← πατέρας(v)

ενόσω u ≠ κενό **και** v = δεξιός(u)

v ← u, u ← πατέρας(v)

επιστροφή u

Η αναζήτηση του διαδόχου του κόμβου v θα επισκεφθεί είτε ένα μονοπάτι στο δεξί υποδένδρο του v είτε ένα μονοπάτι από προγόνους του v, δηλαδή h κόμβους το πολύ. Εκτελείται, άρα, σε O(h) χρόνο.

7.3.4 Εισαγωγή

Η παρακάτω μέθοδος δημιουργεί ένα νέο κόμβο v, ο οποίος αποθηκεύει το στοιχείο x με κλειδί k.

Αλγόριθμος νέος_κόμβος(x, k)

δημιούργησε νέο κόμβο v και αποθήκευσε το στοιχείο x στον v

κλειδί(v) ← k, πατέρας(v) ← κενό, αριστερός(v) ← κενό, δεξιός(v) ← κενό

επιστροφή v

Ο αλγόριθμος εισαγωγής ενός νέου στοιχείου x με κλειδί k επεκτείνει την αναζήτηση του k. Αν η αναζήτηση καταλήξει σε κενό κόμβο, τότε το x τοποθετείται σε ένα νέο κόμβο που λαμβάνει τη θέση του κενού κόμβου στο δένδρο. Δείτε την Εικόνα 7.9. Διαφορετικά, η αναζήτηση συναντά ένα κόμβο v με το ίδιο κλειδί k, οπότε το x αντικαθιστά το στοιχείο που ήταν αποθηκευμένο στον v.

Αλγόριθμος εισαγωγή(x, k)

v ← T. ρίζα

ενόσω v ≠ κενό

z ← v

αν k < κλειδί(v), **τότε** v ← αριστερός(v)

αν k > κλειδί(v), **τότε** v ← δεξιός(v)

διαφορετικά

αντικατάστησε το στοιχείου του v με το x

επιστροφή

w ← νέος_κόμβος(x, k)

αν z = κενό, **τότε**

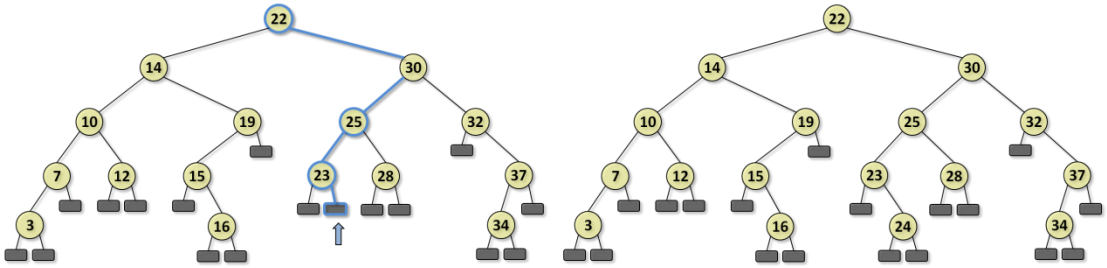
κάνε τον w ρίζα του T

επιστροφή

διαφορετικά

πατέρας(w) ← z

αν κλειδί(w) ≤ κλειδί(z), **τότε** αριστερός(z) ← w **διαφορετικά** δεξιός(z) ← w



Εικόνα 7.9: Εισαγωγή στοιχείου με δεδομένο κλειδί σε δυαδικό δένδρο αναζήτησης.

Όταν επιτρέπουμε πολλαπλά στοιχεία να έχουν το ίδιο κλειδί, τότε η αναζήτηση της θέσης εισαγωγής του νέου στοιχείου με κλειδί k δεν σταματά, αν συναντήσουμε κόμβο v με κλειδί(v) = k. Σε αυτήν την περίπτωση, η διαδικασία εισαγωγής μπορεί να συνεχιστεί είτε στο αριστερό είτε στο δεξί υποδένδρο του κόμβου v. Δείτε την Άσκηση 7.5.

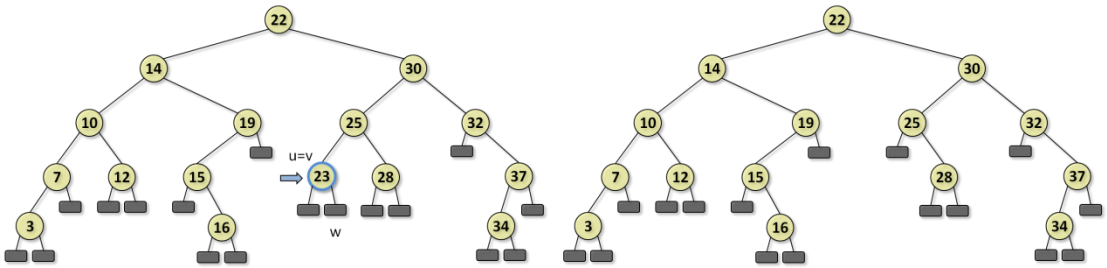
Ο χρόνος εισαγωγής είναι ανάλογος του χρόνου αναζήτησης της θέσης που θα τοποθετηθεί ο νέος κόμβος, δηλαδή O(h) σε δυαδικό δένδρο ύψους h.

7.3.5 Διαγραφή

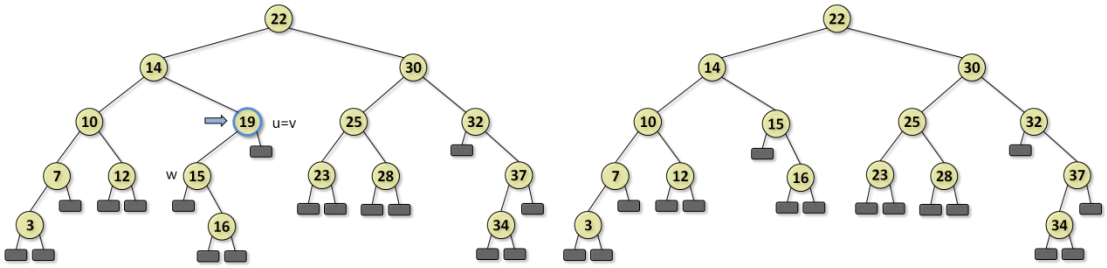
Η διαγραφή είναι πιο περίπλοκη από την εισαγωγή, γιατί θα πρέπει να ξεχωρίσουμε τρεις περιπτώσεις ανάλογα με το πλήθος των παιδιών του κόμβου v που διαγράφουμε:

1) Αν ο v δεν έχει παιδιά, τότε απλώς αντικαθίσταται από τον κενό κόμβο.
2) Αν ο v έχει μόνο ένα παιδί w, τότε ο w αντικαθιστά τον v.
3) Αν ο v έχει δύο παιδιά, τότε πρέπει να αντικατασταθεί από ένα κόμβο u με το πολύ ένα παιδί, έτσι ώστε να διατηρείται η συμμετρική διάταξη των κλειδιών. Κατάλληλες επιλογές για τον κόμβο u είναι ο προκάτοχος ή ο διάδοχος του v. (Και οι δύο υπάρχουν, αφού ο v έχει δύο παιδιά.) Για λόγους συνέπειας, εδώ θα επιλέγουμε πάντα το διάδοχο του κόμβου v.

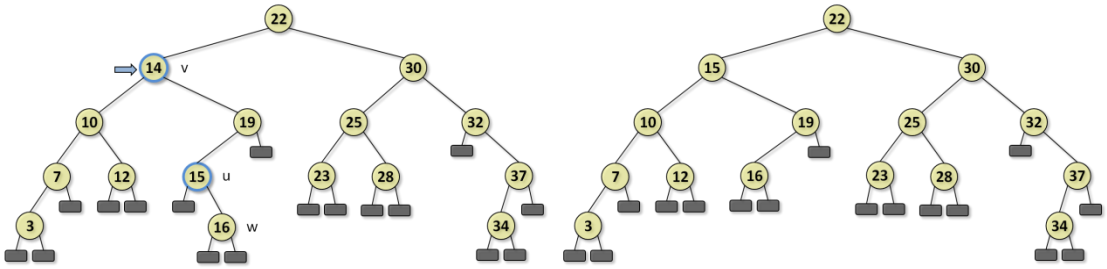
Οι παραπάνω περιπτώσεις απεικονίζονται στην Εικόνα 7.10.



Περίπτωση 1: διαγραφή του κόμβου με κλειδί 23



Περίπτωση 2: διαγραφή του κόμβου με κλειδί 19



Περίπτωση 3: διαγραφή του κόμβου με κλειδί 14

Εικόνα 7.10: Διαγραφή κόμβου σε δυαδικό δένδρο αναζήτησης.

Ο ακόλουθος αλγόριθμος αναλαμβάνει τις περιπτώσεις 1 και 2, όπου διαγράφεται ένας κόμβος u με το πολύ ένα παιδί w, το οποίο –εφόσον υπάρχει– λαμβάνει τη θέση του u στο δένδρο.

Αλγόριθμος απλή_διαγραφή(u)

αν αριστερός(u) = κενό, **τότε** w ← δεξιός(u) **διαφορετικά** w ← αριστερός(u)

αν w ≠ κενό, **τότε** πατέρας(w) ← πατέρας(u)

αν u = T. ρίζα, **τότε** κάνε τον w ρίζα του T

διαφορετικά

αν u = αριστερός(πατέρας(u)), **τότε** αριστερός(πατέρας(u)) ← w

διαφορετικά δεξιός(πατέρας(u)) ← w

διάγραψε τον κόμβο u

Ο παραπάνω αλγόριθμος ελέγχει, επίσης, αν ο κόμβος u είναι η ρίζα του δένδρου. Σε αυτήν την περίπτωση, ο θυγατρικός κόμβος w γίνεται η νέα ρίζα του δένδρου. Αν ο w είναι κενός κόμβος, τότε το νέο δένδρο είναι κενό.

Η περίπτωση 3 ανάγεται στην περίπτωση 1 ή 2 με την εναλλαγή του ρόλου του κόμβου v με τον διάδοχό του κόμβο u. Είναι εύκολο να δείξουμε ότι, αν ο v έχει δύο παιδιά, τότε ο u έχει το πολύ ένα παιδί. Δείτε την Άσκηση 7.4. Καταλήγουμε, λοιπόν, στον παρακάτω αλγόριθμο διαγραφής, ο οποίος αναλαμβάνει και τις τρεις περιπτώσεις.

Αλγόριθμος διαγραφή(v)

αν αριστερός(v) ≠ κενό ή δεξιός(v) ≠ κενό, **τότε** u ← v

διαφορετικά

u ← διάδοχος(v)

κλειδί(v) ← κλειδί(u)

αντίγραψε στον v το στοιχείο που αποθηκεύει ο u

απλή_διαγραφή(u)

Ο χρόνος εκτέλεσης της λειτουργίας απλή_διαγραφή είναι σταθερός. Επομένως, η λειτουργία διαγραφή εκτελείται, στη χειρότερη περίπτωση, σε χρόνο ανάλογο της εύρεσης του διαδόχου ενός κόμβου, δηλαδή $O(h)$.

7.3.6 Επιλογή

Η λειτουργία αυτή εκτελεί αναζήτηση στο δυαδικό δένδρο με βάση τη σειρά διάταξης των κόμβων του ως προς τα κλειδιά που αποθηκεύουν. Για να υποστηρίξουμε την επιλογή με αποδοτικό τρόπο, πρέπει να γνωρίζουμε για κάθε κόμβο v το πλήθος των απογόνων του στο δένδρο. Θεωρούμε ότι η πληροφορία αυτή δίνεται από το πεδίο πλήθος(v), όπου συμπεριλαμβάνουμε τον v στο πλήθος των απογόνων του. Ο παρακάτω αλγόριθμος δίνει μια αναδρομική υλοποίηση.

Αλγόριθμος επιλογή(v, j)

αν αριστερός(v) ≠ κενό, **τότε** i ← πλήθος(αριστερός(v))

διαφορετικά i ← 0

αν $i + 1 > j$, **τότε επιστροφή** επιλογή(αριστερός(v), j)

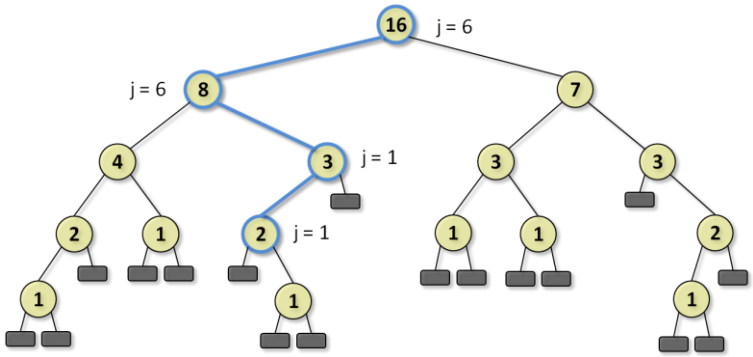
αν $i + 1 < j$, **τότε επιστροφή** επιλογή(δεξιός(v), $j - i - 1$)

επιστροφή v

Η μέθοδος δέχεται δύο ορίσματα, ένα ακέραιο j και έναν κόμβο v. Επιστρέφει τον απόγονο του v ο οποίος περιέχει το j-οστό μικρότερο κλειδί στο υποδένδρο με ρίζα v. Η αναζήτηση βασίζεται στην ακόλουθη παρατήρηση. Έστω i το πλήθος των κλειδιών που είναι αποθηκευμένα στο αριστερό υποδένδρο του v. Αν $j = i + 1$, τότε ο v περιέχει το j-οστό μικρότερο κλειδί και είναι ο κόμβος που αναζητάμε. Αν $j < i + 1$, τότε αναζητούμε τον κόμβο με το j-οστό μικρότερο κλειδί στο αριστερό υποδένδρο του v. Τέλος, αν $j > i + 1$, τότε αναζητούμε το $(j - i + 1)$-οστό μικρότερο κλειδί στο δεξί υποδένδρο του v. Δείτε την Εικόνα 7.11.

Για να εντοπίσουμε τον κόμβο με το j-οστό μικρότερο κλειδί σε ολόκληρο το δένδρο, εκτελούμε τη λειτουργία επιλογή(T.ρίζα, j). Όπως και στην περίπτωση της λειτουργίας

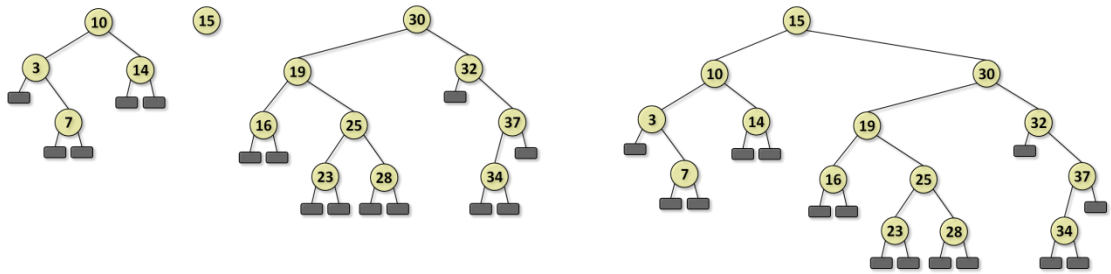
αναζήτηση, ο χρόνος εκτέλεσης της λειτουργίας επιλογή είναι O(h) στη χειρότερη περίπτωση.



Εικόνα 7.11: Επιλογή κόμβου σε δυαδικό δένδρο αναζήτησης. Σε κάθε κόμβο **v** δίνεται η τιμή του πεδίου **πλήθος(v)**. Επιλέγεται ο κόμβος με το έκτο μικρότερο κλειδί.

7.3.7 Ένωση

Η λειτουργία ένωση(x, D′) πραγματοποιείται πολύ εύκολα σε σταθερό χρόνο με την εκτέλεση των παρακάτω βημάτων. Έστω T και T′ τα δυαδικά δένδρα αναζήτησης, που αναπαριστούν τα λεξικά D και D′, αντίστοιχα, και έστω k το κλειδί του στοιχείου x. Δημιουργούμε ένα νέο κόμβο v με κλειδί k, ο οποίος αποθηκεύει το στοιχείο x, και τον θέτουμε ως ρίζα του νέου δυαδικού δένδρου με αριστερό υποδένδρο το T και δεξί υποδένδρο το T′. Η Εικόνα 7.12 δείχνει αυτή τη διαδικασία.



Εικόνα 7.12: Ένωση δύο δυαδικών δένδρων αναζήτησης με νέα ρίζα.

Αλγόριθμος ένωση(T_1, x, T_2)

$r_1 \leftarrow T_1.$ρίζα, $r_2 \leftarrow T_2.$ρίζα

v ← νέος_κόμβος(x, k)

κάνε τον v ρίζα ενός νέου δυαδικού δένδρου T

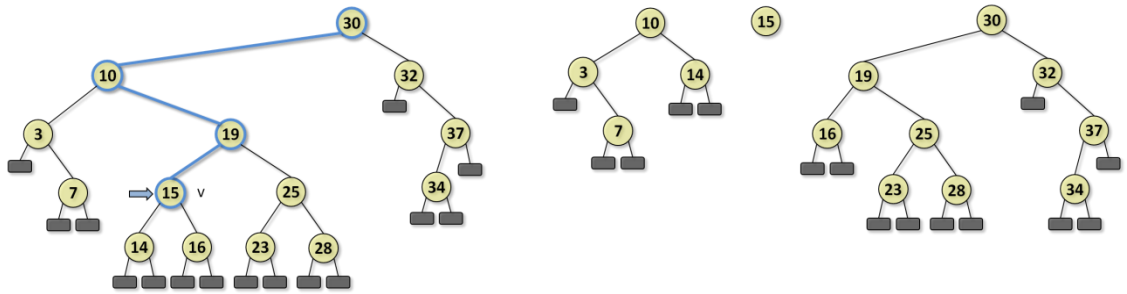
αριστερός(v) ← r_1, δεξιός(v) ← r_2

πατέρας(r_1) ← v, πατέρας(r_2) ← v

επιστροφή v

7.3.8 Διαχωρισμός

Ο διαχωρισμός ενός δυαδικού δένδρου μπορεί να επιτευχθεί με την εκτέλεση μιας ακολουθίας συνδέσεων. Η γενική ιδέα έχει ως εξής: Έστω v ο κόμβος που περιέχει το στοιχείο στο οποίο χωρίζεται το δυαδικό δένδρο T. Αφαιρούμε από το T τις ακμές του μονοπατιού P(v) από τη ρίζα προς το v. Με τον τρόπο αυτό, χωρίζουμε το T σε ένα σύνολο κόμβων, που ανήκουν στο P(v), και σε ένα σύνολο υποδένδρων, όπου η ρίζα κάθε τέτοιου υποδένδρου είναι παιδί ενός κόμβου του P(v). Τέλος, ενώνουμε διαδοχικά σε ένα δένδρο τα υποδένδρα, που περιέχουν κλειδιά μικρότερα του κλειδιού του v και, ομοίως, ενώνουμε διαδοχικά σε ένα δένδρο τα υποδένδρα που περιέχουν κλειδιά μεγαλύτερα του κλειδιού του v. Ένα παράδειγμα αυτής της διαδικασίας φαίνεται στην Εικόνα 7.13.



Εικόνα 7.13: Διαχωρισμός δυαδικού δένδρου αναζήτησης. Ο διαχωρισμός γίνεται στον κόμβο με κλειδί 15.

Αυτή η διαδικασία υλοποιείται από τον αλγόριθμο που ακολουθεί.

Αλγόριθμος διαχωρισμός(v)

u ← v, w ← πατέρας(u)

ενόσω w ≠ T.ρίζα

x ← στοιχείο του κόμβου w

l ← αριστερός(u), r ← δεξιός(u)

αν u = αριστερός(w), **τότε** w ← ένωση(r, x, δεξιός(w))

διαφορετικά w ← ένωση(αριστερός(w), x, l)

u ← w, w ← πατέρας(u)

Η ορθότητα του αλγόριθμου διαχωρισμού προκύπτει εύκολα από τη διάταξη των κλειδιών στο μονοπάτι T(v). Δείτε την Άσκηση 7.7. Αφού η ένωση εκτελείται σε σταθερό χρόνο, η παραπάνω μέθοδος δαπανά O(1) χρόνο σε κάθε κόμβο w που επισκέπτεται. Άρα, και η λειτουργία διαχωρισμός έχει χρόνο εκτέλεσης O(h) στη χειρότερη περίπτωση.

7.4. Τυχαία κατασκευασμένα δυαδικά δένδρα αναζήτησης

Οι επιδόσεις ενός δυαδικού δένδρου αναζήτησης για την εκτέλεση των λειτουργιών που έχουμε δει στις προηγούμενες ενότητες εξαρτώνται από το ύψος h του δένδρου. Στο επόμενο κεφάλαιο, παρουσιάζουμε μερικές τεχνικές που εγγυώνται ότι το h βρίσκεται πολύ κοντά στη μικρότερη δυνατή τιμή του, που είναι περίπου $\lg n$ για n κόμβους. Αυτό επιτυγχάνεται με το να επεκτείνουμε τους αλγόριθμους εισαγωγής και διαγραφής, που περιγράψαμε στις ενότητες 7.3.4 και 7.3.5, έτσι ώστε να εκτελούν πράξεις ισορρόπησης του δένδρου.

Στην ενότητα αυτή, θα δείξουμε ότι μια παρόμοια ιδιότητα ισχύει για δένδρα που κατασκευάζονται από n τυχαία κλειδιά.

Ιδιότητα 7.1 Το μέσο ύψος ενός κόμβου σε ένα δυαδικό δένδρο αναζήτησης που κατασκευάζεται από n τυχαία κλειδιά είναι περίπου $1{,}39 \lg n$.

Για να αποδείξουμε την παραπάνω ιδιότητα μπορούμε να θεωρήσουμε ότι τα κλειδιά ενός δεδομένου συνόλου n κλειδιών εισάγονται με τυχαία σειρά στο δυαδικό δένδρο. Παρατηρούμε ότι το κλειδί k, που εισάγεται πρώτο, έχει πιθανότητα $1/n$ να είναι το $(j+1)$-οστό μικρότερο για οποιοδήποτε τιμή του j από 0 έως $n-1$. Η επιλογή του πρώτου κλειδιού χωρίζει την ακολουθία των κλειδιών σε j κλειδιά μικρότερα του k, τα οποία εισάγονται στο αριστερό υποδένδρο της ρίζας, και σε $n-j-1$ κλειδιά μεγαλύτερα του k, που εισάγονται στο δεξί υποδένδρο της ρίζας.

Εκφράζουμε μαθηματικά τον παραπάνω συλλογισμό με τη βοήθεια αναδρομής, αφού το αριστερό και το δεξί υπόδενδρο της ρίζας είναι επίσης τυχαία κατασκευασμένα δυαδικά δένδρα αναζήτησης. Έστω $\Delta(n)$ το μέσο μήκος διαδρομής στο δένδρο (δηλαδή το άθροισμα του ύψους όλων των κόμβων). Η επιλογή του $(j+1)$-οστού μικρότερου κλειδιού στη ρίζα δίνει δυαδικό δένδρο με μέσο μήκος διαδρομής $(n-1) + \Delta(j) + \Delta(n-j-1)$. Αυτό προκύπτει από το γεγονός ότι η ρίζα συνεισφέρει μία μονάδα ύψους σε καθένα από τους $n-1$ υπόλοιπους κόμβους του δένδρου. Υπολογίζοντας το μέσο όρο για κάθε δυνατή τιμή του j, λαμβάνουμε την ακόλουθη αναδρομική σχέση

$$\Delta(n) = (n-1) + \frac{1}{n}\sum_{j=0}^{n-1}\big(\Delta(j) + \Delta(n-j-1)\big)$$

όπου $\Delta(1) = 0$.

Καθένας από τους όρους $\Delta(j)$ του αθροίσματος εμφανίζεται δύο φορές, οπότε η παραπάνω σχέση απλοποιείται και λαμβάνει την ακόλουθη μορφή

$$\Delta(n) = (n-1) + \frac{2}{n}\sum_{j=0}^{n-1}\Delta(j)\,.$$

Άρα έχουμε

$$n\Delta(n) - (n-1)\Delta(n-1) = n(n-1) - (n-1)(n-2) + 2\Delta(n-1)$$

$$n\Delta(n) = (n+1)\Delta(n-1) + 2(n-1)$$

Αντικαθιστούμε $\Delta(n) = (n+1)A(n)$, οπότε λαμβάνουμε

$$A(n) = A(n-1) + \frac{2(n-1)}{n(n+1)}$$

με αρχική συνθήκη $A(1) = A(0) = 0$.

Αναλύοντας το δεύτερο όρο του αθροίσματος σε μερικά κλάσματα, έχουμε

$$A(n) = A(n-1) + \frac{4}{n+1} - \frac{2}{n}$$

Η παραπάνω αναδρομική σχέση επιλύεται εύκολα με τη μέθοδο της αντικατάστασης (δείτε το Κεφάλαιο 2), οπότε και λαμβάνουμε

$$A(n) = \sum_{j=1}^{n} \left(\frac{4}{j+1} - \frac{2}{j} \right) = 4 \sum_{j=2}^{n+1} \frac{1}{j} - 2 \sum_{j=1}^{n} \frac{1}{j} = 4H(n+1) - 2H(n) - 4$$

όπου $H(N) = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N} \approx \ln N$ ο N-οστός αρμονικός αριθμός. Επομένως, $A(n) \approx 2 \ln n \approx 1{,}39 \log n$. Τέλος, η ιδιότητα προκύπτει από την παρατήρηση ότι η ποσότητα $A(n)$ προσεγγίζει το μέσο ύψος ενός κόμβου στο δένδρο.

7.5. Υλοποίηση δυαδικών δένδρων αναζήτησης σε Java

Περιγράφουμε μια κλάση BinarySearchTree δυαδικού δένδρου αναζήτησης, η οποία περιλαμβάνει μια εμφωλευμένη κλάση BSTreeNode των κόμβων του δένδρου. Κάθε κόμβος αποθηκεύει αντικείμενα τύπου Item με κλειδιά συγκρίσιμου τύπου Key. Για την υποστήριξη της λειτουργίας επιλογή, αποθηκεύουμε την τιμή πλήθος(v), δηλαδή το πλήθος των απογόνων του κόμβου v στο δένδρο, στο πεδίο v.size. Η σύνδεση των κόμβων γίνεται με τα πεδία v.left (αριστερό παιδί του v), v.right (δεξί παιδί του v) και v.parent (πατέρας του v).

```
public class BinarySearchTree<Key extends Comparable<Key>, Item>
{
    private BSTreeNode root;        // ρίζα του δένδρου

    private class BSTreeNode
    {
        private Key key;            // κλειδί κόμβου
        private Item item;          // αντικείμενο που αποθηκεύει ο κόμβος
        private int size;           // πλήθος απογόνων
        private BSTreeNode left;    // αριστερό παιδί
        private BSTreeNode right;   // δεξί παιδί
        private BSTreeNode parent;  // πατέρας

        /* δημιουργία νέου κόμβου με size=1 */
        public BSTreeNode(Key key, Item item, BSTreeNode parent) {
            this.key     = key;
            this.item    = item;
            this.parent  = parent;
            this.size    = 1;
        }
    }

    /* αναζήτηση κόμβου με κλειδί key */
    private BSTreeNode searchNode(Key key) {
        BSTreeNode v = root;
        while ( v != null ) {
            int c = key.compareTo(v.key);  // σύγκριση με το κλειδί του κόμβου v
            if (c < 0)      v = v.left;
            else if (c > 0) v = v.right;
            else return v;                 // βρέθηκε
```

```
        }
        return null;                        // δε βρέθηκε
    }

    /* αναζήτηση αντικειμένου με κλειδί key */
    public Item search(Key key) {
        BSTreeNode v = searchNode(key);
        return ( v != null ) ? v.item : null;
    }

    /* οι υπόλοιπες μέθοδοι της κλάσης BinarySearchTree περιγράφονται παρακάτω */
}
```

Μετά από την εισαγωγή ή διαγραφή ενός κόμβου v πρέπει να ανανεώσουμε τις τιμές των πεδίων u.size για κάθε πρόγονο u του v στο δένδρο. Αυτό μπορεί να γίνει με τη βοήθεια των συνδέσμων προς τον πατέρα που διαθέτει κάθε κόμβος, όπως φαίνεται στη μέθοδο update.

```
    /* επικαιροποίηση του πεδίου size των προγόνων του v */
    private void update(BSTreeNode v) {
        int leftSize, rightSize;
        BSTreeNode u = v;
        while ( u != null ) {
            leftSize  = (u.left != null)  ?  u.left.size  : 0;
            rightSize = (u.right != null) ?  u.right.size : 0;
            u.size = leftSize + rightSize + 1;
            u = u.parent;
        }
    }
```

Η μέθοδος insert υλοποιεί τον αλγόριθμο εισαγωγής της Ενότητας 7.3.4. Μετά την εισαγωγή ενός νέου κόμβου v, καλεί τη μέθοδο update για την επικαιροποίηση του πεδίου size των προγόνων του v.

```
    /* εισαγωγή αντικειμένου item με κλειδί key */
    public void insert(Key key, Item item) {
        BSTreeNode  v = root;
        BSTreeNode pv = null;  // πατέρας του v
        boolean left  = false; // αληθές αν v == pv.left;
        while ( v!=null ) {
            int c = key.compareTo(v.key);  // σύγκριση με το κλειδί του v
            pv = v;
            if      ( c < 0 )  { v = v.left;  left = true;  }
            else if ( c > 0 )  { v = v.right; left = false; }
            else { /* το κλειδί key υπάρχει στο δένδρο - αποθηκεύουμε το item
                      στη θέση του προηγούμενου αντικειμένου του κόμβου        */
                   v.item = item;
                   return;
            }
        }
        v = new BSTreeNode(key, item, pv);
        if ( pv == null ) root = v;   // ο νέος κόμβος γίνεται ρίζα του δένδρου
        else {
            if ( left ) pv.left = v;
            else pv.right = v;
            update(pv);
        }
    }
```

Στη συνέχεια, υλοποιούμε τη μέθοδο delete η οποία διαγράφει από το δένδρο το αντικείμενο με κλειδί key. Η μέθοδος αυτή χρησιμοποιεί τον αλγόριθμο διαγραφής κόμβου της Ενότητας 7.3.4. Χρησιμοποιούμε δύο βοηθητικές μεθόδους. Η μέθοδος min(v) επιστρέφει τον κόμβο με το ελάχιστο κλειδί στο υποδένδρο του κόμβου v. Την χρησιμοποιούμε για την εύρεση του

διαδόχου του κόμβου, που θέλουμε να διαγράψουμε, όταν αυτός έχει δύο παιδιά. Η μέθοδος simpleDelete υλοποιεί τον αλγόριθμο της απλής διαγραφής ενός κόμβου με το πολύ ένα παιδί. Μετά τη διαγραφή, η μέθοδος delete καλεί την update για την επικαιροποίηση του πεδίου size των προγόνων του κόμβου που διαγράφηκε.

```
/* επιστρέφει τον απόγονο του v με το ελάχιστο κλειδί */
private BSTreeNode min(BSTreeNode v) {
    BSTreeNode u = v;
    while ( u.left != null ) u = u.left;
    return u;
}

/* διαγραφή κόμβου u με το πολύ ένα παιδί */
private void simpleDelete(BSTreeNode u) {
    BSTreeNode parent  = u.parent;
    BSTreeNode child   = u.left;
    if ( child == null ) child = u.right; // ο u δεν έχει αριστερό παιδί

    if ( parent == null ) {  // ο u είναι η ρίζα του δένδρου
        root = child;
        root.parent = null;
        return;
    }

    if ( parent.left == u ) parent.left = child;
    else parent.right = child;

    if (child != null) child.parent = parent;

    u = null;
}

/* διαγραφή κόμβου με κλειδί key */
public void delete(Key key) {
    BSTreeNode v = searchNode(key);

    if (v == null) return; // ο κόμβος δεν υπάρχει

    if (root.size == 1) {  // η ρίζα είναι ο μοναδικός κόμβος του δένδρου
        root = null;
        v = null;
        return;
    }

    BSTreeNode u;

    if ( ( v.left == null ) || ( v.right == null ) ) u = v;
    else {  // ο v έχει δύο παιδιά – βρίσκουμε το διάδοχο του u
        u = min(v.right);
        v.item = u.item;
        v.key = u.key;
    }

    v = null;
    BSTreeNode parent = u.parent;
    simpleDelete(u);
    update(parent);
}
```

Τέλος, η μέθοδος select υλοποιεί τον αναδρομικό αλγόριθμο της Ενότητας 7.3.5 για την επιλογή του j-οστού μικρότερου κλειδιού.

```
/* αναδρομικός αλγόριθμος επιλογής */
private BSTreeNode select(BSTreeNode v, int j) {
    if ( v == null ) return null;
    int i = ( v.left != null ) ? v.left.size : 0;
    if ( i+1 > j ) return select(v.left, j);
    if ( i+1 < j ) return select(v.right, j-i-1);
    return v;
}

public Key select(int j) {
    BSTreeNode v = select(root, j);
    return v.key;
}
```

Ασκήσεις

7.1 Υλοποιήστε σε Java τη λειτουργία ένωσης δύο λεξικών, όταν αυτά αναπαρίστανται με α) δύο μη διατεταγμένες λίστες και β) δύο διατεταγμένες λίστες.

7.2 Υλοποιήστε σε Java τη λειτουργία διαχωρισμού ενός λεξικού, όταν αναπαρίσταται με α) μια μη διατεταγμένη λίστα και β) μια διατεταγμένη λίστα.

7.3 Δείξτε πώς μπορεί να υλοποιηθεί η εύρεση προκάτοχου και διάδοχου κλειδιού σε ένα δυαδικό δένδρο αναζήτησης.

7.4 Περιγράψτε πώς ένα δυαδικό δένδρο αναζήτησης μπορεί να υποστηρίξει τις λειτουργίες εξαγωγή_ελάχιστου και εξαγωγή_μέγιστου.

7.5 Δείξτε πώς μπορεί να υλοποιηθεί ο αλγόριθμος εισαγωγής σε δυαδικό δένδρο αναζήτησης, όταν επιτρέπεται να έχουμε πολλαπλά στοιχεία με το ίδιο κλειδί.

7.6 Θεωρήστε ένα δυαδικό δένδρο αναζήτησης, που μπορεί να έχει πολλαπλά στοιχεία με το ίδιο κλειδί. Δώστε έναν αποδοτικό αλγόριθμο για τη λειτουργία εύρεση_όλων(k) η οποία βρίσκει όλα τα στοιχεία με κλειδί k. Ποιος είναι ο χρόνος εκτέλεσης του αλγόριθμού σας σε ένα δυαδικό δένδρο αναζήτησης ύψους h το οποίο έχει s στοιχεία με κλειδί k;

7.7 Περιγράψτε έναν αποδοτικό αλγόριθμο για τη λειτουργία εύρεση_διαστήματος(k, m), η οποία βρίσκει όλα τα στοιχεία με κλειδί στο διάστημα [k, ... , m]. Ποιος είναι ο χρόνος εκτέλεσης του αλγόριθμού σας σε ένα δυαδικό δένδρο αναζήτησης ύψους h το οποίο έχει s στοιχεία με κλειδιά στο ζητούμενο διάστημα.

7.8 Περιγράψτε έναν αποδοτικό αλγόριθμο για τη λειτουργία πλήθος(k, m) η οποία επιστρέφει το πλήθος των στοιχείων με κλειδί στο διάστημα [k, ... , m]. Ποιος είναι ο χρόνος εκτέλεσης του αλγόριθμού σας σε ένα δυαδικό δένδρο αναζήτησης ύψους h;

7.9 Δώστε αποδοτικούς αλγόριθμους για την εύρεση προκάτοχου και διάδοχου κόμβου σε ένα δυαδικό δένδρο όπου οι κόμβοι δε διαθέτουν σύνδεσμο προς τους γονείς.

7.10 Δείξτε ότι ο διάδοχος ενός κόμβου με δύο παιδιά έχει το πολύ ένα παιδί.

7.11 Δώστε μια μη αναδρομική υλοποίηση του αλγόριθμου επιλογής.

7.12 Περιγράψτε έναν αποδοτικό αλγόριθμο για τη λειτουργία τάξη(k) η οποία επιστρέφει το πλήθος των κλειδιών που είναι μικρότερα ή ίσα του k.

7.13 Έστω ότι θέλουμε να διατηρήσουμε σε κάθε κόμβο v ενός δυαδικού δένδρου αναζήτησης ένα ακόμα πεδίο, ύψος(v), το οποίο αποθηκεύει το ύψος του κόμβου στο δένδρο. Δείξτε πώς μπορεί να ενημερωθεί το πεδίο αυτό μετά από μια εισαγωγή ή διαγραφή. Ο συνολικός χρόνος εισαγωγής και διαγραφής πρέπει να παραμείνει O(h).

7.14 Αιτιολογείστε την ορθότητα του αλγόριθμου διαχωρισμού της Ενότητας 7.3.7.

7.15 Υλοποιήστε σε Java τους αλγόριθμους ένωσης και διαχωρισμού των Ενοτήτων 7.3.6 και 7.3.7.

7.16 Δείξτε πώς μπορεί να υλοποιηθεί η λειτουργία συγχώνευσης δύο λεξικών από τα οποία το καθένα αναπαρίσταται από ένα δυαδικό δένδρο αναζήτησης με χρήση των λειτουργιών διαχωρισμού και ένωσης.

Πειραματικές Μελέτες

7.17 Συγκρίνετε πειραματικά την απόδοση της δυαδικής αναζήτησης και της αναζήτησης με παρεμβολή σε ένα διατεταγμένο πίνακα με n τυχαίους ακέραιους για διάφορες τιμές του n.

7.18 Συγκρίνετε πειραματικά την απόδοση των στοιχειωδών υλοποιήσεων ενός διατεταγμένου λεξικού και της υλοποίησης με δυαδικό δένδρο αναζήτησης, με την εισαγωγή και αναζήτηση n τυχαίων κλειδιών για διάφορες τιμές του n.

7.19 Υλοποιήστε ένα πρόγραμμα πελάτη λεξικού, το οποίο διαβάζει ένα κείμενο από το ρεύμα εισόδου και μετρά τη συχνότητα εμφάνισης της κάθε λέξης.

7.20 Προσδιορίστε πειραματικά πώς μεταβάλλεται το ύψος ενός δυαδικού δένδρου αναζήτησης με την εκτέλεση μιας ακολουθίας $m \geq n$ εναλλασσομένων τυχαίων λειτουργιών εισαγωγής και διαγραφής σε ένα τυχαία κατασκευασμένο δένδρο με n κόμβους για διάφορες τιμές των n και m.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μπoζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Κεφάλαιο 8
Ισορροπημένα Δένδρα Αναζήτησης

Περιεχόμενα

8.1 Κατηγορίες ισορροπημένων δένδρων αναζήτησης 155
8.1.1 Περιστροφές 156
8.2 Δένδρα AVL 157
8.2.1 Αποκατάσταση συνθήκης ισορροπίας 158
8.3 Αρθρωτά Δένδρα 166
8.3.1 Ιδιότητες των αρθρωτών δένδρων 171
8.3.2 Υλοποίηση σε Java 172
8.4 (a,b)-Δένδρα 174
8.4.1 Ύψος ενός (a,b)-δένδρου 175
8.4.2 Διάσπαση και συγχώνευση κόμβων 176
8.4.3 (2,4)-δένδρα 176
8.5 Κοκκινόμαυρα δένδρα 184
8.5.1 Αποκατάσταση των συνθήκων ισορροπίας 186
Ασκήσεις 195
Βιβλιογραφία 196

8.1 Κατηγορίες ισορροπημένων δένδρων αναζήτησης

Με τον όρο *ισορροπημένο δένδρο αναζήτησης* αναφερόμαστε τυπικά σε ένα δένδρο αναζήτησης του οποίου το ύψος είναι ανάλογο του πλήθους των κλειδιών του, δηλαδή $O(\log n)$ για n κλειδιά. Όπως έχουμε δει στο Κεφάλαιο 7, ένα δυαδικό δένδρο με n κλειδιά έχει ύψος τουλάχιστον ίσο με $\log n$, επομένως το ύψος ενός ισορροπημένου δυαδικού δένδρου αναζήτησης είναι πολύ κοντά (κατά ένα σταθερό παράγοντα) στο ελάχιστο δυνατό ύψος. Αυτό είναι σημαντικό, καθώς ο χρόνος εκτέλεσης ορισμένων βασικών λειτουργιών, όπως εισαγωγή, διαγραφή και αναζήτηση, σε ένα δένδρο αναζήτησης είναι ανάλογος του ύψους του.

Με βάση την τελευταία παρατήρηση, μπορούμε να δώσουμε μια πιο χαλαρή συνθήκη και να θεωρήσουμε ότι ένα δένδρο αναζήτησης είναι ισορροπημένο, αν εκτελεί τις λειτουργίες εισαγωγής, διαγραφής και αναζήτησης σε $O(\log n)$ χρόνο. Έτσι, ανάλογα με το πώς ορίζουμε το χρόνο εκτέλεσης, μπορούμε να έχουμε δένδρα αναζήτησης τα οποία επιτυγχάνουν λογαριθμικό χρόνο χειρότερης περίπτωσης, αναμενόμενης περίπτωσης ή κατά την αντισταθμιστική έννοια. Στη βιβλιογραφία έχουν προταθεί διάφοροι τύποι δένδρων με αυτά τα χαρακτηριστικά. Σε αυτό το κεφάλαιο θα εστιάσουμε σε ορισμένα από αυτά τα δένδρα, τα οποία δίνονται συνοπτικά στον παρακάτω πίνακα και είναι πολύ διαδεδομένα στην πράξη.

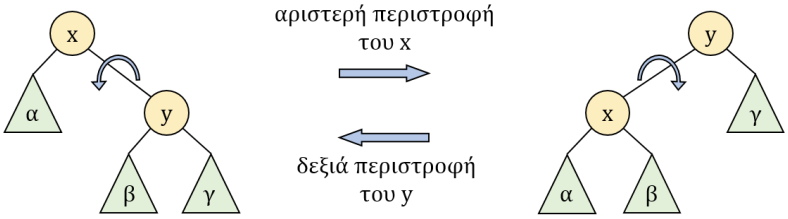
Πίνακας 8.1: Μερικές κατηγορίες ισορροπημένων δένδρων αναζήτησης.

δένδρο	λογαριθμικός χρόνος
AVL	χειρότερης περίπτωσης
(2,4)	χειρότερης περίπτωσης
κοκκινόμαυρο	χειρότερης περίπτωσης
αρθρωτό	αντισταθμιστικός
τυχαιοποιημένο	αναμενόμενος

Από τη στιγμή που υπάρχουν δένδρα αναζήτησης με λογαριθμικό χρόνο χειρότερης περίπτωσης, είναι εύλογο να αναρωτηθούμε αν αξίζει τον κόπο να ασχοληθούμε με δένδρα που παρέχουν εγγυήσεις χρόνου εκτέλεσης μόνο στην αντισταθμιστική ή στην αναμενόμενη περίπτωση. Τα δένδρα που παρέχουν εγγυήσεις απόδοσης χειρότερης περίπτωσης, επιτυγχάνουν αυτό το στόχο επιβάλλοντας μία ή περισσότερες αναλλοίωτες συνθήκες στη δομή του δένδρου. Η εισαγωγή ή διαγραφή ενός κλειδιού μπορεί να έχουν ως αποτέλεσμα την προσωρινή παραβίαση ορισμένων αναλλοίωτων συνθηκών. Επομένως, απαιτούνται διαδικασίες αποκατάστασης τους, οι οποίες καθιστούν τις λειτουργίες εισαγωγής και διαγραφής πιο περίπλοκες σε σχέση με τα δυαδικά δένδρα αναζήτησης που είναι ισορροπημένα κατά την αναμενόμενη ή αντισταθμιστική περίπτωση. Επίσης, ο έλεγχος των αναλλοίωτων συνθηκών απαιτεί πρόσθετο αποθηκευτικό χώρο ανά κόμβο του δένδρου.

8.1.1 Περιστροφές

Για να ελέγξουμε το ύψος ενός δυαδικού δένδρου αναζήτησης χρειαζόμαστε έναν τρόπο να αλλάζουμε την τοπολογία των κόμβων του, χωρίς, όμως να επηρεάσουμε τη δυνατότητα αναζήτησης στο δένδρο. Η περιστροφή ενός ζεύγους κόμβων, απεικονίζεται στην Εικόνα 8.1, μας παρέχει αυτόν τον τρόπο.



Εικόνα 8.2: Αριστερή και δεξιά περιστροφή.

Έστω κ_α ένα οποιοδήποτε κλειδί του υποδένδρου α. Ομοίως, έστω κ_β και κ_γ οποιοδήποτε κλειδιά των υποδένδρων β και γ αντίστοιχα. Παρατηρούμε ότι και στα δύο σχήματα της Εικόνας 8.1 ισχύει η $\kappa_\alpha \leq \kappa\lambda\epsilon\iota\delta\acute{\iota}(x) \leq \kappa_\beta \leq \kappa\lambda\epsilon\iota\delta\acute{\iota}(y) \leq \kappa_\gamma$. Άρα, η περιστροφή διατηρεί την ιδιότητα ενός δυαδικού δένδρου αναζήτησης.

Τυπικά, σε ένα ισορροπημένο δυαδικό δένδρο αναζήτησης, τόσο η εισαγωγή όσο και η διαγραφή αποτελούνται από δύο φάσεις. Στην πρώτη εκτελείται ο αλγόριθμος εισαγωγής ή διαγραφής όπως σε ένα απλό δυαδικό δένδρο αναζήτησης, ενώ στη δεύτερη φάση γίνεται αποκατάσταση των αναλλοίωτων συνθηκών με τη βοήθεια περιστροφών.

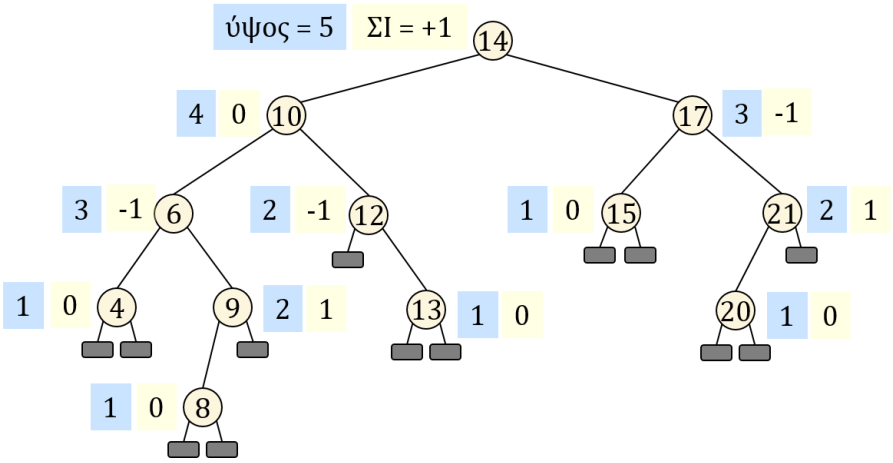
8.2 Δένδρα AVL

Τα δένδρα AVL διατηρούν μια αναλλοίωτη συνθήκη η οποία συνδέει το ύψος των δύο παιδιών του κάθε κόμβου. Ορίζουμε το συντελεστή ισορροπίας $\Sigma\mathrm{I}(v)$ ενός κόμβου v, ως τη διαφορά του ύψους του αριστερού παιδιού από το δεξί παιδί, δηλαδή

$$\Sigma\mathrm{I}(v) = \text{ύψος}\big(\text{αριστερός}(v)\big) - \text{ύψος}\big(\text{δεξιός}(v)\big)$$

Ένα δένδρο T είναι δένδρο AVL, αν ικανοποιεί την παρακάτω συνθήκη:

Αναλλοίωτη συνθήκη δένδρου AVL: Για κάθε κόμβο v ισχύει $-1 \leq \Sigma\mathrm{I}(v) \leq 1$.



Εικόνα 8.3: Ένα δένδρο AVL. Δίπλα σε κάθε κόμβο δίνονται το ύψος του και ο συντελεστής ισορροπίας.

Έστω n_h το ελάχιστο πλήθος κλειδιών ενός δένδρου AVL με ύψος h. Τότε $n_h = n_{h-1} + n_{h-2} + 1$. Από αυτήν τη σχέση μπορούμε να καταλήξουμε εύκολα σε ένα κάτω φράγμα για την τιμή του n_h, παρατηρώντας ότι $n_{h-1} \geq n_{h-2}$. Έτσι, λαμβάνουμε $n_h \geq 2n_{h-2} + 1 \geq 2n_{h-2}$. Άρα, $n_h \geq 2^{h/2} n_1 = 2^{h/2}$, αφού προφανώς ένα δένδρο AVL με ύψος ένα έχει ακριβώς έναν εσωτερικό κόμβο. Από την τελευταία σχέση συνεπάγεται ότι $h \leq 2 \lg n$.

Μπορούμε να καταλήξουμε σε ένα καλύτερο άνω φράγμα παρατηρώντας ότι η αναδρομική σχέση του n_h θυμίζει την ακολουθία Fibonacci. Η μόνη διαφορά είναι ο όρος $+1$, από τον οποίον, όμως, μπορούμε να απαλλαγούμε με ένα απλό μετασχηματισμό. Θέτουμε $N_h = n_h + 1$ και προσθέτουμε τον όρο +1 στα δύο μέλη της . Τότε έχουμε $(n_h + 1) = (n_{h-1} + 1) + (n_{h-2} + 1)$, δηλαδή $N_h = N_{h-1} + N_{h-2}$, που είναι ακριβώς η αναδρομική σχέση της ακολουθίας Fibonacci. Άρα, γνωρίζουμε ότι $N_h \leq \varphi^h/\sqrt{5}$, όπου $\varphi = (1+\sqrt{5})/2 \approx 1.62$. Όμως, $n_h = N_h - 1 \leq \varphi^h/\sqrt{5}$, δηλαδή $h \geq \log_\varphi\big(\sqrt{5} n_h\big) \geq \log_\varphi n_h = \lg n_h / \lg \varphi \approx 1.44 \lg n_h$.

8.2.1 Αποκατάσταση συνθήκης ισορροπίας

Η αναλλοίωτη συνθήκη μπορεί να παραβιαστεί τόσο μετά από μια εισαγωγή όσο και μετά από μια διαγραφή ενός κόμβου x. Μια απλή αλλά χρήσιμη παρατήρηση είναι ότι μεταβάλλεται μόνο το ύψος των κόμβων που βρίσκονται στο μονοπάτι από τη ρίζα προς τον x, επομένως η συνθήκη του συντελεστή ισορροπίας μπορεί να παραβιαστεί μόνο σε αυτούς τους κόμβους. Επιπλέον, αν η συνθήκη παραβιαστεί σε ένα κόμβο v, τότε έχουμε $\Sigma\text{I}(v) = 2$ ή $\Sigma\text{I}(v) = -2$. Η αποκατάσταση της συνθήκης γίνεται μέσω κατάλληλων περιστροφών.

Εισαγωγές

Μετά την εισαγωγή του νέου κόμβου x ανεβαίνουμε το μονοπάτι εισαγωγής P από τον x προς τη ρίζα και υπολογίζουμε το νέο ύψος και το νέο συντελεστή ισορροπίας κάθε κόμβου του P. Αν όλοι οι συντελεστές ικανοποιούν την αναλλοίωτη συνθήκη, τότε το δένδρο παραμένει AVL χωρίς καμία περαιτέρω αλλαγή. Διαφορετικά, έστω z ο κοντινότερος πρόγονος του x στο δένδρο, τέτοιος ώστε $\Sigma\text{I}(z) = 2$ ή $\Sigma\text{I}(z) = -2$. Δηλαδή ο z είναι πρόγονος του x με το ελάχιστο ύψος, στον οποίο παραβιάζεται η αναλλοίωτη συνθήκη. Θα αναφερόμαστε στον z ως τον *κρίσιμο κόμβο* του μονοπατιού P, γιατί, όπως θα δούμε, η αποκατάσταση της αναλλοίωτης συνθήκης στον z συνεπάγεται ότι και όλοι οι πρόγονοί του θα ικανοποιούν την αναλλοίωτη συνθήκη.

Το είδος των περιστροφών καθορίζεται από τη θέση των δύο επόμενων απογόνων του κρίσιμου κόμβου z πάνω στο P. Έστω, λοιπόν, y το παιδί του z στο P και w το παιδί του y στο P. Διαχωρίζουμε τις εξής περιπτώσεις:

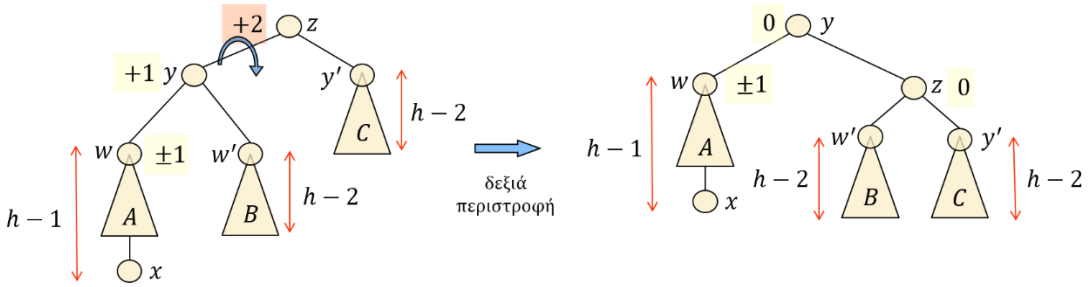
Περίπτωση ΑΑ (αριστερά-αριστερά): Ο y είναι αριστερό παιδί του z και ο w είναι αριστερό παιδί του y. Εκτελούμε μια δεξιά περιστροφή του ζεύγους (z, y).

Περίπτωση ΑΔ (αριστερά-δεξιά): Ο y είναι αριστερό παιδί του z και ο w είναι δεξί παιδί του y. Εκτελούμε μια διπλή αριστερή-δεξιά περιστροφή της τριάδας (z, y, w). Δηλαδή, εκτελούμε πρώτα την αριστερή περιστροφή του ζεύγους (y, w) και, στη συνέχεια, τη δεξιά περιστροφή του ζεύγους (z, w).

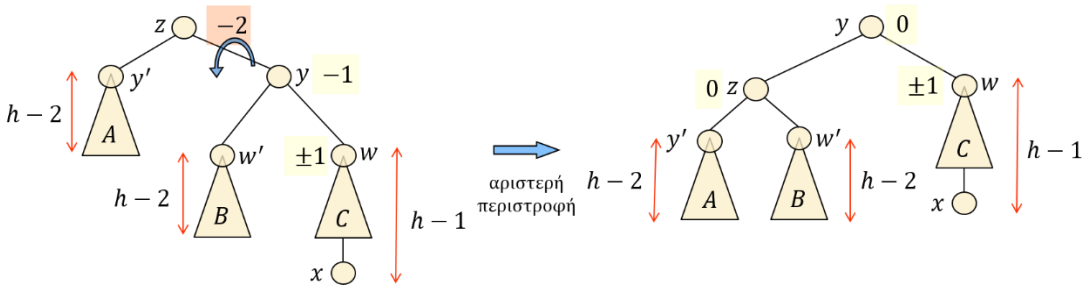
Περίπτωση ΔΑ (δεξιά-αριστερή): Ο y είναι δεξί παιδί του z και ο w είναι αριστερό παιδί του y. Εκτελούμε μια διπλή δεξιά-αριστερή περιστροφή της τριάδας (z, y, w). Δηλαδή, εκτελούμε πρώτα τη δεξιά περιστροφή του ζεύγους (y, w) και, στη συνέχεια, την αριστερή περιστροφή του ζεύγους (z, w).

Περίπτωση ΔΔ (δεξιά-δεξιά): Ο y είναι δεξί παιδί του z και ο w είναι δεξί παιδί του y. Εκτελούμε μια αριστερή περιστροφή του ζεύγους (z, y).

Περίπτωση ΑΑ: y = αριστερό παιδί του z και w = αριστερό παιδί του y

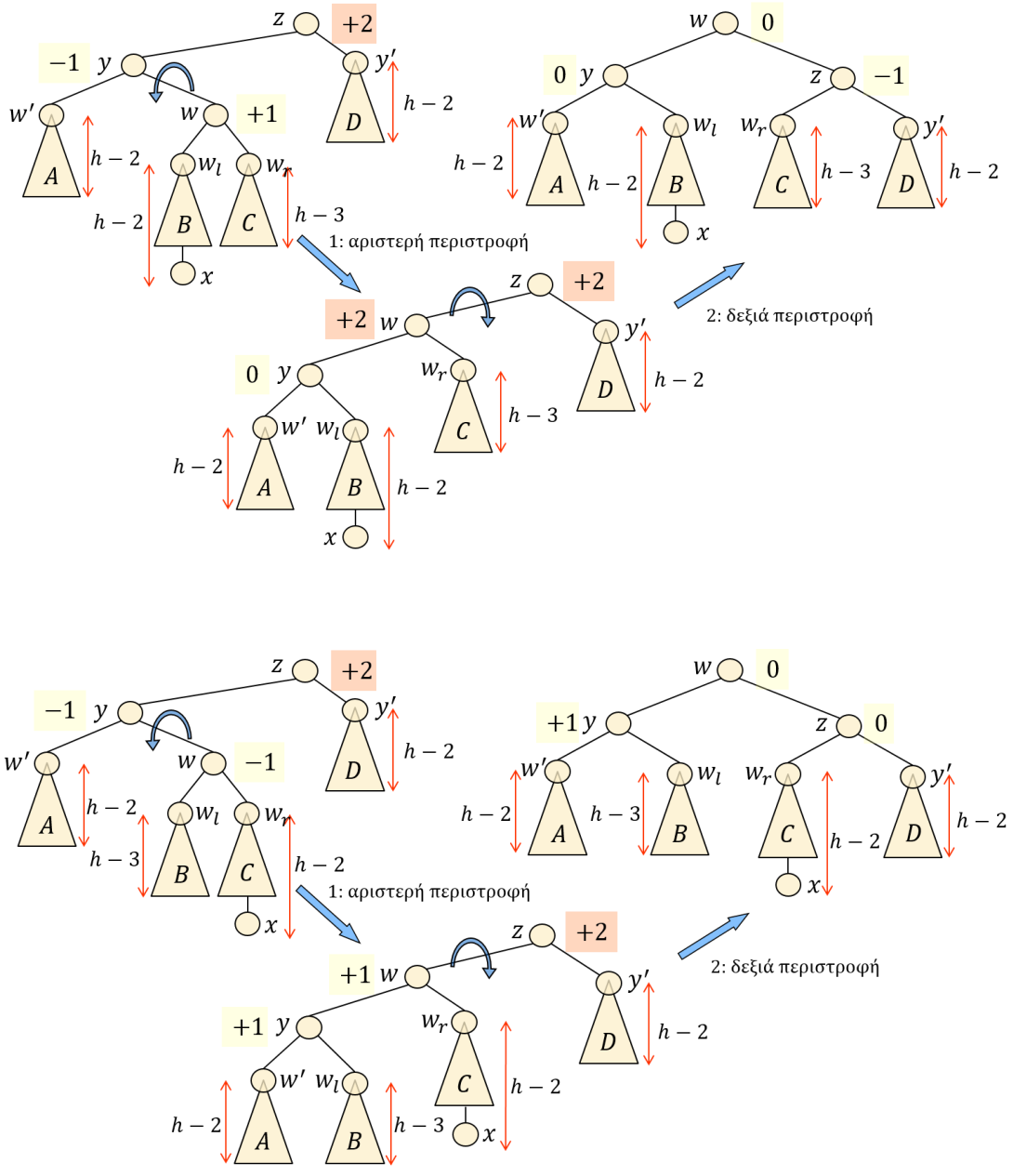


Περίπτωση ΔΔ: y = δεξί παιδί του z και w = δεξί παιδί του y



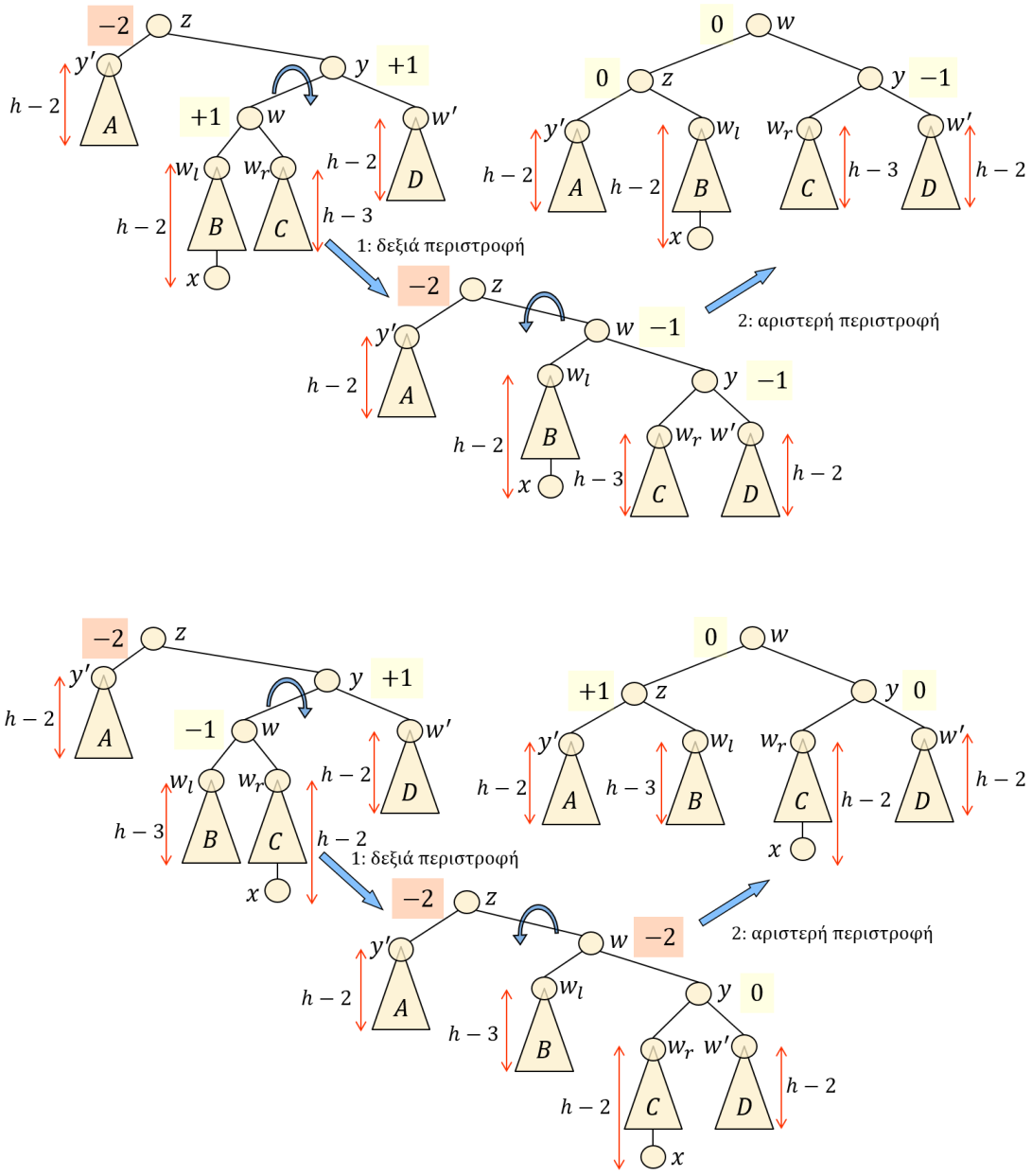
Εικόνα 8.4: Διαδικασία αποκατάστασης της ισορροπίας ενός δένδρου AVL μετά την εισαγωγή ενός νέου κόμβου x. Στις περιπτώσεις ΑΑ και ΔΔ μια απλή περιστροφή είναι αρκετή, για να αποκατασταθεί η συνθήκη ισορροπίας. Ο κόμβος y, μετά την τοποθέτηση του στη θέση του z, έχει το ίδιο ύψος με αυτό που είχε ο z πριν από την εισαγωγή του x.

Περίπτωση ΑΔ: y = αριστερό παιδί του z και w = δεξί παιδί του y

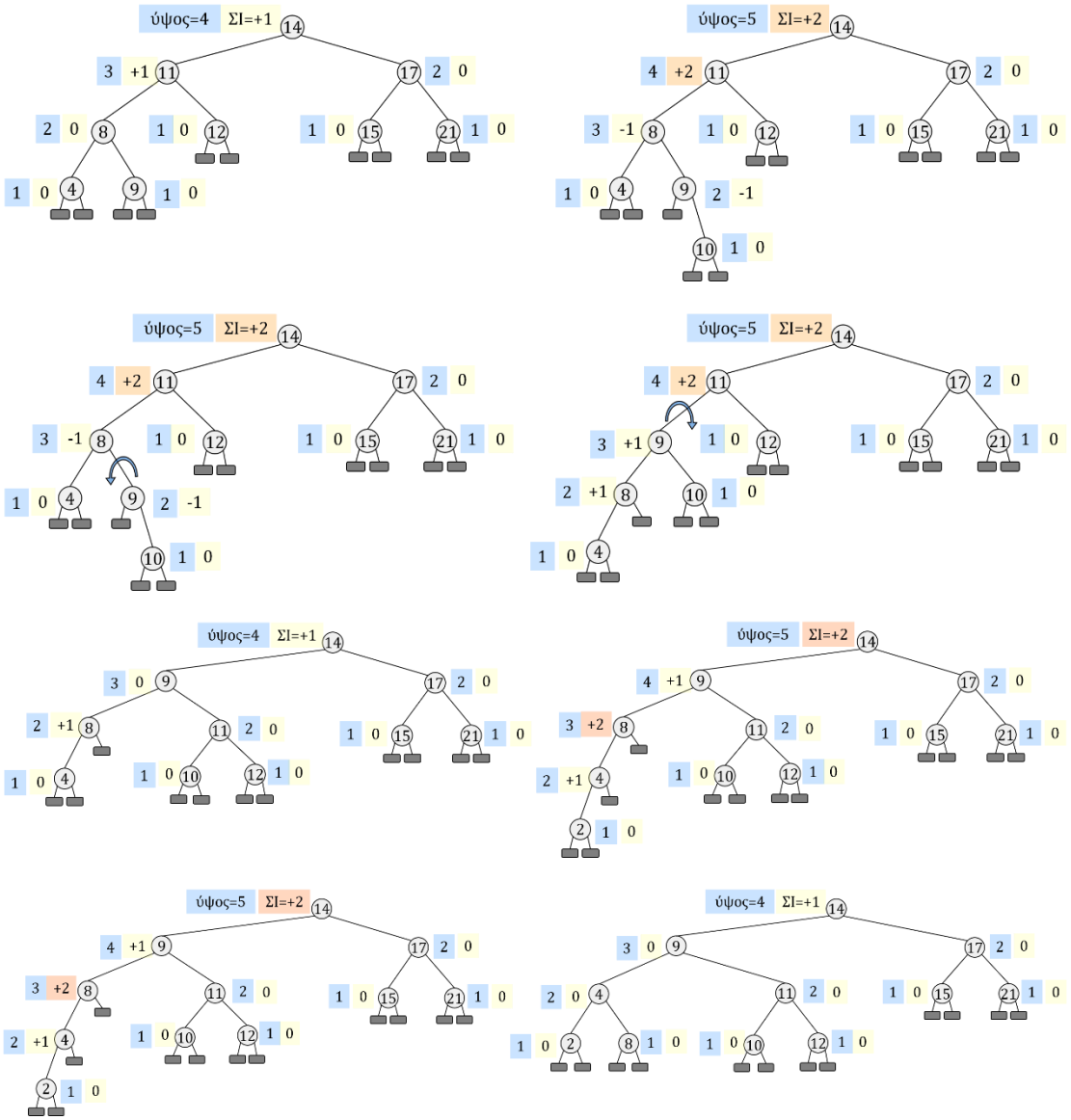


Εικόνα 8.5: Διαδικασία αποκατάστασης της ισορροπίας ενός δένδρου AVL μετά την εισαγωγή ενός νέου κόμβου x. Στην περίπτωση ΑΔ μια διπλή αριστερή-δεξιά περιστροφή της τριάδας (z, y, w) είναι αρκετή, για να αποκατασταθεί η συνθήκη ισορροπίας. Ο κόμβος w, μετά την τοποθέτηση του στη θέση του z, έχει το ίδιο ύψος με αυτό που είχε ο z πριν από την εισαγωγή του x.

Περίπτωση ΔΑ: y = δεξί παιδί του z και w = αριστερό παιδί του y



Εικόνα 8.6: Διαδικασία αποκατάστασης της ισορροπίας ενός δένδρου AVL μετά την εισαγωγή ενός νέου κόμβου x. Στην περίπτωση ΑΔ μια διπλή δεξιά-αριστερή περιστροφή της τριάδας (z, y, w) είναι αρκετή, για να αποκατασταθεί η συνθήκη ισορροπίας. Ο κόμβος w, μετά την τοποθέτηση του στη θέση του z, έχει το ίδιο ύψος με αυτό που είχε ο z πριν από την εισαγωγή του x.



Εικόνα 8.7: Εισαγωγή των κλειδιών 10 και 2 σε ένα δένδρο AVL. Μετά την εισαγωγή του 10 ο κοντινότερος πρόγονος του νέου κόμβου είναι ο z με κλειδί 11. Οι επόμενοι δύο κόμβοι μετά τον z στο μονοπάτι εισαγωγής είναι ο y με κλειδί 8 και ο w με κλειδί 9. Ο y είναι αριστερό παιδί και ο w δεξί παιδί, άρα εφαρμόζουμε την περίπτωση ΑΔ. Μετά την εισαγωγή του 2 ο κοντινότερος πρόγονος του νέου κόμβου είναι ο z με κλειδί 8. Οι επόμενοι δύο κόμβοι μετά τον z στο μονοπάτι εισαγωγής είναι ο y με κλειδί 4 και ο w με κλειδί 2. Και ο y και ο w είναι αριστερά παιδιά, άρα εφαρμόζουμε την περίπτωση ΑΑ.

Ιδιότητα 8.1 Το δένδρο που προκύπτει μετά τη διαδικασία αποκατάστασης είναι δένδρο AVL.

Απόδειξη Έστω h το ύψος του κρίσιμου κόμβου z πριν από την εισαγωγή και έστω t ο κόμβος που λαμβάνει τη θέση του z μετά την απλή ή τη διπλή περιστροφή. Δηλαδή, $t = y$, αν έχουμε την περίπτωση ΑΑ ή ΔΔ, και $t = w$, αν έχουμε την περίπτωση ΑΔ ή ΔΑ. Αρκεί να δείξουμε ότι μετά τη διαδικασία αποκατάστασης, η αναλλοίωτη συνθήκη ισχύει για τους κόμβους z, y και w και, επιπλέον, ότι ο κόμβος t μετά την απλή ή τη διπλή περιστροφή έχει το ύψος h. Το γεγονός αυτό συνεπάγεται ότι οι κόμβοι του μονοπατιού εισαγωγής P που είναι γνήσιοι πρόγονοι του κρίσιμου κόμβου z πριν από τη διαδικασία αποκατάστασης (και, επομένως,

γνήσιοι πρόγονοι του t μετά τη διαδικασία αποκατάστασης), έχουν το ίδιο ύψος που είχαν πριν από την εισαγωγή. Άρα, ικανοποιούν την αναλλοίωτη συνθήκη.

Παρατηρούμε ότι η εισαγωγή του νέου κόμβου x προκαλεί την αύξηση του ύψους όλων των απογόνων του z στο P. Επομένως, πριν από την εισαγωγή ισχύει $\Sigma\text{Ι}(v) = 0$ για κάθε γνήσιο απόγονο του z στο P. Συμβολίζουμε με y' τον αδελφό του y και με w' τον αδελφό του w. (Οποιοσδήποτε από αυτούς τους αδελφικούς κόμβους μπορεί να είναι κενός.)

Ας υποθέσουμε πρώτα ότι στη διαδικασία αποκατάστασης εφαρμόζεται η περίπτωση ΑΑ. Τότε, αμέσως μετά την εισαγωγή έχουμε $\text{ύψος}(z) = h + 1$, $\text{ύψος}(y) = h$ και $\text{ύψος}(w) = h - 1$, ενώ το ύψος των κόμβων y' και w' παραμένει ίσο με $h - 2$. Επομένως, αμέσως μετά την εισαγωγή, $\Sigma\text{Ι}(z) = \text{ύψος}(y) - \text{ύψος}(y') = 2$. Μετά τη δεξιά περιστροφή του ζεύγους (z, y), ο κόμβος z έχει παιδιά τους w' και y', άρα το νέο ύψος του z είναι ίσο με $h - 1$ και ο νέος συντελεστής ισορροπίας του z είναι μηδέν. Απομένει να εξετάσουμε τον κόμβο y. Το ύψος του μετά την περιστροφή γίνεται ίσο με h, δηλαδή όσο ήταν το ύψος του z πριν από την εισαγωγή. Τέλος, μετά την περιστροφή ο συντελεστής ισορροπίας του y γίνεται μηδέν. Άρα, η συνθήκη ισορροπίας αποκαταστάθηκε για όλους τους κόμβους του δένδρου. Η ανάλυση της περίπτωσης ΔΔ είναι συμμετρική και την παραλείπουμε.

Συνεχίζουμε την ανάλυση μας με την περίπτωση ΑΔ. Συμβολίζουμε με w_l και w_r, αντίστοιχα, το αριστερό και δεξί παιδί του w. (Οποιοσδήποτε από αυτούς τους αδελφικούς κόμβους μπορεί να είναι κενός.) Ας υποθέσουμε πρώτα ότι μετά την εισαγωγή, ο κόμβος x είναι απόγονος του w_l, άρα έχουμε $\text{ύψος}(w_l) = h - 2$ και $\text{ύψος}(w_r) = h - 3$. Μετά τη διπλή αριστερή-δεξιά περιστροφή της τριάδας (z, y, w), ο κόμβος w τοποθετείται στη θέση του z και λαμβάνει τον y και τον z ως παιδιά, ο w_l γίνεται δεξί παιδί του y και ο w_r γίνεται αριστερό παιδί του z. Τότε, ο κόμβος y έχει παιδιά τους w' και w_l, άρα το νέο ύψος του y είναι ίσο με $h - 1$ και ο νέος συντελεστής ισορροπίας του y είναι μηδέν. Από την άλλη, ο κόμβος z έχει παιδιά τους w_r και y', οπότε το νέο ύψος του z είναι ίσο με $h - 1$ και ο νέος συντελεστής ισορροπίας του z είναι -1. Όσο για τον κόμβο w, το ύψος του μετά την περιστροφή γίνεται ίσο με h, δηλαδή όσο ήταν το ύψος του z πριν από την εισαγωγή, ενώ ο συντελεστής ισορροπίας του γίνεται μηδέν. Ας υποθέσουμε τώρα ότι μετά την εισαγωγή, ο κόμβος x είναι απόγονος του w_r, άρα έχουμε $\text{ύψος}(w_l) = h - 3$ και $\text{ύψος}(w_r) = h - 2$. Η μόνη διαφορά σε αυτήν την περίπτωση είναι ότι ο κόμβος y έχει νέο συντελεστή ισορροπίας ίσο με ένα, ενώ ο συντελεστής ισορροπίας του z γίνεται μηδέν.

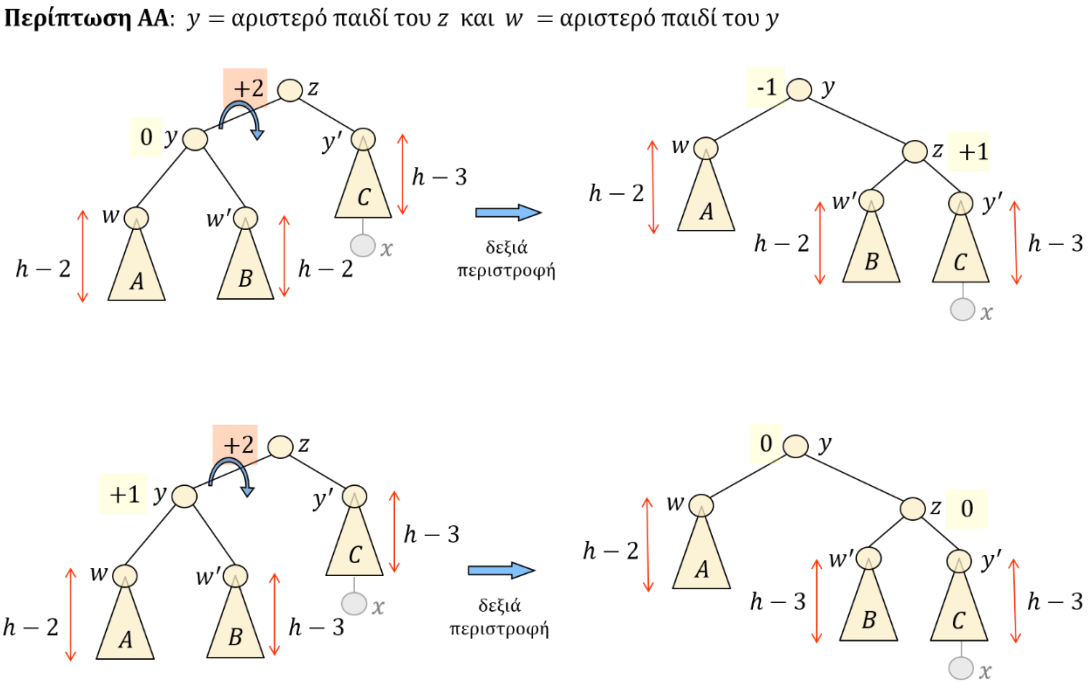
Επομένως, σε κάθε περίπτωση, η συνθήκη ισορροπίας αποκαταστάθηκε για όλους τους κόμβους του δένδρου. Η ανάλυση της περίπτωσης ΔΑ είναι συμμετρική και την παραλείπουμε.

Διαγραφές

Για τη διαγραφή ενός κλειδιού k εκτελούμε πρώτα, όπως και στην εισαγωγή, τον αλγόριθμο διαγραφής κλειδιού σε απλό δυαδικό δένδρο αναζήτησης που περιγράψαμε στην Ενότητα 7.3.5. Έστω x ο κόμβος που περιέχει το κλειδί k. Όπως είδαμε στην Ενότητα 7.3.5, ο αλγόριθμος διαγραφής απομακρύνει από το δένδρο έναν κόμβο x', όπου ο x' είναι ο ίδιος ο κόμβος x, αν δεν έχει παιδιά, ή το μη κενό παιδί του x, αν ο x έχει ακριβώς ένα μη κενό παιδί, ή ο διάδοχος του x, αν και τα δύο παιδιά του x είναι μη κενά. Στις περιπτώσεις όπου ο x' είναι διαφορετικός από τον x, τότε αντιγράφουμε τα περιεχόμενα του x' στον x πριν από τη διαγραφή του x'.

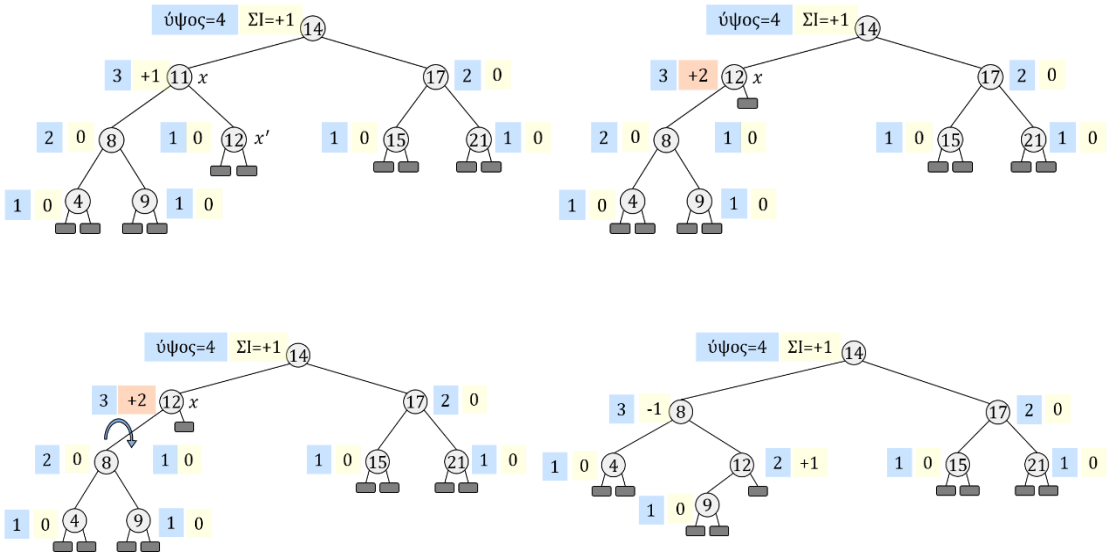
Στη συνέχεια, πρέπει να ελέγξουμε αν η διαγραφή προκάλεσε την παραβίαση της αναλλοίωτης συνθήκης σε κάποιο πρόγονο του x'. Αν δεν υπάρχει τέτοιος πρόγονος του x', τότε το δένδρο παραμένει AVL και η διαδικασία διαγραφής τερματίζεται. Διαφορετικά, εξετάζουμε τον χαμηλότερο πρόγονο z του x', ο οποίος αμέσως μετά τη διαγραφή παραβιάζει τη συνθήκη ισορροπίας. Συμβολίζουμε με y' το παιδί του z που είναι πρόγονος του x' και με y τον

αδελφικό κόμβο του y' (όπου μπορεί να έχουμε $y' = x'$). Μετά τη διαγραφή του x' ο συντελεστής ισορροπίας του z γίνεται ± 2, άρα πριν από τη διαγραφή ισχύει $\text{ύψος}(y) = \text{ύψος}(y') + 1$. Αυτό σημαίνει ότι μπορούμε να χειριστούμε τη διαγραφή του x' σαν να πρόκειται για εισαγωγή στο υποδένδρο του y, εφαρμόζοντας τις ίδιες περιπτώσεις (ΑΑ, ΑΔ, ΔΑ και ΔΔ) με την εισαγωγή. Συγκεκριμένα, ορίζουμε ως w το παιδί του y με το μεγαλύτερο ύψος. Σε περίπτωση που ο y έχει δύο παιδία με το ίδιο ύψος επιλέγουμε το παιδί w του y για το οποίο οι σύνδεσμοι (z, y) και (y, w) έχουν την ίδια φορά. (Αυτό γίνεται για να εφαρμόσουμε τις απλούστερες περιπτώσεις ΑΑ ή ΔΔ, οι οποίες απαιτούν μόνο μία περιστροφή.)

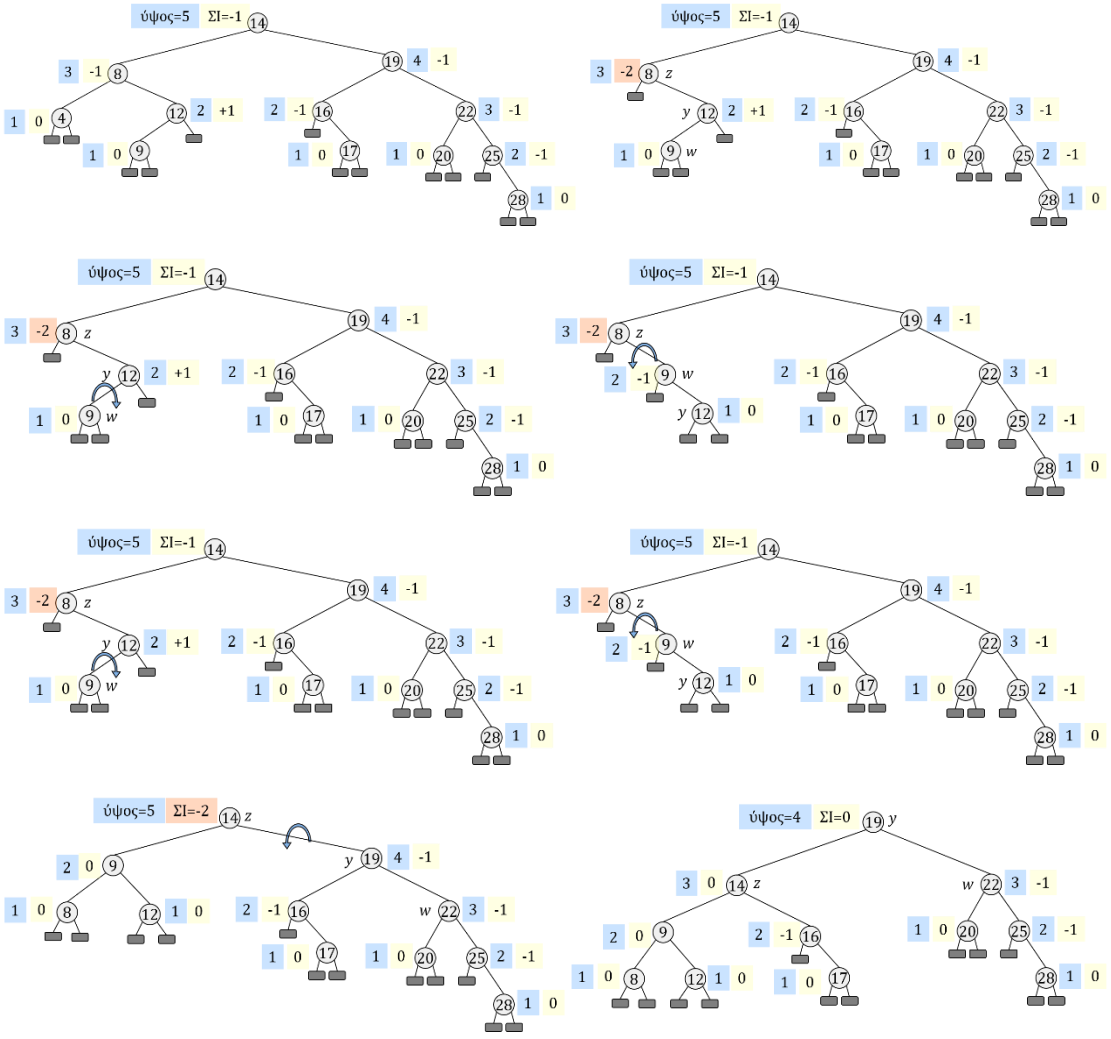


Εικόνα 8.8: Διαδικασία αποκατάστασης της ισορροπίας ενός δένδρου AVL μετά τη διαγραφή ενός κόμβου x. Στην περίπτωση ΑΑ μια απλή περιστροφή αποκαθιστά την αναλλοίωτη συνθήκη για τον κόμβο z, όμως ο κόμβος y, μετά την τοποθέτηση του στη θέση του z, μπορεί να έχει το μικρότερο ύψος από αυτό που είχε ο z πριν από τη διαγραφή του x. Στην πάνω εικόνα, τόσο ο κόμβος z πριν την περιστροφή όσο και ο κόμβος y μετά την περιστροφή έχουν ύψος h. Στην κάτω εικόνα, ο κόμβος y μετά την περιστροφή έχει ύψος $h - 1$.

Η ίδια ανάλυση που εφαρμόσαμε στην περίπτωση της εισαγωγής δείχνει ότι η αναλλοίωτη συνθήκη αποκαθίσταται για τους κόμβους z, y και w. Η μόνη διαφορά είναι ότι ο κόμβος t, που λαμβάνει τη θέση του z μετά την απλή περιστροφή ($t = w$) ή μετά τη διπλή περιστροφή ($t = w$), έχει κατά μία μονάδα μικρότερο ύψος από το ύψος που είχε ο z πριν από τις περιστροφές. Έτσι, θα πρέπει να συνεχίσουμε τον έλεγχο για το αν παραβιάζεται η αναλλοίωτη συνθήκη σε κάποιον πρόγονο του t. Στη χειρότερη περίπτωση, μπορεί να χρειαστεί να γίνουν περιστροφές σε όλους τους κόμβους στο μονοπάτι από τον κόμβο z έως τη ρίζα.



Εικόνα 8.9: Διαγραφή του κλειδιού 11 σε ένα δένδρο AVL. Ο κόμβος x με κλειδί 11 έχει δύο μη κενά παιδιά, οπότε διαγράφεται ο διάδοχος κόμβος x' του x με κλειδί 12. Πριν από τη διαγραφή του x', το κλειδί του 12 αντιγράφεται στον x. Μετά τη διαγραφή ο κοντινότερος πρόγονος του x' στον οποίο παραβιάζεται η αναλλοίωτη συνθήκη είναι ο $z = x$. Βρίσκουμε τους επόμενους δύο κόμβους, y και w, σε ένα μέγιστο μονοπάτι από τον z. Επειδή ο z έχει θετικό συντελεστή ισορροπίας, επιλέγουμε ως y το αριστερό του παιδί με κλειδί 8. Τώρα, αφού έχουμε $\Sigma\text{I}(y) = 0$, μπορούμε να επιλέξουμε ως w το αριστερό παιδί του y με κλειδί 4, οπότε εφαρμόζουμε την περίπτωση AA. Μετά την απλή αριστερή περιστροφή του ζεύγους (z, y) αποκαθίσταται η αναλλοίωτη συνθήκη στο δένδρο.



Εικόνα 8.10: Παράδειγμα διαγραφής κλειδιού σε δένδρο AVL, η οποία προκαλεί πολλαπλές περιστροφές. Μετά τη διαγραφή του κλειδιού 4 παραβιάζεται η αναλλοίωτη συνθήκη στον κόμβο z με κλειδί 8. Βρίσκουμε τους επόμενους δύο κόμβους, y με κλειδί 12 και w με κλειδί 9, στο μέγιστο μονοπάτι από τον z. Αφού ο y είναι δεξί παιδί και ο w αριστερό, εφαρμόζουμε την περίπτωση ΔΑ. Μετά τη διπλή δεξιά-αριστερή περιστροφή της τριάδας (z, y, w) παραβιάζεται η αναλλοίωτη συνθήκη στη ρίζα του δένδρου, επομένως ο νέος κόμβος z είναι η ρίζα. Βρίσκουμε τους επόμενους δύο κόμβους, y με κλειδί 19 και w με κλειδί 22, στο μέγιστο μονοπάτι από τον z. Και οι δύο αυτοί κόμβοι είναι δεξιά παιδιά, οπότε εφαρμόζουμε την περίπτωση ΔΔ. Μετά την απλή αριστερή περιστροφή του ζεύγους (z, y) αποκαθίσταται η αναλλοίωτη συνθήκη στο δένδρο.

8.3 Αρθρωτά Δένδρα

Τα αρθρωτά δένδρα είναι ισορροπημένα κατά την αντισταθμιστική έννοια. Δηλαδή, παρόλο που η εκτέλεση κάποιας λειτουργίας μπορεί να χρειαστεί $O(n)$ χρόνο στη χειρότερη περίπτωση, οποιαδήποτε ακολουθία από m λειτουργίες εκτελείται σε συνολικό χρόνο $O(m \log n)$, όσο και σε ένα AVL δένδρο.

Το χαρακτηριστικό αυτών των δένδρων είναι ότι μεταφέρουν στη ρίζα του δένδρου τον κόμβο πάνω στον οποίο εκτελείται μια λειτουργία. Η μεταφορά αυτού του κόμβου γίνεται μέσω μιας

βοηθητικής διαδικασίας, η οποία ονομάζεται $splay$ και χρησιμοποιεί ζεύγη περιστροφών, καθώς μετακινεί έναν κόμβο στη ρίζα.

Έστω y γονέας του x και έστω z ο γονέας του y. Όπως και στα AVL δένδρα, το είδος των περιστροφών που θα εκτελέσουμε εξαρτάται από τη φορά των συνδέσμων (z, y) και (y, x), όμως σε αντίθεση με τα AVL δένδρα, εκτελούμε πάντα διπλή περιστροφή, εκτός εάν ο y είναι η ρίζα.

Συγκεκριμένα, όταν οι σύνδεσμοι έχουν την ίδια φορά, τότε οι περιστροφές ξεκινούν από **πάνω προς τα κάτω**, δηλαδή από το ζεύγος (z, y).

Περίπτωση ΑΑ (αριστερά-αριστερά): Ο y είναι αριστερό παιδί του z και ο x είναι αριστερό παιδί του y. Εκτελούμε πρώτα μια δεξιά περιστροφή του ζεύγους (z, y) και μετά μια δεξιά περιστροφή του ζεύγους (y, x).

Περίπτωση ΔΔ (δεξιά-δεξιά): Ο y είναι δεξί παιδί του z και ο x είναι δεξί παιδί του y. Εκτελούμε πρώτα μια αριστερή περιστροφή του ζεύγους (z, y) και μετά μια αριστερή περιστροφή του ζεύγους (y, x).

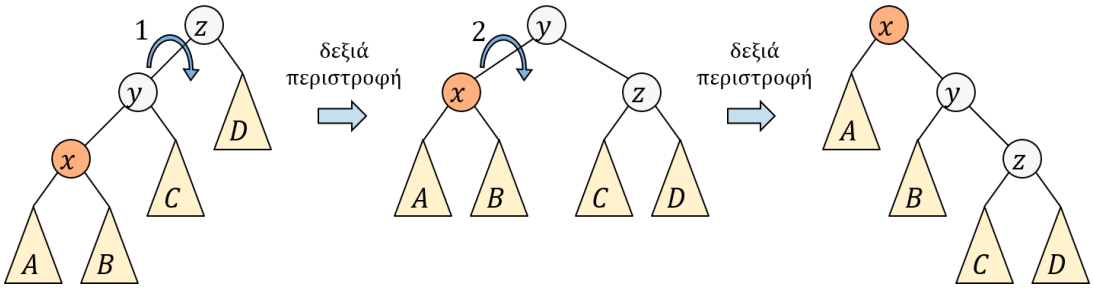
Όταν οι σύνδεσμοι έχουν διαφορετική φορά, τότε οι περιστροφές ξεκινούν, ως συνήθως, από **κάτω προς τα πάνω**, δηλαδή από το ζεύγος (y, x).

Περίπτωση ΑΔ (αριστερά-δεξιά): Ο y είναι αριστερό παιδί του z και ο x είναι δεξί παιδί του y. Εκτελούμε μια διπλή αριστερή-δεξιά περιστροφή της τριάδας (z, y, x), ξεκινώντας από την αριστερή περιστροφή του ζεύγους (y, x) και συνεχίζοντας με τη δεξιά περιστροφή του ζεύγους (z, x).

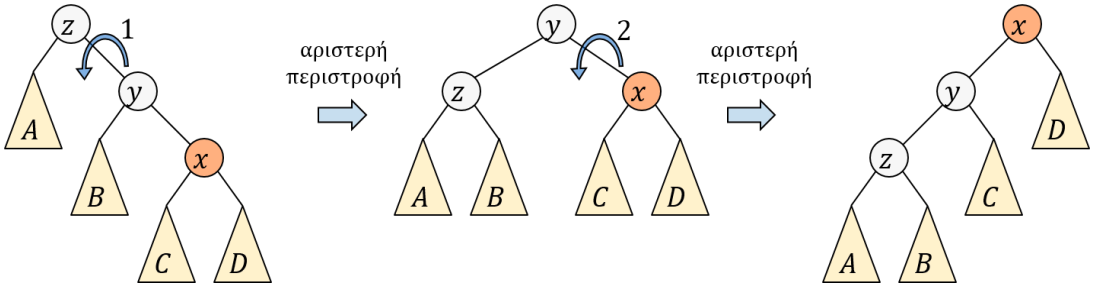
Περίπτωση ΔΑ (δεξιά-αριστερή): Ο y είναι δεξί παιδί του z και ο x είναι αριστερό παιδί του y. Εκτελούμε μια διπλή δεξιά-αριστερή περιστροφή της τριάδας (z, y, x), ξεκινώντας από την δεξιά περιστροφή του ζεύγους (y, x) και συνεχίζοντας με τη δεξιά περιστροφή του ζεύγους (z, x).

Στην περίπτωση που ο y είναι η ρίζα του δένδρου, οπότε ο κόμβος z δεν ορίζεται, εκτελούμε μια απλή δεξιά περιστροφή του ζεύγους (y, x) αν ο x είναι αριστερό παιδί του y ή μια απλή αριστερή περιστροφή του ζεύγους (y, x) αν ο x είναι δεξί παιδί του y.

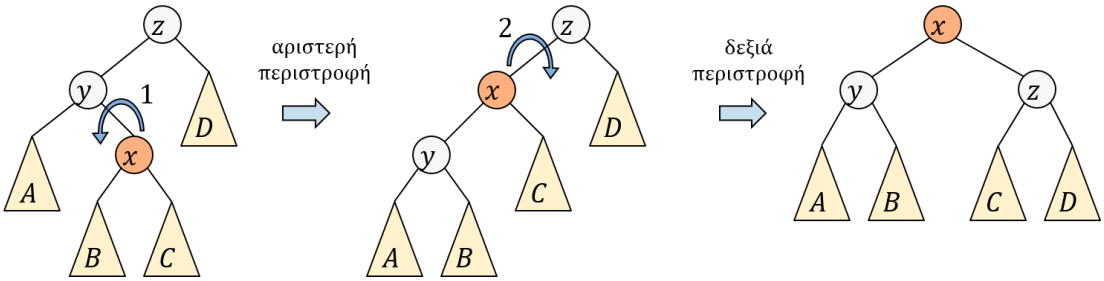
Περίπτωση ΑΑ: $y =$ αριστερό παιδί του z και $x =$ αριστερό παιδί του y



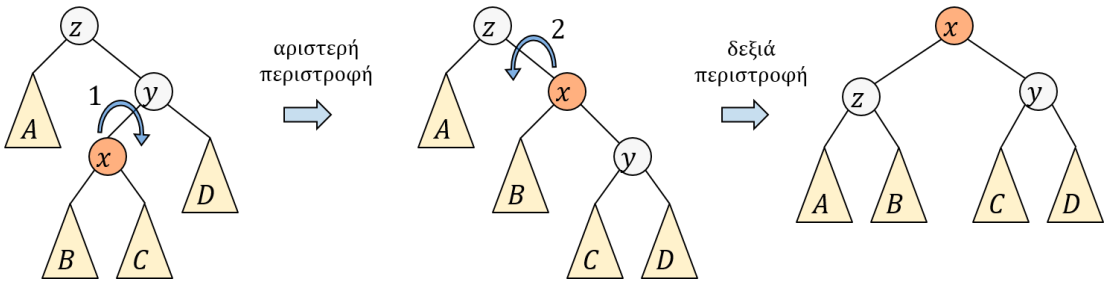
Περίπτωση ΔΔ: $y =$ δεξί παιδί του z και $x =$ δεξί παιδί του y



Περίπτωση ΑΔ: $y =$ αριστερό παιδί του z και $x =$ δεξί παιδί του y



Περίπτωση ΑΔ: $y =$ αριστερό παιδί του z και $x =$ δεξί παιδί του y

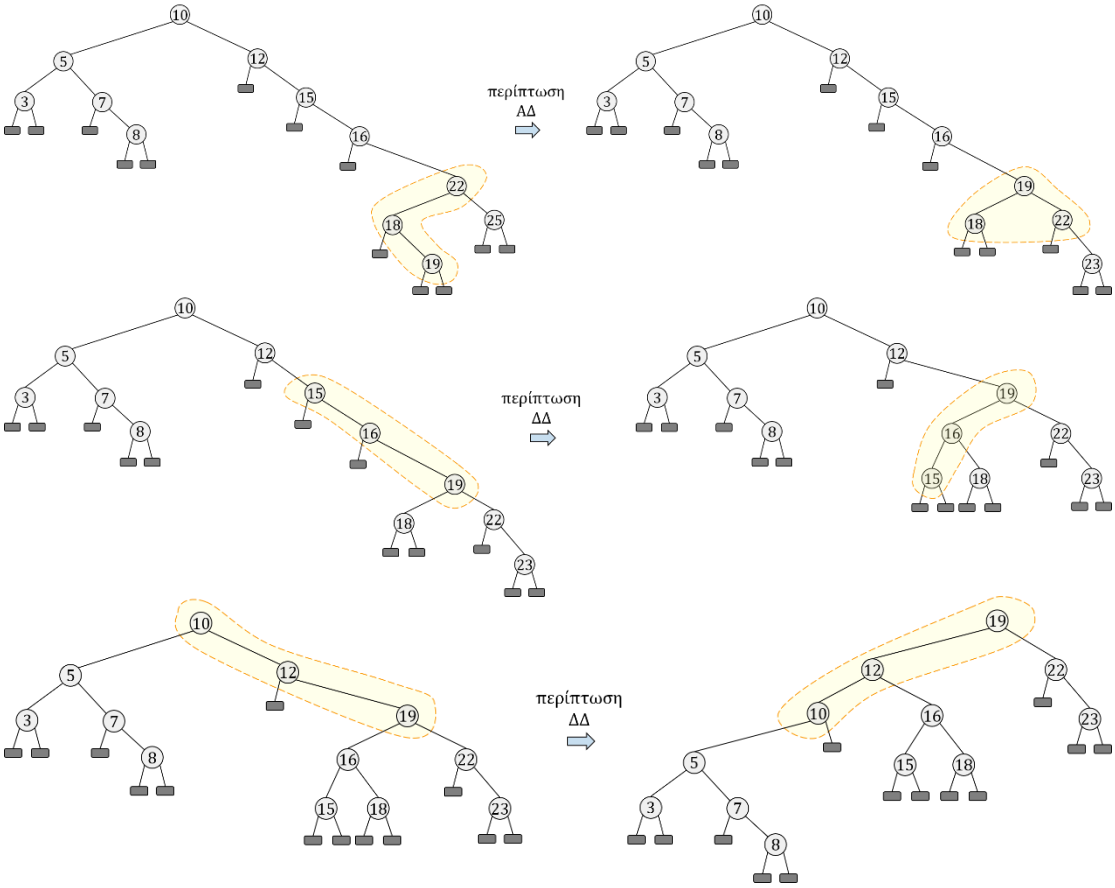


Εικόνα 8.11: Περιπτώσεις διπλών περιστροφών κατά την εκτέλεση της μεθόδου $splay(x)$, η οποία ανεβάζει τον κόμβο x στη ρίζα του δένδρου.

Όλες οι λειτουργίες των αρθρωτών δένδρων βασίζονται στη διαδικασία $splay$. Ουσιαστικά, εκτελούμε την κάθε λειτουργία, εισαγωγή, διαγραφή ή αναζήτηση, όπως στα απλά δυαδικά δένδρα αναζήτησης και, στη συνέχεια, εκτελούμε τη μέθοδο $splay$ για τον τελευταίο κόμβο του δένδρου στον οποίο είχαμε πρόσβαση.

Αλγόριθμος αναζήτηση(k)

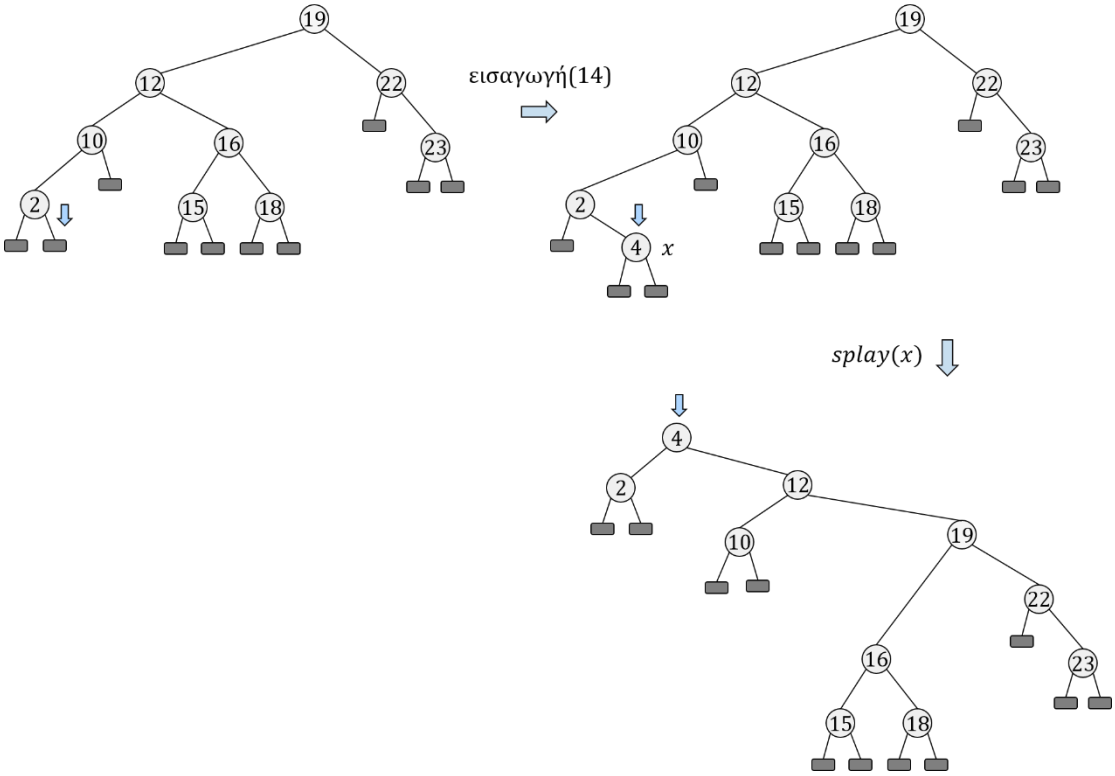
1. Εκτελούμε τη διαδικασία αναζήτησης όπως στο απλό δυαδικό. Έστω v ο τελευταίος μη κενός κόμβος στο μονοπάτι αναζήτησης του k. (Αν το κλειδί k είναι αποθηκευμένο στο δένδρο, τότε κλειδί$(v) = k$.)
2. Εκτελούμε $splay(v)$.



Εικόνα 8.12: Αναζήτηση κλειδιού σε αρθρωτό δένδρο. Μετά την εύρεση του κόμβου x που περιέχει το ζητούμενο κλειδί 19 εκτελούμε τη μέθοδο $splay(x)$, η οποία ανεβάζει τον κόμβο x στη ρίζα του δένδρου.

Αλγόριθμος εισαγωγή(x, k)

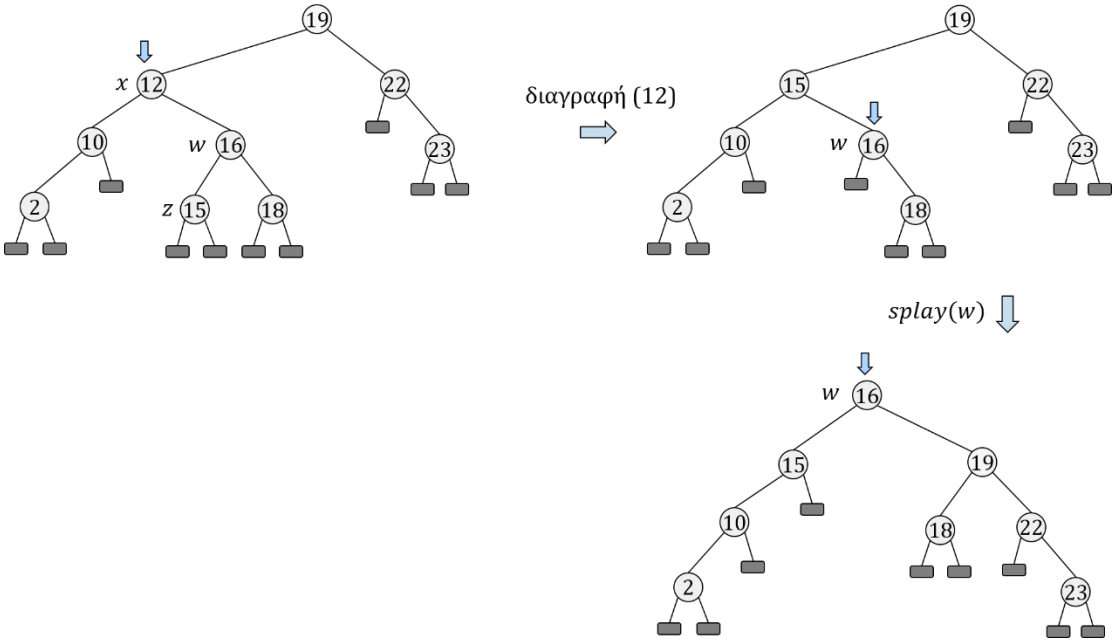
1. Εκτελούμε τη διαδικασία εισαγωγής όπως στο απλό δυαδικό. Έστω v ο κόμβος με κλειδί k. (Αν το κλειδί k υπήρχε ήδη στο δένδρο, τότε ο v είναι ο κόμβος που βρίσκει η αναζήτηση του κλειδιού k. Διαφορετικά, ο v είναι νέος κόμβος που προστέθηκε στο δένδρο.
2. Εκτελούμε $splay(v)$.



Εικόνα 8.13: Εισαγωγή κλειδιού σε αρθρωτό δένδρο. Μετά την τοποθέτηση του νέου κόμβου x εκτελούμε τη μέθοδο $splay(x)$.

Αλγόριθμος διαγραφή(x)

1. Εκτελούμε τη διαδικασία διαγραφής του κόμβου x όπως στο απλό δυαδικό. Έστω z ο κόμβος τον οποίο διαγράφουμε από το δένδρο και έστω w ο γονέας του z. (Ο z είναι ο ίδιος ο κόμβος x, αν δεν έχει παιδιά, ή το μη κενό παιδί του x, αν ο x έχει ακριβώς ένα μη κενό παιδί, ή ο διάδοχος του x, αν και τα δύο παιδιά του x είναι μη κενά.)
2. Εκτελούμε $splay(w)$.



Εικόνα 8.14: Διαγραφή κλειδιού σε αρθρωτό δένδρο. Το κλειδί 12 που θέλουμε να διαγράψουμε βρίσκεται σε κόμβο με δύο μη κενά παιδιά. Έτσι, βρίσκουμε τον κόμβο z με το διάδοχο κλειδί του 12, αντιγράφουμε το 15 στον x, διαγράφουμε τον z και εκτελούμε τη μέθοδο $splay(w)$ για το γονέα w του z.

8.3.1 Ιδιότητες των αρθρωτών δένδρων

Τα αρθρωτά δένδρα διαθέτουν μερικές αξιόλογες ιδιότητες, τις οποίες παραθέτουμε παρακάτω, χωρίς απόδειξη. Αποδεικνύουμε ορισμένες από αυτές στο Κεφάλαιο 13, με τη βοήθεια της αντισταθμιστικής ανάλυσης.

Ιδιότητα Έστω ότι εκτελούμε m λειτουργίες σε αρχικά κενό αρθρωτό δένδρο, όπου κάθε λειτουργία είναι αναζήτηση, εισαγωγή ή διαγραφή. Αν ο συνολικός αριθμός εισαγωγών είναι n, τότε ο συνολικός χρόνος εκτέλεσης των m λειτουργιών είναι $O(m \log n)$, δηλαδή $O(\log n)$ αντισταθμιστικός χρόνος ανά λειτουργία.

Ιδιότητα Έστω ότι εκτελούμε m λειτουργίες σε αρχικά κενό αρθρωτό δένδρο, όπου κάθε λειτουργία είναι αναζήτηση, εισαγωγή ή διαγραφή. Αν ο συνολικός αριθμός εισαγωγών είναι n και το πλήθος των λειτουργιών που εκτελούνται στο i-οστό κλειδί είναι $f(i)$, τότε ο συνολικός χρόνος εκτέλεσης των m λειτουργιών είναι

$$O\left(m + \sum_{i=1}^{n} f(i) \log(m/f(i))\right)$$

.

Ιδιότητα Έστω μια ακολουθία m προσπελάσεων $i_1, i_2, \ldots, i_m$ σε ένα αρθρωτό δένδρο με n κόμβους. Έστω $n(j)$ ο αριθμός των διαφορετικών κλειδιών που προσπελάστηκαν πριν από την j-οστή προσπέλαση, η οποία γίνεται στο κλειδί $k = i_j$, από την προηγούμενη προσπέλαση του ίδιου κλειδιού k. Τότε, ο συνολικός χρόνος εκτέλεσης της ακολουθίας προσπελάσεων είναι

$$O\left(n \log n + m + \sum_{j=1}^{m} \log(n(j) + 1)\right)$$

Συνοψίζοντας, τα πλεονεκτήματα των αρθρωτών δένδρων είναι ότι δεν απαιτούν την αποθήκευση καμίας πληροφορίας για την εξισορρόπηση τους και ότι προσαρμόζονται στη μορφή της ακολουθίας προσπελάσεων. Ωστόσο, έχουν το μειονέκτημα ότι πραγματοποιούν πολλές περιστροφές.

8.3.2 Υλοποίηση σε Java

Περιγράφουμε μια κλάση SplayTree, η οποία υλοποιεί ορισμένες λειτουργίες ενός αρθρωτού δένδρου και αποτελεί επέκταση της BinarySearchTree. Υπενθυμίζουμε ότι κάθε κόμβος αποθηκεύει αντικείμενα τύπου Item με κλειδιά συγκρίσιμου τύπου Key. Η σύνδεση των κόμβων γίνεται με τα πεδία v.left (αριστερό παιδί του v), v.right (δεξί παιδί του v) και v.parent (πατέρας του v).

```
public class SplayTree<Key extends Comparable<Key>, Item> extends BinarySearchTree<Key, Item> { ... }
```

Δίνουμε πρώτα δύο βοηθητικές μεθόδους, που εκτελούν μια αριστερή και μία δεξιά περιστροφή ενός κόμβου x και του γονέα του.

```
private BSTreeNode rotateLeft(BSTreeNode x) {
    BSTreeNode y = x.right;
    x.right = y.left;
    if (x.right != null) {
        x.right.parent = x;
    }

    y.left = x;
    y.parent = x.parent;
    if (x.parent != null) {
        if (x.parent.left == x) {
            x.parent.left = y;
        } else {
            x.parent.right = y;
        }
    }
    x.parent = y;

    return y;
}

private BSTreeNode rotateRight(BSTreeNode y) {
    BSTreeNode x = y.left;
    y.left = x.right;
    if (y.left != null) {
        y.left.parent = y;
    }

    x.right = y;
    x.parent = y.parent;
    if (y.parent != null) {
        if (y.parent.left == y) {
            y.parent.left = x;
        } else {
            y.parent.right = x;
        }
    }
    y.parent = x;
```

```
        return x;
    }
```

Η επόμενη μέθοδος υλοποιεί τη διαδικασία $splay(x)$.

```
    private void splay(BSTreeNode x) {
        if (x == root) {
            return;
        }

        BSTreeNode px, ppx;
        while (x.parent != null) {
            px = x.parent;    // γονέας του x
            ppx = px.parent;  // παππούς του x

            // απλή περιστροφή αν ο γονέας του x είναι η ρίζα
            if (ppx == null) {
                if (x == root.left) {
                    root = rotateRight(root);
                } else {
                    root = rotateLeft(root);
                }

                return;
            }

            if ((px.left == x) && (ppx.left == px)) { // περίπτωση ΑΑ
                rotateRight(ppx);
                rotateRight(px);
            } else if ((px.right == x) && (ppx.left == px)) { // περίπτωση ΑΔ
                rotateLeft(px);
                rotateRight(ppx);
            } else if ((px.left == x) && (ppx.right == px)) { // περίπτωση ΔΑ
                rotateRight(px);
                rotateLeft(ppx);
            } else {  // περίπτωση ΔΔ
                rotateLeft(ppx);
                rotateLeft(px);
            }
        }
        root = x;
    }
```

Στη συνέχεια δίνουμε τις μεθόδους αναζήτησης και εισαγωγής.

```
    /* αναζήτηση αντικειμένου με κλειδί key */
    public Item search(Key key) {
        if (root == null) {
            return null; // tree is empty
        }
        BSTreeNode v = searchNode(key);
        int c = key.compareTo(v.key);
        splay(v);
        if (c == 0) {
            return v.item;    // το αντικείμενο βρέθηκε
        } else {
            return null;      // το αντικείμενο δεν βρέθηκε
        }
    }

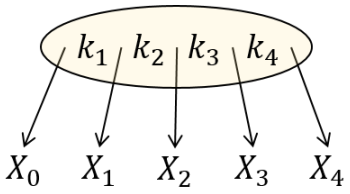
    /* εισαγωγή αντικειμένου item με κλειδί key */
```

```
public void insert(Key key, Item item) {
    BSTreeNode v = insertNode(key, item);
    splay(v);
}
```

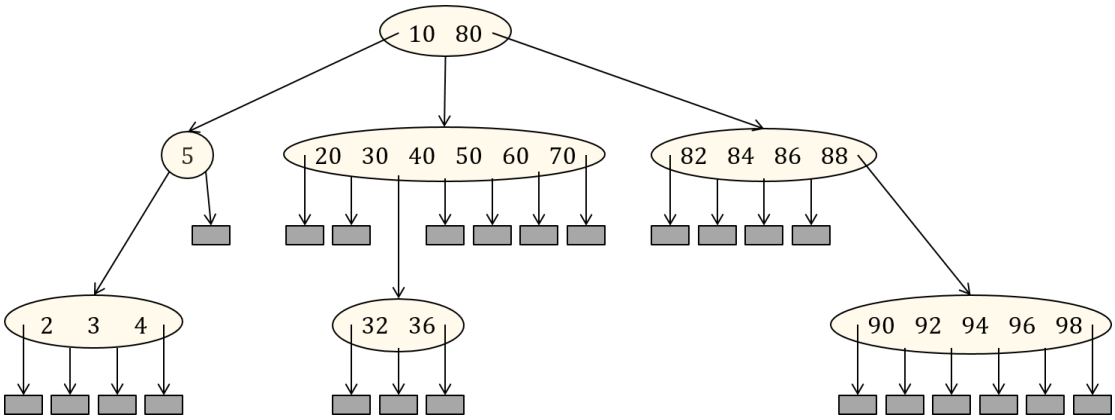
8.4 (a,b)-Δένδρα

Εδώ θα μελετήσουμε μια άλλη κατηγορία ισορροπημένων δένδρων αναζήτησης, τα οποία αποτελούνται από κόμβους πολλαπλής διακλάδωσης. Ένας κόμβος X με d διακλαδώσεις (d-κόμβος για συντομία) αποθηκεύει $d-1$ διατεταγμένα κλειδιά $k_1,\ k_2, \dots, k_{d-1}$, όπου $k_i < k_{i+1}$ για $i = 1,2,\dots,d-2$. Ο X έχει d διατεταγμένα παιδιά $X_0,\ X_1, \dots, X_{d-1}$, έτσι ώστε για κάθε κλειδί k ενός μη κενού παιδιού X_i να ισχύει ότι:

- Αν $i = 0$ τότε $k < k_0$.
- Αν $1 \le i \le d-1$ τότε $k_{i-1} < k < k_i$.
- Αν $i = d$ τότε $k > k_{d-1}$.



Εικόνα 8.39: Ένας 5-κόμβος.



Εικόνα 8.16: Ένα δένδρο πολλαπλής διακλάδωσης.

Ένα δένδρο αναζήτησης πολλαπλής διακλάδωσης αποτελεί γενίκευση του δυαδικού δένδρου αναζήτησης και απαρτίζεται από κόμβους πολλαπλής διακλάδωσης, όπως φαίνεται στην Εικόνα 8.11. Η αναζήτηση ενός κλειδιού k σε ένα τέτοιο δένδρο T γίνεται με παρόμοιο τρόπο, όπως και στα δυαδικά δένδρα. Ξεκινάμε από τη ρίζα του δένδρου T και επισκεπτόμαστε τους κόμβους ενός προς κάποιο φύλλο, μέχρι να βρούμε έναν κόμβο X που να περιέχει το κλειδί k, οπότε η αναζήτηση είναι επιτυχής, ή να καταλήξουμε σε κενό κόμβο, οπότε η αναζήτηση είναι ανεπιτυχής. Ο αλγόριθμος αναζήτησης, όταν βρεθεί σε ένα d-κόμβο X με διατεταγμένα κλειδιά $k_1,\ k_2, \dots, k_{d-1}$ και διατεταγμένα παιδιά $X_0,\ X_1, \dots, X_{d-1}$, θα μεταβεί στο αριστερότερο παιδί X_0 αν $k < k_1$, στο δεξιότερο παιδί X_{d-1} αν $k > k_{d-1}$ ή σε ένα ενδιάμεσο παιδί X_j αν $k_j < k < k_{j+1}$.

Αλγόριθμος αναζήτηση(k)

1. $X \leftarrow$ Τ. ρίζα
2. **ενόσω** $X \neq$ κενό
3. **αν** το k ανήκει στα κλειδιά του X, τότε **επιστροφή** X
4. έστω $k_1,\ k_2, \dots, k_{d-1}$ τα κλειδιά του X σε αύξουσα σειρά και $X_0,\ X_1, \dots, X_{d-1}$ τα
5. **αν** $k < k_0$, **τότε** $i \leftarrow 0$
6. **αλλιώς** $i \leftarrow$ ελάχιστος ακέραιος, τέτοιος ώστε $k < k_i$
7. $X \leftarrow X$.παιδί$(i-1)$
8. **επιστροφή** X

Ένα (a, b)-δένδρο είναι δένδρο αναζήτησης πολλαπλής διακλάδωσης με παραμέτρους $a \geq 2$ και $b > a$, το οποίο διατηρεί τις ακόλουθες αναλλοίωτες συνθήκες:

Αναλλοίωτες συνθήκες (a, b)-δένδρου:

1. Η ρίζα έχει $d-1$ κλειδιά και d παιδιά, όπου $2 \leq d \leq b$.
2. Οι εσωτερικοί κόμβοι εκτός της ρίζας έχουν $t-1$ κλειδιά και t παιδιά, όπου $a \leq t \leq b$.
3. Οι κενοί κόμβοι (φύλλα) ισαπέχουν από τη ρίζα, δηλαδή βρίσκονται όλοι στο ίδιο επίπεδο του δένδρου.

8.4.1 Ύψος ενός (a,b)-δένδρου

Θα υπολογίσουμε τον ελάχιστο και το μέγιστο δυνατό αριθμό κλειδιών σε ένα (a, b)-δένδρο ύψους h.

Έστω n_{min} και N_{min} ο ελάχιστος αριθμός κόμβων και ο ελάχιστος αριθμός κλειδιών, αντίστοιχα, σε ένα (a, b)-δένδρο ύψους h. Ομοίως, έστω n_{max} και N_{max} ο μέγιστος αριθμός κόμβων και ο μέγιστος αριθμός κλειδιών, αντίστοιχα, σε ένα (a, b)-δένδρο ύψους h.

Για τον υπολογισμό του n_{min} θεωρούμε ότι η ρίζα έχει 2 παιδιά και κάθε άλλος κόμβος έχει a παιδιά. Επομένως, έχουμε $2a$ κόμβους στο επίπεδο 2, $2a^2$ κόμβους στο επίπεδο 3, $2a^3$ κόμβους στο επίπεδο 4, κοκ.

Συνολικά έχουμε

$$n_{min} = 1 + 2\sum_{i=1}^{h-1} a^{i-1} = 1 + 2\frac{a^{h-1} - 1}{a - 1}$$

Κάθε κόμβος εκτός της ρίζας αποθηκεύει $a-1$ κλειδιά και η ρίζα αποθηκεύει 1 κλειδί, άρα

$$N_{min} = 1 + (a-1)2\frac{a^{h-1} - 1}{a - 1} = 2a^{h-1} - 1$$

Για τον υπολογισμό του n_{max} θεωρούμε ότι κάθε κόμβος έχει b παιδιά. Επομένως, έχουμε b κόμβους στο επίπεδο 1, b^2 κόμβους στο επίπεδο 2, b^3 κόμβους στο επίπεδο 3, κοκ. Συνολικά έχουμε

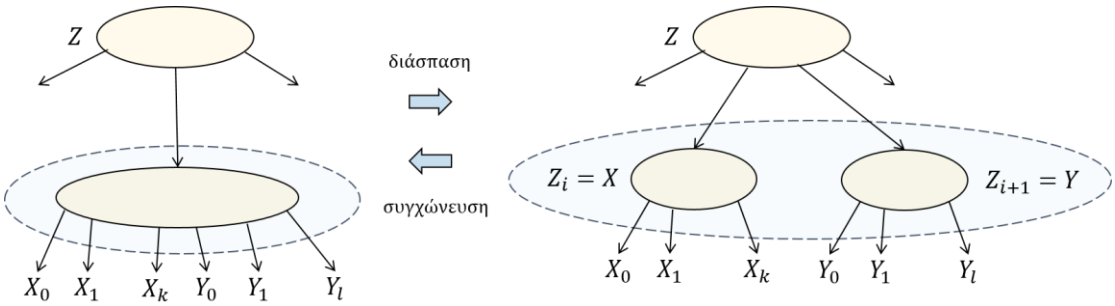
$$n_{max} = \sum_{i=0}^{h-1} b^i = \frac{b^h - 1}{b - 1}$$

Κάθε κόμβος αποθηκεύει $b-1$ κλειδιά, άρα

$$N_{max} = (b-1)\frac{b^h - 1}{b-1} = b^h - 1$$

8.4.2 Διάσπαση και συγχώνευση κόμβων

Όπως και στα δένδρα AVL έτσι και στα (a,b)-δένδρα θα πρέπει να εξασφαλίσουμε ότι διατηρούνται οι αναλλοίωτες συνθήκες μετά από την κάθε εισαγωγή ή διαγραφή κλειδιού. Για την εισαγωγή ενός κλειδιού k εκτελούμε τον αλγόριθμο αναζήτησης του k στο δένδρο, για να εντοπίσουμε τη θέση στην οποία πρέπει να εισαχθεί το κλειδί. Η θέση αυτή, όμως, μπορεί να βρίσκεται σε ένα b-κόμβο X, ο οποίος δεν μπορεί να δεχθεί άλλο κλειδί. Η λύση είναι η διάσπαση του X σε δύο κόμβους, οι οποίοι μοιράζονται τα κλειδιά του X. Αντίστοιχα, στη διαγραφή ενός κλειδιού k, που βρίσκεται σε ένα a-κόμβο X, μπορεί να χρειαστεί να συγχωνεύσουμε τον X με ένα γειτονικό αδελφικό κόμβο. Οι ενέργειες της διάσπασης και συγχώνευσης απεικονίζονται στην Εικόνα 8.16.

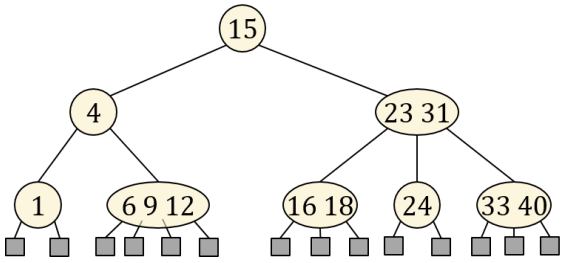


Εικόνα 8.17: Διάσπαση κόμβου και συγχώνευση δύο αδελφικών κόμβων σε (a,b)-δένδρο.

Τόσο η διαδικασία εισαγωγής όσο και η διαδικασία διαγραφής κλειδιού απαιτούν τη μετακίνηση ενός ή περισσότερων κλειδιών μεταξύ γονέα και παιδιού ή αδελφικών κόμβων. Αυτές οι μετακινήσεις, με τη σειρά τους, μπορεί να προκαλέσουν περαιτέρω διασπάσεις ή συγχωνεύσεις, μέχρι να αποκατασταθεί η δομή του (a,b)-δένδρου. Στην επόμενη ενότητα θα εξετάσουμε με λεπτομέρεια πώς γίνεται η εισαγωγή και η διαγραφή στην ειδική περίπτωση των $(2,4)$-δένδρων.

8.4.3 (2,4)-δένδρα

Θα μελετήσουμε διεξοδικά μια συγκεκριμένη κατηγορία (a,b)-δένδρων, τα $(2,4)$-δένδρα. Από την ανάλυση της Ενότητας 8.4.2 προκύπτει ότι το ύψος ενός $(2,4)$-δένδρου είναι $O(\log n)$.



Εικόνα 8.18: Ένα $(2,4)$-δένδρο.

Εισαγωγές

Για την εισαγωγή ενός κλειδιού k εκτελούμε τον αλγόριθμο αναζήτησης σε δένδρο πολλαπλής διακλάδωσης, ο οποίος εντοπίζει τη θέση στην οποία πρέπει να εισαχθεί το νέο κλειδί. Υποθέτουμε ότι το κλειδί k δεν είναι αποθηκευμένο στο (2,4)-δένδρο, καθώς σε αντίθετη περίπτωση, δεν αλλάζει κάτι στη μορφή του δένδρου. Συγκεκριμένα, αν το k δεν είναι αποθηκευμένο στο δένδρο, τότε ο αλγόριθμος αναζήτησης καταλήγει σε ένα κενό κόμβο. Έστω X ο γονέας αυτού του κενού κόμβου. Εισάγουμε το κλειδί k στην κατάλληλη θέση, ώστε να διατηρείται η διάταξη των κλειδιών του κόμβου και προσθέτουμε ένα ακόμα κενό παιδί στον X. Αν ο X δεν ήταν 4-κόμβος πριν από την εισαγωγή, τότε οι αναλλοίωτες συνθήκες διατηρούνται και μετά την εισαγωγή και, επομένως, η διαδικασία εισαγωγής τερματίζει. Διαφορετικά, στην περίπτωση όπου ο X ήταν 4-κόμβος, έχουμε δημιουργήσει ένα προσωρινό 5-κόμβο τον οποίο πρέπει να διασπάσουμε.

Η διαδικασία της διάσπασης ενός 5-κόμβου X έχει ως εξής. Έστω k_1, k_2, k_3, k_4 τα διατεταγμένα κλειδιά και X_0, X_1, X_2, X_3, X_4 τα διατεταγμένα παιδιά του X. (Όλα τα παιδιά είναι κενά, αν ο X είναι ο κόμβος στον οποίον έχει εισαχθεί το νέο κλειδί k. Διαφορετικά, όλα τα παιδιά είναι μη κενά.) Διασπάμε τον X σε δύο κόμβους, X' και X'', όπου ο X' λαμβάνει τα κλειδιά k_1 και k_2 και ο X'' το k_4. Επίσης, ο X' έχει τα διατεταγμένα παιδιά X_0, X_1, X_2 και ο X'' τα X_3 και X_4. Στη συνέχεια, εισάγουμε το κλειδί k_3 στο γονέα Z του X και κάνουμε τον X' αριστερό παιδί του k_3 και τον X'' δεξί παιδί του k_3. Αν και ο Z ήταν 4-κόμβος, τότε επαναλαμβάνουμε την ίδια διαδικασία με τον Z στη θέση του X. Τέλος, αν ο X είναι η ρίζα του δένδρου, τότε δημιουργούμε ένα νέο 2-κόμβο Z, ο οποίος γίνεται η νέα ρίζα του (2,4)-δένδρου με παιδιά τους κόμβους X' και X'', που προήλθαν από τη διάσπαση του X.

Αλγόριθμος διασπαση 5-κόμβου (X)

Έστω k_1, k_2, k_3, k_4 τα διατεταγμένα κλειδιά και X_0, X_1, X_2, X_3, X_4 τα διατεταγμένα παιδιά του X.

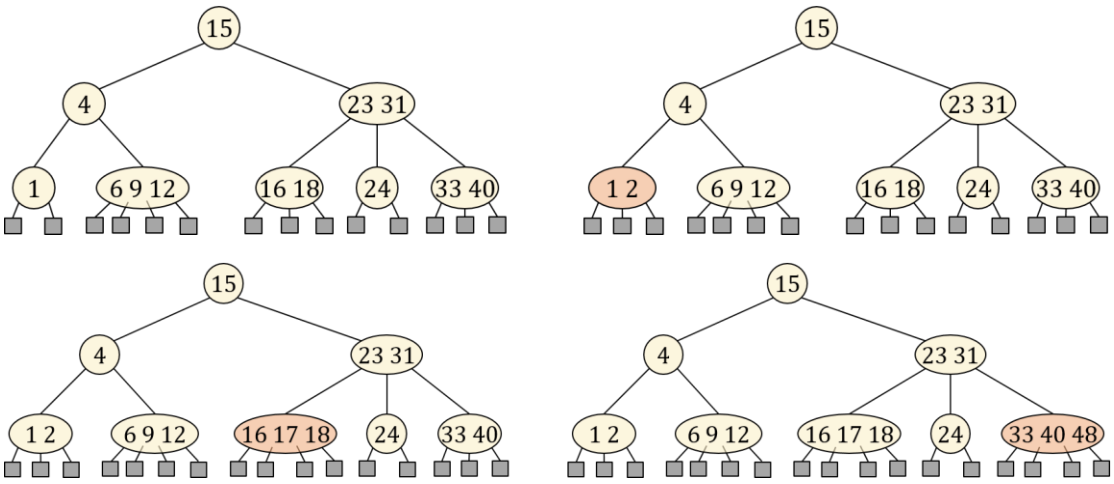
1. Δημιουργούμε δύο νέους κόμβους X' και X'', όπου ο X' είναι 3-κόμβος με κλειδιά k_1 και k_2 και παιδιά X_0, X_1, X_2, και ο X'' είναι 2-κόμβος με κλειδί k_4 και παιδιά X_3 και X_4.
2. Επιστρέφουμε την τριάδα (X', X'', k_3).

Αλγόριθμος εισαγωγή(k)

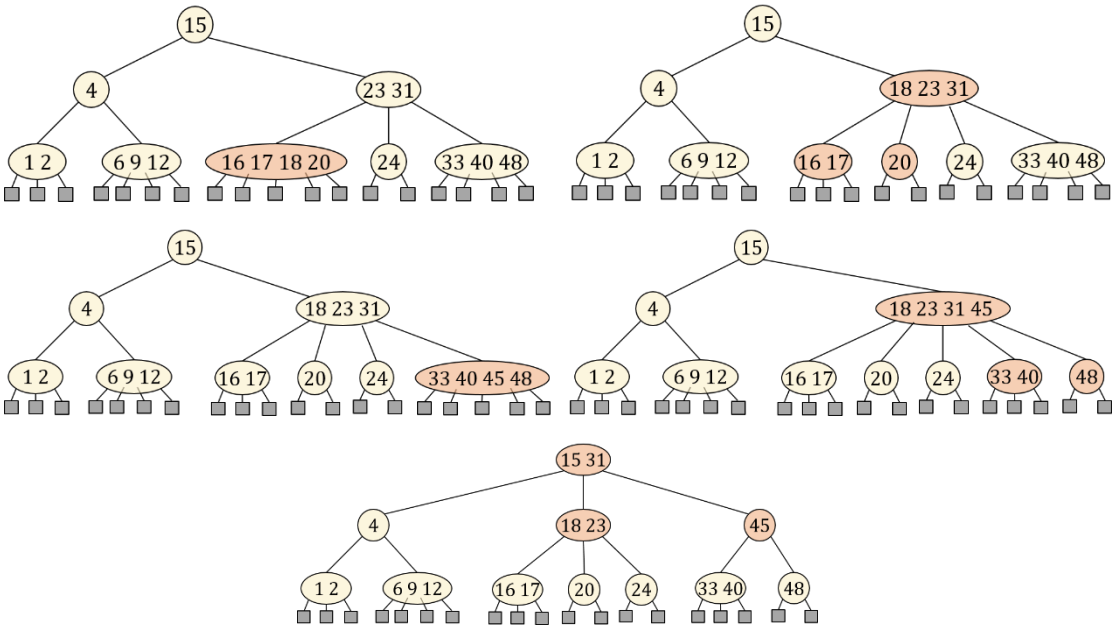
1. Εκτελούμε τον αλγόριθμο αναζήτησης του k. Αν βρεθεί, τότε η διαδικασία τερματίζει. Διαφορετικά, έστω X ο τελευταίος μη κενός κόμβος στο μονοπάτι αναζήτησης.
2. Εισάγουμε το κλειδί k στον X.
3. Ενόσω ο X είναι 5-κόμβος
4. $(X', X'', k_3) \leftarrow$ διασπαση 5-κόμβου (X)
5. $Z \leftarrow$ γονέας(X)
6. Αν ο Z είναι κενός, τότε δημιουργούμε ένα νέο 2-κόμβο Z και τον κάνουμε ρίζα του δένδρου.
7. Εισάγουμε το κλειδί k_3 στον Z και κάνουμε τους κόμβους X' και X'' αριστερό και δεξί παιδί, αντίστοιχα, του k_3.
8. Θέτουμε $X \leftarrow Z$

Αν ο X δεν είναι 4-κόμβος, τότε απλώς εισάγουμε το k στην κατάλληλη θέση, ώστε να διατηρείται η διάταξη των κλειδιών του κόμβου και προσθέτουμε ένα ακόμα κενό παιδί στον X. Στην περίπτωση όπου ο X είναι 4-κόμβος, δημιουργούμε ένα προσωρινό 5-κόμβο τον οποίο θα πρέπει να διασπάσουμε για να δημιουργηθεί χώρος για το νέο κλειδί. Η διαδικασία της

διάσπασης έχει ως εξής. Έστω k_1, k_2, k_3 τα κλειδιά του X. Διασπάμε τον X σε δύο κόμβους X' και X'', όπου ο X' λαμβάνει το κλειδί k_1 και ο X'' το k_3.



Εικόνα 8.19: Απλές περιπτώσεις εισαγωγών σε (2,4)-δένδρο. Εισάγουμε διαδοχικά τα κλειδιά 2, 17 και 48 στο (2,4)-δένδρο της Εικόνα 8.187. Καμία από αυτές τις εισαγωγές δεν προκαλεί παραβίαση των αναλλοίωτων συνθηκών.



Εικόνα 8.20: Περιπτώσεις εισαγωγών σε (2,4)-δένδρο, οι οποίες προκαλούν διάσπαση κόμβων. Εισάγουμε διαδοχικά τα κλειδιά 20 και 45 στο τελευταίο (2,4)-δένδρο της Εικόνα 8.19. Η εισαγωγή του 20 δημιουργεί ένα προσωρινό 5-κόμβο X, με κλειδιά 16, 17, 18 και 20, ο οποίος διασπάται σε ένα 3-κόμβο, με κλειδιά 16 και 17 και ένα 2-κόμβο, με κλειδί 20. Το κλειδί 18 μεταφέρεται στο γονέα του X, ο οποίος μετατρέπεται σε 4-κόμβο και η διαδικασία εισαγωγής τερματίζει. Στη συνέχεια, η εισαγωγή του 45 δημιουργεί ένα προσωρινό 5-κόμβο X με κλειδιά 33, 40, 45 και 48, ο οποίος διασπάται σε ένα 3-κόμβο με κλειδιά 33 και 40, και ένα 2-κόμβο με κλειδί 48. Το κλειδί 45 μεταφέρεται στο γονέα Z του X, ο οποίος μετατρέπεται

σε 5-κόμβο και επομένως η διαδικασία εισαγωγής συνεχίζεται με τη διάσπαση του Z. Από τη διάσπαση του κόμβου Z λαμβάνουμε ένα 3-κόμβο με κλειδιά 18 και 23 και ένα 2-κόμβο με κλειδί 45. Το κλειδί 31 μεταφέρεται στη ρίζα του δένδρου, η οποία μετατρέπεται σε 3-κόμβο και η διαδικασία εισαγωγής τερματίζει.

Στη συνέχεια, περιγράφουμε μια εναλλακτική μέθοδο εισαγωγής.

Εισαγωγή με διάσπαση από πάνω. Διαιρούμε κάθε 4-κόμβο που βρίσκεται στο μονοπάτι εισαγωγής.

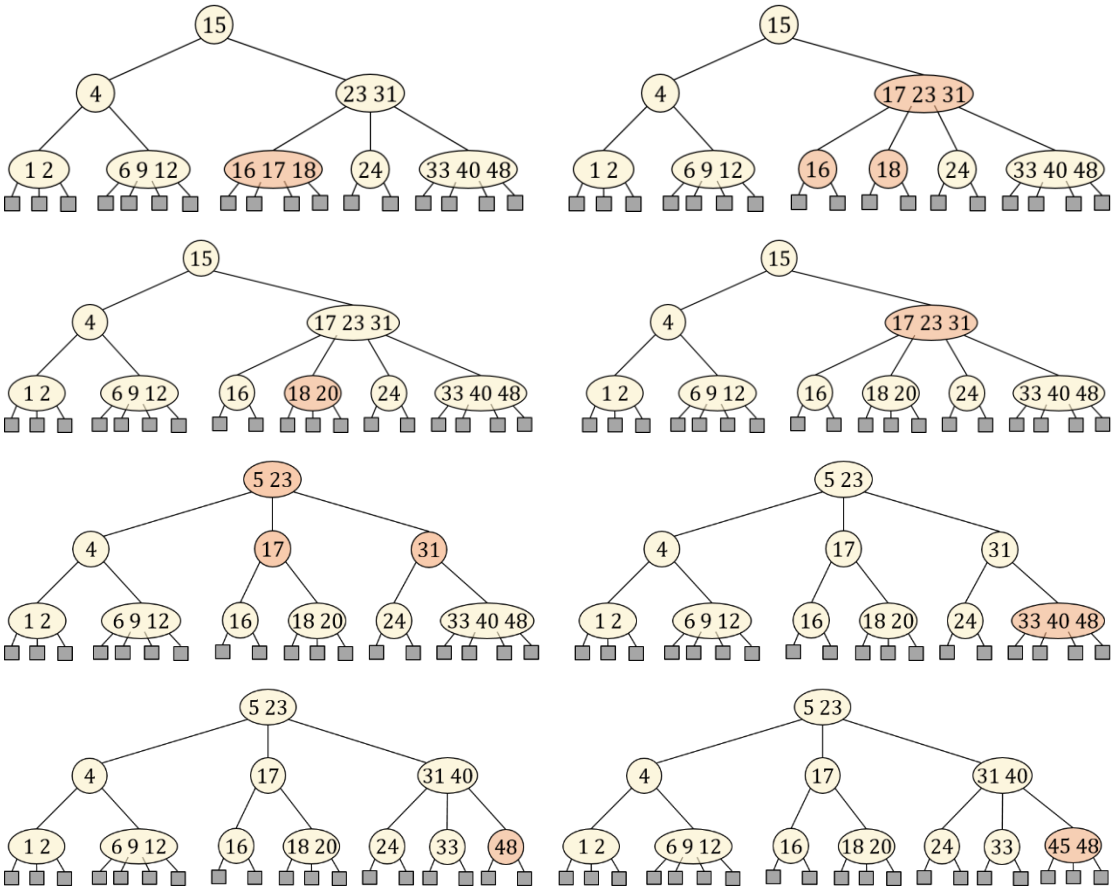
Αλγόριθμος διασπαση 4-κόμβου (X)

Έστω k_1, k_2, k_3 τα διατεταγμένα κλειδιά και X_0, X_1, X_2, X_3 τα διατεταγμένα παιδιά του X.

1. Δημιουργούμε δύο νέους 2-κόμβους X' και X'', όπου ο X' έχει κλειδί k_1και παιδιά X_0 και X_1 και ο X'' έχει κλειδί k_3 και παιδιά X_2 και X_3.
2. Επιστρέφουμε την τριάδα (X', X'', k_2).

Αλγόριθμος εισαγωγή με διάσπαση από πάνω (k)

1. $X \leftarrow$ Τ. ρίζα
2. Ενόσω $X \neq$ κενό
3. Αν το k ανήκει στα κλειδιά του X, τότε επιστροφή X
4. Αν ο X είναι 4-κόμβος, τότε
5. $(X', X'', k_2) \leftarrow$ διάσπαση 4-κόμβου (X)
6. $Z \leftarrow$ γονέας(X)
7. Αν ο Z είναι κενός, τότε δημιουργούμε ένα νέο 2-κόμβο Z και τον κάνουμε ρίζα του δένδρου.
8. Εισάγουμε το κλειδί k_2 στον Z και κάνουμε τους κόμβους X' και X'' αριστερό και δεξί παιδί, αντίστοιχα, του k_2.
9. $X \leftarrow$ επόμενος κόμβος στο μονοπάτι εισαγωγής του k.
10. Εισάγουμε το κλειδί k στον τελευταίο μη κενό κόμβο X.

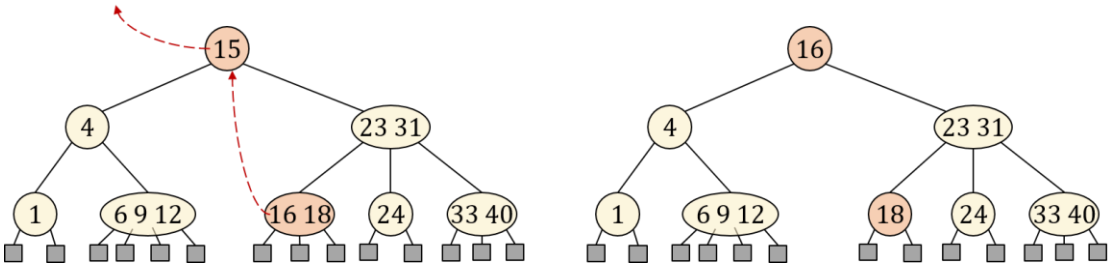


Εικόνα 8.21: Εισαγωγές με διάσπαση από πάνω. Εισάγουμε διαδοχικά τα κλειδιά 20 και 45 στο τελευταίο (2,4)-δένδρο της Εικόνα 8.19. Στο μονοπάτι εισαγωγής του 20 βρίσκουμε ένα 4-κόμβο X με κλειδιά $16, 17$ και 18, ο οποίος διασπάται σε δύο 2-κόμβους X' και X'', με κλειδιά 16 και 18 αντίστοιχα. Το κλειδί 17 μεταφέρεται στο γονέα του X, ο οποίος μετατρέπεται σε 4-κόμβο, ενώ το κλειδί 20 τοποθετείται στον X'' και η διαδικασία εισαγωγής τερματίζει. Στη συνέχεια, στο μονοπάτι εισαγωγής του 45 βρίσκουμε ένα 4-κόμβο W, με κλειδιά $17, 23$ και 31, ο οποίος διασπάται σε δύο 2-κόμβους, με κλειδιά 17 και 31 αντίστοιχα, ενώ το κλειδί 23 μεταφέρεται στο γονέα του W. Συνεχίζοντας την κάθοδο στο μονοπάτι εισαγωγής από τον W'', συναντάμε έναν ακόμα 4-κόμβο X, με κλειδιά $33, 40$ και 48, ο οποίος διασπάται σε δύο 2-κόμβους X' και X'' με κλειδιά 33 και 48 αντίστοιχα. Το κλειδί 40 μεταφέρεται στο γονέα του X, ο οποίος μετατρέπεται σε 4-κόμβο, ενώ το κλειδί 45 τοποθετείται στον X'' και η διαδικασία εισαγωγής τερματίζει.

Διαγραφές

Όπως και στα δυαδικά δένδρα, η διαδικασία διαγραφής ενός κλειδιού είναι πιο περίπλοκη από την εισαγωγή. Έστω k το κλειδί που θέλουμε να διαγράψουμε, το οποίο βρίσκεται στον κόμβο X. Αν ο X δεν είναι κόμβος του τελευταίου επιπέδου, τότε για να είναι εφικτή η διαγραφή του k, θα πρέπει να βρούμε ένα κλειδί που να αντικαταστήσει το k στον X. Αυτή η κατάσταση είναι όμοια με τη διαγραφή ενός κόμβου με δύο μη κενά παιδιά σε δυαδικό δένδρο αναζήτησης και, πράγματι, έχει παρόμοια λύση. Αρκεί να τοποθετήσουμε στον X το διάδοχο κλειδί k' του k. Είναι εύκολο να παρατηρήσουμε ότι το k' θα βρίσκεται σε κόμβο του τελευταίου επιπέδου.

Το γεγονός αυτό μας επιτρέπει να εστιάσουμε την προσοχή μας στην περίπτωση όπου το κλειδί που διαγράφεται βρίσκεται στο τελευταίο επίπεδο.



Εικόνα 8.22: Διαγραφή κλειδιού από κόμβο X με μη κενά παιδιά. Για να μπορέσουμε να διαγράψουμε το κλειδί 15, το αντικαθιστούμε με το 16, το οποίο είναι το διάδοχο κλειδί του και βρίσκεται σε ένα 3-κόμβο X' με κενά παιδιά. Στη συνέχεια, διαγράφουμε το 16 από τον X', με αποτέλεσμα ο X' να μετατραπεί σε 2-κόμβο. Μετά τη διαγραφή οι αναλλοίωτες συνθήκες του (2,4)-δένδρου έχουν αποκατασταθεί.

Η επόμενη πρόκληση που καλούμαστε να αντιμετωπίσουμε είναι αν X ήταν 2-κόμβος πριν από τη διαγραφή. Τότε, μετά τη διαγραφή ο X μετατρέπεται σε ένα προσωρινό 1-κόμβο με ένα παιδί και χωρίς κανένα κλειδί. Για να αποκαταστήσουμε τη βλάβη, μπορούμε να εφαρμόσουμε μια από τις παρακάτω ιδέες: τη μεταφορά κλειδιών και τη συγχώνευση κόμβων. Ας υποθέσουμε αρχικά, ότι ο X δεν είναι το αριστερότερο παιδί του Z. Έστω X' ο αριστερός αδελφός του X και έστω l το κλειδί του Z που αντιστοιχεί στους X και X' και έστω q το μέγιστο κλειδί του X'.

Αν ο X' είναι 2-κόμβος, τότε συγχωνεύουμε τους X και X' σε ένα νέο κόμβο W και μεταφέρουμε το κλειδί l από τον Z στον W. Η μετακίνηση του κλειδιού l αφήνει ένα λιγότερο κλειδί στον κόμβο Z. Αν αυτό έχει ως αποτέλεσμα να γίνει 1-κόμβος, τότε επαναλαμβάνουμε την ίδια διαδικασία με τον Z στη θέση του X.

Η μεταφορά κλειδιών για τον X γίνεται θέτοντας το l ως κλειδί του X, ο οποίος γίνεται 2-κόμβος και λαμβάνει το δεξιότερο παιδί του X' ως αριστερό παιδί, θέτοντας το q ως το μικρότερο κλειδί του Z.

Αν ο X' είναι 2-κόμβος τότε συγχωνεύουμε τους X και X' σε ένα νέο κόμβο W και μεταφέρουμε το κλειδί l από τον Z στον W.

Αλγόριθμος συγχώνευση (X, X')

Έστω Z ο γονέας των κόμβων X και X' και έστω l το κλειδί του Z που αντιστοιχεί στους X και X'.

1. Δημιουργούμε ένα νέο κόμβο W, ο οποίος προκύπτει από τη συγχώνευση του 1-κόμβου X και του 2-κόμβου X', μαζί με την προσθήκη του κλειδιού l.
2. Τοποθετούμε τον W στη διατεταγμένη λίστα των παιδιών του Z στη θέση των X και X' και διαγράφουμε το l από τη διατεταγμένη λίστα των κλειδιών του Z.
3. Επιστρέφουμε τον κόμβο W.

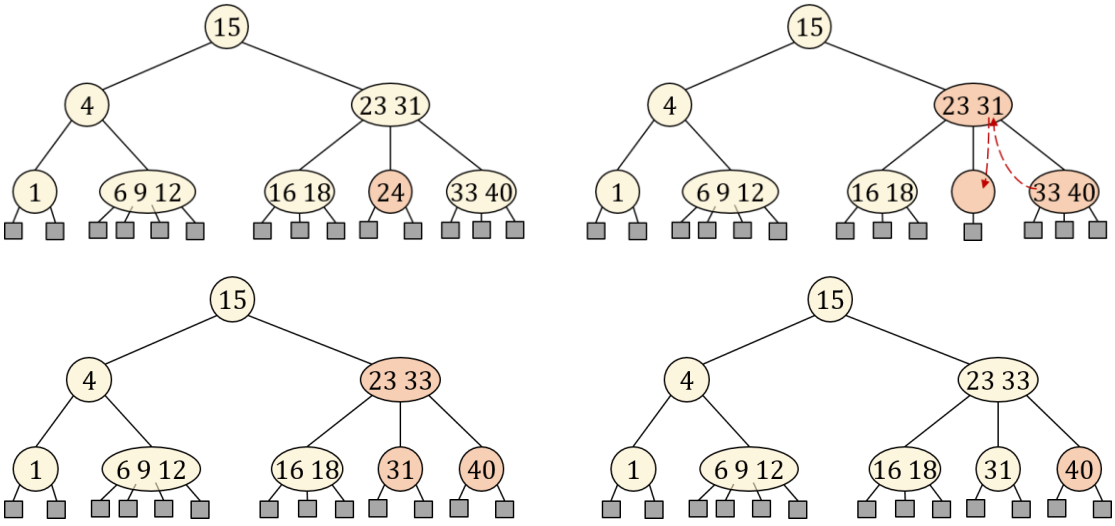
Αλγόριθμος μεταφορά κλειδιού (X, X')

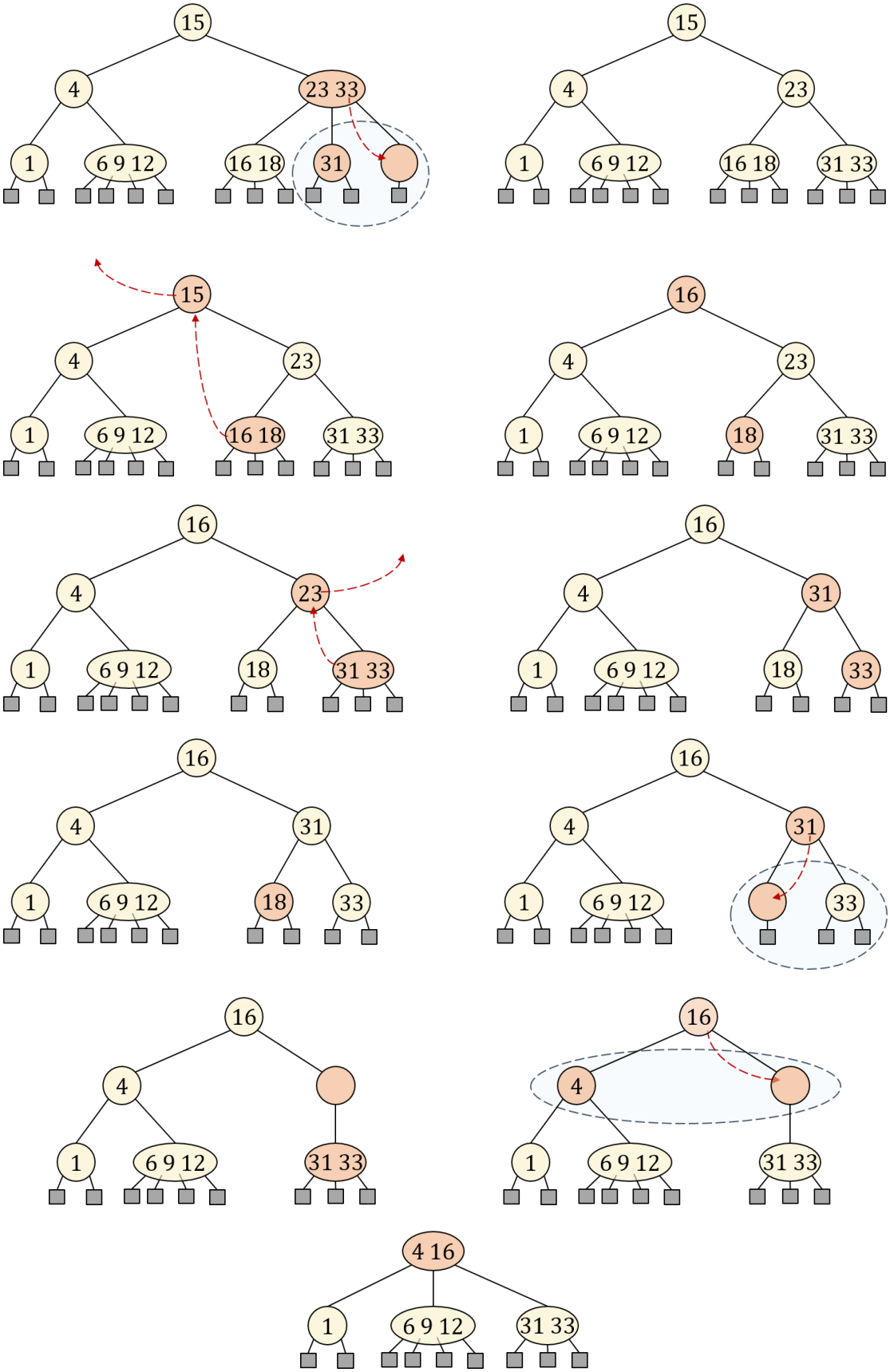
Έστω Z ο γονέας των κόμβων X και X' και έστω l το κλειδί του Z που αντιστοιχεί στους X και X'. Επίσης, έστω q και Y, αντίστοιχα, το μικρότερο κλειδί και το αριστερότερο παιδί του X', αν ο X είναι αριστερός αδελφός του X'. Διαφορετικά, το q και το Y είναι, αντίστοιχα, το μεγαλύτερο κλειδί και το δεξιότερο παιδί του X'.

1. Μετατρέπουμε τον 1-κόμβο X σε 2-κόμβο, τοποθετώντας στον X το κλειδί l και κάνοντας τον Y παιδί του X.
2. Διαγράφουμε το l και τον Y από τη διατεταγμένη λίστα των κλειδιών και των παιδιών του X', μετατρέποντας τον X' από 4-κόμβο σε 3-κόμβο ή από 3-κόμβο σε 2-κόμβο.
3. Αντικαθιστούμε στον Z το κλειδί l με το κλειδί q.
4. Επιστρέφουμε τον κόμβο X.

Αλγόριθμος διαγραφή (k)

1. Εκτελούμε τον αλγόριθμο αναζήτησης του k. Αν δε βρεθεί, τότε η διαδικασία τερματίζει. Διαφορετικά έστω X ο κόμβος που περιέχει το k.
2. Αν ο X έχει μη κενά παιδιά, τότε βρίσκουμε το διάδοχο κλειδί k' του k και τον κόμβο X' που το περιέχει. Αντικαθιστούμε το k με το k' στον X. Θέτουμε $k \leftarrow k'$ και $X \leftarrow X'$.
3. Διαγράφουμε από τον κόμβο X το κλειδί k και ένα αντίστοιχο κενό παιδί.
4. Ενόσω ο X είναι 1-κόμβος
5. Αν ο X είναι η ρίζα, τότε διαγράφουμε τον X, θέτουμε το μοναδικό παιδί του ως νέα ρίζα και επιστρέφουμε.
6. Διαφορετικά, αν ο X έχει ένα γειτονικό αδελφό X' ο οποίος είναι 3-κόμβος ή 4-κόμβος, τότε εκτελούμε μεταφορά κλειδιού (X, X') και επιστρέφουμε.
7. Διαφορετικά, έστω X' ένας γειτονικός αδελφός του X ο οποίος είναι 2-κόμβος. Εκτελούμε $W \leftarrow$ συγχώνευση (X, X').
8. Θέτουμε $X \leftarrow$ γονέας(W).





Εικόνα 8.23: Ακολουθία διαγραφών στο (2,4)-δένδρο της Εικόνα 8.18. Διαγράφουμε διαδοχικά τα κλειδιά 24, 40, 15, 23 και 18. Μετά τη διαγραφή του 24, ο κόμβος X ο οποίος περιείχε αυτό το κλειδί μετατρέπεται σε προσωρινό 1-κόμβο. Η βλάβη αποκαθίσταται με τη

μεταφορά του κλειδιού 31 από το γονέα Z του X. Το κλειδί 31 αντικαθίσταται στον Z με το 33, το οποίο είναι το ελάχιστο κλειδί του δεξιού αδελφού του X. Το κλειδί 40 βρίσκεται αποθηκευμένο σε 2-κόμβο X, με αποτέλεσμα η διαγραφή του να μετατρέπει τον X σε 1-κόμβο. Αυτή τη φορά, ο X έχει μόνο αριστερό αδελφό X', ο οποίος είναι 2-κόμβος, οπότε εκτελούμε τη συγχώνευση των X και X' σε ένα νέο κόμβο W, ο οποίος λαμβάνει επίσης το κλειδί 33 από το γονέα. Στη συνέχεια, για τη διαγραφή του 15, εντοπίζουμε το διάδοχο κλειδί 16 σε 3-κόμβο X του προτελευταίου επιπέδου, το οποίο αντικαθιστά το 15 στη ρίζα και διαγράφουμε το 16 από τον X. Η διαγραφή του 23 γίνεται με παρόμοιο τρόπο. Τέλος, το κλειδί 18 βρίσκεται σε 2-κόμβο X, ο οποίος μετατρέπεται σε προσωρινό 1-κόμβο μετά τη διαγραφή. Ο X έχει μόνο δεξιό αδελφό X', ο οποίος είναι 2-κόμβος, οπότε εκτελούμε τη συγχώνευση των X και X' σε ένα νέο κόμβο W, ο οποίος λαμβάνει επίσης το κλειδί 31 από το γονέα Z. Τώρα, όμως, ο Z γίνεται με τη σειρά του προσωρινός 1-κόμβος και ο μοναδικός αδελφός του Z' είναι 2-κόμβος. Συγχωνεύουμε τους Z και Z' σε ένα νέο κόμβο W, ο οποίος λαμβάνει επίσης το κλειδί 16 από το γονέα τους που είναι η ρίζα του δένδρου. Τώρα ή ρίζα είναι 1-κόμβος και, επομένως, διαγράφεται από το δένδρο.

8.5 Κοκκινόμαυρα δένδρα

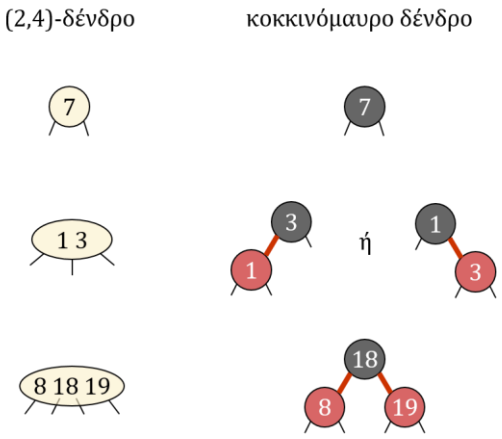
Τα κοκκινόμαυρα δένδρα αποτελούν μια σημαντική κατηγορία ισορροπημένων δυαδικών δένδρων αναζήτησης, καθώς διαθέτουν την ιδιότητα ότι για την αποκατάσταση των συνθηκών ισορροπίας τους αρκεί μια απλή ή μια διπλή περιστροφή, τόσο μετά από μια εισαγωγή όσο και μετά από μια διαγραφή.

Οι συνθήκες ισορροπίας των κοκκινόμαυρων δένδρων εκφράζονται μέσω ενός έγκυρου χρωματισμού των κόμβων του δένδρου με δύο χρώματα, το κόκκινο και το μαύρο. Δηλαδή, κάθε κόμβος είναι χρωματισμένος είτε κόκκινος είτε μαύρος, με τέτοιο τρόπο, ώστε να ικανοποιούνται οι αναλλοίωτες συνθήκες που δίνουμε παρακάτω. Ορίζουμε το **μαύρο βάθος** $\mathrm{MB}(x)$ ενός κόμβου x ως το πλήθος των μαύρων κόμβων που είναι γνήσιοι πρόγονοι του x.

Αναλλοίωτες συνθήκες κοκκινόμαυρου δένδρου:

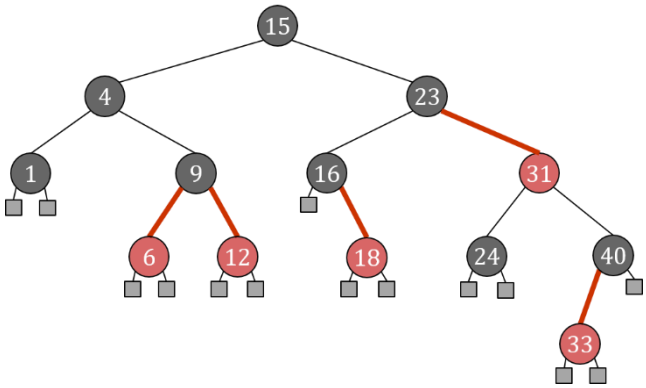
1. Η ρίζα είναι μαύρη.
2. Οι εξωτερικοί (κενοί) κόμβοι είναι μαύροι.
3. Τα παιδιά ενός κόκκινου κόμβου είναι μαύρα.
4. Όλοι οι εξωτερικοί κόμβοι έχουν το ίδιο μαύρο βάθος.

Υπάρχει μια ωραία αλλά και χρήσιμη αντιστοίχιση μεταξύ των κοκκινόμαυρων δένδρων και των (2,4)-δένδρων, η οποία δίνεται στην Εικόνα . Ουσιαστικά, ένας κόκκινος κόμβος ομαδοποιείται με το μαύρο γονέα του, για να σχηματίσει ένα 3-κόμβο ή ένα 4-κόμβο.



Εικόνα 8.24: Αντιστοίχιση των μεταξύ των κόμβων ενός (2,4)-δένδρου και ενός κοκκινόμαυρου δένδρου.

Με την παραπάνω αντιστοίχιση μπορούμε να λάβουμε για κάθε κοκκινόμαυρο δένδρο ένα μοναδικό (2,4)-δένδρο. Αντίστροφα, σε ένα (2,4)-δένδρο μπορεί να αντιστοιχούν περισσότερα από ένα κοκκινόμαυρα δένδρα λόγω των δύο ισοδύναμων διατάξεων για τους 3-κόμβους.



Εικόνα 8.25: Ένα κοκκινόμαυρο δένδρο που αντιστοιχεί στο (2,4)-δένδρο της Εικόνα 8.18: Ένα (2,4)-δένδρο..

Η παραπάνω αντιστοίχιση μας είναι χρήσιμη για δύο λόγους. Πρώτα, μας επιτρέπει να δώσουμε άμεσα ένα άνω φράγμα για το ύψος ενός κοκκινόμαυρου δένδρου. Πράγματι, έστω T ένα κοκκινόμαυρο δένδρο με n κλειδιά, το οποίο αντιστοιχεί σε ένα (2,4)-δένδρο T'. Από την αντιστοίχιση προκύπτει ότι το μαύρο βάθος των εξωτερικών κόμβων του T είναι ίσο με το ύψος του T', το οποίο είναι το πολύ $\log(n+1)$. Επιπλέον, η αναλλοίωτη συνθήκη 3 συνεπάγεται ότι το ύψος του T είναι το πολύ διπλάσιο από το μαύρο βάθος των εξωτερικών του κόμβων.

Ιδιότητα Έστω T ένα κοκκινόμαυρο δένδρο με n κλειδιά. Ισχύει $\log(n+1) \leq \text{ύψος}(T) \leq 2\log(n+1)$.

Ο δεύτερος λόγος που μας είναι χρήσιμη η αντιστοίχιση με τα (2,4)-δένδρα είναι ότι μας επιτρέπει να αναπτύξουμε αλγόριθμους αποκατάστασης των συνθηκών ισορροπίας μετά από την εισαγωγή ή διαγραφή ενός κλειδιού.

8.5.1 Αποκατάσταση των συνθήκων ισορροπίας

Όπως και στα δένδρα AVL, οι λειτουργίες εισαγωγής και διαγραφής πραγματοποιούνται σε δύο φάσεις. Πρώτα εκτελούμε τις αντίστοιχες διαδικασίες, όπως στα απλά δυαδικά δένδρα αναζήτησης και, στη συνέχεια, ελέγχουμε αν έχει παραβιαστεί κάποια από τις αναλλοίωτες συνθήκες. Σε μια τέτοια περίπτωση πρέπει να καθορίσουμε διαδικασίες αποκατάστασης των αναλλοίωτων συνθηκών. Η αποκατάσταση μπορεί να χρησιμοποιήσει περιστροφές, αλλά και ένα επιπλέον εργαλείο: τον αναχρωματισμό των κόμβων.

Εισαγωγές

Έστω ότι εισάγουμε ένα νέο κόμβο x στο κοκκινόμαυρο δένδρο T. Αν το T ήταν προηγουμένως κενό, τότε ο x γίνεται η ρίζα του δένδρου και τον χρωματίζουμε μαύρο, όπως και τα κενά παιδιά του. Διαφορετικά, χρωματίζουμε τον x κόκκινο και τα κενά παιδιά του μαύρα. Με αυτόν τον τρόπο διατηρούμε τις αναλλοίωτες συνθήκες 1, 2 και 4, αλλά μπορεί να έχουμε παραβιάσει τη συνθήκη 3. Έστω y ο γονέας του x, ο οποίος δεν είναι η ρίζα του δένδρου, καθώς είναι κόκκινος. Επίσης, έστω z ο γονέας του y και έστω y' ο αδελφός του y. Από τα παραπάνω γνωρίζουμε ότι ο x και ο y είναι κόκκινοι, ενώ ο z μαύρος. Διαχωρίζουμε δύο περιπτώσεις ανάλογα με το χρώμα του y':

Περίπτωση 1) Ο y' είναι μαύρος. Σε αυτήν την περίπτωση οι κόμβοι x, y και z σχηματίζουν ένα 4-κόμβο στο αντίστοιχο (2,4)-δένδρο αλλά χωρίς τη σωστή διάταξη που δίνεται στην Εικόνα 8.24. Για να σχηματίσουμε τη σωστή διάταξη, θα πρέπει μεταξύ των κόμβων x, y και z αυτός με το μεσαίο κλειδί να γίνει γονέας των άλλων δύο. Αυτό μπορούμε να το επιτύχουμε με μια απλή ή με μία διπλή περιστροφή, ανάλογα με τη φορά των συνδέσμων (z, y) και (y, x), όπως στα δένδρα AVL. Έχουμε, λοιπόν, τις εξής υποπεριπτώσεις:

Υποπερίπτωση ΑΑ (αριστερά-αριστερά): Ο y είναι αριστερό παιδί του z και ο x είναι αριστερό παιδί του y. Εκτελούμε μια δεξιά περιστροφή του ζεύγους (z, y). Χρωματίζουμε τον y μαύρο και τον z κόκκινο.

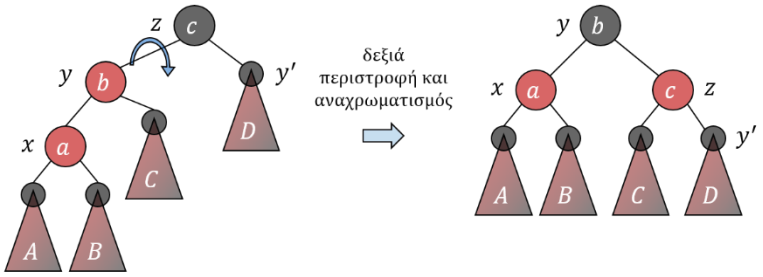
Υποπερίπτωση ΑΔ (αριστερά-δεξιά): Ο y είναι αριστερό παιδί του z και ο x είναι δεξί παιδί του y. Εκτελούμε μια διπλή αριστερή-δεξιά περιστροφή της τριάδας (z, y, x). Χρωματίζουμε τον x μαύρο και τον z κόκκινο.

Υποπερίπτωση ΔΑ (δεξιά-αριστερή): Ο y είναι δεξί παιδί του z και ο x είναι αριστερό παιδί του y. Εκτελούμε μια διπλή δεξιά-αριστερή περιστροφή της τριάδας (z, y, x). Χρωματίζουμε τον x μαύρο και τον z κόκκινο.

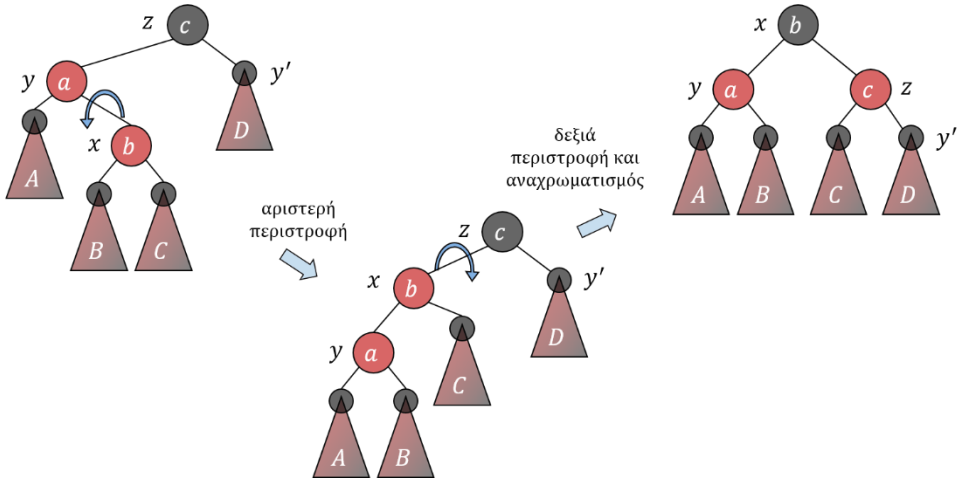
Υποπερίπτωση ΔΔ (δεξιά-δεξιά): Ο y είναι δεξί παιδί του z και ο x είναι δεξί παιδί του y. Εκτελούμε μια αριστερή περιστροφή του ζεύγους (z, y). Χρωματίζουμε τον y μαύρο και τον z κόκκινο.

Περίπτωση 1: Ο θείος y' του x είναι μαύρος

Υποπερίπτωση ΑΑ: y = αριστερό παιδί του z και x = αριστερό παιδί του y



Υποπερίπτωση ΑΔ: y = αριστερό παιδί του z και x = δεξί παιδί του y

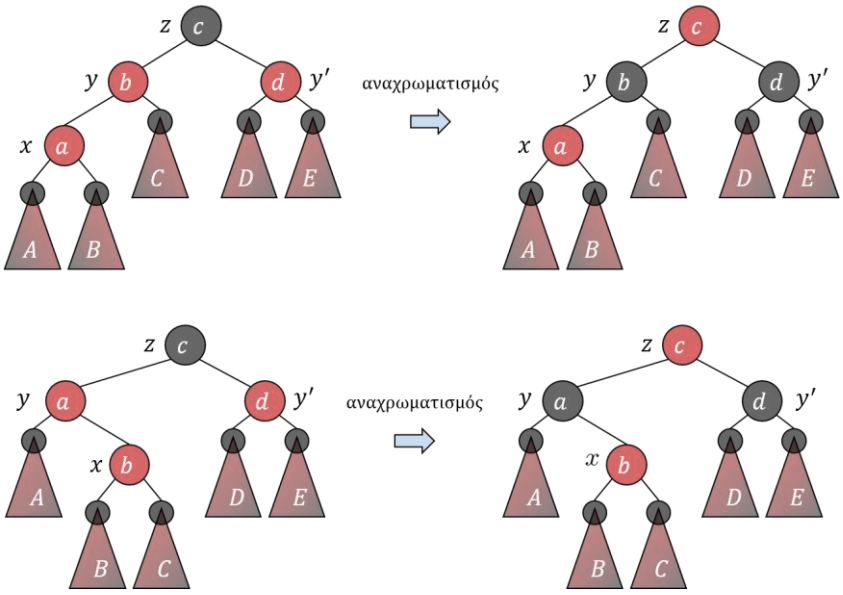


Εικόνα 8.26: Διαδικασία αποκατάστασης των αναλλοίωτων συνθηκών ενός κοκκινόμαυρου δένδρου μετά από εισαγωγή. Στην περίπτωση 1, ο κόμβος x και ο γονέας y του x είναι κόκκινοι, ενώ ο θείος y' του x (δηλαδή ο αδελφός του y) είναι μαύρος. Αρκεί μια απλή περιστροφή (στην υποπερίπτωση ΑΑ και τη συμμετρική της ΔΔ) ή μια διπλή περιστροφή (στην υποπερίπτωση ΑΔ και τη συμμετρική της ΔΑ) και αναχρωματισμός, για να αποκαταστήσει τις συνθήκες ισορροπίας.

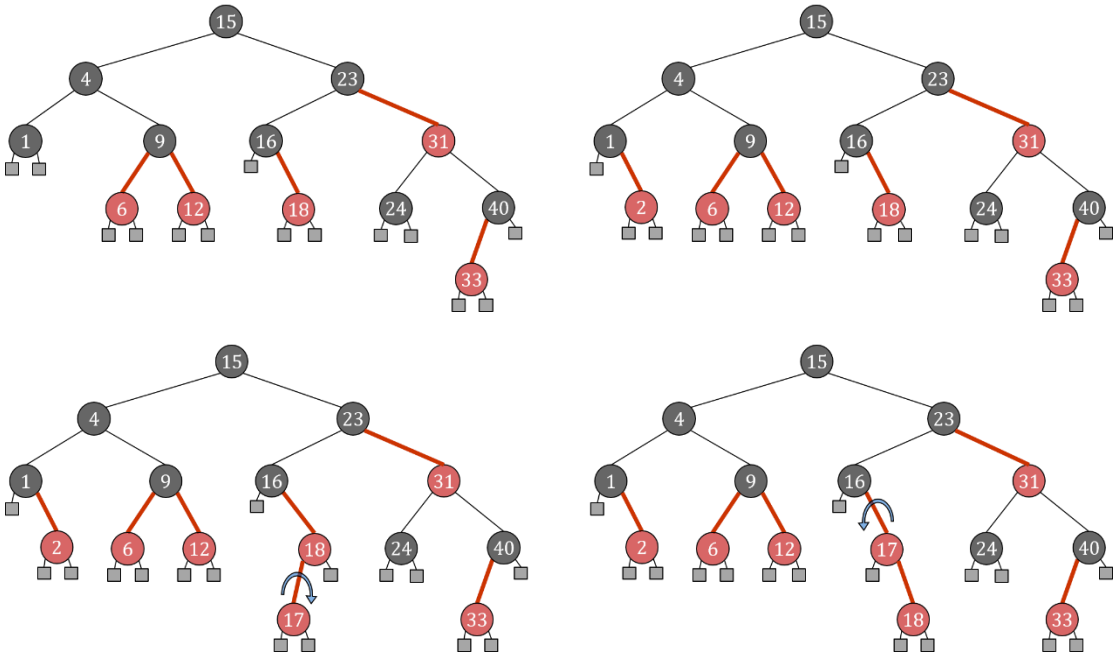
Περίπτωση 2) Ο y' είναι κόκκινος. Σε αυτήν την περίπτωση οι κόμβοι x, y, y' και z σχηματίζουν ένα προσωρινό 5-κόμβο ο οποίος θα πρέπει να διασπαστεί στο αντίστοιχο (2,4)-δένδρο. Η διάσπαση μπορεί να επιτευχθεί με αναχρωματισμό. Συγκεκριμένα, αλλάζουμε το χρώμα των y και y' σε μαύρο και του z σε κόκκινο, εκτός αν ο z είναι η ρίζα, οπότε παραμένει μαύρος.

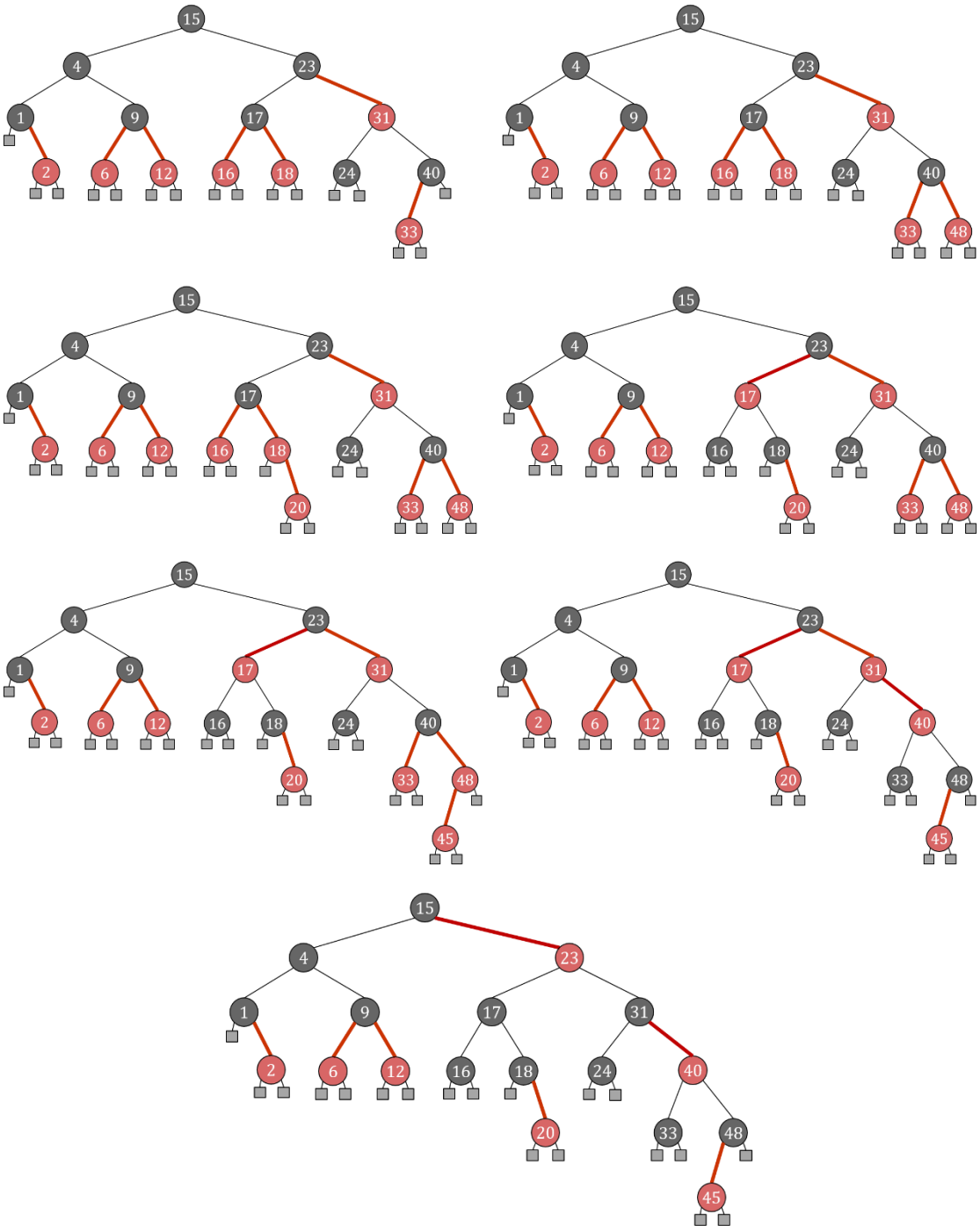
Αυτό έχει ως αποτέλεσμα οι κόμβοι x και y να σχηματίζουν ένα 3-κόμβο και ο y' ένα 2-κόμβο. Ωστόσο, η αλλαγή του χρώματος του z μπορεί να προκαλεί εκ νέου παραβίαση της αναλλοίωτης συνθήκης 3, οπότε θα πρέπει να επαναλάβουμε τη διαδικασία αποκατάστασης με τον κόμβο z στο ρόλο του x.

Περίπτωση 2: Ο θείος y' του x είναι κόκκινος



Εικόνα 8.27: Διαδικασία αποκατάστασης των αναλλοίωτων συνθηκών ενός κοκκινόμαυρου δένδρου μετά από εισαγωγή. Στην περίπτωση 2, ο κόμβος x και ο γονέας y του x είναι κόκκινοι, όπως και ο θείος y' του x (δηλαδή ο αδελφός του y) είναι μαύρος. Στο σχήμα απεικονίζονται οι περιπτώσεις όπου ο y είναι αριστερό παιδί. Οι περιπτώσεις όπου ο y είναι δεξί παιδί είναι συμμετρικές. Μπορούμε να αποκαταστήσουμε την αναλλοίωτη συνθήκη 3 για τους κόμβους x και y με αναχρωματισμό των y και y' σε μαύρους και του γονέα τους z σε κόκκινο, εκτός αν ο z είναι η ρίζα, οπότε παραμένει μαύρος. Ο αναχρωματισμός μπορεί να προκαλέσει νέα παραβίαση της συνθήκης 3 για τον κόμβο z και το γονέα του.



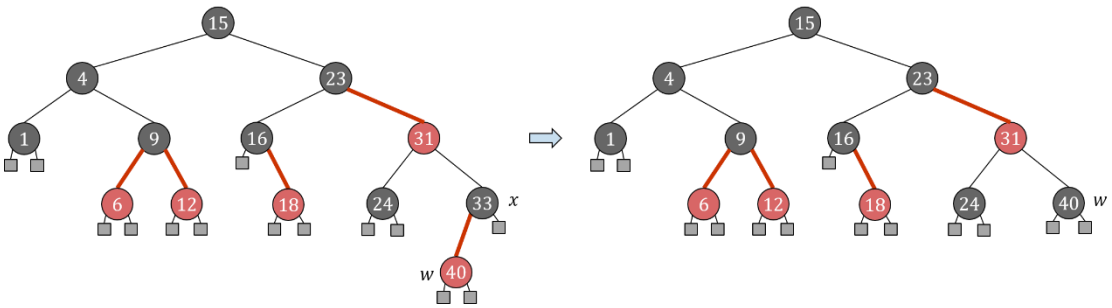


Εικόνα 8.28: Ακολουθία εισαγωγών σε κοκκινόμαυρο δένδρο. Εισάγουμε διαδοχικά τα κλειδιά $2, 17, 48, 20$ και 45 στο κοκκινόμαυρο δένδρο της Εικόνα 8.254. Η εισαγωγή του 2 δεν προκαλεί την παραβίαση των αναλλοίωτων συνθηκών, καθώς ο γονέας του νέου κόμβου είναι μαύρος. Με την εισαγωγή του κόμβου x με κλειδί 17 έχουμε παραβίαση της αναλλοίωτης συνθήκης 3, καθώς ο γονέας y του x είναι και αυτός κόκκινος. Ο θείος y' του x είναι μαύρος, άρα βρισκόμαστε στην περίπτωση 1, υποπερίπτωση ΔΑ. Εκτελούμε μια διπλή αριστερή-δεξιά περιστροφή της τριάδας (z, y, x), όπου z είναι ο γονέας του y με κλειδί 16. Μετά την περιστροφή, ο x γίνεται γονέας των y και z και χρωματίζεται μαύρος, ενώ ο z χρωματίζεται κόκκινος. Αυτές οι πράξεις αποκαθιστούν τις αναλλοίωτες συνθήκες του κοκκινόμαυρου δένδρου. Το επόμενο κλειδί είναι το 28, το οποίο τοποθετείται σε κόμβο με γονέα μαύρο κόμβο, οπότε δεν παραβιάζεται καμία συνθήκη ισορροπίας. Στη συνέχεια, με την εισαγωγή του κόμβου x με κλειδί 20 έχουμε παραβίαση της αναλλοίωτης συνθήκης 3, καθώς ο γονέας

y του x είναι και αυτός κόκκινος. Ο θείος y' του x είναι κόκκινος, άρα βρισκόμαστε στην περίπτωση 2, όπου χρωματίζουμε τον γονέα z του y κόκκινο, ενώ χρωματίζουμε τους y και y' μαύρους. Καθώς ο γονέας του z είναι μαύρος, η διαδικασία αποκατάστασης έχει ολοκληρωθεί. Με την επόμενη εισαγωγή, τοποθετούμε ένα νέο κόμβο x με κλειδί 45 ως αριστερό παιδί του κόκκινου κόμβου y με κλειδί 48. Έτσι, έχουμε και πάλι παραβίαση της αναλλοίωτης συνθήκης 3. Αφού ο θείος y' του x είναι κόκκινος, βρισκόμαστε ξανά στην περίπτωση 2, όπου χρωματίζουμε το γονέα z του y κόκκινο, ενώ χρωματίζουμε τους y και y' μαύρους. Αυτή τη φορά, ο γονέας w του z είναι κόκκινος και, επομένως, η διαδικασία αποκατάστασης πρέπει να συνεχιστεί από τον y. Ο θείος z' του y, με κλειδί 17, είναι κόκκινος, επομένως εκτελούμε την περίπτωση 2 ακόμα μια φορά και χρωματίζουμε τον w κόκκινο και τους z και z' μαύρους. Η διαδικασία αποκατάστασης ολοκληρώνεται, καθώς ο γονέας του w είναι μαύρος.

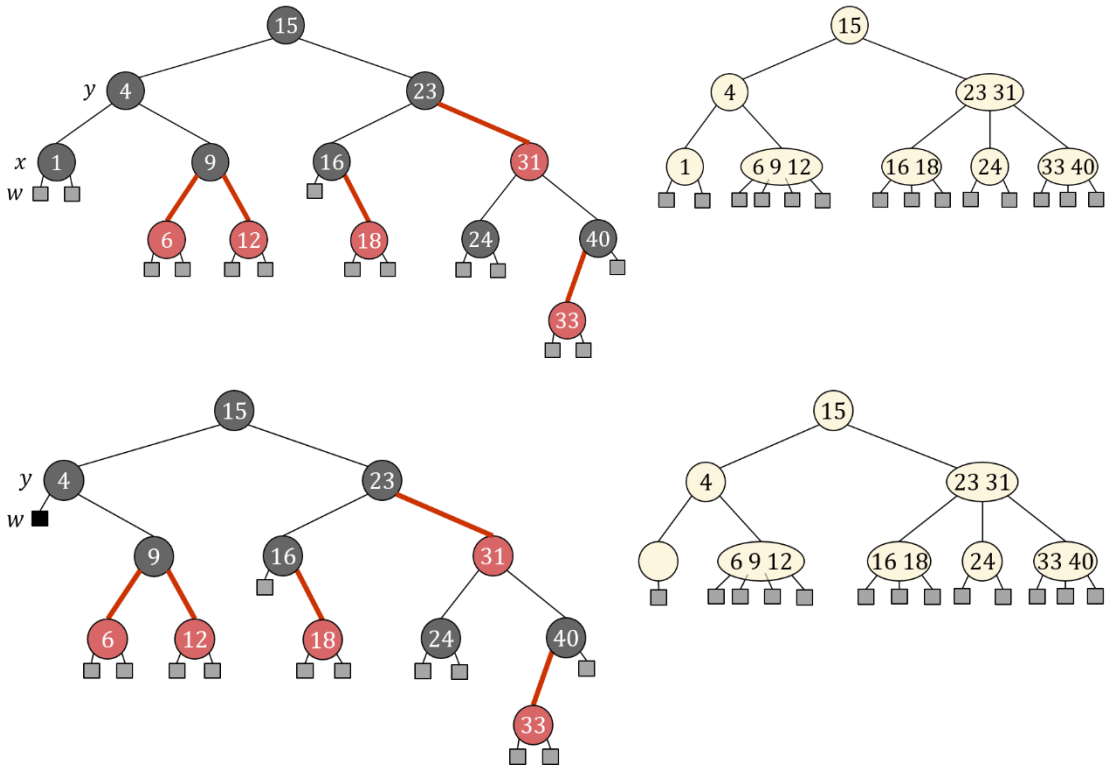
Διαγραφές

Εκτελούμε τον αλγόριθμο διαγραφής κλειδιού σε απλό δυαδικό δένδρο αναζήτησης, που περιγράψαμε στην Ενότητα 7.3.5, ο οποίος αφαιρεί από το δένδρο έναν κόμβο x με το πολύ ένα μη κενό παιδί w. Έστω y ο γονέας του x. Μετά τη διαγραφή, ο w παίρνει τη θέση του x ως παιδί του y. Αν ο x ήταν κόκκινος ή αν ο x ήταν μαύρος αλλά ο w κόκκινος, τότε χρωματίζοντας τον w μαύρο αποκαθιστούμε την αναλλοίωτη συνθήκη 4 και ο αλγόριθμος διαγραφής τερματίζει.



Εικόνα 8.29: Μια απλή περίπτωση διαγραφής σε κοκκινόμαυρο δένδρο. Διαγράφουμε τον μαύρο κόμβο x, ο οποίος έχει μόνο ένα μη κενό παιδί w. Αφού ο w είναι κόκκινος, αρκεί να πάρει τη θέση του x και να χρωματιστεί μαύρος. Αυτό είναι αρκετό, για να αποκατασταθούν οι τις συνθήκες ισορροπίας.

Διαφορετικά, στην περίπτωση που τόσο ο x όσο και ο w είναι μαύροι, θεωρούμε ότι ο w έχει υπερχείλιση μαύρου χρώματος και τον αποκαλούμε *διπλά μαύρο*. Η ύπαρξη ενός διπλά μαύρου κόμβου ερμηνεύεται ως εμφάνιση ενός 1-κόμβου στο αντίστοιχο (2,4)-δένδρο. Επομένως, μπορούμε να χρησιμοποιήσουμε τη μέθοδο αποκατάστασης μετά από διαγραφή σε (2,4)-δένδρο ως οδηγό.

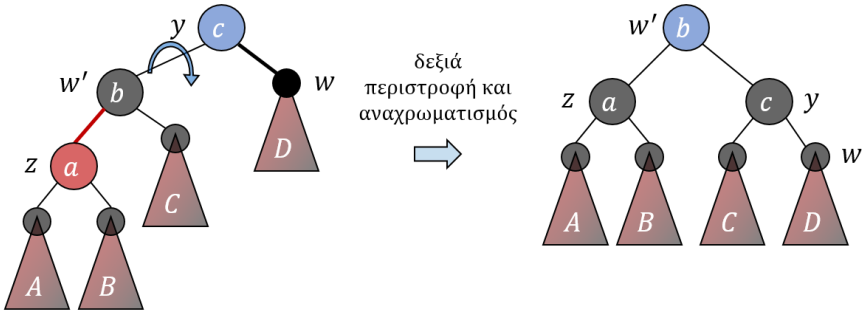


Εικόνα 8.30: Περίπτωση διαγραφής η οποία προκαλεί εμφάνιση ενός διπλά μαύρου κόμβου. Διαγράφουμε τον μαύρο κόμβο x, ο οποίος έχει μόνο ένα μη κενό παιδί w. Αφού ο w είναι και αυτός μαύρος, μετά τη διαγραφή γίνεται διπλά μαύρος κόμβος. Στο αντίστοιχο $(2,4)$-δένδρο κάνει την εμφάνιση του ένας 1-κόμβος.

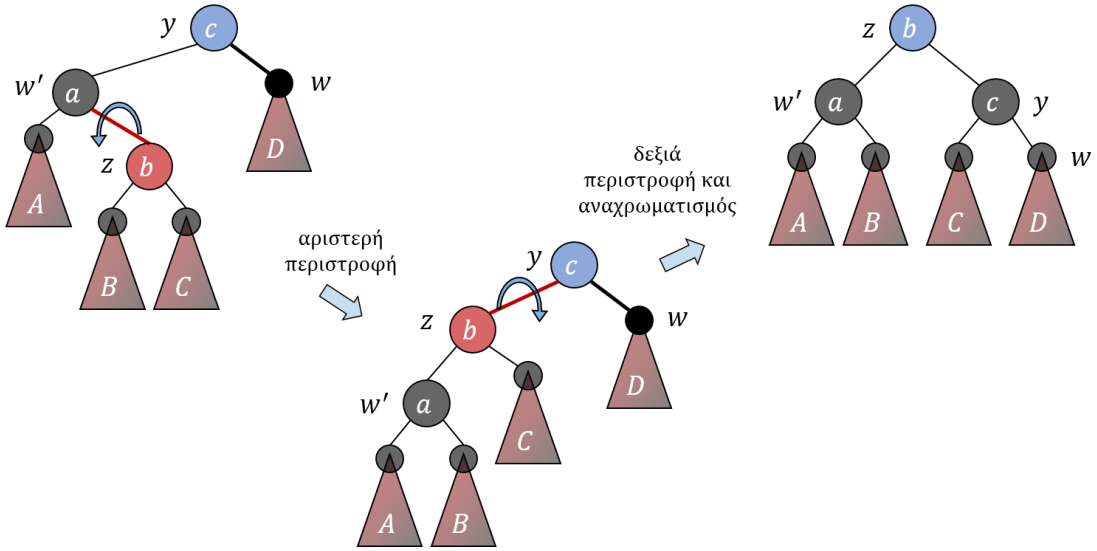
Περίπτωση 1) Ο αδελφός w' του w είναι μαύρος και έχει ένα κόκκινο παιδί z. Στο αντίστοιχο $(2,4)$-δένδρο, ο 1-κόμβος έχει γειτονικό αδελφό με τουλάχιστον δύο κλειδιά, επομένως μπορούμε να εφαρμόσουμε τη μέθοδο της μεταφοράς κλειδιών. Στο κοκκινόμαυρο δένδρο μπορούμε να επιτύχουμε το αντίστοιχο αποτέλεσμα με περιστροφές και αναχρωματισμό των y, w' και z. Συγκεκριμένα, εξετάζουμε τη φορά των συνδέσμων (y, w') και (w', z), και διακρίνουμε τις ίδιες υποπεριπτώσεις ΑΑ, ΑΔ, ΔΑ και ΔΔ με την εισαγωγή. Έτσι, εκτελούμε μια απλή περιστροφή του ζεύγους (y, w'), αν έχουμε την περίπτωση ΑΑ ή ΔΔ, και μια διπλή περιστροφή της τριάδας (y, w', z), αν έχουμε την περίπτωση ΑΔ ή ΔΑ. Έστω a, b και c οι κόμβοι y, w' και z σε αύξουσα σειρά ως προς τα κλειδιά τους. Τότε, ο κόμβος b γίνεται γονέας των άλλων δύο μετά την απλή ή τη διπλή περιστροφή. Χρωματίζουμε τους κόμβους a και c μαύρους και δίνουμε στον κόμβο b το προηγούμενο χρώμα του y. Με τον τρόπο αυτό απορροφούμε το πλεονάζον μαύρο χρώμα και, επομένως, έχουμε αποκαταστήσει τις συνθήκες ισορροπίας.

Περίπτωση 1: Ο αδελφός w' του w είναι μαύρος και έχει ένα κόκκινο παιδί z

Υποπερίπτωση ΑΑ: w' = αριστερό παιδί του y και z = αριστερό παιδί του w'



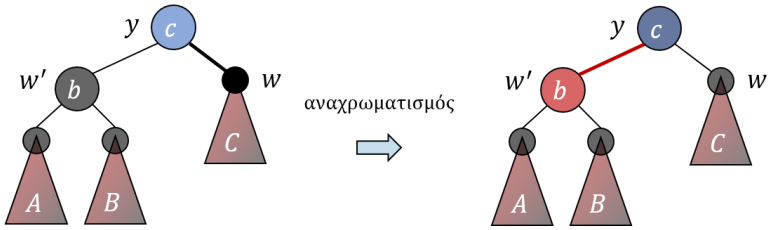
Υποπερίπτωση ΑΔ: w' = αριστερό παιδί του y και z = δεξί παιδί του w'



Εικόνα 8.31: Διαδικασία αποκατάστασης των αναλλοίωτων συνθηκών ενός κοκκινόμαυρου δένδρου μετά από διαγραφή. Ο κόμβος w είναι διπλά μαύρος. Στην περίπτωση 1, ο αδελφός w' του w είναι μαύρος και έχει ένα κόκκινο παιδί z. Αρκεί μια απλή περιστροφή (στην υποπερίπτωση ΑΑ και τη συμμετρική της ΔΔ) ή μια διπλή περιστροφή (στην υποπερίπτωση ΑΔ και τη συμμετρική της ΔΑ) και αναχρωματισμός, για να αποκαταστήσει τις συνθήκες ισορροπίας.

Περίπτωση 2) Ο αδελφός w' του w είναι μαύρος και έχει δύο μαύρα παιδιά. Χρωματίζουμε τον κόμβο w' κόκκινο τον w μαύρο (δηλαδή απομακρύνουμε το πλεονάζον μαύρο χρώμα από τον w). Μεταβιβάζουμε το πλεονάζον μαύρο χρώμα από τον w στον y. Αν ο y ήταν κόκκινος, τότε γίνεται μαύρος, διαφορετικά, αν ήταν μαύρος, γίνεται διπλά μαύρος. Στην τελευταία περίπτωση μεταφέρουμε τη βλάβη ένα επίπεδο προς τα πάνω στο δένδρο.

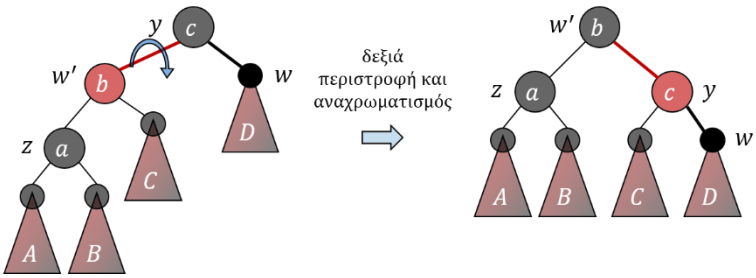
Περίπτωση 2: Ο αδελφός w' του w είναι μαύρος και έχει δύο μαύρα παιδιά



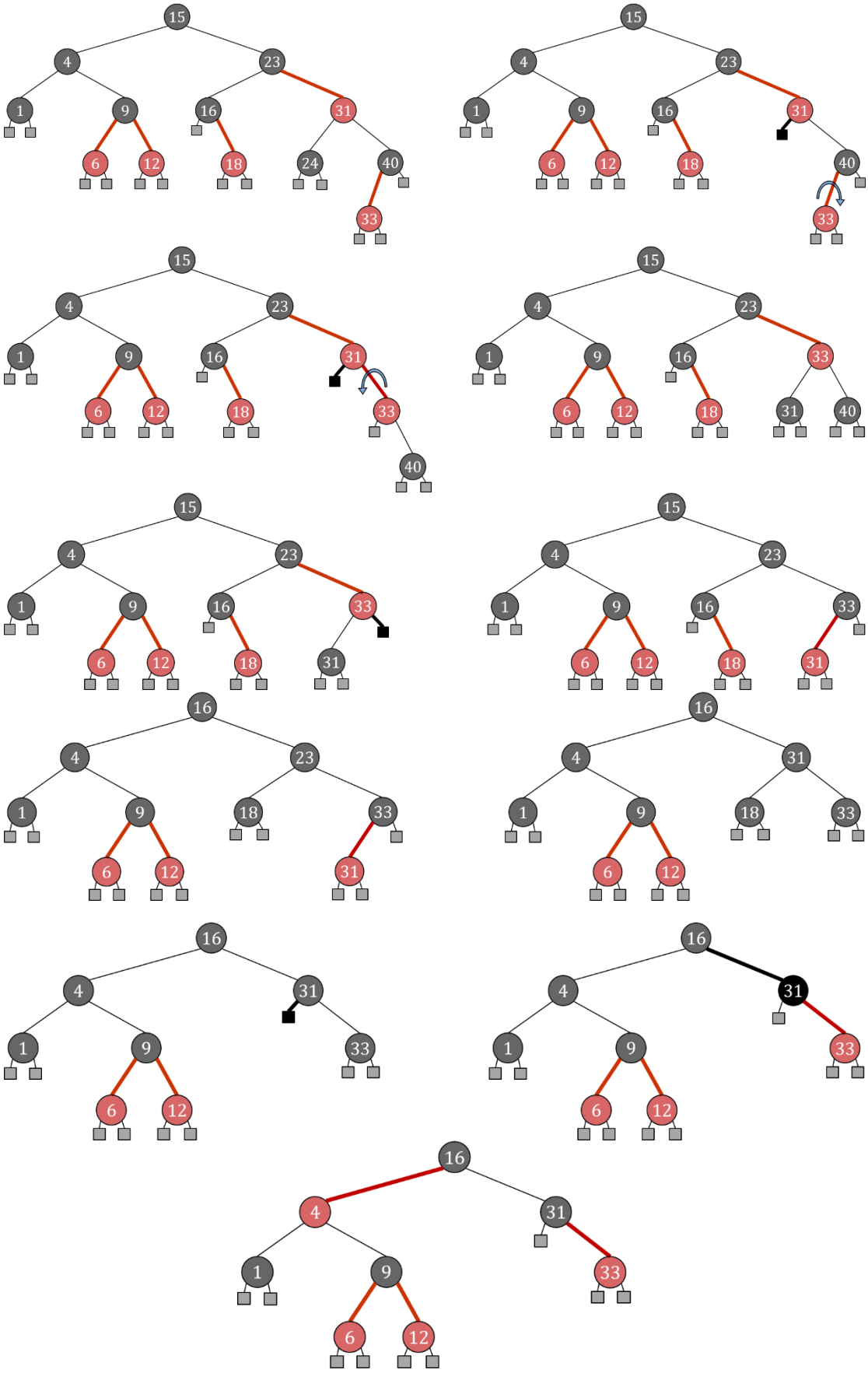
Εικόνα 8.32: Διαδικασία αποκατάστασης των αναλλοίωτων συνθηκών ενός κοκκινόμαυρου δένδρου μετά από διαγραφή. Ο κόμβος w είναι διπλά μαύρος ενώ ο y μπορεί να είναι είτε μαύρος είτε κόκκινος. Στην περίπτωση 2, ο αδελφός w' του w είναι μαύρος και έχει μόνο μαύρα παιδιά. Στο σχήμα απεικονίζεται η περίπτωση όπου ο w είναι δεξί παιδί. Η περίπτωση όπου ο w είναι αριστερό παιδί είναι συμμετρική. Με αναχρωματισμό των w, w' και y μεταβιβάζουμε το πλεονάζον μαύρο χρώμα από τον w στον y. Αν ο y ήταν προηγουμένως κόκκινος, τότε έχουμε αποκαταστήσει τις αναλλοίωτες συνθήκες, διαφορετικά ο y γίνεται διπλά μαύρος και η διαδικασία αποκατάστασης συνεχίζεται με τον y να παίρνει το ρόλο του w.

Περίπτωση 3) Ο αδελφός w' του w είναι κόκκινος. Αυτή η περίπτωση μπορεί να αναχθεί σε μια από τις προηγούμενες περιπτώσεις με μια απλή περιστροφή του ζεύγους (y, w') και αναχρωματισμό. Μετά την περιστροφή, ο w' γίνεται γονέας του y και τον χρωματίζουμε μαύρο, ενώ χρωματίζουμε τον y κόκκινο. Τώρα ο w παραμένει διπλά μαύρος κόμβος, αλλά ο νέος αδελφός του είναι μαύρος, άρα μπορούμε να εφαρμόσουμε μια από τις δύο προηγούμενες περιπτώσεις. Επιπλέον, αν ισχύει η περίπτωση 2, τότε μετά τον αναχρωματισμό του w, του νέου αδελφού του και του y, ο y γίνεται μαύρος και οι συνθήκες ισορροπίας έχουν αποκατασταθεί.

Περίπτωση 3: Ο αδελφός w' του w είναι κόκκινος



Εικόνα 8.33: Διαδικασία αποκατάστασης των αναλλοίωτων συνθηκών ενός κοκκινόμαυρου δένδρου μετά από διαγραφή. Ο κόμβος w είναι διπλά μαύρος. Στην περίπτωση 3, ο αδελφός w' του w είναι κόκκινος. Με μια απλή περιστροφή του ζεύγους (y, w') και αναχρωματισμό αναγόμαστε στην περίπτωση 1 ή στην περίπτωση 2.



Εικόνα 8.34: Ακολουθία διαγραφών στο κοκκινόμαυρο δένδρο της Εικόνα 8.25. Διαγράφουμε διαδοχικά τα κλειδιά 24, 40, 15, 23 και 18. Μετά τη διαγραφή του 24 δημιουργείται ένας διπλά

μαύρος κόμβος w ως αριστερό παιδί του κόμβου y με κλειδί 31. Ο αδελφός w' του w είναι μαύρος και έχει ένα κόκκινο αριστερό παιδί z με κλειδί 33. Επομένως, βρισκόμαστε στην περίπτωση 1, υποπερίπτωση ΑΔ, και εφαρμόζουμε μια διπλή δεξιά-αριστερή περιστροφή της τριάδας (y, w', z). Στη συνέχεια, χρωματίζουμε τον y μαύρο και έτσι απορροφούμε το πλεονάζον μαύρο χρώμα. Με τη διαγραφή του 40 δημιουργείται ένας διπλά μαύρος κόμβος w ως δεξί παιδί του κόμβου y με κλειδί 33. Ο αδελφός w' του w είναι κόκκινος, επομένως εφαρμόζουμε την περίπτωση 2. Χρωματίζουμε τον y μαύρο, απορροφώντας το πλεονάζον μαύρο χρώμα του w και χρωματίζουμε τον w' κόκκινο. Οι αναλλοίωτες συνθήκες έχουν πια αποκατασταθεί. Για τη διαγραφή του 15, διαγράφουμε τον κόμβο x με κλειδί 16, το οποίο είναι το διάδοχο κλειδί του 15, και αντικαθιστούμε στη ρίζα το 15 με το 16. Έπειτα χρωματίζουμε τον κόκκινο παιδί του x, δηλαδή τον κόμβο με κλειδί 18, μαύρο και τον κάνουμε παιδί του γονέα του x. Για τη διαγραφή του 23, διαγράφουμε τον κόμβο x με κλειδί 31, το οποίο είναι το διάδοχο κλειδί του 23, και αντικαθιστούμε στη ρίζα το 23 με το 31. Η διαγραφή του x δεν προκαλεί παραβίαση των αναλλοίωτων συνθηκών, αφού ο x είναι κόκκινος. Τέλος, με τη διαγραφή του 18 δημιουργείται ένας διπλά μαύρος κόμβος w ως αριστερό παιδί του κόμβου y με κλειδί 31. Ο αδελφός w' του w είναι μαύρος και έχει δύο μαύρα παιδιά, οπότε τώρα εφαρμόζουμε την περίπτωση 3. Χρωματίζουμε τον w' κόκκινο και μεταβιβάζουμε το πλεονάζον μαύρο χρώμα από τον w στον y. Τώρα ο y είναι με τη σειρά του διπλά μαύρος και ο αδελφός του y' με κλειδί 4 είναι μαύρος με μαύρα παιδιά. Βρισκόμαστε στην περίπτωση 3, όπου κάνουμε τον y' κόκκινο και μεταβιβάζουμε το πλεονάζον μαύρο χρώμα του y στο γονέα του. Καθώς ο γονέας του y είναι η ρίζα, απορροφά το πλεονάζον μαύρο χρώμα, χωρίς να απαιτηθεί κάποια άλλη πράξη αποκατάστασης.

Ασκήσεις

8.1 Υλοποιήστε σε Java τη λειτουργία ένωσης δύο λεξικών, όταν αυτά αναπαρίστανται με α) δύο μη διατεταγμένες λίστες και β) δύο διατεταγμένες λίστες.

8.2 Περιγράψτε αποδοτικούς αλγόριθμους για την εύρεση του προκάτοχου και του διάδοχου ενός κλειδιού k σε (2,4)-δένδρο.

8.3 Θεωρήστε την παρακάτω λειτουργία σε ένα AVL δένδρο αναζήτησης T :

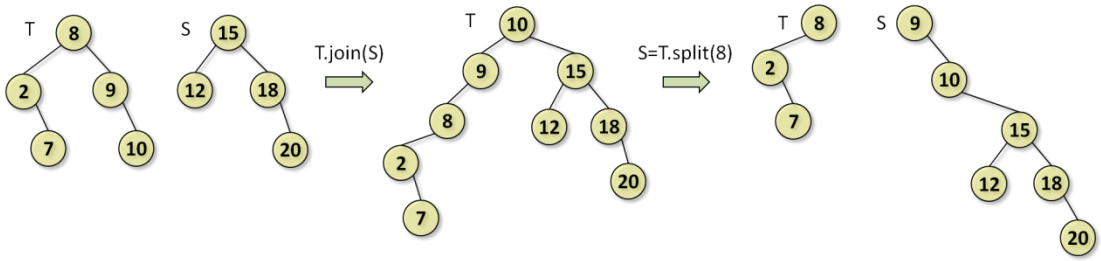
πλήθος(k, m) : Επιστρέφει το πλήθος των στοιχείων του δένδρου με κλειδιά στο διάστημα [k, ... , m].

Περιγράψτε έναν αποδοτικό αλγόριθμο για τη λειτουργία αυτή. Ποιος είναι ο χρόνος εκτέλεσης του αλγόριθμού σας σε ένα AVL δένδρο με n κλειδιά; *Υπόδειξη:* Θεωρήστε ότι κάθε κόμβος του AVL δένδρου αποθηκεύει το πλήθος των απογόνων του. Η πληροφορία αυτή θα πρέπει να ανανεώνεται μετά από κάθε περιστροφή.

8.4 Περιγράψτε αποδοτικούς αλγόριθμους που να υλοποιούν τις παρακάτω δύο λειτουργίες σε ένα αρθρωτό δένδρο T:

void join(SplayTree S) Ενώνει στο T το δένδρο S. Προϋποθέτει ότι τα κλειδιά του T είναι μικρότερα από τα κλειδιά του S. Μετά την ένωση το S είναι κενό. (Δείτε το παρακάτω σχήμα.)

SplayTree split(Key key)	Χωρίζει το T σε δύο αρθρωτά δένδρα T και S. Το T διατηρεί τα στοιχεία με κλειδιά μικρότερα ή ίσα του k και το S περιέχει τα στοιχεία με κλειδιά μεγαλύτερα του k. Επιστρέφει το S. (Δείτε το παρακάτω σχήμα.)



Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

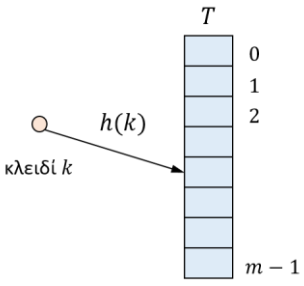
Κεφάλαιο 9
Κατακερματισμός

Περιεχόμενα

9.1 Εισαγωγή 197
9.2 Συναρτήσεις Κατακερματισμού 199
9.3 Επίλυση συγκρούσεων 200
9.3.1 Ξεχωριστές αλυσίδες 200
9.3.2 Μεταβλητές διευθύνσεις 201
Ανάλυση αναμενόμενης περίπτωσης 204
9.3.3 Κατακερματισμός του κούκου 208
9.4 Καθολικές οικογένειες συναρτήσεων κατακερματισμού 209
Ασκήσεις 210
Βιβλιογραφία 212

9.1 Εισαγωγή

Η βασική ιδέα της τεχνικής του κατακερματισμού είναι το ότι ερμηνεύουμε την τιμή ενός κλειδιού ως διεύθυνση σε ένα πίνακα ο οποίος αποθηκεύει τα κλειδιά στις αντίστοιχες θέσεις. Τυπικά, χρησιμοποιούμε ένα πίνακα κατακερματισμού T, χωρητικότητας m θέσεων και μια συνάρτηση κατακερματισμού h, η οποία αντιστοιχεί τα κλειδιά σε θέσεις του πίνακα T, όπως φαίνεται στο διπλανό σχήμα.



Αν το σύνολο όλων των δυνατών κλειδιών (σύμπαν) είναι το Ω, τότε η συνάρτηση κατακερματισμού έχει πεδίο ορισμού το Ω και πεδίο τιμών τους ακέραιους $\{0,1,\dots,m-1\}$, δηλαδή

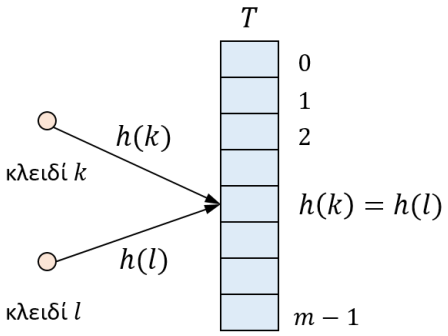
$$h : \Omega \rightarrow \{0,1,\dots,m-1\}$$

Η επιτυχία της μεθόδου καθορίζεται από την επιλογή της συνάρτησης κατακερματισμού. Στο πιο απλό σενάριο, το Ω είναι το υποσύνολο των φυσικών αριθμών $\{0,1,\dots,m-1\}$, οπότε μπορούμε να επιλέξουμε την ταυτοτική συνάρτηση $h(k) = k$ ως συνάρτηση κατακερματισμού. Με τον τρόπο αυτό λαμβάνουμε άμεσα μια πολύ απλή λύση με σταθερό χρόνο εκτέλεσης των λειτουργιών της εισαγωγής, διαγραφής και αναζήτησης κλειδιού.

Το μεγάλο μειονέκτημα της λύσης που μόλις προτείναμε είναι ότι μπορεί να έχει απαγορευτικές απαιτήσεις σε χώρο. Για παράδειγμα, ας υποθέσουμε ότι το Ω περιλαμβάνει τους μη αρνητικούς ακέραιους που μπορούν να αναπαρασταθούν σε έναν υπολογιστή των 64

bit. Τότε θα χρειαζόμασταν να δεσμεύσουμε χώρο $m = 2^{64}$ λέξεων, για να αποθηκεύσουμε ένα σύνολο κλειδιών που μπορεί να είναι πολύ μικρό.

Ας συμβολίσουμε με K το υποσύνολο των κλειδιών που έχουν εισαχθεί στη δομή κατακερματισμού. Από την παραπάνω συζήτηση, κατανοούμε ότι η πρόκληση είναι να σχεδιάσουμε μια δομή κατακερματισμού η οποία να είναι αποδοτική από άποψη χρόνου εκτέλεσης των βασικών λειτουργιών, αλλά, επιπλέον, να καταλαμβάνει χώρο ο οποίος να είναι ανάλογος του μεγέθους του συνόλου K και όχι του σύμπαντος Ω. Σε αυτήν την περίπτωση έχουμε $m < |\Omega|$, που σημαίνει ότι αναπόφευκτα θα έχουμε κλειδιά τα οποία αντιστοιχίζονται στην ίδια θέση. Λέμε ότι δύο κλειδιά k και l **συγκρούονται**, όταν $h(k) = h(l)$, όπως δείχνει η Εικόνα 9.35.



Εικόνα 9.35: Σύγκρουση δύο κλειδιών k και l.

Θα πρέπει, λοιπόν, να καθορίσουμε μια μέθοδο επίλυσης των συγκρούσεων. Οι δομές κατακερματισμού διακρίνονται σε δύο βασικές κατηγορίες, ανάλογα με το πώς διευθετούν τις συγκρούσεις. Δύο βασικές μέθοδοι που θα εξετάσουμε στη συνέχεια είναι οι ξεχωριστές αλυσίδες και οι μεταβλητές διευθύνσεις. Και με τις δύο μεθόδους μπορούμε να επιτύχουμε σταθερό αναμενόμενο χρόνο για εισαγωγή, διαγραφή και αναζήτηση ενός κλειδιού (βλέπε Πίνακας 9.1), υπό την προϋπόθεση ότι η συνάρτηση κατακερματισμού κατανέμει «αρκετά τυχαία» τα κλειδιά που εισάγονται.

Δυστυχώς, η υπόθεση αυτή μπορεί να απέχει πολύ από την πραγματικότητα για ένα δεδομένο σύνολο κλειδιών K και μια δεδομένη συνάρτηση κατακερματισμού h.

Πίνακας 9.1: Χρόνοι εκτέλεσης χειρότερης και αναμενόμενης περίπτωσης μερικών βασικών λειτουργιών μη διατεταγμένου λεξικού με n στοιχεία, υλοποιημένου με κατακερματισμό.

	χειρότερη περίπτωση			αναμενόμενη περίπτωση		
	εισαγωγή	διαγραφή	αναζήτηση	εισαγωγή	διαγραφή	αναζήτηση
ξεχωριστές αλυσίδες	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$
μεταβλητές διευθύνσεις	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$

9.2 Συναρτήσεις Κατακερματισμού

Όπως υπαινιχθήκαμε παραπάνω, μια καλή συνάρτηση κατακερματισμού θα πρέπει να έχει όσο το δυνατό πιο «τυχαιόμορφη» συμπεριφορά, έτσι ώστε να ελαχιστοποιεί την πιθανότητα συγκρούσεων. Από την άλλη, η χρήση μιας πραγματικά τυχαίας συνάρτησης δεν είναι εφικτή, καθώς η συνάρτηση κατακερματισμού θα πρέπει να είναι υπολογίσιμη, ώστε να μπορεί να μας βρει τη θέση στην οποία έχει τοποθετήσει ένα κλειδί. Η θεωρία των αλγορίθμων μάς προσφέρει ορισμένους τρόπους συμβιβασμού αυτών των απαιτήσεων, τους οποίους θα εξετάσουμε στη συνέχεια.

Εδώ θα αναφέρουμε κάποιες επιλογές συναρτήσεων κατακερματισμού που συναντάμε συχνά στην πράξη.

Για αλφαριθμητικά με χαρακτήρες κάθε χαρακτήρας αντιστοιχεί σε ένα ακέραιο σε κωδικοποίηση ASCII, δηλαδή έχουμε 'a' = 97, 'b' = 98, 'c' = 99, κλπ. Μπορούμε, λοιπόν, να μετατρέψουμε ένα αλφαριθμητικό σε ένα ακέραιο στο διάστημα $[0, m-1]$ προσθέτοντας τις ακέραιες τιμές των χαρακτήρων του και επιστρέφοντας το υπόλοιπο της διαίρεσης με το m. Π.χ., για $m = 67$ και το αλφαριθμητικό "have", υπολογίζουμε την τιμή $(104 + 97 + 118 + 101) \bmod 67 = 420 \bmod 67 = 18$.

Η ιδέα αυτή αντιστοιχεί στον ίδιο ακέραιο αλφαριθμητικά που προκύπτουν από μεταθέσεις των ίδιων χαρακτήρων, όπως stop, tops, pots, spot ή γραφή, φραγή. Για να το αποφύγουμε, μπορούμε να πολλαπλασιάσουμε με ένα συντελεστή βάρους w^j σε κάθε θέση j. Στο προηγούμενο παράδειγμα, με m=67 και το αλφαριθμητικό "have", έχουμε την τιμή $(104 \cdot 128^3 + 97 \cdot 128^2 + 118 \cdot 128^1 + 101 \cdot 128^0) \bmod 67 = 219708261 \bmod 67 = 52$. Για μεγάλα αλφαριθμητικά ένας τέτοιος υπολογισμός μπορεί να είναι πιο αργός από το επιθυμητό, ενώ μπορεί να οδηγήσει σε υπερχείλιση. Και τα δύο προβλήματα λύνονται με τη βοήθεια του κανόνα του Horner για τον υπολογισμό της τιμής ενός πολυωνύμου. Συγκεκριμένα, έχουμε $104*128^3 + 97*128^2 + 118*128^1 + 101*128^0 = ((104*128 + 97)*128 + 118)*128 + 101$.

Εκμεταλλευόμενοι τις αριθμητικές ιδιότητες της συνάρτησης υπολοίπου ακέραιας διαίρεσης, καταλήγουμε στον παρακάτω τρόπο υπολογισμού

```
int w = 128;
int k = 0;
for (i=0; i<N; i++){ k = (k*w + s.charAt(i)) % m; }
```

hashCode() της Java

Για τη μετατροπή ενός αυθαίρετου αντικειμένου σε ακέραιο, μπορούμε να χρησιμοποιήσουμε τη μέθοδο hashCode() της Java, οποία επιστρέφει ακέραιο 32-bit. Για να λάβουμε ακέραιο στο διάστημα $[0, m-1]$, μπορούμε να υπολογίσουμε

```
private int hash(Key k)
{
    return ( ( k.hashCode() & 0x7fffffff ) % m );
}
```

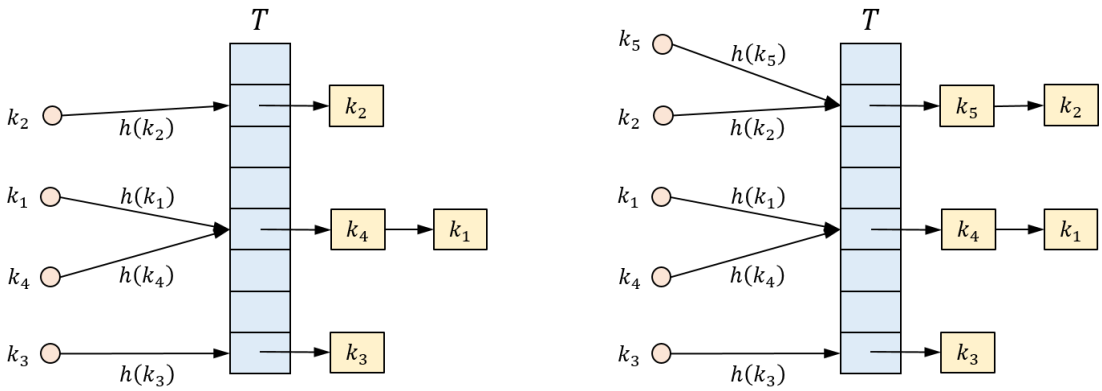
Προσοχή: Η μέθοδος hashCode() μπορεί να μην είναι καλή επιλογή για ορισμένα αντικείμενα. Π.χ. μπορεί να επιστρέφει αναφορά σε θέση μνήμης του αντικειμένου.

9.3 Επίλυση συγκρούσεων

Σε αυτήν την ενότητα θα μελετήσουμε ορισμένες βασικές τεχνικές αντιμετώπισης των συγκρούσεων, οι οποίες εφαρμόζονται συχνά σε πρακτικές εφαρμογές. Θα αναλύσουμε την απόδοση των μεθόδων στην αναμενόμενη περίπτωση ως συνάρτηση του συντελεστή πληρότητας $\lambda = n/m$.

9.3.1 Ξεχωριστές αλυσίδες

Ένας απλός τρόπος χειρισμού των συγκρούσεων είναι να αποθηκεύουμε τα κλειδιά που τοποθετούνται στην ίδια θέση του πίνακα κατακερματισμού σε μία συνδεδεμένη λίστα.



Εικόνα 9.36: Κατακερματισμός με χωριστές αλυσίδες. Ένα νέο κλειδί k_5 τοποθετείται στην αρχή της λίστας στην οποία αναφέρεται η θέση $T[h(k_5)]$.

Αλγόριθμος εισαγωγή (k)

1. Υπολόγισε $i \leftarrow h(k)$
2. Τοποθέτησε το κλειδί k στην αρχή της λίστας $T[i]$.

Έστω ότι υλοποιούμε ένα πίνακα κατακερματισμού μεγέθους $m = 13$ θέσεων, με συνάρτηση κατακερματισμού

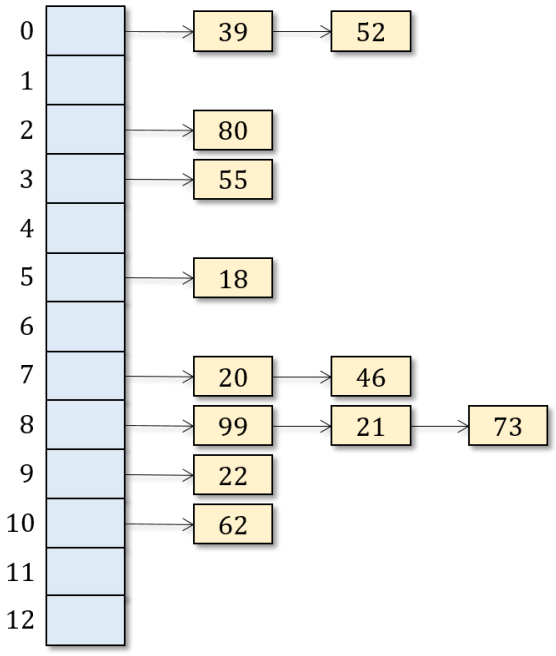
$$h(k) = k \bmod m$$

χρησιμοποιώντας την τεχνική των ξεχωριστών αλυσίδων. Η μορφή που έχει η δομή μετά τη διαδοχική εισαγωγή των κλειδιών

$$52, 46, 62, 39, 73, 21, 80, 18, 20, 22, 99, 55$$

δίνεται στο διπλανό σχήμα. Υπολογίζουμε την τιμή της h:

$h(52) = 0,$ $h(46) = 7,$ $h(62) = 10,$
$h(39) = 0,$ $h(73) = 8,$ $h(21) = 8,$



$h(80) = 2, h(18) = 5, h(20) = 7, h(22) = 9, h(99) = 8, h(55) = 3.$

Αλγόριθμος αναζήτηση(k)

1. Υπολόγισε $i \leftarrow h(k)$
2. Αναζήτησε το κλειδί k στη λίστα $T[i]$.

Αλγόριθμος διαγραφή (k)

1. Υπολόγισε $i \leftarrow h(k)$
2. Αναζήτησε το κλειδί k στη λίστα $T[i]$ και διάγραψέ το, αν βρεθεί στη λίστα.

Ο χρόνος εισαγωγής είναι σταθερός αν υποθέσουμε ότι η συνάρτηση κατακερματισμού h μπορεί να υπολογιστεί σε σταθερό χρόνο. Ο χρόνος εκτέλεσης της αναζήτησης ή της διαγραφής ενός κλειδιού k εξαρτάται από το πλήθος των στοιχείων στη λίστα $T[h(k)]$. Ο αναμενόμενος αριθμός αυτών των στοιχείων είναι ίσος με το συντελεστή πληρότητας λ, επομένως, ο αναμενόμενος χρόνος εκτέλεσης είναι λ. Αν επιλέξουμε το μέγεθος του πίνακα T έτσι ώστε $n = O(m)$, ο αναμενόμενος χρόνος αναζήτησης είναι σταθερός.

9.3.2 Μεταβλητές διευθύνσεις

Ο κατακερματισμός με ξεχωριστές αλυσίδες έχει το μειονέκτημα ότι απαιτεί τη χρήση μιας δευτερεύουσας δομής, μιας συνδεδεμένης λίστας, για την αποθήκευση των κλειδιών που συγκρούονται. Μια εναλλακτική ιδέα είναι να μπορούμε να αποθηκεύσουμε ένα κλειδί σε ένα σύνολο θέσεων του πίνακα κατακερματισμού. Η ιδέα είναι ότι η συνάρτηση διασποράς αντιστοιχεί σε κάθε κλειδί k μία ακολουθία A_k από διευθύνσεις του πίνακα T, την οποία ονομάζουμε *βολιδοσκοπική ακολουθία*. Για να το κάνουμε αυτό, ορίζουμε μια συνάρτηση διασποράς με δύο ορίσματα

$$h : \Omega \times \{0, \dots, m-1\} \rightarrow \{0,1, \dots, m-1\}.$$

Σε μια τέτοια συνάρτηση, το πρώτο όρισμα είναι ένα κλειδί από το σύμπαν Ω και το δεύτερο όρισμα είναι η θέση της βολιδοσκοπικής ακολουθίας. Επομένως, η βολιδοσκοπική ακολουθία ενός κλειδιού k είναι η $A_k = \langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$. Ιδανικά, θα θέλαμε κάθε βολιδοσκοπική ακολουθία A_k να είναι μια μετάθεση των θέσεων $\langle 0, 1, \dots, m-1 \rangle$ του πίνακα κατακερματισμού, έτσι ώστε το κλειδί k να έχει τη δυνατότητα να τοποθετηθεί σε οποιαδήποτε θέση του πίνακα T.

Ο αλγόριθμος εισαγωγής ενός κλειδιού k ελέγχει διαδοχικά τις θέσεις της βολιδοσκοπικής ακολουθίας A_k, μέχρι να βρει την πρώτη κενή θέση και τοποθετεί εκεί το κλειδί.

Αλγόριθμος εισαγωγή (k)

1. $i \leftarrow 0$
2. Ενόσω η θέση $T[h(k,i)]$ είναι κατειλημμένη
3. $\quad i \leftarrow i + 1$
4. Τοποθέτησε $T[h(k,i)] \leftarrow k$.

Αντίστοιχα, ο αλγόριθμος αναζήτησης του κλειδιού k ελέγχει διαδοχικά τις θέσεις της βολιδοσκοπικής ακολουθίας A_k, μέχρι να βρει το k ή μία κενή θέση.

Αλγόριθμος αναζήτηση (k)

1. $i \leftarrow 0$
2. Ενόσω η θέση $T[h(k,i)]$ είναι κατειλημμένη και $T[h(k,i)] \neq k$
3. $\quad i \leftarrow i + 1$
4. Αν $T[h(k,i)] = k$, τότε κλειδί βρέθηκε, διαφορετικά δεν υπάρχει.

Η διαγραφή ενός κλειδιού παρουσιάζει την εξής δυσκολία. Ας θεωρήσουμε ότι διαγράφουμε ένα κλειδί το οποίο βρίσκεται στη θέση $T[r]$. Αν ο πίνακας κατακερματισμού περιέχει κάποιο άλλο κλειδί k το οποίο έχει αποθηκευτεί σε μια θέση $T[h(k,i)]$, έτσι ώστε $h(k,j) = r$ για κάποιο $j < i$, τότε ο παραπάνω αλγόριθμος αναζήτησης θα αποτύχει να βρει το k. Ένας τρόπος αντιμετώπισης του προβλήματος είναι να χρησιμοποιήσουμε ένα βοηθητικό πίνακα Δ, με ίδιο αριθμό θέσεων με τον πίνακα κατακερματισμού T, στον οποίον επισημαίνουμε τις θέσεις του Τ που είναι κενές, επειδή έχει διαγραφεί το κλειδί που αποθήκευαν. Αρχικά, πριν από την εισαγωγή οποιουδήποτε κλειδιού, έχουμε $\Delta[i] = 0$ για κάθε i. Στο προηγούμενο παράδειγμα, μετά τη διαγραφή του κλειδιού της θέσης $T[r]$, θα θέσουμε $\Delta[r] = 1$. Έτσι, κατά την αναζήτηση του κλειδιού k, όταν ο αλγόριθμος αναζήτησης βολιδοσκοπήσει τη θέση $h(k,j) = r$, θα ελέγξει ότι ισχύει $\Delta[r] = 1$ και θα συνεχίσει την αναζήτηση του κλειδιού k στην επόμενη θέση $h(k,j+1)$ της βολιδοσκοπικής ακολουθίας. Για τον αλγόριθμο εισαγωγής, από την άλλη πλευρά, δεν απαιτείται κάποια τροποποίηση, αφού οι θέσεις του πίνακα κατακερματισμού από τις οποίες διαγράφηκε κάποιο κλειδί είναι διαθέσιμες για την τοποθέτηση ενός νέου κλειδιού.

Γραμμική βολιδοσκόπηση

Η πιο απλή εφαρμογή της μεθόδου των ανοικτών διευθύνσεων είναι να δοκιμάζουμε διαδοχικά τις επόμενες θέσεις του πίνακα, μέχρι να βρεθεί μια κενή θέση. Δηλαδή, θέτουμε ως συνάρτηση κατακερματισμού

$$h(k,i) = (f(k) + i) \bmod m.$$

Η γραμμική βολιδοσκόπηση δίνει μια αρκετά απλή υλοποίηση του κατακερματισμού μεταβλητών διευθύνσεων αλλά πάσχει από δύο σημαντικά μειονεκτήματα. Πρώτον, δίνει μόνο m διαφορετικές βολιδοσκοπικές ακολουθίες, $\langle 0, 1, 2, \ldots, m-1 \rangle$, $\langle 1, 2, \ldots, m-1, 0 \rangle$, …, $\langle m-1, 0, 1, \ldots, m-2 \rangle$. Δεύτερον, τείνει να τοποθετεί τα κλειδιά σε μακριές αλληλουχίες διαδοχικών θέσεων του πίνακα κατακερματισμού. Το φαινόμενο αυτό είναι γνωστό ως **πρωτεύουσα ομαδοποίηση** και έχει ως συνέπεια την υποβάθμιση της απόδοσης.

Έστω ότι υλοποιούμε ένα πίνακα κατακερματισμού μεγέθους $m = 13$ θέσεων, με συνάρτηση κατακερματισμού

$$h(k,i) = (f(k) + i) \bmod m$$

όπου $f(k) = k \bmod m$. Η μορφή που έχει η δομή μετά τη διαδοχική εισαγωγή των κλειδιών

$$52, 46, 62, 39, 73, 21, 80, 18, 20, 22, 99, 55$$

δίνεται στο διπλανό σχήμα.

0	52
1	39
2	80
3	99
4	55
5	18
6	
7	46
8	73
9	21
10	62
11	20
12	22

Η ακολουθία βολιδοσκοπήσεων για κάθε κλειδί (μέχρι να εισαχθεί σε κενή θέση) έχει ως εξής: 52→0, 46→7, 62→10, 39→0,1, 73→8, 21→8,9, 80→2, 18→5, 20→7,8,9,10,11, 22→9,10,11,12, 99→8,9,10,11,12,0,1,2,3, 55→3,4.

Τετραγωνική βολιδοσκόπηση

Προκειμένου να αποφύγουμε το φαινόμενο της πρωτεύουσας ομαδοποίησης, θα πρέπει να ορίσουμε μια συνάρτηση κατακερματισμού η οποία να πραγματοποιεί άλματα από θέση σε θέση του πίνακα κατακερματισμού. Ένας τρόπος να το επιτύχουμε αυτό το στόχο είναι να εισαγάγουμε τετραγωνικούς όρους στη συνάρτηση κατακερματισμού. Για παράδειγμα, μπορούμε να επιλέξουμε ως συνάρτηση κατακερματισμού

$$h(k,i) = (f(k) + i^2) \bmod m,$$

όπου $f(k)$ μια βοηθητική συνάρτηση κατακερματισμού. Πιο γενικά, μπορούμε να έχουμε μια συνάρτηση της μορφής $h(k,i) = (f(k) + r(i)) \bmod m$, όπου $r(i) = ai^2 + bi$ για κάποιες σταθερές $a \neq 0$ και b. Χάρη στα άλματα που πραγματοποιούνται από τους όρους $r(i)$, η μέθοδος αυτή αποφεύγει την πρωτεύουσα ομαδοποίηση της γραμμικής βολιδοσκόπησης. Ωστόσο, η ακολουθία των αλμάτων $r(i)$ είναι μοναδική και, επομένως, όμοια με τη γραμμική βολιδοσκόπηση, η τετραγωνική βολιδοσκόπηση δίνει μόνο m διαφορετικές βολιδοσκοπικές. Το γεγονός αυτό έχει ως συνέπεια η τετραγωνική βολιδοσκόπηση να εμφανίζει μια ηπιότερη μορφή ομαδοποίησης, γνωστή ως **δευτερεύουσα ομαδοποίηση**. Επιπλέον, η βολιδοσκοπική ακολουθία μπορεί να μην καλύπτει όλες τις θέσεις του πίνακα (σε αντίθεση με τη γραμμική βολιδοσκόπηση), με αποτέλεσμα ο αλγόριθμος εισαγωγής να μην μπορεί να βρει κενή θέση.

Στο προηγούμενο παράδειγμα, όπου εισάγουμε τα κλειδιά

$$52, 46, 62, 39, 73, 21, 80, 18, 20, 22, 99, 55$$

σε ένα πίνακα κατακερματισμού μεγέθους $m = 13$ θέσεων, αλλά αυτή τη φορά με συνάρτηση κατακερματισμού $h(k,i) = (f(k) + i + i^2) \bmod m$. Η ακολουθία βολιδοσκοπήσεων για κάθε κλειδί (μέχρι να εισαχθεί σε κενή θέση) έχει ως εξής:

52→0, 46→7, 62→10, 39→0,2, 73→8, 21→8,10,1, 80→2,4, 18→5, 20→7,9, 22→9,11, 99→8,10,1,7,2,12, 55→3.

0	52
1	21
2	39
3	55
4	80
5	18
6	
7	46
8	73
9	20
10	62
11	22
12	99

Διπλός κατακερματισμός

Εδώ χρησιμοποιούμε δύο βοηθητικές συναρτήσεις κατακερματισμού $f(k)$ και $g(k)$ και θέτουμε

$$h(k,i) = (f(k) + ig(k)) \bmod m.$$

Με αυτόν τον τρόπο αποφεύγουμε την ομαδοποίηση που εμφανίζουν τόσο η γραμμική όσο και η τετραγωνική βολιδοσκόπηση. Ένα σημείο που πρέπει να προσέξουμε στην επιλογή της $g(k)$ είναι ότι δεν πρέπει να λαμβάνει την τιμή μηδέν για κανένα κλειδί. Έτσι, για παράδειγμα, μπορούμε να θέσουμε $g(k) = 1 + (k \bmod m - 1)$.

Ας εφαρμόσουμε τώρα στο παράδειγμα μας διπλό κατακερματισμό με συνάρτηση $h(k,i) = \big(f(k) + ig(k)\big) \bmod m$, όπου $g(x) = 1 + (k \bmod (m-1))$. Υπολογίζουμε την τιμή της $g(k)$:

$g(52) = 5,\ g(46) = 11,\ g(62) = 3,\ g(39) = 4,\ g(73) = 2,\ g(21) = 10,$
$g(80) = 9, g(18) = 7, g(20) = 9, g(22) = 11, g(99) = 4, g(55) = 8.$

Η ακολουθία βολιδοσκοπήσεων για κάθε κλειδί (μέχρι να εισαχθεί σε κενή θέση)έχει ως εξής:

52→0, 46→7, 62→10, 39→0,4, 73→8, 21→8,5, 80→2, 18→5,12, 20→7,3, 22→9, 99→8,12,3,7,11, 55→3,11,6.

0	52
1	
2	80
3	20
4	39
5	21
6	55
7	46
8	73
9	22
10	62
11	99
12	18

Ανάλυση αναμενόμενης περίπτωσης

Αναλύουμε την απόδοση του κατακερματισμού μεταβλητών διευθύνσεων, θεωρώντας ότι ισχύει η υπόθεση ομοιόμορφης διασποράς. Η πιθανότητα η j-οστή βολιδοσκόπηση να βρει κατειλημμένη θέση δεδομένου ότι οι πρώτες $(j-1)$ βολιδοσκοπήσεις βρήκαν κατειλημμένες θέσεις είναι

$$\frac{n-(j-1)}{m-(j-1)}$$

Άρα η πιθανότητα να χρειαστούν τουλάχιστον i βολιδοσκοπήσεις είναι

$$\frac{n}{m}\cdot\frac{n-1}{m-1}\cdot\frac{n-2}{m-2}\cdots\frac{n-(i-2)}{m-(i-2)} \le \left(\frac{n}{m}\right)^{i-1} = \lambda^{i-1}$$

Η αναμενόμενη τιμή είναι

$$\sum_{i=1}^{\infty}\lambda^{i-1} = \frac{1}{1-\lambda}$$

Το αναμενόμενο πλήθος προσπελάσεων σε επιτυχημένη αναζήτηση είναι ίσο με το αναμενόμενο πλήθος προσπελάσεων για την εισαγωγή καθενός από τα n κλειδιά. Το αναμενόμενο πλήθος προσπελάσεων για την εισαγωγή του i-οστού κλειδιού είναι ίσο με το αναμενόμενο πλήθος προσπελάσεων σε ανεπιτυχή αναζήτηση, όταν έχουν ήδη εισαχθεί $i-1$ κλειδιά.

Επομένως, έχουμε

$$\frac{1}{n}\sum_{i=0}^{n-1}\frac{1}{1-\frac{i}{m}} = \frac{m}{n}\sum_{i=0}^{n-1}\frac{1}{m-i} = \frac{m}{n}\left(\sum_{i=1}^{m}\frac{1}{i} - \sum_{i=1}^{m-n}\frac{1}{i}\right) = \frac{m}{n}(H_m - H_{m-n})$$

όπου για θετικό ακέραιο N

$$H_N = 1 + \frac{1}{2} + \frac{1}{3} + \cdots \frac{1}{N}$$

ο N-οστός αρμονικός αριθμός. Χρησιμοποιώντας την προσέγγιση $H_N \approx \ln N$ λαμβάνουμε

$$\frac{m}{n}(\ln m - \ln(m-n)) = \frac{m}{n} \ln \frac{m}{m-n} = \frac{1}{\lambda} \ln \frac{1}{1-\lambda}$$

Υλοποίηση σε Java

Το πρόγραμμα HashTable.java υλοποιεί τη μέθοδο μεταβλητών διευθύνσεων με διπλό κατακερματισμό, για αντικείμενα γενικού τύπου Item. Τα αντικείμενα αποθηκεύονται στον πίνακα T[0: m − 1] τύπου Item. Χρησιμοποιούμε επίσης ένα πίνακα ακέραιων count[0: m − 1] ο οποίος μετράει σε κάθε θέση j πόσες φορές έχει εισαχθεί το αντικείμενο T[j].

Η θέση ενός αντικειμένου item στον πίνακα κατακερματισμού υπολογίζεται με τη συνάρτηση κατακερματισμού

$$h(k,i) = \big(h_1(k) + i \cdot h_2(k)\big) \bmod m$$

όπου k = hash(item) = (item.hashCode() & 0x7fffffff), $h_1(k) = k \bmod m$ και $h_2(k) = 1 + (k \bmod m - 1)$.

```
public class HashTable<Item> {

    private int m;       // μέγεθος πίνακα κατακερματισμού
    private Item[] T;    // πίνακας κατακερματισμού
    private int[] count; // count[j] = πλήθος εμφανίσεων του στοιχείου T[j]
    private int n;       // πλήθος μη κενών θέσεων

    private int collisions; // πλήθος συγκρούσεων

    public int collisions() { return collisions; }

    private int h1(int k)
    {
        return (k % m);
    }
    private int h2(int k)
    {
        //return ( 1 + (k % (m-1)) );
        return 1;
    }
    private int h(int k, int i)
    {
        return ( (h1(k) + i*h2(k)) % m );
    }
    private int hash(Item item)
    {
        return ( item.hashCode() & 0x7fffffff );
    }
```

```
HashTable2(int M)
{
    m = M;
    n = 0;
    T = (Item []) new Object[m];
    count = new int[m];
}

// αντιγραφή σε νέο πίνακα μεγέθους M
private void resize(int M) {
    System.out.println("resize " + M);
    Item[] tempT = (Item[]) new Object[M];
    int[] tempCount = new int[M];
    m = M;
    for (int l = 0; l < T.length; l++) {
        Item item = T[l];
        int c = count[l];
        if (item != null) {
            int k = hash(item);
            int i = 0;
            int j = h(k,i);
            while (tempT[j] != null)
            {
                collisions++;
                i++;
                j = h(k,i);
            }

            tempT[j] = item;
            tempCount[j] = c;
        }
    }
    T = tempT;
    count = tempCount;
}

// συντελεστής πληρότητας
public double loadFactor()
{
    return (double) 100*n/m;
}

public void insert(Item item)
{
    //System.out.println("insert " + item);
    int k = hash(item);
    int i = 0;

    int j = h(k,i);
    while (T[j] != null)
    {
        if (T[j].equals(item)) {
            //System.out.println("found " + T[j]);
            count[j]++; return;
        }
        collisions++;
        i++;
        j = h(k,i);
```

```
        }

        T[j] = item;
        count[j] = 1;
        n++;

        if ( loadFactor() > 70 ) { resize(2*m); }
    }

    // βρίσκει το αντικείμενο με τις περισσότερες εμφανίσεις
    public void findMax()
    {
        int max = count[0];
        int pos = 0;
        for (int l = 1; l < T.length; l++) {
            if (count[l] > max) {
                max = count[l];
                pos = l;
            }
        }
        System.out.println("max count : " + T[pos] + ", " + count[pos]);
    }

    public int contains(Item item)
    {
        int k = hash(item);
        int i = 0;
        int j = h(k,i);
        while ( T[j] != null )
        {
            if ( (T[j] != null) && ( T[j].equals(item) ) ) return count[j];
            i++;
            j = h(k,i);
        }
        return 0;
    }
}
```

Έστω αντικείμενο item με hash(item) $= k$. Για την εισαγωγή του item βολιδοσκοπούμε τις θέσεις $h(k, 0)$, $h(k, 1)$, $h(k, 2)$,…, του πίνακα κατακερματισμού T μέχρι να βρούμε την πρώτη θέση $j = h(k, i)$ τέτοια ώστε να ισχύει η συνθήκη (T[j]==null) || (item.equals(T[j])).

Στην πρώτη περίπτωση (T[j]==null), η θέση j είναι κενή και επομένως το αντικείμενο item δεν είχε εισαχθεί προηγουμένως. Τότε τοποθετούμε το στοιχείο item στη θέση j του πίνακα T, δηλαδή θέτουμε T[j] = item και count[j]=1. Αν ο συντελεστής πληρότητας του T είναι μεγαλύτερος από 70%, τότε δημιουργούμε ένα νέο πίνακα tempT με διπλάσιο μέγεθος και τοποθετούμε εκεί όλα τα αντικείμενα του T. Δηλαδή εισάγουμε τα αντικείμενα του T στην κατάλληλη θέση του tempT, χρησιμοποιώντας τη συνάρτηση κατακερματισμού $h(k, i)$ αλλά με διπλάσιο m.

Στη δεύτερη περίπτωση (item.equals(T[j])), το αντικείμενο item έχει ήδη εισαχθεί στον πίνακα, επομένως αρκεί να αυξήσουμε τον μετρητή count[j] που δίνει τον αριθμό των εμφανίσεων του item.

Για την αναζήτηση του item, βολιδοσκοπούμε τις θέσεις $h(k,0)$, $h(k,1)$, $h(k,2)$,…, του πίνακα κατακερματισμού T μέχρι να βρούμε την πρώτη θέση $j = h(k,i)$ όπου είτε α) $\mathrm{T[j].equals(item)} == \mathrm{true}$, οπότε η αναζήτηση είναι επιτυχής, είτε β) $\mathrm{T[j]} == \mathrm{null}$ οπότε η αναζήτηση είναι ανεπιτυχής.

9.3.3 Κατακερματισμός του κούκου

Θα περιγράψουμε τώρα μια διαφορετική προσέγγιση επίλυσης συγκρούσεων, η οποία μπορεί να θεωρηθεί ως μια παραλλαγή του κατακερματισμού μεταβλητών διευθύνσεων. Χρησιμοποιούμε δύο πίνακες κατακερματισμού T_1 και T_2, ίσου μεγέθους m, με αντίστοιχες συναρτήσεις κατακερματισμού h_1 και h_2. Σε αντίθεση με τη μέθοδο των μεταβλητών διευθύνσεων, ο κατακερματισμός του κούκου τοποθετεί ένα κλειδί σε μια από δύο δυνατές θέσεις. Συγκεκριμένα, η μέθοδος διατηρεί την αναλλοίωτη συνθήκη ότι κάθε κλειδί k που έχει εισαχθεί βρίσκεται είτε στη θέση $T_1[h_1(k)]$ είτε στη θέση $T_2[h_2(k)]$. Επομένως, η αναζήτηση και η διαγραφή ενός κλειδιού γίνονται άμεσα σε σταθερό χρόνο.

Αλγόριθμος αναζήτηση (k)

1. Αν $T_1[h_1(k)] = k$ ή $T_2[h_2(k)] = k$ τότε κλειδί βρέθηκε, διαφορετικά δεν υπάρχει.

Αλγόριθμος διαγραφή (k)

1. Αν $T_1[h_1(k)] = k$, τότε διάγραψε το κλειδί k από τη θέση $T_1[h_1(k)]$.
2. Διαφορετικά, αν $T_2[h_2(k)] = k$, τότε διάγραψε το κλειδί k από τη θέση $T_2[h_2(k)]$.

Η πολυπλοκότητα αυτής της μεθόδου έγκειται στη διαδικασία εισαγωγής, από την οποία λαμβάνει το όνομα της. Η εισαγωγή ενός νέου κλειδιού k γίνεται άμεσα όταν μια από τις θέσεις $T_1[h_1(k)]$, $T_2[h_2(k)]$ είναι κενή. Στην αντίθετη περίπτωση, προκειμένου να διατηρήσουμε την αναλλοίωτη συνθήκη, πρέπει να απομακρύνουμε ένα από τα κλειδιά που βρίσκονται στις θέσεις $T_1[h_1(k)]$ και $T_2[h_2(k)]$. Το κλειδί αυτό επανεισάγεται, ακολουθώντας παρόμοια διαδικασία.

Αλγόριθμος εισαγωγή (k)

1. Αν κάποια από τις θέσεις $T_1[h_1(k)]$ και $T_2[h_2(k)]$ είναι κενή, τοποθετούμε εκεί το k.
2. Διαφορετικά, έστω ότι $T_1[h_1(k)] = y$. Απομακρύνουμε το κλειδί y από την τρέχουσα θέση του και τοποθετούμε το k στη θέση αυτή.
3. Όταν απομακρύνουμε ένα κλειδί z από τη θέση $T_j[h_j(z)]$ (για κάποιο $j \in \{1,2\}$), τοποθετούμε το z στη θέση $T_{3-j}[h_{3-j}(z)]$, απομακρύνοντας ενδεχομένως κάποιο άλλο κλειδί.
4. Αν συμβούν M απομακρύνσεις κλειδιών, επιλέγουμε διαφορετικές συναρτήσεις κατακερματισμού και τοποθετούμε από την αρχή όλα τα αντικείμενα.

Ένα λεπτό σημείο αυτής της μεθόδου είναι ότι η εισαγωγή μπορεί να προκαλέσει μια αλυσίδα από διαδοχικές απομακρύνσεις κλειδιών. Για να αποφύγουμε το ενδεχόμενο να εισέλθουμε σε ατέρμονα βρόχο, θέτουμε ένα άνω όριο M στο πλήθος των απομακρύνσεων κλειδιών που μπορεί να προκαλέσει μια εισαγωγή.

Η ανάλυση της μεθόδου κατακερματισμού του κούκου είναι αρκετά περίπλοκη και ξεφεύγει από τους σκοπούς του συγγράμματος. Επιγραμματικά αναφέρουμε, ότι για να έχουμε καλή

απόδοση στις εισαγωγές, θα πρέπει ο συντελεστής πληρότητας να μην ξεπερνά το 50%. Επίσης, μια καλή επιλογή για την τιμή του M είναι της τάξεως του $O(\log n)$. Τότε, υπό κάποιες σχετικά ήπιες προϋποθέσεις, αποδεικνύεται ότι ο αναμενόμενος χρόνος εισαγωγής είναι σταθερός.

9.4 Καθολικές οικογένειες συναρτήσεων κατακερματισμού

Για κάθε καθορισμένη συνάρτηση διασποράς μπορούμε να επιλέξουμε κλειδιά που θα δίνουν τη χειρότερη δυνατή επίδοση. Προκειμένου να βελτιώσουμε την κατάσταση, μπορούμε να βασιστούμε σε τυχαιοκρατικούς αλγόριθμους. Ορίζουμε μία οικογένεια συναρτήσεων και κατά τη διάρκεια της εκτέλεσης επιλέγουμε τυχαία μία από αυτές ως συνάρτηση διασποράς. Έτσι, περιορίζουμε την πιθανότητα εμφάνισης παθολογικών περιπτώσεων.

Έστω $\mathcal{H}$ μια οικογένεια (συλλογή) συναρτήσεων διασποράς $h : \Omega \to \{0,1,\dots,m-1\}$. Η οικογένεια $\mathcal{H}$ ονομάζεται *καθολική*, αν για κάθε ζεύγος διαφορετικών κλειδιών k και l, υπάρχουν το πολύ $|\mathcal{H}|/m$ συναρτήσεις $h \in \mathcal{H}$, τέτοιες ώστε $h(k) = h(l)$. Αυτό σημαίνει ότι για τυχαία επιλεγμένη συνάρτηση $h \in \mathcal{H}$, η πιθανότητα σύγκρουσης δύο δεδομένων (διαφορετικών) κλειδιών k και l είναι $\Pr_{h\in\mathcal{H}}[h(k) = h(l)] \le \frac{1}{m}$. Παρατηρούμε ότι η τιμή $1/m$ δίνει την πιθανότητα σύγκρουσης των κλειδιών k και l, όταν επιλέγουμε τη θέση τους στον πίνακα κατακερματισμού τυχαία και ανεξάρτητα.

Θα δώσουμε ένα παράδειγμα καθολικής οικογένειας συναρτήσεων διασποράς $\mathcal{H}$ υποθέτοντας ότι :

- Το σύνολο των πιθανών κλειδιών περιλαμβάνει $|U| = 2^{\upsilon}$ τιμές.
- Οι συναρτήσεις $h \in \mathcal{H}$ αντιστοιχούν το U σε ένα σύνολο με $m = 2^{\mu}$ τιμές.

Άρα, ουσιαστικά οι συναρτήσεις της $\mathcal{H}$ αντιστοιχούν διανύσματα x (ακολουθίες) των υ bits σε διανύσματα y των μ bits. Μπορούμε να λάβουμε μια τέτοια αντιστοιχία πολλαπλασιάζοντας το x με ένα $\mu \times \upsilon$ πίνακα A, δηλαδή έχουμε $y = h(x) = Ax$, όπου οι πράξεις γίνονται modulo 2 (δηλαδή $0 + 0 = 1 + 1 = 0$ και $0 + 1 = 1 + 0 = 1$).

Η οικογένεια H ορίζεται από όλους τους $2^{\mu\upsilon}$ Boolean $\mu \times \upsilon$ πίνακες A.

Έστω A ένας τυχαίος Boolean $\mu \times \upsilon$ πίνακας και έστω $x \neq y$ διανύσματα του $\{0,1\}^{\upsilon}$. Θα δείξουμε ότι η πιθανότητα σύγκρουσης $h(x) = h(y)$ είναι το πολύ $1/m$, δηλαδή $\Pr_{h\in H}[h(x) = h(y)] \le 1/m$.

Έστω $h(x) = h(y)$. Θέτουμε $z = x - y$. Έχουμε $Ax = Ay \Rightarrow A(x - y) = \mathbf{0} \Rightarrow Az = \mathbf{0}$, όπου $\mathbf{0} = (0\ 0\ \dots 0)$ το διάνυσμα του $\{0,1\}^{\mu}$ με όλες τις μ συνιστώσες μηδέν. Άρα, θέλουμε να δείξουμε ότι $\Pr_{h\in H}[h(z) = \mathbf{0}] = \Pr_{h\in H}[Az = \mathbf{0}] \le 1/m$.

Έστω $q = (q_1, q_2, \dots, q_{\mu}) = Az$. Η συνιστώσα q_i προκύπτει από το εσωτερικό γινόμενο της γραμμής i του A με το διάνυσμα z. Εφόσον $z = x - y$ και $x \neq y$, το z έχει τουλάχιστον μία μη μηδενική συνιστώσα, έστω $z_k = 1$. Έχουμε

$$q_i = \sum_{j=1}^{\upsilon} A_{ij} z_j = A_{ik} z_k + \sum_{j\neq k} A_{ij} z_j = A_{ik} + \sum_{j\neq k} A_{ij} z_j .$$

Επομένως, $q_i = 0 \Rightarrow A_{ik} = \sum_{j\neq k} A_{ij} z_j$.

Με τι πιθανότητα μπορεί να συμβεί αυτό; Αφού ο πίνακας A είναι τυχαίος, κάθε στοιχείο του επιλέγεται ανεξάρτητα. Επομένως, μπορούμε να υποθέσουμε ότι η τιμή (0 ή 1) του στοιχείου A_{ik} επιλέγεται τελευταία. Τη στιγμή που επιλέγουμε αυτήν την τιμή, το άθροισμα

$$c = \sum_{j \neq k} A_{ij} z_j$$

είναι καθορισμένο, δηλαδή είναι μια σταθερά $c \in \{0,1\}$. Συνεπώς η πιθανότητα να επιλέξουμε $A_{ik} = c$ είναι $1/2$. Το ίδιο ισχύει για κάθε συνιστώσα $i \in \{1,2, \dots, \mu\}$, δηλαδή

$$\Pr_{h \in H}[h(x) = h(y)] = \Pr_{h \in H}[Az = \mathbf{0}] = \left(\frac{1}{2}\right)^{\mu} = \frac{1}{m}.$$

Η παραπάνω ανάλυση αποδεικνύει την ύπαρξη καθολικών οικογενειών συναρτήσεων κατακερματισμού, ωστόσο, η οικογένεια που χρησιμοποιήσαμε δεν μπορεί να εφαρμοστεί αποδοτικά στην πράξη (γιατί;). Κλείνοντας την ενότητα, αναφέρουμε, δίχως απόδειξη, μια πρακτική κατασκευή καθολικής οικογένειας συναρτήσεων κατακερματισμού. Έστω p ένας πρώτος αριθμός μεγαλύτερος του m. Έστω σύνολα $Z_p = \{0, 1, \dots, p-1\}$ και $Z_p^* = \{1,2, \dots, p-1\}$. Ορίζουμε μια οικογένεια συναρτήσεων κατακερματισμού

$$\mathcal{H} = \{ h_{a,b} : a \in Z_p^*, b \in Z_p \},$$

θέτοντας

$$h_{a,b}(k) = [(ak + b) \bmod p] \bmod m.$$

Ασκήσεις

9.1 Έστω ότι υλοποιούμε κατακερματισμό με πίνακα $M = 13$ θέσεων και συνάρτηση $h_1(x) = x \bmod M$. Σχεδιάστε τη μορφή που θα έχει η δομή του κατακερματισμού μετά την εισαγωγή των κλειδιών

$$60, 40, 36, 21, 1, 10, 27, 14, 3$$

όταν χρησιμοποιούμε:

α) Γραμμική βολιδοσκόπηση.

β) Διπλό κατακερματισμό με συνάρτηση $h(x,i) = \left(h_1(x) + ih_2(x)\right) \bmod M$, όπου $h_2(x) = 1 + (x \bmod (M-1))$.

Ποια από τις δύο μεθόδους δίνει τον μικρότερο αριθμό συγκρούσεων κατά την εισαγωγή των παραπάνω κλειδιών;

9.2 Θέλουμε να σχεδιάσουμε μια δομή δεδομένων η οποία να αναπαριστά μαθηματικά σύνολα ακέραιων αριθμών. Ένα σύνολο S υποστηρίζει τις παρακάτω λειτουργίες:

σύνολο(A) :	Δημιουργεί ένα σύνολο S με τους ακέραιους του πίνακα A.
εισαγωγή(x) :	Εισάγει τον ακέραιο x στο σύνολο S.
διαγραφή(x) :	Διαγράφει από το σύνολο S τον ακέραιο x.

περιέχει(x) : Επιστρέφει true, αν και μόνο αν ο ακέραιος x ανήκει στο σύνολο S.

ένωση(S') : Εισάγει στο σύνολο S τους ακέραιους του συνόλου S' που δεν ανήκουν στο S.

τομή(S') : Διαγράφει από το σύνολο S τους ακέραιους που δεν ανήκουν στο σύνολο S'.

Περιγράψτε μια υλοποίηση αυτής της δομής με χρήση κατακερματισμού.

9.3 Στα παρακάτω ερωτήματα οι nκαι mείναι θετικοί ακέραιοι. Συμβολίζουμε με $[n]$και $[m]$ τα σύνολα των ακέραιων $\{0,1,\dots,n-1\}$ και $\{0,1,\dots,m-1\}$, αντίστοιχα.

α) Έστω μια αυθαίρετη συνάρτηση κατακερματισμού $h : U \to [m]$, η οποία αντιστοιχεί αντικείμενα ενός συνόλου U στους ακέραιους από 0 έως και $m-1$. Θέλουμε να χρησιμοποιήσουμε την h, για να αποθηκεύσουμε ένα σύνολο $K \subseteq U$ σε πίνακα κατακερματισμού T με τη μέθοδο της αλυσιδωτής σύνδεσης. Δείξτε ότι, αν το U είναι αρκετά μεγάλο (πόσο μεγάλο;), τότε υπάρχει σύνολο K με n στοιχεία, τέτοιο ώστε κάθε στοιχείο του K να αποθηκεύεται στην ίδια θέση του πίνακα T (δηλαδή όλα τα στοιχεία του K αποθηκεύονται στην ίδια αλυσίδα μήκους n).

β) Έστω $\mathcal{H}$ η οικογένεια **όλων** των δυνατών συναρτήσεων κατακερματισμού

$$h : [n] \to [m]$$

όπου $n > m$. Δείξτε ότι η $\mathcal{H}$ είναι καθολική οικογένεια, δηλαδή για οποιουσδήποτε δύο διαφορετικούς ακέραιους $k, l \in \{0,1,\dots,n-1\}$, η πιθανότητα σύγκρουσης των k και l για τυχαία επιλεγμένη συνάρτηση $h \in \mathcal{H}$ είναι

$$\Pr_{h\in\mathcal{H}}[h(k) = h(l)] \le \frac{1}{m}.$$

Πώς θα μπορούσαμε να χρησιμοποιήσουμε αυτήν την οικογένεια σε μια υλοποίηση;

Υπόδειξη: Μια συνάρτηση $h : \{0,1,\dots,n-1\} \to \{0,1,\dots,m-1\}$ μπορεί να αναπαρασταθεί με ένα πίνακα nθέσεων, όπου η κάθε θέση αποθηκεύει μια τιμή από το σύνολο $\{0,1,\dots,m-1\}$. Το $\mathcal{H}$ περιέχει όλους του δυνατούς πίνακες αυτής της μορφής.

γ) Ορίζουμε μια οικογένεια συναρτήσεων κατακερματισμού $\mathcal{H} = \{ h_{a,b} : a, b \in [m] \}$, οι οποίες αντιστοιχούν ζεύγη ακέραιων (x, y), με $x \in [m]$ και $y \in [m]$, σε ακέραιο στο $[m]$ ($h : [m]^2 \to [m]$), όπου

$$h_{a,b}(x, y) = (ax + by) \bmod m.$$

Δηλαδή η $\mathcal{H}$ περιέχει μια συνάρτηση κατακερματισμού για κάθε διαφορετικό ζεύγος τιμών $a \in [m]$ και $b \in [m]$. Δείξτε ότι αν το mείναι δύναμη του 2 ($m = 2^k$ για κάποιο σταθερό k), τότε η $\mathcal{H}$ δεν είναι καθολική οικογένεια.

Υπόδειξη: Μπορούμε να βρούμε ένα ζεύγος ακέραιων (x, y) το οποίο να συγκρούεται με το ζεύγος $(0,0)$ σε πολλές συναρτήσεις της οικογένειας $\mathcal{H}$.

Βιβλιογραφία

Cormen, T., Leiserson, C., Rivest, R., & Stain, C. (2001). *Introduction to Algorithms.* MIT Press (2nd edition) .

Dasgupta, S., Papadimitriou, C., & Vazirani, U. (2008). *Algorithms.* McGraw-Hill.

Dietzfelbinger, Martin, Anna R. Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, και Robert Endre Tarjan. «Dynamic Perfect Hashing: Upper and Lower Bounds.» *SIAM J. Comput.*, 1994: 738-761.

Goodrich, Michael T., και Roberto Tamassia. *Data Structures and Algorithms in Java, 4th edition.* Wiley, 2006.

Mehlhorn, Kurt, και Peter Sanders. *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag, 2008.

Pagh, Rasmus, και Flemming Friche Rodler. «Cuckoo hashing.» *Journal of Algorithms,* 2004: 122–144.

Sedgewick, Robert, και Kevin Wayne. *Algorithms, 4th edition.* Addison-Wesley, 2011.

Tarjan, Robert E. *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics, 1983.

Μποζάνης, Παναγιώτης Δ. *Δομές Δεδομένων.* Εκδόσεις Τζιόλα, 2006.

Κεφάλαιο 10
Ψηφιακά Λεξικά

Περιεχόμενα

10.1 Εισαγωγή 213
10.2 Ψηφιακά Δένδρα 214
10.3 Υλοποίηση σε Java 222
10.4 Συμπιεσμένα και τριαδικά ψηφιακά δένδρα 223
Ασκήσεις 225
Βιβλιογραφία 226

10.1 Εισαγωγή

Σε αυτό το κεφάλαιο μελετάμε μια διαφορετική κατηγορία λεξικών (δομών αναζήτησης) από αυτή που είδαμε στα Κεφάλαια 7 και 8. Εδώ επιθυμούμε να αποθηκεύσουμε ένα δυναμικό σύνολο στοιχειών τα οποία είναι ακολουθίες συμβόλων $x = x_1 x_2 \dots x_\mu$ από ένα πεπερασμένο αλφάβητο $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_R\}$. Θα αναφερόμαστε σε τέτοιες ακολουθίες συμβόλων x ως «λέξεις» και σε μια δομή η οποία διαχειρίζεται ένα σύνολο λέξεων ως «ψηφιακό λεξικό». Για μια λέξη $x = x_1 x_2 \dots x_\mu$ η οποία αποτελείται από μ χαρακτήρες θα λέμε ότι έχει μήκος μ. Το αλφάβητο καθορίζεται από το είδος των λέξεων που θέλουμε να αποθηκεύσουμε. Για παράδειγμα, $R = 256$ για ακολουθίες χαρακτήρων των 8 bit, $R = 10$ για ακολουθίες δεκαδικών ψηφίων, $R = 4$ για ακολουθίες DNA κλπ.

Ένα ψηφιακό λεξικό D υποστηρίζει τις βασικές λειτουργίες ενός λεξικού :

κατασκευή() : Επιστρέφει ένα κενό λεξικό.

αναζήτηση(x) : Αν το D περιέχει τη λέξη x τότε επιστρέφει «αληθές». Διαφορετικά επιστρέφει «ψευδές».

εισαγωγή(x) : Εισαγάγει στο D μια νέα λέξη x.

διαγραφή(x) : Διαγράφει από το D τη λέξη x.

Στα ψηφιακά λεξικά μπορούμε να εκμεταλλευτούμε το γεγονός ότι αποθηκεύουμε ακολουθίες συμβόλων, για να υποστηρίξουμε κάποιες χρήσιμες λειτουργίες, τις οποίες φαίνεται δύσκολο να υλοποιήσουμε αποδοτικά με τις δομές των Κεφαλαίων 7 και 8. Έστω δύο λέξεις $y = y_1 y_2 \dots y_\lambda$ και $x = x_1 x_2 \dots x_\mu$, όπου $\lambda \leq \mu$. Η λέξη y αποτελεί *πρόθεμα* της x αν $x_i = y_i$ για $1 \leq i \leq \lambda$. Μια *μερική λέξη* $y = y_1 y_2 \dots y_\lambda$ είναι μια ακολουθία συμβόλων από το αλφάβητο $\Sigma \cup \{?\}$, όπου το σύμβολο «?» είναι ένας ειδικός χαρακτήρας ο οποίος μπορεί να αντικατασταθεί από οποιοδήποτε χαρακτήρα του Σ. Μια λέξη $x = x_1 x_2 \dots x_\mu$ ταυτίζεται με τη μερική λέξη $y = y_1 y_2 \dots y_\lambda$, αν για $1 \leq i \leq \lambda$ έχουμε $x_i = y_i$ ή $y_i = ?$.

Ένα ψηφιακό λεξικό D υποστηρίζει ακόμα τις ακόλουθες λειτουργίες:

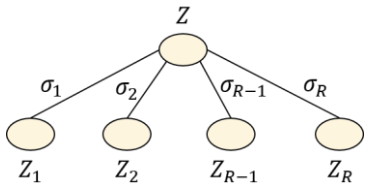
$\text{πρόθεμα}(y)$: Επιστρέφει όλες τις λέξεις x του D οι οποίες έχουν ως πρόθεμα τη λέξη y.

$\text{ταύτιση}(y)$: Επιστρέφει όλες τις λέξεις x του D οι οποίες ταυτίζονται με τη μερική λέξη y.

Στη συνέχεια, θα σχεδιάσουμε μια δομή δεδομένων η οποία υποστηρίζει τις παραπάνω λειτουργίες.

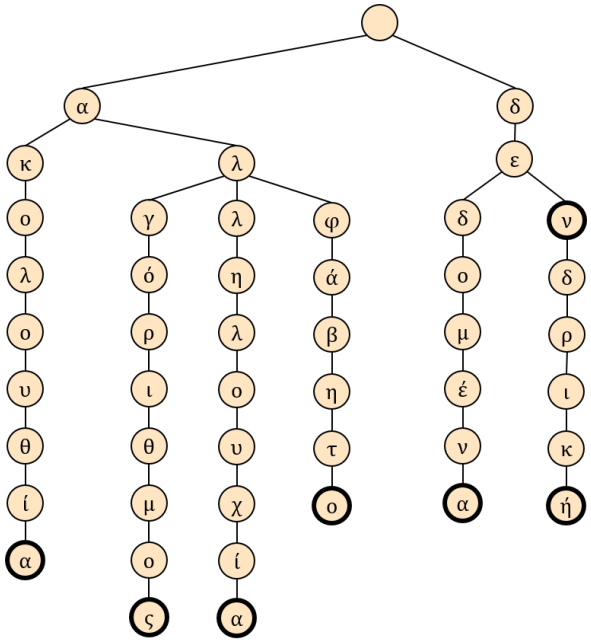
10.2 Ψηφιακά Δένδρα

Ένα ψηφιακό δένδρο T είναι ένα δένδρο αναζήτησης, κάθε εσωτερικός κόμβος Z του οποίου έχει ακριβώς $R = |\Sigma|$ παιδιά $Z_1, Z_2, \dots, Z_R$. Κάθε παιδί Z_i του Z αντιστοιχεί στο σύμβολο σ_i του αλφάβητου Σ, όπως φαίνεται στην Εικόνα 10.37.



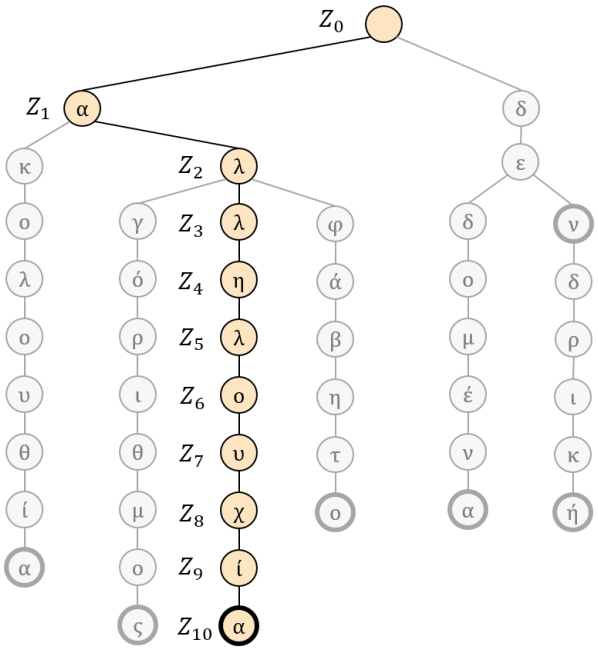
Εικόνα 10.37: Εσωτερικός κόμβος Z ενός ψηφιακού δένδρου και τα παιδιά του $Z_1, Z_2, \dots, Z_R$.

Όπως στο Κεφάλαιο 8, θα συμβολίζουμε το i-οστό παιδί ενός κόμβου Z ως $Z.\text{παιδί}(i)$. Σε αντίθεση με τα λεξικά των Κεφαλαίων 7 και 8, όπου κάθε λέξη αποθηκεύεται αυτούσια σε ένα ξεχωριστό κόμβο, σε ένα ψηφιακό δένδρο οι λέξεις αποθηκεύονται με έμμεσο τρόπο. Κάθε κόμβος αντιστοιχεί σε ένα χαρακτήρα του αλφάβητου και μια λέξη αντιστοιχεί σε ένα μονοπάτι από τη ρίζα προς κάποιο εσωτερικό κόμβο. Μια αναπαράσταση ενός τέτοιου δένδρου φαίνεται στην Εικόνα 10.38. Σε αυτήν την αναπαράσταση παραλείπουμε τις αναφορές σε κενούς κόμβους, και σημειώνουμε με έντονη γραμμή τους κόμβους που αντιστοιχούν σε τερματικούς χαρακτήρες των λέξεων.



Εικόνα 10.38: Γραφική αναπαράσταση ενός ψηφιακού δένδρου για τις λέξεις «ακολουθία», «αλγόριθμος», «αλληλουχία», «αλφάβητο», «δεδομένα», «δεν», «δενδρική».

Συγκεκριμένα, ας θεωρήσουμε ότι έχουμε αποθηκεύσει στο δένδρο τη λέξη $\sigma_{i_1}\sigma_{i_2}$ ... σ_{i_μ}. Η λέξη αυτή αντιστοιχεί στο μονοπάτι $Z_0, Z_1, \ldots, Z_\mu$ του ψηφιακού δένδρου, όπου Z_0 είναι η ρίζα του δένδρου και $Z_{j+1} = Z_j.\text{παιδί}(i_j)$ για $0 \leq j \leq \mu - 1$. Δείτε την Εικόνα 10.39.



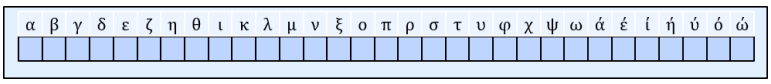
Εικόνα 10.39: Το μονοπάτι στο ψηφιακό δένδρο της Εικόνα 10.38 που αντιστοιχεί στη λέξη αλληλουχία.

Με την παραπάνω δομή μπορούμε να επιτύχουμε τους χρόνους εκτέλεσης που δίνονται στον Πίνακας 10.1.

Πίνακας 10.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης των βασικών λειτουργιών ενός ψηφιακού δένδρου T. Η λέξη η οποία δίνεται ως παράμετρος έχει μήκος μ. Για τις λειτουργίες πρόθεμα και ταύτιση, ν είναι το πλήθος των λέξεων που επιστρέφει η αντίστοιχη λειτουργία. Επιπλέον, για τη λειτουργία πρόθεμα, λ είναι το μέσο μήκος των λέξεων που επιστρέφονται.

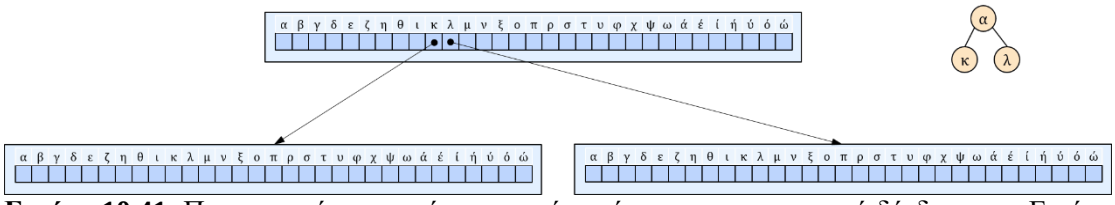
αναζήτηση	εισαγωγή	διαγραφή	πρόθεμα	ταύτιση
$O(\boldsymbol{\mu})$	$O(\mu)$	$O(\mu)$	$O(\mu + (\lambda - \mu)\nu)$	$O(\mu\nu)$

Θα περιγράψουμε αναλυτικά την υλοποίηση ενός τέτοιου ψηφιακού δένδρου. Κάθε κόμβος Z του δένδρου περιλαμβάνει ένα πίνακα $R = |\Sigma|$ θέσεων, όπου η i-οστή θέση αντιστοιχεί στον i-οστό χαρακτήρα σ_i του αλφάβητου Σ και αποθηκεύει μια αναφορά στον κόμβο $Z.\text{παιδί}(i)$. Δείτε την Εικόνα 10.40. Επιπλέον, διατηρούμε ένα bit επισήμανσης, με το οποίο ελέγχουμε αν ο τρέχων κόμβος σηματοδοτεί το τέλος μιας λέξης του δένδρου. Καλούμε ένα τέτοιο κόμβο *επισημασμένο*.



Εικόνα 10.40: Υλοποίηση ενός κόμβου ψηφιακού δένδρου για λέξεις από το ελληνικό αλφάβητο.

Ας θεωρήσουμε τώρα ένα κόμβο Z του ψηφιακού δένδρου, ο οποίος βρίσκεται σε απόσταση λ από τη ρίζα. Το μονοπάτι από τη ρίζα προς τον κόμβο Z κωδικοποιεί μια λέξη $y = \sigma_{i_1}\sigma_{i_2} \ldots \sigma_{i_\lambda}$, η οποία αποτελεί πρόθεμα μιας ή περισσότερων λέξεων που έχουν αποθηκευτεί στο ψηφιακό δένδρο. Έτσι, για $1 \leq i \leq R$, η αναφορά $Z.\text{παιδί}(i)$ δείχνει σε εσωτερικό κόμβο του δένδρου αν και μόνο αν έχουμε αποθηκεύσει στο δένδρο κάποια λέξη με πρόθεμα $y\sigma_i$. Δείτε την Εικόνα 10.41.



Εικόνα 10.41: Πραγματική αναπαράσταση ενός τμήματος του ψηφιακού δένδρου της Εικόνα 10.38.

Ο αλγόριθμος αναζήτησης μιας λέξης x στο ψηφιακό δένδρο ακολουθεί το μονοπάτι από τη ρίζα το οποίο αντιστοιχεί στους χαρακτήρες της x. Παρακάτω δίνουμε μια αναδρομική περιγραφή αυτής της διαδικασίας.

Αλγόριθμος αναζήτησης λέξης σε ψηφιακό δένδρο

αναζήτηση(Z, x, d)
1. αν ο κόμβος Z είναι κενός, τότε επίστρεψε τον κενό κόμβο
2. αν η λέξη x έχει μήκος d, τότε επίστρεψε τον κόμβο Z
3. έστω σ_i ο χαρακτήρας της λέξης x στη θέση $d + 1$
4. εκτέλεσε αναδρομικά αναζήτηση$(Z.\text{παιδί}(i), x, d + 1)$

τέλος αναζήτηση(Z, x, d)

αρχή αναζήτησης

5. εκτέλεσε $Z \leftarrow$ αναζήτηση$(T.\text{ρίζα}, x, 0)$
6. αν ο Z είναι κενός ή δεν είναι επισημασμένος τότε επίστρεψε ψευδές
7. διαφορετικά επίστρεψε αληθές

τέλος αναζήτησης

Για την εισαγωγή μιας λέξης ακολουθούμε μια ανάλογη διαδικασία, με τη διαφορά ότι αν ο επόμενος κόμβος Z.παιδί(i) του μονοπατιού αναζήτησης είναι κενός, τότε τον δημιουργούμε και τον συνδέουμε με τον τρέχοντα κόμβο Z. Έπειτα, συνεχίζουμε την ίδια διαδικασία αναδρομικά. Τέλος, θέτουμε το bit επισήμανσης στον τερματικό κόμβο του μονοπατιού εισαγωγής.

Αλγόριθμος εισαγωγής λέξης σε ψηφιακό δένδρο

εισαγωγή(Z, x, d)
1. αν ο κόμβος Z είναι κενός, τότε δημιούργησε ένα νέο εσωτερικό κόμβο Z
2. αν η λέξη x έχει μήκος d, τότε θέσε το bit επισήμανσης του Z και επίστρεψε τον Z
3. έστω σ_i ο χαρακτήρας της λέξης x στη θέση $d + 1$
4. εκτέλεσε αναδρομικά $Z.\text{παιδί}(i) \leftarrow$ εισαγωγή$(Z.\text{παιδί}(i), x, d + 1)$

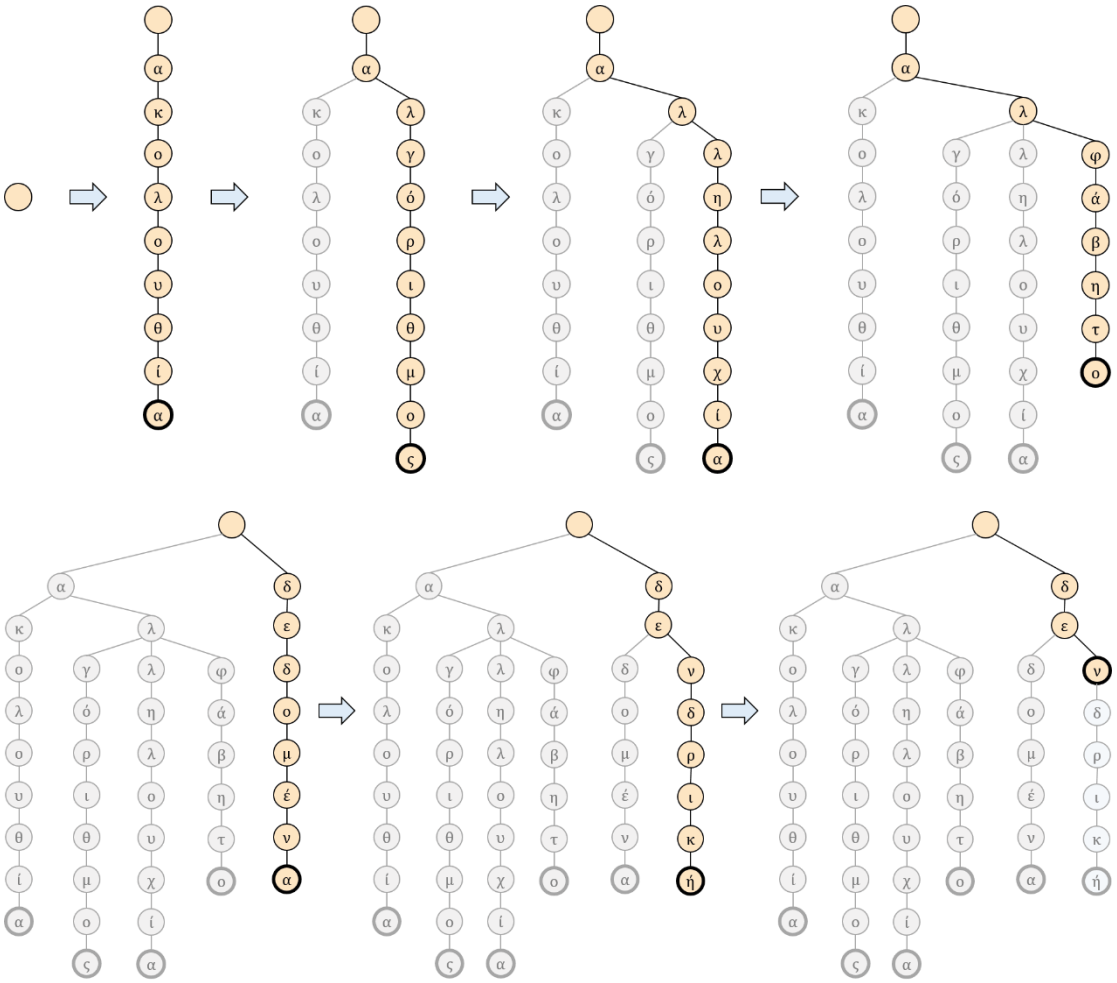
τέλος εισαγωγή(Z, x, d)

αρχή εισαγωγής

5. εκτέλεσε $T.\text{ρίζα} \leftarrow$ εισαγωγή$(T.\text{ρίζα}, x, 0)$

τέλος εισαγωγής

Η Εικόνα 10.42 δίνει ένα παράδειγμα διαδοχικών εισαγωγών λέξεων σε αρχικά κενό ψηφιακό δένδρο.



Εικόνα 10.42: Κατασκευή ψηφιακού δένδρου με διαδοχική εισαγωγή λέξεων. Εισάγουμε, με τη σειρά, τις λέξεις «ακολουθία», «αλγόριθμος», «αλληλουχία», «αλφάβητο», «δεδομένα», «δενδρική» και «δεν».

Σε αυτό το σημείο είναι χρήσιμο να αναφέρουμε κάποια χαρακτηριστικά των επιδόσεων των ψηφιακών δένδρων:

- Η μορφή του ψηφιακού δένδρου είναι ανεξάρτητη από τη σειρά εισαγωγής των στοιχείων. Κάθε δεδομένο σύνολο διακριτών λέξεων στοιχείων δημιουργεί ένα μοναδικό ψηφιακό δένδρο.

- Η αναζήτηση ή εισαγωγή μιας λέξης μήκους μ χαρακτήρων απαιτεί $O(\mu)$ χρόνο στη χειρότερη περίπτωση.

- Έστω N το πλήθος των συνδέσμων (κενών ή μη) σε ένα ψηφιακό δένδρο κατασκευασμένο από n λέξεις μέσου μήκους μ από αλφάβητο R χαρακτήρων. Ισχύει $Rn \leq N \leq Rn\lambda$.

Ας εξετάσουμε τώρα την περίπτωση της διαγραφής μιας λέξης x. Πρώτα αναζητούμε τη λέξη x στο δένδρο. Αν δεν υπάρχει, τότε δε γίνεται καμία αλλαγή στο δένδρο. Διαφορετικά, αν η x υπάρχει, βρίσκουμε τον κόμβο Z που αντιστοιχεί στο τελευταίο της γράμμα και σβήνουμε την

επισήμανση του. Αν ο Z δεν έχει απογόνους στο δένδρο, τότε τον διαγράφουμε και συνεχίζουμε την ίδια διαδικασία στους προγόνους του Z, μέχρι να φτάσουμε σε κόμβο με απογόνους ή με επισήμανση.

Αλγόριθμος διαγραφής σε ψηφιακό δένδρο

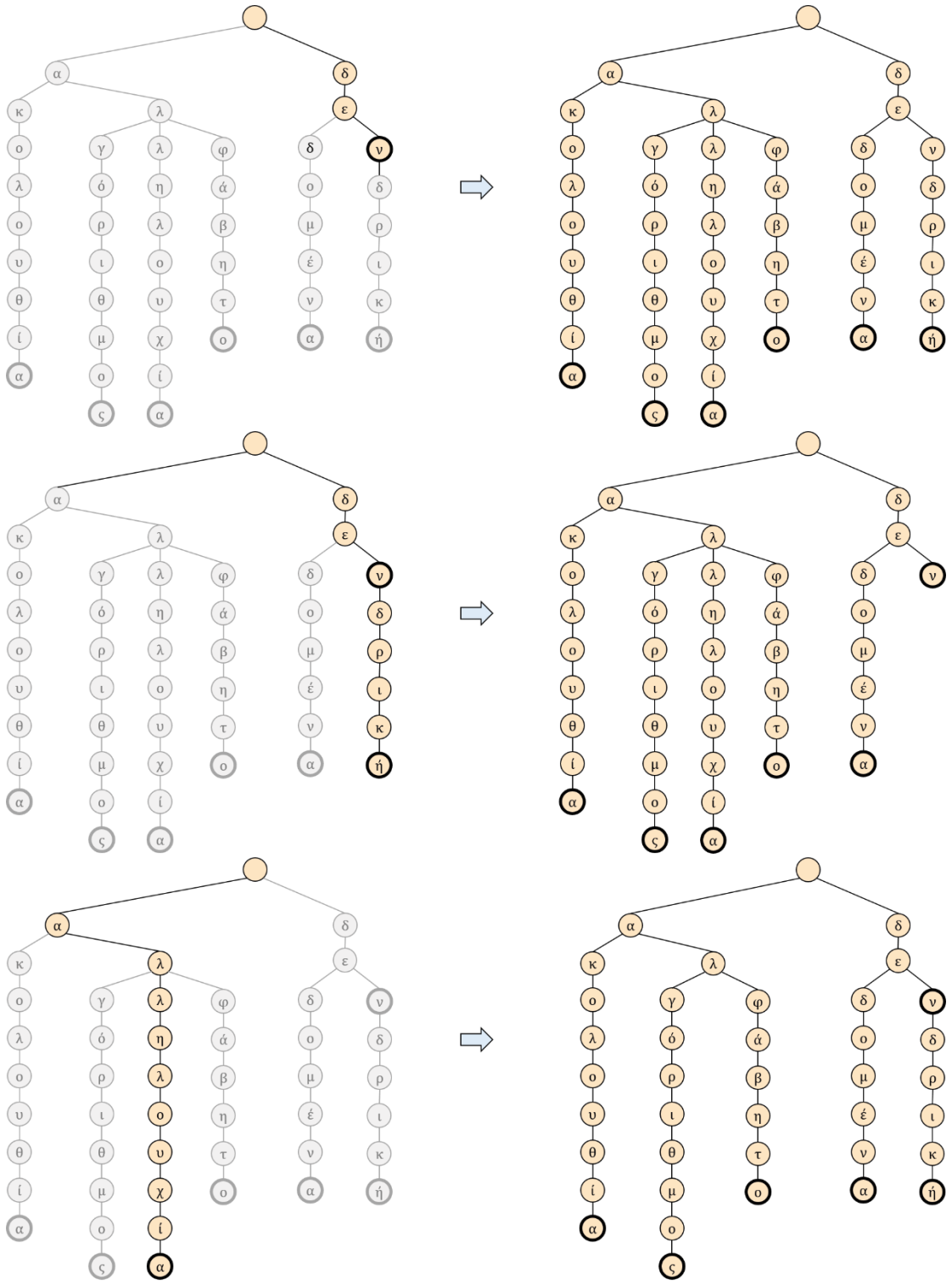
αρχή διαγραφής

1. εκτέλεσε $Z \leftarrow \text{αναζήτηση}(T.\text{ρίζα}, x, 0)$
2. αν ο Z είναι κενός ή δεν είναι επισημασμένος, τότε επίστρεψε (δεν γίνεται καμία αλλαγή στο δένδρο)
3. σβήσε την επισήμανση του Z
4. ενόσω ο Z δεν είναι κενός και δεν έχει επισήμανση και δεν έχει παιδιά
5. έστω Y ο γονέας του Z στο δένδρο
6. διάγραψε τον κόμβο Z
7. θέσε $Z \leftarrow Y$

τέλος διαγραφής

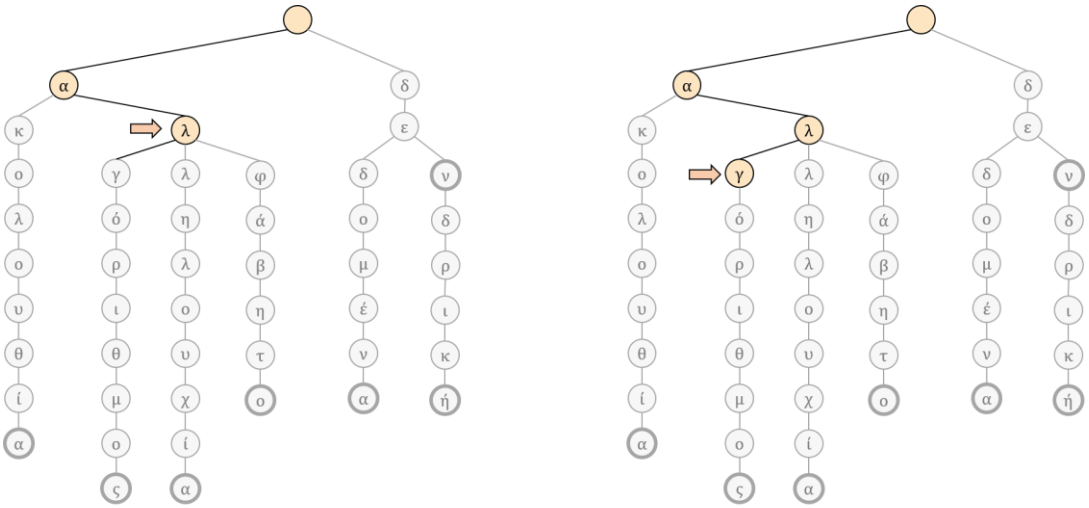
Στην Εικόνα 10.43 φαίνονται οι διάφορες περιπτώσεις διαγραφής λέξης σε ένα ψηφιακό δένδρο. Στην πρώτη περίπτωση, για τη διαγραφή της λέξης «δεν», αρκεί να σβήσουμε την επισήμανση του κόμβου Z, ο οποίος αντιστοιχεί στο τελευταίο γράμμα της λέξης. Καθώς ο κόμβος Z έχει παιδί, το δένδρο δεν υφίσταται καμία άλλη αλλαγή. Στη δεύτερη περίπτωση, όπου διαγράφουμε τη λέξη «δενδρική», η αναζήτηση της λέξης καταλήγει σε κόμβο Z χωρίς κανένα παιδί. Έτσι, διαγράφουμε όλους τους προγόνους του Z στο δένδρο μέχρι τον κοντινότερο πρόγονο ο οποίος είναι επισημασμένος. Ομοίως, στην τρίτη περίπτωση, κατά τη διαγραφή της λέξης «αλληλουχία» καταλήγουμε σε κόμβο Z χωρίς κανένα παιδί. Αυτή τη φορά, διαγράφουμε όλους τους προγόνους του Z στο δένδρο μέχρι τον κοντινότερο πρόγονο ο οποίος έχει παιδιά.

Όπως και με τις λειτουργίες της αναζήτησης και της εισαγωγής, η διαγραφή μιας λέξης μήκους μ χαρακτήρων απαιτεί $O(\mu)$ χρόνο στη χειρότερη περίπτωση.



Εικόνα 10.43: Διαγραφή των λέξεων «δεν», «δενδρική» και «αλληλουχία» από το ψηφιακό δένδρο της Εικόνα 10.38. Κάθε διαγραφή εκτελείται στο αρχικό δένδρο.

Πριν προχωρήσουμε στις υπόλοιπες δύο εντολές, είναι χρήσιμο να εξετάσουμε πώς μπορούμε να συλλέξουμε όλες τις λέξεις οι οποίες είναι αποθηκευμένες σε ένα ψηφιακό δένδρο. Καλούμε αυτή τη διαδικασία συλλογή. Εύκολα μπορούμε να διαπιστώσουμε ότι η διαδικασία της συλλογής μπορεί να πραγματοποιηθεί με μια προδιατεταγμένη διάσχιση του δένδρου. Κατά τη διάρκεια της διάσχισης, σχηματίζουμε ένα τρέχον πρόθεμα $y = \sigma_{i_1}\sigma_{i_2} \dots \sigma_{i_\mu}$ το οποίο αντιστοιχεί στο μονοπάτι $Z_0, Z_1, \dots, Z_\mu$ του ψηφιακού δένδρου, όπου Z_0 είναι η ρίζα του δένδρου, Z_μ είναι ο τρέχων κόμβος της διάσχισης, και $Z_{j+1} = Z_j.\text{παιδί}(i_j)$ για $0 \le j \le \mu - 1$. Δείτε την Εικόνα 10.44. Αν ο τρέχων κόμβος Z_μ είναι επισημασμένος, τότε το πρόθεμα y είναι λέξη που έχει αποθηκευτεί στο δένδρο και την τοποθετούμε σε μια ουρά Q. Παρατηρήστε ότι, αν κατά τη διάρκεια της διάσχισης επισκεπτόμαστε τα παιδιά κάθε κόμβου σε αλφαβητική σειρά, τότε στο τέλος αυτής της διαδικασίας η ουρά Q θα περιέχει όλες τις λέξεις του ψηφιακού δένδρου, διατεταγμένες σε λεξικογραφική σειρά.



Εικόνα 10.44: Σχηματισμός προθέματος κατά τη προδιατεταγμένη διάσχιση του ψηφιακού δένδρου της Εικόνα 10.38.

Μπορούμε να εκτελέσουμε την ίδια διαδικασία, με αφετηρία κάποιο αυθαίρετο κόμβο Z του ψηφιακού δένδρου αντί για τη ρίζα. Έστω y το πρόθεμα το οποίο αντιστοιχεί στο μονοπάτι του ψηφιακού δένδρου από τη ρίζα προς τον Z. Το αποτέλεσμα θα είναι να συλλέξουμε στην ουρά Q τις καταλήξεις των λέξεων του δένδρου οι οποίες έχουν ως πρόθεμα το y. Καλούμε αυτή τη διαδικασία συλλογή(Z).

Από τα παραπάνω, γίνεται κατανοητό ότι μπορούμε να υλοποιήσουμε τη λειτουργία πρόθεμα(y) όπως φαίνεται στον επόμενο αλγόριθμο.

Αλγόριθμος εύρεσης όλων των λέξεων του ψηφιακού δένδρου με πρόθεμα y

αρχή πρόθεμα(y)

1. εκτέλεσε $Z \leftarrow$ αναζήτηση$(T.\text{ρίζα}, y, 0)$
2. αν ο Z είναι κενός τότε επίστρεψε την κενή ουρά Q
3. επίστρεψε την ουρά $Q \leftarrow$ συλλογή(Z)

τέλος πρόθεμα(y)

10.3 Υλοποίηση σε Java

Το πρόγραμμα StringDT.java υλοποιεί ένα ψηφιακό δένδρο το οποίο αποθηκεύει λέξεις, δηλαδή αντικείμενα τύπου String όπου ο κάθε χαρακτήρας είναι γράμμα του λατινικού αλφάβητου a-z. Κάθε κόμβος Z του ψηφιακού δένδρου περιέχει έναν πίνακα Node next[R] συνδέσμων σε το πολύ R = 26 παιδιά, που ο σύνδεσμος $Z.\text{next}[j]$ αντιστοιχεί στο γράμμα ('a' + j). Δηλαδή ο $Z.\text{next}[0]$ στο 'a', ο $Z.\text{next}[1]$ στο 'b', …, και ο $Z.\text{next}[25]$ στο 'z'. Κάθε κόμβος Z του ψηφιακού δένδρου περιλαμβάνει επίσης μια μεταβλητή σήμανσης boolean mark, όπου $Z.\text{mark} ==$ true, αν ο Z αντιστοιχεί στο τέλος μιας λέξης που έχει εισαχθεί στη δομή.

```
public class StringDT {

    private static int R = 26;  // πλήθος διαφορετικών χαρακτήρων
    private static int N = 0;   // πλήθος λέξεων στο ψηφιακό δένδρο
    private Node root;          // ρίζα του ψηφιακού δένδρου

    // κόμβος ψηφιακού δένδρου
    private static class Node {
        private boolean mark;  // true αν είναι το τέλος μιας λέξης
        private Node[] next = new Node[R];  // αναφορές σε R παιδιά
    }

    // επιστρέφει true αν βρει τη λέξη x
    public boolean contains(String x) {
        Node Z = contains(root, x, 0);
        if (Z == null) {
            return false;
        } else {
            return Z.mark;
        }
    }

    // αναζήτηση στους απόγονους του Z για τη λέξη που περιέχει το x
    // από τη θέση d και μετά
    private Node contains(Node Z, String x, int d) {
        if (Z == null) {
            return null;
        }
        if (d == x.length()) {
            return Z;  // τέλος της λέξης
        }
        char c = x.charAt(d);   // επόμενος χαρακτήρας
        int j = (int) c - 'a';  // αντίστοιχη θέση του επόμενου χαρακτήρα
        return contains(Z.next[j], x, d + 1);
    }

    // εισαγωγή της λέξης x
    public void insert(String x) {
        root = insert(root, x, 0);
    }

    // εισαγωγή στους απόγονους του Z της λέξης που περιέχει το x
    // από τη θέση d και μετά
    private Node insert(Node Z, String x, int d) {
        if (Z == null) {
            Z = new Node();
```

```
        }
        if (d == x.length()) {
            Z.mark = true;
            N++;
            return Z;
        }  // τέλος της λέξης
        char c = s.charAt(d);   // επόμενος χαρακτήρας
        int j = (int) c - 'a';  // αντίστοιχη θέση του επόμενου χαρακτήρα
        Z.next[j] = insert(Z.next[j], x, d + 1);
        return Z;
    }

    // διαγραφή της λέξης x
    public void delete(String x) {
        root = delete(root, x, 0);
    }

    // διαγραφή από τους απόγονους του Z της λέξης που περιέχει το x
    // από τη θέση d και μετά
    private Node delete(Node Z, String x, int d) {
        if (Z == null) {
            return null;
        }
        if (d == x.length()) {
            Z.mark = false;  // τέλος της λέξης
            N--;
        } else {
            char c = s.charAt(d);   // επόμενος χαρακτήρας
            int j = (int) c - 'a';  // αντίστοιχη θέση του επόμενου χαρακτήρα
            Z.next[j] = delete(Z.next[j], x, d + 1);
        }

        if (Z.mark) {
            return Z; // ο Z είναι επισημασμένος και δεν τον διαγράφουμε
        }
        for (int j = 0; j < R; j++) {
            if (Z.next[j] != null) {
                return Z; // ο Z έχει μη κενά παιδιά και δεν τον διαγράφουμε
            }
        }
        return null;
    }
}
```

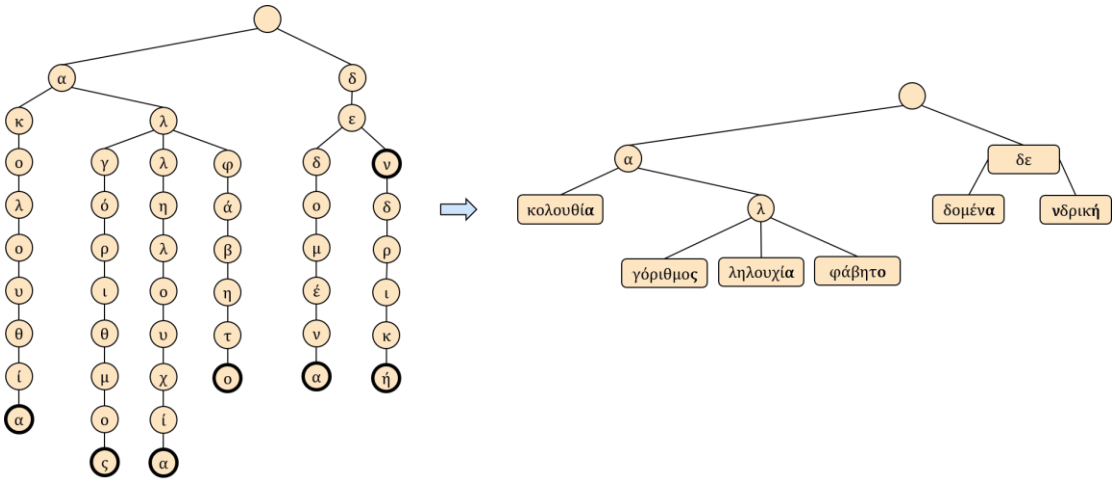
10.4 Συμπιεσμένα και τριαδικά ψηφιακά δένδρα

Το ψηφιακό δένδρο της Ενότητας 10.2 είναι υπερβολικά σπάταλο σε χώρο μνήμης, όταν αποθηκεύουμε λέξεις με μεγάλο μήκος και με χαρακτήρες από μεγάλο αλφάβητο. Σε αυτήν την ενότητα αναφέρουμε, εν συντομία, δύο εναλλακτικές μορφές ψηφιακών δένδρων, οι οποίες αποσκοπούν στη βελτίωση των απαιτήσεων σε μνήμη.

Συμπιεσμένα ψηφιακά δένδρα

Ένας τρόπος, για να ελαττώσουμε τον αποθηκευτικό χώρο που απαιτεί το ψηφιακό δένδρο, είναι να απαλείψουμε τους κόμβους του δένδρου που δεν δημιουργούν πολλαπλές διακλαδώσεις. Συγκεκριμένα, έστω $Z_0, Z_1, \ldots, Z_\mu$ ένα μονοπάτι του ψηφιακού δένδρου, όπου

$Z_{j+1} = Z_j.\text{παιδί}(i_j)$ και ο κόμβος Z_{j+1} είναι το μοναδικό παιδί του Z_j, για $0 \leq j \leq \mu - 1$. Τότε, μπορούμε να αντικαταστήσουμε το μονοπάτι αυτό με ένα νέο κόμβο Z, όπως φαίνεται στην Εικόνα 10.45. Για να διατηρήσουμε την πληροφορία του αρχικού μονοπατιού $Z_0, Z_1, \ldots, Z_\mu$, αποθηκεύουμε στον κόμβο Z του συμπιεσμένου ψηφιακού δένδρου τη λέξη $y = \sigma_{i_1}\sigma_{i_2} \ldots \sigma_{i_\mu}$. Επιπλέον, σημειώνουμε τους χαρακτήρες της y οι οποίοι αποτελούν τερματικούς χαρακτήρες κάποιας λέξης. Έτσι, στο παράδειγμα της Εικόνα 10.45, στον κόμβο ο οποίος αποθηκεύει τη λέξη «νδρική», σημειώνουμε ότι ο πρώτος και ο τελευταίος χαρακτήρας είναι τερματικοί χαρακτήρες λέξεων του δένδρου.

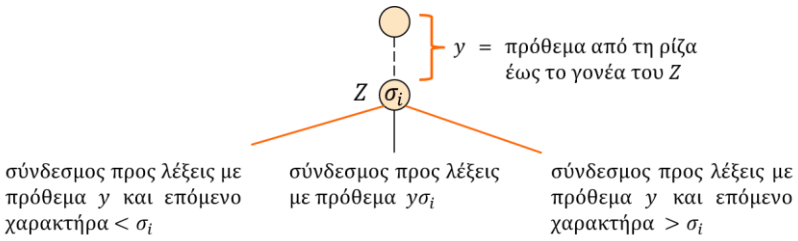


Εικόνα 10.45: Κατασκευή συμπιεσμένου ψηφιακού δένδρου.

Η παραπάνω αντιμετωπίζει το πρόβλημα χώρου, αλλά ο χειρισμός του ψηφιακού δένδρου γίνεται αρκετά πιο περίπλοκος. Δείτε την Άσκηση 10.3.

Τριαδικά ψηφιακά δένδρα

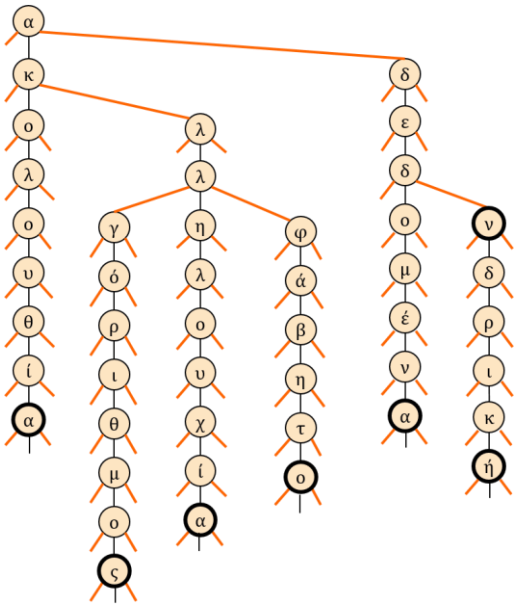
Ένας διαφορετικός τρόπος για να ελαττώσουμε τον αποθηκευτικό χώρο που απαιτεί το ψηφιακό δένδρο, είναι να μειώσουμε το πλήθος των συνδέσμων του κάθε κόμβου. Αυτή την ιδέα εφαρμόζει ένα τριαδικό ψηφιακό δένδρο, οι κόμβοι του οποίου έχουν ακριβώς τρεις συνδέσμους. Δείτε την Εικόνα 10.46.



Εικόνα 10.46: Κόμβος και σύνδεσμοι τριαδικού ψηφιακού δένδρου.

Όπως και στα τυπικά ψηφιακά δένδρα, ένα μονοπάτι από τη ρίζα προς κάποιο κόμβο Z αντιστοιχεί σε ένα πρόθεμα y. Ο κόμβος Z ελέγχει μόνο ένα χαρακτήρα σ_i του αλφάβητου. Ο μεσαίος σύνδεσμος οδηγεί σε λέξεις με πρόθεμα $y\sigma_i$. Ο αριστερός σύνδεσμος αντιστοιχεί στις λέξεις με πρόθεμα y στις οποίες ο επόμενος χαρακτήρας είναι μικρότερος του σ_i. Τέλος, ο

δεξιός σύνδεσμος αντιστοιχεί στις λέξεις με πρόθεμα y και επόμενο χαρακτήρα μεγαλύτερο του σ_i. Δείτε την Εικόνα 10.47.



Εικόνα 10.47: Γραφική αναπαράσταση ενός τριαδικού ψηφιακού δένδρου μετά την εισαγωγή των λέξεων «ακολουθία», «αλληλουχία», «αλγόριθμος», «αλφάβητο», «δεδομένα», «δεν», «δενδρική». Οι λέξεις έχουν εισαχθεί με αυτήν τη σειρά.

Σε αντίθεση με τα τυπικά ψηφιακά δένδρα (και τα συμπιεσμένα ψηφιακά δένδρα), η μορφή ενός τριαδικού ψηφιακού δένδρου δεν καθορίζεται μόνο από το σύνολο των λέξεων που εισάγουμε, αλλά επίσης και από τη σειρά εισαγωγής.

Τέλος, σημειώνουμε ότι είναι δυνατό να συνδυάσουμε δύο ή παραπάνω δομές ψηφιακών δένδρων και να κατασκευάσουμε υβριδικές μορφές τους. Δείτε την Άσκηση 10.6.

Ασκήσεις

10.1 Έστω N το πλήθος των συνδέσμων (κενών ή μη) σε ένα ψηφιακό δένδρο κατασκευασμένο από n λέξεις μέσου μήκους μ από αλφάβητο R χαρακτήρων. Δικαιολογήστε γιατί ισχύει η σχέση $Rn \leq N \leq Rn\lambda$. Δώστε κατάλληλα παραδείγματα ψηφιακών δένδρων τα οποία απαιτούν Rn και $Rn\lambda$ συνδέσμους αντίστοιχα.

10.2 Περιγράψτε μια αποδοτική υλοποίηση της λειτουργίας ταύτιση σε ένα ψηφιακό δένδρο.

10.3 Περιγράψτε πώς μπορούν να υλοποιηθούν οι λειτουργίες αναζήτησης, εισαγωγής και διαγραφής μιας λέξης σε ένα συμπιεσμένο ψηφιακό δένδρο.

10.4 Περιγράψτε πώς μπορούν να υλοποιηθούν οι λειτουργίες αναζήτησης, εισαγωγής και διαγραφής μιας λέξης σε ένα συμπιεσμένο ψηφιακό δένδρο.

10.5 Έστω ένα τριαδικό ψηφιακό δένδρο κατασκευασμένο από n λέξεις μέσου μήκους μ από αλφάβητο R χαρακτήρων. Δώστε ένα άνω και ένα κάτω φράγμα για το πλήθος των συνδέσμων (κενών ή μη) σε ένα τέτοιο δένδρο.

10.6 Περιγράψτε την υλοποίηση ενός υβριδικού ψηφιακού δένδρου το οποίο συνδυάζει τα τυπικά ψηφιακά δένδρα με τα τριαδικά ψηφιακά δένδρα ως εξής. Στη ρίζα του δένδρου βρίσκεται ένας κόμβος τυπικού ψηφιακού δένδρου με R συνδέσμους. Κάθε μη κενός σύνδεσμος δείχνει σε ένα τριαδικό ψηφιακό δένδρο. Δηλαδή, το i-οστό παιδί της ρίζας είναι η ρίζα ενός τριαδικού ψηφιακού δένδρου για τις λέξεις που ξεκινούν με το χαρακτήρα σ_i.

10.7 Υλοποιήστε σε Java και συγκρίνετε πειραματικά την απόδοση των ψηφιακών δένδρων, των τριαδικών ψηφιακών δένδρων και των υβριδικών ψηφιακών δένδρων της Άσκησης 10.6. Μετρήστε το χρόνο κατασκευής των δένδρων καθώς και το χώρο που καταλαμβάνουν στη μνήμη.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Κεφάλαιο 11
Ένωση Ξένων Συνόλων

Περιεχόμενα

11.1 Εισαγωγή 227
11.2 Εφαρμογή στο Πρόβλημα της Συνεκτικότητας 228
11.3 Δομή Ξένων Συνόλων με Συνδεδεμένες Λίστες 229
11.4 Δομή Ξένων Συνόλων με Ανοδικά Δένδρα 232
11.4.1 Συμπίεση Διαδρομής 234
11.5 Υλοποίηση σε Java 235
Ασκήσεις 237
Βιβλιογραφία 237

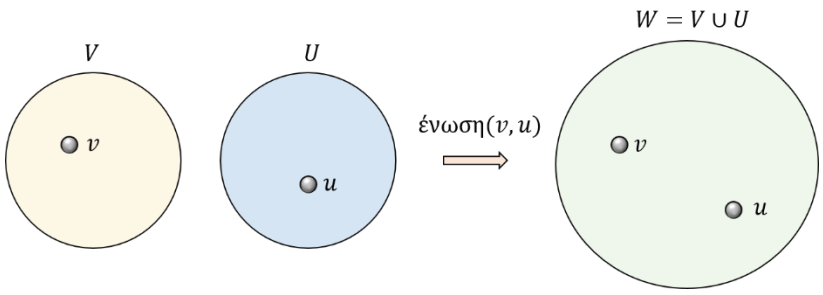
11.1 Εισαγωγή

Έστω S ένα σύνολο στοιχειών. Θέλουμε να διαχειριστούμε μια συλλογή C ξένων μεταξύ τους υποσυνόλων του S, όπου κάθε σύνολο έχει ένα διακεκριμένο όνομα, μέσω των ακόλουθων λειτουργιών:

νέο σύνολο(v) : Επιστρέφει ένα νέο σύνολο V με μοναδικό του στοιχείο το v. Πριν από την εκτέλεση της λειτουργίας, το v δεν ανήκει σε κανένα σύνολο της συλλογής C.

ένωση(v, u) : Επιστρέφει ένα νέο σύνολο W, το οποίο προκύπτει από την ένωση του συνόλου V, που περιέχει το στοιχείο v, με το σύνολο U, που περιέχει το στοιχείο u. Τα σύνολα V και U καταστρέφονται μετά την εκτέλεση της ένωσης.

εύρεση(v) : Επιστρέφει το όνομα του συνόλου το οποίο περιέχει το στοιχείο v.



Εικόνα 11.48: Ένωση δύο ξένων συνόλων.

Η επιλογή των ονομάτων που δίνουμε στα σύνολα της συλλογής μπορεί να εξαρτάται από το είδος της εφαρμογής για την οποία προορίζεται η δομή. Για παράδειγμα, σε αρκετές εφαρμογές μάς αρκεί να ελέγχουμε αν δύο στοιχεία ανήκουν σε διαφορετικά σύνολα, χωρίς να έχει σημασία το όνομα του κάθε συνόλου. Σε τέτοιες περιπτώσεις μπορούμε να δίνουμε σε κάθε σύνολο ένα αυθαίρετο όνομα, για παράδειγμα έναν αύξοντα αριθμό, με μόνη προϋπόθεση να μην υπάρχουν δύο διαφορετικά σύνολα με το ίδιο όνομα. Αντίθετα, σε άλλες περιπτώσεις θέλουμε, αντί για ένα αυθαίρετο όνομα, να μας επιστρέφεται ένα συγκεκριμένο στοιχείο του συνόλου το οποίο αποκαλούμε *αντιπρόσωπο*. Στις δομές που θα περιγράψουμε στη συνέχεια ακολουθούμε τη μέθοδο του αντιπρόσωπου.

Πίνακας 11.1: Χρόνοι εκτέλεσης χειρότερης περίπτωσης των βασικών λειτουργιών μιας δομής ένωσης ξένων συνόλων με n στοιχεία.

	νέο σύνολο	ένωση	εύρεση	ακολουθία m λειτουργιών
συνδεδεμένη λίστα με κόμβο-αντιπρόσωπο	$O(1)$	$O(n)$	$O(1)$	$O(m + n \log n)$
ανοδικά δένδρα με σταθμισμένη ένωση	$O(1)$	$O(\log n)$	$O(\log n)$	$O(m \log n)$
ανοδικά δένδρα με σταθμισμένη ένωση και συμπίεση διαδρομής	$O(1)$	$O(\log n)$	$O(\log n)$	$O(m\, \alpha(m,n))$

Ο Πίνακας 11.1 συνοψίζει τις επιδόσεις των δομών που περιγράφουμε στη συνέχεια. Η πιο αποδοτική δομή που θα δούμε επιτυγχάνει σχεδόν γραμμικό χρόνο για την εκτέλεση οποιασδήποτε ακολουθίας λειτουργιών. Συγκεκριμένα, εκτελεί οποιαδήποτε (μεικτή) ακολουθία $m \geq n$ λειτουργιών για $|S| = n$ στοιχεία σε χρόνο $O(m\alpha(m,n))$, όπου $\alpha(m,n)$ είναι η αντίστροφη συνάρτηση Ackermann, γνωστή για τον εξαιρετικά αργό ρυθμό αύξησης. Για όλες τις πρακτικές τιμές των m και n ισχύει $\alpha(m,n) \leq 4$.

11.2 Εφαρμογή στο Πρόβλημα της Συνεκτικότητας

Ας θυμηθούμε το πρόβλημα της συνεκτικότητας ενός γραφήματος. Θέλουμε να επεξεργαστούμε ένα γράφημα $G = (V, E)$, έτσι ώστε να μπορούμε να απαντάμε γρήγορα σε ερωτήματα του τύπου «υπάρχει μονοπάτι στο G μεταξύ των κορυφών x και y;» Στο Κεφάλαιο 3 είδαμε ότι αλγόριθμοι διερεύνησης γραφήματος, όπως η κατά πλάτος και η κατά βάθος διερεύνηση, λύνει το παραπάνω πρόβλημα σε γραμμικό χρόνο επεξεργασίας του G και σταθερό χρόνο ανά ερώτημα.

Η χρήση μιας δομής ξένων συνόλων δίνει μια εναλλακτική λύση στο πρόβλημα της συνεκτικότητας σε περίπου γραμμικό χρόνο. Ο αλγόριθμος αρχικοποιεί μια δομή όπου το σύνολο των στοιχείων είναι οι κόμβοι του γραφήματος. Αρχικά, κάθε κόμβος αποτελεί ένα ξεχωριστό σύνολο. Στη συνέχεια εισάγουμε μια προς μια τις ακμές του γραφήματος, π.χ. με τη σειρά με την οποία μας δίνονται στην είσοδο, και για κάθε ακμή $\{u, v\}$ εκτελούμε ένωση(u,v).

Αλγόριθμος αρχικοποίηση (V)

Εκτελούμε νέο σύνολο (x) για κάθε κορυφή $x \in V$.

Αλγόριθμος εισαγωγή ακμής (x, y)

Εκτελούμε ένωση(x, y).

Αλγόριθμος συνεκτικότητα (V, E)

3. Εκτελούμε αρχικοποίηση(V)
4. Για κάθε ακμή $\{u, v\}$ του E
5. Εκτελούμε εισαγωγή ακμής (u, v)

Αφού ολοκληρωθεί η επεξεργασία των ακμών του γραφήματος, μπορούμε να απαντήσουμε γρήγορα αν οποιαδήποτε ζεύγος κόμβων x και y συνδέεται με κάποιο μονοπάτι στο γράφημα G. Αρκεί να συγκρίνουμε τους αντιπροσώπους των συνόλων που περιέχουν τους x και y.

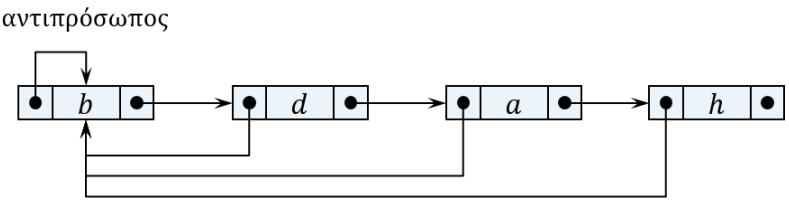
Αλγόριθμος συνδέονται (x, y)

3. Εκτελούμε $k =$ εύρεση(x) και $l =$ εύρεση(y).
4. Αν $k = l$ απαντάμε «ναι» διαφορετικά απαντάμε «όχι».

Το πλεονέκτημα αυτής της μεθόδου έναντι της οριζόντιας ή καθοδικής διερεύνησης είναι ότι μπορεί να χειριστεί την εισαγωγή νέων ακμών στο γράφημα G. Όπως βλέπουμε στον Πίνακα 11.1, αν χρησιμοποιήσουμε την υλοποίηση της δομής ξένων συνόλων με ανοδικά δένδρα με σταθμισμένη ένωση και συμπίεση διαδρομής, τότε μπορούμε να εκτελέσουμε μια ακολουθία m πράξεων εισαγωγής ακμών και ερωτημάτων συνδέονται(x, y) σε ένα γράφημα με n κόμβους σε συνολικό χρόνο $O((m + n)\alpha(m + n, n))$.

11.3 Δομή Ξένων Συνόλων με Συνδεδεμένες Λίστες

Μια απλή ιδέα είναι να αναπαραστήσουμε κάθε σύνολο S της συλλογής C με μια συνδεδεμένη λίστα, όπου κάθε κόμβος της λίστας αποθηκεύει ένα στοιχείο του S. Τα στοιχεία του συνόλου S μπορούν να αποθηκεύονται σε αυθαίρετη σειρά, αλλά θα πρέπει να ξεχωρίσουμε ένα από αυτά ως τον αντιπρόσωπο του συνόλου ο οποίος τοποθετείται στην πρώτη θέση της λίστας. Για να μπορούμε να εκτελέσουμε τη λειτουργία της ένωσης, θα πρέπει κάθε κόμβος της λίστας να έχει πρόσβαση στο πρώτο στοιχείο. Έτσι, κάθε κόμβος της λίστας, εκτός από την αναφορά στον επόμενο κόμβο, διαθέτει και μια αναφορά στον πρώτο κόμβο.



Εικόνα 11.49: Αναπαράσταση του συνόλου $S = \{a, b, d, h\}$ με συνδεδεμένη λίστα. Στην αρχή της λίστας τοποθετούμε τον αντιπρόσωπο του συνόλου που στην προκειμένη περίπτωση είναι το b.

Για να δημιουργήσουμε ένα νέο μονοσύνολο $\{v\}$, δημιουργούμε ένα νέο κόμβο λίστας x στον οποίο τοποθετούμε το στοιχείο v.

Αλγόριθμος νέο σύνολο(v)

1. Δημιουργούμε ένα νέο κόμβο x ο οποίος αποθηκεύει το στοιχείο v.
2. Θέτουμε επόμενος$(x) =$ κενός και αντιπρόσωπος$(x) = x$.
3. Επιστρέφουμε τον κόμβο x.

Η λειτουργία εύρεση(v) γίνεται άμεσα με χρήση της αναφοράς στον αντιπρόσωπο. Υποθέτουμε ότι δοθέντος του στοιχείου v έχουμε άμεση πρόσβαση στον κόμβο της συνδεδεμένης λίστας που περιέχει το v.

Αλγόριθμος εύρεση(v)

1. Έστω x ο κόμβος που περιέχει το στοιχείο v.
2. Ορίζουμε τον κόμβο $p =$ αντιπρόσωπος(x).
3. Επιστρέφουμε το στοιχείο του κόμβου p.

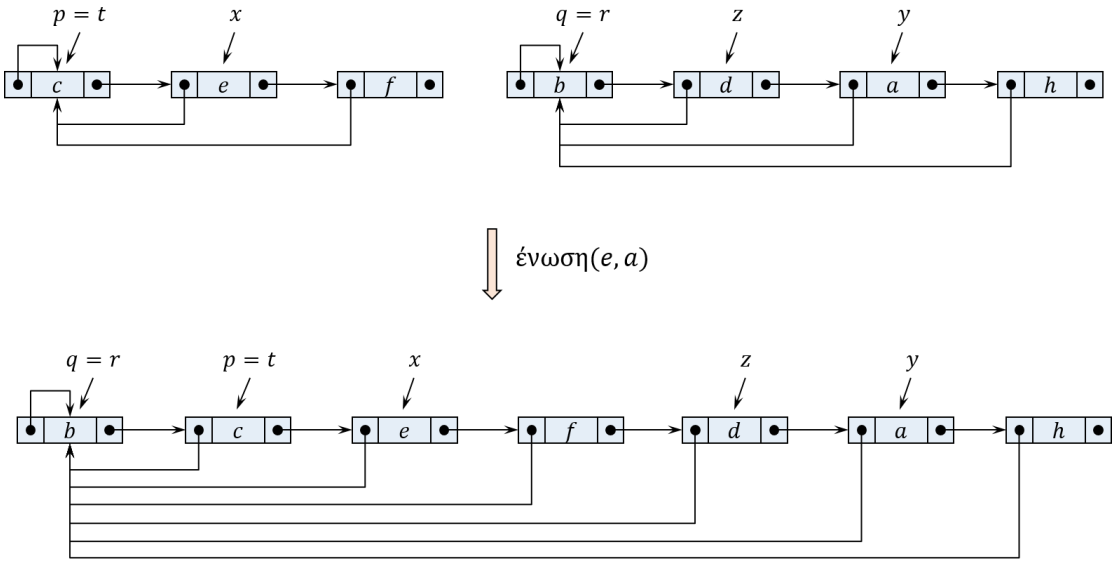
Η λειτουργία ένωση(v, u) είναι η πιο περίπλοκη σε αυτή τη δομή, καθώς απαιτεί τη συγχώνευση των συνδεδεμένων λιστών, οι οποίες περιέχουν τα στοιχεία u και v. Υπάρχουν διάφοροι τρόποι να γίνει αυτή η συγχώνευση, αλλά δεν είναι όλοι εξίσου αποδοτικοί. Για παράδειγμα, θα μπορούσαμε να διατρέξουμε τη λίστα του v, για να βρούμε τον τελευταίο κόμβο της και να τον συνδέσουμε με τον πρώτο κόμβο της λίστας του u. Στη συνέχεια, πρέπει να διατρέξουμε κάθε κόμβο x στη λίστα του u, για να αλλάξουμε την αναφορά αντιπρόσωπος(x), έτσι ώστε να δείχνει τον πρώτο κόμβο της λίστας του v. Αυτή η μέθοδος προσπελαύνει όλους τους κόμβους των δύο συνδεδεμένων λιστών, με αποτέλεσμα να εκτελεί μια ακολουθία $n - 1$ ενώσεων σε χρόνο $O(n^2)$ στη χειρότερη περίπτωση.

Με μια πιο προσεκτική ματιά μπορούμε να κάνουμε την ένωση σε χρόνο ανάλογο του πλήθους των στοιχείων του μικρότερου συνόλου. Η ιδέα είναι να παρεμβάλουμε τη μικρότερη λίστα ανάμεσα στους δύο πρώτους κόμβους της μεγαλύτερης λίστας.

Αλγόριθμος ένωση(v, u)

1. Έστω x και y οι κόμβοι που περιέχουν τα στοιχεία v και u αντίστοιχα.
2. Ορίζουμε τους κόμβους $p =$ αντιπρόσωπος(x) και $q =$ αντιπρόσωπος(y).
3. Αν $p = q$, τότε επιστρέφουμε τον κόμβο p.
4. Διαφορετικά, έστω r ο κόμβος με τον αντιπρόσωπο του μεγαλύτερου συνόλου και έστω t ο κόμβος με τον αντιπρόσωπο του άλλου συνόλου. Σε περίπτωση ισοπαλίας, έστω $r = p$.
5. Ορίζουμε $z =$ επόμενος(r).

6. Διατρέχουμε τη λίστα με πρώτο κόμβο τον t και για κάθε κόμβο x θέτουμε $\text{αντιπρόσωπος}(x) = r$. Αν ο x είναι ο τελευταίος κόμβος της λίστας, τότε θέτουμε επιπλέον $\text{επόμενος}(x) = z$.
7. Θέτουμε $\text{επόμενος}(r) = t$.
8. Επιστρέφουμε τον κόμβο r.



Εικόνα 11.50: Ένωση δύο συνόλων τα οποία αναπαριστούμε με συνδεδεμένες λίστες.

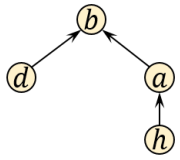
Ιδιότητα 11.1 Η υλοποίηση της δομής ξένων συνόλων με συνδεδεμένες λίστες επιτυγχάνει τους ακόλουθους χρόνους εκτέλεσης στη χειρότερη περίπτωση: $O(1)$ για τις λειτουργίες της κατασκευής νέου συνόλου και της εύρεσης και $O(n')$ για την λειτουργία της ένωσης, όπου n' το πλήθος των στοιχείων του μικρότερου συνόλου από τα δύο σύνολα που συμμετέχουν στην ένωση.

Ιδιότητα 11.2 Έστω ότι εκτελούμε μια ακολουθία από $n - 1$ ενώσεις συνόλων. Ο συνολικός χρόνος εκτέλεσης όλων των ενώσεων είναι $O(n \log n)$

Ας θεωρήσουμε την εκτέλεση της λειτουργίας $\text{ένωση}(v, u)$, υποθέτοντας χωρίς βλάβη της γενικότητας ότι το σύνολο V, που περιέχει το στοιχείο v, έχει τουλάχιστον τόσα στοιχεία όσα το σύνολο U, που περιέχει το στοιχείο u. Μπορούμε να παρατηρήσουμε πρώτα ότι όλα τα βήματα της ένωσης, με εξαίρεση το βήμα 6, εκτελούνται σε σταθερό χρόνο. Το βήμα 6 εκτελείται σε χρόνο ανάλογο του πλήθους των στοιχείων του συνόλου U. Έστω x ένας κόμβος της λίστας του συνόλου U. Το σύνολο W, το οποίο προκύπτει από την ένωση, έχει τουλάχιστον $2|U|$ στοιχεία, άρα κάθε φορά που προσπελαύνουμε τον κόμβο x δημιουργούμε ένα σύνολο με διπλάσια στοιχεία. Αυτό σημαίνει ότι, αν ο κόμβος x ενημερωθεί j φορές τότε βρίσκεται σε ένα σύνολο με τουλάχιστον 2^j στοιχεία. Αφού το πλήθος των στοιχείων είναι n, έχουμε $n \leq 2^j$, δηλαδή $j \leq \lg n$. Επομένως, η συνεισφορά ενός κόμβου σε όλες τις ενώσεις είναι το πολύ $\lg n$, άρα συνολικά $n \lg n$ για όλους τους κόμβους.

11.4 Δομή Ξένων Συνόλων με Ανοδικά Δένδρα

Η δομή που περιγράψαμε στην προηγούμενη ενότητα υποστηρίζει τη γρήγορη εύρεση, ωστόσο ο χρόνος μιας ένωσης μπορεί να είναι γραμμικός στη χειρότερη περίπτωση. Προκειμένου να βελτιώσουμε το χρόνο εκτέλεσης της ένωσης, χρειαζόμαστε μια αναπαράσταση των συνόλων η οποία θα επιτρέπει να γίνεται η ένωση, χωρίς να επεξεργαστούμε αναγκαστικά όλα τα στοιχεία ενός εκ των δύο συνόλων. Για το σκοπό αυτό, θα αναπτύξουμε μια δομή δεδομένων η οποία αναπαριστά το κάθε σύνολο με ένα δένδρο με ρίζα. Φυσιολογικά, κάθε κόμβος του δένδρου αποθηκεύει ένα στοιχείο του συνόλου με τον αντιπρόσωπο να βρίσκεται στη ρίζα του δένδρου.



Εικόνα 11.51: Αναπαράσταση του συνόλου $S = \{a, b, d, h\}$ με ανοδικό δένδρο. Ο αντιπρόσωπος του συνόλου, το στοιχείο b, βρίσκεται στη ρίζα.

Για να δημιουργήσουμε ένα νέο μονοσύνολο $\{v\}$, δημιουργούμε ένα νέο κόμβο-ρίζα ανδοδικού δένδρου x, στον οποίο τοποθετούμε το στοιχείο v.

Αλγόριθμος νέο σύνολο(v)

1. Δημιουργούμε ένα νέο κόμβο x ο οποίος αποθηκεύει το στοιχείο v.
2. Θέτουμε γονέας$(x) =$ κενός.
3. Επιστρέφουμε τον κόμβο x.

Η λειτουργία εύρεση(v) γίνεται με χρήση της αναφοράς στο γονέα του κάθε κόμβου. Ξεκινώντας από τον κόμβο ο οποίος περιέχει το στοιχείο v, ακολουθούμε το μονοπάτι προς τη ρίζα. Τέλος, επιστρέφουμε το στοιχείο που βρίσκεται στη ρίζα του ανοδικού δένδρου.

Αλγόριθμος εύρεση(v)

1. Έστω x ο κόμβος που περιέχει το στοιχείο v.
2. Ενόσω γονέας$(x) \neq$ κενός θέτουμε $x =$ γονέας(x).
3. Επιστρέφουμε το στοιχείο του κόμβου x.

Για την υλοποίηση της ένωσης θα χρειαστούμε μια βοηθητική μέθοδο, εύρεση ρίζας(x), η οποία βρίσκει τη ρίζα του ανοδικού δένδρου, που περιέχει ένα κόμβο x. Αυτό γίνεται με παρόμοιο τρόπο με τη λειτουργία εύρεση(v), με μόνη διαφορά ότι επιστρέφουμε την αναφορά στη ρίζα του δένδρου.

Αλγόριθμος εύρεση ρίζας(x)

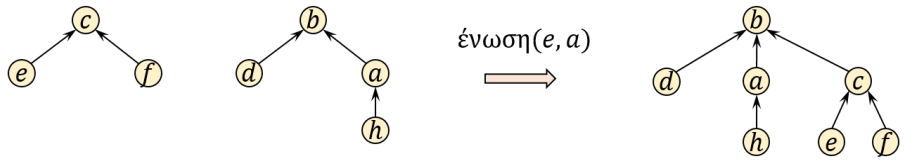
1. Ενόσω γονέας$(x) \neq$ κενός, θέτουμε $x =$ γονέας(x).

2. Επιστρέφουμε τη ρίζα x.

Η λειτουργία ένωση(v, u) μπορεί να υλοποιηθεί τώρα ως εξής. Πρώτα βρίσκουμε τις ρίζες των δένδρων που περιέχουν τα στοιχεία v και u. Στη συνέχεια, κάνουμε μια από αυτές τις ρίζες παιδί της άλλης. Η επιλογή της ρίζας του τελικού δένδρου είναι σημαντική και επηρεάζει το χρόνο εκτέλεσης όλων των λειτουργιών της δομής. Αν η επιλογή γίνει αυθαίρετα, π.χ. επιλέγοντας πάντα τη ρίζα του δένδρου που περιέχει το v, τότε μπορούμε εύκολα να κατασκευάσουμε δένδρα με ύψος $O(n)$. Έτσι, η δομή εκτελεί μια ακολουθία $n-1$ ενώσεων σε χρόνο $O(n^2)$ στη χειρότερη περίπτωση. Μια καλύτερη ιδέα, την οποία θα αναλύσουμε παρακάτω, είναι να κάνουμε τη ρίζα του μικρότερου δένδρου παιδί της άλλης ρίζας.

Αλγόριθμος ένωση(v, u)

1. Έστω x και y οι κόμβοι που περιέχουν τα στοιχεία v και u αντίστοιχα.
2. Ορίζουμε τους κόμβους (ρίζες) $p =$ εύρεση ρίζας(x) και $q =$ εύρεση ρίζας(y).
3. Αν $p = q$ τότε επιστρέφουμε τον κόμβο p.
4. Διαφορετικά έστω r η ρίζα του μεγαλύτερου συνόλου και έστω t η ρίζα του άλλου συνόλου. Σε περίπτωση ισοπαλίας, έστω $r = p$.
5. Θέτουμε γονέας$(t) = r$ και πλήθος$(r) =$ πλήθος$(r) +$ πλήθος(t).
6. Επιστρέφουμε τον κόμβο r.



Εικόνα 11.52: Ένωση δύο συνόλων τα οποία αναπαριστούμε με ανοδικά δένδρα.

Ιδιότητα 11.3 Η δομή ξένων συνόλων με ανοδικά δένδρα για n στοιχεία δημιουργεί δένδρα με ύψος το πολύ $\lg n$.

Θα δείξουμε την ιδιότητα με επαγωγή ως προς το πλήθος των ενώσεων. Αρχικά, κάθε στοιχείο αποτελεί ένα ξεχωριστό σύνολο, το οποίο αναπαρίσταται από ένα δένδρο με μόνο κόμβο τη ρίζα, δηλαδή με ύψος μηδέν. Άρα, η βάση της επαγωγής ισχύει, αφού $\lg 1 = 0$. Ας υποθέσουμε ότι η ιδιότητα ισχύει για σύνολα με το πολύ k στοιχεία. Για το επαγωγικό θέμα, θεωρούμε ένα σύνολο W με $k+1$ στοιχεία, το οποίο προκύπτει από την ένωση δύο ξένων συνόλων V και U με κ και λ στοιχεία, αντίστοιχα. Άρα, έχουμε $k+1 = \kappa + \lambda \leq \kappa + \kappa = 2\kappa$. Θα δείξουμε ότι το βάθος του κάθε στοιχείου στο δένδρο του W είναι το πολύ ίσο με $\lg 2\kappa \leq \lg(k+1)$, το οποίο αποδεικνύει το επαγωγικό βήμα. Χωρίς βλάβη της γενικότητας, μπορούμε να υποθέσουμε ότι το V έχει τουλάχιστον το ίδιο πλήθος στοιχείων με το U. Κάθε σύνολο που συμμετέχει σε μία ένωση έχει τουλάχιστον ένα στοιχείο, επομένως ισχύει $1 \leq \kappa \leq \lambda \leq k$. Μετά την ένωση, η ρίζα του V γίνεται παιδί της ρίζας του U και το βάθος των στοιχείων του U δεν αλλάζει. Έστω v ένα στοιχείο του V. Από την επαγωγική υπόθεση έχουμε ότι το βάθος του v στο δένδρο του V ήταν το πολύ $\lg \kappa$. Μετά την ένωση το βάθος αυξάνει κατά ένα και, άρα, γίνεται το πολύ $\lg \kappa + 1 \leq \lg \kappa + \lg 2 = \lg 2\kappa$.

11.4.1 Συμπίεση Διαδρομής

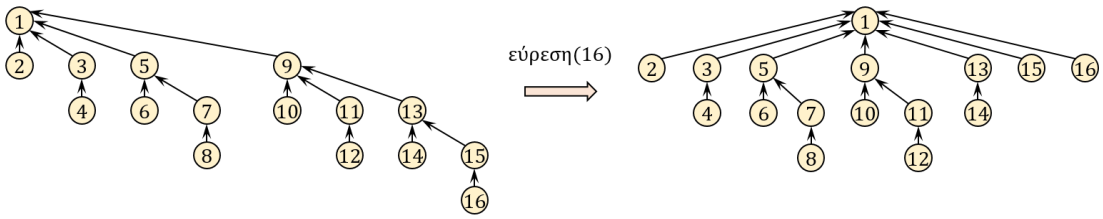
Όπως έχουμε ήδη αναφέρει, στην αναπαράσταση ενός συνόλου με ανοδικό δένδρο η συγκεκριμένη μορφή που έχει το δένδρο δεν επηρεάζει την ορθότητα των λειτουργιών, με την προϋπόθεση, βέβαια, ότι η ρίζα περιέχει τον αντιπρόσωπο του συνόλου. Χρησιμοποιήσαμε την ευελιξία αυτή στην πράξη της ένωσης, όπου επιλέγουμε να κάνουμε τη ρίζα του μικρότερου από τα δύο σύνολα παιδί της ρίζας του μεγαλύτερου συνόλου. Μπορούμε, όμως, να εκμεταλλευτούμε περαιτέρω την ευελιξία της αναπαράστασης, έτσι ώστε να επιταχύνουμε την πράξη της εύρεσης. Η ιδέα είναι ότι, αν κάθε κόμβος του δένδρου έχει μικρό βάθος, τότε η εύρεση θα γίνεται γρήγορα. Το να διατηρήσουμε αυτή τη συνθήκη γρήγορα μετά από κάθε ένωση είναι αδύνατο, επομένως θα αρκεστούμε σε αλλαγές γονέων τις οποίες μπορούμε να πραγματοποιήσουμε κατά την εκτέλεση της λειτουργίας εύρεση(v). Ξεκινώντας την εύρεση από τον κόμβο x που περιέχει το στοιχείο v, κάνουμε κάθε κόμβο στο μονοπάτι από τον x προς τη ρίζα r παιδί της r.

Αλγόριθμος συμπίεση(v)

1. Έστω x ο κόμβος που περιέχει το στοιχείο v.
2. Δημιουργούμε μια κενή στοίβα S.
3. Ενόσω γονέας$(x) \neq$ κενός εκτελούμε S.ώθηση(x) και θέτουμε $x =$ γονέας(x).
4. Θέτουμε ρίζα $= x$.
5. Ενόσω η S δεν είναι κενή
6. Θέτουμε $x = S.$απώθηση()
7. Θέτουμε γονέας$(x) =$ ρίζα

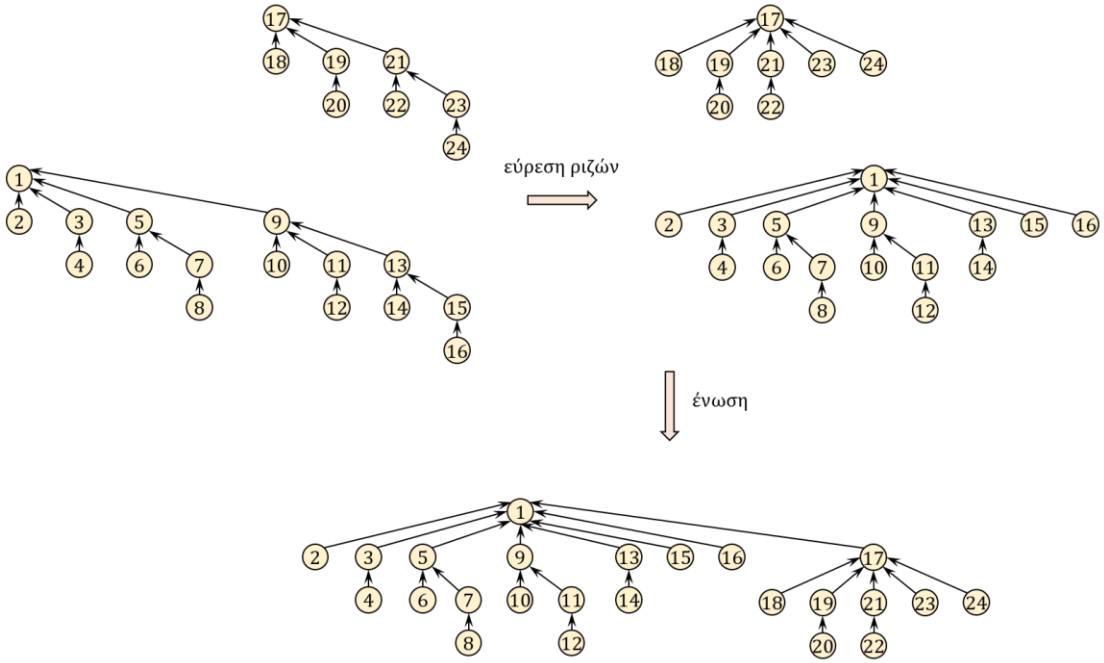
Αλγόριθμος εύρεση(v)

1. Έστω x ο κόμβος που περιέχει το στοιχείο v.
2. Εκτελούμε συμπίεση(x).
3. Επιστρέφουμε το στοιχείο του κόμβου γονέας(x).



Εικόνα 11.53: Εύρεση με συμπίεση διαδρομής. Κατά την αναζήτηση του αντιπροσώπου του συνόλου του 16 επισκεπτόμαστε τους κόμβους των στοιχείων 16, 15, 13 και 9 πριν καταλήξουμε στη ρίζα η οποία περιέχει τον αντιπρόσωπο του συνόλου, που είναι το 1. Στη συνέχεια, οι κόμβοι των 16, 15, 13 και 9 γίνονται παιδιά της ρίζας.

Ο αλγόριθμος εύρεσης ρίζας, τον οποίο χρειαζόμαστε για την ένωση, μπορεί να υλοποιηθεί με τον ίδιο τρόπο. Έτσι, η συμπίεση διαδρομής λαμβάνει χώρα και κατά την εκτέλεση της ένωσης, όπως φαίνεται στην Εικόνα 11.54.



Εικόνα 11.54: Ένωση συνόλων, όταν η εύρεση των ριζών γίνεται με συμπίεση διαδρομής. Στο παράδειγμα εκτελούμε τη λειτουργία ένωση(24,16). Πρώτα βρίσκουμε τις ρίζες των δένδρων τα οποία περιέχουν τα στοιχεία 24 και 16, που είναι κόμβοι με τα στοιχεία 17 και 1, αντίστοιχα. Η εύρεση στο δένδρο του 24 έχει ως αποτέλεσμα τη συμπίεση της διαδρομής 24, 23, 21, ενώ η εύρεση στο δένδρο του 16 συμπιέζει τη διαδρομή 16, 15, 13, 9. Τέλος, αφού το δένδρο με ρίζα το 17 έχει λιγότερα στοιχεία, τη θέτουμε ως παιδί της ρίζας του 1.

11.5 Υλοποίηση σε Java

Εδώ θα περιγράψουμε μια υλοποίηση της δομής ξένων συνόλων με ανοδικά δένδρα, με χρήση σταθμισμένης ένωσης και συμπίεσης διαδρομής. Υποθέτουμε, για απλούστευση, ότι τα αντικείμενα του συνόλου S που χειριζόμαστε έχουν μια ακέραιη ταυτότητα στο διάστημα $[1, n]$, όπου n το πλήθος των στοιχείων του S ($|S| = n$). Έτσι, θα αναφερόμαστε στα στοιχεία του S με τις ταυτότητες του, το οποίο είναι ισοδύναμο με το να θεωρούμε ότι $S = \{1,2, \ldots, n\}$.

Μπορούμε, τώρα, να αναπαραστήσουμε τα ανοδικά δένδρα της δομής απλά με ένα πίνακα ακεραίων parent, όπου η τιμή parent[i] είναι ο γονέας του αντικειμένου i στο ανοδικό δένδρο που περιέχει το αντικείμενο i. Στην περίπτωση όπου το i είναι ο αντιπρόσωπος του συνόλου του και, επομένως, βρίσκεται στη ρίζα του δένδρου που το περιέχει, υιοθετούμε τη σύμβαση parent[i] == i. Για παράδειγμα, στο τελικό δένδρο της Εικόνα 11.54 έχουμε parent[8] == 7 και parent[1] == 1. Θα χρειαστούμε έναν ακόμα πίνακα ακεραίων size, στον οποίο διατηρούμε το πλήθος των κόμβων σε κάθε ανοδικό δένδρο. Συγκεκριμένα, αν το στοιχείο i είναι ο αντιπρόσωπος ενός συνόλου, τότε η τιμή size[i] είναι το πλήθος των στοιχείων στο σύνολο που περιέχει το i. Αρχικά, προτού εκτελεστεί κάποια ένωση, κάθε στοιχείο i του S σχηματίζει ένα μονοσύνολο {i}. Έτσι, αρχικοποιούμε τη δομή θέτοντας parent[i] = i και size[i] = 1.

```
public class DisjointSetUnion {
```

```
    private int[] parent;   // γονείς στο ανοδικό δένδρο
    private int[] size;     // πλήθος απογόνων μιας ρίζας
    private int n;          // πλήθος στοιχείων της δομής

    DisjointSetUnion(int n) {
        this.n = n;
        parent = new int[n + 1];
        size = new int[n + 1];
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    /* οι υπόλοιπες μέθοδοι της κλάσης BinarySearchTree περιγράφονται παρακάτω */
}
```

Στη συνέχεια, περιγράφουμε τις υλοποιήσεις των λειτουργιών της εύρεσης και της ένωσης. Πρώτα θα ορίσουμε μια βοηθητική μέθοδο compress, η οποία χρειαστούμε έναν ακόμα πίνακα ακεραίων size, στον οποίο διατηρούμε το πλήθος των κόμβων σε κάθε ανοδικό δένδρο. Συγκεκριμένα, αν το στοιχείο i είναι ο αντιπρόσωπος ενός συνόλου, τότε η τιμή size[i] είναι το πλήθος των στοιχείων στο σύνολο που περιέχει το i.

```
    // αναδρομική συμπίεση διαδρομής
    private void compress(int v) {
        int p;
        if ((p = parent[v]) != v) {
            compress(p);
            parent[v] = parent[p];
        }
    }
```

Τώρα είμαστε σε θέση να υλοποιήσουμε τη λειτουργία της εύρεσης μέσω συμπίεσης διαδρομής. Έστω ότι εκτελούμε τη λειτουργία της εύρεσης για το στοιχείο v. Εκτελούμε τη μέθοδο compress(v), η οποία συμπιέζει το μονοπάτι από το v προς τη ρίζα. Αυτό έχει ως αποτέλεσμα ο αντιπρόσωπος του συνόλου που περιέχει το v να είναι ο γονέας του v, τον οποίο και επιστρέφουμε.

```
    // εύρεση με συμπίεση διαδρομής
    int find(int v) {
        compress(v);
        return parent[v];
    }
```

Τέλος, περιγράφουμε τη λειτουργία της ένωσης unite(v,u). Πρώτα υπολογίζουμε τους αντιπροσώπους p και q των συνόλων που περιέχουν τα στοιχεία v και u, αντίστοιχα. Αν έχουν τον ίδιο αντιπρόσωπο, τότε τα δύο σύνολα ταυτίζονται και δεν εκτελούμε καμία άλλη ενέργεια. Διαφορετικά κάνουμε τον αντιπρόσωπο του μικρότερου συνόλου παιδί του αντιπροσώπου του μεγαλύτερου από τα δύο σύνολα.

```
    // σταθμισμένη ένωση
    void unite(int v, int u) {
        int p = find(v);  // ρίζα του δένδρου που περιέχει το v
        int q = find(u);  // ρίζα του δένδρου που περιέχει το u
        if (p == q) {     // τα στοιχεία v και u βρίσκονται στο ίδιο σύνολο
            return;
        }
        if (size[p] > size[q]) {
            // εναλλαγή των p και q
```

```
            int t = p;
            p = q;
            q = t;
        }
        size[q] += size[p]++;
        parent[p] = q;
    }
```

Ασκήσεις

11.1 Πραγματοποιούμε την παρακάτω ακολουθία ενώσεων σε μια δομή σταθμισμένης γρήγορης ένωσης
(1,2), (2,3), (3,4), (5,6), (6,7), (7,8), (9,10), (10,11), (11,12), (13,14), (14,15), (15,16), (4,8), (12,16), (8,16)

α) Σχεδιάστε τη μορφή των δένδρων εύρεσης-ένωσης μετά από κάθε ένωση.

β) Σχεδιάστε τη μορφή των δένδρων εύρεσης-ένωσης, όταν χρησιμοποιούμε και συμπίεση διαδρομής.

11.2 Περιγράψτε μια ακολουθία $n-1$ ενώσεων χειρότερης περίπτωσης για τη δομή ξένων συνόλων με συνδεδεμένες λίστες, όταν η ένωση γίνεται, χωρίς να λαμβάνουμε υπόψη το μέγεθος της κάθε λίστας. Ποιος είναι ο συνολικός χρόνος εκτέλεσης για τις $n-1$ ενώσεις;

11.3 Περιγράψτε μια ακολουθία $n-1$ ενώσεων χειρότερης περίπτωσης για τη δομή ξένων συνόλων με ανοδικά δένδρα, όταν η ένωση γίνεται χωρίς να λαμβάνουμε υπόψη το πλήθος των απογόνων της κάθε ρίζας. Ποιος είναι ο συνολικός χρόνος εκτέλεσης για τις $n-1$ ενώσεις;

11.4 Θα μελετήσουμε μια εναλλακτική μέθοδο σταθμισμένης ένωσης στη δομή των ανοδικών δένδρων. Αντί να αποθηκεύουμε το πλήθος των απογόνων, διατηρούμε για κάθε κόμβο v μια μεταβλητή τάξη(v), η οποία δίνει ένα άνω φράγμα για το ύψος του v. Αρχικά κάθε κόμβος έχει τάξη μηδέν. Όταν εκτελούμε μια ένωση, συγκρίνουμε την τάξη των δύο ριζών p και q. Αν τάξη$(p) <$ τάξη(q), κάνουμε τον κόμβο p παιδί του κόμβου q. Αντίστοιχα, αν τάξη$(p) >$ τάξη(q), κάνουμε τον κόμβο q παιδί του κόμβου p. Τέλος, αν τάξη$(p) =$ τάξη(q), κάνουμε τον κόμβο p παιδί του κόμβου q και αυξάνουμε την τάξη του q κατά ένα.
α) Υλοποιήστε την παραπάνω μέθοδο και συγκρίνετε την απόδοση της σε σχέση με τη σταθμισμένη ένωση της Ενότητας 11.4.
β) Δείξτε ότι ένα ανοδικό δένδρο με ρίζα τον κόμβο v έχει τουλάχιστον $2^{\text{τάξη}(v)}$ κόμβους.

11.5 Δώστε μια μη αναδρομική υλοποίηση της μεθόδου συμπίεσης διαδρομής compress, όπου οι κόμβοι του μονοπατιού που συμπίεζεται αποθηκεύονται σε μια στοίβα. Τι μέγεθος πρέπει να έχει η στοίβα αυτή, αν υλοποιηθεί με πίνακα;

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Μποζάνης, Π. Δ. (2006). *Δομές Δεδομένων.* Εκδόσεις Τζιόλα.

Κεφάλαιο 12
Διαχείριση Μνήμης

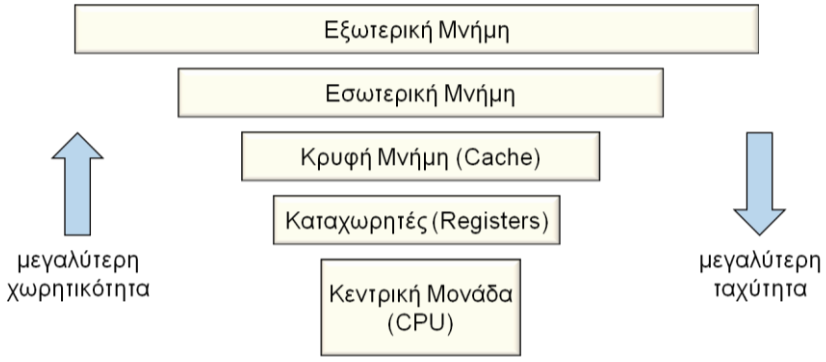
Περιεχόμενα

12.1 Ιεραρχία Μνήμης 239
12.2 Εξωτερική Μνήμη 241
12.2.1 Μοντέλο εξωτερικής μνήμης 241
12.2.2 Διατεταγμένο αρχείο με ευρετήριο 241
12.2.3 Β-δένδρα 242
12.3 Συλλογή Απορριμμάτων 244
Βιβλιογραφία 247

Οι δομές δεδομένων που είδαμε στα προηγούμενα κεφάλαια δεν θα μπορούν να χρησιμοποιηθούν, εάν δεν μπορούμε να αποθηκεύσουμε σε κάποιο μέσο τα δεδομένα. Για αυτό ακριβώς χρησιμοποιούμε τη μνήμη του υπολογιστή.

12.1 Ιεραρχία Μνήμης

Οι υπολογιστές, προκειμένου να διαχειριστούν σύνολα δεδομένων μεγάλου όγκου χρησιμοποιούν μια ιεραρχία από διαφορετικές μνήμες, οι οποίες διαφέρουν μεταξύ τους σε σχέση με το μέγεθός τους και την απόστασή τους από την Κεντρική Μονάδα Επεξεργασίας - ΚΜΕ (Central Processing Unit-CPU). Πιο κοντά στην ΚΜΕ βρίσκονται οι *καταχωρητές (registers)*, οι οποίοι χρησιμοποιούνται από την ίδια την ΚΜΕ. Η ταχύτητά τους είναι πολύ υψηλή, σε αντίθεση με τη χωρητικότητά τους, που είναι πολύ χαμηλή. Στο επόμενο επίπεδο της ιεραρχίας βρίσκεται η *κρυφή μνήμη (cache)*. Αυτού του είδους η μνήμη θεωρείται μεγαλύτερης χωρητικότητας από το σύνολο των καταχωρητών της ΚΜΕ, όμως μειονεκτεί ως προς το χρόνο που απαιτείται για την προσπέλασή της. Στο τρίτο επίπεδο της ιεραρχίας της μνήμης βρίσκεται η εσωτερική μνήμη, η οποία ονομάζεται *κύρια ή εσωτερική μνήμη (main memory)*. Η εσωτερική μνήμη είναι πιο αργή από την κρυφή μνήμη, όμως έχει μεγαλύτερη αποθηκευτική ικανότητα. Τέλος, στο υψηλότερο επίπεδο της ιεραρχίας βρίσκεται η *εξωτερική μνήμη (external memory)*. Ως εξωτερική μνήμη θεωρούνται οι μαγνητικοί-οπτικοί δίσκοι (CD or DVD drives) και οι μνήμες USB. Οι μνήμες αυτές, ενώ έχουν τη δυνατότητα αποθήκευσης μεγάλου όγκου δεδομένων, είναι πολύ αργές ως προς το χρόνο προσπέλασής τους. Στην Εικόνα 12.1, παρουσιάζεται η ιεραρχία μνήμης αποτελούμενη από τα τέσσερα επίπεδα που περιγράψαμε πιο πάνω.



Εικόνα 12.1: Η ιεραρχία της μνήμης.

Στις πιο πολλές εφαρμογές δύο είναι τα σημαντικά επίπεδα της ιεραρχίας: αυτό που χρησιμοποιείται για την αποθήκευση όλων των δεδομένων και το επίπεδο που βρίσκεται κάτω από αυτό. Η μεταφορά των δεδομένων από και προς τη μνήμη του υψηλότερου επιπέδου, η οποία είναι ικανή να αποθηκεύσει όλα τα δεδομένα, είναι το σημείο που καθορίζει ποια επίπεδα θα επιλεγούν, προκειμένου να επιτευχθεί η μικρότερη υπολογιστική συμφόρηση.

Σύμφωνα με τα παραπάνω, η σημασία του κάθε επιπέδου της ιεραρχίας της μνήμης εξαρτάται από το είδος του προβλήματος που προσπαθούμε να επιλύσουμε. Για ένα πρόβλημα το οποίο μπορεί να φορτωθεί ολόκληρο στην κύρια μνήμη τα δύο πιο σημαντικά επίπεδα μνήμης είναι η κρυφή μνήμη και η κύρια μνήμη. Ο χρόνος προσπέλασης της κύριας μνήμης είναι 10 έως 100 φορές μεγαλύτερος από αυτόν της κρυφής μνήμης. Έτσι, για αυτού του είδους τα προβλήματα, είναι επιθυμητό να μπορούν να πραγματοποιούνται περισσότερες προσπελάσεις στην κρυφή μνήμη. Στην περίπτωση που ένα πρόβλημα δεν μπορεί να φορτωθεί ολόκληρο στην κύρια μνήμη τα δύο πιο σημαντικά επίπεδα της ιεραρχίας της μνήμης είναι η κύρια μνήμη και η εξωτερική μνήμη. Οι διαφορές στην περίπτωση αυτή ως προς το χρόνο προσπέλασης είναι ακόμη πιο δραματικές, καθώς ο χρόνος προσπέλασης στις εξωτερικές μνήμες είναι περίπου 100.000 με 1.000.000 φορές μεγαλύτερος από αυτόν της κύριας μνήμης.

Για να γίνουν οι παραπάνω διαφορές πιο κατανοητές, ας δούμε κάποιες ενδεικτικές τιμές από τον Intel Haswell Mobile processor 2013. Οι καταχωρητές του επεξεργαστή παρέχουν τη γρηγορότερη δυνατή πρόσβαση: μπορούν να αποθηκεύσουν μερικά KB σε 1 κύκλο της Κεντρικής Μονάδας Επεξεργασίας. Η κρυφή μνήμη περιέχει 4 επίπεδα. Το πρώτο επίπεδο έχει μέγεθος 128 KB και η ταχύτητα πρόσβασης είναι 700 GB το δευτερόλεπτο, το δεύτερο έχει μέγεθος 1 MB με ταχύτητα πρόσβασης 200 GB το δευτερόλεπτο, το τρίτο 6 MB με ταχύτητα 100 GB το δευτερόλεπτο και τέλος το τέταρτο 128 MB με ταχύτητα 40 GB το δευτερόλεπτο. Η κύρια μνήμη έχει μέγεθος μερικά GB και ταχύτητα 10 GB το δευτερόλεπτο. Τέλος, ο δίσκος (εξωτερική μνήμη) έχει μέγεθος μερικά TB και ταχύτητα μόλις 600 MB το δευτερόλεπτο. Συμπεραίνουμε, λοιπόν, ότι οι προσπελάσεις στην εξωτερική μνήμη πρέπει να είναι όσο το δυνατόν λιγότερες.

Για να καταλάβουμε καλύτερα τις παραπάνω αναλογίες, ως προς την τάξη μεγέθους, φανταστείτε το εξής σενάριο: ένας φοιτητής που ζει στα Ιωάννινα θέλει να ζητήσει από τους γονείς του, που μένουν στην Αθήνα, να του στείλουν χρήματα. Εάν ο φοιτητής αυτός επιλέξει

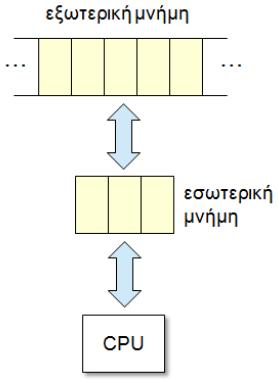
να στείλει μήνυμα στους γονείς του με email, τότε το μήνυμα θα φτάσει σε αυτούς σε περίπου 5 δευτερόλεπτα. Φανταστείτε ότι αυτού του είδους η επικοινωνία αντιστοιχεί με την επικοινωνία της ΚΜΕ με την εσωτερική μνήμη. Η προσπέλαση μιας εξωτερικής μνήμης που είναι 500.000 φορές πιο αργή, στην περίπτωση του φοιτητή, είναι να μεταφέρει το μήνυμα στους γονείς του ταξιδεύοντας με τα πόδια μέχρι την Αθήνα. Συμπεραίνουμε, ότι οι προσπελάσεις στην εξωτερική μνήμη πρέπει να είναι όσο το δυνατόν λιγότερες.

12.2 Εξωτερική Μνήμη

Πολλές σημαντικές εφαρμογές διαχειρίζονται ένα μεγάλο όγκο δεδομένων που είναι αποθηκευμένα σε εξωτερική μνήμη. Στις περιπτώσεις αυτές πρέπει να λάβουμε υπόψη το χρόνο που απαιτείται για τη μεταφορά από και προς την εξωτερική μνήμη.

12.2.1 Μοντέλο εξωτερικής μνήμης

Πρόκειται για ένα απλοποιημένο μοντέλο ανάλυσης αλγορίθμων. Η εξωτερική μνήμη διαιρείται σε σελίδες και μία σελίδα περιέχει ένα μεγάλο αριθμό δεδομένων (Εικόνα 12.2). Θεωρούμε ότι ο χρόνος των λειτουργιών εισόδου/εξόδου που απαιτείται για την ανάγνωση μιας σελίδας από την εξωτερική μνήμη είναι πολύ μεγαλύτερος από το χρόνο επεξεργασίας των δεδομένων της σελίδας.

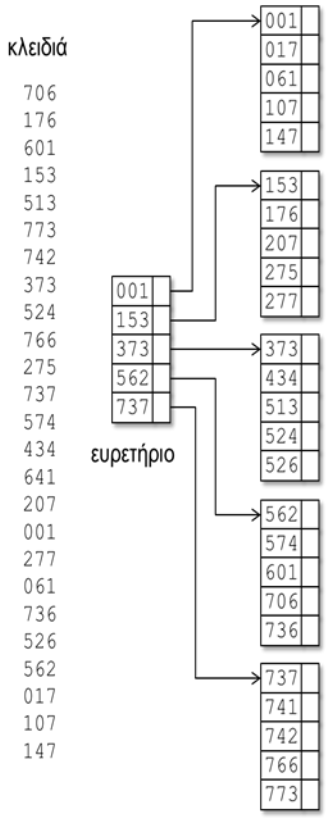


Εικόνα 12.2: Μεταφορά δεδομένων από την εξωτερική μνήμη στην Κεντρική Μονάδα Επεξεργασίας (CPU) μέσω της εσωτερικής μνήμης.

Σημειώνεται και πάλι ότι μια τυχαία προσπέλαση στην εξωτερική μνήμη είναι πολύ ακριβή σε σχέση με μια τυχαία προσπέλαση στην εσωτερική μνήμη (π.χ. 100000 φορές πιο αργή σε τυπικά συστήματα).

12.2.2 Διατεταγμένο αρχείο με ευρετήριο

Τα κλειδιά βρίσκονται σε ακολουθιακή διάταξη στις σελίδες, ενώ το ευρετήριο οδηγεί προς το μικρότερο κλειδί κάθε σελίδας (Εικόνα 12.3). Σε αυτήν την περίπτωση αρκεί να αναζητήσουμε το αμέσως μικρότερο κλειδί στο ευρετήριο, από το οποίο θα ανακατευθυνθούμε στη σελίδα που ενδεχομένως περιέχει το κλειδί που αναζητούμε. Εάν το ευρετήριο χωρά σε μία σελίδα της μνήμης, τότε αρκούν δύο προσβάσεις σε σελίδες της μνήμης, για να βρούμε το κλειδί που μας ενδιαφέρει.



Εικόνα 12.3: Αναπαράσταση της αποθήκευσης ενός διατεταγμένου αρχείου με 25 κλειδιά.

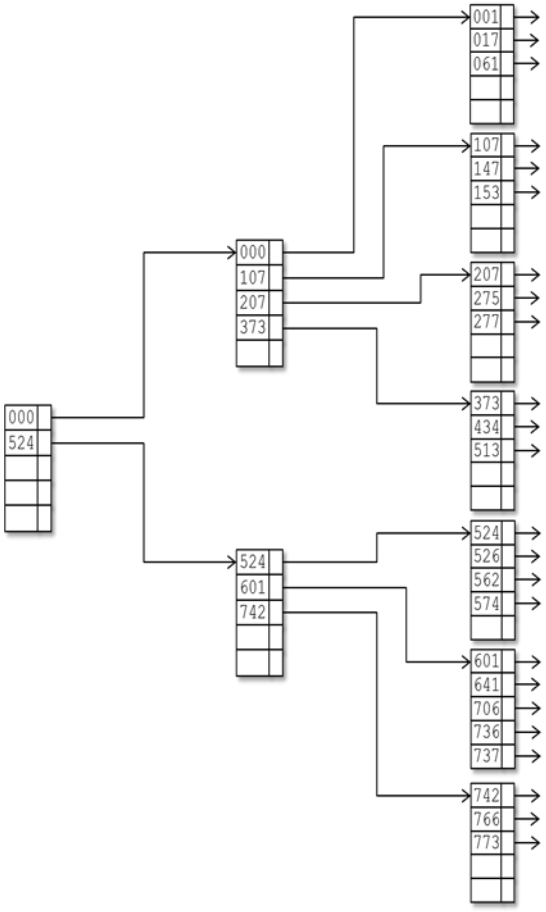
Βέβαια, σε περίπτωση που μας ενδιαφέρει να προσθέσουμε ένα νέο κλειδί, θα πρέπει να ανακατασκευάσουμε ολόκληρη τη δομή.

12.2.3 B-δένδρα

Ένα (a, b)-δένδρο είναι δένδρο αναζήτησης πολλαπλής διακλάδωσης όπου:

- a, b ακέραιοι με $a \geq 2$ και $b > a$ (συνήθως $b \geq 2 \cdot a$)
- η ρίζα έχει $d - 1$ κλειδιά και d παιδιά, όπου $2 \leq d \leq b$
- οι υπόλοιποι εσωτερικοί κόμβοι έχουν $t - 1$ κλειδιά και t παιδιά, όπου $a \leq t \leq b$
- οι κενοί κόμβοι (φύλλα) ισαπέχουν από τη ρίζα (δηλ., βρίσκονται στο ίδιο επίπεδο).

Ένα B-δένδρο βαθμού m είναι ένα (a, b)-δένδρο με $a = \lceil m/2 \rceil$ και $b = m$. Στην Εικόνα 12.4, παρουσιάζεται ένα B-δένδρο βαθμού 5.



Εικόνα 12.4: Ένα B-δένδρο βαθμού 5.

12.3 Συλλογή Απορριμμάτων

Η διαδικασία αιτήματος δέσμευσης μιας περιοχής μνήμης βασίζεται στον εντοπισμό ενός κατάλληλου-μεγέθους και μη-κατειλημμένου χώρου στη μνήμη (block). Τα αιτήματα που αφορούν τη δέσμευση μια περιοχής μνήμης ικανοποιούνται μέσω της απόκτησης τμημάτων μνήμης μέσα σε έναν μεγάλο χώρο μνήμης, που αποκαλείται Σωρός (Heap). Έτσι, σε μια δεδομένη χρονική στιγμή, ορισμένα τμήματα του Σωρού είναι σε χρήση (δεσμευμένα), ενώ κάποια άλλα είναι διαθέσιμα για χρήση, ώστε να δεσμευτούν στο μέλλον. Ωστόσο, περιοχές μνήμης που δεσμεύτηκαν στο παρελθόν και πλέον δε χρησιμοποιούνται δημιουργούν κορεσμό στην πεπερασμένη μνήμη ενός συστήματος, με αποτέλεσμα να δυσχεραίνεται η ικανοποίηση αιτημάτων για νέα δεδομένα, για τα οποία απαιτείται να δεσμευτεί χώρος. Έτσι, στις περιοχές του Σωρού μνήμης οι οποίες περιέχουν μη χρησιμοποιούμενα στοιχεία (θα αναφερόμαστε σε αυτά ως απορρίμματα) θα πρέπει να εκτελεστεί μια διαδικασία συλλογής απορριμμάτων (garbage collection).

Η διαδικασία της συλλογής απορριμμάτων αναφέρεται στην αποδέσμευση μνήμης για δεδομένα (μεταβλητές, αντικείμενα, κλπ.) τα οποία δε χρησιμοποιούνται και, ως εκ τούτου, δεν υπάρχει λόγος να δεσμεύεται μνήμη για τη διατήρησή τους. Η συλλογή απορριμμάτων σε γλώσσες προγραμματισμού, όπως για παράδειγμα οι γλώσσες C και C++, αποτελεί ευθύνη του προγραμματιστή, ενώ, αντίθετα, στη γλώσσα προγραμματισμού Java η διαδικασία αυτή είναι ενσωματωμένη στη γλώσσα από τους σχεδιαστές της, λειτουργώντας στο περιβάλλον εκτέλεσής της (run-time environment).

Ας παρουσιάσουμε, όμως, πιο αναλυτικά τη διαδικασία. Η μνήμη για τα αντικείμενα δεσμεύεται από τον Σωρό Μνήμης (Memory Heap), όπου χώρος για τις μεταβλητές ενός προγράμματος Java ανατίθεται στις στοίβες των μεθόδων (μια ανά τρέχον νήμα). Συγκεκριμένα, με τον όρο Σωρός Μνήμης αναφερόμαστε σε μια περιοχή μνήμης όπου η μνήμη μπορεί να ανατίθεται αυθαίρετα. Σε αντίθεση με τη στοίβα, όπου η μνήμη δεσμεύεται και απελευθερώνεται κατά μια αυστηρά ορισμένη διάταξη, στο σωρό τα στοιχεία στα οποία έχει ανατεθεί χώρος απελευθερώνονται με ασύγχρονο τρόπο μεταξύ τους, καθώς κάθε στοιχείο και, ως εκ τούτου, ο χώρος τον οποίο καταλαμβάνει δύναται να αποδεσμευθεί κατά τη στιγμή που ο προγραμματιστής διαγράψει ρητά το δείκτη προς αυτό το στοιχείο. Δεδομένου ότι τα στιγμιότυπα των μεταβλητών μέσα στις στοίβες των μεθόδων δύνανται να αναφέρονται σε αντικείμενα στο Σωρό Μνήμης, όλες οι μεταβλητές και τα αντικείμενα στις στοίβες των μεθόδων καλούνται Πηγαία Αντικείμενα (root objects), ενώ όλα τα υπόλοιπα αντικείμενα, τα οποία μπορούν να προσπελαστούν μέσω ακολουθιακών αναφορών σε αντικείμενα, ονομάζονται Ενεργά Αντικείμενα (live objects) και αποτελώντας τα αντικείμενα τα οποία χρησιμοποιούνται από το κατ' εκτέλεση πρόγραμμα, ο χώρος τον οποίο καταλαμβάνουν δεν πρέπει να αποδεσμεύεται.

Εν γένει, ένα αντικείμενο θεωρείται απόρριμμα, όταν δεν υφίσταται καμία αναφορά σε αυτό. Ως εκ τούτου, μια καλή πρακτική θα ήταν να ιχνηλατήσουμε με κάποιο τρόπο τις αναφορές που γίνονται σε κάθε αντικείμενο, για παράδειγμα αναθέτοντας σε κάθε ένα εξ αυτών ένα ειδικό πεδίο το οποίο καλείται Μετρητής Αναφορών (reference count), το οποίο δεν είναι προσπελάσιμο παρά μόνο από την εικονική μηχανή της Java (JVM) . Η εικονική μηχανή της Java, αντιλαμβανόμενη ότι η μνήμη αρχίζει να εξαντλείται, ξεκινά να ανακαλεί τη μνήμη η οποία έχει ανατεθεί-δεσμευτεί από αντικείμενα τα οποία δε χρησιμοποιούνται πλέον (δεν είναι Ενεργά Αντικείμενα), επιστρέφοντας τελικά αυτήν τη μνήμη στη λίστα του ελεύθερου χώρου. Για την αποδέσμευση της μνήμης που καταλαμβάνεται από αντικείμενα στα οποία δε γίνεται

καμία αναφορά (*συλλογή απορριμμάτων*), υπάρχουν αρκετοί αλγόριθμοι, με γνωστότερο όλων τον αλγόριθμο *Mark and Sweep*.

Ο αλγόριθμος συλλογής απορριμμάτων Mark and Sweep συσχετίζει ένα διακριτικό-σηματοδότη bit (mark-bit) με κάθε αντικείμενο, σημαίνοντας αν αυτό το αντικείμενο είναι ενεργό (live object) ή όχι. Στην απλούστερη έκδοση του αλγορίθμου Mark and Sweep, κάθε στοιχείο στη μνήμη έχει έναν διακριτικό-σηματοδότη (bit) ο οποίος έχει εκ προοιμίου δεσμευτεί, ώστε να αξιοποιηθεί εν συνεχεία αποκλειστικά για τη διαδικασία της συλλογής απορριμμάτων. Το διακριτικό αυτό (ενίοτε αναφέρεται και ως *flag*) έχει συνεχώς την τιμή "*καθαρισμένο*" (cleared) εκτός της διάρκειας της διαδικασίας συλλογής απορριμμάτων. Ας περιγράψουμε όμως τα στάδια που ακολουθούνται κατά την εκτέλεση του αλγορίθμου Mark and Sweep.

Ο αλγόριθμος Mark and Sweep περιλαμβάνει τρία στάδια:

- Το πρώτο στάδιο ελέγχει αν σε κάθε αντικείμενο του συστήματος έχει τεθεί το bit προσήμανσης και, εν συνεχεία, επισημαίνεται ως καθαρισμένο (cleared).
- Το δεύτερο στάδιο διασχίζει όλους τους δείκτες ξεκινώντας από τα Πηγαία Αντικείμενα ενός προγράμματος κι επισημαίνοντας (mark) κάθε αντικείμενο.
- Το τρίτο στάδιο προσπελαύνει γραμμικά το Σωρό, για μια ακόμη φορά, και απομακρύνει (sweep) όλα τα αντικείμενα τα οποία δεν έχουν επισημανθεί.

Κατά τα στάδια του αλγορίθμου, εκτελούνται δύο βασικές λειτουργίες, ονομαστικά: η επισήμανση (mark) και η απομάκρυνση (sweep). Κατά τη λειτουργία της επισήμανσης (mark), προσπελαύνεται δενδρικά ολόκληρο το σύνολο των Πηγαίων Αντικειμένων (root objects) και επισημαίνεται ως Ενεργό Αντικείμενο (live object) οποιοδήποτε αντικείμενο "δείχνεται" από (ή αντίστοιχα προς το οποίο δείχνει) ένα Πηγαίο Αντικείμενο (root object), καθώς επίσης με τον ίδιο τρόπο επισημαίνονται και όλα τα αντικείμενα στα οποία δείχνουν τα αντικείμενα αυτά. Κατά τη λειτουργία της απομάκρυνσης (sweep), εκτελείται μια διαδικασία σάρωσης ολόκληρης της μνήμης ανιχνεύοντας όλα τα ελεύθερα ή χρησιμοποιούμενα τμήματα (blocks), εκείνα που δεν έχουν επισημανθεί ως χρησιμοποιούμενα και, ως εκ τούτου, δε δείχνει σε αυτά κανένα Πηγαίο Αντικείμενο και αποδεσμεύεται αντίστοιχος ο χώρος μνήμης. Για τα τμήματα εκείνα τα οποία έχουν επισημανθεί ως χρησιμοποιούμενα το διακριτικό bit, που δρα ως σηματοδότης τους (flag), επαναπροσδιορίζεται ως "καθαρισμένο" (cleared) προετοιμάζοντάς το για την εκτέλεση του επόμενου κύκλου.

Έτσι, κατά την εκκίνηση της διαδικασίας συλλογής απορριμμάτων, η λειτουργία των νημάτων αναστέλλεται προσωρινά και όλοι οι διακριτικοί-σηματοδότες bit (mark-bits) κάθε αντικειμένου που βρίσκονται εκείνη τη στιγμή στο Σωρό Μνήμης σημαίνονται ως "καθαρισμένοι" (cleared). Εν συνεχεία, μέσα στις στοίβες της Java, για όλα τα τρέχοντα νήματα, επισημαίνουμε όλα τα Πηγαία Αντικείμενα (root objects), καθώς επίσης και τα αντικείμενα τα οποία αυτά δείχνουν, ως ενεργά (live objects).

Για την επίτευξη του στόχου αυτού μπορούμε για λόγους απόδοσης να αξιοποιήσουμε την έκδοση της εις βάθος διάσχισης (Depth First Search –DFS) για κατευθυνόμενα γραφήματα. Συγκεκριμένα, κάθε αντικείμενο στο Σωρό Μνήμης αποτελεί ένα κόμβο στο κατευθυνόμενο γράφημα, ενώ κατ' αντιστοιχία μια αναφορά από ένα αντικείμενο σε ένα άλλο συνιστά μια κατευθυνόμενη ακμή. Με τον τρόπο αυτόν υλοποιείται η λειτουργία της επισήμανσης (mark),

εκτελώντας κατευθυνόμενη εις βάθος διάσχιση (DFS) από κάθε Πηγαίο Αντικείμενο (root object) σε οποιοδήποτε αντικείμενο δείχνει αυτό, εντοπίζοντας και σημαίνοντας με τον τρόπο αυτό οποιοδήποτε Ενεργό Αντικείμενο (live object). Στο επόμενο βήμα, εκτελώντας τη λειτουργία της απομάκρυνσης (sweep), ελέγχουμε ολόκληρο το Σωρό Μνήμης, όπου, όταν βρεθεί ένα αντικείμενο το οποίο δεν έχει επισημανθεί, αποδεσμεύεται ο χώρος ο οποίος του έχει ανατεθεί. Έτσι, ο αλγόριθμος Mark and Sweep για την ανάκτηση του μη-χρησιμοποιούμενου χώρου απαιτεί χρόνο ανάλογο του αθροίσματος του αριθμού των Ενεργών Αντικειμένων (live objects), των αναφορών τους και του μεγέθους του Σωρού Μνήμης.

Εκτελώντας DFS ενδομνημιακά

Εκτελώντας εις βάθος διάσχιση (DFS) τη στιγμή που η μνήμη έχει αρχίσει να λιγοστεύει πρέπει να φροντίσουμε να μην απαιτήσουμε περισσότερη μνήμη για την εκτέλεση της διαδικασίας συλλογής απορριμμάτων. Η αναδρομική εκδοχή της εις βάθος διάσχισης (DFS) απαιτεί χώρο ανάλογο του αριθμού των κόμβων στο γράφημα, όπου οι κόμβοι εν προκειμένω αναπαριστούν τα Ενεργά Αντικείμενα στο Σωρό Μνήμης απαιτώντας μνήμη την οποία ενδεχομένως να μην είμαστε σε θέση να διαθέσουμε. Έτσι, καλούμαστε να χρησιμοποιήσουμε την ενδομνημιακή (in-place) εκδοχή της εις βάθος διάσχισης (DFS), παρά την αναδρομική, χρησιμοποιώντας σταθερό χώρο για την διεργασία αυτή. Συγκεκριμένα, για την ενδομνημιακή εκδοχή της DFS, επιλέγουμε να αναπαραστήσουμε την στοίβα αναδρομής με τις ακμές του γραφήματος (αναφορές σε αντικείμενα).

Συγκεκριμένα, στο δέντρο που αναπτύσσεται κατά την εκτέλεση της εις βάθος διάσχισης (DFS) αναστρέφουμε μια ακμή (u, v) από τη λίστα γειτνίασης του u ώστε να δείχνει στον προκάτοχό του (απαριθμώντας τις εξερχόμενες ακμές μέσω ενός *αναγνωριστικού μετρητή* προκειμένου να γνωρίζουμε ποιες εξ αυτών έχουν υποστεί μετατροπή), κάθε φορά που διασχίζουμε μια ακμή από έναν κόμβο u, τον οποίο έχουμε επισκεφθεί, σε ένα νέο κόμβο v, ενώ μπορούμε να αναστρέψουμε την ακμή, ώστε να δείχνει τον v επιστρέφοντας στον κόμβο u. Στο σημείο αυτό, εύλογα κάποιος μπορεί να διερωτάται περί του επιπρόσθετου χώρου ο οποίος απαιτείται για την αποθήκευση του *αναγνωριστικού μετρητή* που αναφέρθηκε παραπάνω και χρησιμοποιείται, για να επισημαίνονται οι ακμές οι οποίες έχουν αναστραφεί. Ωστόσο, όπως θα εξηγήσουμε και παρακάτω, ο ασυμπτωτικός χρόνος εκτέλεσης της ενδομνημιακής εκδοχής της εις βάθος διάσχισης (DFS) δε μεταβάλλεται χρησιμοποιώντας είτε την αλλαγή ακμών (αναστροφή) είτε τον *αναγνωριστικό μετρητή*.

Το ερώτημα αυτό απαντάται εύκολα σε διάφορες υλοποιήσεις της JVM, όπου τα αντικείμενα αναπαρίστανται ως συνθέσεις δύο πεδίων - αναφορών, όπου το πρώτο πεδίο εμπεριέχει μια αναφορά στον τύπο του αντικειμένου, ενώ το δεύτερο μια αναφορά σε ένα άλλο αντικείμενο. Έτσι, μια αποδοτική λύση θα ήταν να αξιοποιηθεί η δυνατότητα μεταβολής στο πρώτο πεδίο της σύνθεσης κατά την αναφορά σε ένα άλλο αντικείμενο (σημαίνει πως δημιουργείται ακμή στο γράφημα). Με την προσέγγιση αυτή, για δύο αντικείμενα u και v, όπου το u αναφέρεται στο v, αλλάζουμε το πρώτο πεδίο του u (περιέχει την αναφορά στον τύπο του u) με την αναφορά στο v, όπου καθίσταται δυνατή η ανίχνευση της ακμής που ανεστράφη, καθώς επίσης και τη θέση της ακμής στη λίστα γειτνίασης του u, κατά την επιστροφή μας στο u, αφού η αναφορά στο v βρίσκεται στη θέση της αναφοράς στον τύπο του u.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Κεφάλαιο 13
Αντισταθμιστική Ανάλυση

Περιεχόμενα

13.1 Αντισταθμιστική Ανάλυση 248

13.2 Μέθοδοι Αντισταθμιστικής Ανάλυσης 250

13.2.1 Η χρεωπιστωτική μέθοδος 250

13.2.2 Η ενεργειακή μέθοδος 251

Ασκήσεις 252

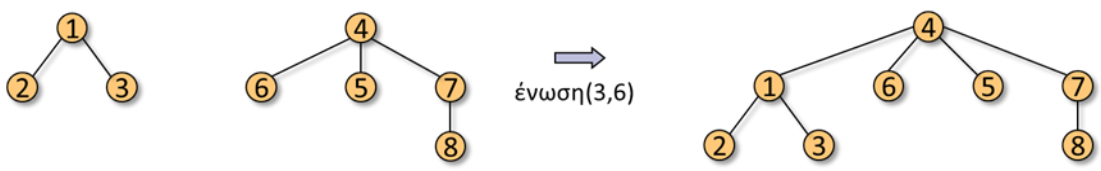
Βιβλιογραφία 254

Στο κεφάλαιο αυτό μελετάμε την Αντισταθμιστική Ανάλυση. Σε αντίθεση με τη ανάλυση πολυπλοκότητας στη χειρότερη περίπτωση, που σε πολλές περιπτώσεις μπορεί να υπερεκτιμά το κόστος μιας πράξης, η αντισταθμιστική ανάλυση βασίζεται στον υπολογισμό του μέσου κόστους μιας πράξης μετά από μια ακολουθία πράξεων, συχνά καταλήγοντας σε ακριβέστερες εκτιμήσεις της πολυπλοκότητας των πράξεων.

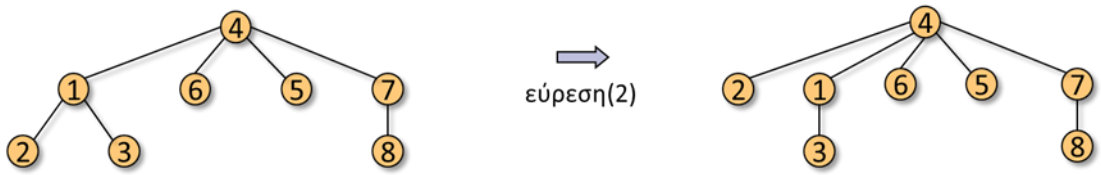
13.1 Αντισταθμιστική Ανάλυση

Ας θεωρήσουμε έναν αλγόριθμο Α ο οποίος χρησιμοποιεί μια δομή δεδομένων Δ. Κατά τη διάρκεια εκτέλεσης του αλγορίθμου Α, η δομή Δ πραγματοποιεί μια ακολουθία από πράξεις. Υπό κάποιες συνθήκες η εκτέλεση μιας τέτοιας πράξης μπορεί να είναι ιδιαίτερα ακριβή. Όμως, κάτι τέτοιο μπορεί να μη συμβαίνει συχνά, οπότε η ανάλυση χειρότερης περίπτωσης υπερεκτιμά το συνολικό κόστος μιας ακολουθίας από τέτοιες πράξεις. Αντίθετα, μπορούμε να θεωρήσουμε το συνολικό κόστος ολόκληρης της ακολουθίας, από το οποίο να υπολογίσουμε το μέσο κόστος μιας τέτοιας πράξης, το οποίο να δίνει πιο αντιπροσωπευτική εκτίμηση του κόστους της πράξης.

Για παράδειγμα, ας θεωρήσουμε το πρόβλημα της Εύρεσης-Ένωσης που είδαμε νωρίτερα. Μια λύση που μελετήσαμε βασιζόταν στη χρήση δενδρικών δομών στα οποία εφαρμόζαμε «σταθμισμένη ένωση με συμπίεση διαδρομής» (Στις Εικόνες 13.1 και 13.2 φαίνονται οι εκτελέσεις μίας πράξης ένωσης και μίας εύρεσης, αντίστοιχα.)



Εικόνα 13.1: Εκτέλεση της πράξης της ένωσης των στοιχείων 3 και 6 που ανήκουν στα δύο δένδρα στα αριστερά.



Εικόνα 13.2: Εκτέλεση της πράξης της εύρεσης του συνόλου που περιέχει το στοιχείο 2.

Ο χρόνος χειρότερης περίπτωσης για μια πράξη εύρεσης ή ένωσης είναι ανάλογος του ύψους του δένδρου που αναπαριστά το σύνολο το οποίο περιέχει το στοιχείο που μας ενδιαφέρει, το οποίο για n στοιχεία συνολικά μπορεί να είναι $\Theta(\log n)$. Όμως, ο συνολικός χρόνος εκτέλεσης m πράξεων εύρεσης-ένωσης αποδεικνύεται ότι είναι $O(m\,\alpha(m,n))$, με αποτέλεσμα ο μέσος χρόνος εκτέλεσης μιας τέτοιας πράξης να είναι $O(\alpha(m,n))$, το οποίο αυξάνεται με ιδιαίτερα αργότερο ρυθμό από ό,τι ο λογάριθμος του n.

Συνοψίζοντας, έστω $f(n)$ το κόστος μιας πράξης στη χειρότερη πείπτωση. Τότε ο συνολικός χρόνος για m πράξεις είναι $O(m\,f(n))$. Ωστόσο, σε κάποιες περιπτώσεις, η ίδια πράξη μπορεί να έχει διαφορετικό κόστος ανάλογα με τη στιγμή που εκτελείται. Έτσι, αν και το κόστος μιας πράξης στη χειρότερη περίπτωση μπορεί να είναι μεγάλο, το *μέσο κόστος* ανά πράξη σε *οποιαδήποτε ακολουθία πράξεων* μπορεί να είναι αρκετά μικρότερο. Στην *Αντισταθμιστική Ανάλυση*, λαμβάνουμε το *αντισταθμιστικό κόστος* ανά πράξη, δηλαδή, το μέσο κόστος εκτέλεσης μιας πράξης, όταν εκτελούμε μια ακολουθία πράξεων χειρότερης περίπτωσης.

Παράδειγμα 1. Διαχείριση στοίβας.

Ας θεωρήσουμε μια στοίβα S στην οποία εκτός από τις λειτουργίες ώθησης $S.\text{push}(x)$ και απώθησης $S.\text{pop}(\,)$ έχουμε και μια λειτουργία πολλαπλών απωθήσεων $S.\,\text{multipop}(k)$ η οποία αφαιρεί από τη στοίβα S τα k πρώτα στοιχεία στην κορυφή της. Δεν ειναι δύσκολο να παρατηρήσει κανείς ότι ο χρόνος χειρότερης περίπτωσης για την εκτέλεση μιας πράξης ώθησης ή απώθησης είναι $\Theta(1)$, ενώ η εκτέλεση μιας πράξης πολλαπλής απώθησης ενδέχεται να απαιτήσει έως και $\Theta(k)$ χρόνο. Τότε, όμως, ο συνολικός χρόνος εκτέλεσης μιας ακολουθίας N πράξεων στη στοίβα S μπορεί να είναι $\Theta(N\,k)$, καθώς οι περισσότερες από αυτές τις πράξεις μπορεί να είναι πολλαπλές απωθήσεις και καθεμία από αυτές να απαιτήσει $\Theta(k)$ χρόνο. Όμως, με πιο προσεκτική ανάλυση μπορούμε να παρατηρήσουμε ότι ο συνολικός χρόνος για τις N πράξεις είναι $O(N)$, οπότε το αντισταθμιστικό κόστος, δηλαδή ο μέσος χρόνος ανά πράξη, είναι μόνο $O(1)$.

Παράδειγμα 2. Επαύξηση δυαδικού μετρητή.

Έστω ένας δυαδικός μετρητής C με k δυαδικά ψηφία. Μια πράξη επαύξησης θέτει την τιμή C του μετρητή ίση με $C + 1 \pmod{2^k}$. Ο χρόνος για μία επαύξηση είναι ίσος με το πλήθος των δυαδικών ψηφίων τα οποία αλλάζουν τιμή. Στη χειρότερη περίπτωση ο χρόνος αυτός μπορεί να είναι $\Theta(k)$, οπότε ο συνολικός χρόνος για N επαυξήσεις μπορεί να είναι $\Theta(N\,k)$. Και στην περίπτωση αυτή, με πιο προσεκτική ανάλυση μπορούμε να δείξουμε ότι ο συνολικός χρόνος για N επαυξήσεις (ξεκινώντας με μηδενισμένο μετρητή) είναι $\Theta(N)$. Συνεπώς, το αντισταθμιστικό κόστος ανά πράξη είναι μόνο $O(1)$. Αρκεί να παρατηρήσουμε ότι το 1ο ψηφίο από το τέλος αλλάζει με κάθε επαύξηση, το 2ο αλλάζει με κάθε δεύτερη επαύξηση, το 3ο με κάθε τέταρτη επαύξηση κ.ο.κ. Σε μια ακολουθία N επαυξήσεων το i-οστό ψηφίο από το τέλος αλλάζει συνολικά $\left\lfloor \frac{N}{2^{i-1}} \right\rfloor$ φορές, οπότε το συνολικό πλήθος αλλαγών είναι

$$\sum_{i=1}^{k} \left\lfloor \frac{N}{2^{i-1}} \right\rfloor < 2N.$$

Δηλαδή, ο συνολικός χρόνος για N επαυξήσεις είναι $O(N)$.

13.2 Μέθοδοι Αντισταθμιστικής Ανάλυσης

Υπάρχουν 3 μέθοδοι αντισταθμιστικής ανάλυσης:

- Αθροιστική μέθοδος
- Χρεωπιστωτική μέθοδος
- Ενεργειακή μέθοδος

Στην ανάλυση της επαύξησης του δυαδικού μετρητή χρησιμοποιήσαμε την αθροιστική μέθοδο. Είναι σημαντικό να παρατηρήσουμε ότι σε αντίθεση με την αθροιστική μέθοδο, η χρεωπιστωτική μέθοδος και η ενεργειακή μέθοδος μπορούν να αποδώσουν *διαφορετικό αντισταθμιστικό κόστος σε διαφορετικούς τύπους πράξεων*.

13.2.1 Η χρεωπιστωτική μέθοδος

Σύμφωνα με την χρεωπιστωτική μέθοδο (γνωστή και ως μέθοδος του τραπεζίτη) αποδίδουμε σε κάθε πράξη έναν αριθμό από *πιστώσεις* (*credits*). Οι πιστώσεις αυτές χρησιμοποιούνται, για να αποπληρωθούν οι πράξεις. Όταν μια πράξη κοστίζει λιγότερο από την αντίστοιχη πίστωση, τότε το υπόλοιπο αποθηκεύεται σε κάποια αντικείμενα της δομής. Όταν μια πράξη κοστίζει περισσότερο από την αντίστοιχη πίστωση, τότε η υπολειπόμενη χρέωση καλύπτεται από αποθηκευμένες πιστώσεις. Αν οι πιστώσεις που αποδώσαμε στις πράξεις αρκούν, για να αποπληρώσουν *οποιαδήποτε ακολουθία πράξεων*, τότε το αντισταθμιστικό κόστος μιας πράξης είναι ίσο με τον αριθμό των πιστώσεων που της αποδώσαμε.

Ας επιστρέψουμε στο πρόβλημα της διαχείρισης λίστας. Υπενθυμίζουμε ότι το κόστος μιας ώθησης και μιας απώθησης είναι σταθερό, ενώ το κόστος μιας πολλαπλής απώθησης είναι $\min\{k, |S|\}$. Ας θεωρήσουμε ότι αναθέτουμε 2 μονάδες πίστωσης για κάθε ώθηση και 0 μονάδες πίστωσης σε κάθε απλή και πολλαπλή απώθηση. Πρέπει να δείξουμε ότι οι πιστώσεις αρκούν, για να αποπληρώσουν *οποιαδήποτε ακολουθία πράξεων*. Για κάθε ώθηση, η μία μονάδα πίστωσης πληρώνει για την εκτέλεση της ώθησης, ενώ η δεύτερη αποθηκεύεται στο στοιχείο που ωθήθηκε. Σε κάθε απλή η πολλαπλή απώθηση, η απομάκρυνση των αντικειμένων που απωθούνται απόπληρώνεται από τη μία μονάδα πίστωσης που είναι αποθηκευμένη σε κάθε ένα από αυτά τα αντικείμενα. Άρα, οι πιστώσεις που αποδόθηκαν επαρκούν για οποιαδήποτε ακολουθία πράξεων και, άρα ,ο συνολικός χρόνος για την εκτέλεση N πράξεων είναι $O(N)$.

Με παρόμοιο τρόπο μπορούμε να δείξουμε ότι ο συνολικός χρόνος που απαιτείται για N επαυξήσεις ενός δυαδικού μετρητή ξεκινώντας από το 0 είναι $O(N)$. Σημειώνουμε ότι η αλλαγή ενός δυαδικού ψηφίου απαιτεί σταθερό χρόνο. Ας θεωρήσουμε ότι αναθέτουμε 2 μονάδες πίστωσης σε κάθε αλλαγή ενός δυαδικού ψηφίου από 0 σε 1 και 0 μονάδες πίστωσης σε κάθε αλλαγή ενός δυαδικού ψηφίου από 1 σε 0. Συνεπώς, για κάθε αλλαγή από 0 σε 1, μία μονάδα πίστωσης χρησιμοποιείται για την αλλαγή αυτή, ενώ η δεύτερη μονάδα αποθηκεύεται

στο 1. Άρα, δεδομένου ότι ο δυαδικός μετρητής ξεκινά από την τιμή 0, σε κάθε στιγμή κάθε δυαδικό ψηφίου που είναι 1 φέρει μία μονάδα πίστωσης. Αυτή η μονάδα αρκεί για την αποπληρωμή τυχόν αλλαγής του δυαδικού ψηφίου ξανά σε 0. Για παράδειγμα, ας θεωρήσουμε ότι ο μετρητής έχει τη δυαδική τιμή 00101111. Η τιμή του μετρητή μετά από μία επαύξηση είναι 00110000. Συνολικά έχουν αλλάξει 5 δυαδικά ψηφία και, άρα, το κόστος της επαύξησης είναι 5. Αυτό αποπληρώνεται από τις 4 αποθηκευμένες μονάδες πίστωσης στα 4 δεξιότερα δυαδικά ψηφία της τιμής 00101111, που είναι 1 (και γίνονται 0) και από τη μία από τις 2 μονάδες πίστωσης λόγω της αλλαγής του 5ου από το τέλος δυαδικού ψηφίου από 0 σε 1. Η δεύτερη μονάδα αποθηκεύεται στο 1 (που είναι 5ο από το τέλος).

13.2.2 Η ενεργειακή μέθοδος

Σύμφωνα με την ενεργειακή μέθοδο (γνωστή και ως μέθοδος του φυσικού) αποδίδουμε στη δομή δεδομένων D μια *συνάρτηση δυναμικού* ή «δυναμικής ενέργειας» (*potential function*), που συνήθως συμβολίζεται με $\Phi(D)$ και λαμβάνει πραγματικές τιμές. Έστω D_0 η αρχική δομή δεδομένων και D_i η δομή μετά την i-οστή πράξη. Επίσης, έστω c_i το κόστος της i-οστής πράξης. Το *αντισταθμιστικό κόστος* της i-οστής πράξης είναι

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Τότε το συνολικό αντισταθμιστικό κόστος μετά από N πράξεις είναι:

$$\sum_{i=1}^{N} \hat{c}_i = \sum_{i=1}^{N} \left(c_i + \Phi(D_i) - \Phi(D_{i-1})\right) = \left(\sum_{i=1}^{N} c_i\right) + \Phi(D_N) - \Phi(D_0).$$

Θέλουμε $\Phi(D_N) \geq \Phi(D_0)$ έτσι ώστε $\sum_{i=1}^{N} c_i \leq \sum_{i=1}^{N} \hat{c}_i$.

Το αντισταθμιστικό κόστος $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ της i-οστής πράξης συνεπάγεται ότι:

- Αν $\Phi(D_i) - \Phi(D_{i-1}) > 0$, τότε η δομή συγκεντρώνει δυναμικό και $\hat{c}_i > c_i$.
- Αν $\Phi(D_i) - \Phi(D_{i-1}) < 0$, τότε η δομή χάνει δυναμικό και $\hat{c}_i < c_i$.

Δηλαδή, το κόστος μιας ακριβής πράξης αποπληρώνεται από τη διαφορά δυναμικού. Σημειώνουμε, επίσης, ότι, επειδή θέλουμε $\Phi(D_N) \geq \Phi(D_0)$ για οποιοδήποτε πλήθος N πράξεων, απαιτούμε να ισχύει ότι $\Phi(D_i) \geq \Phi(D_0)$ για κάθε $i = 1, 2, \ldots$.

Για το πρόβλημα της διαχείρισης στοίβας, επιλέγουμε

$$\Phi(S) = \text{πλήθος αντικειμένων στη στοίβα } S.$$

Για αρχικά κενή στοίβα S_0 έχουμε $\Phi(S_0) = 0$, το οποίο συνεπάγεται ότι για κάθε ακέραιο $i \geq 0$, $\Phi(S_i) \geq \Phi(S_0) = 0$, που με τη σειρά του συνεπάγεται ότι $\sum_{j=1}^{i} c_j \leq \sum_{j=1}^{i} \hat{c}_j$ για κάθε $i \geq 0$. Επομένως, το συνολικό αντισταθιστικό κόστος αποτελεί άνω φράγμα του συνολικού πραγματικού κόστους. Μένει να υπολογίσουμε το αντισταθμιστικό κόστος $\hat{c}_i$ για καθεμία από τις 3 πράξεις της στοίβας.

- Έστω ότι η i-οστή πράξη είναι ώθηση (push). Έχουμε ότι $c_i = 1$ και $\Phi(S_i) = \Phi(S_{i-1}) + 1$, επομένως $\hat{c}_i = c_i + \Phi(S_i) - \Phi(S_{i-1}) = 2$.
- Έστω ότι η i-οστή πράξη είναι (απλή) απώθηση (pop). Έχουμε ότι $c_i = 1$ και $\Phi(S_i) = \Phi(S_{i-1}) - 1$, επομένως $\hat{c}_i = c_i + \Phi(S_i) - \Phi(S_{i-1}) = 0$.
- Έστω ότι η i-οστή πράξη είναι πολλαπλή απώθηση (multipop). Έχουμε ότι $c_i = \min\{k, |S|\}$ και $\Phi(S_i) = \Phi(S_{i-1}) - \min\{k, |S|\}$, επομένως $\hat{c}_i = c_i + \Phi(S_i) - \Phi(S_{i-1}) = 0$.

Άρα, ο συνολικός χρόνος για την εκτέλεση N πράξεων είναι $O(N)$.

Για το πρόβλημα της επαύξησης δυαδικού μετρητή, επιλέγουμε

$$\Phi(D) = \text{πλήθος ψηφίων ίσων με 1 στη δυαδική τιμή } D \text{ του μετρητή.}$$

Για αρχικά μηδενισμένο μετρητή, έχουμε $\Phi(D_0) = 0$ το οποίο συνεπάγεται ότι για κάθε ακέραιο $i \geq 0$, $\Phi(D_i) \geq \Phi(D_0) = 0$, που με τη σειρά του συνεπάγεται ότι $\sum_{j=1}^{i} c_j \leq \sum_{j=1}^{i} \hat{c}_j$ για κάθε $i \geq 0$. Επομένως, και σε αυτήν την περίπτωση το συνολικό αντισταθιστικό κόστος αποτελεί άνω φράγμα του συνολικού πραγματικού κόστους. Μένει να υπολογίσουμε το αντισταθμιστικό κόστος $\hat{c}_i$ για μία επαύξηση της τιμής του μετρητή. Έστω $\Phi(D_{i-1}) = b_{i-1}$ και $\Phi(D_i) = b_i$. Επίσης, έστω ότι η i-οστή πράξη μηδενίζει t_i δυαδικά ψηφία. Έχουμε $c_i \leq t_i + 1$ και $b_i \leq b_{i-1} - t_i + 1$, επομένως $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \leq t_i + 1 + b_i - b_{i-1} \leq 2$.

Εάν έχουμε $\Phi(D_0) = b_0$ και $\Phi(D_N) = b_N$, τότε

$$\sum_{i=1}^{N} c_i = \sum_{i=1}^{N} (\hat{c}_i - (\Phi(D_N) - \Phi(D_0))) \leq 2N - b_N + b_0 \leq 2N + k.$$

Επομένως, αν πραγματοποιήσουμε $N = \Omega(k)$ επαυξήσεις τότε ο συνολικός χρόνος που αυτές απαιτούν είναι $\sum_{i=1}^{N} c_i = O(N)$ ανεξάρτητα από την αρχική τιμή του μετρητή.

Ασκήσεις

13.1 Θέλουμε να κατασκευάσουμε μια δομή δεδομένων που να υποστηρίζει δυαδική αναζήτηση αλλά και την αποδοτική εισαγωγή νέων στοιχείων. Θυμηθείτε ότι για τη δυαδική αναζήτηση ενός στοιχείου x σε ένα ταξινομημένο πίνακα $A[1:n]$ συγκρίνουμε το x με το μεσαίο στοιχείο $A[m]$, όπου $m = \left\lfloor \frac{n+1}{2} \right\rfloor$: (α) αν $x = A[m]$, τότε το στοιχείο έχει βρεθεί, (β) αν $x < A[m]$, τότε ψάχνουμε αναδρομικά τον υποπίνακα $A[1:m-1]$ και (γ), αν $x > A[m]$, τότε ψάχνουμε αναδρομικά τον υποπίνακα $A[m+1:n]$.

Ωστόσο, η εισαγωγή ενός νέου στοιχείου y στο διατεταγμένο πίνακα A δεν είναι αποδοτική, ακόμα και αν ο πίνακας έχει κενές θέσεις στο τέλος (όπως θα συνέβαινε με μία υλοποίηση με δυναμικό πίνακα). Π.χ. αν $A = [2, 5, 19, 22, 35, 48, 81, 95, 101, 110, 134, 149, 256, \text{κενό}]$ και $y = 1$, τότε όλα τα στοιχεία του A πρέπει να μετακινηθούν μία θέση δεξιά, με αποτέλεσμα η εισαγωγή να γίνεται σε $O(n)$ χρόνο.

Για αυτό το λόγο προτείνουμε την ακόλουθη δομή. Έστω $k = \lceil \lg(n+1) \rceil$, ο αριθμός των bits που χρειάζονται για τη δυαδική αναπαράσταση $\langle n_{k-1}, n_{k-2}, \ldots, n_1, n_0 \rangle$ του n. Π.χ. για $n = 13$ έχουμε $k = \lceil \lg(n+1) \rceil = \lceil \lg 14 \rceil = 4$ και $n = 13 = \langle 1101 \rangle$. Η δομή αποτελείται από k διατεταγμένους πίνακες A_i, $0 \leq i \leq k-1$. Αν $n_i = 0$, τότε ο A_i είναι κενός, διαφορετικά (όταν $n_i = 1$) ο A_i περιέχει 2^i διατεταγμένα στοιχεία. Π.χ. για το παραπάνω παράδειγμα με $n = 13$ θα μπορούσαμε να έχουμε τους πίνακες $A_0 = [35]$, $A_1 = [\,]$ (κενός πίνακας), $A_2 = [2, 48, 81, 149]$ και $A_3 = [5, 19, 22, 95, 101, 110, 134, 256]$. Προσέξτε ότι τα στοιχεία ενός πίνακα είναι διατεταγμένα αλλά δεν υπάρχει κάποια συγκεκριμένη σχέση μεταξύ των στοιχείων διαφορετικών πινάκων.

α) Δώστε ένα αλγόριθμο αναζήτησης σε αυτή τη δομή. Ποιος είναι ο χρόνος εκτέλεσης χειρότερης περίπτωσης;

β) Περιγράψτε έναν αλγόριθμο εισαγωγής ενός νέου στοιχείου στη δομή μας. Αναλύστε το χρόνο εκτέλεσης χειρότερης περίπτωσης καθώς και τον αντισταθμιστικό χρόνο εκτέλεσης της εισαγωγής.

γ) Μελετήστε τρόπους αποδοτικής υλοποίησης της διαδικασίας διαγραφής ενός στοιχείου από τη δομή. Υποθέστε ότι δεν υπάρχουν διπλά στοιχεία αποθηκευμένα στη δομή.

13.2 Έχουμε δει ότι ο αντισταθμιστικός χρόνος επαύξησης (πρόσθεσης με το 1) ενός δυαδικού μετρητή $\langle b_{k-1}, b_{k-2}, \dots, b_1, b_0 \rangle$ με k bits είναι $O(1)$, παρόλο που στη χειρότερη περίπτωση μια επαύξηση χρειάζεται $O(k)$ χρόνο. Σε αυτήν την άσκηση θα δούμε πως μπορούμε να κάνουμε επαύξηση του μετρητή σε $O(1)$ χρόνο *χειρότερης περίπτωσης* (δηλαδή κάθε επαύξηση θα γίνεται σε σταθερό αριθμό βημάτων).

Η ιδέα βασίζεται στη χρήση ενός *πλεονάζοντος αριθμητικού συστήματος*: επιτρέπουμε κάποια bits να λάβουν την τιμή 2 με ελεγχόμενο, όμως, τρόπο. Ο αριθμός που αναπαριστά μια τέτοια ακολουθία $\langle b_{k-1}, b_{k-2}, \dots, b_1, b_0 \rangle$ σε αυτό το σύστημα υπολογίζεται με τον τύπο $\sum_{i=0}^{k-1} b_i \cdot 2^i$, όπως και στην κανονική δυαδική αναπαράσταση, μόνο που τώρα η ίδια τιμή μπορεί να αντιστοιχεί σε περισσότερες ακολουθίες. Π.χ. $9 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = \langle 1001 \rangle$ αλλά, επίσης, $9 = 1 \times 2^2 + 2 \times 2^1 + 1 \times 2^0 = \langle 121 \rangle$ και $9 = 2 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = \langle 201 \rangle$.

Για να επιτύχουμε γρήγορη επαύξηση πρέπει να αποτρέψουμε την εμφάνιση διαδοχικών 2 στην ακολουθία $\langle b_{k-1}, b_{k-2}, \dots, b_1, b_0 \rangle$, οπότε πρέπει να θέσουμε κάποιους κανόνες. Θα χρειαστούμε μερικούς χρήσιμους ορισμούς :

- Μια ακολουθία στο πλεονάζον αριθμητικό σύστημα είναι *κανονική*, εάν τα 2 και τα 0 εναλλάσσονται (δηλαδή ανάμεσα από διαδοχικά 2 μεσολαβεί κάποιο 0 και αντιστρόφως). Π.χ. η $\langle 2012 \rangle$ είναι κανονική αλλά όχι και η $\langle 2120 \rangle$.
- Όταν το τελευταίο ψηφίο που δεν είναι 1 έχει τιμή 0, τότε λέμε ότι η ακολουθία έχει *εκτεθειμένο* 0. Π.χ. η ακολουθία $\langle 2001 \rangle$ έχει εκτεθειμένο 0 (αλλά δεν είναι κανονική), όπως και η $\langle 20201 \rangle$ (η οποία είναι κανονική).
- Όταν το τελευταίο ψηφίο που δεν είναι 1 έχει τιμή 2, τότε λέμε ότι η ακολουθία έχει *εκτεθειμένο* 2. Π.χ. η ακολουθία $\langle 2102 \rangle$ έχει εκτεθειμένο 2 (και είναι κανονική).

Παρατηρήστε ότι ακόμα και με κανονικές ακολουθίες υπάρχει πλεονασμός, π.χ. $18 = \langle 1202 \rangle = \langle 1210 \rangle$.

Εξηγήστε πώς μπορούμε να επιτύχουμε επαύξηση μιας κανονικής ακολουθίας σε $O(1)$ χρόνο χειρότερης περίπτωσης. Το αποτέλεσμα της επαύξησης πρέπει να είναι κανονική ακολουθία έτσι ώστε να μπορούμε να συνεχίσουμε τις επαυξήσεις.

Υπόδειξη: Η προβληματική περίπτωση είναι όταν η ακολουθία μας έχει εκτεθειμένο 2. Πριν γίνει η επαύξηση, πρέπει να μετατραπεί σε ακολουθία με εκτεθειμένο 0 σε $O(1)$ χρόνο. Για να επιτευχθεί αυτό, πρέπει να γνωρίζουμε τη θέση του τελευταίου 2. Αυτό γίνεται εύκολα, αν διατηρούμε τις θέσεις των 2 σε μια στοίβα (πίνακα) με το τελευταίο 2 να βρίσκεται στην κορυφή της.

13.3 Θέλουμε να αναλύσουμε την απόδοση μιας δομής δεδομένων που χειρίζεται διατεταγμένες λίστες ακεραίων από το σύνολο $\{1,2,3, \dots, n\}$ και υποστηρίζει την πράξη της συγχώνευσης δύο λιστών. Αρχικά, κάθε ακέραιος αποτελεί μια ξεχωριστή λίστα. Η πράξη συγχώνευση(A,B) δημιουργεί μια νέα λίστα C με τα στοιχεία των λιστών A και B. Μετά τη συγχώνευση οι δύο λίστες A και B έχουν καταστραφεί. Για παράδειγμα, αν έχουμε $A =$

$\langle 2, 5, 8, 12, 21 \rangle$ και $B = \langle 1, 15, 18, 22, 34, 55 \rangle$, τότε η συγχώνευσή τους δίνει μια νέα λίστα $C = \langle 1, 2, 5, 8, 12, 15, 18, 21, 22, 34, 55 \rangle$.

Για να λύσουμε το παραπάνω πρόβλημα, αποθηκεύουμε κάθε λίστα σε μια δομή δεδομένων η οποία εκτελεί όλες τις παρακάτω πράξεις σε χρόνο $f(n)$.

- ελάχιστο(A): Επιστρέφει το ελάχιστο στοιχείο της λίστας A.
- αναζήτηση(A,x): Βρίσκει τη θέση του ακέραιου x στη λίστα A.
- διαχωρισμός(A,x): Χωρίζει τη λίστα A στη θέση x. Επιστρέφει δύο νέες λίστες B και C όπου η B περιέχει τους ακέραιους της A που είναι $\leq x$ και η C περιέχει τους ακέραιους της A που είναι $> x$.
- ένωση(A,B): Προϋποθέτει ότι όλοι οι ακέραιοι της λίστας A είναι μικρότεροι από τους ακέραιους της λίστας B. Επιστρέφει μια νέα λίστα C με την ένωση των A και B.

α) Δείξτε πώς υλοποιείται η πράξη συγχώνευση(A,B) συνδυάζοντας τις παραπάνω πράξεις. Ποιος είναι ο χρόνος εκτέλεσης της;

β) Τώρα θέλουμε να δείξουμε ότι ο αντισταθμιστικός χρόνος της συγχώνευσης είναι $O(f(n)\log n)$, ξεκινώντας από n λίστες με ένα ακέραιο η κάθε μια. Για το σκοπό αυτό θα ορίσουμε το δυναμικό $\varphi(a_j)$ ενός ακέραιου a_j ως εξής. Έστω ότι ο a_j ανήκει στη λίστα $A = \langle a_1, \dots, a_{j-1}, a_j, a_{j+1}, \dots, a_k \rangle$. Αν το a_j είναι το μοναδικό στοιχείο της λίστας, τότε $\varphi(a_j) = 2\log n$. Διαφορετικά, έχουμε τις ακόλουθες περιπτώσεις. Αν το a_j είναι το πρώτο στοιχείο της λίστας ($j = 1$), τότε $\varphi(a_j) = \log n + \log(a_{j+1} - a_j)$. Αν το a_j είναι το τελευταίο στοιχείο της λίστας ($j = k$) τότε $\varphi(a_j) = \log(a_j - a_{j-1}) + \log n$. Διαφορετικά ($1 < j < k$) έχουμε $\varphi(a_j) = \log(a_j - a_{j-1}) + \log(a_{j+1} - a_j)$. Το ολικό δυναμικό Φ της δομής είναι το άθροισμα των επιμέρους δυναμικών $\varphi(a_j)$. Αρχικά, αφού κάθε λίστα έχει μόνο ένα στοιχείο, έχουμε $\Phi = 2n\log n$. Στη συνέχεια, κάθε πράξη συγχώνευσης έχει ως αποτέλεσμα τη μείωση του δυναμικού. Με τη βοήθεια της παραπάνω συνάρτησης δείξτε ότι σε οποιαδήποτε ακολουθία m συγχωνεύσεων θα γίνουν το πολύ $O(m\log n)$ πράξεις αναζήτησης, διαχωρισμού και ένωσης. Καταλήξτε στο συμπέρασμα ότι ο αντισταθμιστικός χρόνος της συγχώνευσης είναι $O(f(n)\log n)$.

Βιβλιογραφία

Cormen, T., Leiserson, C., Rivest, R., & Stain, C. (2001). *Introduction to Algorithms.* MIT Press (2nd edition) .

Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures: The Basic Toolbox.* Springer-Verlag.

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Γεωργακόπουλος, Γ. Φ. (2011). *Δομές Δεδομένων.* Πανεπιστημιακές εκδόσεις Κρήτης.

Κεφάλαιο 14
Προηγμένες Ουρές Προτεραιότητας

Περιεχόμενα

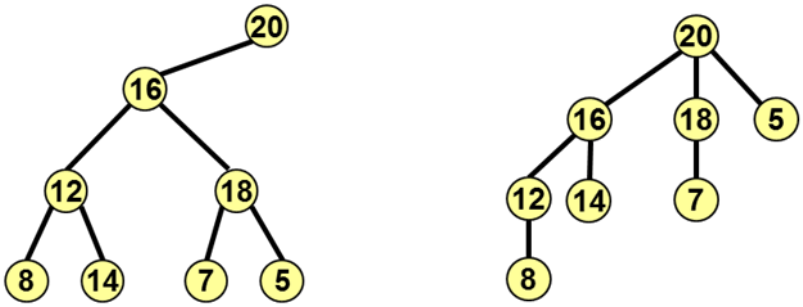
14.1 Διωνυμικά Δένδρα 255
14.2 Διωνυμικές Ουρές 258
14.1.1 Εισαγωγή στοιχείου σε διωνυμική ουρά 258
14.1.2 Διαγραφή μεγίστου από διωνυμική ουρά 262
14.1.3 Ένωση δύο διωνυμικών ουρών 263
14.1.4 Κατασκευή διωνυμικής ουράς με *N* κλειδιά 265
14.2 Σωροί Fibonacci 265
14.2.1 Δυναμικό σωρού Fibonacci 266
14.2.2 Εύρεση ελάχιστου κλειδιού 267
14.2.3 Εισαγωγή κλειδιού 267
14.2.4 Ένωση δύο σωρών Fibonacci 268
14.2.5 Εξαγωγή ελάχιστου κλειδιού 269
14.2.6 Μείωση κλειδιού 276
14.2.7 Διαγραφή κλειδιού 278
Βιβλιογραφία 279

Στο κεφάλαιο αυτό μελετάμε τους σωρούς Fibonacci οι οποίοι βασίζονται στις διωνυμικές ουρές. Οι σωροί Fibonacci επιπλέον των πράξεων που επιτρέπουν οι κλασικοί σωροί (δηλαδή, εισαγωγή, εύρεση ελαχίστου και εξαγωγή ελαχίστου) επιτρέπουν την ταχεία εκτέλεση και άλλων πράξεων, όπως ένωση, διαγραφή, και μείωση κλειδιού.

14.1 Διωνυμικά Δένδρα

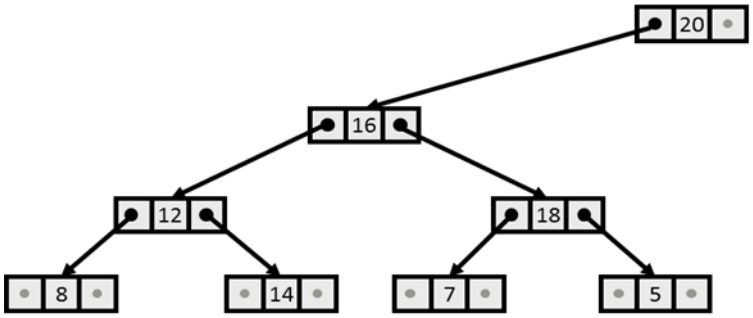
Ένα *δυαδικό δένδρο αριστερά διατεταγμένο σε σωρό* είναι ένα δυαδικό δένδρο στο οποίο το κλειδί κάθε κόμβου είναι μεγαλύτερο από ή ίσο με όλα τα κλειδιά του αριστερού υποδένδρου αυτού του κόμβου. Ένα δυαδικό δένδρο αριστερά διατεταγμένο σε σωρό, στο οποίο το δεξί υποδένδρο της ρίζας είναι κενό και το αριστερό υποδένδρο είναι πλήρες, σχηματίζει έναν *σωρό δύναμης του 2*. Στην Εικόνα 14.1 (αριστερά) φαίνεται ένας σωρός δύναμης 2 με 8 κλειδιά, στον οποίον μπορούμε να παρατηρήσουμε ότι το δεξί υποδένδρο της ρίζας είναι κενό ένω το αριστερό είναι πλήρες και το κλειδί κάθε κόμβου είναι μεγαλύτερο από ή ίσο με κάθε κλειδί του αριστερού του υποδένδρου. Ο σωρός δύναμης του 2 οφείλει το όνομά του στο γεγονός ότι το πλήθος κόμβων ενός τέτοιου σωρού είναι δύναμη του 2. Ένα *διωνυμικό δένδρο (binomial tree)* είναι ένα δένδρο το οποίο με την αντιστοίχιση αριστερού παιδιού και δεξιού αδελφού δίνει σωρό δύναμης 2 (στην Εικόνα 14.1 (δεξιά) φαίνεται ένα διωνυμικό δένδρο το οποίο με την αντιστοίχιση αριστερού παιδιού και δεξιού αδελφού δίνει το σωρό δύναμης 2 στα

αριστερά). Ένα διωνυμικό δένδρο δεν είναι απαραίτητα δυαδικό. Το δυωνυμικό δένδρο της Εικόνας 14.1 (δεξιά) έχει βαθμό 3.



Εικόνα 14.1: (αριστερά) Ένας σωρός δύναμης 2 με 8 κλειδιά. **(δεξιά)** Ένα δυωνυμικό δένδρο το οποίο με την αντιστοίχιση αριστερού παιδιού και δεξιού αδελφού δίνει το σωρό δύναμης 2 στα αριστερά.

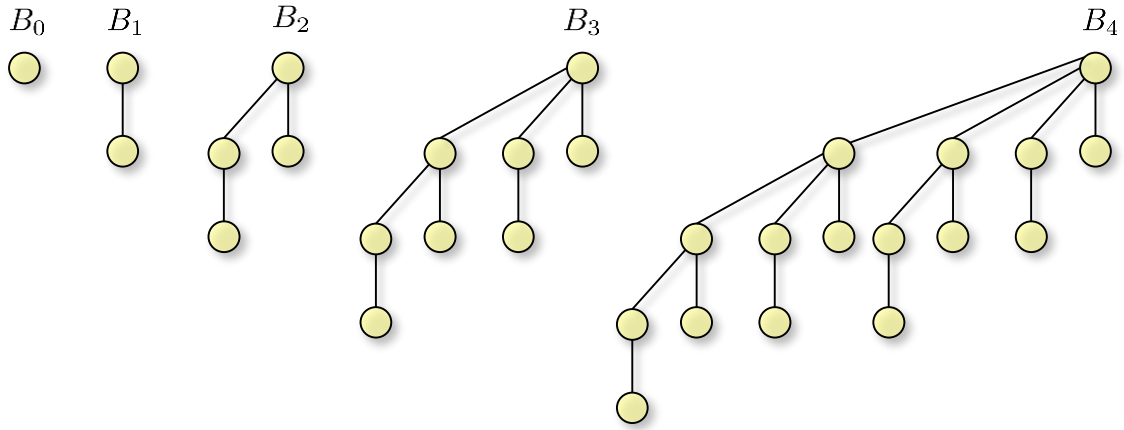
Ένα διωνυμικό δένδρο νοητικά παρίσταται, όπως φαίνεται στην Εικόνα 14.1 (δεξιά) ωστόσο υλοποιείται ως δυαδικό δένδρο με την αντιστοίχιση αριστερού παιδιού και δεξιού αδελφού (Εικόνα 14.2).



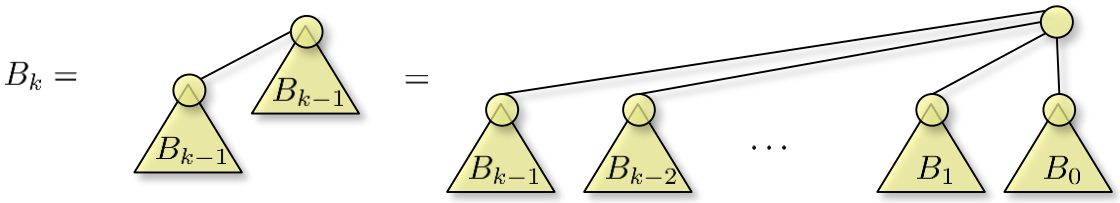
Εικόνα 14.2: Υλοποίηση του διωνυμικού δένδρου της Εικόνας 14.1 (δεξιά).

Καθώς ένας σωρός δύναμης 2 έχει πλήθος κλειδιών ίσο με δύναμη του 2, και τα διωνυμικά δένδρα έχουν πλήθος κλειδιών ίσο με δύναμη του 2. Το διωνυμικό δένδρο, με 2^k κλειδιά αναπαρίσταται με B_k (Εικόνα 14.3). Μάλιστα, το διωνυμικό δένδρο B_k μπορει να προκύψει από τη σύνδεση της ρίζας ενός διωνυμικού δένδρου B_{k-1} ως αριστερότερο παιδί της ρίζας ενός άλλου διωνυμικού δένδρου B_{k-1}. Έτσι, το B_k συνίσταται από τον κόμβο-ρίζα με k παιδιά, που από δεξιά προς τα αριστερά είναι κόμβοι-ρίζες διωνυμικών δένδρων B_0, B_1, B_2, ..., B_{k-1} (Εικόνα **14.4**). Συνεπώς, το διωνυμικό δένδρο B_k, που έχει 2^k κόμβους, έχει $\binom{k}{j}$ κόμβους στο επίπεδο $j \geq 0$. Υπενθυμίζεται ότι

$$\sum_{j=0}^{k} \binom{k}{j} = 2^k.$$



Εικόνα 14.3: Τα διωνυμικά δένδρα B_0, B_1, B_2, B_3 και B_4.

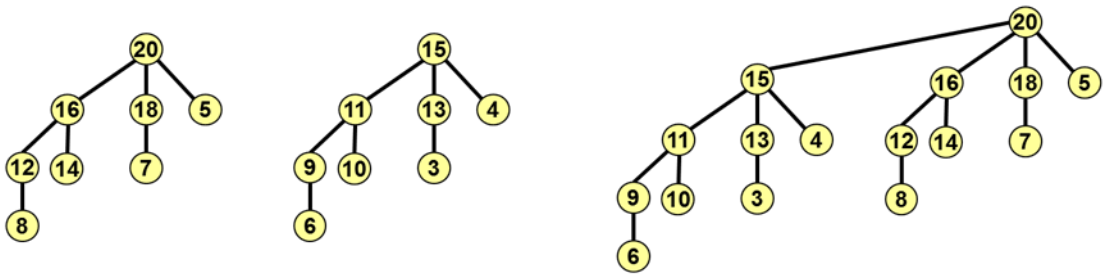


Εικόνα 14.4: Γενική μορφή του διωνυμικού δένδρου B_k.

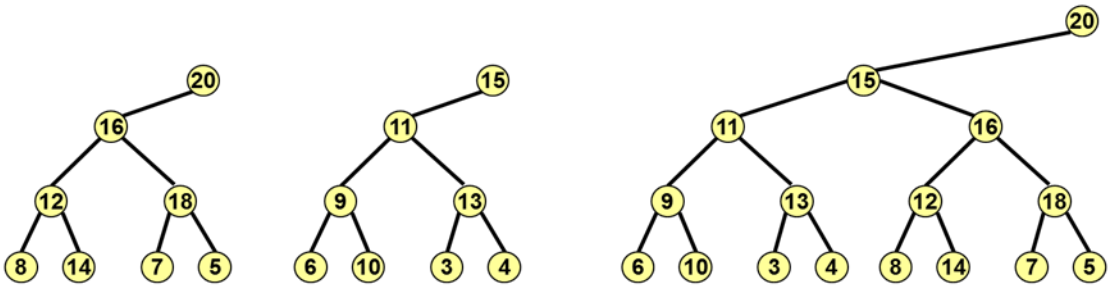
Συνοψίζοντας, υπενθυμίζουμε ότι

- το πλήθος των κόμβων σε ένα διωνυμικό δένδρο είναι δύναμη του 2,
- κανένας κόμβος δεν έχει κλειδί μεγαλύτερο από το κλειδί της ρίζας και
- τα διωνυμικά δένδρα είναι διατεταγμένα σε σωρό (heap-ordered).

Επιπλέον, δύο διωνυμικά δένδρα *ίδιου μεγέθους* συνενώνονται εύκολα ως εξής: Βρίσκουμε το διωνυμικό δένδρο στον κόμβο-ρίζα του οποίου βρίσκεται το μεγαλύτερο κλειδί και προσθέτουμε ως παιδί τον κόμβο-ρίζα του άλλου διωνυμικού δένδρου προς συνένωση (Εικόνες 14.5 και 14.6). Τονίζεται και πάλι ότι δύο διωνυμικά δένδρα συνενώνονται, μόνον εάν έχουν ίδιο μέγεθος.



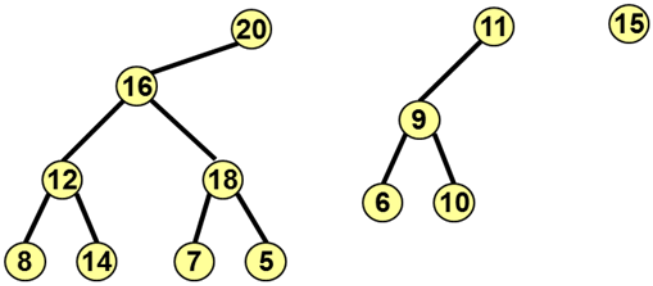
Εικόνα 14.5: Συνένωση δύο διωνυμικών δένδρων με 8 κλειδιά σε ένα δυωνυμικό δένδρο με 16 κλειδιά.



Εικόνα 14.6: Η συνένωση της Εικόνας 14.5 με αναπαράσταση των διωνυμικών δένδρων με δυαδικά δένδρα.

14.2 Διωνυμικές Ουρές

Μια διωνυμική ουρά (binomial queue) είναι ένα σύνολο διωνυμικών δένδρων διαφορετικού μεγέθους ανά δύο (Εικόνα 14.7). Ο ορισμός των διωνυμικών ουρών συνεπάγεται ότι η δομή μιας τέτοιας ουράς καθορίζεται από τη δυαδική αναπαράσταση του πλήθους των κόμβων της. Για παράδειγμα, η διωνυμική ουρά της Εικόνας 14.7 έχει μέγεθος 13 = $(1101)_2$ και, άρα, αποτελείται από διωνυμικά δένδρα B_0, B_2, B_3 (τα ψηφία της δυαδικής αναπαράστασης του 13 που είναι ίσα με 1 βρίσκονται στις θέσεις 0, 2 και 3 (από δεξιά προς τα αριστερά)). Ως αποτέλεσμα, μια διωνυμική ουρά με N κλειδιά αποτελείται από το πολύ $\lfloor \log N \rfloor + 1$ διωνυμικά δένδρα.

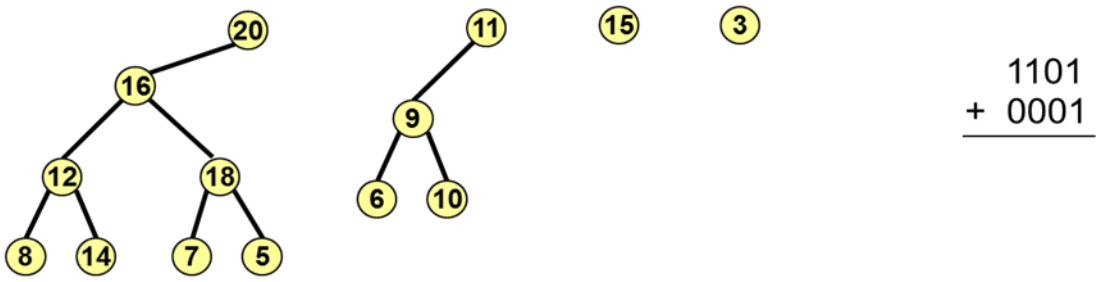


Εικόνα 14.7: Μια διωνυμική ουρά που αποτελείται από διωνυμικά δένδρα B_0, B_2, B_3.

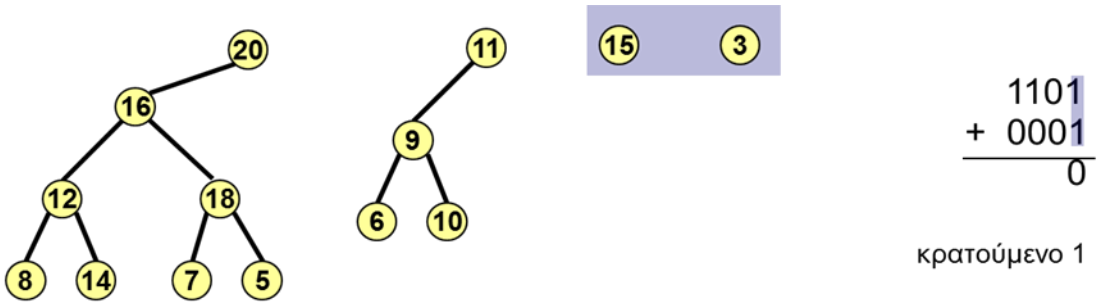
14.1.1 Εισαγωγή στοιχείου σε διωνυμική ουρά

Η εισαγωγή ενός επιπλέον στοιχείου σε διωνυμική ουρά ξεκινά με το σχηματισμό ενός νέου διωνυμικού δένδρου B_0, που περιέχει το στοιχείο αυτό και, κατόπιν, όσο υπάρχουν δύο διωνυμικά δένδρα ίδιου μεγέθους, αυτά συνενώνονται, όπως περιγράψαμε στο τέλος της προηγούμενης παραγράφου, δημιουργώντας ένα νέο διωνυμικό δένδρο διπλάσιου μεγέθους.

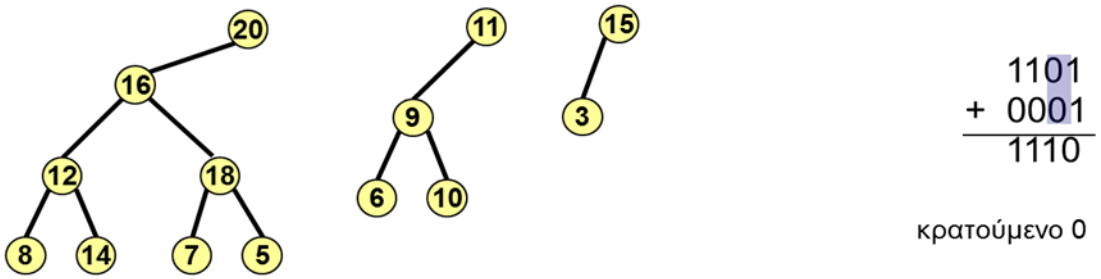
Για παράδειγμα, η εισαγωγή του κλειδιού 3 στη διωνυμική ουρά της Εικόνας **14.7** φαίνεται στις Εικόνες 14.8 – 14.10, όπου στα δεξιά κάθε εικόνας εμφανίζονται τα βήματα της πρόσθεσης του αριθμού 1 στο πλήθος των στοιχείων (13) της διωνυμικής ουράς της Εικόνας 14.7.



Εικόνα 14.8: Το 3 αρχικά εισάγεται ως ένα διωνυμικό δένδρο B_0.

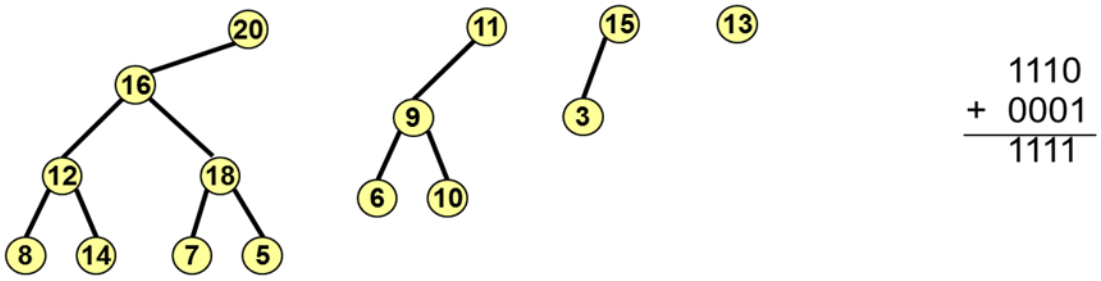


Εικόνα 14.9: Το δύο διωνυμικά δένδρα B_0 συνενώνονται σε ένα δυωνυμικό δένδρο B_1.



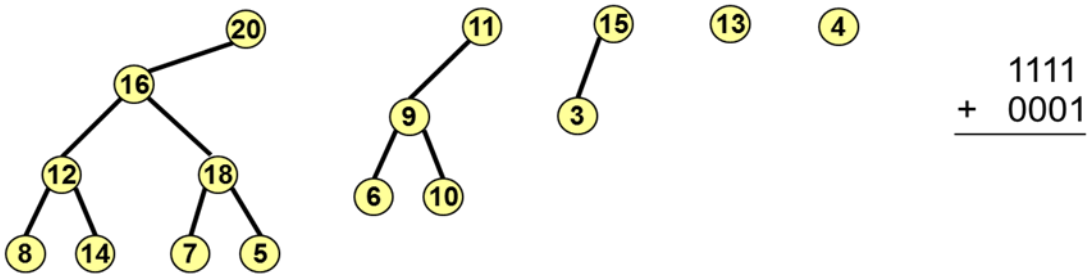
Εικόνα 14.10: Η τελική διωνυμική ουρά.

Αν στη διωνυμική ουρά της Εικόνας 14.10 εισαχθεί το 13, τότε απλώς προκύπτει η διωνυμική ουρά της Εικόνας 14.11.

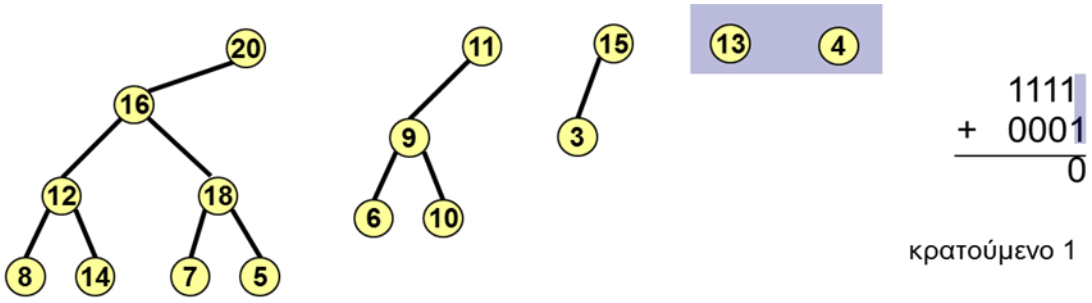


Εικόνα 14.11

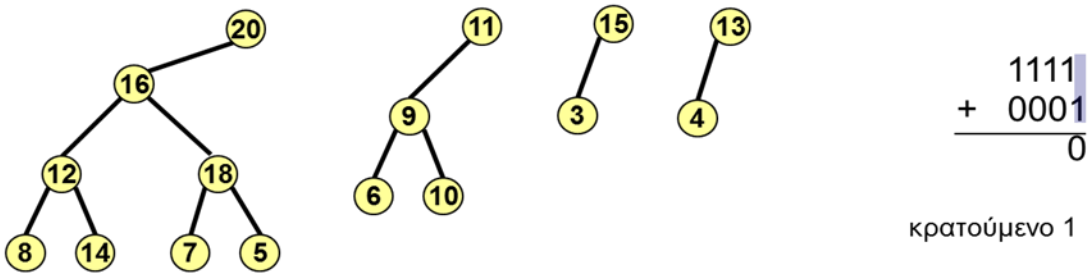
Τέλος εάν στη διωνυμική ουρά της Εικόνας 14.11 εισαχθεί το 4, τότε έχουμε συνεχείς συνενώσεις διωνυμικών δένδρων, έως ότου προκύψει τελικά ένα διωνυμικό δένδρο B_4 (Εικόνα 14.20).



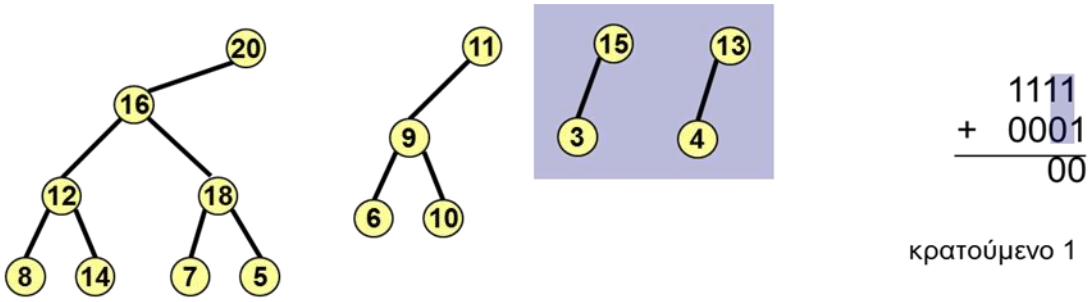
Εικόνα 14.12



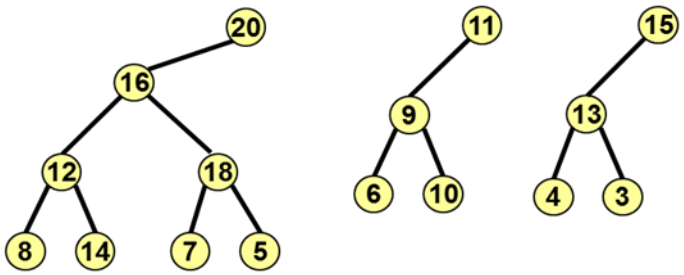
Εικόνα 14.13



Εικόνα 14.14



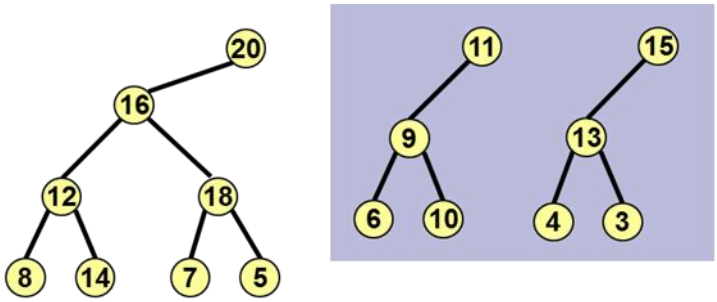
Εικόνα 14.15




```
  1111
+ 0001
    00
```

κρατούμενο 1

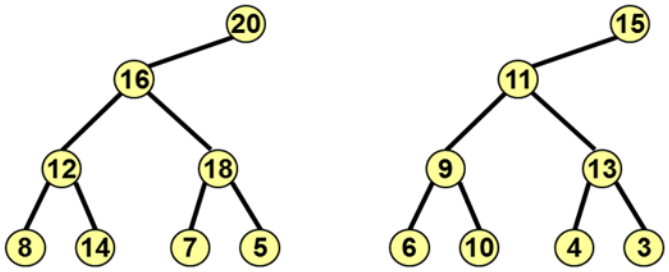
Εικόνα 14.16




```
  1111
+ 0001
   000
```

κρατούμενο 1

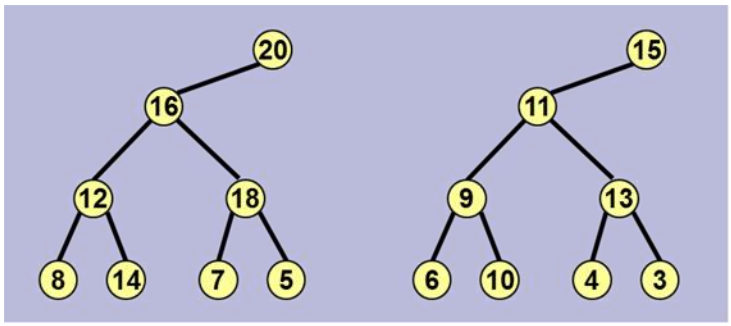
Εικόνα 14.17




```
  1111
+ 0001
   000
```

κρατούμενο 1

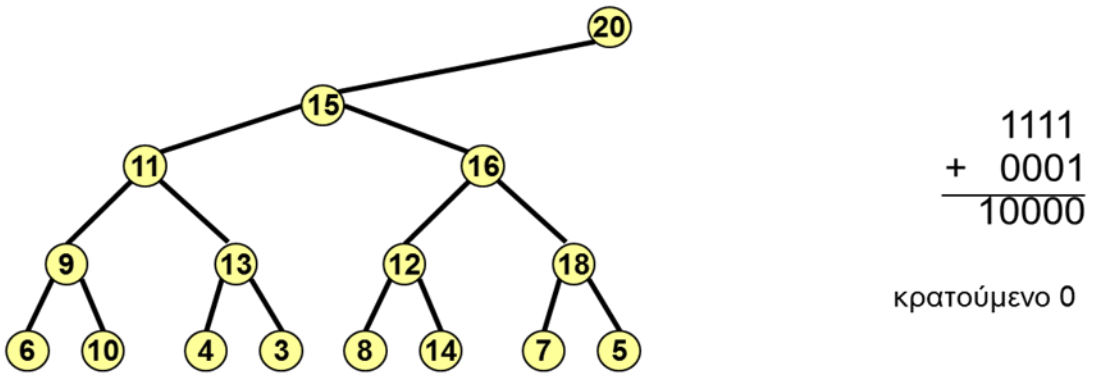
Εικόνα 14.18




```
  1111
+ 0001
  0000
```

κρατούμενο 1

Εικόνα 14.19

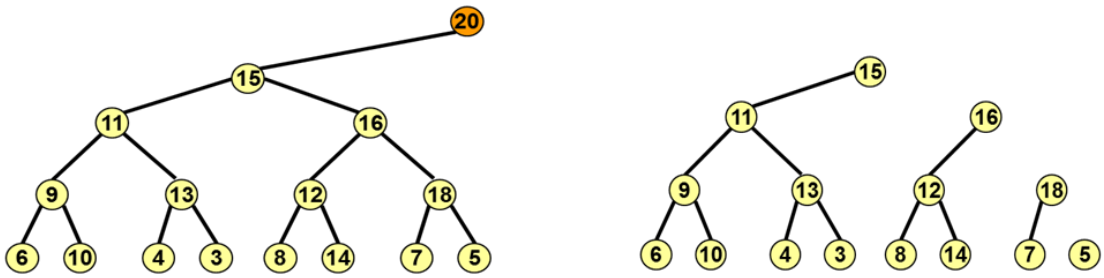


Εικόνα 14.20

Καθώς η συνένωση δύο διωνυμικών δένδρων γίνεται σε σταθερό χρόνο, η εισαγωγή ενός στοιχείου σε διωνυμική ουρά με N στοιχεία απαιτεί $O(\log N)$ χρόνο.

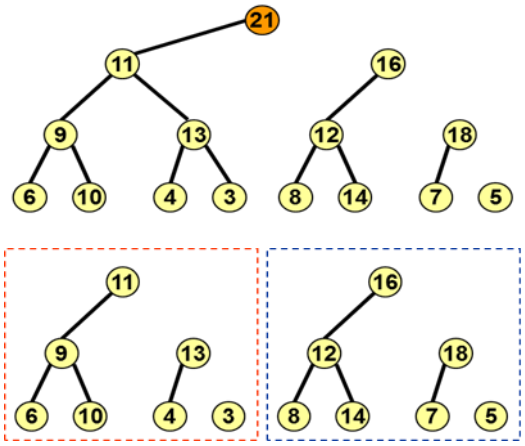
14.1.2 Διαγραφή μεγίστου από διωνυμική ουρά

Εάν η διωνυμική ουρά αποτελείται από ένα διωνυμικό δένδρο, έστω B_k, τότε το μέγιστο βρίσκεται στη ρίζα του δένδρου. Η διαγραφή του γίνεται με διαγραφή του κόμβου-ρίζας και αποσύνδεση των παιδιών του, που θα δημιουργήσει k διωνυμικά δένδρα, ένα B_0, ένα B_1, ... και ένα B_{k-1} (Εικόνα 14.21).



Εικόνα 14.21: Διαγραφή μεγίστου από διωνυμικό δένδρο.

Εάν η διωνυμική ουρά αποτελείται από περισσότερα από ένα διωνυμικά δένδρα, τότε βρίσκουμε το μέγιστο (σε κάποιον από τους κόμβους-ρίζες των διωνυμικών δένδρων που αποτελούν τη διωνυμική ουρά) και το διαγράφουμε από το δένδρο, το οποίο αποσυνδέεται σε μικρότερα διωνυμικά δένδρα (Εικόνα 14.22) με αποτέλεσμα να χρειάζεται τελικά να ενώσουμε δύο διωνυμικές ουρές. Την ένωση διωνυμικών ουρών θα δούμε στην επόμενη παράγραφο.

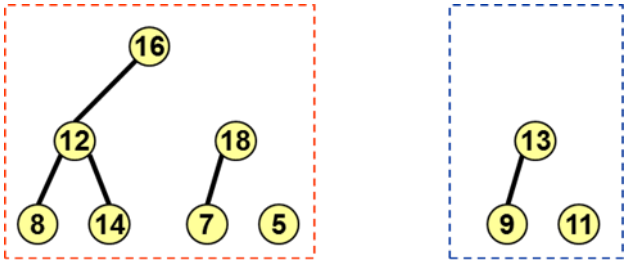


Εικόνα 14.22: Η διαγραφή του μεγίστου απαιτεί ένωση δύο διωνυμικών ουρών.

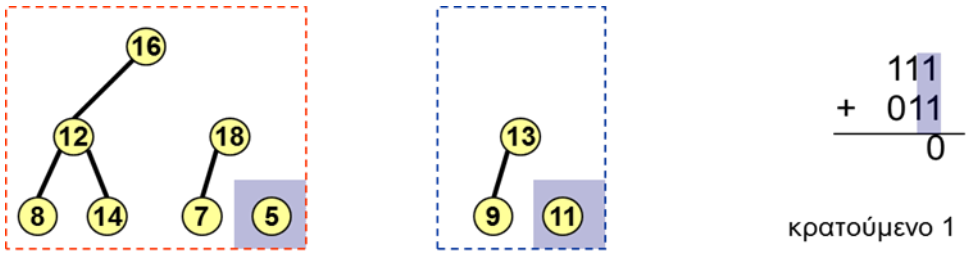
Είτε η διωνυμική ουρά αποτελείται από ένα διωνυμικό δένδρο είτε από περισσότερα, η διαγραφή του μεγίστου από διωνυμική ουρά με N κλειδιά μπορεί να εκτελεσθεί σε $O(\log N)$ χρόνο.

14.1.3 Ένωση δύο διωνυμικών ουρών

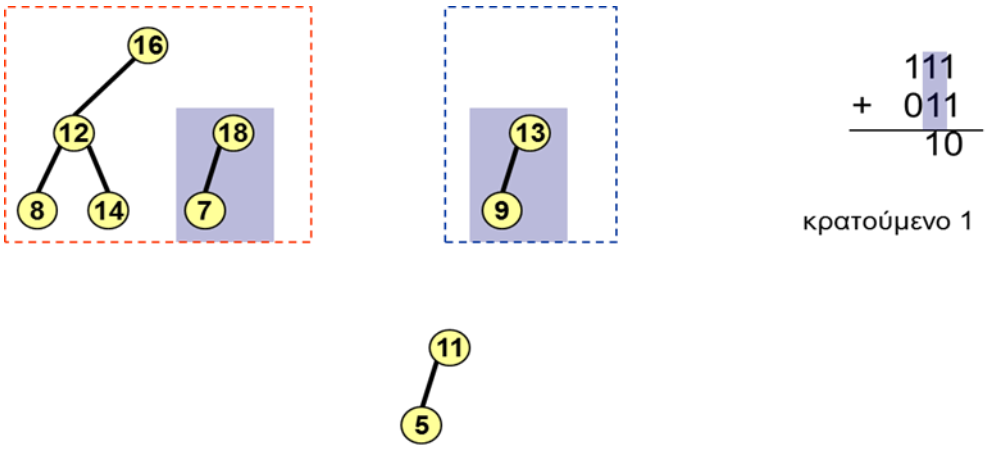
Η ένωση δύο διωνυμικών ουρών συνίσταται στην ένωση των αντίστοιχων διωνυμικών δένδρων ίδιου μεγέθους (εάν υπάρχουν) ξεκινώντας από το μικρότερο μέγεθος προς το μεγαλύτερο. Για παράδειγμα, ας θεωρήσουμε την ένωση των δύο διωνυμικών ουρών που φαίνονται στην Εικόνα 14.23. Τα βήματα της διαδικασίας ένωσης και τα αντίστοιχα βήματα της πρόσθεσης των δυαδικών αναπαραστάσεων του πλήθους κόμβων των δύο ουρών φαίνονται στις Εικόνες 14.24 – 14.28.



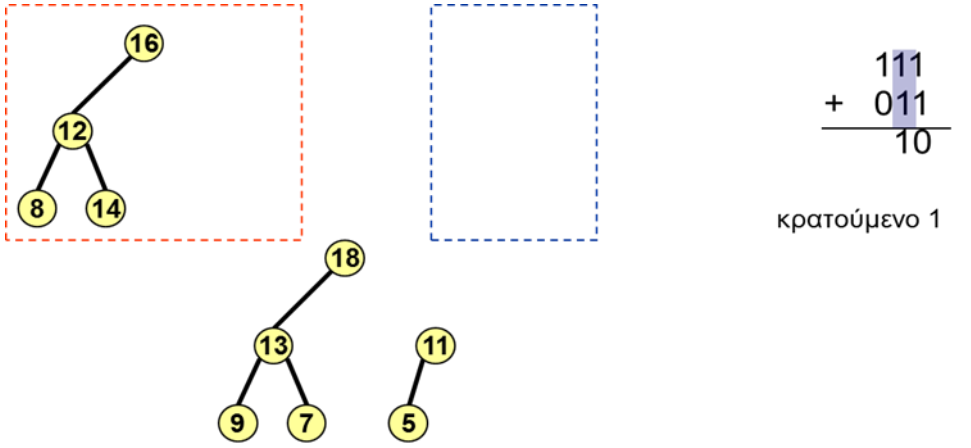
Εικόνα 14.23: Δύο διωνυμικές ουρές.



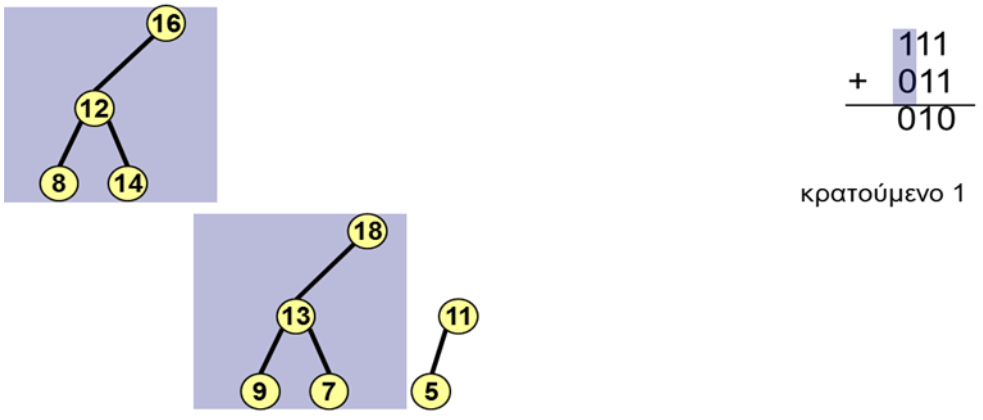
Εικόνα 14.24



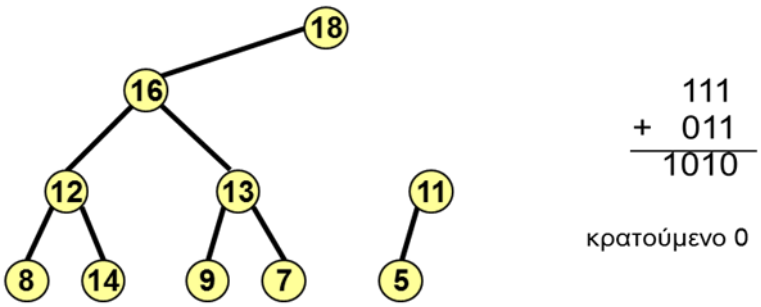
Εικόνα 14.25



Εικόνα 14.26



Εικόνα 14.27



Εικόνα 14.28: Η τελική διωνυμική ουρά.

Η διαδικασία ένωσης διωνυμικών ουρών συνεπάγεται ότι μπορεί να εκτελεσθεί σε $O(\log N)$ χρόνο.

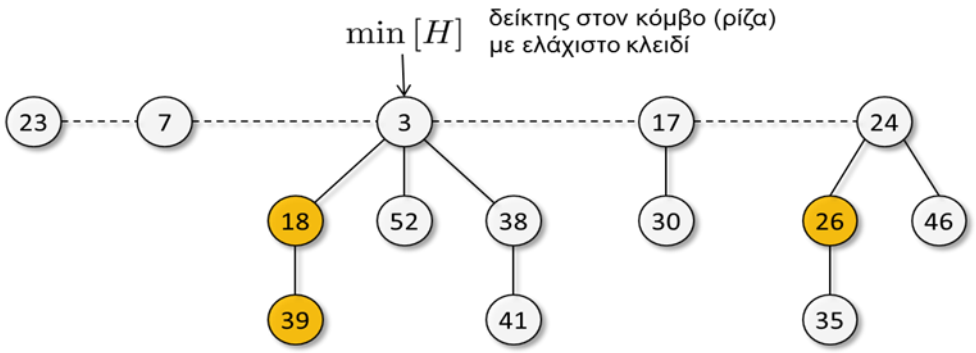
14.1.4 Κατασκευή διωνυμικής ουράς με *N* κλειδιά

Μια διωνυμική ουρά μπορεί να κατασκευασθεί με διαδοχκές εισαγωγές των στοιχείων της σε αρχικά κενή ουρά. Δεδομένου ότι η εισαγωγή ενός στοιχείου σε διωνυμική ουρά με K κλειδιά μπορεί να εκτελεσθεί σε $O(\log K)$ χρόνο, η κατασκευή μιας διωνυμικής ουράς με N κλειδιά μπορεί να εκτελεσθεί σε $O(N \log N)$ χρόνο.

Με πιο προσεκτική ανάλυση, ωστόσο, μπορούμε να αποδείξουμε ότι η κατασκευή της διωνυμικής ουράς απαιτεί $O(N)$ χρόνο. Παρατηρούμε ότι για κάθε εισαγωγή στοιχείου έχουμε μια πράξη ένωσης διωνυμικών δένδρων για κάθε δυαδικό ψηφίο που αλλάζει από 1 σε 0 στη δυαδική αναπαράσταση του πλήθους κλειδιών στη διωνυμική ουρά η οποία προκύπτει μετά την εισαγωγή του στοιχείου. Δεδομένου ότι προσθέτουμε ένα στοιχείο κάθε φορά, η δυαδική αναπαράσταση του πλήθους κλειδιών της ουράς μεταβάλλεται, όπως κατά την αύξηση δυαδικού μετρητή με $\log N$ δυαδικού ψηφία. Αλλά τότε το 1ο ψηφίο από το τέλος αλλάζει με κάθε αύξηση, το 2ο αλλάζει με κάθε δεύτερη αύξηση, το 3ο με κάθε τέταρτη αύξηση κ.ο.κ. Σε ακολουθία N αυξήσεων, το i-οστό ψηφίο από το τέλος αλλάζει συνολικά $\lfloor \frac{N}{2^{i-1}} \rfloor$ φορές, οπότε το συνολικό πλήθος αλλαγών είναι $\sum_{i=1}^{k} \lfloor \frac{N}{2^{i-1}} \rfloor < 2N$. Καθώς η ένωση δύο διωνυμικών δένδρων του ίδιου μεγέθους απαιτεί σταθερό χρόνο, η κατασκευή της διωνυμικής ουράς μπορεί να εκτελεσθεί σε $O(N)$ χρόνο.

14.2 Σωροί Fibonacci

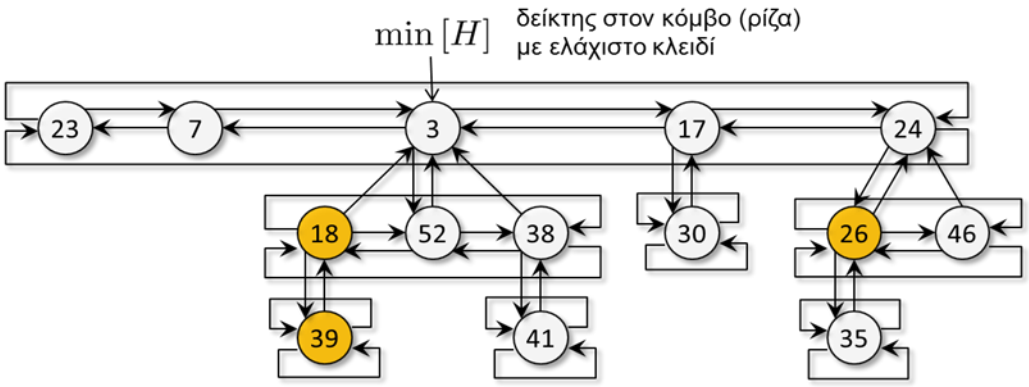
Ο σωρός Fibonacci (Fibonacci heap) βασίζεται στη διωνυμική ουρά (δηλαδή αποτελεί ένα σύνολο από δένδρα), αλλά έχει πιο χαλαρή δομή. Στην Εικόνα 14.29 φαίνεται ένας σωρός Fibonacci H με $n[H] = 14$ κλειδιά. Ο σωρός αποτελείται από δένδρα (τα οποία ενδέχεται να μην είναι διωνυμικά, αλλά προέρχονται από διωνυμικά δένδρα, από τα οποία έχουν αποκοπεί κλάδοι – οι κόμβοι που έχουν χάσει παιδιά φαίνονται με κίτρινο χρώμα), ενώ το ελάχιστο στοιχείο του σωρού δείχνεται από το δείκτη $\min[H]$.



Εικόνα 14.29: Ένας σωρός Fibonacci.

Με χρήση αντισταθμιστικής ανάλυσης μπορεί να δειχθεί ότι σε έναν σωρό Fibonacci με N κλειδιά η εισαγωγή, η ένωση, η εύρεση ελαχίστου και η μείωση κλειδιού εκτελούνται σε $O(1)$ χρόνο και η διαγραφή και η εξαγωγή ελαχίστου σε $O(\log N)$ χρόνο.

Η αποτελεσματική υλοποίηση ενός σωρού Fibonacci βασίζεται στο ότι κάθε κόμβος x αποθηκεύει εκτός από το κλειδί του: δείκτη *parent*[x] στον κόμβο-γονέα του, δείκτη *child*[x] σε ένα από τα παιδιά του και δείκτες *left*[x] και *right*[x] στον αριστερό και τον δεξιό αδελφό του σχηματίζοντας μια κυκλική διπλά συνδεδεμένη λίστα.



Εικόνα 14.30: Η υλοποίηση του σωρού Fibonacci της Εικόνας 14.29.

Επιπλέον, για κάθε κόμβο x αποθηκεύουμε το βαθμό του *degree*[x] και ένα δυαδικό ψηφίο *mark*[x] για την επισήμανη του κόμβου, εάν χρειαστεί. Έτσι, στην Εικόνα 14.30, αν x είναι ο κόμβος που αποθηκεύει το κλειδί 3, έχουμε *degree*[x] = 3 και *mark*[x] = 0, ενώ, αν x είναι ο κόμβος που αποθηκεύει το κλειδί 18, έχουμε *degree*[x] = 1 και *mark*[x] = 1.

Ακόμη, σημειώνουμε ότι σε ένα σωρό Fibonacci με n κλειδιά, ο μέγιστος βαθμός που μπορεί να έχει οποιοσδήποτε κόμβος είναι $D(n) = O(\log n)$.

14.2.1 Δυναμικό σωρού Fibonacci

Για την ανάλυση της πολυπλοκότητας χρόνου των λειτουργιών σε ένα σωρό Fibonacci θα χρησιμοποιήσουμε αντισταθμιστική ανάλυση και, συγκεκριμένα, την ενεργειακή μέθοδο. Γι' αυτό, ορίζουμε το ακόλουθο δυναμικό:

$$\Phi(H) = c \cdot (t(H) + 2\, m(H)),$$

όπου $t(H)$ είναι το πλήθος των δένδρων στο σωρό, $m(H)$ είναι το πλήθος των επισημασμένων κόμβων και c σταθερά. Για απλότητα, θεωρούμε ότι $c = 1$ υποθέτοντας ότι μία μονάδα δυναμικού αντιστοιχεί σε κάποια συγκεκριμένη σταθερή ποσότητα εργασίας. Έτσι το δυναμικό απλοποιείται σε

$$\Phi(H) = t(H) + 2\, m(H).$$

Για παράδειγμα, η τιμή του δυναμικού για το σωρό Fibonacci της Εικόνας 14.29 είναι $\Phi(H) = 5 + 2 \cdot 3 = 11$, καθώς ο σωρός αποτελείται από 5 δένδρα και έχει 3 επισημασμένους κόμβους.

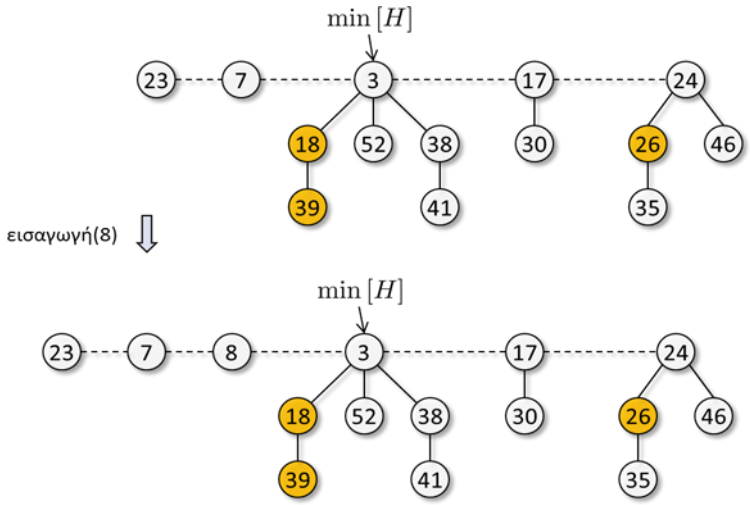
14.2.2 Εύρεση ελάχιστου κλειδιού

Αρκεί να επιστρέψουμε το κλειδί στον κόμβο που δείχνεται από το δείκτη $\min[H]$. Το πραγματικό κόστος c_{findmin} της εύρεσης είναι $O(1)$. Πρέπει όμως να φράξουμε και το αντισταθμιστικό κόστος. Το δυναμικό της δομής μετά την εύρεση είναι $\Phi'(H) = t(H) + 2\, m(H) = \Phi(H)$ και, άρα, το αντισταθμιστικό κόστος είναι $c_{\text{findmin}} + \Phi'(H) - \Phi(H) = O(1)$.

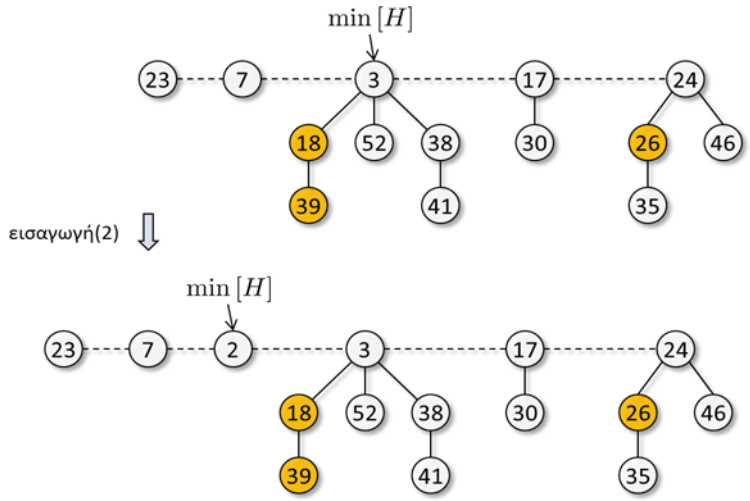
14.2.3 Εισαγωγή κλειδιού

Για την εισαγωγή ενός κλειδιού δημιουργείται ένα νέο δένδρο με μόνο έναν κόμβο που αποθηκεύει το κλειδί προς εισαγωγή και το δένδρο συνδέεται δίπλα στον κόμβο που δείχνεται από το δείκτη $\min[H]$. Επιπλέον, εάν το εισαγόμενο κλειδί είναι το ελάχιστο, τότε ο δείκτης $\min[H]$ μετατοπίζεται να δείχνει το νέο κόμβο. Η Εικόνα 14.31 δείχνει το αποτέλεσμα της εισαγωγής του κλειδιού 8 στον σωρό Fibonacci της Εικόνας 14.29, ενώ η Εικόνα 14.32 το αποτέλεσμα της εισαγωγής του κλειδιού 2.

Το πραγματικό κόστος c_{insert} της εισαγωγής είναι $O(1)$. Πρέπει και πάλι να φράξουμε και το αντισταθμιστικό κόστος. Το δυναμικό της δομής μετά την εισαγωγή είναι $\Phi'(H) = t'(H) + 2\, m(H) = t(H) + 1 + 2\, m(H) = \Phi(H) + 1$ και, άρα, το αντισταθμιστικό κόστος είναι $c_{\text{insert}} + \Phi'(H) - \Phi(H) = O(1)$.



Εικόνα 14.31: Εισαγωγή του κλειδιού 8 στο σωρό Fibonacci της Εικόνας 14.29.

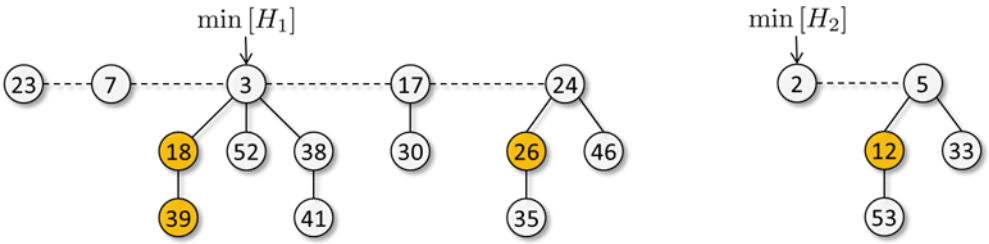


Εικόνα 14.32: Εισαγωγή του κλειδιού 2 στο σωρό Fibonacci της Εικόνας 14.29.

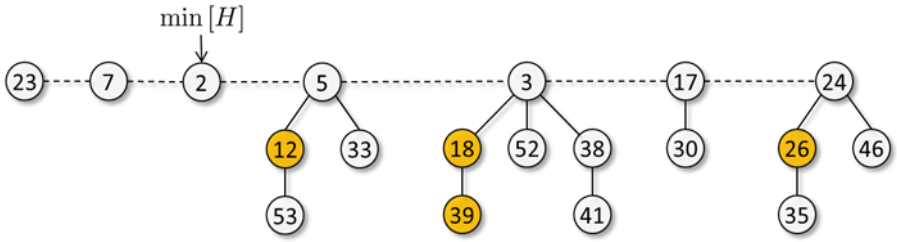
14.2.4 Ένωση δύο σωρών Fibonacci

Η ένωση δύο σωρών Fibonacci H_1 και H_2 πραγματοποιείται με ένωση των λιστών των ριζικών κόμβων τους χρησιμοποιώντας τους δείκτες $\min[H_1]$ και $\min[H_2]$. Ο δείκτης $\min[H]$ του σωρού που προκύπτει δείχνει στον κόμβο με το ελάχιστο κλειδί μεταξύ των κόμβων που δείχνονται από τους $\min[H_1]$ και $\min[H_2]$.

Για παράδειγμα, η Εικόνα 14.34 δείχνει το αποτέλεσμα της ένωσης των δύο σωρών Fibonacci της Εικόνας 14.33.



Εικόνα 14.33: Δύο σωροί Fibonacci.



Εικόνα 14.34: Ένωση των σωρών Fibonacci της Εικόνας 14.33.

Το πραγματικό κόστος c_{unite} της ένωσης δύο σωρών Fibonacci είναι $O(1)$. Το δυναμικό της δομής μετά την ένωση είναι $\Phi(H) = t(H) + 2\ m(H) = t(H_1) + t(H_2) + 2\ m(H_1) + 2\ m(H_2) = \Phi(H_1) + \Phi(H_2)$ και, άρα, το αντισταθμιστικό κόστος είναι $c_{\text{unite}} + \Phi(H) - (\Phi(H_1) + \Phi(H_2)) = O(1)$.

14.2.5 Εξαγωγή ελάχιστου κλειδιού

Η εξαγωγή του ελάχιστου κλειδιού συνίσταται στη διαγραφή του κόμβου με το ελάχιστο κλειδί, μεταφορά των παιδιών του στο ριζικό επίπεδο και ενοποίηση δένδρων. Συγκεκριμένα, η διαδικασία έχει ως εξής:

1. z := min[H];
2. **εάν** $z \neq$ NULL
3. **τότε για** κάθε παιδί x του z
4. πρόσθεσε τον κόμβο x στο ριζικό επίπεδο του σωρού H;
5. *parent*[x] := NULL;
6. διάγραψε τον κόμβο z από το ριζικό επίπεδο του σωρού H;
7. **εάν** z = *right*[z]
8. **τότε** min[H] := NULL;
9. **άλλως** min[H] := *right*[z];
10. CONSOLIDATE(H);
11. n[H] := n[H] + 1;
12. **επίστρεψε** τον κόμβο z;

όπου η διαδικασία CONSOLIDATE(H) ενοποιεί δένδρα στο ριζικό επίπεδο ως εξής (πριν από την ενοποίηση θεωρούμε πίνακα A μεγέθους $D(n[H]) + 1$, τα στοιχεία του οποίου (θέσεις 0, 1, ... $D(n[H])$) έχουν αρχικοποιηθεί σε NULL):

CONSOLIDATE(H)

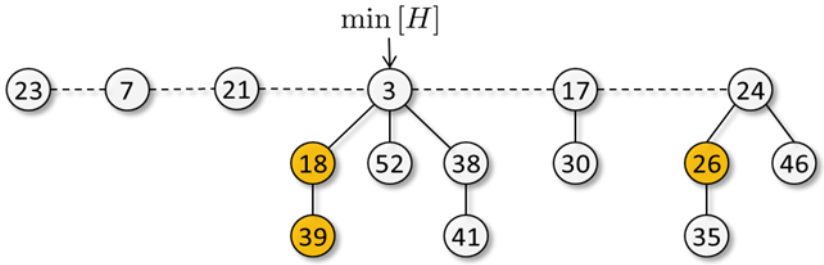
1. **για** κάθε κόμβο *w* στο ριζικό επίπεδο του σωρού *H*
2. *x* := *w*; *d* := *degree*[*x*];
3. **ενόσω** A[*d*] ≠ NULL
4. *y* := A[*d*];
5. **εάν** *key*[*x*] > *key*[*y*]
6. **τότε** ενάλλαξε τα *x* και *y*;
7. LINK(*H*, *y*, *x*);
8. A[*d*] := NULL; *d* := *d* + 1;
9. A[*d*] := *x*;
10. min[*H*] := NULL;
11. **για** *i* = 0, 1, …, *D*(*n*[*H*])
12. **εάν** A[*i*] ≠ NULL
13. **τότε** πρόσθεσε τον κόμβο A[*i*] στο ριζικό επίπεδο του σωρού *H*;
14. **εάν** min[*H*] = NULL **ή** *key*[A[*i*]] < *key*[min[*H*]]
15. **τότε** min[*H*] := A[*i*];

και

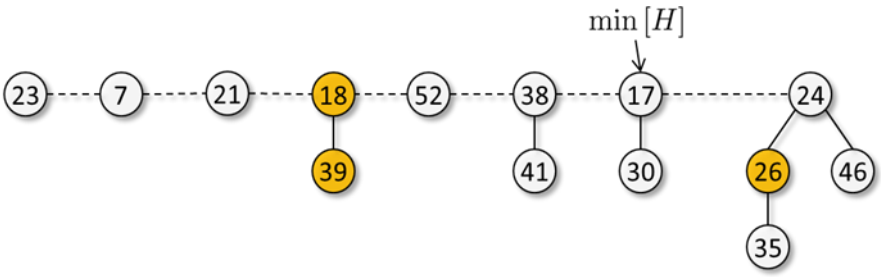
LINK(*H*, *y*, *x*)

1. διάγραψε τον κόμβο *y* από το ριζικό επίπεδο του σωρού *H*;
2. κάνε τον *y* παιδί του *x* και αύξησε κατά 1 το βαθμό *degree*[*x*];
3. *mark*[*y*] := 0;

Για παράδειγμα, ας θεωρήσουμε το σωρό Fibonacci που φαίνεται στην Εικόνα 14.35 από τον οποίο θέλουμε να εξαγάγουμε το ελάχιστο κλειδί 3. Ως πρώτο βήμα προσθέτουμε στο ριζικό επίπεδο του σωρού όλα τα παιδιά του κόμβου που αποθηκευει το ελάχιστο κλειδί και τον διαγράφουμε από το ριζικό επίπεδο (Εικόνα 14.36).

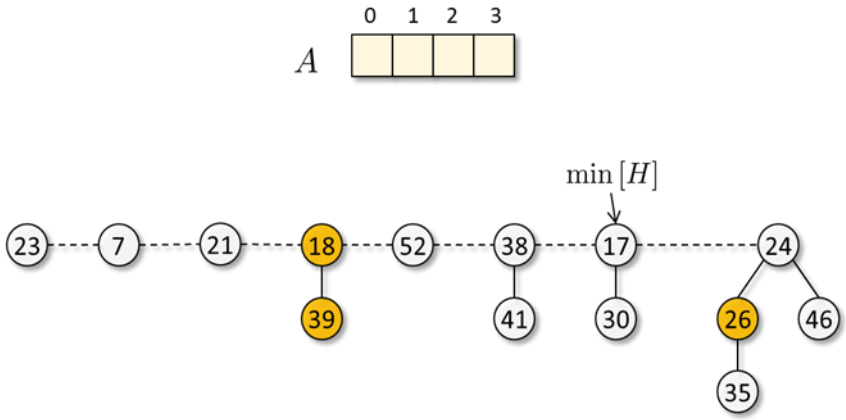


Εικόνα 14.35

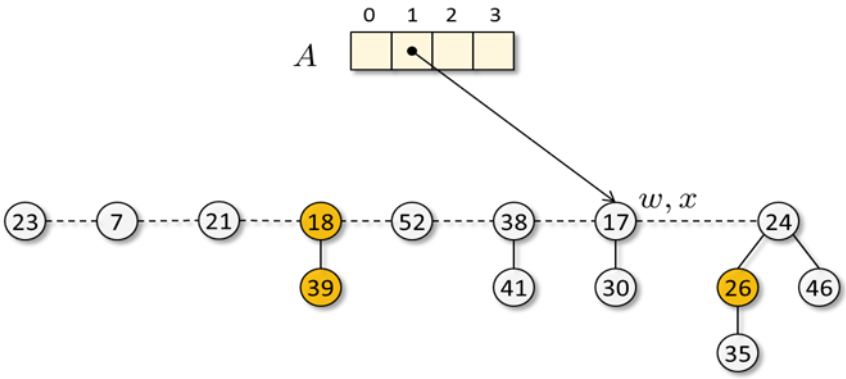


Εικόνα 14.36

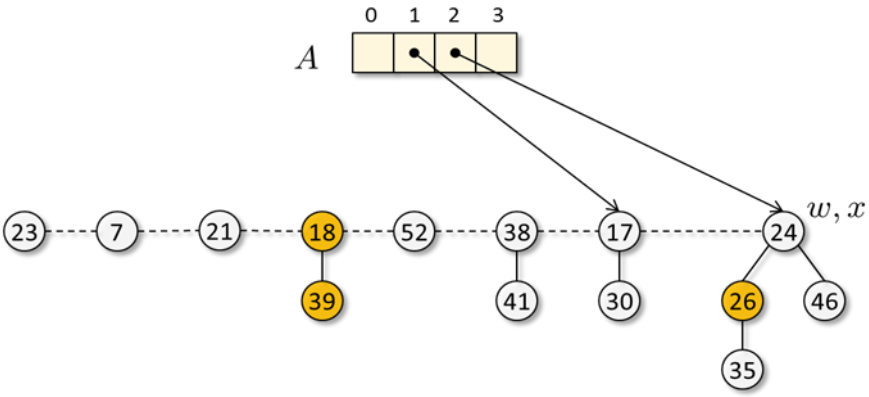
Στη συνέχεια εκτελούμε ενοποίηση με τη διαδικασία CONSOLIDATE(*H*), για την οποία χρειάζομαστε έναν πίνακα A με θέσεις για βαθμούς δένδρων 0, 1, 2, 3 (Εικόνα 14.37). Τα βήματα της διαδικασίας φαίνονται στις Εικόνες 14.38 – 14.52.



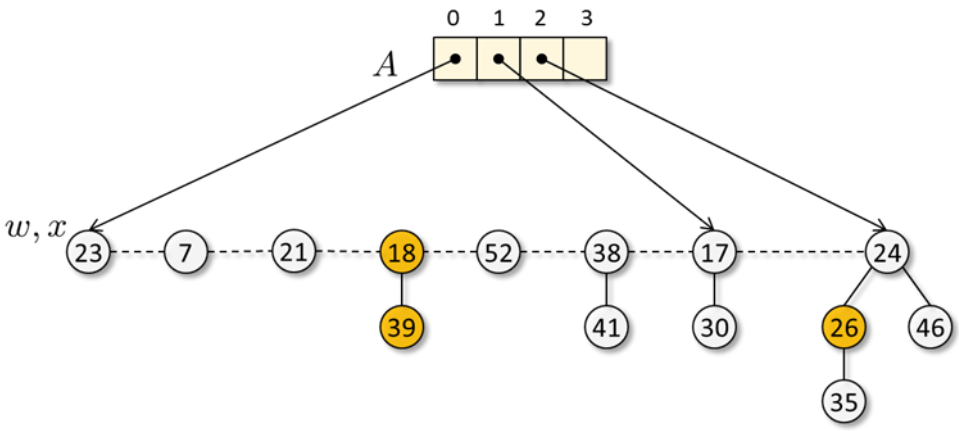
Εικόνα 14.37



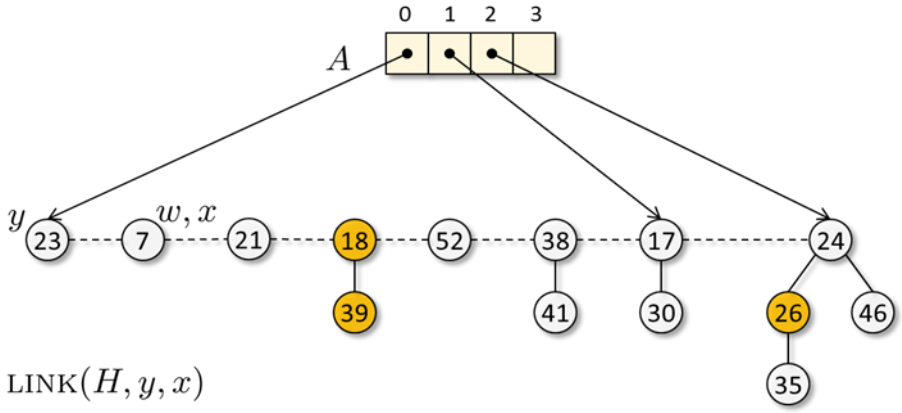
Εικόνα 14.38



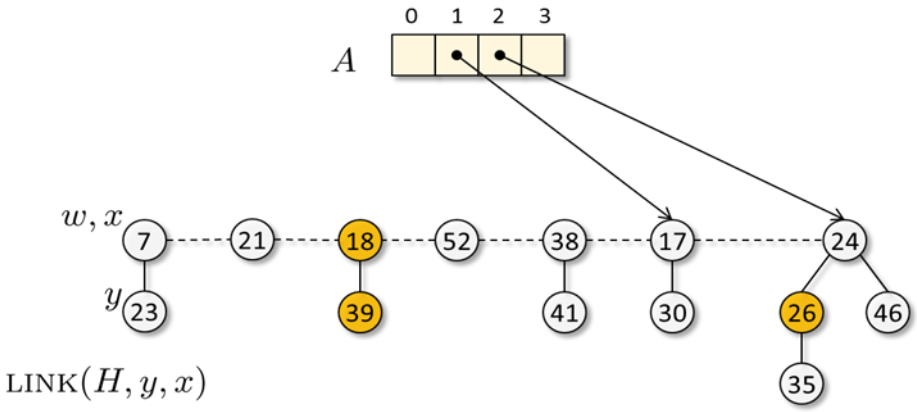
Εικόνα 14.39



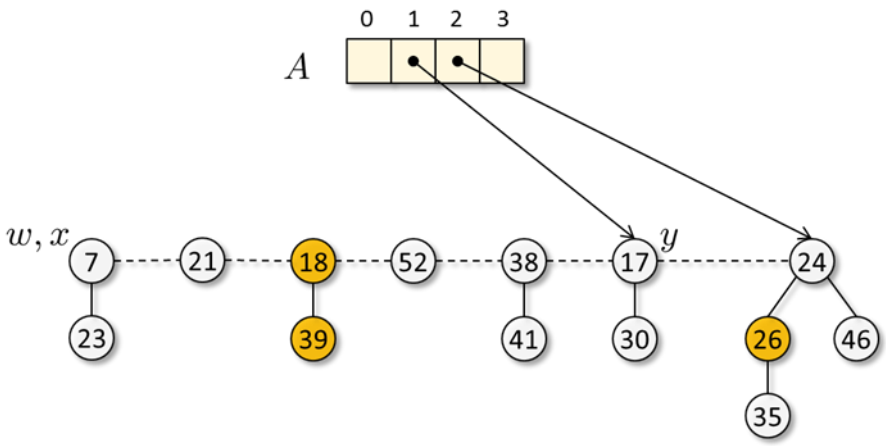
Εικόνα 14.40



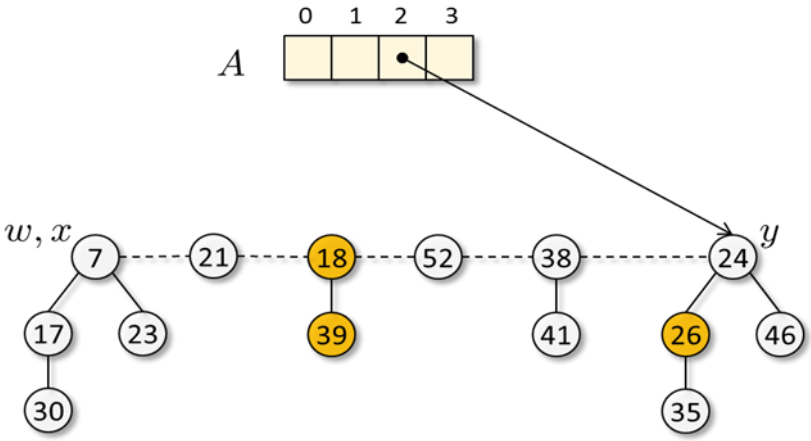
Εικόνα 14.41



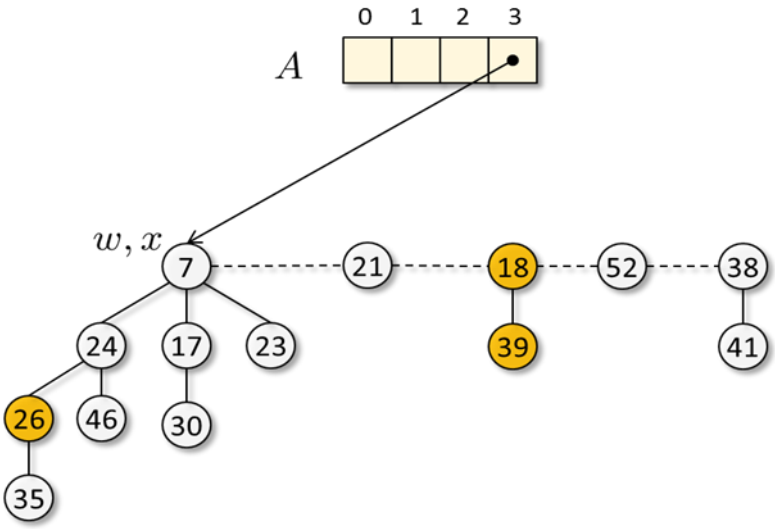
Εικόνα 14.42



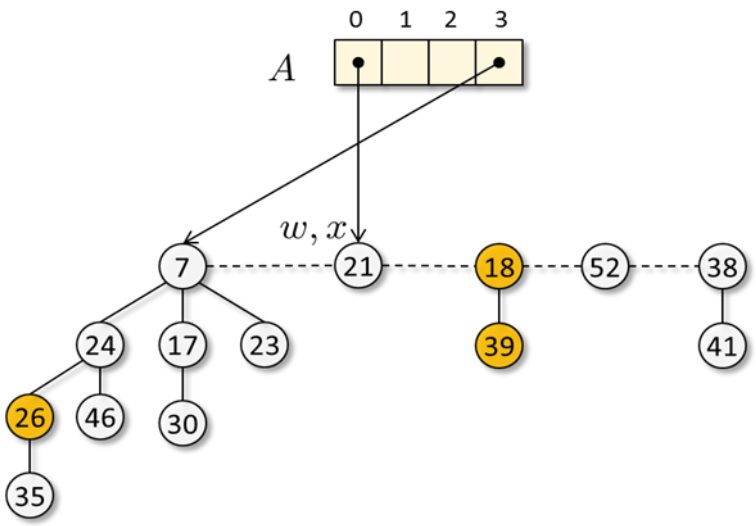
Εικόνα 14.43



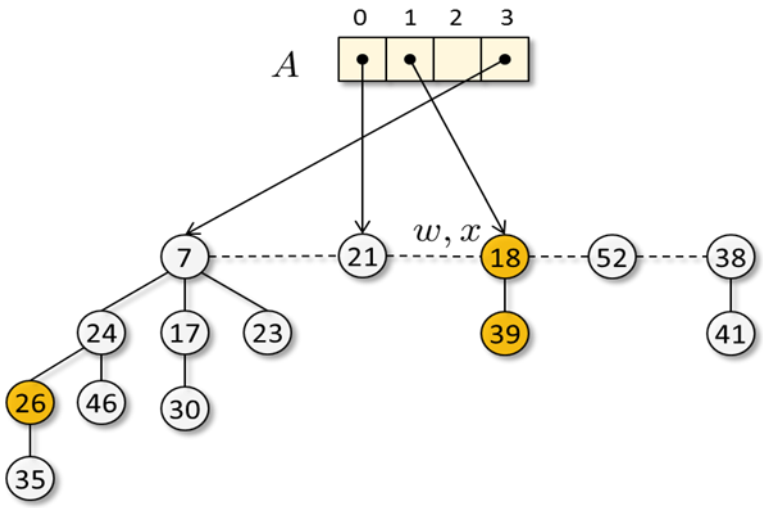
Εικόνα 14.44



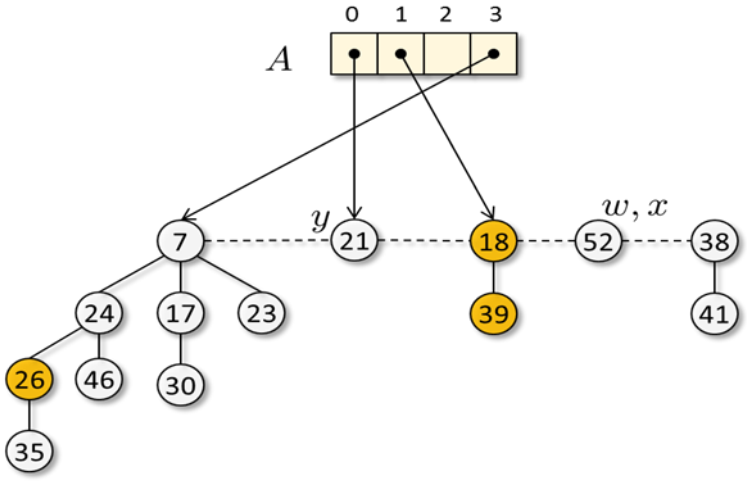
Εικόνα 14.45



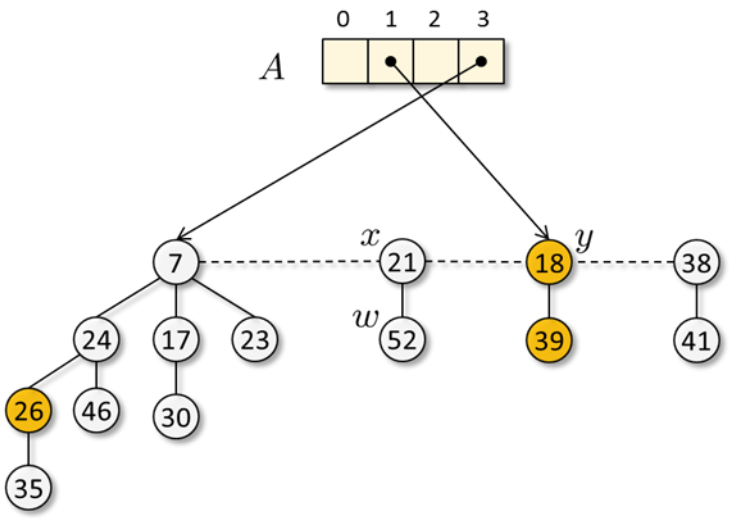
Εικόνα 14.46



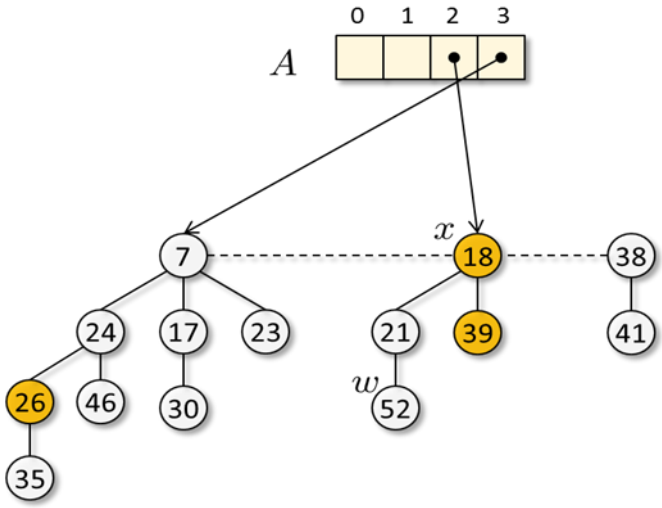
Εικόνα 14.47



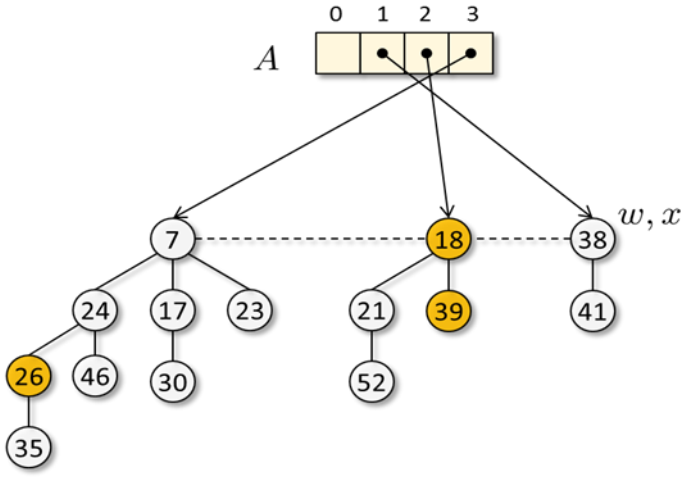
Εικόνα 14.48



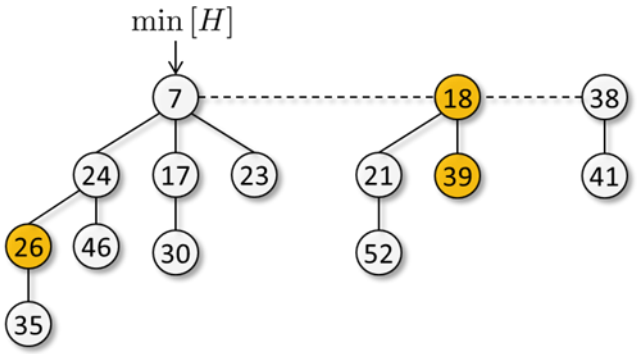
Εικόνα 14.49



Εικόνα 14.50



Εικόνα 14.51



Εικόνα 14.52: Ο τελικός σωρός Fibonacci μετά την ενοποίηση.

Το πραγματικό κόστος $c_{\text{extractmin}}$ της εξαγωγής του ελάχιστου κλειδιού είναι $O(D(n) + t(H))$. Το δυναμικό της δομής πριν από την εξαγωγή είναι $\Phi(H) = t(H) + 2\,m(H)$. Το δυναμικό της δομής μετά την εξαγωγή είναι $\Phi'(H) \leq (D(n) + 1) + 2\,m(H)$. Άρα, η μεταβολή του δυναμικού είναι $\Phi'(H) - \Phi(H) \leq (D(n) + 1) + 2\,m(H) - t(H) - 2\,m(H) = D(n) - t(H) + 1$ και, άρα, το αντισταθμιστικό κόστος είναι $c_{\text{extractmin}} + \Phi'(H) - \Phi(H) = O(D(n)) = O(\log n)$.

14.2.6 Μείωση κλειδιού

Η εκτέλεση αυτής της πράξης έχει ως αποτέλεσμα τα δένδρα του σωρού Fibonacci να μην παραμένουν δυωνυμικά. Για να μειώσουμε το κλειδί ενός κόμβου x από $key[x]$ σε $k < key[x]$ εκτελούμε τα εξής:

1. $key[x] := k$; $y := parent[x]$;
2. **εάν** $y \neq$ NULL **και** $key[x] < key[y]$
3. **τότε** CUT(H, x, y);
4. CASCADINGCUT(H, y);
5. **εάν** $key[x] < key[\min[H]]$
6. **τότε** $\min[H] := x$;

όπου οι διαδικασίες CUT(H,x,y) και CASCADINGCUT(H,y) είναι ως εξής:

CUT(H, x, y)

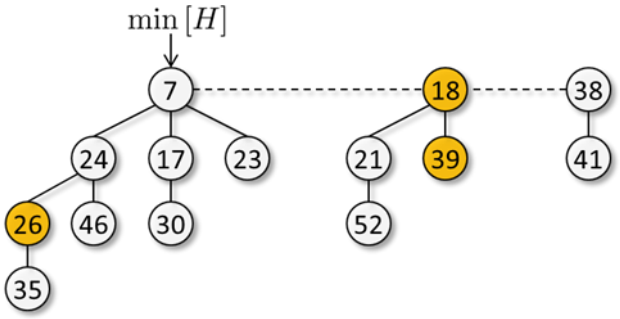
1. διάγραψε τον κόμβο x από τη λίστα παιδιών του κόμβου y και μείωσε κατά 1 τον βαθμό $degree[y]$ του y;
2. εισάγαγε τον x στη λίστα ριζικών κόμβων του σωρού H;
3. $parent[x] :=$ NULL; $mark[x] := 0$;

CASCADINGCUT(H, y)

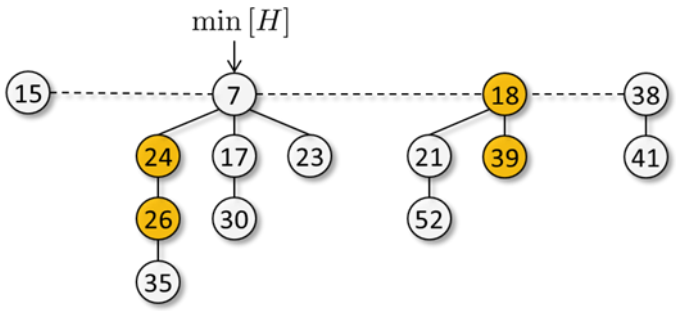
1. $z := parent[y]$;
2. **εάν** $z \neq$ NULL
3. **τότε εάν** $mark[y] = 0$
4. **τότε** $mark[y] := 1$;
7. **άλλως** CUT(H, y, z);
8. CASCADINGCUT(H, z);

Για παράδειγμα, ας θεωρήσουμε το σωρό Fibonacci της Εικόνας 14.53 και έστω ότι θέλουμε να μειώσουμε την τιμή του κλειδιού 46 σε 15. Το κλειδί του κόμβου που

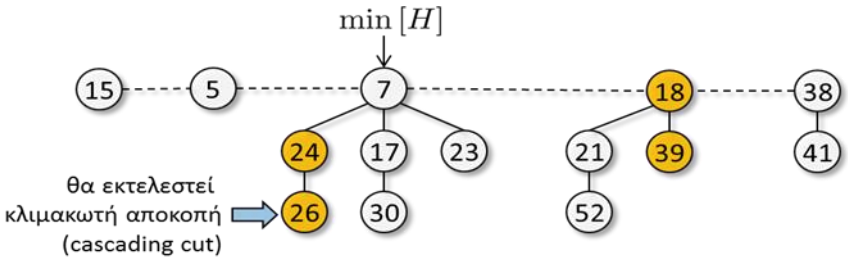
αποθηκεύει το 46 μειώνεται σε 15 και ο κόμβος μετατοπίζεται στη λίστα ριζικών κόμβων του σωρού, ενώ, επίσης, ο κόμβος-γονέας (που αποθηκεύει το 24) επισημαίνεται (Εικόνα 14.54). Ελέγχεται εάν χρειάζεται να ενημερωθεί ο δείκτης min[H], αλλά η τιμή του ελάχιστου κλειδιού δεν έχει αλλάξει, οπότε δεν αλλάζει και ο δείκτης min[H]. Παρόμοια, εάν στον σωρό που προκύπτει μειώσουμε το κλειδί 35 σε 5, το κλειδί του κόμβου που αποθηκεύει το 35 μειώνεται σε 5 και ο κόμβος μετατοπίζεται στη λίστα ριζικών κόμβων του σωρού (Εικόνα 14.55). Επίσης, επειδή ο κόμβος-γονέας (που αποθηκεύει το 26) είναι ήδη επισημασμένος, εκτελείται κλιμακωτή αποκοπή σε αυτόν τον κόμβο, με αποτέλεσμα να μεταφερθεί στη λίστα ριζικών κόμβων του σωρού (Εικόνα 14.56) και να συνεχίσουμε με τον κόμβο-γονέα του. Επειδή και αυτός ο κόμβος είναι επισημασμένος, εκτελείται και πάλι κλιμακωτή αποκοπή σε αυτόν τον κόμβο που μεταφέρεται στη λίστα ριζικών κόμβων του σωρού (Εικόνα 14.57) και συνεχίζουμε με τον κόμβο-γονέα του. Ο κόμβος-γονέας είναι η ρίζα, οπότε η κλιμακωτή αποκοπή σταματά. Τέλος, παρατηρούμε ότι άλλαξε η τιμή του ελάχιστου κλειδιού, οπότε ενημερώνεται και ο δείκτης min[H] (Εικόνα 14.58).



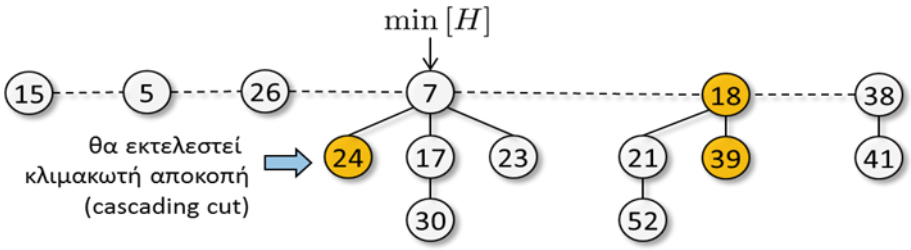
Εικόνα 14.53



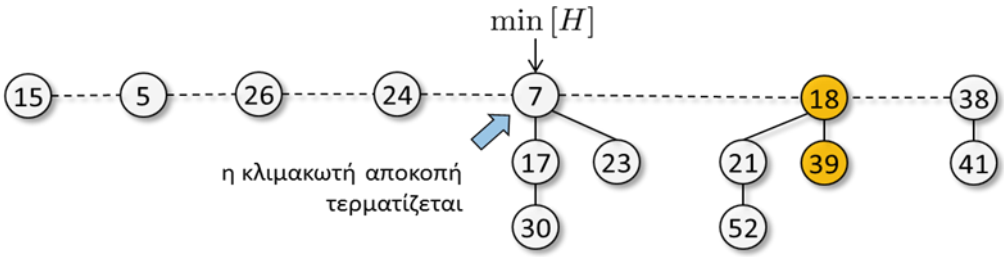
Εικόνα 14.54



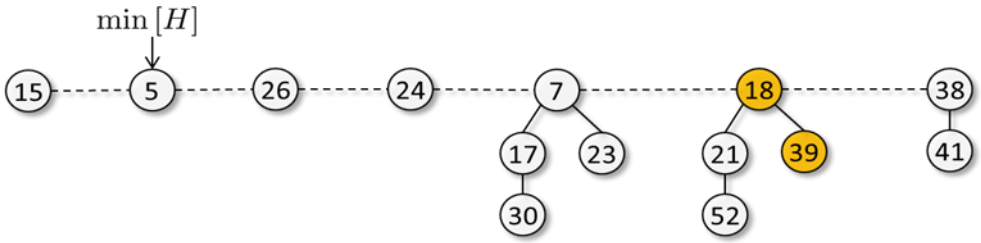
Εικόνα 14.55



Εικόνα 14.56



Εικόνα 14.57



Εικόνα 14.58

Για τον υπολογισμό του αντισταθιστικού κόστους της διαγραφής, ας υποθέσουμε ότι η διαδικασία κλιμακωτής αποκοπής εκτελέστηκε k φορές. Το πραγματικό κόστος $c_{\text{decreasekey}}$ της μείωσης κλειδιού είναι $O(k)$. Το δυναμικό της δομής πριν από τη μείωση είναι $\Phi(H) = t(H) + 2\,m(H)$. Το δυναμικό της δομής μετά τη μείωση είναι $\Phi'(H) = t'(H) + 2\,m'(H) \leq (t(H) + k) + 2\,(m(H) - k + 2)$, καθώς τα δένδρα στο σωρό αυξάνονται κατά k, ενώ τουλάχιστον k-2 παύουν να είναι επισημασμένοι. Άρα η μεταβολή του δυναμικού είναι $\Phi'(H) - \Phi(H) \leq t(H) + k + 2\,m(H) - 2\,k + 4 - t(H) - 2\,m(H) = 4 - k$. Συνεπώς, το αντισταθμιστικό κόστος της μείωσης κλειδιού είναι $c_{\text{decreasekey}} + \Phi'(H) - \Phi(H) = O(1)$.

14.2.7 Διαγραφή κλειδιού

Έστω x ο κόμβος τον οποίο θέλουμε να διαγράψουμε. Η διαγραφή γίνεται σε δύο βήματα:

1. Μειώνουμε το κλειδί του x σε τιμή μικρότερη από το ελάχιστο κλειδί στο σωρό.
2. Εκτελούμε εξαγωγή του ελάχιστου κλειδιού (που είναι το μειωμένο κλειδί του κόμβου x).

Το αντισταθμιστικό κόστος της διαγραφής ισούται με το άθροισμα των αντισταθμιστικών κοστών της μείωσης κλειδιού και της εξαγωγής του ελάχιστου κλειδιού. Από τις παραγράφους 14.2.5 και 14.2.6 συμπεραίνουμε ότι το αντισταθμιστικό κόστος της διαγραφής είναι $O(1) + O(\log n) = O(\log n)$.

Βιβλιογραφία

Cormen, T., Leiserson, C., Rivest, R., & Stain, C. (2001). *Introduction to Algorithms.* MIT Press (2nd edition).

Tarjan, R. E. (1983). *Data Structures and Network Algorithms.* Society for Industrial and Applied Mathematics.

Γεωργακόπουλος, Γ. Φ. (2011). *Δομές Δεδομένων*. Πανεπιστημιακές εκδόσεις Κρήτης.

Κεφάλαιο 15
Γλώσσα προγραμματισμού Java

Περιεχόμενα

15.1 Δομή της Java 280
15.1.1 Βασικοί Τύποι 283
15.1.2 Κλάσεις, Αντικείμενα, και Μέθοδοι 286
15.1.3 Πίνακες 290
15.1.4 Η κλάση String 290
15.2 Κληρονομικότητα και Πολυμορφισμός 291
15.3 Διασυνδέσεις και Αφηρημένες Κλάσεις 291
15.4 Γενικοί Τύποι 292
15.5 Ασκήσεις 293
Βιβλιογραφία 294

15.1 Δομή της Java

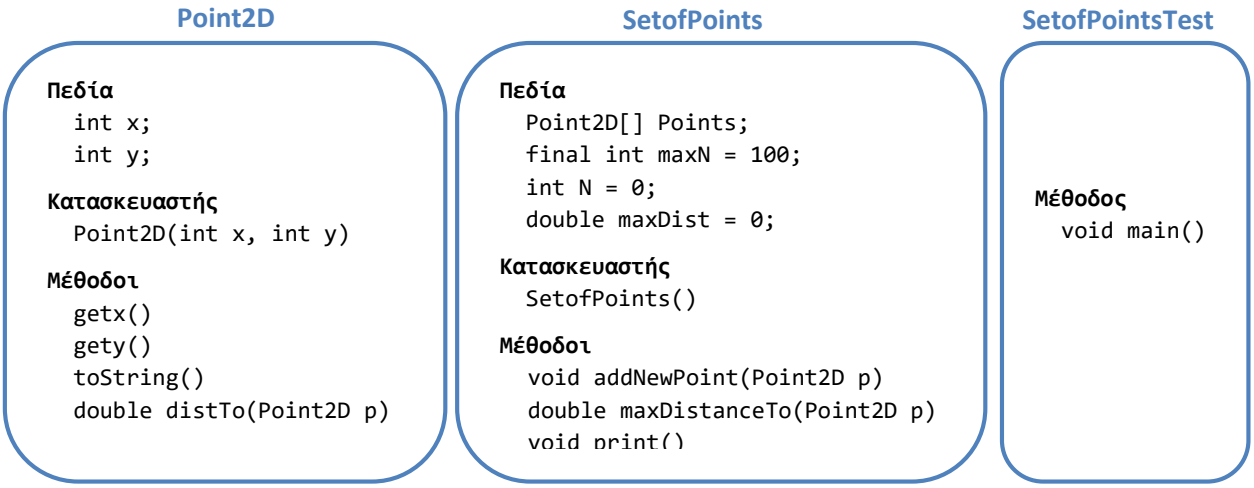
Στο παρόν κεφάλαιο στόχος είναι να γίνει μια σύντομη επισκόπηση στη γλώσσα προγραμματισμού Java, ξεκινώντας την παρουσίαση από απλά δομικά στοιχεία της γλώσσας και φτάνοντας στην ανάλυση του αντικειμενοστρεφούς προγραμματισμού και των αρχών που το διέπουν, δίνοντας στον αναγνώστη ένα βοήθημα το οποίο θα καλύπτει πλήρως τα θέματα που παρουσιάζονται στα παρόντα κεφάλαια αυτού του βιβλίου. Θα πρέπει να αναφέρουμε πως θεωρούμε ότι ο αναγνώστης έχει κάποιες γνώσεις σε μία γλώσσα προγραμματισμού υψηλού επιπέδου.

Ένα πρόγραμμα σε Java είναι ουσιαστικά μια κλάση, έστω class <Όνομα>, όπου το αρχείο που θα αποθηκεύει την κλάση αυτή θα είναι το <Όνομα>.java. Τα βασικά δομικά στοιχεία μιας κλάσης είναι τα πεδία, οι κατασκευαστές (constructors) και οι μέθοδοι (methods). Στην κλάση <Όνομα> θα πρέπει να περιέχεται μια μέθοδος main, η οποία είναι το σημείο εκκίνησης του προγράμματος. Ένα πρόγραμμα μπορεί να περιέχει περισσότερες από μία κλάσεις, όμως σε μία από αυτές θα πρέπει να υπάρχει η μέθοδος main.

Ακολουθεί ένα παράδειγμα προγράμματος σε Java το οποίο δημιουργεί τυχαία σημεία στο επίπεδο, υπολογίζει την ευκλείδεια απόσταση μεταξύ δύο σημείων και κρατάει σε ένα πίνακα τα στοιχεία με τη μεγαλύτερη απόσταση.

Παράδειγμα 1. Δημιουργία ενός συνόλου από τυχαία σημεία στο επίπεδο.

Στην Εικόνα 15.1 παρουσιάζονται διαγραμματικά οι κλάσεις που υλοποιούνται στο Παράδειγμα 1 και ακολουθεί ο κώδικας σε Java.



Εικόνα 15.1. Περιγραφή των κλάσεων του Παραδείγματος 1.

Αρχείο `SetofPointsTest.java`.

```
import java.util.Random;

class Point2D
{
   private int x;
   private int y;

   public Point2D(int x, int y) {
               this.x = x;
               this.y = y;
   }

   public int getx(){ return x;}
   public int gety(){ return y;}

  public String toString(){ return "x:"+getx()+" y:"+gety();}
   //public String toString(){ return "x:"+x+" y:"+y;}

   public double distTo(Point2D p) {
        double dx = (double)(this.x-p.x);
        double dy = (double)(this.y-p.y);
        return Math.sqrt(dx*dx+dy*dy);
   }
}
```

```
class SetofPoints
{
   Point2D[] Points;
   final int maxN = 100;
   int N = 0;
   double maxDist = 0;

   public SetofPoints() { Points = new Point2D[maxN];}

   public void addNewPoint(Point2D p) {
     double dist = maxDistanceTo(p);
     if (dist > maxDist) maxDist=dist;
     Points[N++] = p;
   }

   double maxDistanceTo(Point2D p) {
     double dist = 0;
     double newdist = 0;

     for (int i=0; i<N; i++){
        newdist = p.distTo(Points[i]);
        if (dist < newdist) dist=newdist;
     }
     return dist;
   }

   public void print(){
       for(int i=0; i<Points.length && Points[i]!=null; i++)
              System.out.println(Points[i]);
       }
}
```

```
public class SetofPointsTest
{
   public static void main(String[] args){
     int min=1;
     int max=10;
     Random rand = new Random();
     Point2D p1 = new Point2D(rand.nextInt((max-min)+1) + min,rand.nextInt((max-
     min) + 1) + min);
     Point2D p2 = new Point2D(rand.nextInt((max-min)+1) + min,rand.nextInt((max-
     min) + 1) + min);


     System.out.println(p1);
     System.out.println(p1.distTo(p2));

     SetofPoints S=new SetofPoints();
     S.addNewPoint(p1);
     S.addNewPoint(p2);
     S.print();
   }
}
```

15.1.1 Βασικοί Τύποι

Οι μεταβλητές, οι εκφράσεις, οι αναθέσεις, καθώς και η ροή ελέγχου στη Java είναι παρόμοιες με τις αντίστοιχες στις περισσότερες γλώσσες υψηλού επιπέδου. Σε αυτήν την ενότητα περιγράφουμε σχετικές λεπτομέρειες. Στον Πίνακα 15.1 αναφέρονται οι βασικοί τύποι δεδομένων που υποστηρίζει η Java.

Πίνακας 15.1: Βασικοί τύποι δεδομένων της Java.

Όνομα τύπου	Τιμή	Μνήμη
boolean	true/false	1 byte
char	Χαρακτήρας (Unicode)	2 bytes
byte	Ακέραιος	1 byte
short	Ακέραιος	2 bytes
int	Ακέραιος	4 bytes
long	Ακέραιος	8 bytes
float	Πραγματικός	4 bytes
double	Πραγματικός	8 bytes

Στον Πίνακα 15.2 έχουμε συγκεντρώσει ορισμένες από τις βασικές προτάσεις που χρησιμοποιούνται κατά το προγραμματισμό με τη Java και, στη συνέχεια, θα τις αναλύσουμε περαιτέρω.

Δήλωση. Μια μεταβλητή μπορεί να δηλωθεί ακολουθώντας οποιονδήποτε από τους παραπάνω βασικούς τύπους και θα πρέπει να δηλώνεται πάντα πριν από τη χρήση της. Οφείλουμε να σημειώσουμε ότι μια μεταβλητή μπορεί, επίσης, να είναι και άλλου τύπου, όπως είναι η κλάση String, στην οποία θα αναφερθούμε σε επόμενη ενότητα και η οποία, αν και δεν αποτελεί βασικό τύπο, χρησιμοποιείται ευρέως στη Java.

Ανάθεση. Μια πρόταση ανάθεσης χρησιμοποιείται, για να αλλάξει μια μεταβλητή. Ο τελεστής ανάθεσης είναι το σύμβολο της ισότητας «=» και κάθε πρόταση αποτελείται από το όνομα μιας μεταβλητής αριστερά από το σύμβολο «=» και από μια σταθερή τιμή ανάλογα με τον τύπο της μεταβλητής ή από μία έκφραση, δεξιά του «=». Μια σταθερή τιμή μπορεί να είναι είτε κάποιο λεκτικό null, true, false, ένα αλφαριθμητικό, είτε μια τιμή int, float, double, char, ανάλογα με το τύπο πάντα της μεταβλητής. Μια έκφραση είναι μια σύνθεση λεκτικών και μεταβλητών μέσω τελεστών. Οι τελεστές που χρησιμοποιεί η Java είναι ο τελεστής ανάθεσης, όπως αναφέραμε και παραπάνω, είναι το σύμβολο της ισότητας «=», οι αριθμητικοί, λογικοί τελεστές, αυτοί της αύξησης/μείωσης αλλά και οι τελεστές σε μπιτ. Ακολουθεί μια επισκόπηση των τελεστών στη Java.

Αριθμητικοί		Λογικοί	
+	πρόσθεση	<	μικρότερο από
-	αφαίρεση	<=	μικρότερο από ή ίσο με
*	πολλαπλασιασμός	==	ίσο με
/	διαίρεση	!=	διαφορετικό
%	υπόλοιπο ακεραίας διαίρεσης	>=	μεγαλύτερο από ή ίσο με
		>	μεγαλύτερο από
		!	λογική άρνηση (not)
		&&	λογικό και (and)
		||	λογικό ή (or)

Αύξησης/Μείωσης		Μπιτ ή Δυαδικοί	
++	αύξηση κατά ένα	~	συμπλήρωμα
--	μείωση κατά ένα	&	λογικό και
		|	λογικό ή
		<<	αριστερή μπιτ ολίσθηση
		>>	δεξιά μπιτ ολίσθηση

Επίσης, στη Java συναντούμε και ορισμένους ειδικούς τελεστές, όπως η εναλλακτική αποτίμηση υπό συνθήκη (?:), η εκχώρηση με πράξη (+=, -=, *=, /=, &=, %=) και ο τελεστής new για τη δημιουργία αντικειμένου.

Πίνακας 15.2: Βασικές προτάσεις στη Java.

Πρόταση	Παράδειγμα	Εξήγηση
Δήλωση	`int i;` `double k;`	Δημιουργία μιας ακέραιης μεταβλητής με όνομα i και μιας μεταβλητής κινητής υποδιαστολής με όνομα k.
Ανάθεση	`k=2;` `exp=Math.sqrt(dx8dx+dy*dy);`	Ανάθεση σε μια μεταβλητή τιμή, είτε απευθείας είτε αποτιμώντας μια αριθμητική έκφραση.
Αρχικοποίηση	`int i=0;` `double k=3.14;`	Ανάθεση μιας αρχικής τιμής σε μεταβλητές.
Τελεστές αύξησης/μείωσης	`i++; j--;` `i+=1; j-=1`	i=i+1; j=j-1
Δομή ελέγχου (if)	`if (i <0) i++;`	Εάν η λογική έκφραση (i <0) είναι αληθής, τότε εκτελείται η εντολή i++;
Δομή ελέγχου (if-else)	`if (i>j) m=i;` `else m=j;`	Εάν η λογική έκφραση (i >j) είναι ψευδής, τότε εκτελείται η εντολή m=j;

Δομή επανάληψης (while)	`while ( u != null ) {`
	`  leftSize=(u.left != null) ?`
	`u.left.size:0;`
	
	`rightSize=(u.right!=null)?u.right.size:0;`
	`  u.size = leftSize + rightSize + 1;`
	`  u = u.parent;`
	`}`
		Όσο η συνθήκη (u != null) είναι αληθής, εκτελούνται οι εντολές μέσα στο block της while.

Δομή επανάληψης (for)	`for(int i=0; i<10;i++)` `  System.out.println(i);`	Η δομή επανάληψης θα εκτελέσει την εντολή που είναι στο block της 10 φορές.
Κλήση μεθόδου	`String line = input.nextInt();`	Κλήση άλλων μεθόδων της Java
Επιστροφή τιμής	`return true;`	Επιστροφή τιμής από μια μέθοδο.

Μπορούμε να αλλάξουμε τον τύπο μιας έκφρασης κάνοντας casting, μετατρέποντας δηλαδή τον τύπο της τιμής της έκφρασης σε ισοδύναμη τιμή άλλου τύπου. Η σύνταξη μετατροπής τύπου είναι: *type* var = (*type*) exp, όπου *type* είναι ο τύπος που επιθυμούμε να έχει η έκφραση exp. Για παράδειγμα, εάν η τιμή της έκφρασης είναι 5.3, θέτοντας (int) 5.3, το αποτέλεσμα θα είναι 5.

Αρχικοποίηση. Μια δήλωση μεταβλητής μπορεί να συνδυαστεί και με μια ανάθεση αρχικής τιμής.

Δομές ελέγχου και επανάληψης. Οι δομές ελέγχου και επανάληψης στη Java είναι ίδιες με αυτές των γλωσσών C και C++. Συνοπτικά οι δομές αυτές είναι οι εξής:

- ***if*** (λογική έκφραση) {Ακολουθία προτάσεων} ***else*** {Ακολουθία προτάσεων}
- ***while*** (λογική έκφραση) {Ακολουθία προτάσεων}
- ***do*** {Ακολουθία προτάσεων} ***while*** (λογική έκφραση)
- ***for*** (Αρχικοποίηση; λογική έκφραση; Ενημέρωση) {Ακολουθία προτάσεων}
- ***switch*** (λογική έκφραση) {case 1: Ακολουθία προτάσεων; break; … case n: Ακολουθία προτάσεων; break; default: Ακολουθία προτάσεων; break;}

Κάθε μία δομή ουσιαστικά ελέγχει μια λογική έκφραση εάν είναι αληθής (true) ή ψευδής (false). Η Java έχει εντολές που επιτρέπουν την αλλαγή της ροής ενός προγράμματος, τις **return**, **break** και **continue**.

15.1.2 Κλάσεις, Αντικείμενα, και Μέθοδοι

Στην ενότητα αυτή παρουσιάζονται τα χαρακτηριστικά του αντικειμενοστραφούς προγραμματισμού της Java, που αποτελούν και τον πυρήνα της γλώσσας.

Όπως αναφέρθηκε και στη εισαγωγή, ένα πρόγραμμα σε Java αποτελείται από μία ή περισσότερες κλάσεις. Η τιμή ενός τύπου κλάσης ονομάζεται **αντικείμενο** (object), όμως συνήθως αναφέρεται ως αντικείμενο της κλάσης ή ως στιγμιότυπο της κλάσης. Δεδομένου ότι μια κλάση αποτελείται από πεδία, κατασκευαστές και μεθόδους, ένα αντικείμενο είναι ένας συγκεκριμένος συνδυασμός αυτών. Ουσιαστικά τα πεδία είναι τα δεδομένα που κρατάει η κλάση, οι κατασκευαστές (constructors) είναι οι συναρτήσεις αρχικοποίησης αυτών των δεδομένων, ενώ οι μέθοδοι (methods) είναι οι ενέργειες επί των δεδομένων.

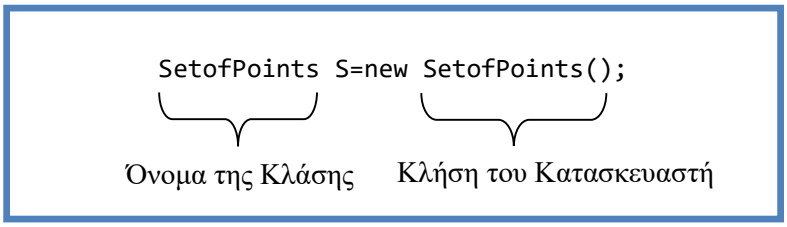
Η σύνταξη μιας κλάσης είναι της μορφής:

public class Όνομα_Κλάσης {Πεδία, Κατασκευαστές, Μέθοδοι}

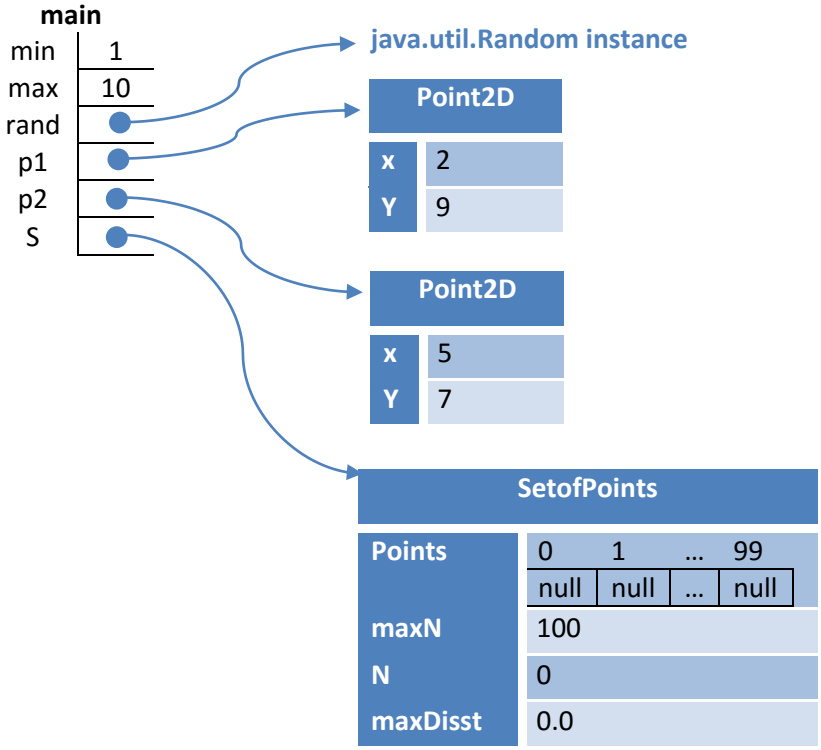
Η κλάση αυτή αποθηκεύεται σε ένα αρχείο που έχει το ίδιο όνομα με την κλάση. Η λέξη κλειδί public ονομάζεται τροποποιητής της κλάσης και περιγράφει μια κλάση η οποία είναι ορατή σε κλάσεις που έχουν οριστεί σε άλλα αρχεία.

Στο Παράδειγμα 1, έχουμε ορίσει τις κλάσεις Point2D, SetofPoints και SetofPointsTest. Στην αρχή κάθε κλάσης ορίζουμε τα δεδομένα που αντιπροσωπεύουν την κλάση. Η κλάση Point2D είναι μια κλάση που κρατάει δύο σημεία στο χώρο x και y, ενώ η κλάση SetofPoints κρατάει έναν πίνακα από σημεία, έναν πίνακα από αντικείμενα τύπου Point2D. Ένα νέο αντικείμενο δημιουργείται χρησιμοποιώντας τον τελεστή **new**. Στο παράδειγμά μας, η πρόταση `SetofPoints S=new SetofPoints();` δημιουργεί ένα αντικείμενο της κλάσης SetofPoints με το

όνομα S. Ουσιαστικά, καλώντας τον τελεστή new αυτό που συμβαίνει είναι να δημιουργείται ένα νέο αντικείμενο της κλάσης δυναμικά στη μνήμη, όπου αρχικοποιούνται τα πεδία της κλάσης σε προκαθορισμένες τιμές. Έπειτα καλείται ο κατασκευαστής της κλάσης `SetofPoints()`, που δίνει τιμές στα πεδία της κλάσης και με την επιστροφή του κατασκευαστή επιστρέφεται μια αναφορά στο νέο αυτό αντικείμενο μέσω της μεταβλητής S.



Στο παράδειγμά μας, στην κλάση SetofPointsTest δημιουργούμε αρχικά ένα αντικείμενο τύπου Random, δύο αντικείμενα τύπου Point2D, τα αντικείμενα p1 και p2, καθώς και ένα αντικείμενο τύπου SetofPoints. Στην Εικόνα 15.2 παρουσιάζεται σχηματικά η κατάσταση της μνήμης μετά τη δημιουργία αυτών των αντικειμένων.



Εικόνα 15.2: Κατάσταση μνήμης μετά της δημιουργία των αντικειμένων του Παραδείγματος 1.

Εάν θέλουμε να προσπελάσουμε τη μεταβλητή x, τυπώνοντας, για παράδειγμα, την τιμή της στη μέθοδο main, τότε θα χρησιμοποιήσουμε τον τελεστή «.» και θα γράψουμε την εντολή `System.out.println(p1.x);`

Γενικότερα, για να προσπελάσουμε τα περιεχόμενα μιας κλάσης της οποίας έχουμε δημιουργήσει ένα αντικείμενο, χρησιμοποιούμε τον τελεστή τελεία «.» μετά το όνομα του αντικειμένου και ακολουθεί το πεδίο ή η μέθοδος της κλάσης που θέλουμε να προσπελάσουμε.

Στη Java οι **μέθοδοι (methods)** είναι παρόμοιες με τις συναρτήσεις και τις διαδικασίες σε άλλες γλώσσες υψηλού επιπέδου. Η γενική σύνταξη μιας μεθόδου είναι η εξής:

public *Επιστρεφόμενος_Τύπος Όνομα_Μεθόδου* (*Λίστα Παραμέτρων*)

{

Ακολουθία Προτάσεων

}

Υπάρχουν δύο ειδών μέθοδοι, αυτές που επιστρέφουν μια τιμή και αυτές που είναι τύπου void, επιτελούν κάποιες ενέργειες, αλλά δεν επιστρέφουν μια τιμή. Έτσι, ο *Επιστρεφόμενος_Τύπος* μιας μεθόδου είναι είτε void είτε οποιοσδήποτε από τους βασικούς τύπους του Πίνακα 15.1, είτε ένας πίνακας είτε ακόμη και ένα αντικείμενο. Στην περίπτωση που η μέθοδος επιστρέφει ένα αντικείμενο ως *Επιστρεφόμενος_Τύπος* γράφουμε το όνομα του τύπου του αντικειμένου. Παρομοίως, και η λίστα παραμέτρων μπορεί να είναι οποιουδήποτε τύπου, είτε κενή, οπότε δεν δηλώνουμε τίποτε μέσα στις παρενθέσεις, είτε κάποιων από τους βασικούς τύπους του Πίνακα 15.1, είτε κάποια αντικείμενα. Στη περίπτωση που ως παράμετρο δηλώνουμε αντικείμενο, τότε στην παρένθεση γράφουμε το τύπου του αντικειμένου και ένα όνομα μεταβλητής. Για παράδειγμα, η δήλωση:

Point2D example (int x, double k, SetofPoints P)

Περιγράφει μια μέθοδο με όνομα example, η οποία επιστρέφει ένα αντικείμενο τύπου Point2D και ως παραμέτρους δέχεται μια μεταβλητή τύπου int με όνομα x, μια μεταβλητή τύπου double με όνομα k και ένα αντικείμενο τύπου SetofPoints με όνομα P. Στο παράδειγμά μας, η κλάση Point2D έχει τη μέθοδο `public double distTo(Point2D p).` Όπως και με τις μεταβλητές, έτσι και με τις μεθόδους, η κλήση μιας μεθόδου από μία κλήση γίνεται με τη βοήθεια του τελεστή «.» και υπό την προϋπόθεση ότι έχουν δημιουργηθεί αντικείμενα της κλάσης που περιέχει τη μέθοδο. Έτσι, η κλήση της μεθόδου `distTo` πραγματοποιείται στη `main` ως εξής: `p1.distTo(p2).` Παρατηρούμε ότι το αντικείμενο p2 τύπου Point2D μπαίνει ως όρισμα στη μέθοδο `distTo` σύμφωνα με τη περιγραφή της μεθόδου.

Παρατηρούμε στο παράδειγμά μας ότι αριστερά από τις δηλώσεις των πεδίων και των μεθόδων υπάρχουν οι δεσμευμένες λέξεις public, private, final. Ό,τι είναι ορισμένο ως public σε μία κλάση είναι προσβάσιμο από μία άλλη κλάση, ό,τι είναι ορισμένο ως private σε μία κλάση δεν είναι προσβάσιμο από μία άλλη κλάση, ενώ ό,τι είναι δηλωμένο ως final δεν μπορεί να παρακαμφθεί από άλλη κλάση.

Ένας **κατασκευαστής** (constructor) είναι ένα ειδικό είδος μεθόδου, που χρησιμοποιείται για την αρχικοποίηση των πεδίων ενός αντικειμένου και καλείται κατά τη δημιουργία των αντικειμένων μιας κλάσης, δηλαδή με τη χρήση του τελεστή new. Ο κατασκευαστής σε μια κλάση έχει πάντα το ίδιο όνομα με την κλάση, δεν έχει τύπο επιστρεφόμενης τιμής και μπορεί

να πάρει είτε καμία είτε οσεσδήποτε παραμέτρους είναι αναγκαίες. Στο παράδειγμά μας οι κατασκευαστές που χρησιμοποιούμε δεν έχουν καμία παράμετρο. Μία κλάση μπορεί να έχει οσουσδήποτε κατασκευαστές, οι οποίοι θα διαφοροποιούνται στη λίστα των παραμέτρων. Κατά τη δημιουργία ενός αντικειμένου καλείται πάντα ένας κατασκευαστής, που ανάλογα με τις παραμέτρους που του θέτουμε καθορίζει και ποιος θα κληθεί από αυτούς που έχουν ορισθεί στην κλάση.

Στο παράδειγμά μας, ο κατασκευαστής της κλάσης Point2D έχει δύο παραμέτρους, int x και int y, και η κλάση έχει δύο πεδία, int x και int y. Προκειμένου να εκχωρηθούν οι τιμές των παραμέτρων στα πεδία της κλάσης, χρησιμοποιούμε τη παράμετρο **this**. Η παράμετρος this είναι ένα είδος κρυφής παραμέτρου. Όταν καλείται μια μέθοδος, στη θέση της παραμέτρου this εισάγεται αυτομάτως το αντικείμενο κλήσης της μεθόδου.

Επίσης, σε μια κλάση μπορεί να υπάρχουν μέθοδοι με το ίδιο όνομα. Ακόμη και μεταξύ δύο ή περισσοτέρων κλάσεων μπορεί να υπάρχουν μέθοδοι με το ίδιο όνομα. Αυτό ονομάζεται **υπερφόρτωση** (**overloading**). Κάθε μία από τις μεθόδους με το ίδιο όνομα, θα έχει τον ίδιο τύπο επιστρεφόμενης τιμής και θα διαφοροποιείται στη λίστα των παραμέτρων.

Είναι, επίσης, σημαντικό να αναφέρουμε ότι η Java αποτελείται από ένα σύνολο από έτοιμες κλάσεις, τις οποίες ο προγραμματιστής έχει στη διάθεσή του. Στο παράδειγμά μας, χρησιμοποιούμε την κλάση **Random** (δείτε περιγραφή της κλάσης στο σχετικό σύνδεσμο: https://docs.oracle.com/javase/8/docs/api/java/util/Random.html), για να παραγάγουμε τυχαίους ακεραίους αριθμούς. Για να χρησιμοποιήσουμε την έτοιμη αυτή κλάση της Java, πρέπει στην αρχή του προγράμματος να δηλώσουμε **import** java.util.Random; Η δήλωση αυτή εισάγει την κλάση Random από το πακέτο java.util. Η δήλωση αυτή έχει πληροφοριακό χαρακτήρα, δηλαδή ο κώδικας της Random δεν εισάγεται στην πραγματικότητα και το μέγεθος του αρχείου δε μεταβάλλεται. Με παρόμοιο τρόπο μπορούν να εισαχθούν όλα τα περιεχόμενα του πακέτου java.util, χρησιμοποιώντας τον ειδικό χαρακτήρα * ως εξής:

```
import java.util.*;
```

Στην περίπτωση που πρέπει να εισαχθούν περισσότερα πακέτα, είναι δυνατόν να εισαχθούν περισσότερες δηλώσεις import στην αρχή του αρχείου.

Η κλάση **Math** είναι ένα ακόμη χαρακτηριστικό παράδειγμα έτοιμης κλάσης της Java, η χρήση της οποίας δεν απαιτεί κάποια δήλωση import. Η κλάση Math προσφέρει τυπικές μαθηματικές μεθόδους. Στο παράδειγμά μας χρησιμοποιήσαμε τη Math για τον υπολογισμό της τετραγωνικής ρίζας. Μερικές μέθοδοι της Math είναι οι εξής:

- double pow(double a, double b),
- double abs(double a),
- double min(double n1, double n2)
- double max(double n1, double n2)
- int round(double a)

Η κλάση Math έχει, επίσης, δύο προκαθορισμένες σταθερές, την E και την PI. Η σταθερά PI, η οποία συμβολίζεται με το σύμβολο π στους μαθηματικούς υπολογισμούς, ισούται με 3.14159. Η σταθερά E αποτελεί τη βάση του φυσικού λογάριθμου, που συμβολίζεται με *e* στους μαθηματικούς τύπους και ισούται κατά προσέγγιση με 2.72. Η παρακάτω έκφραση υπολογίζει το εμβαδόν του κύκλου δεδομένης της ακτίνας του:

E = Math.PI*radius*radius;

15.1.3 Πίνακες

Ένας πίνακας είναι μια δομή δεδομένων που χρησιμοποιείται για την επεξεργασία μιας συλλογής δεδομένων ίδιου τύπου, όπως είναι για παράδειγμα μια ακολουθία αριθμών τύπου int ή ακόμη ένα σύνολο αντικειμένων, όπως στο παράδειγμά μας ο πίνακας Points, που αποθηκεύει ένα σύνολο από αντικείμενα τύπου Point2D.

Ένας τρόπος δήλωσης και αρχικοποίησης ενός πίνακα στη Java είναι ο εξής:

Τύπος_Πίνακα[] Όνομα_Μεταβλητής = {*val*1, *val*2, …, *val*n}

όπου ο *Τύπος_Πίνακα* μπορεί να είναι οποιοσδήποτε βασικός τύπος της Java (βλ. Πίνακας 15.1). Για παράδειγμα θεωρείστε τη παρακάτω δήλωση ενός πίνακα με άρτιους ακεραίους αριθμούς:

int[] oddN = {2, 4, 6, 8}

Στην περίπτωση που θέλουμε να δημιουργήσουμε έναν πίνακα χωρίς να αρχικοποιήσουμε τις τιμές του ακολουθούμε τη παρακάτω σύνταξη:

Τύπος_Πίνακα[] Όνομα_Μεταβλητής = **new** *Τύπος_Πίνακα*[Μέγεθος_Πίνακα];

Στην κλάση SetofPoints δημιουργήσαμε τον πίνακα Point ως εξής:

```
Point2D[] Points = new Point2D[maxN];
```

Ο πίνακας Points έχει μέγεθος maxN και η αναφορά σε κάθε κελί του πίνακα ξεκινάει από το 0 έως το maxN-1. Η προσπέλαση στο μήκος ενός πίνακα μπορεί να γίνει και με τη δήλωση: Points.length, άρα τα κελιά του πίνακα Points αριθμούνται από 0 έως Points.length -1.

Επίσης, στη Java μπορούμε να ορίσουμε και πολυδιάστατους πίνακες με παρόμοιο τρόπο:

double[][] array2D;
int[][][] array3D;

Πρέπει να σημειώσουμε ότι η προκαθορισμένη (default) τιμή των στοιχείων ενός πίνακα είναι null.

15.1.4 Η κλάση String

Στον Πίνακα 15.1 αναφερθήκαμε στους βασικούς τύπους της Java, όπου είδαμε ότι δεν υπάρχει κάποιος τύπος για τα αλφαριθμητικά. Η Java χειρίζεται τα αλφαριθμητικά μέσω της κλάσης String. Ένα αλφαριθμητικό είναι ένα σύνολο από χαρακτήρες γραμμένο μέσα σε εισαγωγικά, για παράδειγμα “Hello Java”. Έτσι, ένα αλφαριθμητικό στη Java είναι ένα αντικείμενο τύπου String. Η δήλωση και η ανάθεση τιμής μιας μεταβλητής τύπου String είναι η εξής: String var = “Data Structures”;

Στην περίπτωση των αλφαριθμητικών μπορούμε να χρησιμοποιήσουμε τον τελεστή της πρόσθεσης για τη συνένωση δύο αλφαριθμητικών. Για παράδειγμα η δήλωση:

String var2 = var1 + “!”;

θα επιστρέψει το εξής αποτέλεσμα: “Data Structures!”

15.2 Κληρονομικότητα και Πολυμορφισμός

Στην ενότητα αυτή θα αναφερθούμε σε μια από τις κύριες τεχνικές του αντικειμενοστραφούς προγραμματισμού, που είναι η κληρονομικότητα και ο πολυμορφισμός.

Κληρονομικότητα είναι η διαδικασία με την οποία μια νέα κλάση – που ονομάζεται παράγωγη κλάση – δημιουργείται από μια άλλη κλάση που ονομάζεται βασική κλάση. Μια παράγωγη κλάση έχει αυτομάτως όλα τα πεδία και τις μεθόδους που έχει και η βασική κλάση, ενώ μπορεί να έχει επιπρόσθετες μεθόδους ή/και επιπρόσθετα πεδία.

Θα μπορούσαμε να επεκτείνουμε το παράδειγμά μας, ορίζοντας μια νέα κλάση παράγωγη της Point2D, η οποία θα κρατάει τρία σημεία στο χώρο και η οποία θα κληρονομεί τα πεδία και τις μεθόδους της κλάσης; Point2D. Ονομάζουμε την κλάση αυτή Point3D. Προκειμένου να αναφερθούμε στα πεδία ή τις μεθόδους της βασικής κλάσης, χρησιμοποιούμε τη λέξη κλειδί super. Εάν θέλουμε να χρησιμοποιήσουμε τη μέθοδο toString() της παράγωγης κλάσης, τότε μπορούμε να χρησιμοποιήσουμε το **this**.Μια μέθοδος που ορίζεται στη βασική κλάση μπορούμε να την ορίσουμε ξανά στην παράγωγη κλάση με διαφορετικό τρόπο, όπως, για παράδειγμα, δείχνουμε στον κώδικα που ακολουθεί με τη μέθοδο toString(). Αυτό λέγεται **υπέρβαση** της μεθόδου (method overriding).

```
class Point3D extends Point2D
{
	int z;

	public Point3D(int x, int y,int z) {
		super(x,y);
		this.z = z;
	}
	public String toString(){ return "x:"+super.toString()+" z:"+this.z;}
}
```

Ο Πολυμορφισμός αναφέρεται στη δυνατότητα που μας δίνεται να συνδέουμε πολλές έννοιες στο όνομα μιας μεθόδου. Ένα παράδειγμα πολυμορφισμού είναι και η υπερφόρτωση, που αναφέρθηκε στην Ενότητα 15.1.2.

15.3 Διασυνδέσεις και Αφηρημένες Κλάσεις

Μια διασύνδεση (interface) στη Java προσδιορίζει ένα σύνολο μεθόδων που πρέπει να έχει οποιαδήποτε κλάση υλοποιεί τη διασύνδεση. Μια διασύνδεση δεν είναι κλάση, είναι μια ιδιότητα μιας κλάσης που περιγράφει ποιες μεθόδους πρέπει να έχει αυτή η κλάση. Μια διασύνδεση περιέχει τις επικεφαλίδες των μεθόδων που πρέπει να ορίζονται στην κλάση που υλοποιεί τη διασύνδεση. Μια κλάση, για να υλοποιεί μια διασύνδεση, πρέπει να

συμπεριλαμβάνει τη φράση implements Όνομα_Διασύνδεσης στην αρχή του ορισμού της και να υλοποιεί όλες τις μεθόδους των οποίων οι επικεφαλίδες περιέχονται στους ορισμούς της διασύνδεσης.

Μια διασύνδεση ορίζεται χρησιμοποιώντας τη δεσμευμένη λέξη interface και το όνομα της διασύνδεσης. Το παράδειγμά μας θα μπορούσε να τροποποιηθεί προσθέτοντας τη διασύνδεση Points και τροποποιώντας κατάλληλα την κλάση Point2D, η οποία υλοποιεί τη διασύνδεση, όπως φαίνεται παρακάτω:

```
interface Points
{
	public double distTo();
}

class Point2D implements Points{…}
```

Μια αφηρημένη κλάση είναι μια κλάση με μεθόδους που δεν έχουν πλήρεις ορισμούς. Με τη χρήση ενός δημιουργού αφηρημένης κλάσης δεν μπορούν να δημιουργηθούν αντικείμενα. Μια αφηρημένη κλάση, όμως, μπορεί να χρησιμοποιηθεί ως βασική κλάση για τον ορισμό των παράγωγων κλάσεων.

Για να ορίσουμε μια κλάση ως αφηρημένη, χρησιμοποιούμε τη δεσμευμένη λέξη **abstract** στον ορισμό της. Επίσης, η αφηρημένη κλάση θα πρέπει να έχει τουλάχιστον μία αφηρημένη μέθοδο. Μια αφηρημένη μέθοδος έχει τύπο επιστρεφόμενης τιμής, όνομα και λίστα παραμέτρων, αλλά δεν έχει σώμα. Σύμφωνα με τους κανόνες σύνταξης της Java στη δήλωση μιας αφηρημένης μεθόδου βάζουμε τη δεσμευμένη λέξη abstract αριστερά του τύπου επιστρεφόμενης τιμής.

15.4 Γενικοί Τύποι

Οι κλάσεις και οι μέθοδοι μπορούν να περιέχουν έναν τύπο δεδομένων ο οποίος ορίζεται παραμετρικά. Ο παραμετρικά οριζόμενος τύπος μπορεί να είναι οποιοσδήποτε τύπος αναφοράς, άρα και οποιοσδήποτε τύπος κλάσης. Στο Παράδειγμα 2, παρουσιάζεται ένας πολύ απλός ορισμός κλάσης, που περιέχει έναν παραμετρικά οριζόμενο τύπο T.

Παράδειγμα 2. Χρήση γενικευμένης κλάσης.

```
public class Example<T>
{
	private T input;

	public void setInput(T input) {this.input = input}

	public T getInput() { return input }
}
```

Το κώδικαν του Παραδείγματος 2, τον αποθηκεύουμε σε ένα αρχείο με το όνομα Example.java. Σε ένα άλλο αρχείο που είναι η μέθοδος main γράφουμε τον παρακάτω κώδικα, προκειμένου να χρησιμοποιήσουμε την κλάση Example με το γενικευμένο τύπο.

```
Example<String> p1 = new Example<String>();
P1.setInput(“Hello”);
System.out.println(p1.getInput());
Example<int> p2 = new Example<int>();
P1.setInput(1982);
System.out.println(p.getInput());
```

Έτσι, ο παραμετρικά οριζόμενος τύπος είναι String στην περίπτωση του αντικειμένου p1, ενώ σε αυτή του αντικειμένου p2 είναι τύπου int.

Σε μία κλάση μπορεί να δηλώσουμε οσουσδήποτε παραμετρικούς τύπους. Το όνομα της παραμέτρου που παριστά τον τύπο μπορεί να είναι οποιοδήποτε, αρκεί να μην είναι λέξη κλειδί, και κατά σύμβαση το πρώτο γράμμα γράφεται πάντα κεφαλαίο. Ο παραμετρικά οριζόμενος τύπος μπορεί να χρησιμοποιείται, όπως οποιοσδήποτε άλλος τύπος, στον ορισμό μιας κλάσης.

15.5 Ασκήσεις

15.1. Γράψτε μια σύντομη συνάρτηση σε Java, την isMultiple, που δέχεται ως παράμετρο δύο αριθμούς n και m τύπου double και επιστρέφει true, αν και μόνο αν τον είναι πολλαπλάσιο του m, δηλαδή $n = mi$ για κάποιο i.

15.2. Να ορίσετε μια κλάση με όνομα Payment, που θα αναπαριστά μια πληρωμή. Η κλάση αυτή θα πρέπει να περιέχει μια μεταβλητή τύπου double, στην οποία θα αποθηκεύεται το ποσό μιας πληρωμής και κατάλληλες μεθόδους προσπέλασης και μεταλλαγής (get() και set()). Επίσης, να δημιουργήσετε μια μέθοδο με όνομα paymentDetails, η οποία θα πρέπει να τυπώνει στην οθόνη μια φράση που θα περιγράφει το ποσό της πληρωμής.
Έπειτα να ορίσετε μια παράγωγη κλάση της Payment, με όνομα CashPayment, που θα αναπαριστά μια πληρωμή σε μετρητά. Η κλάση αυτή θα πρέπει να ορίζει ξανά τη μέθοδο paymentDetails, υποδεικνύοντας πως η πληρωμή είναι σε μετρητά. Να συμπεριλάβετε τον κατάλληλο κατασκευαστή ή κατασκευαστές.

Να ορίσετε μια παράγωγη κλάση της κλάσης Payment, με όνομα CreditCardPayment, που θα αναπαριστά μια πληρωμή με πιστωτική κάρτα. Η κλάση αυτή θα πρέπει να περιέχει ως μέλη της μεταβλητές για το όνομα, την ημερομηνία λήξης και τον αριθμό της πιστωτικής κάρτας. Να συμπεριλάβετε τον κατάλληλο κατασκευαστή. Τέλος, να ορίσετε ξανά τη μέθοδο paymentDetails ώστε στην τυπωμένη αναφορά να συμπεριλαμβάνονται όλες οι πληροφορίες της πιστωτικής κάρτας.

Να κατασκευάσετε μια μέθοδο main που να δημιουργεί δύο τουλάχιστον αντικείμενα της κλάσης CreditCardPayment με διαφορετικές τιμές και κλήσεις της μεθόδου paymentDetails για κάθε ένα από αυτά.

Βιβλιογραφία

Goodrich, M. T., & Tamassia, R. (2006). *Data Structures and Algorithms in Java, 4th edition.* Wiley.

Savitch, W. (2008). *Απόλυτη Java.* Ίων.

Sedgewick, R., & Wayne, K. (2011). *Algorithms, 4th edition.* Addison-Wesley.

Λεξικό Αγγλοελληνικών Όρων

A

abstract data type	αφηρημένος τύπος δεδομένων
address	διεύθυνση
acyclic graph	άκυκλο γράφημα
access	πρόσβαση
adjacency list	λίστα γειτνίασης
adjacency matrix	πίνακας γειτνίασης
adjacent vertex	γειτονικός (κόμβος)
algorithm	αλγόριθμος
amortized	αντισταθμιστικός
ancestor	πρόγονος
array	πίνακας
asymptotic complexity	ασυμπτωτική πολυπλοκότητα
average-case analysis	ανάλυση μέσης-περίπτωσης
AVL tree	AVL δένδρο

B

back-edge	οπισθο-ακμή ή ανιούσα-ακμή
balance factor	συντελεστής ισορροπίας
balanced tree	ισορροπημένο δένδρο
binary search	δυαδική αναζήτηση
binary search tree	δυαδικό δένδρο αναζήτησης
binomial tree	διωνυμικό δένδρο
binomial queue	διωνυμική ουρά
bipartite graph	διμερές γράφημα
bound	φράγμα
breadth-first search	κατά-πλάτος ή οριζόντια διερεύνηση

C

chain	αλυσίδα
child	παιδί
class	κλάση
collision	σύγκρουση
complete	πλήρες
compress	συμπιέζω
computation	υπολογισμός
concatenation	συνένωση
connected component	συνεκτική συνιστώσα
connected graph	συνεκτικό γράφημα
connectivity	συνεκτικότητα
constructor	κατασκευαστής
cross-edge	διασχίζουσα-ακμή ή εγκάρσια-ακμή
cycle	κύκλος
cuckoo hashing	κατακερματισμός του κούκου

D

degree	βαθμός

depth-first search	κατά-βάθος ή καθοδική διερεύνηση
descendant	απόγονος
dequeue	ουρά δύο άκρων
dictionary	λεξικό
digital tree	ψηφιακό δένδρο
dimension	διάσταση
directed acyclic graph	κατευθυνόμενο άκυκλο γράφημα
directed graph	κατευθυνόμενο γράφημα
distance	απόσταση
dynamic	δυναμικός, δυναμική

E

edge	ακμή
elementary	στοιχειώδης
examine	εξετάζω
exception	εξαίρεση
exponential	εκθετικός
external memory	εξωτερική μνήμη
external node	εξωτερικός κόμβος

F

family	οικογένεια
Fibonacci heap	σωρός Fibonacci
find	εύρεση
forward-edge	εμπρόσθια-ακμή ή κατιούσα-ακμή

G

garbage collection	συλλογή απορριμάτων
graph	γράφημα

H

hash function	συνάρτηση κατακερματισμού ή διασποράς
hash table	πίνακας κατακερματισμού ή διασποράς
heap	σωρός
height	ύψος

I

implementation	υλοποίηση
index	θέση ή ευρετήριο
induced subgraph	επαγόμενο υπογράφημα
inheritance	κληρονομικότητα
instance	στιγμιότυπο
interface	διασύνδεση
internal node	εσωτερικός κόμβος
intersection	τομή
interval	διάστημα
invariant	αναλλοίωτη συνθήκη

L

leaf	φύλλο ή τερματικός κόμβος
length	μήκος
linear	γραμμικός
linked list	συνδεδεμένη λίστα
logarithmic	λογαριθμικός

loop	βρόχος
lower bound	κάτω φράγμα

M

matrix	πίνακας
matrix multiplication	πολλαπλασιασμός πινάκων
median	διάμεσος
memory hierarchy	ιεραρχία μνήμης
method	μέθοδος

N

node	κόμβος (δένδρου ή λίστας)

O

object	αντικείμενο
operand	όρισμα
operator	τελεστής
order	τάξη (συνάρτησης)

P

parent	γονέας
partition	διαμέριση
path	διαδρομή ή μονοπάτι
permutation	μετάθεση
pointer	δείκτης
polymorphism	πολυμορφισμός
polynomial	πολυωνυμικός
predecessor	προκάτοχος
priority queue	ουρά προτεραιότητας
property	ιδιότητα

Q

queue	ουρά
quotient	πηλίκο

R

rank	τάξη (στοιχείου)
red-black tree	κοκκινόμαυρο δένδρο
reference	αναφορά
remainder	υπόλοιπο
root	ρίζα
rooted tree	δένδρο με ρίζα ή έρριζο δένδρο
rotation	περιστροφή

S

searching	διερεύνηση
sibling (node)	αδελφικός (κόμβος)
sink vertex	τερματικός ή καταληκτικός κόμβος
size	μέγεθος
sorting	ταξινόμηση
source vertex	πηγαίος ή αφετηριακός κόμβος
spanning tree/forest	συνδετικό δένδρο/δάσος
splay tree	αρθρωτό δένδρο
split	διάσπαση

stack	στοίβα
subgraph	υπογράφημα
subroutine	υποπρόγραμμα
successor	διάδοχος

T

tree	δένδρο
tree-edge	δενδρική-ακμή

U

undirected graph	μη-κατευθυνόμενο γράφημα
union	ένωση
universal	καθολική
upper bound	άνω φράγμα

V

vector	διάνυσμα
vertex	κόμβος (γραφήματος)
visit	επισκέπτομαι

W

walk	περίπατος
worst-case analysis	ανάλυση χειρίστης-περίπτωσης

Λεξικό Ελληνοαγγλικών Όρων

Α

AVL δένδρο	AVL tree
αδελφικός (κόμβος)	sibling (node)
ακμή	edge
άκυκλο γράφημα	acyclic graph
αλγόριθμος	algorithm
αλυσίδα	chain
αναλλοίωτη συνθήκη	invariant
ανάλυση μέσης-περίπτωσης	average-case analysis
ανάλυση χειρίστης-περίπτωσης	worst-case analysis
αναφορά	reference
αντικείμενο	object
αντισταθμιστικός	amortized
άνω φράγμα	upper bound
απόγονος	descendant
απόσταση	distance
αρθρωτό δένδρο	splay tree
ασυμπτωτική πολυπλοκότητα	asymptotic complexity
αφηρημένος τύπος δεδομένων	abstract data type
αφετηριακός κόμβος	source vertex

Β

βαθμός	degree
βρόχος	loop

Γ

γειτονικός (κόμβος)	adjacent vertex
γονέας	parent
γραμμικός	linear
γράφημα	graph

Δ

δείκτης	pointer
δενδρική-ακμή	tree-edge
δένδρο	tree
δένδρο με ρίζα	rooted tree
διάδοχος	successor
διαδρομή ή μονοπάτι	path
διαμέριση	partition
διάμεσος	median
διάνυσμα	vector
διάσπαση	split
διάσταση	dimension
διάστημα	interval
διασχίζουσα-ακμή ή εγκάρσια-ακμή	cross-edge
διερεύνηση	searching
διμερές γράφημα	bipartite graph
διεύθυνση	address

διωνυμικό δένδρο	binomial tree
διωνυμική ουρά	binomial queue
δυαδική αναζήτηση	binary search
δυαδικό δένδρο αναζήτησης	binary search tree
δυναμικός, δυναμική	dynamic

Ε

εκθετικός	exponential
εμπρόσθια-ακμή ή κατιούσα-ακμή	forward-edge
ένωση	union
εξαίρεση	exception
εξετάζω	examine
εξωτερικός κόμβος	external node
εξωτερική μνήμη	external memory
επαγόμενο υπογράφημα	induced subgraph
επισκέπτομαι	visit
εσωτερικός κόμβος	internal node
εύρεση	find
ευρετήριο	index

Θ

θέση	index

Ι

ιδιότητα	property
ιεραρχία μνήμης	memory hierarchy
ισορροπημένο δένδρο	balanced tree

Κ

καθολική	universal
κατά-βάθος ή καθοδική διερεύνηση	depth-first search
κατά-πλάτος ή οριζόντια διερεύνηση	breadth-first search
κατακερματισμός	hashing
κατακερματισμός του κούκου	cuckoo hashing
κατασκευαστής	constructor
κατευθυνόμενο γράφημα	directed graph
κάτω φράγμα	lower bound
κλάση	class
κληρονομικότητα	inheritance
κοκκινόμαυρο δένδρο	red-black tree
κόμβος (γραφήματος)	vertex
κόμβος (δένδρου ή λίστας)	node
κύκλος	cycle

Λ

λεξικό	dictionary
λίστα γειτνίασης	adjacency list
λογαριθμικός	logarithmic

Μ

μέγεθος	size
μέθοδος	method
μετάθεση	permutation
μη-κατευθυνόμενο γράφημα	undirected graph

μήκος	length

Ο

οικογένεια	family
οπισθο-ακμή ή ανιούσα-ακμή	back-edge
όρισμα	operand
ουρά	queue
ουρά δύο άκρων	dequeue
ουρά προτεραιότητας	priority queue

Π

παιδί	child
περίπατος	walk
περιστροφή	rotation
πηλίκο	quotient
πίνακας	matrix
πίνακας ή παράταξη	array
πίνακας γειτνίασης	adjacency matrix
πίνακας κατακερματισμού ή διασποράς	hash table
πλήρες	complete
πολλαπλασιασμός πινάκων	matrix multiplication
πολυμορφισμός	polymorphism
πολυωνυμικός	polynomial
πρόγονος	ancestor
προκάτοχος	predecessor
πρόσβαση	access

Ρ

ρίζα	root

Σ

στιγμιότυπο	instance
στοίβα	stack
στοιχειώδης	elementary
σύγκρουση	collision
συλλογή απορριμάτων	garbage collection
συμπιέζω	compress
συνάρτηση κατακερματισμού ή διασποράς	hash function
συνδεδεμένη λίστα	linked list
συνεκτική συνιστώσα	connected component
συνεκτικό γράφημα	connected graph
συνεκτικότητα	connectivity
συνένωση	concatenation
συντελεστής ισορροπίας	balance factor
συντελεστής πληρότητας	load factor
σωρός	heap
σωρός Fibonacci	Fibonacci heap

Τ

τάξη (στοιχείου)	rank
τάξη (συνάρτησης)	order
ταξινόμηση	sorting
τελεστής	operator
τερματικός κόμβος	sink vertex

τομή	intersection

Υ

υλοποίηση	implementation
υπογράφημα	subgraph
υπολογισμός	computation
υπόλοιπο	remainder
υποπρόγραμμα	subroutine
ύψος	height

Φ

φράγμα	bound
φύλλο ή τερματικός κόμβος	leaf

Ψ

ψηφιακό λεξικό	digital dictionary

Κύριος συγγραφέας

ΛΟΥΚΑΣ ΓΙΕΩΡΓΙΑΔΗΣ

Συν-συγγραφείς

ΣΤΑΥΡΟΣ Δ. ΝΙΚΟΛΟΠΟΥΛΟΣ
ΛΕΩΝΙΔΑΣ ΠΑΛΗΟΣ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ